CS 61A Fall 2016

Structure and Interpretation of Computer Programs

MIDTERM 1

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 study guide.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

1. (10 points) Exeggcute

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If a function value is displayed, write "Function".

The first two rows have been provided as examples.

Recall: The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have started `python3` and executed the following statements:

```
equals = lambda a, b: a == b
nemo = lambda n: lambda: print(n)
ray = nemo(1)

def if_function(f, g, h):
    if h:
        f()
    elif h:
        f(f())
    else:
        print(5 or 6)
    g()

def dory():
    print('fish')
    return lambda: 1/0
```

Expression	**Interactive Output**
`pow(2, 3)`	8
`print(4, 5) + 1`	4 5 Error
`equals(3==4, equals(5, equals(5, 5)))`	
`print(print(print(2)), print(3))`	
`print(nemo(print(5))())`	
`if_function(nemo(3), dory, 2)`	
`if_function(dory, nemo(2), ray())`	

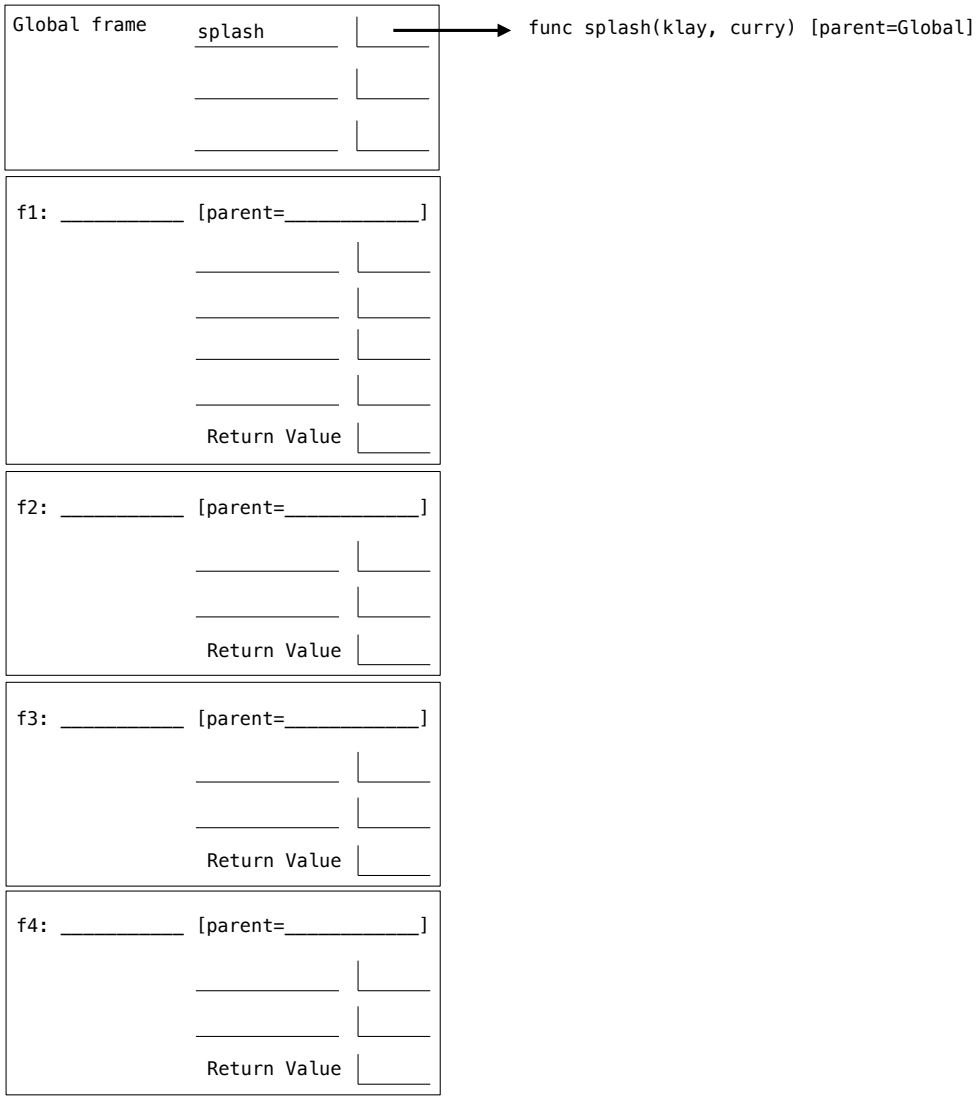
2. (8 points) Goldeen State

Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*

A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  def splash(klay, curry):
2    while curry == 3:
3      steph = klay
4      klay = lambda klay: steph(curry + 1)
5      curry = curry + 2
6      return klay
7    return curry // 5
8
9  steph = lambda klay: splash(11, curry)-3
10 steph, curry = splash(steph, 3), 30
11 steph(4)
```



3. (9 points) Countizard

(a) (6 pt) Implement `counter`, which takes a non-negative single-digit integer `d`. It returns a function `count` that takes a non-negative integer `n` and returns the number of times that `d` appears as a digit in `n`. You may not use recursive calls or any features of Python not yet covered in the course.

```
def counter(d):
    """Return a function of N that returns the number of times D appears in N.

    >>> counter(8)(8018)
    2
    >>> counter(0)(2016)
    1
    >>> counter(0)(0)
    0
    """
    def count(_____________________________________________________________):

        k = 0

        while _____________________________________________________________:


            _____________, last = _____________________________________, n % 10


            if _____________________________________________________________:


                ____________________________________________________________


        ____________________________________________________________________


    ____________________________________________________________________________
```

(b) (3 pt) Implement `significant`, which takes positive integers `n` and `k`. It returns the `k` most significant digits of `n` as an integer. These are the first `k` digits of `n`, starting from the left. If `n` has fewer than `k` digits, it returns `n`. You may not use `round`, `int`, `str`, or any functions from the `math` module. You may use `pow`, which raises its first argument to the power of its second: `pow(9, 2)` is 81 and `pow(9, 0.5)` is 3.0.

```
def significant(n, k):
    """Return the K most significant digits of N.

    >>> significant(12345, 3)
    123
    >>> significant(12345, 7)
    12345
    """
    if ____________________________________________________________________:

        return n

    return significant(_____________________________, _____________________________)
```

4. (6 points) Caterepeat

(a) (4 pt) Implement `repeat_sum`, which takes a one-argument function `f`, a value `x`, and a non-negative integer `n`. It returns the sum of $n + 1$ terms. Each term, indexed by `k` starting at 0, is the result of applying `f` to `x` repeatedly `k` times. You may assign to only one name in each of the three assignment statements. You may not use recursive calls or any features of Python not yet covered in the course.

```
def repeat_sum(f, x, n):
    """Compute the following summation of N+1 terms, where the last term
    calls F N times: x + f(x) + f(f(x)) + f(f(f(x))) + ... + f(f(...f(x)))

    >>> repeat_sum(lambda x: x*x, 3, 0) # 3
    3
    >>> repeat_sum(lambda x: x*x, 3, 1) # 3 + 9
    12
    >>> repeat_sum(lambda x: x+2, 3, 4) # 3 + 5 + 7 + 9 + 11
    35
    """
    total, k = 0, 0

    while ____________________________________________________________________:

        _______________ = ____________________________________________________

        _______________ = ____________________________________________________

        _______________ = ____________________________________________________

    return total
```

(b) (2 pt) Implement `sum_squares`, which takes a non-negative integer `n` and uses `repeat_sum` to return the sum of the squares of the first `n` positive integers. Assume `repeat_sum` is implemented correctly. You may use `pow`, which raises its first argument to the power of its second: `pow(9, 2)` is 81 and `pow(9, 0.5)` is 3.0.

```
def sum_squares(n):
    """Return the sum of the first N perfect squares.

    >>> sum_squares(0)
    0
    >>> sum_squares(3) # 1**2 + 2**2 + 3**2
    14
    >>> sum_squares(5) # 1**2 + 2**2 + 3**2 + 4**2 + 5**2
    55
    """

    f = ____________________________________________________________________

    return repeat_sum(f, 0, n)
```

5. (7 points) Multikarp

Terminology. An *order 1 numeric function* is a function that takes a number and returns a number. An order 2 numeric function is a function that takes a number and returns an order 1 numeric function. Likewise, an order n numeric function is a function that takes a number and returns an order $n-1$ numeric function.

The *argument sequence* of a nested call expression is the sequence of all arguments in all subexpressions, in the order they appear. For example, the expression `f(3)(4)(5)(6)(7)` has the argument sequence 3, 4, 5, 6, 7.

(a) (4 pt) Implement `multiadder`, which takes a positive integer `n` and returns an order n numeric function that sums an argument sequence of length n.

```
def multiadder(n):
    """Return a function that takes N arguments, one at a time, and adds them.

    >>> f = multiadder(3)
    >>> f(5)(6)(7)                # 5 + 6 + 7
    18
    >>> multiadder(1)(5)
    5
    >>> multiadder(2)(5)(6)       # 5 + 6
    11
    >>> multiadder(4)(5)(6)(7)(8) # 5 + 6 + 7 + 8
    26
    """
    assert n > 0


    if ____________________________________________________________________:


        return ____________________________________________________________


    else:


        return ____________________________________________________________
```

(b) (3 pt) Complete the expression below by writing one integer in each blank so that the whole expression evaluates to 2016. The `compose1` function appears on your midterm 1 study guide in the middle of the left column of page 2. Assume `multiadder` is implemented correctly.

```
compose1(multiadder(_______)(1000), multiadder(_______)(10)(_______))(1)(2)(3)
```

CS 61A Fall 2016 — Structure and Interpretation of Computer Programs

MIDTERM 2

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A study guides.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

1. (8 points) Halloween

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. The `Link` class appears on page 2 of the midterm 2 study guide.

The first two rows have been provided as examples.

Recall: The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have started `python3` and executed the following statements:

```
class Party:
    guests = Link.empty

    def __init__(self, time):
        Party.guests = Link(time+1, Party.guests)

    def attend(self):
        self.guests.rest = Link(self.guests.rest)
        return self.guests

class Costume(Party):
    def __init__(self, bow, tie):
        Party.guests.rest = Link(bow)
        self.ie = Link(self)

    def attend(self):
        print(repr(self.ie))
        Party.attend = lambda self: Party(9).guests

    def __repr__(self):
        print('Nice')
        return 'Costume'
```

Expression	**Interactive Output**
`Link(1, Link.empty)`	Link(1)
`Link(1, Link(2))`	Link(1, Link(2))
`Party(1).guests`	
`Party(3).attend()`	
`Costume(5, 6).attend()`	
`Party(7).attend()`	

2. (8 points) A List with a Twist

Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*

A complete answer will:

- Add all missing names and parent annotations to frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.
- Use box-and-pointer notation for list values. You do not need to write index numbers or the word "list".

```
1  lamb = 'da'
2  def da(da):
3    def lamb(lamb):
4      nonlocal da
5      def da(nk):
6        da = nk + ['da']
7        da.append(nk[0:2])
8        return nk.pop()
9    da(lamb)
10   return da([[1], 2]) + 3
11
12 da(lambda da: da(lamb))
```

Global frame _____ _____

_____ _____

f1: _____ [parent=_____]

_____ _____

_____ _____

_____ _____

Return Value _____

f2: _____ [parent=_____]

_____ _____

_____ _____

_____ _____

Return Value _____

f3: _____ [parent=_____]

_____ _____

_____ _____

_____ _____

Return Value _____

f4: _____ [parent=_____]

_____ _____

_____ _____

_____ _____

Return Value _____

3. (6 points) Return Policy

Implement `quota`, which takes a one-argument function `f` and a non-negative integer `limit`. The function it returns has the same behavior as `f`, except that each value is only returned up to `limit` times. After that, the function returns an "Over quota" message instead, and the `limit` is decreased by 1 for future calls.

```
def quota(f, limit):
    """A decorator that limits the number of times a value can be returned.

    >>> square = lambda x: x * x
    >>> square = quota(square, 3)
    >>> square(6)                     # 1st call with return value 36
    36
    >>> [square(5) for x in range(3)] # 3 calls when the limit is 3
    [25, 25, 25]
    >>> square(5)                     # 4th call with return value 25
    'Over quota! Limit is now 2'
    >>> square(-6)                    # 2nd call with return value 36
    36
    >>> square(-6)                    # 3rd call when the limit is 2
    'Over quota! Limit is now 1'
    >>> square(7)                     # 1st call when the limit is 1
    49
    >>> square(5)                     # 5th call with return value 25
    'Over quota! Limit is now 0'
    """

    values = []

    def limited(n):

        ____________________________________________________________________


        y = ____________________________________________________________________


        count = len(____________________________________________________________)

        values.append(y)


        if ____________________________________________________________________:


            return ____________________________________________________________


        limit = ____________________________________________________________


        return 'Over quota! Limit is now ' ___________________________________

    return limited
```

4. (6 points) A Classy Election

Implement the `VotingMachine` and `Ballot` classes based on the doctest below. The voting machine must determine which choice has the most votes (the `winner`) and detect if a ballot is used more than once. In case of a tie, the `winner` among choices with maximal votes is the one that most recently received a vote. `Ballot.vote` takes a string, and a `VotingMachine` must handle an arbitrary number of choices.

```
class VotingMachine:
    """A machine that creates and records ballots.

    >>> machine = VotingMachine(4)
    >>> a, b, c, d = machine.ballots
    >>> d.vote('Bruin')
    'Bruin is winning'
    >>> b.vote('Bruin')
    'Bruin is winning'
    >>> c.vote('Bear')
    'Bear is losing'
    >>> a.vote('Bear')
    'Bear is winning'
    >>> c.vote('Tree')
    'Fraud: multiple votes from the same ballot!'
    >>> machine.winner
    'Bear'
    """
    def __init__(self, k):
        self.ballots =   [___________________________________ for i in range(k)]
        self.votes = {}

    def record(self, ballot, choice):
        if ballot.used:
            return 'Fraud: multiple votes from the same ballot!'


        ____________________________________________________________________


        self.votes[choice] = _____________________________________________ + 1


        if ____________________________________________________________________:
            return choice + ' is losing'
        else:

            ____________________________________________________________________
            return choice + ' is winning'
class Ballot:

    ____________________________________________________________________________

    def __init__(self, machine):
        self.machine = machine
    def vote(self, x):

        return ____________________________________________________________
```

5. (6 points) Trick or Tree

Implement `path`, which takes a linked list `s` and a `Tree` instance `t`. It returns whether `s` is a path from the root of `t` to some leaf. The `Tree` and `Link` classes are on page 2 of the midterm 2 study guide.

Restrictions:

- You may not call the built-in `len` function on a linked list or invoke its `__len__` method.
- You may not apply element selection (e.g., `s[2]`) on a linked list or invoke its `__getitem__` method.

```
def path(s, t):
    """Return whether Link S is a path from the root to a leaf in Tree T.

    >>> t = Tree(1, [Tree(2), Tree(3, [Tree(4), Tree(5)]), Tree(6)])
    >>> a = Link(1, Link(3, Link(4)))  # A full path
    >>> path(a, t)
    True
    >>> b = Link(1, Link(3))           # A partial path
    >>> path(b, t)
    False
    >>> c = Link(1, Link(2, Link(7)))  # A path and an extra value
    >>> path(c, t)
    False
    >>> d = Link(3, Link(4))           # A path of a branch
    >>> path(d, t)
    False
    """

    if ____________________________________________________________________:


        return False


    if ____________________________________________________________________:


        return True


    return ______________([_______________________________ for b in t.branches])
```

6. (6 points) Left it Right There

(a) (4 pt) Implement `binary`, which takes a list `s` of unique integers. It returns a *binary search tree* containing all of those integers, represented as a `BTree` instance or `BTree.empty`. The values in any path of this tree must appear in the same order as they did in `s`. The `BTree` class is on page 2 of the midterm 2 study guide.

```
def binary(s):
    """Construct a binary search tree from S for which all paths are in order.

    >>> binary([3, 5, 1])
    BTree(3, BTree(1), BTree(5))
    >>> binary([4, 3, 7, 6, 2, 9, 8])
    BTree(4, BTree(3, BTree(2)), BTree(7, BTree(6), BTree(9, BTree(8))))
    """
    assert len(s) == len(set(s)), 'All elements of s should be unique'

    if ____________________________________________________________________:

        return ____________________________________________________________

    root = ____________________________________________________________

    left = ____________________________________________________________

    right = ____________________________________________________________

    return BTree(root, binary(left), binary(right))
```

(b) (1 pt) Circle the Θ expression that describes the **smallest** possible height of the tree returned by `binary(s)` for a list `s` of length n. The height of a tree is the length of the longest path from its root to a leaf.

$\Theta(1)$ $\Theta(\log n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(2^n)$ None of these

(c) (1 pt) Circle the Θ expression that describes the **largest** possible height of the tree returned by `binary(s)` for a list `s` of length n. The height of a tree is the length of the longest path from its root to a leaf.

$\Theta(1)$ $\Theta(\log n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(2^n)$ None of these

7. (10 points) Summer Camp

(a) (6 pt) Implement `sums`, which takes two positive integers `n` and `k`. It returns a list of lists containing all the ways that a list of `k` positive integers can sum to `n`. Results can appear in any order.

```
def sums(n, k):
    """Return the ways in which K positive integers can sum to N.

    >>> sums(2, 2)
    [[1, 1]]
    >>> sums(2, 3)
    []
    >>> sums(4, 2)
    [[3, 1], [2, 2], [1, 3]]
    >>> sums(5, 3)
    [[3, 1, 1], [2, 2, 1], [1, 3, 1], [2, 1, 2], [1, 2, 2], [1, 1, 3]]
    """

    if ____________________________________________________________________:

        return ____________________________________________________________

    y = []

    for x in ___________________________________________________________:

        y.extend([_____________________ for s in sums(_____________________)])

    return y
```

(b) (4 pt) *Why so many lines?* Implement `f` and `g` for this alternative version of the `sums` function.

```
f = lambda x, y: (x and [_____________ for z in y] + f(_______, _______)) or []

def sums(n, k):
    """Return the ways in which K positive integers can sum to N."""

    g = lambda w: (w and f(_____________________________________________)) or [[]]

    return [v for v in g(k) if sum(v) == n]
```

CS 61A Fall 2017

Structure and Interpretation of Computer Programs

MIDTERM 1

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 study guide.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

1. (10 points) I Wonder What Python Would Display

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
aaron, burr = 2, 5
aaron, burr = 4, aaron + 1

hamil = 10

def alex(hamil):
    def g(w):
        hamil = 2 * w
        print(hamil, w)
        w = hamil
        return hamil
    w = 5
    alex = g(w + 1)
    print(w, alex, hamil)

def el(i, za):
    def angelica():
        return i + 1
    if i > 10:
        return za()
    elif i > 4:
        print(angelica())
        return el(i * i, za)
    else:
        return el(i * i,
                  angelica)

K = lambda x: lambda y: x

def pr(x):
    print(x)
    return x
```

Expression	Interactive Output
`pow(2, 3)`	8
`print(4, 5) + 1`	4 5 Error
`print(aaron, burr)`	
`alex(3)`	
`el(3, el)`	
`K`	
`K(3)`	
`K(3)(2)`	
`pr(True) and pr(0) and pr(1)`	

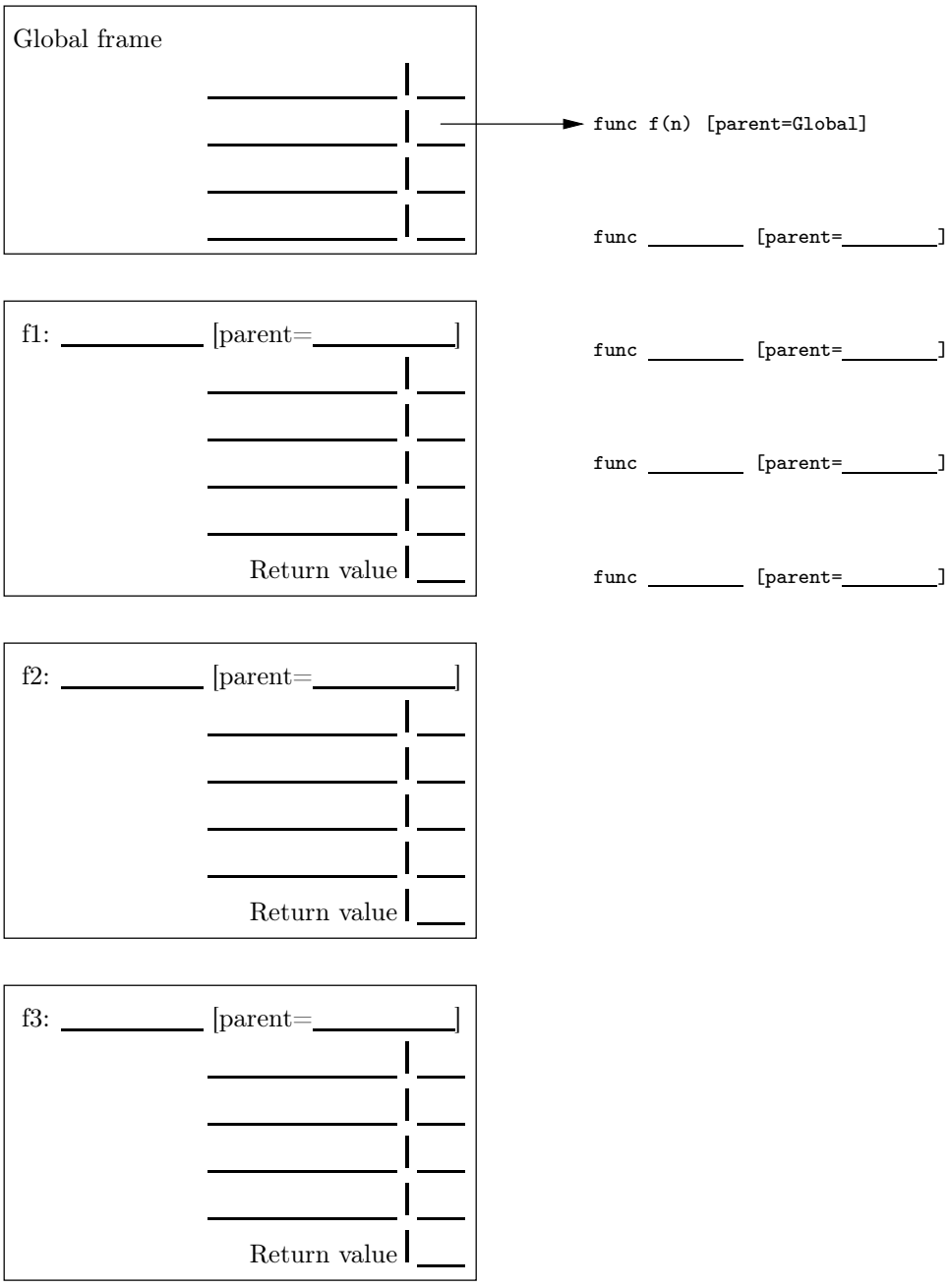
2. (8 points) Environmental Influences

Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces, frames, or function values.*

A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  x = 1
2  def f(n):
3      def g():
4          return n + x
5      x = n + 5
6      if n % 3 == 0:
7          return g
8      else:
9          return f(n + 1)
10 x = 10
11 z = f(2)
12 q = x + z()
```



3. (3 points) Triangulate

It's easy to see that in any triangle, each side must be shorter than the sum of the other two. Implement `triangle`, which takes three positive numbers, `a`, `b`, and `c`, and returns whether these three numbers could possibly be the lengths of the three sides of a triangle.

```
def triangle(a, b, c):
    """Return whether a, b, and c could be the legs of a triangle.

    >>> triangle(3, 4, 5)
    True
    >>> triangle(3, 4, 6)
    True
    >>> triangle(6, 3, 4)
    True
    >>> triangle(3, 6, 4)
    True
    >>> triangle(9, 2, 2)
    False
    >>> triangle(2, 4, 2)
    False
    """




    longest = ____________________________________________________________________________




    sum_of_others = ____________________________________________________________________




    return longest ____________________________________________________________ sum_of_others
```

4. (9 points) Digital

(a) (3 pt) Implement `collapse`, which takes a non-negative integer, and returns the result of removing all digits from it that duplicate the digit immediately to their right.

```
def collapse(n):
    """For non-negative N, the result of removing all digits that are equal
    to the digit on their right, so that no adjacent digits are the same.

    >>> collapse(1234)
    1234
    >>> collapse(12234441)
    12341
    >>> collapse(0)
    0
    >>> collapse(3)
    3
    >>> collapse(11200000013333)
    12013
    """



    left, last = n // 10, n % 10



    if ____________________________________________________________________________:



        return last



    elif ________________ == ____________________________________________________:



        return collapse(____________________________________________________________)



    else:



        return collapse(_____________________________) * 10 + _______________________________
```

(b) (6 pt) Implement `find_pair`, which takes a two-argument function, `p`, as input and returns another function. The returned function takes a non-negative integer `n`; it returns `True` if and only if `p` returns a true value when called on at least one pair of adjacent digits in `n`, and `False` otherwise.

```
def find_pair(p):
    """Given a two-argument function P, return a function that takes a
    non-negative integer and returns True if and only if two adjacent digits
    in that integer satisfy P (that is, cause P to return a true value).

    >>> z = find_pair(lambda a, b: a == b)   # Adjacent equal digits
    >>> z(1313)
    False
    >>> z(12334)
    True
    >>> z = find_pair(lambda a, b: a > b)
    >>> z(1234)
    False
    >>> z(123412)
    True
    >>> find_pair(lambda a, b: a <= b)(9753)
    False
    >>> find_pair(lambda a, b: a == 1)(1)    # Only one digit; no pairs.
    False
    """

    def find(n):


        while ____________________________________________________________________________:


            if ____________________________________________________________________________:


                return ____________________________________________________________


            else:


                _______________ = ____________________________________________________


        ____________________________________________________________________________


    ____________________________________________________________________________
```

5. (10 points) Please Confirm

Definition. A *confirming function* for a sequence of digits, called a *code*, takes a single digit as its only argument. If the digit does not match the first (left-most) digit of the code to be confirmed, it returns `False`. If the digit does match, then the confirming function returns `True` if the code has only one digit, or another confirming function for the rest of the code if there are more digits to confirm.

(a) (5 pt) Implement `confirmer` so that when `confirmer` takes a positive integer `code`, it returns a confirming function for the digits of that `code`.

```
def confirmer(code):
    """Return a confirming function for CODE.

    >>> confirmer(204)(2)(0)(4) # The digits of 204 are 2, then 0, then 4.
    True
    >>> confirmer(204)(2)(0)(0) # The third digit of 204 is not 0.
    False
    >>> confirmer(204)(2)(1)    # The second digit of 204 is not 1.
    False
    >>> confirmer(204)(20)      # The first digit of 204 is not 20.
    False
    """
    def confirm1(d, t):
        def result(digit):
            if d == digit:
                return t
            else:
                return False
        return result

    def extend(prefix, rest):
        """Return a confirming function that returns REST when given the digits of PREFIX.
        For example, if c = extend(12, confirmer(34)), then c(1)(2) returns confirmer(34),
        so that c is a confirming function for 1234."""

        left, last = prefix // 10, prefix % 10



        if ____________________________________________________________________________:



            return ____________________________________________________________________



        else:



            return ____________________________________________________________________



    return ________________________________________________________________________
```

(b) (5 pt) Given a confirming function, one can find the code it confirms, one digit at a time. Implement `decode`, which takes a confirming function `f` and returns the code that it confirms.

```
def decode(f, y=0):
    """Return the code for a confirming function f.

    >>> decode(confirmer(12001))
    12001
    >>> decode(confirmer(56789))
    56789
    """


    d = 0


    while d < 10:


        x, code = _____________________________________ , _____________________________________


        if x == True:


            return code


        elif x == False:


            ____________________________________________________________________________


        else:


            ____________________________________________________________________________
```

CS 61A Fall 2017 — Structure and Interpretation of Computer Programs

MIDTERM 2

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except two hand-written 8.5" × 11" crib sheets of your own creation and the two official CS 61A midterm study guides.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- Before asking a question, read the announcements on the screen/board. We will not answer your question directly. If we decide to respond, we'll add our response to the screen/board so everyone can see the clarification.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.

1. (12 points) By Any Other Name

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. The interactive interpreter displays the repr string of the value of a successfully evaluated expression, unless it is `None`. If an error occurs, write "Error", but include all output displayed before the error. The first row has been provided as an example.

Assume that you have started `python3` and executed the code shown on the left first, then you evaluate each expression on the right in order. Statements and expressions sent to the interpreter have a cumulative effect.

```
class Plant:
    k = 1
    kind = "green"

    def __init__(self):
        self.k = Plant.k
        Plant.k = self.k + 1
        if self.k > 3:
            Plant.name = lambda t: "tree"
            Plant.k = 6

    def name(self):
        return kind

    def __repr__(self):
        s = self.name() + " "
        return s + str(self.k)

class Flower(Plant):
    kind = "pretty"

    def __repr__(self):
        s = self.smell() + " "
        return s + Plant.__repr__(self)

    def smell(self):
        return "bad"

class Rose(Flower):
    def name(self):
        return "rose"

    def smell(self):
        return "nice"

class Garden:
    def __init__(self, kind):
        self.name = kind
        self.smell = kind().smell

    def smell(self):
        return self.name.kind

f1 = Flower()
f2 = Flower()
```

Expression	**Interactive Output**
`[2, 3]`	[2, 3]
(1 pt) `f1.name()`	
(1 pt) `f1.k`	
(1 pt) `Plant().k`	
(1 pt) `Rose.k`	
(2 pt) `Plant()`	
(2 pt) `Rose()`	
(2 pt) `Garden(Flower).smell()`	
(2 pt) `Garden(Flower).name()`	

2. (8 points) Buy Local

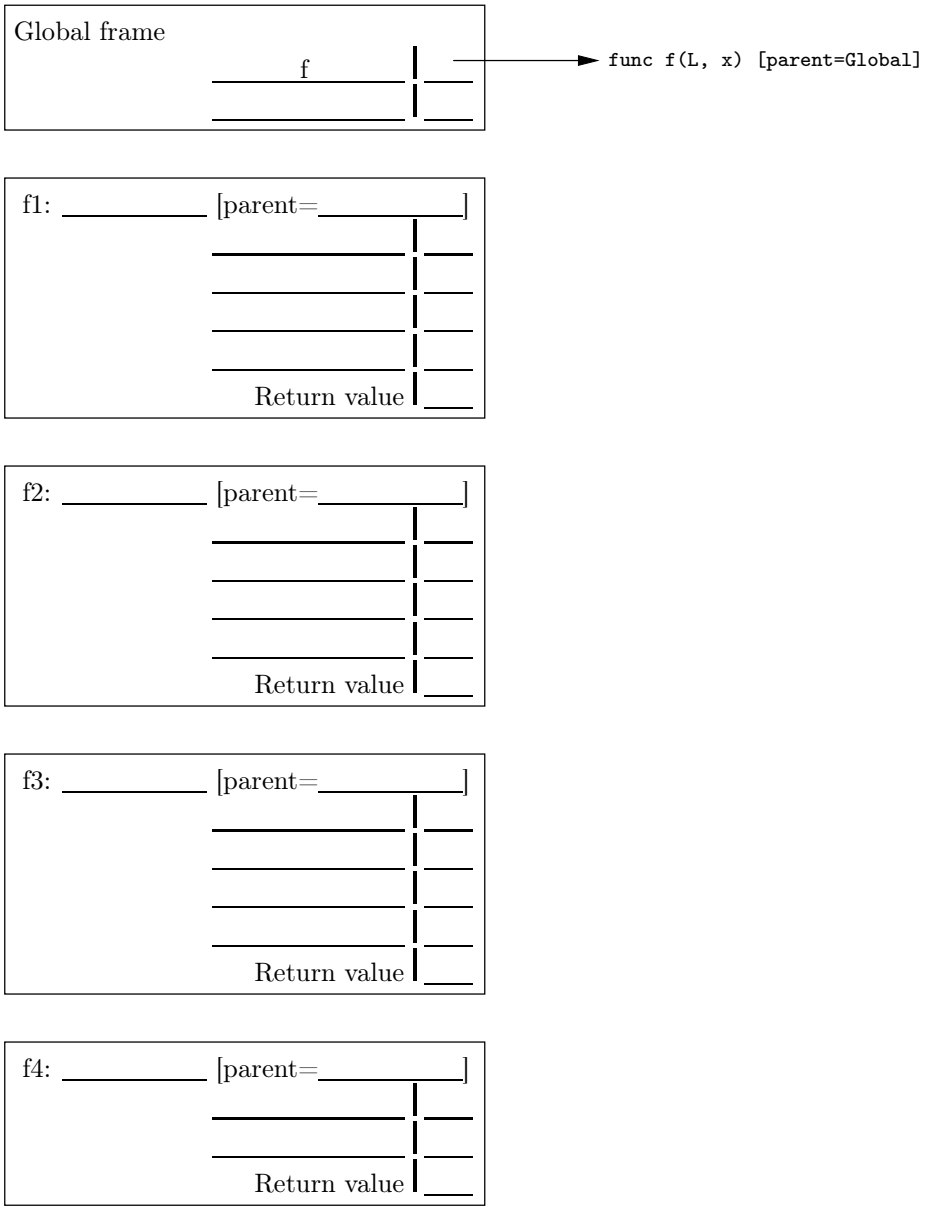
Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*

A complete answer will:

- Add all missing names and parent annotations to frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.
- Use box-and-pointer notation for list values. You do not need to write index numbers or the word "list".

Important: The slash on line 11 means that the return expression continues on the next line.

```
1  def f(L, x):
2    L[1] = L
3    def g(c, h, b):
4      nonlocal x
5      x = c
6      if c == 0:
7        b.append(5)
8        b = L + b
9        return h
10     else:
11       return \
12         g(c-1,
13           lambda: [c, x],
14           b)
15   p = g(1, None, [4])
16   x += 3
17   return p()
18 r = f([0, 0], 1)
```



3. (10 points) Pumpkin Splice Latte

(a) (2 pt) Implement `splice`, which takes two lists `a` and `b` and a non-negative integer `k` that is less than or equal to the length of `a`. It returns the result of *splicing* `b` into `a` at `k`. That is, it returns a new list containing the first `k` elements of `a`, then all elements of `b`, then the remaining elements of `a`.

```
def splice(a, b, k):
    """Return a list of the first k elements of a, then all of b, then the rest of a.

    >>> splice([2, 3, 4, 5], [6, 7], 2)
    [2, 3, 6, 7, 4, 5]
    """

    return ____________________________________________________________________________
```

(b) (3 pt) Implement `all_splice`, which returns a list of all the non-negative integers `k` such that splicing list `b` into list `a` at `k` creates a list with the same contents as `c`. Assume that `splice` is implemented correctly.

```
def all_splice(a, b, c):
    """Return a list of all k such that splicing b into a at position k gives c.

    >>> all_splice([1, 2], [3, 4], [1, 3, 4, 2])
    [1]
    >>> all_splice([1, 2, 1, 2], [1, 2], [1, 2, 1, 2, 1, 2])
    [0, 2, 4]
    """

    return ____________________________________________________________________________
```

(c) (5 pt) Implement `splink`, which takes two `Link` instances `a` and `b` and a non-negative integer `k` that is less than or equal to the length of `a`. It returns a `Link` instance containing the first `k` elements of `a`, then all elements of `b`, then the remaining elements of `a`. The `Link` class is defined on the midterm 2 study guide. **Important**: You may **not** use `len`, `in`, `for`, `list`, slicing, element selection, addition, or list comprehensions.

```
def splink(a, b, k):
    """Return a Link containing the first k elements of a, then all of b, then the rest of a.

    >>> splink(Link(2, Link(3, Link(4, Link(5)))), Link(6, Link(7)), 2)
    Link(2, Link(3, Link(6, Link(7, Link(4, Link(5))))))
    """

    if ____________________________________________________________________________:


        return a


    elif ____________________________________________________________________________:


        return ____________________________________________________________________


    return ____________________________________________________________________________
```

4. (11 points) Both Ways

(a) (4 pt) Implement `both`, which takes two *sorted* linked lists composed of `Link` objects and returns whether some value is in both of them. The `Link` class is defined on the midterm 2 study guide.
Important: You may **not** use `len`, `in`, `for`, `list`, slicing, element selection, addition, or list comprehensions.

```
def both(a, b):
    """Return whether there is any value that appears in both a and b, two sorted Link instances.

    >>> both(Link(1, Link(3, Link(5, Link(7)))), Link(2, Link(4, Link(6))))
    False
    >>> both(Link(1, Link(3, Link(5, Link(7)))), Link(2, Link(7, Link(9))))  # both have 7
    True
    >>> both(Link(1, Link(4, Link(5, Link(7)))), Link(2, Link(4, Link(5))))  # both have 4 and 5
    True
    """
    if ____________________________________________________________________________________:

        return False

    if ____________________________________________________________________________________:

        a, b = b, a

    return ____________________________________________________________________________
```

(b) (2 pt) Circle the Θ expression that describes the minimum number of comparisons (e.g., `<`, `>`, `<=`,`==`, or `>=` expressions) required to verify that two sorted lists of length n contain no values in common.

$\Theta(1)$ $\Theta(\log n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(2^n)$ None of these

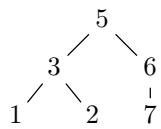
(c) (5 pt) Implement `ways`, which takes two values *start* and *end*, a non-negative integer k, and a list of one-argument functions `actions`. It returns the number of ways of choosing functions f_1, f_2, ..., f_j from `actions`, such that $f_1(f_2(...(f_j(start))))$ equals *end* and $j \leq k$. The same action function can be chosen multiple times. If a sequence of actions reaches *end*, then no further actions can be applied (see the first example below).

```
def ways(start, end, k, actions):
    """Return the number of ways of reaching end from start by taking up to k actions.

    >>> ways(-1, 1, 5, [abs, lambda x: x+2])   # abs(-1) or -1+2, but not abs(abs(-1))
    2
    >>> ways(1, 10, 5, [lambda x: x+1, lambda x: x+4])  # 1+1+4+4, 1+4+4+1, or 1+4+1+4
    3
    >>> ways(1, 20, 5, [lambda x: x+1, lambda x: x+4])
    0
    >>> ways([3], [2, 3, 2, 3], 4, [lambda x: [2]+x, lambda x: 2*x, lambda x: x[:-1]])
    3
    """
    if ____________________________________________________________________________________:

        return 1

    elif ____________________________________________________________________________________:

        return 0

    return ________([_____________________________________________________ for f in actions])
```

5. (9 points) Autumn Leaves

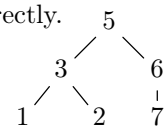
Definition. A *pile* (of leaves) for a tree t with no repeated leaf labels is a dictionary in which the label for each leaf of t is a key, and its value is the path from that leaf to the root. Each path from a node to the root is either an empty tuple, if the node is the root, or a two-element tuple containing the label of the node's parent and the rest of the path (i.e., the path to the root from the node's parent).

(a) (5 pt) Implement `pile`, which takes a tree constructed using the `tree` data abstraction. It returns a *pile* for that tree. You may use the `tree`, `label`, `branches`, and `is_leaf` functions from the midterm 2 study guide.




```
def pile(t):
    """Return a dict that contains every path from a leaf to the root of tree t.

    >>> pile(tree(5, [tree(3, [tree(1), tree(2)]), tree(6, [tree(7)])]))
    {1: (3, (5, ())), 2: (3, (5, ())), 7: (6, (5, ()))}
    """
    p = {}

    def gather(_____________________________________, _____________________________________):

        if is_leaf(u):


            ___________________________________________________________________________

        for b in branches(u):


            ___________________________________________________________________________


    ___________________________________________________________________________________

    return p
```

(b) (4 pt) Implement `Path`, a class whose constructor takes a tree `t` constructed by `tree` and a `leaf_label`. Assume all leaf labels of `t` are unique. When a `Path` is printed, labels in the path from the root to the leaf of `t` with label `leaf_label` are displayed, separated by dashes. Assume `pile` is implemented correctly.




```
class Path:
    """A path through a tree from the root to a leaf, identified by its leaf label.

    >>> a = tree(5, [tree(3, [tree(1), tree(2)]), tree(6, [tree(7)])])
    >>> print(Path(a, 7), Path(a, 2))
    5-6-7 5-3-2
    """
    def __init__(self, t, leaf_label):
        self.pile, self.end = pile(t), leaf_label

    def __str__(self):

        path, s = _____________________________________ , _____________________________________

        while path:

            path, s = _____________________________________ , _____________________________________

        return s
```

CS 61A Fall 2018 — Structure and Interpretation of Computer Programs

MIDTERM 1

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 study guide.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, and `abs`.
- You **may not** use example functions defined on your study guides unless a problem clearly states you can.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.

1. (10 points) What Would Python Display

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
identity = lambda x: x
increment = lambda x: x + 1


def fif(c, t, f, x):
    if c(x):
        return t(x)
    else:
        return f(x)


def bounce(x, y):
    while x < y:
        if x <= (y and x):
            print('a')
        if x > 0:
            print('b')
        elif x > -5:
            print('c')
        x, y = -y, increment(x - y)
        print(y)


crazy = lambda rich: 100 * rich
crazy = lambda rich: crazy(crazy(rich))


def ok(py):
    def h(w):
        print(py // 10)
        return ok(py)
    return lambda h: h(py)
```

Expression	Interactive Output
`pow(10, 2)`	100
`print(4, 5) + 1`	4 5 Error
`print(None, print(1, 2))`	
`fif(abs, print, print, -2)`	
`bounce(1, 2)`	
`crazy(88)`	
`ok(314)(identity)`	

2. (4 points) Implement `factorial`, which computes the factorial of a positive integer n. You may use any of the functions defined above. **You may <u>not</u> write `if`, `else`, `and`, `or`, or `lambda` in your solution.**

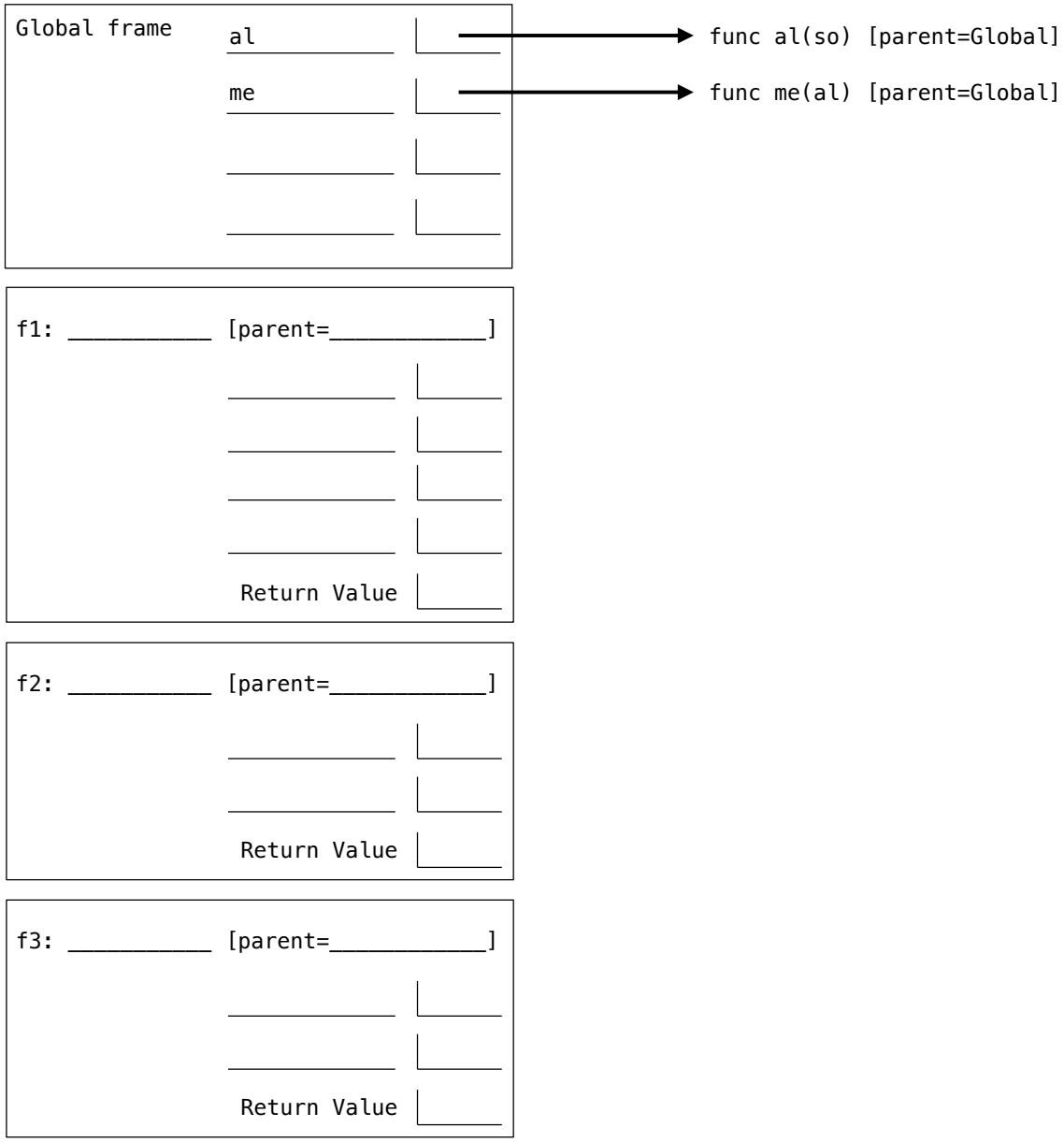
```
def factorial(n):
    """The factorial of positive n: n * (n-1) * (n-2) * ... * 1

    >>> factorial(4)  # 4 * 3 * 2 * 1
    24
    >>> factorial(1)
    1
    """
    return n * fif(_________________, _________________, _________________, _________________)
```

3. (6 points) Also Some Meal

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.* A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  def al(so):
2      me = 1
3      def al(to):
4          return so + me
5      so = 2
6      return al
7
8  def me(al):
9      me = 3
10     return al(lambda: 4) + so
11
12 so = 5
13 so = me(al(6)) + so
```



4. (5 points) Getting Rect

Implement `rect`, which takes two positive integer arguments, `perimeter` and `area`. It returns the **integer** length of the longest side of a rectangle with integer side lengths ℓ and h which has the given perimeter and area. If no such rectangle exists, it returns `False`.

The perimeter of a rectangle with sides ℓ and h is $2\ell + 2h$. The area is $\ell \cdot h$.

Hint: The built-in function `round` takes a number as its argument and returns the nearest integer. For example, `round(2.0)` evaluates to `2`, and `round(2.5)` evaluates to `3`.

```
def rect(area, perimeter):
    """Return the longest side of a rectangle with area and perimeter that has integer sides.

    >>> rect(10, 14)  # A 2 x 5 rectangle
    5
    >>> rect(5, 12)   # A 1 x 5 rectangle
    5
    >>> rect(25, 20)  # A 5 x 5 rectangle
    5
    >>> rect(25, 25)  # A 2.5 x 10 rectangle doesn't count because sides are not integers
    False
    >>> rect(25, 29)  # A 2 x 12.5 rectangle doesn't count because sides are not integers
    False
    >>> rect(100, 50) # A 5 x 20 rectangle
    20
    """


    side = 1


    while side * side ____________________________________________________________________ area:


        other = round(_____________________________________________________________________________)


        if ___________________________________________________________________________________:


            ___________________________________________________________________________


        side = side + 1


    return False
```

5. (6 points) Dig It

Implement `sequence`, which takes a positive integer `n` and a function `term`. It returns an integer whose digits show the n elements of the sequence `term(1)`, `term(2)`, ..., `term(n)` in order. Assume the `term` function takes a positive integer argument and returns a positive integer.

Important: You may not use `pow`, `**`, `log`, `str`, or `len` in your solution.

```
def sequence(n, term):
    """Return the first n terms of a sequence as an integer.

    >>> sequence(6, abs)                  # Terms are 1, 2, 3, 4, 5, 6
    123456
    >>> sequence(5, lambda k: k+8)        # Terms are 9, 10, 11, 12, 13
    910111213
    >>> sequence(4, lambda k: pow(10, k)) # Terms are 10, 100, 1000, 10000
    10100100010000
    """


    t, k = 0, 1


    while ____________________________________________________________________________________:


        m = 1


        x = ____________________________________________________________________________


        ____________________________________________________________________________ m <= x:


            ____________________________________________________________________________


        t = ____________________________________________________________________________


        k = k + 1


    return t
```

6. (9 points) This Again?

Definitions. A *repeatable integer* function takes an integer argument and returns a repeatable integer function.

(a) (6 pt) Implement `repeat`, which is a repeatable integer function that detects repeated arguments. As a side effect of repeated calls, it prints each argument that has been used before in a sequence of repeated calls. Therefore, if an argument appears n times, it is printed $n - 1$ times in total, each time other than the first. The `detector` function is part of the implementation of `repeat`; you must determine how it is used.

Important: You may <u>not</u> use a list, set, or any other data type not covered yet in the course.

```
def repeat(k):
    """When called repeatedly, print each repeated argument.

    >>> f = repeat(1)(7)(7)(3)(4)(2)(5)(1)(6)(5)(1)
    7
    1
    5
    1
    """

    return ____________________________________________________________________(k)


def detector(f):

    def g(i):


        if ____________________________________________________________________:


            ____________________________________________________________________


        return ____________________________________________________________________


    return g
```

(b) (3 pt) Implement `repeat_digits`, which takes a non-negative integer `n`. For the digits of `n` from right to left, it prints each digit that also appears somewhere to its right. Assume `repeat` is implemented correctly.

```
def repeat_digits(n):
    """Print the repeated digits of non-negative integer n.

    >>> repeat_digits(581002821)
    2
    0
    1
    8
    """

    f = ____________________________________________________________________________

    while n:

        f, n = ___________________________________ , ___________________________________
```

CS 61A Fall 2018 — Structure and Interpretation of Computer Programs

MIDTERM 2

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except two hand-written 8.5" × 11" crib sheets of your own creation and the official CS 61A study guides.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, `abs`, `sum`, `next`, `iter`, `list`, `tuple`, `map`, `filter`, `zip`, `all`, and `any`.
- You **may not** use example functions defined on your study guides unless a problem clearly states you can.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.
- You may use the `Tree`, `Link`, and `BTree` classes defined on Page 2 (left column) of the Midterm 2 Study Guide.

1. (12 points) A/B Test

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
x = [1, 2]

class A:
    x = 3
    y = 4

    def __init__(self, y):
        self.a = y
        self.x = b
        self.__str__ = lambda: str(y)

    def __str__(self):
        return 'BB'

class B(A):
    x = [5, 6]

    def __init__(self, y):
        self.a = x[1]
        y[1] = 8

b = B(x)
a = A(6)

def dash(x):
    return print(self.x)

elastigirl = Link(7, Link(8))
elastigirl.first = elastigirl.rest
```

Expression	Interactive Output
`pow(10, 2)`	100
`print(Link(2, Link(3)))`	`<2, 3>`
`[c.a for c in [a, b]]`	
`print(a.x, b.x)`	
`print(b.y, x)`	
`a.__str__()`	
`dash(b)`	
`print(elastigirl)`	

2. (3 points) Implement `lowest`, which takes a list of numbers `s` and returns a list of only the elements of `s` with the smallest absolute value. **You may only write a single name in each blank.**

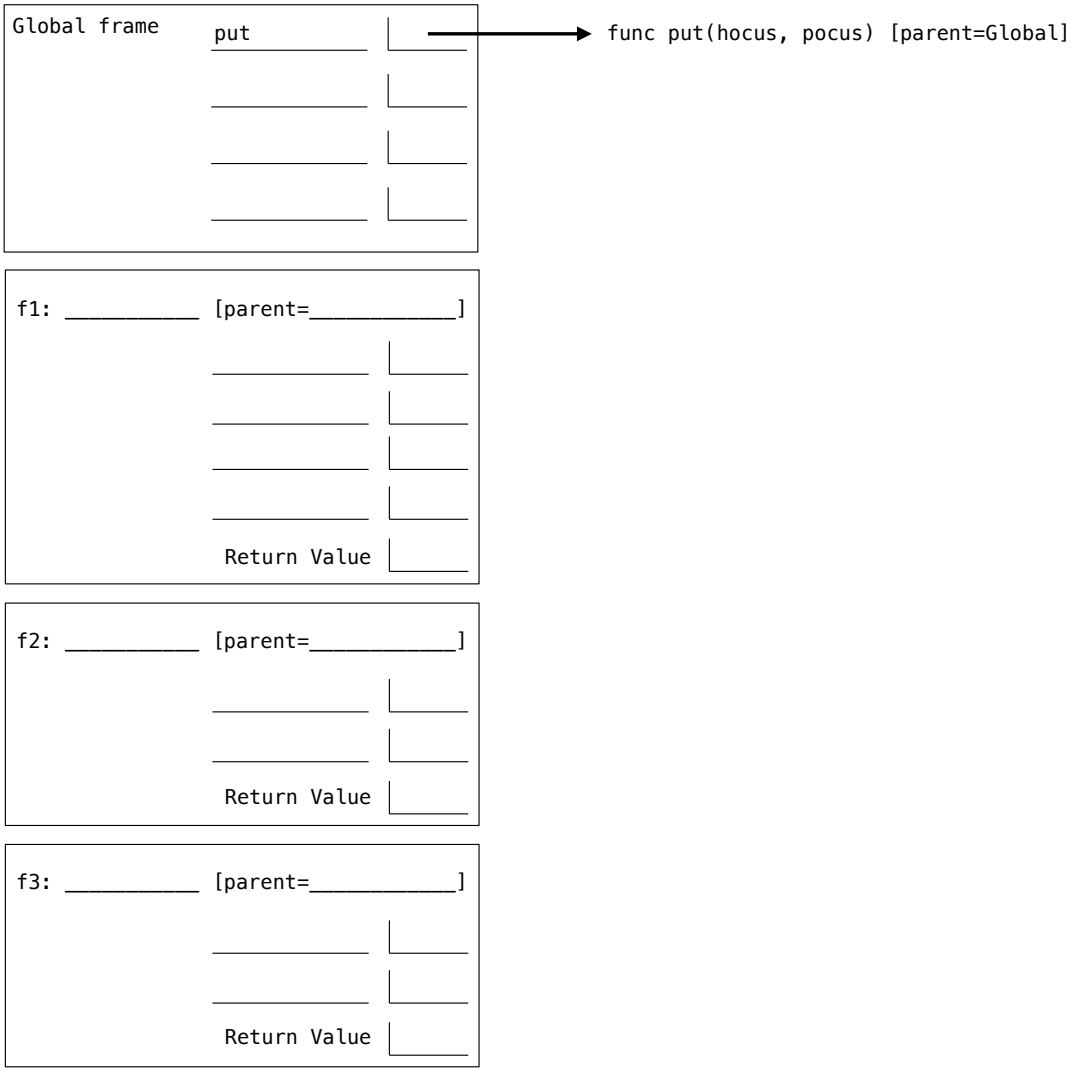
```
def lowest(s):
    """Return a list of the elements in s with the smallest absolute value.

    >>> lowest([3, -2, 2, -3, -4, 2, 3, 4])
    [-2, 2, 2]
    >>> lowest(range(-5, 5))
    [0]
    """
    return (lambda y: [x for ______ in ______ if abs(______)==y])(abs(______(______, key=______)))
```

3. (8 points) Hocus Pocus

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*
A complete answer will:

- Use box-and-pointer notation for all lists.
- Add missing names and parents to all local frames.
- Add missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  def put(hocus, pocus):
2      hocus = 2
3      you = pocus[hocus]
4      def pocus():
5          nonlocal hocus
6          if type(spell.pop()) == list:
7              you.append(hocus)
8              return [pocus(), hocus]
9          else:
10             hocus = 3
11             return spell[0:1]
12     return pocus
13 spell = [6, 5, [4]]
14 you = spell
15 put(spell, you)()
```



4. (10 points) Nonplussed

Definition. A *plus expression* for a non-negative integer `n` is made by inserting `+` symbols in between digits of `n`, such that there are **never more than two consecutive digits** in the resulting expression. For example, one plus expression for 2018 is `20+1+8`, and its value is 29. Assume that a two-digit number starting with 0 evaluates to its one's digit. For example, another plus expression for 2018 is `2+01+8`, and its value is 11.

(a) (3 pt) Implement `plus`, which takes a non-negative integer `n`. It returns the largest value of any plus expression for `n`.

```
def plus(n):
    """Return the largest sum that results from inserting +'s into n.

    >>> plus(123456)     # 12 + 34 + 56 = 102
    102
    >>> plus(1604)       # 1 + 60 + 4 = 65
    65
    >>> plus(160450)     # 1 + 60 + 4 + 50 = 115
    115
    """
    if n:

        return ____________________________________________________________________________

    return 0
```

(b) (5 pt) Implement `plusses`, which takes non-negative integers `n` and `cap`. It returns the number of plus expressions for `n` that have a value less than `cap`.

```
def plusses(n, cap):
    """Return the number of plus expressions for n with values below cap.

    >>> plusses(123, 16)  # 1+2+3=6 and 12+3=15, but 1+23=24 isn't below cap.
    2
    >>> plusses(2018, 38) # 2+0+1+8, 20+1+8, 2+0+18, and 2+01+8, but not 20+18.
    4
    >>> plusses(1, 2)
    1
    """
    if ____________________________________________________________________________________:

        return 1

    elif ____________________________________________________________________________________:

        return 0

    else:

        return ____________________________________________________________________________
```

(c) (2 pt) Circle the Θ expression that describes how many addition operations are required to evaluate a plus expression for a positive integer n.

$\Theta(1)$ $\Theta(\log n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(2^n)$ $\Theta(10^n)$ None of these

5. (8 points) Midterm Elections

(a) (6 pt) Implement the `Poll` class and the `tally` function, which takes a choice `c` and returns a list describing the number of votes for `c`. This list contains pairs, each with a name and the number of times `vote` was called on that `choice` at the `Poll` with that name. Pairs can be in any order. Assume all `Poll` instances have distinct names. *Hint*: the dictionary `get(key, default)` method (MT 2 guide, page 1 top-right) returns the value for a `key` if it appears in the dictionary and `default` otherwise.

```
class Poll:
    s = []
    def __init__(self, n):

        self.name = ____________________________________________________________________________

        self.votes = {}

        ____________________________________________________________________________________________

    def vote(self, choice):

        self._________________________ = ____________________________________________________

def tally(c):
    """Tally all votes for a choice c as a list of (poll name, vote count) pairs.

    >>> a, b, c = Poll('A'), Poll('B'), Poll('C')
    >>> c.vote('dog')
    >>> a.vote('dog')
    >>> a.vote('cat')
    >>> b.vote('cat')
    >>> a.vote('dog')
    >>> tally('dog')
    [('A', 2), ('C', 1)]
    >>> tally('cat')
    [('A', 1), ('B', 1)]
    """

    return ____________________________________________________________________________________
```

(b) (2 pt) Implement the `vote` method of the `Crooked` class, which only records every other `vote` call for each `Crooked` instance. Only odd numbered calls to `vote` are recorded, e.g., first, third, fifth, etc.

```
class Crooked(Poll):
    """A poll that ignores every other call to vote.

    >>> d = Crooked('D')
    >>> for s in ['dog', 'cat', 'dog', 'cat', 'cat']:
    ...     d.vote(s)
    >>> d.votes
    {'dog': 2, 'cat': 1}
    """
    record = True
    def vote(self, choice):
        if self.record:

            _________________ . ____________________________(_____________________________________)

        self._________________ = ____________________________________________________________
```

6. (4 points) Dr. Frankenlink

Implement `replace`, which takes two non-empty linked lists `s` and `t`, as well as **positive** integers `i` and `j` with `i < j`. It mutates `s` by removing elements with indices from `i` to `j` (removing element i but not removing element `j`) and replacing them with `t`. Afterward, `s` contains all of the objects in `t`, so a change to `t` would be reflected in `s` as well. `t` may change as a result of calling `replace`. Assume `s` has at least `j` elements.

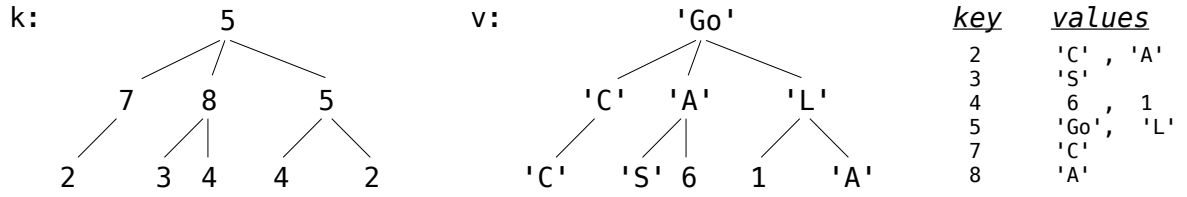
```
def replace(s, t, i, j):
    """Replace the slice of s from i to j with t.

    >>> s, t = Link(3, Link(4, Link(5, Link(6, Link(7))))), Link(0, Link(1, Link(2)))
    >>> replace(s, t, 2, 4)
    >>> print(s)
    <3, 4, 0, 1, 2, 7>
    >>> t.rest.first = 8
    >>> print(s)
    <3, 4, 0, 8, 2, 7>
    """
    assert s is not Link.empty and t is not Link.empty and i > 0 and i < j

    if i > 1:

        ____________________________________________________________________________

    else:

        for k in range(j - i):

            _______________ = ____________________________________________________

        end = t

        while ____________________________________________________________________________:

            end = end.rest

        s.rest, end.rest = ______________________________, ______________________________
```

7. (5 points) Trictionary or Treat

Definitions. A *trictionary* is a pair of `Tree` instances `k` and `v` that have identical structure: each node in `k` has a corresponding node in `v`. The labels in `k` are called *keys*. Each key may be the label for multiple nodes in `k`, and the *values* for that key are the labels of all the corresponding nodes in `v`.

A *lookup function* returns one of the values for a key. Specifically, a lookup function for a node in `k` is a function that takes `v` as an argument and returns the label for the corresponding node in `v`.

Implement the generator function `lookups`, which takes a `Tree` instance `k` and some `key`. It yields all *lookup functions* for nodes in `k` that have `key` as their label. The `new_lookup` function is part of the implementation.




```
k = Tree(5, [Tree(7, [Tree(2)]), Tree(8, [Tree(3), Tree(4)]), Tree(5, [Tree(4), Tree(2)])])
v = Tree('Go', [Tree('C', [Tree('C')]), Tree('A', [Tree('S'), Tree(6)]), Tree('L', [Tree(1), Tree('A')])])

def lookups(k, key):
    """Yield one lookup function for each node of k that has the label key.

    >>> [f(v) for f in lookups(k, 2)]
    ['C', 'A']
    >>> [f(v) for f in lookups(k, 3)]
    ['S']
    >>> [f(v) for f in lookups(k, 6)]
    []
    """

    if ____________________________________________________________________________________________:

        yield lambda v: ____________________________________________________________________________

    for i in range(len(k.branches)):

        ____________________________________________________________________________________________:

            yield new_lookup(i, lookup)

def new_lookup(i, f):

    def g(v):

        return ____________________________________________________________________________________

    return g
```

No more questions.

CS 61A Fall 2019 — Structure and Interpretation of Computer Programs

MIDTERM 1

INSTRUCTIONS

- You have 1 hour and 50 minutes to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 study guide.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, and `abs`.
- You **may not** use example functions defined on your study guide unless a problem clearly states you can.
- For fill-in-the-blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.

1. (12 points) What Would Python Display

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
def mint(y):
    return print(-2)

def snooze(e, f):
    if e and f():
        print(e)
    if e or f():
        print(f)
    if not e:
        print('naughty')

def lose():
    return -1

def alarm():
    print('Midterm')
    1 / 0
    print('Time')

def sim(b, a):
    while a > 1:
        def sc(ar):
            a = b + 4
            return b
        a, b = a // 2, b - a
        print(a)
    print(sc(b - 1), a)

pumbaa = lambda f: lambda x: f(f(x))
pumbaa = pumbaa(pumbaa)
rafiki = 1
timon = lambda y: y + rafiki
rafiki = -1
```

	Expression	Interactive Output
	`pow(10, 2)`	100
	`print(4, 5) + 1`	4 5 Error
(2 pt)	`print(mint(print))`	
(3 pt)	`print(snooze(1, lose))`	
(3 pt)	`snooze(print(1), alarm)`	
(2 pt)	`sim(3, 3)`	
(2 pt)	`pumbaa(timon)(5)`	

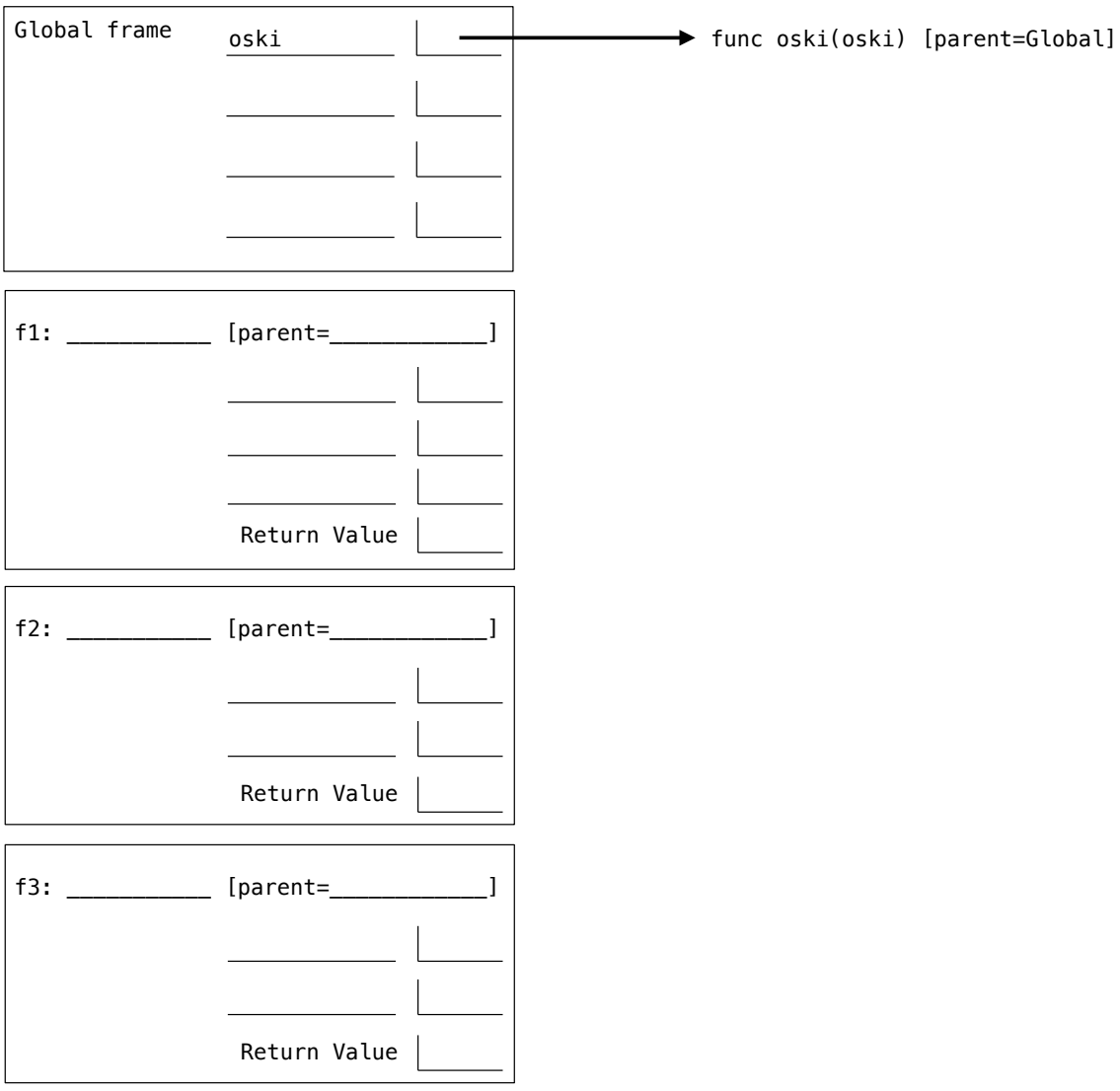
2. (6 points) Environmental Studies

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*

A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1 def oski(oski):
2     x = 1
3     if oski(2) > x:
4         return oski
5 bear = lambda z: (lambda y: z+3)(x+4)
6 x, z = 11, 12
7 x = oski(bear)
```



3. (5 points) You Again

Implement `again`, which takes a function `f` as an argument. The `again` function returns the smallest non-negative integer `n` for which `f(n)` is equal to `f(m)` for some non-negative `m` that is less than `n`. Assume that `f` takes non-negative integers and returns the same value for at least two different non-negative arguments.

Constraints:

- Lines numbered 2, 4, and 5 must begin with either `while` or `if`.
- Lines numbered 6 and 7 must contain either `return` or `=`.

```
def parabola(x):
    """A parabola function (for testing the again function)."""
    return (x-3) * (x-6)

def vee(x):
    """A V-shaped function (for testing the again function)."""
    return abs(x-2)

def again(f):
    """Return the smallest non-negative integer n such that f(n) == f(m) for some m < n.

    >>> again(parabola)  # parabola(4) == parabola(5)
    5
    >>> again(vee)       # vee(1) == vee(3)
    3
    """
```

```
1       n = 1
2       ____________________________________________________________________________________:
3           m = 0
4           ____________________________________________________________________________:
5               ________________________________________________________________________:
6                   ____________________________________________________________________
7               ________________________________________________________________________
8           n = n + 1
```

4. (17 points) Ups and Downs

Definition. Two adjacent digits in a non-negative integer are an *increase* if the left digit is smaller than the right digit, and a *decrease* if the left digit is larger than the right digit.

For example, 61127 has 2 increases ($1 \to 2$ and $2 \to 7$) and 1 decrease ($6 \to 1$).

You may use the `sign` function defined below in all parts of this question.

```
def sign(x):
    if x > 0:
        return 1
    elif x < 0:
        return -1
    else:
        return 0
```

(a) (5 pt) Implement `ramp`, which takes a non-negative integer `n` and returns whether it has more *increases* than *decreases* when reading its digits from left to right (see the definition above).

```
def ramp(n):
    """Return whether non-negative integer N has more increases than decreases.

    >>> ramp(123)   # 2 increases (1 -> 2, 2 -> 3) and 0 decreases
    True
    >>> ramp(1315)  # 2 increases (1 -> 3, 1 -> 5) and 1 decrease (3 -> 1)
    True
    >>> ramp(176)   # 1 increase (1 -> 7) and 1 decrease (7 -> 6)
    False
    >>> ramp(5)     # 0 increases and 0 decreases
    False
    """

    n, last, tally = ______________________________ , ______________________________ , 0

    while n:

        n, last, tally = n // 10, n % 10, ____________________________________________


    return ____________________________________________________________________________
```

(b) (3 pt) Implement `over_under`, which takes a number `y` and returns a function that takes a number `x`. This function returns 1 if `x` is greater than `y`, 0 if `x` equals `y`, and -1 if `x` is less than `y`.

You may <u>not</u> use `if`, `and`, or `or`.

```
def over_under(y):
    """Return a function that takes X and returns:
         -1 if X is less than Y
          0 if X is equal to Y
          1 if X is greater than Y

    >>> over_under(5)(3)  # 3 < 5
    -1
    >>> over_under(5)(5)  # 5 == 5
    0
    >>> over_under(5)(7)  # 7 > 5
    1
    """
    return ____________________________________________________________________________
```

Read this first. The `process` function below uses `tally` and `result` functions to analyze all adjacent pairs of digits in a non-negative integer `n`. A `tally` function is called on each pair of adjacent digits.

```
def process(n, tally, result):
    """Process all pairs of adjacent digits in N using functions TALLY and RESULT."""
    while n >= 10:
        tally, result = tally(n % 100 // 10, n % 10)
        n = n // 10
    return result()
```

(c) (6 pt) Implement `ups`, which returns two functions that can be passed as `tally` and `result` arguments to `process`, so that `process` computes whether a non-negative integer `n` has exactly `k` *increases*.

Hint: You can use `sign` from the previous page and the built-in `max` and `min` functions.

```
def ups(k):
    """Return tally and result functions that compute whether N has exactly K increases.

    >>> f, g = ups(3)
    >>> process(1200849, f, g)    # Exactly 3 increases: 1 -> 2, 0 -> 8, 4 -> 9
    True
    >>> process(94004, f, g)      # 1 increase: 0 -> 4
    False
    >>> process(122333445, f, g)  # 4 increases: 1 -> 2, 2 -> 3, 3 -> 4, 4 -> 5
    False
    >>> process(0, f, g)          # 0 increases
    False
    """
    def f(left, right):


        return ____________(____________________________________________________________)


    return ___________________________________ , ___________________________________
```

(d) (3 pt) Implement `at_most`, which returns `True` if the number of *increases* in a non-negative integer `n` is less than or equal to `k`, and `False` otherwise. Assume `ups` is implemented correctly. You may use any of the functions from previous parts of this question: `sign`, `ramp`, `over_under`, and `process`.

```
def at_most(n, k):
    """Return whether non-negative integer N has at most K increases.

    >>> at_most(567, 3)
    True
    >>> at_most(567, 2)
    True
    >>> at_most(567, 1)
    False
    """
    result = ____________________________________________________________________________

    while k >= 0:

        a, b = ups(k)

        k, result = k - 1, ____________________________________________________________

    return result
```

CS 61A Fall 2019 — Structure and Interpretation of Computer Programs

MIDTERM 2

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except two hand-written 8.5" × 11" crib sheet of your own creation and the provided CS 61A study guides.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, `abs`, `sum`, `next`, `iter`, `list`, `sorted`, `reversed`, `tuple`, `map`, `filter`, `zip`, `all`, and `any`.
- You **may not** use example functions defined on your study guide unless a problem clearly states you can.
- For fill-in-the-blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.
- You may use the `Tree` and `Link` classes defined on Page 2 (left column) of the Midterm 2 Study Guide.

1. (12 points) What Would Python Display

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the contents of the `repr` string of the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left, which cause no errors. Assume that expressions on the right are executed in the order shown. Expressions evaluated by the interpreter have a cumulative effect.

```
class C:
    x = 'e'
    def f(self, y):
        self.x = self.x + y
        return self

    def __str__(self):
        return 'go'

class Big(C):
    x = 'u'
    def f(self, y):
        C.x = C.x + y
        return C.f(self, 'm')

    def __repr__(self):
        return '<bears>'

m = C().f('i')
n = Big().f('o')

def g(y):
    def h(x):
        nonlocal h
        k = 2
        if len(x) == 1:
            h = lambda x: y
            k = 1
        return [-x[0]] + h(x[k:])
    return h(y)

py = {3: {5: [7]}}
thon = {5: 6, 7: py}
thon[7][3] = thon

q = range(3, 5)
r = iter(q)
code = [next(r), next(iter(q)), list(r)]
```

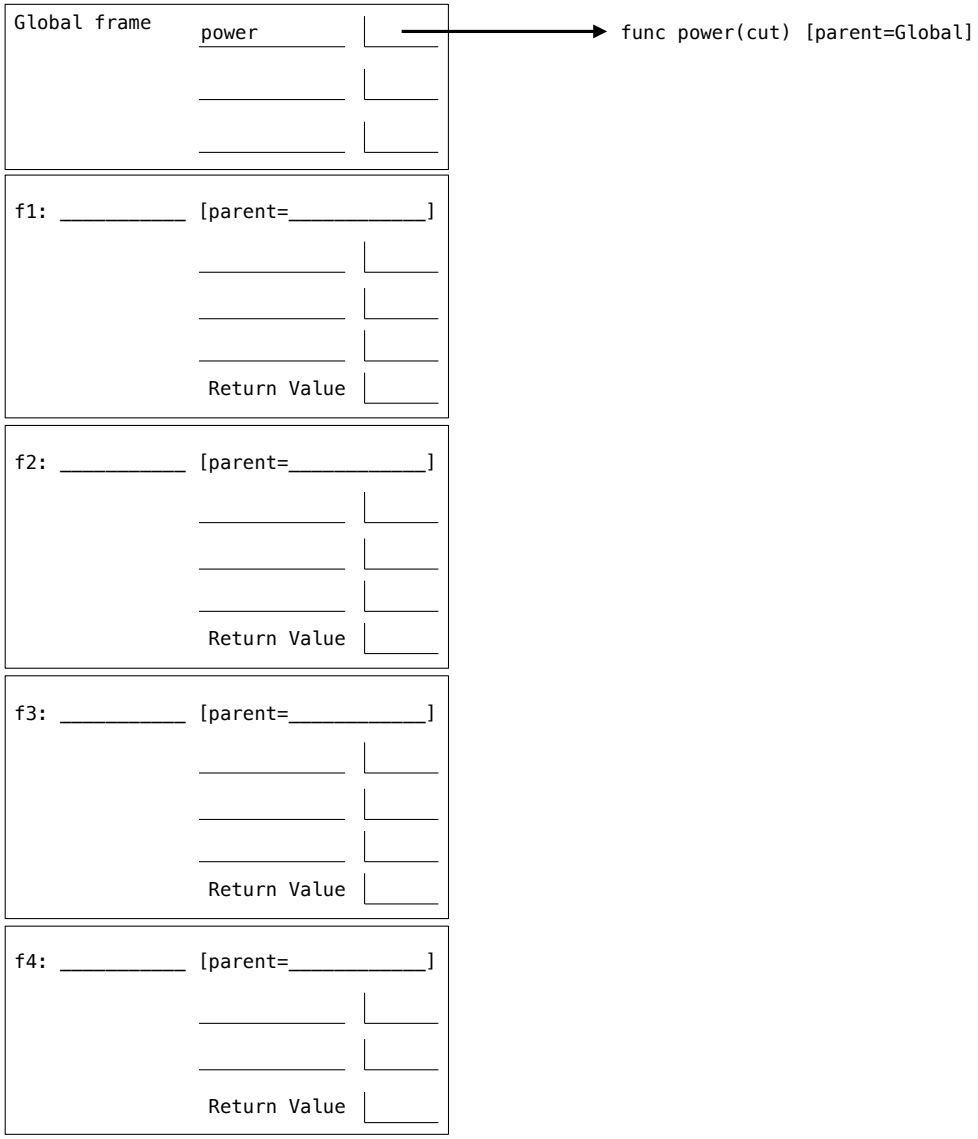
	Expression	Interactive Output
	`pow(10, 2)`	100
	`print(4, 5) + 1`	4 5 Error
(2 pt)	`[m.x, n.x]`	
(2 pt)	`[C.f(n, 'a').x, C().x]`	
(2 pt)	`print(m, n)`	
(1 pt)	`n`	
(2 pt)	`g([3, 4, 5])`	
(1 pt)	`py[3][5]`	
(2 pt)	`code`	

2. (8 points) Environmental Studies

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*
A complete answer will:

- Use box-and-pointer notation for lists.
- Add all missing names and parent annotations.
- Add all missing values created or referenced.
- Show the return value for each local frame.

```
1  def power(cut):
2      def wind(y):
3          nonlocal cut
4          cut, g = cut - 1, cut
5          y = [e]
6          return y is e
7      if cut and not wind('y'):
8          return power(cut)
9      else:
10         return [g, 'e']
11 g = 'p'
12 e = [g]
13 g = 2
14 e.append(power(1))
```



3. (6 points) Max Tree

(a) (4 pt) Implement `max_tree`, which takes a `Tree` instance `t` and a function `key` that takes one argument and returns a number. The `max_tree` function returns the label `n` of `t` for which `key(n)` is largest. If there is more than one label for which `key` returns the same largest value, `max_tree` can return any of those labels. The `Tree` class is defined on the Midterm 2 Study Guide. You may **not** call `min` or `max`.

```
def max_tree(t, key):
    """Return the label n of t for which key(n) returns the largest value.

    >>> t = Tree(6, [Tree(3, [Tree(5)]), Tree(2), Tree(4, [Tree(7)])])
    >>> max_tree(t, key=lambda x: x)
    7
    >>> max_tree(t, key=lambda x: -x)
    2
    >>> max_tree(t, key=lambda x: -abs(x - 4))
    4
    """
    if t.is_leaf():

        return ____________________________________________________________________

    x = ____________________________________________________________________________

    for b in t.branches:

        m = ____________________________________________________________________

        if ____________________________________________________________________:

            x = m

    return x
```

(b) (2 pt) Now implement `max_tree` in one line using a call to the built-in `max` function.
You may **not** include any additional calls to `max` beyond the one included in the template below.

```
def max_tree(t, key):
    "Return the label n of t for which key(n) returns the largest value."

    return max(____________________________________________________________, key=key)
```

4. (4 points) Seek Once

Implement `stable`, which takes a list of numbers `s`, a positive integer `k`, and a non-negative number `n`. It returns whether all pairs of values in `s` with indices that differ by at most `k` have an absolute difference in value of at most `n`.

You may **not** use `lambda`, `if`, `and`, or `or` in your solution.

Note: The "functions that aggregate iterable arguments" on the Midterm 2 Study Guide have the following behavior when called on an empty list:

- `sum([])` evaluates to 0.
- `max([])` causes a `ValueError`.
- `min([])` causes a `ValueError`.
- `all([])` evaluates to `True`.
- `any([])` evaluates to `False`.

```
def stable(s, k, n):
    """Return whether all pairs of elements of S within distance K differ by at most N.

    >>> stable([1, 2, 3, 5, 6], 1, 2)  # All adjacent values differ by at most 2.
    True
    >>> stable([1, 2, 3, 5, 6], 2, 2)  # abs(5-2) is a difference of 3.
    False
    >>> stable([1, 5, 1, 5, 1], 2, 2)  # abs(5-1) is a difference of 4.
    False
    """

    for i in range(len(s)):


        near = range(_____________________________________________________________, i)


        if _________________([ ____________________________________ > n for j in near]):


            return False


    return True
```

5. (7 points) Do You Yield?

(a) (6 pt) Implement `partitions`, which is a generator function that takes positive integers `n` and `m`. It yields strings describing all partitions of `n` using parts up to size `m`. Each partition is a sum of non-increasing positive integers less than or equal to `m` that totals `n`. The `partitions` function yields a string for each partition exactly once.

You may **not** use `lambda`, `if`, `and`, `or`, lists, tuples, or dictionaries in your solution (other than what already appears in the template).

```
def partitions(n, m):
    """Yield all partitions of N using parts up to size M.

    >>> list(partitions(1, 1))
    ['1']
    >>> list(partitions(2, 2))
    ['2', '1 + 1']
    >>> list(partitions(4, 2))
    ['2 + 2', '2 + 1 + 1', '1 + 1 + 1 + 1']
    >>> for p in partitions(6, 4):
    ...     print(p)
    4 + 2
    4 + 1 + 1
    3 + 3
    3 + 2 + 1
    3 + 1 + 1 + 1
    2 + 2 + 2
    2 + 2 + 1 + 1
    2 + 1 + 1 + 1 + 1
    1 + 1 + 1 + 1 + 1 + 1
    """
    if ____________________________________________________________________________:

        yield ____________________________________________________________

    if n > 0 and m > 0:

        for p in partitions(_________________________, _________________________):

            yield ____________________________________________________________

        yield from partitions(_________________________, _________________________)
```

(b) (1 pt) Circle the order of growth of the value of `len(list(partitions(n, 2)))` as a function of `n`. For example, the value of `len(list(partitions(3, 2)))` is 2, because there are 2 partitions of 3 using parts up to size 2: $2 + 1$ and $1 + 1 + 1$. Assume `partitions` is implemented correctly.

Constant Logarithmic Linear Quadratic Exponential None of these

6. (4 points) Best of Both

Definition. A *switch list* `r` for two source lists `s` and `t`, both of length n, is a list where each element `r[i]` for $0 \leq i < n$ is either `s[i]` or `t[i]`. The *switch count* for `r` is the number of indices `i` for which `r[i]` and `r[i-1]` come from different lists. As a special case, index 0 contributes 0 to the switch count if `r[0]` comes from `s` and 1 if it comes from `t`.

Implement `switch`, which takes two lists of numbers `s` and `t` that have the same length, and a non-negative integer `k`. It returns the *switch list* for `s` and `t` that has the largest sum and has a *switch count* of at most `k`.

Constraints:

- The blanks on line 4 must be filled with one of the following combinations:
  - `s, t`
  - `t, s`
  - `s[1:], t`
  - `t, s[1:]`
  - `s[1:], t[1:]`
  - `t[1:], s[1:]`
- The blank on line 5 must use one of the following templates:
  - `_______ + switch(_______, _______, k - 1)`
  - `_______ + switch(_______, _______, k)`
  - `switch(_______, _______, k - 1)`
  - `switch(_______, _______, k)`

```
def switch(s, t, k):
    """Return the list with the largest sum built by switching between S and T at most K times.

    >>> switch([1, 2, 7], [3, 4, 5], 0)
    [1, 2, 7]
    >>> switch([1, 2, 7], [3, 4, 5], 1)
    [3, 4, 5]
    >>> switch([1, 2, 7], [3, 4, 5], 2)
    [3, 4, 7]
    >>> switch([1, 2, 7], [3, 4, 5], 3)
    [3, 4, 7]
    """
1   if k == 0 or len(s) == 0:

2       return s

3   else:

4       a = switch(_____________, ____________________________, k-1)

5       b = ____________________________________________________________________________

6       return max(a, b, key=sum)
```

7. (5 points) Version 2.0

Implement the `Version`, `Insert`, and `Delete` classes as follows:

- A `Version` represents a string that has been edited. A `Version` instance has a `previous` value (which may be a `str` or `Version` instance) and an `edit` (which may be an instance of `Insert` or `Delete`).
- Calling `str` on a `Version` instance should return the string that results from applying its `edit` to the string represented by `previous`. Assume that string is long enough to perform the `edit`.
- An `edit` object represents a change to a string parameterized by some value `c` applied at an index `i`.
- The `apply` method for `Insert` inserts the string `c` into string `t` starting at index `i`.
- The `apply` method for `Delete` removes `c` (a positive `int`) characters from string `t` starting at index `i`.

```
class Version:
    """A version of a string after an edit.

    >>> s = Version('No power?', Delete(3, 6))
    >>> print(Version(s, Insert(3, 'class!')))
    No class!
    >>> t = Version('Beary', Insert(4, 'kele'))
    >>> print(t)
    Bearkeley
    >>> print(Version(t, Delete(2, 1)))
    Berkeley
    >>> print(Version(t, Delete(4, 5)))
    Bear
    """
    def __init__(self, previous, edit):
        self.previous, self.edit = previous, edit
    def __str__(self):


        return ____________________________________________________________________________________

class Edit:
    def __init__(self, i, c):
        self.i, self.c = i, c

class Insert(Edit):
    def apply(self, t):
        "Return a new string by inserting string c into t starting at position i."


        return ____________________________________________________________________________________

class Delete(Edit):
    def apply(self, t):
        "Return a new string by deleting c characters from t starting at position i."


        return ____________________________________________________________________________________
```

8. (4 points) Their Mascot is a Tree?!?

Definitions. The *depth* of the root node of a tree is 0, and the depth of a child is 1 more than the depth of its parent. A linked list is either a `Link` instance or `Link.empty`.

Implement `layer`, which takes a `Tree` instance `t` and a non-negative integer `d`. It returns a linked list containing the labels of all nodes in `t` that have depth `d`. Labels in the result should appear in branch order. The `Tree` and `Link` classes appear on the Midterm 2 Study Guide.

You may **not** use `lambda`, `if`, `and`, or `or` in your solution (other than what already appears in the template).

Note: The built-in `reversed` function returns a reverse iterator over the values of a sequence. For example, `list(reversed([1, 2, 3]))` evaluates to `[3, 2, 1]`.

```
def layer(t, d):
    """Return a linked list containing all labels of Tree T at depth D.

    >>> a_tree = Tree(1, [Tree('b', [Tree('mas')]),
    ...                   Tree('a', [Tree('co')]),
    ...                   Tree('d', [Tree('t', [Tree('!')])])])
    >>> print(layer(a_tree, 0))
    <1>
    >>> print(layer(a_tree, 1))
    <b a d>
    >>> print(layer(a_tree, 2))
    <mas co t>
    >>> print(layer(a_tree, 3))
    <!>
    """
    return helper(t, d, ____________________________________________________________________________)


def helper(t, d, s):


    if d == 0:


        return ____________________________________________________________________________


    else:


        for b in reversed(t.branches):


            s = ____________________________________________________________________________


        return s
```

CS 61A Fall 2020

Structure and Interpretation of Computer Programs

MIDTERM 1

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

1. (14.0 points) Down for the Count

Definition. A *digit* is a non-negative integer less than 10. Integers contain digits.

Examples.

- The integer 21 contains the digits 1 and 2.
- The integer 474 contains the digit 4 twice and the digit 7 once.
- The integer 400 contains the digit 4 once and the digit 0 twice.
- The integer -77 contains the digit 7 twice.
- The integer 0 is a 0-digit number that contains no digits.

Reminders.

- You may call built-in functions that do not require import, such as `min`, `max`, `abs`, and `pow`.
- You may call functions defined in earlier parts of the question in your implementation for later parts, and you may assume that the functions you call are implemented correctly.

RESTRICTION. You may not call `str` or `repr` or use `[` or `]` in any part of this question.

(a) (4.0 points)

Implement `count`, which takes a digit `element` and an integer `box`. It returns the number of times that `element` appears in `box`.

Warning: `n % d` and `n // d` may not behave as you expect for negative `n`. For example, `-123 % 10` evaluates to 7. `-1 // 10` evaluates to -1. You do not need to know how these operators apply to negative `n` in order to solve this problem.

```
def count(element, box):
    """Count how many times digit element appears in integer box.

    >>> count(2, 222122)
    5
    >>> count(0, -2020)
    2
    >>> count(0, 0)  # 0 has no digits
    0
    """
    assert element >= 0 and element < 10


    _________
       (a)

    total = 0

    while box > 0:

        if _________:
              (b)

            total = _________
                       (c)

        box = box // 10

    return total
```

i. (2.0 pt) Fill in blank (a).

ii. (1.0 pt) Which of these could fill in blank (b)?

- ◯ `box == element`
- ◯ `box % 10 == element`
- ◯ `box % element == 0`
- ◯ `box % element > 0`

iii. (1.0 pt) Which of these could fill in blank (c)?

- ◯ `total + 1`
- ◯ `element`
- ◯ `total + element`
- ◯ `box % 10`
- ◯ `total + box % 10`

(b) (5.0 points)

Implement `count_nine`, which takes a digit `element` and a non-negative integer `box`. It returns the number of times that `element` appears in `box` and is not adjacent to a 9.

```
def count_nine(element, box):
    """Count how many times digit element appears in the non-negative integer
    box in a place that is not next to a 9.

    >>> count_nine(2, 222122)
    5
    >>> count_nine(1, 1911191) # Only the middle 1 is not next to a 9
    1
    >>> count_nine(9, 9)
    1
    >>> count_nine(9, 99)
    0
    >>> count_nine(3, 314159265359)
    2
    >>> count_nine(5, 314159265359)
    1
    >>> count_nine(9, 314159265359)
    2
    >>> count_nine(0, 0) # No digits are in 0
    0
    """
    assert element >= 0 and element < 10
    assert box >= 0

    nine, total = False, 0

    while box > 0:

        if _________ and not (nine or _________):
              (a)                       (b)

            total = _________
                      (c)

        nine = _________ == 9
                  (d)

        box = box // 10

    return total
```

i. (1.0 pt) Which of these could fill in blank (a)?

◯ `box == element`

◯ `box % 10 == element`

◯ `box % element == 0`

◯ `box % element > 0`

ii. (2.0 pt) Fill in blank (b).

iii. (1.0 pt) Which of these could fill in blank (c)?

- ◯ `total + 1`
- ◯ `element`
- ◯ `total + element`
- ◯ `box % 10`
- ◯ `total + box % 10`

iv. (1.0 pt) Fill in blank (d).

(c) (5.0 points)

Implement `fit`, which takes two non-negative integers `pegs` and `holes`. It returns whether every digit in `pegs` appears at least as many times in `holes` as it does in `pegs`.

```
def fit(pegs, holes):
    """Return whether every digit in pegs appears at least as many times in
    holes as it does in pegs.

    >>> fit(123, 321)    # Each digit appears once in pegs and in holes.
    True
    >>> fit(1213, 33221) # 1 appears twice in pegs, but only once in holes.
    False
    >>> fit(12, 22)      # 1 appears once in pegs, but not at all in holes.
    False
    >>> fit(314159, 112233456789)
    True
    """
    i = 0

    while i <= _________:
                  (a)

        if _________:
              (b)

            _________
               (c)

        i = i + 1

    return _________
              (d)
```

i. (1.0 pt) Fill in blank (a).

ii. (2.0 pt) Fill in blank (b).

iii. (1.0 pt) Fill in blank (c).

iv. (1.0 pt) Which of these could fill in blank (d)?

- ◯ `True`
- ◯ `False`
- ◯ `holes > pegs`
- ◯ `pegs > holes`
- ◯ `holes >= pegs`
- ◯ `pegs >= holes`

2. (8.0 points) Mystery Function

Assume `mystery` is a deterministic pure function that takes one integer argument, returns an integer, and never errors.

```
def mystery(n):
    ...
```

Assume the following functions are also defined:

```
def add_two(y):
    return y + 2

def two(y):
    return 2

def constant(k):
    def ignore(x):
        return k
    return ignore

def diff(f, g):
    return lambda z: abs(f(z) - g(z))
```

Definition. Two functions `f` and `g` have identical behavior if `f(x)` and `g(x)` return equal values or return functions with identical behavior, for every `x` that does not cause an error.

Complete each statement below so that it is true for all possible deterministic pure `mystery` functions.

(a) (2.0 pt) The result of evaluating `constant(2)` has identical behavior to the result of evaluating the expression...

◯ ... `add_two`

◯ ... `add_two(0)`

◯ ... `add_two(2)`

◯ ... `two`

◯ ... `two(0)`

◯ ... `two(2)`

◯ None of these

(b) (2.0 pt) The result of evaluating `diff(constant(1), constant(-1))` has identical behavior to the result of evaluating the expression...

◯ ... `constant`

◯ ... `constant(0)`

◯ ... `constant(2)`

◯ ... `diff(constant, constant)`

◯ None of these

(c) (2.0 pt) The result of evaluating `diff(mystery, mystery)` has identical behavior to the result of evaluating the expression...

- ◯ ... `constant`
- ◯ ... `constant(0)`
- ◯ ... `constant(2)`
- ◯ ... `diff(constant, constant)`
- ◯ ... `constant(mystery)`
- ◯ ... `mystery`
- ◯ None of these

(d) (2.0 pt) The result of evaluating `diff(mystery, diff(mystery, mystery))` has identical behavior to the result of evaluating the expression...

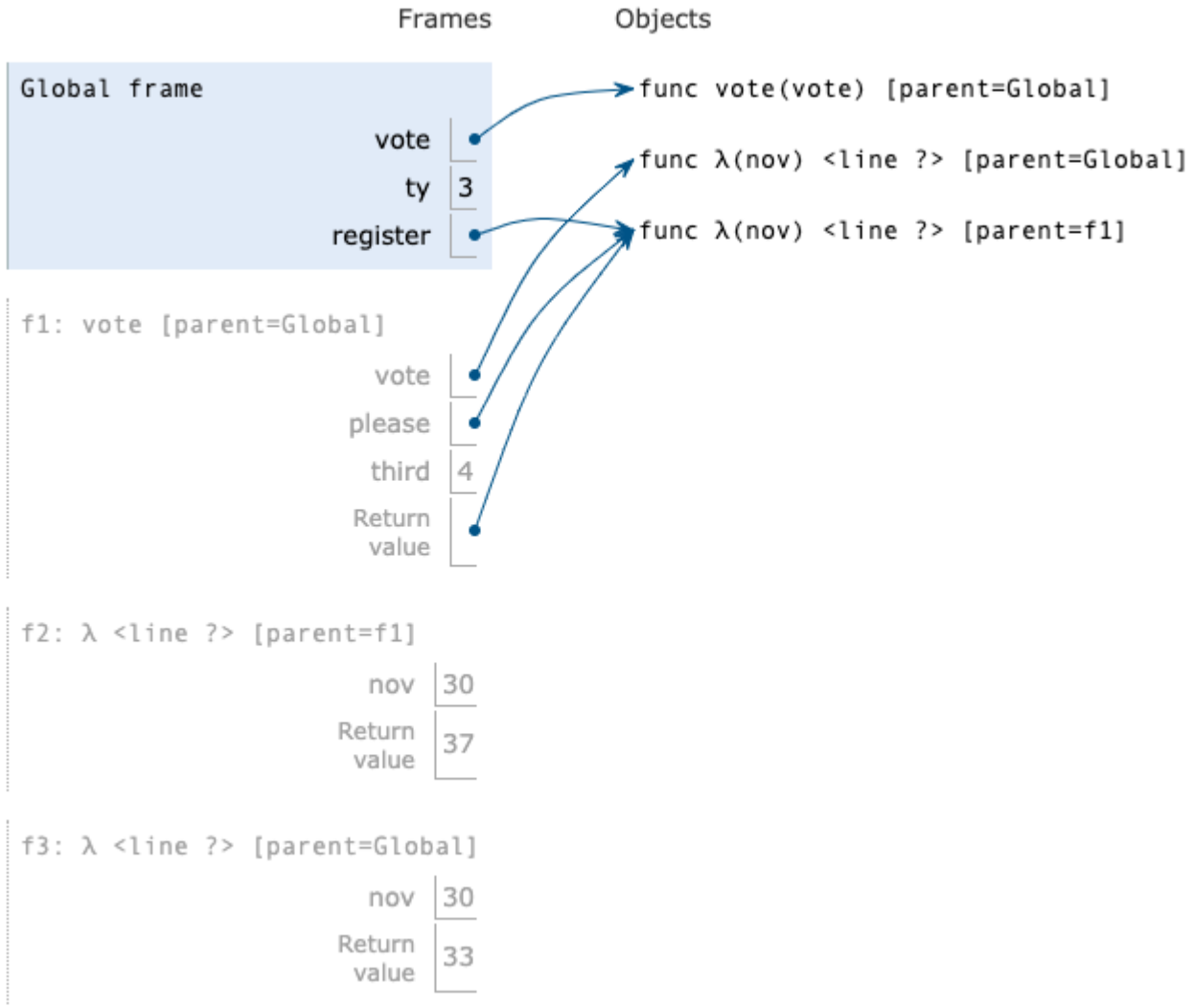
- ◯ ... `mystery`
- ◯ ... `abs(mystery)`
- ◯ ... `lambda y: abs(mystery(y))`
- ◯ ... `lambda y: mystery(abs(y))`
- ◯ ... `lambda y: lambda z: mystery(abs(y)) - mystery(abs(z))`
- ◯ ... `lambda y: lambda z: abs(mystery(y)) - abs(mystery(z))`
- ◯ ... `lambda y: lambda z: abs(mystery(y) - mystery(z))`
- ◯ None of these

3. (8.0 points) Please Register to Vote

Fill in each blank in the code example below so that its environment diagram is the following.

RESTRICTIONS. You must use all of the blanks. Each blank can only include one statement or expression.

Click here to open the diagram in a new window




```
def vote(vote):

    please = _________
                (a)

    _________ = ty + 3
       (b)

    return please

ty = 1

register = _________(lambda nov: nov + ty)
                (c)
```

```
---------
   (d)

register(_________)
            (e)
```

(a) (2.0 pt) Which of these could fill in blank (a)?

- ◯ `vote(ty)`
- ◯ `vote(30)`
- ◯ `vote`
- ◯ `lambda nov: vote(nov) + third`
- ◯ `lambda nov: vote(nov + third)`
- ◯ `lambda nov: vote(nov) + ty`
- ◯ `lambda nov: vote(nov + ty)`

(b) (1.0 pt) Which of these could fill in blank (b)?

- ◯ `third`
- ◯ `ty`
- ◯ `please`
- ◯ `vote`

(c) (1.0 pt) Which of these could fill in blank (c)?

- ◯ `third`
- ◯ `ty`
- ◯ `please`
- ◯ `vote`

(d) (2.0 pt) Fill in blank (d).

(e) (2.0 pt) Which of these could fill blank (e)? **Check all that apply.**

- ☐ `ty * 10`
- ☐ `ty - 1 + 30`
- ☐ `30`
- ☐ `third + 26`
- ☐ `(lambda x: x + x)(15)`

4. (10.0 points) Amazing Job Growth

Definition. A *repeatable* function is a function that returns a repeatable function.

Reminder. You may call built-in functions that do not require import, such as `min`, `max`, `abs`, and `pow`.

(a) (4.0 points)

Implement `growth`, which takes a number `baseline` and returns a repeatable function `increase`. When `increase` is called on a number `observed`, it prints the difference between `observed` and the smallest argument passed to `growth` or `increase` so far among the repeated calls.

```
def growth(baseline):
    """Return a function that can be called repeatedly on numbers and prints
    the difference between its argument and the smallest argument used so far
    (including baseline).

    >>> job = growth(148)(149)(150)(130)(133)(139)(137)
    1
    2
    0
    3
    9
    7
    """
    def increase(observed):

        under = _________
                  (a)

        print(observed - under)

        return _________
                  (b)

    return increase
```

i. (2.0 pt) Fill in blank (a).

ii. (2.0 pt) Which of these could fill in blank (b)?

- ◯ `increase`
- ◯ `increase(under)`
- ◯ `increase(observed)`
- ◯ `increase(baseline)`
- ◯ `growth`
- ◯ `growth(under)`
- ◯ `growth(observed)`
- ◯ `growth(baseline)`

(b) (6.0 points)

Implement `maxer`, a higher-order function that takes a function `smoke`, which takes a number and returns a number. The `maxer` function returns a repeatable function `fire` that takes a number `y` and prints the largest result of calling `smoke` on any value of `y` passed to `fire` so far among the repeated calls.

Assume that `smoke` is a deterministic pure function.

```
def square(x):
    return x * x

def maxer(smoke):
    """Return a repeatable function fire(y) that prints the largest smoke(y) so far.

    >>> g = maxer(square)
    >>> h = g(2)(1)(3)(2)(-4)  # print the largest square(y) so far
    4
    4
    9
    9
    16
    >>> h = maxer(abs)(2)(1)(3)(2)(-4)  # print the largest abs(y) so far
    2
    2
    3
    3
    4
    """

    def fire(y):

        _________
           (a)

        def haze(z):

            if _________:
                  (b)

                z = y

            return _________
                       (c)

        return haze

    return fire
```

i. (2.0 pt) Fill in blank (a). You may not write a `return` statement for this blank.

ii. (2.0 pt) Fill in blank (b).

iii. (2.0 pt) Which of these could fill in blank (c)?

- ◯ `y`
- ◯ `smoke(y)`
- ◯ `fire(y)`
- ◯ `fire(smoke(y))`
- ◯ `haze`
- ◯ `haze(y)`
- ◯ `haze(smoke(y))`
- ◯ `z`
- ◯ `smoke(z)`
- ◯ `fire(z)`
- ◯ `fire(smoke(z))`
- ◯ `haze(z)`
- ◯ `haze(smoke(z))`

No more questions.

CS 61A Fall 2020 — Structure and Interpretation of Computer Programs

MIDTERM 2

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

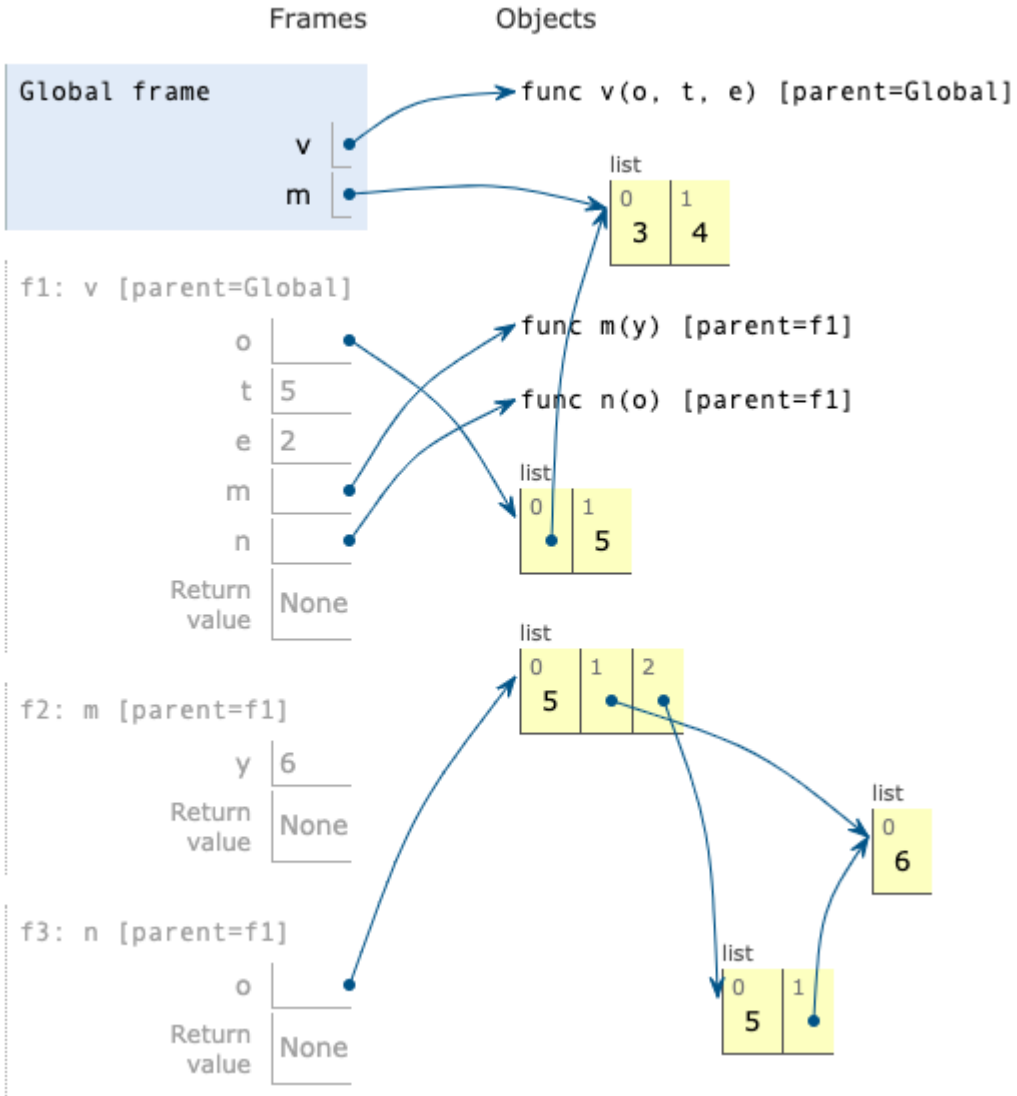
(b) What is your student ID number?

1. (8.0 points) Political Environment

Fill in each blank in the code example below so that executing it would generate the following environment diagram.

RESTRICTIONS. You must use all of the blanks. Each blank can only include one statement or expression.

Click here to open the diagram in a new window/tab




```
def v(o, t, e):
    def m(y):

        ---------
           (a)

        ---------
           (b)

    def n(o):

        o.append(_________)
```

```
                    (c)

        o.append(_________)
                    (d)

    m(e)
    n([t])
    e = 2

m = [3, 4]

v(m, 5, 6)
```

(a) (2.0 pt) Fill in blank (a). **You may not write any numbers or arithmetic operators (`+`, `-`, `*`, `/`, `//`, `**`) in your solution.**

(b) (2.0 pt) Fill in blank (b). **You may not write any numbers or arithmetic operators (`+`, `-`, `*`, `/`, `//`, `**`) in your solution.**

(c) (2.0 pt) Fill in blank (c). **You may not write any numbers or arithmetic operators (`+`, `-`, `*`, `/`, `//`, `**`) in your solution.**

(d) (2.0 pt) Which of these could fill in blank (d)? **Check all that apply.**

- ☐ `o`
- ☐ `[o]`
- ☐ `list(o)`
- ☐ `[list(o)]`
- ☐ `list([o])`
- ☐ `o + []`
- ☐ `[o[0], o[1]]`
- ☐ `o[:]`

2. (10.0 points) Yield, Fibonacci!

(a) (4.0 points)

Implement `fibs`, a generator function that takes a one-argument pure function `f` and yields all Fibonacci numbers `x` for which `f(x)` returns a true value.

The Fibonacci numbers begin with 0 and then 1. Each subsequent Fibonacci number is the sum of the previous two. Yield the Fibonacci numbers in order.

```
def fibs(f):
    """Yield all Fibonacci numbers x for which f(x) is a true value.

    >>> odds = fibs(lambda x: x % 2 == 1)
    >>> [next(odds) for i in range(10)]
    [1, 1, 3, 5, 13, 21, 55, 89, 233, 377]
    >>> bigs = fibs(lambda x: x > 20)
    >>> [next(bigs) for i in range(10)]
    [21, 34, 55, 89, 144, 233, 377, 610, 987, 1597]
    >>> evens = fibs(lambda x: x % 2 == 0)
    >>> [next(evens) for i in range(10)]
    [0, 2, 8, 34, 144, 610, 2584, 10946, 46368, 196418]
    """

    n, m = 0, 1

    while _________:
             (a)

        if _________:
              (b)

            _________
               (c)

        _________
           (d)
```

i. (1.0 pt) Which of these could fill in blank (a)?

- ◯ `f(n)`
- ◯ `f(m)`
- ◯ `f(n) or f(m)`
- ◯ `f(n) and f(m)`
- ◯ `True`
- ◯ `False`

ii. (1.0 pt) Which of these could fill in blank (b)?

- ◯ `f(n)`
- ◯ `f(m)`
- ◯ `f(n) or f(m)`
- ◯ `f(n) and f(m)`
- ◯ `True`
- ◯ `False`

iii. (1.0 pt) Fill in blank (c).

iv. (1.0 pt) Fill in blank (d).

(b) (6.0 points)

Definition. For a linked list `s`, the *index* of an element is the number of times `rest` appears in the smallest dot expression containing only `s`, `rest`, and `first` that evaluates to that element. For example, in the linked list `s = Link(5, Link(7, Link(9, Link(11))))`,

- The index of 5 (`s.first`) is 0.
- The index of 7 (`s.rest.first`) is 1.
- The index of 11 (`s.rest.rest.rest.first`) is 3.

Implement `filter_index`, a function that takes a one-argument pure function `f` and a `Link` instance `s`. It returns a `Link` containing all elements of `s` that have an **index** `i` for which `f(i)` returns a true value.

Assume that `s` is a finite linked list of numbers that contains no repeated elements. The `Link` class appears on Page 2 (left column) of the Midterm 2 Study Guide.

```
def filter_index(f, s):
    """Return a Link containing the elements of Link s that have an index i for
    which f(i) is a true value.

    >>> powers = Link(1, Link(2, Link(4, Link(8, Link(16, Link(32))))))
    >>> filter_index(lambda x: x < 4, powers)
    Link(1, Link(2, Link(4, Link(8))))
    >>> filter_index(lambda x: x % 2 == 1, powers)
    Link(2, Link(8, Link(32)))
    """

    def helper(i, s):

        if s is Link.empty:

            return s

        filtered_rest = _________
                            (a)
        if _________:
              (b)

            return _________
                       (c)

        else:

            return filtered_rest

    return _________
              (d)
```

i. **(1.0 pt)** Which of these could fill in blank (a)?

- ◯ `helper(i + 1, s.rest)`
- ◯ `helper(i + 1, s.rest.rest)`
- ◯ `filter_index(f, s.rest)`
- ◯ `filter_index(f, s.rest.rest)`
- ◯ `Link(helper(i + 1, s.rest))`
- ◯ `Link(helper(i + 1, s.rest.rest))`
- ◯ `Link(filter_index(f, s.rest))`
- ◯ `Link(filter_index(f, s.rest.rest))`

ii. **(1.0 pt)** Fill in blank (b).

iii. **(2.0 pt)** Fill in blank (c).

iv. **(2.0 pt)** Fill in blank (d).

3. (12.0 points) Sparse Lists

The `most_common` function returns the most common element in a non-empty list. You do not need to implement this function. Assume that it is implemented for you.

```
def most_common(s):
    """Return the most common element in non-empty list s. In case of a tie,
    return the most common element that appears first in s.

    >>> most_common([3, 1, 4, 1, 5, 9])
    1
    >>> most_common([3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5])
    5
    >>> most_common([2, 7, 1, 8, 2, 8, 1, 8, 2, 8])
    8
    >>> most_common([3, 5, 7, 7, 7, 5, 5])
    5
    >>> most_common([3, 7, 5, 5, 7, 7])
    7
    """
```

Implement the `SparseList` class. A `SparseList` instance represents a non-empty list `s`.

- Its `common` attribute is the most common value in `s`.
- Its `others` dictionary has a value for every element in `s` that is not `common`. The corresponding key is the **index** for that value in `s`.
- Its `item` method takes a non-negative integer `i` and returns `s[i]` or the string `'out of range'` if `i` is not smaller than the length of `s`.
- Its `items` method returns a list with the same elements as `s` in the same order as `s`.

```
class SparseList:
    """Represent a non-empty list as a most common value and a dictionary from
    indices to values that contains only values that are not the most common.

    >>> pi = SparseList([3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5])
    >>> pi.common
    5
    >>> pi.others
    {0: 3, 1: 1, 2: 4, 3: 1, 5: 9, 6: 2, 7: 6, 9: 3}
    >>> [pi.item(0), pi.item(1), pi.item(2), pi.item(3), pi.item(4)]
    [3, 1, 4, 1, 5]
    >>> pi.item(10)
    5
    >>> pi.item(11)
    'out of range'
    >>> pi.items()
    [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
    """
    def __init__(self, s):
        ...
    def item(self, i):
        ...
    def items(self):
        ...
```

(a) (5.0 points)

Implement the `__init__` method, which takes a list `s`.

```
def __init__(self, s):

    assert s, 's cannot be empty'

    self.n = len(s)

    self.common = most_common(_________)
                              (a)

    self.others = { _________: _________ for i in range(_________) if _________ }
                    (b)        (c)                      (d)           (e)
```

i. **(1.0 pt)** Fill in blank (a).

ii. **(1.0 pt)** Which of these could fill in blank (b)?

- ◯ `i`
- ◯ `self.i`
- ◯ `n`
- ◯ `self.n`
- ◯ `s[i]`
- ◯ `s`

iii. **(1.0 pt)** Fill in blank (c).

iv. **(1.0 pt)** Which of these could fill in blank (d)? **Check all that apply.**

- ☐ `s`
- ☐ `len(s)`
- ☐ `self.s`
- ☐ `len(self.s)`
- ☐ `n`
- ☐ `len(n)`
- ☐ `self.n`
- ☐ `len(self.n)`

v. **(1.0 pt)** Fill in blank (e).

(b) (3.0 points)

Implement the `item` method, which takes a non-negative integer `i`.

```
def item(self, i):
    """Return s[i] or 'out of range' if i is not smaller than the length of s."""

    assert i >= 0, 'index i must be non-negative'

    if _________:
          (a)

        return 'out of range'

    elif _________:
            (b)

        return _________
                  (c)

    else:

        return self.common
```

i. (1.0 pt) Fill in blank (a).

ii. (1.0 pt) Fill in blank (b).

iii. (1.0 pt) Which of these could fill in blank (c)?

- ◯ `others[i]`
- ◯ `self.others[i]`
- ◯ `others[self.i]`
- ◯ `self.others[self.i]`
- ◯ `others`
- ◯ `self.others`

(c) (4.0 points)

Implement the `items` method.

```
def items(self):

    """Return a list with the same elements as s in the same order as s."""

    return [_________ for i in _________]
               (a)                (b)
```

i. (2.0 pt) Fill in blank (a). You may not use `and`, `or`, `if`, `else`, `[`, `]`, or `get`.

Hint: Don't repeat yourself.

ii. (2.0 pt) Fill in blank (b).

4. (20.0 points) Fork It

The *tree* data abstraction, which is implemented by the constructor `tree`, selectors `branches` and `label`, and helper functions `is_leaf` and `is_tree` appear on Page 2 (left column) of the Midterm 2 Study Guide. You may call these functions. Do not violate the abstraction barriers of the tree data abstraction.

(a) (4.0 points)

Implement `max_path`, which takes a tree `t` whose labels are **all positive** numbers and returns the largest sum of the labels along a path from the root of `t` to one of its leaves.

You may call `tree`, `label`, `branches`, `is_leaf`, `is_tree`, and `max_path`.

```
def max_path(t):
    """Return the largest sum of labels along any path from the root to a leaf
    of tree t, which has positive numbers as labels.

    >>> a = tree(1, [tree(2), tree(3), tree(4, [tree(5)])])
    >>> max_path(a) # 1 + 4 + 5
    10
    >>> b = tree(6, [a, a, a])
    >>> max_path(b) # 6 + 1 + 4 + 5
    16
    """

    return _________ + max(_________ + _________)
              (a)              (b)          (c)
```

i. (1.0 pt) Which of the following could fill in blank (a)?

- ◯ `t`
- ◯ `label(t)`
- ◯ `[t]`
- ◯ `[label(t)]`
- ◯ `[0]`
- ◯ `sum([b for b in branches(t)])`
- ◯ `sum([label(b) for b in branches(t)])`

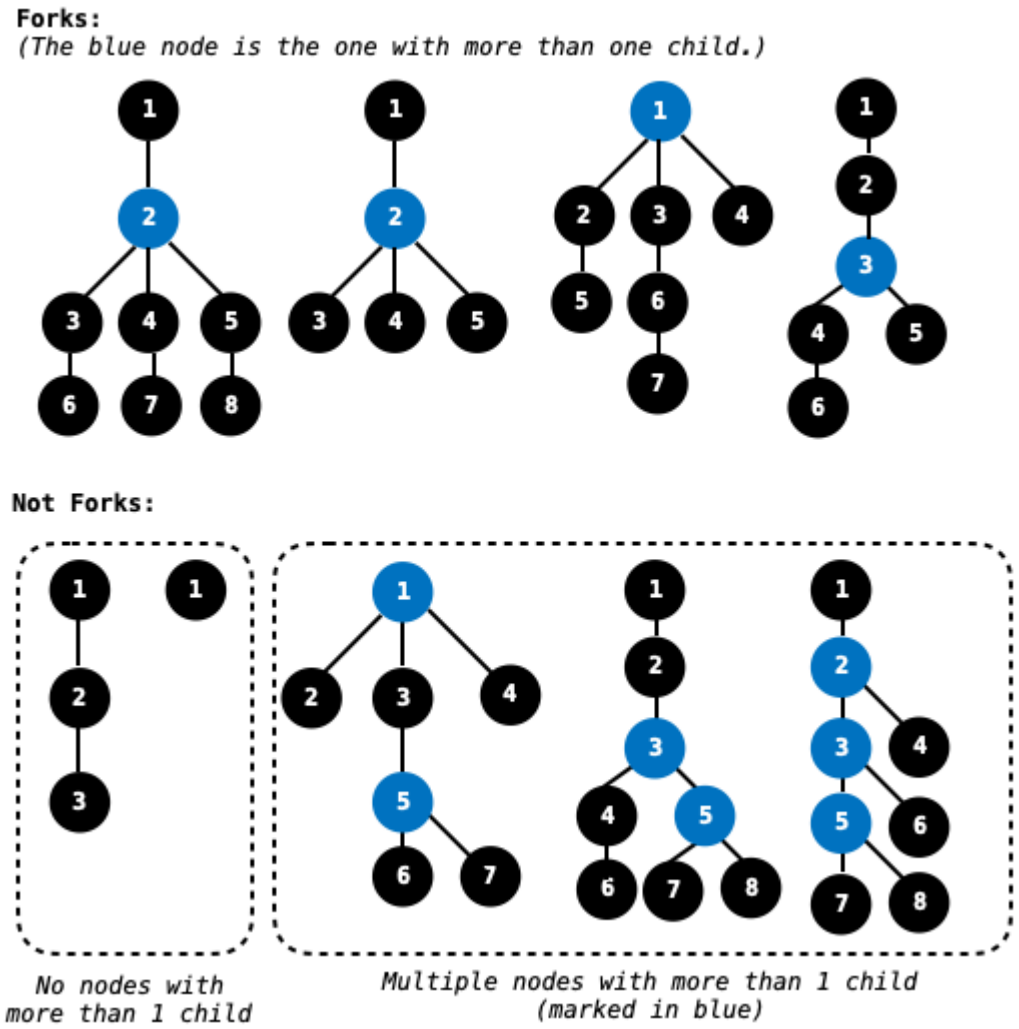
ii. (1.0 pt) Which of the following could fill in blank (b)?

- ◯ `t`
- ◯ `label(t)`
- ◯ `[t]`
- ◯ `[label(t)]`
- ◯ `[0]`
- ◯ `[label(b) for b in branches(t)]`

iii. (2.0 pt) Fill in blank (c). You may not use the word `default`.

(b) (8.0 points)

Definition. A *fork* is a tree in which **exactly one node** has more than one child.



Implement `is_fork` and its helper function `slide`. The `is_fork` function takes a *tree* `t` and returns `True` if `t` is a fork and `False` otherwise.

You may call `tree`, `label`, `branches`, `is_leaf`, `is_tree`, `max_path`, `is_fork`, and `slide`.

```
def is_fork(t):
    """Return whether tree t is a fork.

    >>> is_fork(tree(1, [tree(2, [tree(3), tree(4), tree(5)])]))
    True
    >>> is_fork(tree(1, [tree(2, [tree(3)]), tree(4)]))
    True
    >>> is_fork(tree(1, [tree(2), tree(3), tree(4)]))
    True
    >>> is_fork(tree(1, [tree(2, [tree(3, [tree(5)]), tree(4, [tree(6)])])]))
    True
    >>> is_fork(tree(1))
    False
    >>> is_fork(tree(1, [tree(2, [tree(3)])]))
    False
```

```
    >>> is_fork(tree(1, [tree(2, [tree(3)]), tree(4, [tree(5), tree(6)])]))
    False
    >>> is_fork(tree(1, [tree(2, [tree(3, [tree(5, [tree(7), tree(8)]), tree(6)]), tree(4)])]))
    False
    """

    neck = slide(t)

    if is_leaf(neck):

        _________
           (a)

    return _________([_________ for b in branches(_________)])
              (b)        (c)                         (d)

def slide(t):
    """Return the deepest node within tree t whose ancestors all have exactly one child.

    Definition: The ancestors of a node include its parent and the parents of all its ancestors.

    >>> deepest = slide(tree(1, [tree(2, [tree(3)])]))
    >>> label(deepest)
    3
    >>> label(slide(tree(1, [tree(2, [tree(3), tree(4)])])))
    2
    """

    while _________:
             (e)

        t = _________
               (f)

    return t
```

i. (1.0 pt) Fill in blank (a).

ii. (1.0 pt) Which of the following could fill in blank (b)?

- ◯ `all`
- ◯ `any`
- ◯ `list`
- ◯ `tree`
- ◯ `branches`
- ◯ `label`

iii. (2.0 pt) Fill in blank (c).

iv. (1.0 pt) Which of the following could fill in blank (d)?

- ◯ `t`
- ◯ `tree`
- ◯ `neck`
- ◯ `label(t)`
- ◯ `label(tree)`
- ◯ `label(neck)`

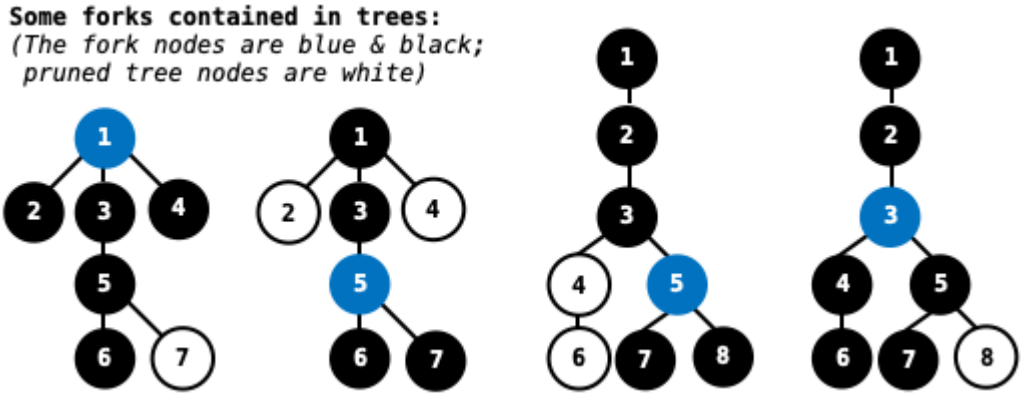
v. (1.0 pt) Which of the following could fill in blank (e)?

- ◯ `len(branches(t)) == 1`
- ◯ `len(branches(t)) > 1`
- ◯ `len(branches(t)) != 1`
- ◯ `branches(t) == 1`
- ◯ `branches(t) > 1`
- ◯ `branches(t) != 1`

vi. (2.0 pt) Fill in blank (f).

(c) (8.0 points)

Definition. A tree `t` *contains* a fork `u` if `u` is the result of pruning zero or more nodes from `t`.



Implement `max_fork`, which takes a tree `t` whose labels are all positive integers. It returns the largest sum of the labels of a fork that is contained in `t`. If `t` does not contain any forks, then `max_fork` returns 0.

You may call `tree`, `label`, `branches`, `is_leaf`, `is_tree`, `max_path`, `is_fork`, `slide`, and `max_fork`.

```
def max_fork(t):
    """Return the largest sum of the labels in any fork contained in tree t,
    which has positive numbers as labels. If t contains no forks, return 0.


    >>> a = tree(1, [tree(2), tree(3), tree(4, [tree(5)])])
    >>> max_fork(a) # 1 + 2 + 3 + 4 + 5
    15
    >>> b = tree(6, [a, a, a])
    >>> max_fork(b)  #          6 + (1 + 4 + 5) + (1 + 4 + 5) + (1 + 4 + 5)
    36
    >>> c = tree(7, [tree(8), b, tree(9)])
    >>> max_fork(c)  #     7 + (6 + (1 + 4 + 5) + (1 + 4 + 5) + (1 + 4 + 5))
    43
    >>> d = tree(9, [c])
    >>> max_fork(d)  # 9 + 7 + (6 + (1 + 4 + 5) + (1 + 4 + 5) + (1 + 4 + 5))
    52
    >>> max_fork(tree(1, [tree(2, [tree(3)])])) # No forks here!
    0
    """

    n = len(branches(t))

    if n == 0:

        return 0

    elif n == 1:

        below = _________
                  (a)

        if _________:
              (b)
```

```
            return _________ + below
                      (c)

        else:

            return 0

    else:

        here = sum([_________ for b in branches(t)])
                       (d)

        there = max([_________ for b in branches(t)])
                        (e)

        return label(t) + max(here, there)
```

i. **(2.0 pt)** Fill in blank (a).

ii. **(1.0 pt)** Which of the these could fill in blank (b)?

- ◯ `below > 0`
- ◯ `label(t) > 0`
- ◯ `max_fork(t) > 0`
- ◯ `True`
- ◯ `max_fork(t) > max_path(t)`
- ◯ `is_fork(t)`

iii. **(1.0 pt)** Which of these could fill in blank (c)?

- ◯ `1`
- ◯ `label(t)`
- ◯ `max_path(t)`
- ◯ `label(slide(t))`
- ◯ `max([label(b) for b in branches(t)])`
- ◯ `label(branches(t)[0])`

iv. **(2.0 pt)** Fill in blank (d).

v. (2.0 pt) Fill in blank (e).

No more questions.

CS 61A Fall 2021

Structure and Interpretation of Computer Programs

MIDTERM 1

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

1. (8.0 points) What Would Python Display?

(a) (4.0 points)

Assume the following code has been executed.

```
def os(ki):
    t = -1

i = 5
t = 7
while i > 3:
    t = i - 4 and i + 4
    os(i - 2)
    i, j = i - 1, i * 2
    s = i
```

i. (1.0 pt) What value is bound to `i` in the global frame?

ii. (1.0 pt) What value is bound to `j` in the global frame?

iii. (1.0 pt) What value is bound to `s` in the global frame?

iv. (1.0 pt) What value is bound to `t` in the global frame?

(b) (4.0 points)

The function `tik` takes an argument `tok` and returns a function `insta` that takes an argument `gram`. The `insta` function prints `tok` and `gram` on the same line separated by a space and has no return statement. Its implementation has been omitted intentionally.

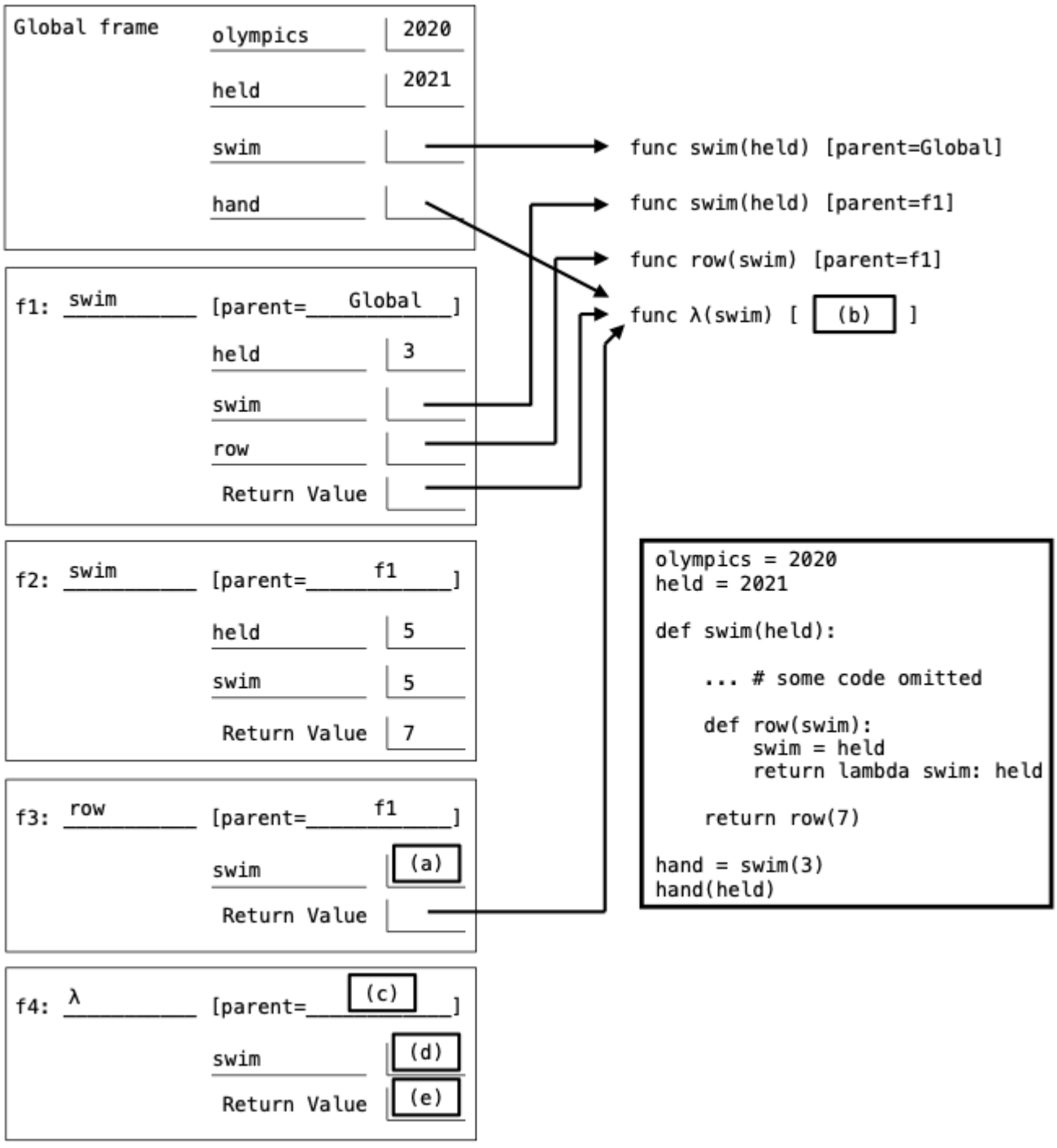
```
def tik(tok):
    """Returns a function that takes gram and prints tok and gram on a line.

    >>> tik(5)(6)
    5 6
    """
    def insta(gram):
        ... # The implementation of this function has been omitted.
    return insta
```

i. (4.0 pt) What would the interactive Python interpreter display upon evaluating the expression:

```
tik(tik(5)(print(6)))(print(7))
```

2. (5.0 points) 31 Cal Olympians

The environment diagram below was generated by code that is only partially provided to the right of the diagram. All of the relevant code needed to fill in the blanks in the environment diagram is shown. Line numbers have been omitted intentionally.



(a) (1.0 pt) Which of these could fill in blank (a)?

- ◯ `3`
- ◯ `5`
- ◯ `7`
- ◯ `2021`

(b) (1.0 pt) Which of these could fill in blank (b)? (The parent of the function returned in `f3`.)

- ◯ `parent=Global`
- ◯ `parent=f1`
- ◯ `parent=f2`
- ◯ `parent=f3`

(c) (1.0 pt) Which of these could fill in blank (c)?

- ◯ `parent=Global`
- ◯ `parent=f1`
- ◯ `parent=f2`
- ◯ `parent=f3`

(d) (1.0 pt) Which of these could fill in blank (d)?

- ◯ `3`
- ◯ `5`
- ◯ `7`
- ◯ `2021`

(e) (1.0 pt) Which of these could fill in blank (e)?

- ◯ `3`
- ◯ `5`
- ◯ `7`
- ◯ `2021`

3. (27.0 points) All Hail the Stone

Definition: A *hailstone sequence* begins with a positive integer `n`. If `n` is even, divide it by 2. If `n` is odd, triple it and add 1. Repeat until 1 is reached. For example, the hailstone sequence starting at 3 is 3, 10, 5, 16, 8, 4, 2, 1. Assume all hailstone sequences are finite and end with 1.

Assume the following code has been executed.

```
from operator import add, mul

def next_hail(k):
    """Return the next element in a hailstone sequence."""
    assert k > 1
    if k % 2 == 0:
        return k // 2
    else:
        return 3 * k + 1
```

(a) (8.0 points)

Implement `hail_min`, which takes a positive integer `n` and a one-argument function `measure`. It returns the element of the hailstone sequence starting with `n` for which calling `measure` on the element returns the smallest value.

If more than one element of the sequence has the smallest `measure` value, return the earliest one.

```
def hail_min(n, measure):
    """Return the element k of the hailstone sequence starting with n for which
    measure(k) is smallest. In case of a tie, return the earliest element.

    >>> hail_min(5, lambda k: -k)          # Among 5, 16, 8, 4, 2, 1; 16 is largest
    16
    >>> hail_min(8, lambda k: -k)          # Among 8, 4, 2, 1; 8 is largest
    8
    >>> hail_min(3, lambda k: abs(k - 7))  # Among 3, 10, 5, 16, 8, 4, 2, 1; 8 is closest to 7
    8
    >>> hail_min(9, lambda k: abs(k - 7))  # Among 9, 28, 14, 7, 22, ...; 7 is closest to 7
    7
    >>> hail_min(8, lambda k: abs(k - 3))  # 4 and 2 are both close to 3, but 4 is earliest
    4
    """

    apple = _________
              (a)

    while n > 1:

        n = next_hail(n)

        if _________:
              (b)

            _________
              (c)

    return _________
              (d)
```

i. (1.0 pt) Fill in blank (a).

ii. (3.0 pt) Fill in blank (b).

iii. (1.0 pt) Fill in blank (c).

iv. (1.0 pt) Which of these could fill in blank (d)? Check all that apply.

- ☐ `n`
- ☐ `apple`
- ☐ `measure(n)`
- ☐ `measure(apple)`
- ☐ `min(n, apple)`
- ☐ `min(measure(n), measure(apple))`

v. (2.0 pt) What element of the hailstone sequence starting with `n` larger than 1 is returned by the expression:

```
hail_min(n, lambda k: 1 - k % 2)
```

- ◯ Always `n` (the first element)
- ◯ Always 1 (the last element)
- ◯ The largest odd element
- ◯ The largest even element
- ◯ The smallest odd element
- ◯ The smallest even element
- ◯ The first odd element
- ◯ The first even element

(b) (6.0 points)

Definition: An *accumulator function* is a function that takes two integers and returns an integer. It can serve as the second argument to `hail_tally` below.

```
def hail_tally(n, f):
    """Accumulate the elements of the hailstone sequence starting with n using
    accumulator function f.

    >>> hail_tally(3, add)   # 3 + 10 + 5 + 16 + 8 + 4 + 2 + 1 = 49
    49
    >>> hail_tally(10, max)  # Largest of 10, 5, 16, 8, 4, 2, 1
    16
    """
    total = 0
    while n > 1:
        total = f(total, n)
        n = next_hail(n)
    return f(total, 1)
```

Implement `sum_some`, which takes a one-argument function `select` and returns an *accumulator function* `f`. For positive integer `n`, the call `hail_tally(n, f)` returns the sum of the elements of the hailstone sequence starting with `n` for which calling `select` on the element returns a true value.

```
def sum_some(select):
    """Return an accumulator function that sums all k for which select(k) is a true value.

    >>> def below_ten(k):
    ...     return k < 10
    >>> sum_below_ten = sum_some(below_ten)
    >>> hail_tally(3, sum_below_ten)  # [3] + 10 + [5] + 16 + [8] + [4] + [2] + [1]
    23
    """
    def f(total, k):

        if _________:
              (a)

            return _________(total, k)
                       (b)

        return _________
                  (c)


    _________
       (d)
```

i. (2.0 pt) Fill in blank (a).

ii. (1.0 pt) Which of these could fill in blank (b)? Check all that apply.

- ☐ `add`
- ☐ `mul`
- ☐ `sum_some`
- ☐ `total`
- ☐ `f`
- ☐ `hail_tally`

iii. (1.0 pt) Which of these could fill in blank (c)?

- ◯ `k`
- ◯ `total`
- ◯ `total + k`
- ◯ `hail_tally(k, sum_some)`
- ◯ `hail_tally(total, sum_some)`
- ◯ `hail_tally(total + k, sum_some)`

iv. (2.0 pt) Fill in blank (d).

(c) (5.0 points)

Implement `hail_odd_sum`, a function that takes a positive integer `n` and returns the sum of all odd elements in the hailstone sequences starting with `n`.

Important: Your solution **must** include `sum_some`.

You may also call other functions defined previously in this question.

```
def hail_odd_sum(n):
    """Sum the odd elements of the hailstone sequence starting with n.

    >>> hail_odd_sum(3)   # [3], 10, [5], 16, 8, 4, [1]; 3 + 5 + 1 = 9
    9
    >>> hail_odd_sum(34)  # 34, [17], 52, 26, [13], 40, 20, 10, [5], ..., [1]
    36
    """
    return _________(_________, _________)
              (a)       (b)        (c)
```

i. (1.0 pt) Fill in blank (a).

ii. (1.0 pt) Fill in blank (b).

iii. (3.0 pt) Fill in blank (c).

(d) (8.0 points)

Definition: To call a function `f` *repeatedly* on a sequence of values `x`, `y`, `z` means to use `f` in a nested call expression in which each element of the sequence is passed in as a single argument in order: `f(x)(y)(z)`.

Implement `hail`, which takes an integer `n` greater than 1. It returns a function that returns `True` when called repeatedly on all of the remaining elements of the hailstone sequence starting with `n`. It returns `False` when called repeatedly on any sequence that differs from the hailstone sequence starting with `n`.

Hint: You may call `next_hail` (and other functions defined previously in this question).

```
def hail(n):
    """Return a function that returns True if called repeatedly on the remaining
    elements of the hailstone sequence starting with n and False otherwise.

    >>> hail(3)(10)(5)(16)(8)(4)(2)(1)
    True
    >>> hail(3)(4)                 # The next element should have been 10.
    False
    >>> hail(3)(10)(5)(16)(8)(1)   # The next element should have been 4.
    False
    """

    assert n > 1

    def check(k):

        if _________:
              (a)

            return True

        if _________:
              (b)

            return False

        return _________
                  (c)

    return _________
              (d)
```

i. (2.0 pt) Fill in blank (a).

ii. (2.0 pt) Fill in blank (b).

iii. (2.0 pt) Fill in blank (c).

iv. (2.0 pt) Fill in blank (d).

No more questions.

CS 61A Fall 2021

Structure and Interpretation of Computer Programs

MIDTERM 2

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

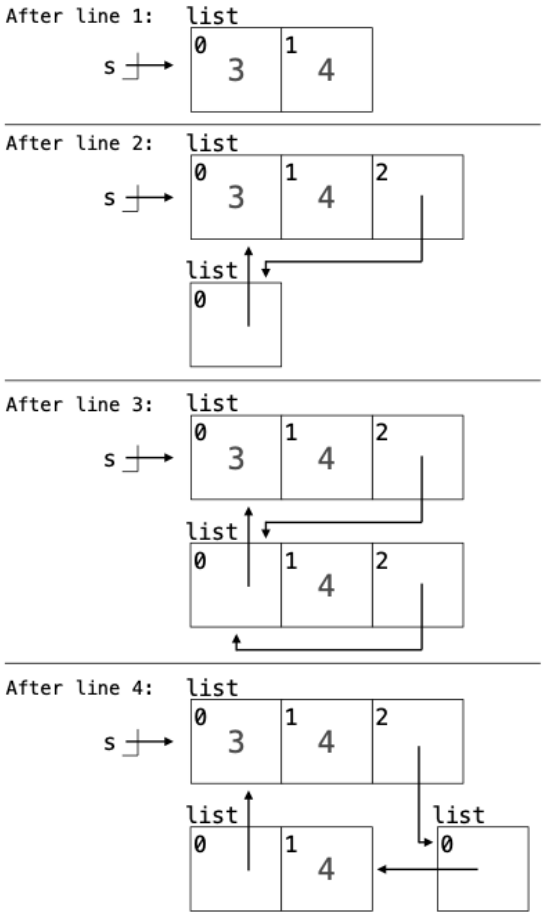
(a) What is your full name?

(b) What is your student ID number? A regex restricts inputs to numerical responses only.

1. (7.0 points) Hawkeye

The box-and-pointer diagram for the value of `s` is shown after each line of the program below. Fill in the blanks to match the diagrams.

```
s = [3, 4]

s.append(_________)
            (a)

_________.________(_________)
   (b)       (c)       (d)

s[2] = _________
           (e)
```



(a) (2.0 pt) Which of these could fill in blank (a)? Check all that apply.

- ☐ `s`
- ☐ `s[:]`
- ☐ `s[0]`
- ☐ `[s]`
- ☐ `[s[:]]`
- ☐ `[s[0]]`
- ☐ `list(s)`
- ☐ `list(s[:])`
- ☐ `list(s[0])`

(b) (1.0 pt) Fill in blank (b).

(c) (1.0 pt) Fill in blank (c).

(d) (1.0 pt) Which of these could fill in blank (d)? (Select one.)

- ◯ `s[1]`
- ◯ `s[1:]`
- ◯ `s[1:2]`
- ◯ `[s[1]]`
- ◯ `[s[1:]]`
- ◯ `[s[1:2]]`
- ◯ `[s[1], s]`
- ◯ `[s[1]] + s`

(e) (2.0 pt) Fill in blank (e).

2. (11.0 points) Doctor Change

(a) (6.0 points)

Implement `change`, a function that takes an integer `n` and a list of positive integers `coins`. It returns whether there is a subset of the values in `coins` that sums to `n`. As a side effect, `change` modifies `coins`.

```
def change(n, coins):
    """Return whether a subset of coins adds up to n.

    >>> change(10, [2, 7, 1, 8, 2])  # e.g., 2 + 8
    True
    >>> change(20, [2, 7, 1, 8, 2])  # e.g., 2 + 7 + 1 + 8 + 2
    True
    >>> change(6, [2, 7, 1, 8, 2])   # Impossible; only two 2's in coins
    False
    """
    if n == 0:
        return True
    elif _________:
            (a)
        return False
    coin = coins.pop()  # remove the end of coins and name it "coin"
    return change(n, _________) _________ change(_________, _________)
                         (b)        (c)              (d)        (e)
```

i. (2.0 pt) Fill in blank (a).

ii. (1.0 pt) Fill in blank (b).

iii. (1.0 pt) Fill in blank (c).

iv. (1.0 pt) Which of these could fill in blank (d)? (Select one.)

◯ `n - coin`

◯ `n - coins[0]`

◯ `n - coins[1]`

◯ `n - coins.pop()`

v. (1.0 pt) Fill in blank (e).

(b) (5.0 points)

Implement `amounts`, which takes a list of positive integers `coins`. It returns a sorted list of all unique non-negative integers `n` for which `change(n, coins)` returns `True`. You **may not** call `change`.

```
def amounts(coins):
    """List all unique n such that change(n, coins) returns True (in sorted order).

    >>> amounts([2, 5, 3])
    [0, 2, 3, 5, 7, 8, 10]
    >>> amounts([2, 7, 1, 8, 2])
    [0, 1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 13, 15, 16, 17, 18, 19, 20]
    """
    if not coins:
        return _________
                  (a)
    coin = coins[0]
    rest = amounts(_________)
                     (b)
    return sorted(_________ + [k + coin for _________])
                    (c)                        (d)
```

i. (1.0 pt) Fill in blank (a).

ii. (1.0 pt) Fill in blank (b).

iii. (1.0 pt) Which of these could fill in blank (c)? (Select one.)

- ◯ `rest`
- ◯ `rest[1:]`
- ◯ `amounts(rest)`
- ◯ `amounts(rest[1:])`
- ◯ `[k + coin for k in rest]`
- ◯ `[k + coin for k in rest[1:]]`

iv. (2.0 pt) Fill in blank (d).

3. (13.0 points) Shang-Chi

(a) (6.0 points)

Implement the `Valet` and `Garage` classes.

A `Valet` instance has two instance attributes: their total `tips` and the `garage` where they work (a `Garage` instance).

- The `park` method takes a string `car` and parks it in the `Garage` where they work.
- The `wash` method takes a string `car` that has been parked in their garage and a number `tip`. The `Valet` who washed the car and the `Valet` who **most recently** parked the car split the tip.

The `Garage` constructor takes a list of the `Valet`s who work in that `Garage`. Assume that `park` and `wash` are only invoked on a `Valet` that already has a `Garage`.

```
class Valet:
    """A valet is tipped after they wash a car,
    or after one of their parked cars is washed.

    >>> shaun = Valet()
    >>> katy = Valet()
    >>> g = Garage([shaun, katy])
    >>> shaun.park('Benz')
    >>> katy.park('BMW')
    >>> shaun.wash('Benz', 1)   # $1.0 to Shaun
    >>> katy.wash('Benz', 2)    # $1.0 to Katy, $1.0 to Shaun

    >>> shaun.park('Rolls')
    >>> katy.park('Rolls')
    >>> katy.wash('BMW', 2)     # $2.0 to Katy
    >>> shaun.wash('Rolls', 2)  # $1.0 to Shaun, $1.0 to Katy
    >>> [shaun.tips, katy.tips]
    [3.0, 4.0]
    """
    def __init__(self):
        self.tips = 0
        self.garage = None

    def park(self, car):
        _________
           (a)

    def wash(self, car, tip):
        self.tips += tip / 2
        _________ += tip / 2
           (b)

class Garage:
    """A garage holds cars parked by the valets who work there."""
    def __init__(self, valets):
        self.cars = {}
        for valet in valets:
            _________ = _________
               (c)         (d)
```

i. (2.0 pt) Fill in blank (a).

ii. (2.0 pt) Fill in blank (b).

iii. (1.0 pt) Fill in blank (c).

iv. (1.0 pt) Which of these could fill in blank (d)? (Select one.)

- ◯ `Garage`
- ◯ `valet`
- ◯ `valets`
- ◯ `garage`
- ◯ `self.valet`
- ◯ `self.valets`
- ◯ `self.garage`
- ◯ `self`

(b) (2.0 points)

Definition. An *infinite* iterator `t` is one for which `next(t)` can be called any number of times and always returns a value.

Implement `ring`, a generator function that takes a non-empty list `s`. It returns an infinite generator that repeatedly yields the values of `s` in the order they appear in `s`.

```
def ring(s):
    """Yield all values of non-empty s in order, repeatedly.

    >>> t = ring([2, 5, 3])
    >>> [next(t), next(t), next(t), next(t), next(t), next(t), next(t)]
    [2, 5, 3, 2, 5, 3, 2]
    """
    _________:
       (a)

        _________
           (b)
```

i. (1.0 pt) Which of these could fill in blank (a)? (Select one.)

◯ `while True`

◯ `while s`

◯ `for x in s`

◯ `for x in ring(s)`

ii. (1.0 pt) Fill in blank (b).

(c) (5.0 points)

Implement `fork`, a function that takes an infinite iterator `t`. It returns two infinite iterators that each iterate over the contents of `t`.

```
def fork(t):
    """Return two iterators with the same contents as infinite iterator t.

    >>> a, b = fork(ring([1, 2, 3]))
    >>> [next(a), [next(b), next(b), next(b)], next(a), [next(b), next(b), next(b)], next(a)]
    [1, [1, 2, 3], 2, [1, 2, 3], 3]
    """
    s = []
    def copy():
        i = 0
        while True:
            if _________:
                  (a)
               s.append(_________)
                           (b)
            yield _________
                     (c)
            i += 1
    return _________
              (d)
```

i. (1.0 pt) Fill in blank (a).

ii. (1.0 pt) Fill in blank (b).

iii. (1.0 pt) Which of these could fill in blank (c)? (Select one.)

- ◯ `from s`
- ◯ `from t`
- ◯ `s[0]`
- ◯ `s[i]`
- ◯ `next(t)`

iv. (2.0 pt) Fill in blank (d).

4. (10.0 points) Thanos

Hint: you may call built-in sequence functions: `sum`, `max`, `min`, `all`, `any`, `map`, `filter`, `zip`, and `reversed`.

(a) (4.0 points)

Implement `snap`, which takes a one-argument function `f`, a one-argument function `g`, and a sequence `s`. It returns a list of `(x, f(x))` pairs (two-element tuples) for all `x` in `s` for which `g(f(x))` is a true value. The implementation of `snap` only calls `f` once per element of `s`; never twice.

Important: For full credit, your implementation may only call `f` **once** on each element of `s`.

```
def snap(f, g, s):
    """Return a list of (x, f(x)) pairs for each x in s such that g(f(x)) is a true value.

    >>> snap(lambda x: x * x, lambda x: x < 10, range(5))
    [(0, 0), (1, 1), (2, 4), (3, 9)]
    >>> snap(lambda x: x * x, lambda x: x > 10, range(5))
    [(4, 16)]
    >>> snap(lambda x: x * x, lambda x: x and x - 9, range(5))
    [(1, 1), (2, 4), (4, 16)]
    """
    return [(x, _________) for _________ in _________ if _________]
                  (a)            (b)          (c)          (d)
```

i. (1.0 pt) Fill in blank (a).

ii. (1.0 pt) Which of these could fill in blank (b)? (Select one.)

- ◯ `x`
- ◯ `x, y`
- ◯ `y`
- ◯ `y, z`

iii. (1.0 pt) Fill in blank (c).

iv. (1.0 pt) Fill in blank (d).

(b) (4.0 points)

Implement `max_diff`, which takes a non-empty sequence `s` and a one-argument function `f`. It returns a pair of elements `(v, w)` in `s` for which `f(v) - f(w)` is largest. `v` and `w` may be the same or different elements of `s`. Also, describe the order of growth of the run time of `max_diff`.

```
def max_diff(s, f):
    """Return two elements (v, w) of s for which f(v) - f(w) is largest.

    >>> max_diff(range(-7, 4), lambda x: x * x)  # (-7 * -7) - (0 * 0) = 49
    (-7, 0)
    >>> max_diff(['what', 'a', 'great', 'film'], len)  # len('great') - len('a')
    ('great', 'a')
    """
    assert s, 's cannot be empty'
    v, w = None, None
    for x in s:
        for y in s:
            if v is None or _________:
                              (a)

                ---------
                   (b)
    return v, w
```

i. (1.0 pt) Fill in blank (a).

ii. (1.0 pt) Fill in blank (b).

iii. (2.0 pt) What is the order of growth of the time required to evaluate `max_diff(s, f)` for a sequence `s` of length n and a function `f` that requires constant time to evaluate.

- ◯ Exponential, $\Theta(b^n)$
- ◯ Quadratic, $\Theta(n^2)$
- ◯ Linear, $\Theta(n)$
- ◯ Logarithmic, $\Theta(\log_2 n)$
- ◯ Constant, $\Theta(1)$

(c) (2.0 points)

Implement `max_diff_fast`, which has the same signature and behavior as `max_diff`, but has a faster order of growth of its run time.

Important. You may **not** use a list comprehension in your solution.

```
def max_diff_fast(s, f):

    return _________ , _________
              (a)         (b)
```

i. (1.0 pt) Fill in blank (a), but do not use a comprehension.

ii. (1.0 pt) Fill in blank (b), but do not use a comprehension.

5. (9.0 points) Groot

Definition. A *twig* is a tree that is not a leaf but whose branches are all leaves.

The `Tree` and `Link` classes appear on your midterm 2 study guide. Assume they are defined.

(a) (4.0 points)

Implement `twig`, which takes a `Tree` instance t. It returns `True` if `t` is a twig and `False` otherwise.

```
def twig(t):
    """Return True if Tree t is a twig and False otherwise.

    >>> twig(Tree(1))
    False
    >>> twig(Tree(1, [Tree(2), Tree(3)]))
    True
    >>> twig(Tree(1, [Tree(2), Tree(3, [Tree(4)])]))
    False
    """
    return _________ and _________
              (a)           (b)
```

i. (2.0 pt) Which of these could fill in blank (a)? Check all that apply. *Hint*: The `bool` function returns `True` for a true value and `False` for a false value.

☐ `t`

☐ `bool(t)`

☐ `t.is_leaf()`

☐ `not t.is_leaf()`

☐ `t.branches`

☐ `bool(t.branches)`

☐ `len(t.branches)`

☐ `len(t.branches) > 0`

☐ `len(t.branches) >= 0`

ii. (2.0 pt) Fill in blank (b).

(b) (5.0 points)

Implement `twigs`, which takes a `Tree` instance `t`. It returns a linked list (either a `Link` instance or `Link.empty`) containing all of the labels of the twigs in `t`. Labels should be in the same order as they appear in `repr(t)`.

```
def twigs(t):
    """Return a linked list of the labels of the twigs in t.

    >>> t = Tree(1, [Tree(2), Tree(3, [Tree(4)]), Tree(5, [Tree(6, [Tree(7), Tree(8)])])])
    >>> print(twigs(t))
    <3 6>
    >>> print(twigs(Tree(0, [t, t, t])))
    <3 6 3 6 3 6>
    >>> twigs(Tree(0)) is Link.empty
    True
    """

    def add_twigs(t, rest):

        if twig(t):

            return _________
                      (a)

        for b in reversed(t.branches):

            rest = _________
                      (b)

        return rest

    return add_twigs(t, _________)
                           (c)
```

i. (2.0 pt) Fill in blank (a).

ii. (2.0 pt) Fill in blank (b).

iii. (1.0 pt) Fill in blank (c).

No more questions.

CS 61A Structure and Interpretation of Computer Programs

Fall 2022 MIDTERM 1

INSTRUCTIONS

You have 1 hour and 50 minutes to complete the exam.

- The exam is closed book, closed notes, closed computer, closed calculator, except one 8.5" × 11" page of your own creation and the provided midterm 1 study guide.
- Mark your answers on the exam itself in the spaces provided. We will not grade answers written on scratch paper or outside the designated answer spaces.
- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `pow`, `len`, `abs`, `bool`, `int`, `float`, `str`, `max`, and `min`.
- You **may not** use example functions defined on your study guide unless a problem clearly states you can.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.
- You may not use ; to place two statements on the same line.

For questions with **circular bubbles**, you should select exactly *one* choice.

◯ You must choose either this option

◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

☐ You could select this choice.

☐ You could select this one too!

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

1. (4.0 points) What Would Python Display?

Assume the following code has been executed.

```
bear = -1
oski = lambda print: print(bear)
bear = -2

s = "Knock"
```

For each expression below, write the output **displayed by the interactive Python interpreter** when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

(a) (1.0 pt) `(3 and 4) - 5`

- ◯ -2
- ◯ -1
- ◯ 1
- ◯ 2
- ◯ Error

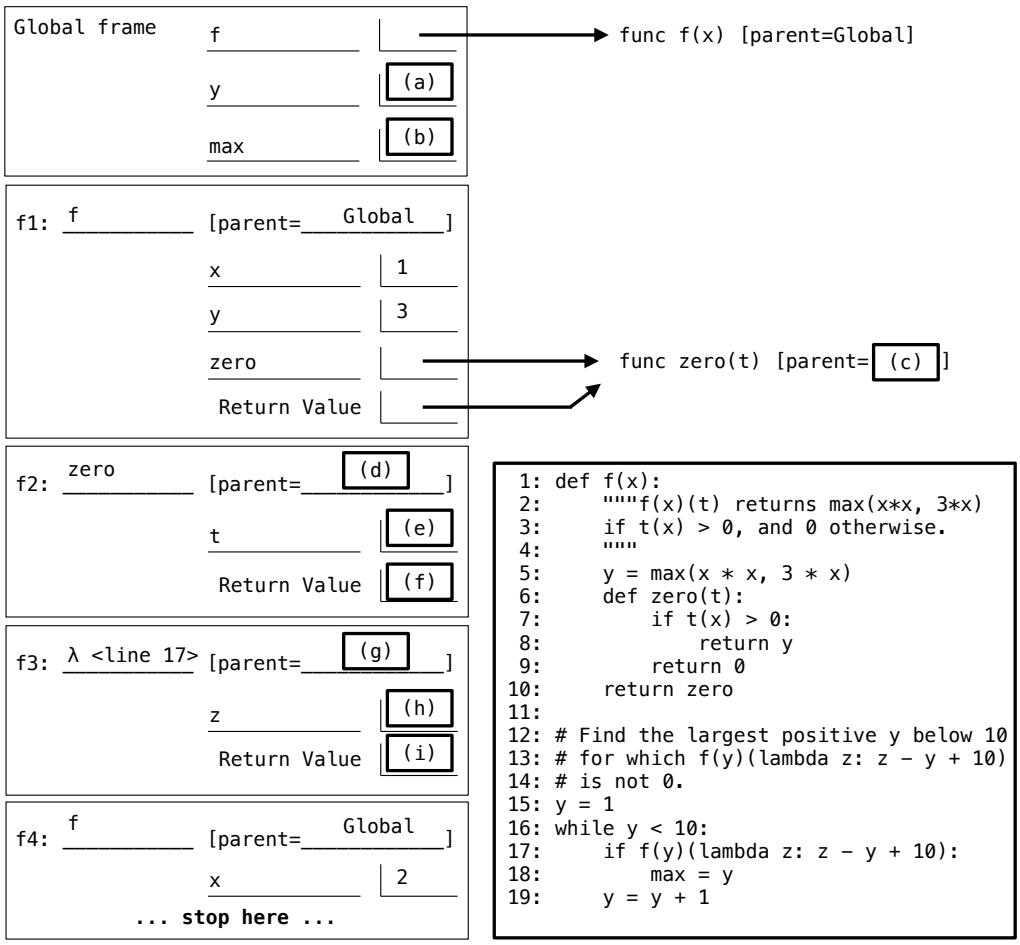
(b) (1.0 pt) `oski(abs)`

- ◯ -2
- ◯ -1
- ◯ 1
- ◯ 2
- ◯ Function
- ◯ Error

(c) (2.0 pt) `print(print(print(s, s) or print("Who's There?")), "Who?")`

2. (12.0 points) Don't Call Me, I'll Call You

When working on a large project, you get the error `'int' object is not callable` in the code below during the second call to `f`. Complete the environment diagram until the error occurs to identify the problem with the code. **If a blank contains an arrow to a function, write the function as it would appear in the diagram.**




```
 1: def f(x):
 2:     """f(x)(t) returns max(x*x, 3*x)
 3:     if t(x) > 0, and 0 otherwise.
 4:     """
 5:     y = max(x * x, 3 * x)
 6:     def zero(t):
 7:         if t(x) > 0:
 8:             return y
 9:         return 0
10:     return zero
11:
12: # Find the largest positive y below 10
13: # for which f(y)(lambda z: z - y + 10)
14: # is not 0.
15: y = 1
16: while y < 10:
17:     if f(y)(lambda z: z - y + 10):
18:         max = y
19:     y = y + 1
```

(a) (1.0 pt) Fill in blank (a).

(b) (1.0 pt) Fill in blank (b).

(c) (1.0 pt) Which of these could fill in blank (c)?

- ◯ `Global`
- ◯ `f`
- ◯ `f1`

(d) (1.0 pt) Which of these could fill in blank (d)?

- ◯ `Global`
- ◯ `f`
- ◯ `f1`

(e) (2.0 pt) Fill in blank (e).

(f) (1.0 pt) Fill in blank (f).

(g) (1.0 pt) Which of these could fill in blank (g)?

- ◯ `Global`
- ◯ `f`
- ◯ `zero`
- ◯ `f1`
- ◯ `f2`

(h) (1.0 pt) Fill in blank (h).

(i) (1.0 pt) Fill in blank (i).

(j) (2.0 pt) Explain in 10 words or less why the `'int' object is not callable` error occurred.

3. (10.0 points) It's Perfect

Definitions. A *perfect* number is a positive integer `n` whose *proper factors* (the factors of `n` below `n`) sum to exactly `n`. A number `n` is *abundant* if the sum of `n`'s proper factors is greater than `n` and *deficient* if that sum is less than `n`.

(a) (4.0 points)

Implement `classify`, a function that takes an integer `n` **greater than** 1. It returns the string `'deficient'`, `'perfect'`, or `'abundant'` that correctly describes `n`.

```
def classify(n):
    """Return whether n > 1 is 'deficient', 'perfect', or 'abundant'.

    >>> classify(6)   # Proper factors 1, 2 and 3 sum to exactly 6.
    'perfect'
    >>> classify(24)  # Proper factors 1, 2, 3, 4, 6, 8, and 12 sum to 36.
    'abundant'
    >>> classify(23)  # Proper factor 1 sums to 1.
    'deficient'
    """
    total, k = 0, 1
    while __<BLANK 1>__:
        if __<BLANK 2>__:
            __<BLANK 3>__
        k = k + 1
    if total == n:
        return 'perfect'
    elif __<BLANK 4>__:
        return 'deficient'
    else:
        return 'abundant'
```

i. (1.0 pt) BLANK 1

- ◯ `k < n`
- ◯ `k <= n`
- ◯ `k < total`
- ◯ `k <= total`

ii. (1.0 pt) BLANK 2

iii. (1.0 pt) BLANK 3

iv. (1.0 pt) BLANK 4

(b) (6.0 points)

Implement `nearest_perfect`, which takes an integer `n` above 5. It returns the nearest perfect number to `n`. If two perfect numbers are equally close to `n`, return the larger one. A number `a` is nearer to `n` than another number `b` if `abs(a - n) < abs(b - n)`. Assume `classify` is implemented correctly.

```
def nearest_perfect(n):
    """Return the nearest perfect number to n. In a tie, return the larger one.

    >>> nearest_perfect(8)   # 6 is perfect and 2 away from 8.
    6
    >>> nearest_perfect(20)  # 28 is perfect and 8 away from 20.
    28
    >>> nearest_perfect(17)  # Both 6 and 28 are 11 away from 17.
    28
    >>> nearest_perfect(6)   # 6 is perfect and 0 away from 6.
    6
    """
    k = 0
    while True:
        if __<BLANK 5>__:
            __<BLANK 6>__
        if __<BLANK 7>__:
            k = -k
        else:
            __<BLANK 8>__
```

i. (2.0 pt) BLANK 5

ii. (1.0 pt) BLANK 6

- ◯ `n = n + k`
- ◯ `n = n + 1`
- ◯ `n = n - k`
- ◯ `n = n - 1`
- ◯ `return n + k`
- ◯ `return n`
- ◯ `return k`

iii. (1.0 pt) BLANK 7

iv. (2.0 pt) BLANK 8

4. (4.0 points) Super Powers

Definition. A *function power*, written $f^n(x)$, describes repeated application of a function f to x. $f^2(x) = f(f(x))$, $f^5(x) = f(f(f(f(x))))$, and so on.

(a) (2.0 pt) Choose **all** correct implementations of `funsquare`, a function that takes a one-argument function `f`. It returns a one-argument function `f2` such that `f2(x)` has the same behavior as `f(f(x))` for all `x`.

```
>>> triple = lambda x: 3 * x
>>> funsquare(triple)(5)  # Equivalent to triple(triple(5))
45
```

```
A:  def funsquare(f):
        return f(f)
```

```
B:  def funsquare(f):
        return lambda: f(f)
```

```
C:  def funsquare(f, x):
        def g(x):
            return f(f(x))
        return g
```

```
D:  def funsquare(f):
        return lambda x: f(f(x))
```

```
E:  def funsquare(f, x):
        return f(f(x))
```

```
F:  def funsquare(f):
        def g(x):
            return f(f(x))
        return g
```

☐ A

☐ B

☐ C

☐ D

☐ E

☐ F

(b) (1.0 pt) Fill in the blank to assign `quadruple` to a function that takes a number x and returns $4x$.

```
>>> quadruple = funsquare(__<BLANK 1>__)
>>> quadruple(3)  # 4 * 3
12
```

BLANK 1

(c) (1.0 pt) Fill in the blank to assign `funfourth` to a function that takes a one-argument function $f(x)$ and returns a one-argument function $f^4(x)$. **You may not use a lambda expression.**

```
>>> funfourth = __<BLANK 2>__
>>> funfourth(triple)(10)  # 10 * 3 * 3 * 3 * 3 (triple is defined near the top of the page)
810
```

BLANK 2

5. (0.0 points) A+ Question 1: Busy Beaver

This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

To receive an A+ in CS 61A, you need at least 300 points in the course overall and must solve 2 A+ questions. To receive credit for this A+ question, you must answer all parts of it entirely correctly.

Introduced by Tibor Radó in 1962, the *Busy Beaver Game* aims to find the terminating program of a given length that produces the most possible output. Here, we'll explore one approach to generating a lot of output.

Implement `b`, which takes a zero-argument function `f` that returns `None`. The `b` function calls its argument `f` 64 times and returns `None`.

- You may not use any numbers or strings or any expressions that evaluate to numbers or strings.
- You may not import any functions, but may use built-in functions that do not need to be imported.
- You may not use lists, tuples, or dictionaries.
- Your answer for blank 2 must be no longer than 50 characters, ignoring whitespace.

```
def b(f):
    """Evaluate f() 64 times.

    >>> a = lambda: print('A', end='')
    >>> b(a)
    AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
    """
    (lambda g: g(g(g(g(g(g(g(__<BLANK 1>__)))))))(__<BLANK 2>__)__<BLANK 3>__
```

(a) (0.0 pt) `BLANK 1`

◯ `f`

◯ `g`

◯ `f()`

◯ `g()`

(b) (0.0 pt) `BLANK 2` (**Your answer must not be longer than 50 characters, ignoring whitespace.**)

(c) (0.0 pt) `BLANK 3`

◯ `()`

◯ `(f)`

◯ `(g)`

CS 61A Fall 2022

Structure and Interpretation of Computer Programs

MIDTERM 2

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

1. (7.0 points) What would Python Python?

Assume the code below has been executed.

```
class Snake:
    legs = 0
    def __init__(self, name):
        self.name = name
    def __str__(self):
        return self.name
    def run(self, s):
        print("Snakes don't run")
        return self.crawl()
    def crawl(self):
        print(f"{self} crawled")
    def eat(self, s):
        self.run(s)
        print("Nom nom")

class Python(Snake):
    def run(self, s):
        print(eval(s))  # eval(s) evaluates string s as a Python expression

snek = Snake("atari")
solidsnake = Snake("David")
solidsnake.legs = 2
solidsnake.run = lambda s: print("He ran")
python = Python("pypy")
```

For each expression below, write the output **displayed by the interactive Python interpreter** when the expression is evaluated.

- If an output has multiple lines, write each line separately.
- If an error occurs, write "Error", but include all output displayed before the error.
- If evaluation would run forever, write "Forever".
- To display a function value, write "Function".

Each expression below should be evaluated independently of previously-evaluated expressions.

(a) (1.0 pt) `[snek.legs, Snake.legs]`

(b) (2.0 pt) `solidsnake.eat("python")`

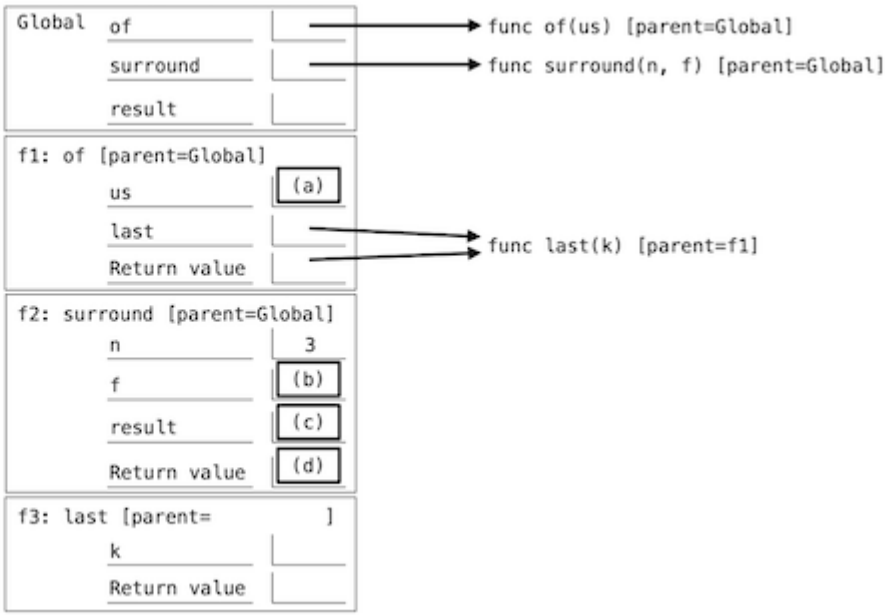
(c) (2.0 pt) `python.eat("snek")`

(d) (2.0 pt) `Snake.run(python, python)`

2. (6.0 points) Environmental Disaster

Your friend asks you to help debug their program below, which doesn't print what they expect. Complete the environment diagram, then answer questions about it. The diagram itself will not be scored; only the questions based on it.

```
def of(us):
    def last(k):
        "The last k items of us"
        while k > 0:
            result.append(us.pop())
            k = k - 1
        return result
    return last

def surround(n, f):
    "n is the first and last item of f(2)"
    result = [n]
    result = f(2)
    result[0] = [n]
    return result.append(n)

result = [1]
surround(3, of([4, 5, 6]))
print(result)
```



(a) (1.0 pt) *Blank (a)*: What is the name `us` bound to in f1?

- ◯ A list `[4, 5, 6]`
- ◯ A list `[4, 5]`
- ◯ A list `[4]`

(b) (1.0 pt) *Blank (b)*: What is the name `f` bound to in frame `f2`?

- ◯ `func of(us) [parent=Global]`
- ◯ `func last(k) [parent=f1]`
- ◯ None of the above

(c) (1.0 pt) *Blank (c)*: What is the name `result` bound to in frame `f2`?

- ◯ The same list bound to the name `result` in the Global frame
- ◯ A list that is **not** bound to any name in the Global frame

(d) (1.0 pt) *Blank (d)*: What is the return value of frame `f2`?

- ◯ The same list bound to the name `result` in `f2`
- ◯ A list that is one element longer than the one bound to the name `result` in `f2`
- ◯ None of the above

(e) (2.0 pt) What is displayed by the last line, `print(result)`?

3. (8.0 points) Hog Revisited

In Project 1, we created a simulator for the game of Hog without using lists or iterators. Let's revise the implementation, but instead of representing dice as zero-argument functions, **we'll represent dice as iterators over outcomes**.

(a) (3.0 points)

Implement `make_test_dice`, which takes a list of `outcomes`. It returns an infinite iterator that repeatedly cycles through the outcomes.

```
def make_test_dice(outcomes):
    """Return an infinite iterator that cycles through the elements of outcomes.

    >>> dice = make_test_dice([1, 2, 3])
    >>> next(dice)
    1
    >>> next(dice)
    2
    >>> next(dice)
    3
    >>> next(dice)
    1
    >>> next(dice)
    2
    >>> next(dice)
    3
    >>> next(dice)
    1
    >>> next(dice)
    2
    """
    outcomes = list(outcomes)
    while True:
        for i in ___(a)___:
            ___(b)___
```

i. (1.0 pt) Which of these could fill in blank (a)? Select **all** that apply.

☐ `outcomes`

☐ `outcomes + make_test_dice(outcomes)`

☐ `outcomes.extend(make_test_dice(outcomes))`

☐ `outcomes[0] + make_test_dice(outcomes[1:])`

☐ `make_test_dice(outcomes) + make_test_dice(outcomes)`

☐ `make_test_dice(outcomes[0]) + make_test_dice(outcomes[1:])`

ii. (2.0 pt) Fill in blank (b).

(b) (5.0 points)

Implement `roll_dice`, which takes two arguments: a positive integer called `num_rolls` giving the number of dice to roll and a `dice` iterator that provides outcomes. It returns the number of points scored by rolling the `dice` `num_rolls` times in a turn. The points scored are either the sum of the dice outcomes or 1 (Sow Sad).

- Sow Sad. If any of the dice outcomes is a 1, the current player's score for the turn is 1.

```
def roll_dice(num_rolls, dice):
    """Return the number of points scored by rolling dice num_rolls times.

    >>> dice = make_test_dice([6, 6, 6, 6, 6, 1])
    >>> roll_dice(5, dice) # 6, 6, 6, 6, 6
    30
    >>> roll_dice(5, dice) # 1, 6, 6, 6, 6
    1
    >>> roll_dice(2, dice) # 6, 1
    1
    >>> roll_dice(10,  make_test_dice([2, 4, 3])) # 2, 4, 3, 2, 4, 3, 2, 4, 3, 2
    29
    """
    rolls = ___(a)___
    if ___(b)___:
        return 1
    ___(c)___
```

i. (2.0 pt) Fill in blank (a).

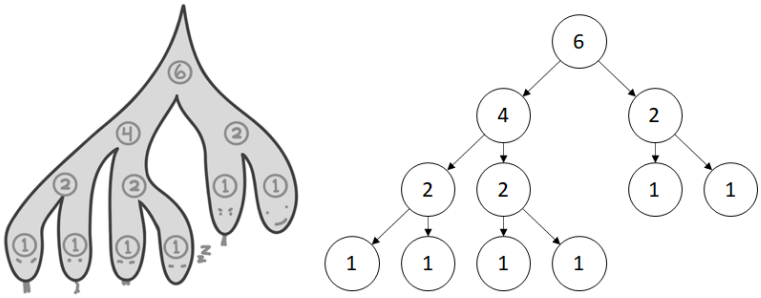
ii. (1.0 pt) Fill in blank (b).

iii. (2.0 pt) Fill in blank (c).

4. (9.0 points) The Lambdanean Hydra

Heracles is fighting the Lambdanean Hydra, a many-headed monster, where two heads regrow when one head gets chopped off. A hydra is represented by a tree, each head is represented by a leaf labeled 1, and each non-leaf node has two branches and is labeled with the number of leaves among its descendants. Here's a six-headed hydra named `lerna`:

```
lerna=Tree(6,[Tree(4,[Tree(2,[Tree(1),Tree(1)]),Tree(2,[Tree(1),Tree(1)])]),Tree(2,[Tree(1),Tree(1)])])
```



(a) (4.0 points)

Implement `is_hydra`, which takes in as input a Tree instance `t` and determines if `t` represents a valid hydra.

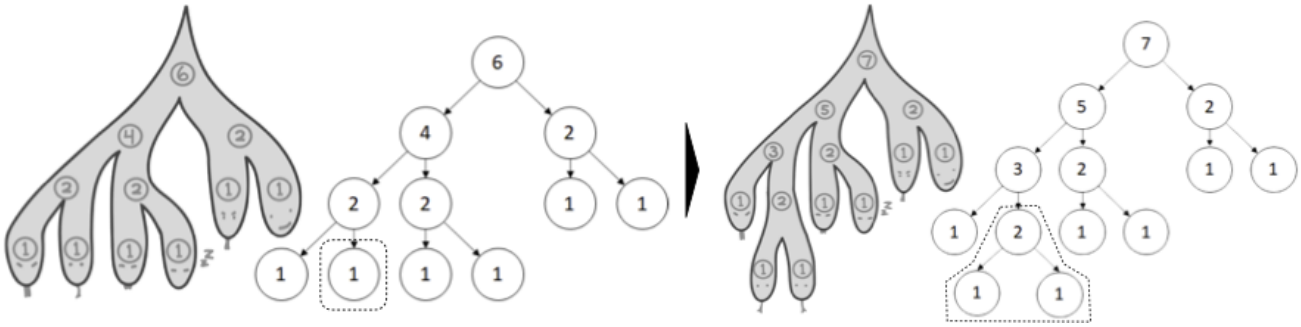
```
def is_hydra(t):
    """Return True if Tree instance t properly represents a hydra and False otherwise.

    >>> is_hydra(Tree(1))
    True
    >>> is_hydra(Tree(2, [Tree(1), Tree(1)]))
    True
    >>> is_hydra(Tree(3, [Tree(1), Tree(2)]))  # Wrong leaf label
    False
    >>> is_hydra(Tree(3, [Tree(1), Tree(1)]))  # Wrong root label
    False
    >>> is_hydra(Tree(3, [Tree(3, [Tree(1), Tree(1)]), Tree(1)]))  # Wrong node label below root
    False
    >>>is_hydra(lerna)  # lerna the six-headed hydra is defined above
    True
    """
    if t.is_leaf():
        return t.label == 1
    if len(t.branches) != 2:
        return False
    return (t.label == ___(a)___) and ___(b)___
```

i. (2.0 pt) Fill in blank (a).

ii. (2.0 pt) Fill in blank (b).

(b) (5.0 points)

If Heracles cuts off the second head from the left, two new heads appear there, and all ancestor labels are updated.



Implement `chop_head(hydra, n)`, which takes as input a Tree instance `hydra` representing a hydra and a positive integer `n`. It mutates `hydra` by chopping off the `n`th head from the left (adding two new adjacent heads in its place).

```
def chop_head(hydra, n):
    """
    >>> lerna = Tree(1)
    >>> chop_head(lerna, 1)  # Note that n is 1-indexed
    >>> chop_head(lerna, 1)
    >>> chop_head(lerna, 3)
    >>> chop_head(lerna, 1)
    >>> chop_head(lerna, 3)
    >>> lerna                # Now lerna is a six-headed hydra (above left)
    Tree(6,[Tree(4,[Tree(2,[Tree(1),Tree(1)]),Tree(2,[Tree(1),Tree(1)])]),
            Tree(2,[Tree(1),Tree(1)])])
    >>> chop_head(lerna, 2)
    >>> lerna                # The mutated lerna now has seven heads (above right)
    Tree(7,[Tree(5,[Tree(3,[Tree(1),Tree(2,[Tree(1),Tree(1)])]),Tree(2,[Tree(1),Tree(1)])]),
            Tree(2,[Tree(1),Tree(1)])])
    """
    assert is_hydra(hydra)
    assert n > 0 and n <= hydra.label
    if hydra.is_leaf():
        ___(a)___  # This blank may be filled with multiple lines of code
        return
    ___(b)___
    left, right = hydra.branches
    if ___(c)___:
        chop_head(right, ___(d)___)
    else:
        chop_head(left, n)
```

i. (2.0 pt) Select **all** options that could fit in blank (a), which may have multiple lines.

```
Option 1:  hydra = Tree(2,[Tree(1), Tree(1)])

Option 2:  for _ in range(2):
               hydra.branches.append(Tree(1))
               hydra.label = 2

Option 3:  hydra.label = Tree(2)
           hydra.branches = [Tree(1),Tree(1)]

Option 4:  hydra.label = 2
           hydra.branches = [Tree(1), Tree(1)]

Option 5:  hydra.label = 2
           hydra.branches = [Tree(1)] * 2
```

☐ Option 1

☐ Option 2

☐ Option 3

☐ Option 4

☐ Option 5

ii. (1.0 pt) Fill in blank (b).

iii. (1.0 pt) Fill in blank (c).

iv. (1.0 pt) Fill in blank (d).

5. (7.0 points) Aim for 100

The function `count_subsets` takes as input a list of positive integers `s`. It returns the number of lists that sum to 100 and contain a subset of the elements of `s` in order.

```
def count_subsets(s):
    """
    >>> count_subsets([25, 50, 75, 100, 125, 150])  # [25, 75], [100]
    2
    >>> count_subsets([25, 50, 25, 75])  # [25, 75] (first 25), [25, 75] (second 25), [25, 50, 25]
    3
    >>> count_subsets(list(range(1,10000)))
    444793
    """
```

(a) (5.0 points)

Complete the following implementation of `count_subsets`.

```
def count_subsets(s):
    def helper(sum_so_far, index):
        if ___(a)___:
            if ___(b)___:
                return 1
            return 0
        return ___(c)___ + ___(d)___
    return helper(0,0)
```

i. (1.0 pt) Fill in blank (a).

- ◯ `index == s`
- ◯ `index == len(s)`
- ◯ `sum_so_far == 100`
- ◯ `sum_so_far != 100`

ii. (1.0 pt) Fill in blank (b).

- ◯ `index == s`
- ◯ `index == len(s)`
- ◯ `sum_so_far == 100`
- ◯ `sum_so_far != 100`

iii. (1.0 pt) Fill in blank (c).

- ◯ `count_subsets(s[index:])`
- ◯ `count_subsets(s[1:])`
- ◯ `helper(sum_so_far, index)`
- ◯ `helper(sum_so_far, index + 1)`

iv. (2.0 pt) Fill in blank (d).

(b) (2.0 points)

i. (1.0 pt) What is the order of growth of the time required to evaluate `count_subsets`, in terms of the length of `s`?

◯ Exponential

◯ Quadratic

◯ Linear

◯ Logarithmic

◯ Constant

ii. (1.0 pt) We decide to rewrite `count_subsets`, following a different approach:

```
def count_subsets(s):
    values = [1]+[0]*100
    for i in s:
        for j in reversed(range(100-i+1)):
            values[j+i]+=values[j]
    return values[100]
```

What is the order of growth of the time required to evaluate the new version of `count_subsets`, in terms of the length of `s`?

◯ Exponential

◯ Quadratic

◯ Linear

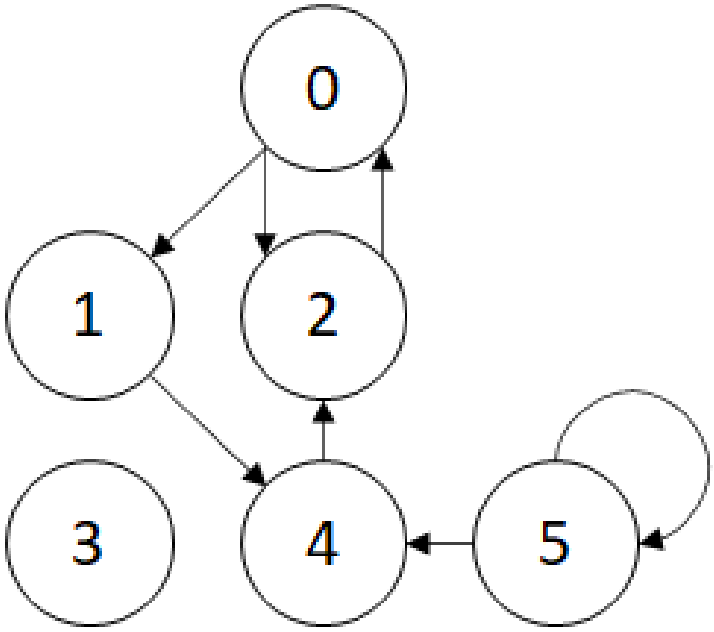
◯ Logarithmic

◯ Constant

6. (8.0 points) Point A to Point B

A *graph* is a structure consisting of two parts:

- A set of n nodes, which are labeled from 0 to n-1.
- A set of directed edges, which go from one node (the source node) to another node (the destination).



The above is an example of a graph with 6 nodes, with edges represented as arrows. Note that a graph can have:

- Nodes that have no edges to/from them (Node 3)
- Two edges between the same two nodes, with source and destination reversed (The edges between Nodes 0 and 2)
- An edge from a node to itself (Node 5)

However, a graph cannot have two distinct edges from one node to another. For example, we cannot have a graph where two edges go from Node 0 to Node 1.

We say that a Node j is a **destination** of Node i if there is a directed edge from Node i to Node j. Thus, in the above example, Node 0 has two destinations (Nodes 1 and 2), and Node 1 has one destination (Node 4). Note that Node 0 is not considered a destination of Node 1.

We say that there is a **path** from Node i to Node j if either `i==j`, or we can go from Node i to Node j by following a sequence of edges in the graph. For example, we can get from Node 1 to Node 0 by following the edges `1->4`, then `4->2`, then `2->0`.

Graphs are often used to represent transportation maps, where nodes represent individual locations, and edges represent a path you can take to go from one node to another.

A `Graph` instance is constructed from a number of nodes and begins with no edges. Edges are added, and then the `destinations` method lists destinations for a node and the `has_path` method determines if a path exists between two nodes.

```
>>> g = Graph(6)       # A graph with 6 nodes and no edges.
>>> g.add_edge(0, 2)   # Add an edge from node 0 (source) to node 2 (destination)
>>> g.add_edge(2, 0)   # Add an edge from node 2 (source) to node 0 (destination)
>>> g.add_edge(0, 1)
>>> g.add_edge(1, 4)
>>> g.add_edge(4, 2)
>>> g.add_edge(5, 4)
>>> g.add_edge(5, 5)
```

```
>>> g.destinations(0)
[1, 2]
>>> g.destinations(5)
[4, 5]
>>> g.destinations(3)
[]
>>> g.has_path(0,2)  # 0 -> 2
True
>>> g.has_path(5,1)  # 5 -> 4 -> 2 -> 0 -> 1
True
>>> g.has_path(3,3)  # 3
True
>>> g.has_path(1,5)  # No path exists
False
```

(a) (2.0 points)

Implement the `destinations` method of the `Graph` class.

```
class Graph:
    "A graph."

    def __init__(self, nodes):
        self.nodes = nodes
        self.edges = []

    def add_edge(self, source, destination):
        "Add an edge from source to destination."
        if source >= 0 and source < self.nodes:
            if destination >= 0 and destination < self.nodes:
                if (source, destination) not in self.edges:
                    self.edges.append((source, destination))

    def destinations(self, source):
        "Returns a list of all destinations of the given node."
        assert source >= 0 and source < self.nodes
        return ___(a)___

    def has_path(self, source, destination):
        "Returns True if a path exists from source to destination, and False otherwise."
        ...  # The implementation of this method is omitted, but assume it works.
```

i. (2.0 pt) Fill in blank (a).

(b) (6.0 points)

The function `compose_path` takes in a list of functions `funcs` and three non-negative integers, `x`, `y`, and `maxval`. It returns whether there exists a list of numbers `s` such that:

- `s` begins with `x` and ends with `y`.
- `s[i+1] = f(s[i])` for some `f` in `funcs`, for every adjacent pair of elements `s[i]` and `s[i+1]`.
- `0 <= s[i]` and `s[i] < maxval` for all elements `s[i]`.

Assume `x` and `y` are less than `maxval` and that all elements of `funcs` are one-argument functions that take in an integer and output an integer. Complete the implementation of `compose_path`. (**Hint**: Treat each function call f(x) as "moving" from x to f(x).)

```
def compose_path(funcs, x, y, maxval):
    """
    >>> f, g, h = lambda x: 2*x-1, lambda x: x*x+1, lambda x: x-4
    >>> #3->10->6->11->122->118->114->110->106->102->98
    >>> #  g   h  f   g    h    h    h    h    h    h
    >>> compose_path([f, g, h], 3, 98, 1000)
    True
    >>> #The above path hits 122, and no other path exists to get from 3 to 98
    >>> compose_path([f, g, h], 3, 98, 122)
    False
    >>> [i for i in range(100) if compose_path([h], 10, i, 100)]
    [2, 6, 10]
    """
    g = Graph(___(a)___)
    for f in funcs:
        ___(b)___:
            ___(c)___
    ___(d)___
```

i. (1.0 pt) Fill in blank (a).

ii. (1.0 pt) Fill in blank (b).

iii. (2.0 pt) Fill in blank (c).

iv. (1.0 pt) Fill in blank (d).

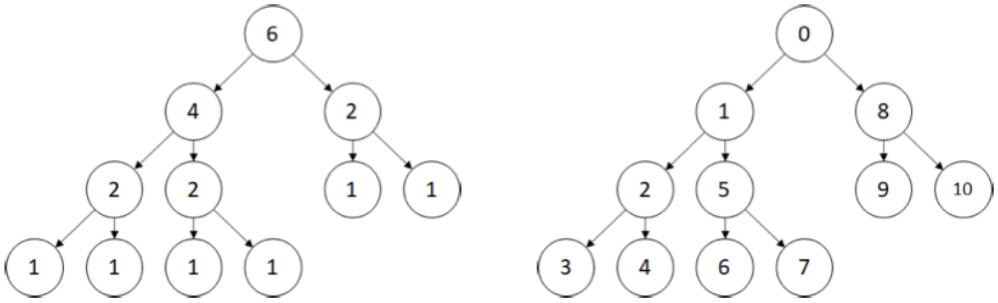
7. (0.0 points) A+ Question 2: Deforestation

This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Implement the function `tree_to_graph`, which takes as input a Tree instance. It returns a Graph instance (defined in the previous question) corresponding to the tree, with nodes ordered in pre-order starting with 0. Pre-order is defined as:

- The root node is assigned the lowest value unassigned.
- After this, the first branch of the tree is assigned values according to pre-order.
- Then, the second branch of the tree is assigned values, and so on.

For example, calling `tree_to_graph` on this `Tree` instance to the left should create this `Graph` instance to the right.




```
def tree_to_graph(tree):
    def helper(tree, op1, op2):
        a = op1()
        return op2(a, [helper(b, op1, op2) for b in tree.branches]) or a
    num_nodes = helper(tree, ___(a)___, ___(b)___)
    g = Graph(num_nodes)
    b = iter(range(num_nodes))
    helper(tree, ___(c)___,___(d)___)
    return g
```

(a) (0.0 pt) Fill in blank (a).

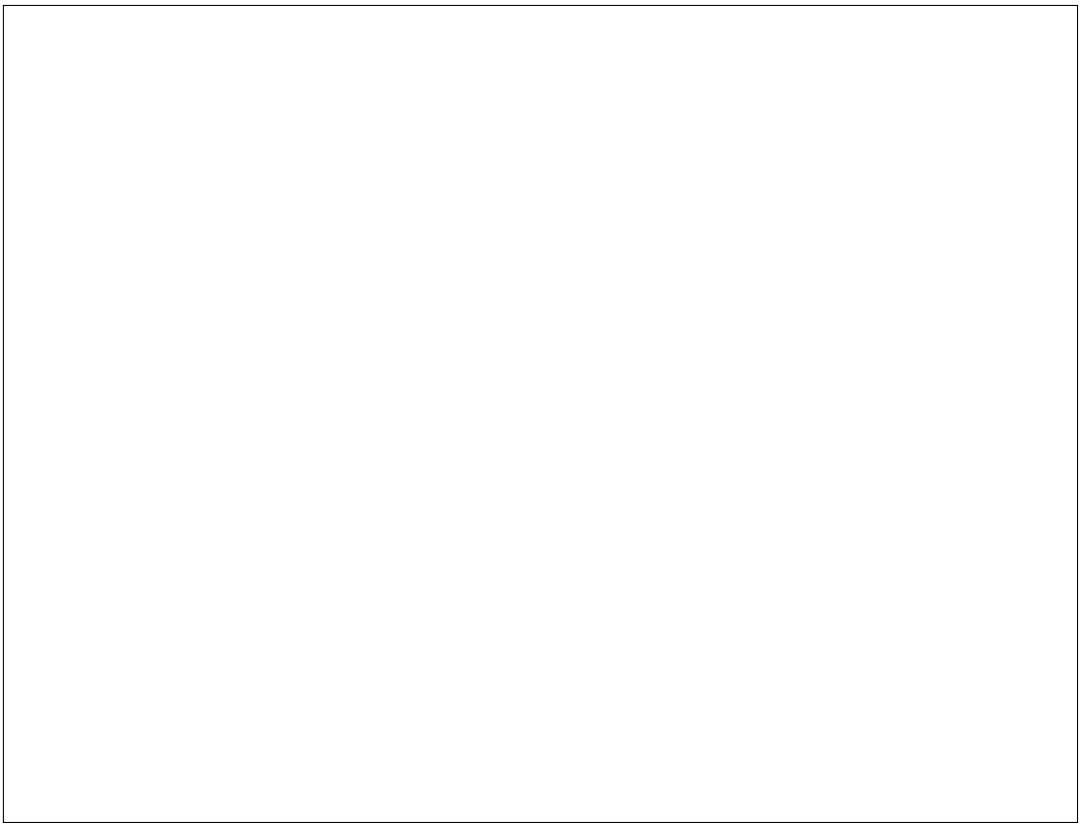
(b) (0.0 pt) Fill in blank (b).

(c) (0.0 pt) Fill in blank (c).

(d) (0.0 pt) Fill in blank (d).

8. (0.0 points) OPTIONAL

(a) (0.0 pt) Draw a picture related to CS 61A or write a program about CS 61A.



No more questions.

CS 61A Fall 2023

Structure and Interpretation of Computer Programs

MIDTERM 1

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (6.0 points) What Would Python Display?

Assume the following code has been executed already.

```
cat = 3
def dog():
    return cat
def hog():
    cat = 4
    print(cat, dog())
cat = 5
```

For each expression below, write the output **displayed by the interactive Python interpreter** when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. To display a function value, write "Function".

(a) (1.0 pt) `dog(cat)`

- ◯ 3
- ◯ 4
- ◯ 5
- ◯ Function
- ◯ Error

(b) (1.0 pt) `dog()`

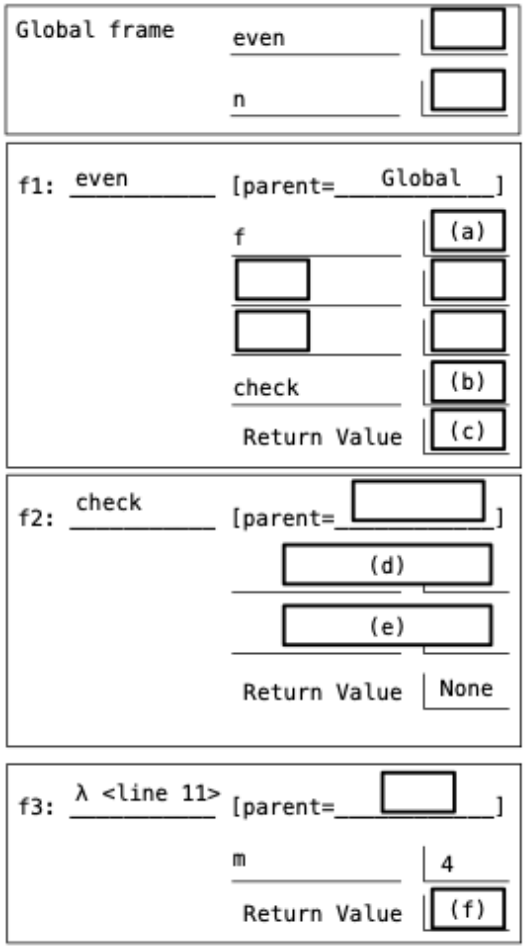
- ◯ 3
- ◯ 4
- ◯ 5
- ◯ Function
- ◯ Error

(c) (2.0 pt) `print((lambda dog: print(dog()))(lambda: 6))`

(d) (2.0 pt) `hog()`

2. (8.0 points) An Odd Implementation of Even

Complete the environment diagram below and then answer the questions that follow. There is one question for each labeled blank in the diagram. The blanks with no labels have no questions associated with them and are not scored. **Some blanks may be empty or unused. If a blank contains an arrow to a function, write the function as it would appear in the diagram. Do not add frames for calls to built-in functions.**




```
 1: def even(f, n):
 2:     "Return whether f(n) is even."
 3:     even = False
 4:     def check(h):
 5:         if h(n) % 2 == 0:
 6:             even = True
 7:     check(f)
 8:     return even
 9:
10: n = 2
11: even(lambda m: m + n, 4)
```

(a) (1.0 pt) Fill in blank (a).

- ◯ `func lambda(m) <line 11> [parent=Global]`
- ◯ `func lambda(m) <line 11> [parent=f1]`
- ◯ `func lambda(m, n) <line 11> [parent=Global]`
- ◯ `func lambda(m, n) <line 11> [parent=f1]`

(b) (1.0 pt) Fill in blank (b).

- ◯ `func check(f) [parent=Global]`
- ◯ `func check(f) [parent=f1]`
- ◯ `func check(h) [parent=Global]`
- ◯ `func check(h) [parent=f1]`

(c) (1.0 pt) Fill in blank (c).

- ◯ `None`
- ◯ A function named `check`
- ◯ `True`
- ◯ `False`
- ◯ Blank: an error occurs before a value is returned.

(d) (1.0 pt) Fill in blank (d).

- ◯ `f` is bound to a lambda function.
- ◯ `h` is bound to a lambda function.
- ◯ `f` is bound to a number.
- ◯ `h` is bound to a number.

(e) (1.0 pt) Fill in blank (e).

- ◯ `n` is bound to a number.
- ◯ `m` is bound to a number.
- ◯ `check` is bound to `True`
- ◯ `check` is bound to `False`
- ◯ `even` is bound to `True`
- ◯ `even` is bound to `False`
- ◯ Blank: there is no name-value binding here.

(f) (1.0 pt) Fill in blank (f).

- ◯ 2
- ◯ 4
- ◯ 6
- ◯ 8
- ◯ Blank: an error occurs before a value is returned.

(g) (2.0 pt) What problems are there in the `even` function's implementation? **Select all that apply.**

- ☐ It does not call `f` on `n` for its original arguments `f` and `n`.
- ☐ It does not correctly test whether `f(n)` is an even number.
- ☐ It always stops without returning due to an error.
- ☐ It always returns `False` regardless of the value of `f(n)`.
- ☐ None of the above, but there is some other error in the implementation.
- ☐ None of the above because there is no error in the implementation.

3. (11.0 points) In Your Prime

Definition: The *prime gap* of a prime number `p` is the difference `q-p` between `p` and the next larger prime `q`.

Assume a function `is_prime` is implemented that takes a positive integer and returns `True` if it is prime and `False` otherwise. You may call `is_prime` in your implementations below.

```
def is_prime(n):
    """Return whether positive integer n is a prime number."""
```

(a) (6.0 points)

Implement `smallest_gap`, which takes a positive integer `k`. It returns the smallest prime number that has a prime gap greater than or equal to `k`.

```
def smallest_gap(k):
    """Return the smallest prime number with prime gap greater than or equal to k.

    >>> smallest_gap(2) # 5 - 3 = 2
    3
    >>> smallest_gap(6) # 29 - 23 = 6
    23
    >>> smallest_gap(9) # 127 - 113 = 14, and no prime below 113 has a gap above 8
    113
    """
    p, q = 2, 3  # The two smallest prime numbers
    while _______:
            (a)
        p, q = _______
                 (b)
        while _______:
                (c)
            q = q + 1
    return p
```

i. (2.0 pt) Fill in blank (a).

- ◯ `q - p == k`
- ◯ `q - p != k`
- ◯ `q - p >= k`
- ◯ `q - p <= k`
- ◯ `q - p > k`
- ◯ `q - p < k`

ii. (2.0 pt) Fill in blank (b).

iii. (2.0 pt) Fill in blank (c). **You may not use `and` or `or`.**

(b) (5.0 points)

Implement `plus_one`, a function that takes two different positive integers `a` and `b`. If the larger one is prime, it returns one more than the smaller one. Otherwise, it returns one more than the average of `a` and `b`.

```
def plus_one(a, b):
    """Return one more than either the smaller (if larger is prime) or average of a and b.

    >>> plus_one(5, 7) # 7 is prime, so return 1 more than 5
    6
    >>> plus_one(15, 7) # 15 is not prime, so return 1 more than the average of 15 and 7
    12.0
    """
    assert a > 0 and b > 0 and a != b

    if _______:
         (d)

        f = _______
              (e)

    else:

        f = _______
              (f)

    return f(a, b) + 1
```

i. (2.0 pt) Fill in blank (d).

ii. (1.0 pt) Fill in blank (e).

- ◯ `max`
- ◯ `min`
- ◯ `max()`
- ◯ `min()`
- ◯ `max(a, b)`
- ◯ `min(a, b)`

iii. (2.0 pt) Fill in blank (f).

4. (15.0 points) Choose Wisely

Definition: A *digit test* is a function that takes a non-negative integer less than 10 and returns `True` or `False`.

(a) (9.0 points)

Implement `only` which takes a non-negative integer `n` and a *digit test* `t`. It returns a non-negative integer containing only the digits of `n` for which `t` returns `True` when called on each of those digits. The digits should appear in the same order as they did in `n`. The number 0 has no digits. **You may not use `str` or `repr` or `[` or `]` or `for`.**

```
def only(n, t):
    """Return only the digits of n for which t returns True when called on each digit.

    >>> only(23344567, lambda d: d % 2 == 0)
    2446
    >>> only(987654349675, lambda d: d < 7)
    6543465
    >>> only(2023, lambda d: False)
    0
    """
    keep = 0
    while n:
        n, d = n // 10, n % 10

        if _______:
             (a)

            keep = 10 * keep + _______
                                  (b)
    _______:
      (c)

        _______
          (d)
    return n
```

i. (2.0 pt) Fill in blank (a).

ii. (1.0 pt) Fill in blank (b).

- ◯ `d`
- ◯ `d // 10`
- ◯ `n`
- ◯ `n % 10`
- ◯ `keep`
- ◯ `keep % 10`
- ◯ `keep // 10`

iii. (2.0 pt) Fill in blank (c)

- ◯ `if True`
- ◯ `if n`
- ◯ `if keep`
- ◯ `if n > keep`
- ◯ `if keep > n`
- ◯ `while True`
- ◯ `while n`
- ◯ `while keep`
- ◯ `while n > keep`
- ◯ `while keep > n`

iv. (4.0 pt) Fill in blank (d) **with an assignment statement**. You are allowed to assign to multiple names.

(b) (6.0 points)

Implement `every` which takes a *digit test* `t` and returns a function `digit` that takes a positive integer `n`. The `digit` function returns whether `t` returns True for every digit of `n`.

```
def every(t):
    """Return a function of n that returns whether t returns True for every digit of n.

    >>> f = every(lambda d: d % 2 == 1)
    >>> f(37511)  # every digit is odd
    True
    >>> f(2023)   # Not every digit is odd
    False
    """
    def digit(n):
        assert n > 0
        while n:
            if _______:
                 (e)
                _______
                  (f)
            n = n // 10
        return _______
                  (g)
    return _______
             (h)
```

i. (3.0 pt) Fill in blank (e).

ii. (1.0 pt) Fill in blank (f).

iii. (1.0 pt) Fill in blank (g).

- ◯ `every`
- ◯ `digit`
- ◯ `True`
- ◯ `False`
- ◯ `n > 0`

iv. (1.0 pt) Fill in blank (h).

- ◯ `digit`
- ◯ `digit()`
- ◯ `digit(n)`
- ◯ `lambda n: digit`

(c) This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Implement `even_odd`, which takes a positive integer `n` that has both even and odd digits. It returns `True` if all of the odd digits of `n` are larger than the last (right-most) even digit of `n`. Otherwise, it returns `False`. You may call `only` and `every`. **You may not use `str` or `repr` or `[` or `]` or `for`.**

```
def even_odd(n):
    """Return whether all odd digits of n are larger than n's last even digit.

    >>> even_odd(8023) # 3 (the only odd digit) is larger than 2 (the last even digit)
    True
    >>> even_odd(5237679) # 3 (an odd digit) is smaller than 6 (the last even digit)
    False
    >>> even_odd(7297679) # All odd digits (7 & 9) are larger than 6 (the last even digit)
    True
    """
    return _______
```

i. Fill in the blank. You may only write one return expression, but if it's long you can use multiple lines.

No more questions.

CS 61A Fall 2023

Structure and Interpretation of Computer Programs

MIDTERM 2

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (4.0 points) What Would Python Display?

Assume the following code has been executed.

```
s = range(3, 7)
t = iter(s)
u = map(lambda x: 2 * x, t)
v = [next(t), next(t)]      # This line does not cause an error
```

Choose the output **displayed by the interactive Python interpreter** when each expression below is evaluated or *Error* if an error occurs. These expressions are evaluated **in order** and the value of later expressions may be affected by evaluating previous expressions.

(a) (1.0 pt) `[-k for k in v if k in s]`

- ◯ `[]`
- ◯ `[3, 4]`
- ◯ `[-3, -4]`
- ◯ *Error*

(b) (1.0 pt) `tuple(u)`

- ◯ `(3, 4, 5, 6)`
- ◯ `(3, 4, 5, 6, 7)`
- ◯ `(6, 8, 10, 12)`
- ◯ `(6, 8, 10, 12, 14)`
- ◯ `(10, 12)`
- ◯ `(10, 12, 14)`
- ◯ *Error*

(c) (1.0 pt) `next(t)`

- ◯ `3`
- ◯ `5`
- ◯ `range(3, 7)`
- ◯ `range(5, 7)`
- ◯ *Error*

(d) (1.0 pt) `next(iter(s)) + next(iter(s))`

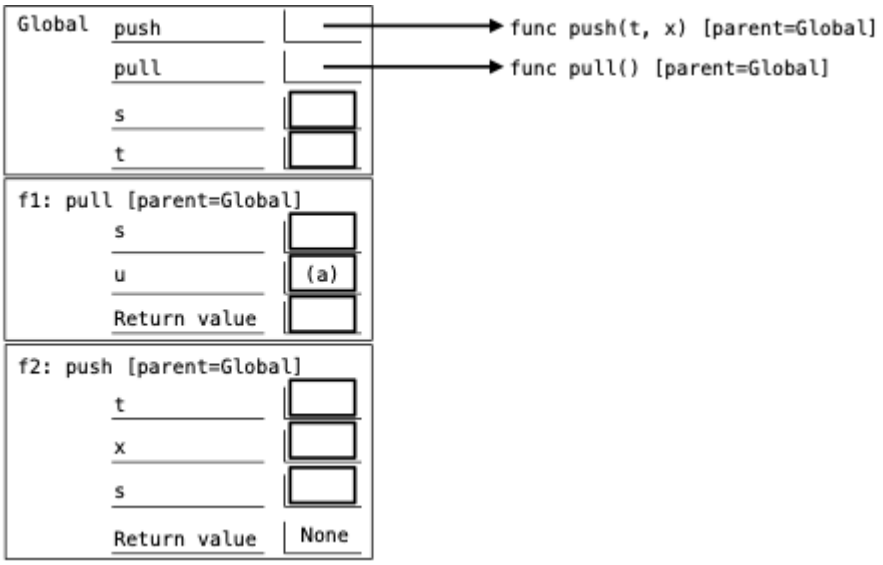
- ◯ `6`
- ◯ `7`
- ◯ `10`
- ◯ `11`
- ◯ *Error*

2. (6.0 points) Making a List, Checking it Twice

Complete the environment diagram below and then answer the questions that follow. There is one question for each labeled blank in the diagram. The blanks with no labels have no questions associated with them and are not scored. **Some blanks may be empty or unused. If a blank contains an arrow to a function, write the function as it would appear in the diagram. Do not add frames for calls to built-in functions.**

Reminder: `list(s)` creates a new list with the same elements as `s`; `s.pop()` removes and returns the last element of `s`.

```
def push(t, x):
    s = list(t)
    s[1].append(x)

def pull():
    s = list(t)
    u = s.pop()
    return u[1][0]

s = [7, [8, 9]]
t = [6, s]

push(s, pull())
print(t)
```



(a) (3.0 pt) What value fills blank (a)? Select **all** that apply.

- ☐ `None`
- ☐ A list
- ☐ A number
- ☐ The object bound to global `s`
- ☐ The object bound to global `t`
- ☐ The same object that `s[0]` would evaluate to in the global frame
- ☐ The same object that `s[1]` would evaluate to in the global frame
- ☐ The same object that `t[0]` would evaluate to in the global frame
- ☐ The same object that `t[1]` would evaluate to in the global frame

(b) (3.0 pt) What would be printed by the expression `print(t)` at the end?

3. (6.0 points) 24-Hour Library

Anyone can check out a book from a library and then bring that book back. Implement the `Library` and `Book` classes. A `Library` takes a list of unique strings called `titles` and constructs a `books` dictionary that has strings as keys and `Book` objects as values. Its `checkout` method takes a string `title` and returns the corresponding `Book` object if that `title` is not checked out, or prints a message. After `bring_back` is invoked on that book, it can be checked out again.

```
class Library:
    """A library with one copy of each title that can be checked out.

    >>> cs = Library(['Composing Programs', 'Python Docs', 'Berkeley Academic Guide'])
    >>> bs = [cs.checkout('Composing Programs'), cs.checkout('Python Docs')]
    >>> cs.checkout('Composing Programs')        # This time, no Book is returned
    Composing Programs is checked out
    >>> bs[0].bring_back()
    >>> cs.checkout('Composing Programs').title  # This time, a Book is returned
    'Composing Programs'
    """
    def __init__(self, titles):
        self.books = {t: Book(t, _______) for t in titles}
                                    (a)
        self.out = []  # A list of Book objects

    def checkout(self, title):
        assert title in self.books, title + " isn't in this library's collection"
        book = _______
                 (b)
        if book not in self.out:
            _______
              (c)
            return book
        else:
            print(book, 'is checked out')

class Book:
    def __init__(self, title, library):
        self.title = title      # a string
        self.library = library  # a Library object

    def bring_back(self):
        _______.remove(_______)
          (d)            (e)

    def __str__(self):
        return _______
                 (f)
```

(a) (1.0 pt) Fill in blank (a).

- ◯ `self`
- ◯ `self.books`
- ◯ `Library`
- ◯ `Library()`
- ◯ `Library(titles)`

(b) (1.0 pt) Fill in blank (b).

(c) (1.0 pt) Fill in blank (c).

- ◯ `out.append(book)`
- ◯ `out.extend(book)`
- ◯ `Library.append(book)`
- ◯ `Library.extend(book)`
- ◯ `self.out.append(book)`
- ◯ `self.out.extend(book)`

(d) (1.0 pt) Fill in blank (d).

- ◯ `self.`
- ◯ `self.out`
- ◯ `library`
- ◯ `library.out`
- ◯ `self.library.out`

(e) (1.0 pt) Fill in blank (e).

- ◯ `self`
- ◯ `title`
- ◯ `library`
- ◯ `self.title`
- ◯ `self.library`

(f) (1.0 pt) Fill in blank (f).

- ◯ `self`
- ◯ `title`
- ◯ `self.title`
- ◯ `repr(self)`
- ◯ `repr(title)`
- ◯ `repr(self.title)`

4. (16.0 points) A Perfect Question

Definition. A *perfect square* is `k*k` for some integer `k`.

(a) (7.0 points)

Implement `fit`, which takes **positive** integers `total` and `n`. It returns `True` or `False` indicating whether there are `n` **positive** perfect squares that sum to `total`. The perfect squares need not be unique.

```
def fit(total, n):
    """Return whether there are n positive perfect squares that sums to total.

    >>> [fit(4, 1), fit(4, 2), fit(4, 3), fit(4, 4)]  # 1*(2*2) for n=1; 4*(1*1) for n=4
    [True, False, False, True]
    >>> [fit(12, n) for n in range(3, 8)]  # 3*(2*2), 3*(1*1)+3*3, 4*(1*1)+2*(2*2)
    [True, True, False, True, False]
    >>> [fit(32, 2), fit(32, 3), fit(32, 4), fit(32, 5)] # 2*(4*4), 3*(1*1)+2*2+5*5
    [True, False, False, True]
    """
    def f(total, n, k):

        if _______:
             (a)
            return True

        elif _______:
               (b)
            return False
        else:
            return _______
                     (c)

    return f(total, n, 1)
```

i. (2.0 pt) Select **all** of the options that could fill blank (a).

- ☐ `total == 0`
- ☐ `n == k`
- ☐ `total == 0 and n == 0`
- ☐ `total == k * k`
- ☐ `total == k * k and n == 1`
- ☐ `total == n * k * k`

ii. (2.0 pt) Fill in blank (b).

iii. (3.0 pt) Fill in blank (c) with an expression of the form `f(___, ___, ___) ___ f(___, ___, ___)`.

(b) (9.0 points)

Implement the generator function `squares`, which takes **positive** integers `total` and `k`. It yields all lists of perfect squares greater or equal to `k*k` that sum to `total`. Each list is in non-increasing order (large to small).

```
def squares(total, k):
    """Yield the ways in which perfect squares greater or equal to k*k sum to total.

    >>> list(squares(10, 1))  # All lists of perfect squares that sum to 10
    [[1, 1, 1, 1, 1, 1, 1, 1, 1, 1], [4, 1, 1, 1, 1, 1, 1], [4, 4, 1, 1], [9, 1]]
    >>> list(squares(20, 2))  # Only use perfect squares greater or equal to 4 (2*2).
    [[4, 4, 4, 4, 4], [16, 4]]
    """
    assert total > 0 and k > 0
    if total == k * k:

        yield _______
                (d)
    elif total > k * k:

        for s in _______:
                    (e)

            yield _______
                    (f)
        yield from squares(total, k + 1)
```

i. (2.0 pt) Fill in blank (d).

ii. (3.0 pt) Fill in blank (e).

iii. (1.0 pt) Select **all** of the options that could fill in blank (f).

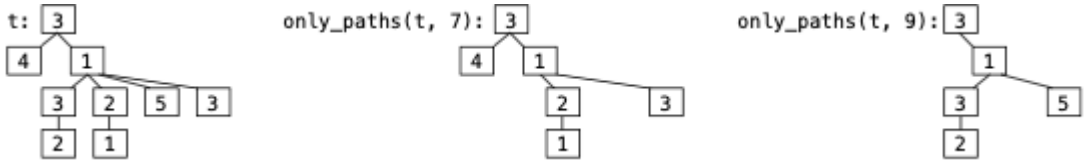
- ☐ `s + k*k`
- ☐ `s.append(k*k)`
- ☐ `s + [k*k]`
- ☐ `[s] + [k*k]`

iv. (3.0 pt) Write an expression containing `a` that evaluates to the **shortest** list of perfect squares that sum to an integer `a`. For example, if `a = 32`, your expression should evaluate to `[16, 16]` (not `[25, 4, 1, 1, 1]`). If there are two or more such lists that are both shortest, it can evaluate to any of them. Assume `squares` is implemented correctly.

5. (7.0 points) Only Paths

Implement `only_paths`, which takes a Tree of numbers `t` and a number `n`. It returns a new Tree with only the nodes of `t` on a path from the root to a leaf with labels that sum to `n`, or `None` if no path sums to `n`. Do not mutate `t`.

The Tree class appears on the midterm 2 study guide. Here is an illustration of the doctest examples involving `t`.




```
def only_paths(t, n):
    """Return a Tree with only the nodes of t along paths from the root to a leaf of t
    for which the node labels of the path sum to n. If no paths sum to n, return None.

    >>> print(only_paths(Tree(5, [Tree(2), Tree(1, [Tree(2)]), Tree(1, [Tree(1)])]), 7))
    5
      2
      1
        1
    >>> t = Tree(3, [Tree(4), Tree(1, [Tree(3, [Tree(2)]), Tree(2, [Tree(1)]), Tree(5), Tree(3)])])
    >>> print(only_paths(t, 7))
    3
      4
      1
        2
          1
        3
    >>> print(only_paths(t, 9))
    3
      1
        3
          2
        5
    >>> print(only_paths(t, 3))
    None
    """
    if _______:
         (a)

        return t

    new_branches = [_______ for b in t.branches]
                       (b)

    if _______(new_branches):
         (c)

        return Tree(t.label, [b for b in new_branches if _______])
                                                           (d)
```

(a) (2.0 pt) Fill in blank (a).

(b) (3.0 pt) Fill in blank (b).

(c) (1.0 pt) Fill in blank (c).

- ◯ `all`
- ◯ `any`
- ◯ `has`
- ◯ `only_paths`

(d) (1.0 pt) Fill in blank (d).

- ◯ `only_paths(b, n)`
- ◯ `b.label == n`
- ◯ `b.label != n`
- ◯ `b is None`
- ◯ `b is not None`

(e) This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first! Fill in the blank to implement `only_long_paths`, which takes a Tree of numbers `t` and a number `n`. It returns a new Tree containing only the nodes of `t` that lie on a path from the root to a leaf for which the sum of the labels **plus the length of the path** is `n`. Do not mutate `t`. You may not write `and`, `or`, `if`, `[` or `]`. Assume `only_paths` is implemented correctly.

```
only_paths = _____________

def only_long_paths(t, n):
    """Return a Tree with only the nodes of t along paths from the root to a leaf of t
    for which the sum of node labels plus the length of the path is n.

    >>> example = Tree(5, [Tree(3), Tree(1, [Tree(2)]), Tree(1, [Tree(1)])])
    >>> only_long_paths(example, 10)  # Result has paths 5-3 (length 2) and 5-1-1 (length 3)
    Tree(5, [Tree(3), Tree(1, [Tree(1)])])
    """
    return only_paths(t, n)
```

6. (6.0 points) After Party

Implement `after`, which takes a linked list `s` and values `a` and `b`. It returns whether an element of `s` equal to `b` appears after an element of `s` equal to `a`. The `Link` class appears on the Midterm 2 study guide.

```
def after(s, a, b):
    """Return whether b comes after a in linked list s.

    >>> t = Link(3, Link(6, Link(5, Link(4))))
    >>> after(t, 6, 4)
    True
    >>> after(t, 4, 6)
    False
    >>> after(t, 6, 6)
    False
    """
    def find(s, n, f):

        if s == Link.empty:

            return _______
                     (a)

        elif s.first == n:
            return f(s.rest)
        else:
            return find(s.rest, n, f)

    return find(s, a, lambda rest: _______)
                                     (b)
```

(a) (1.0 pt) Fill in blank (a).

- ◯ `n`
- ◯ True
- ◯ False
- ◯ `a < b`
- ◯ `f(Link.empty)`

(b) (4.0 pt) Fill in blank (b). **You may not use `or`, `and`, `if`, `[`, or `]`.**

(c) (1.0 pt) What is the order of growth of the run time of an efficient implementation of `after` in terms of the length of `s`? Here, *efficient* means that you have filled in the blanks to give the function the fastest order of growth.

- ◯ Constant
- ◯ Logarithmic
- ◯ Linear
- ◯ Quadratic
- ◯ Exponential

No more questions.

CS 61A Fall 2024

Structure and Interpretation of Computer Programs

MIDTERM 1

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (8.0 points) What Would Python Display?

Answer the following questions about the output printed by this code.

```
def mad(max):
    g = lambda: (print(1) or 2) or (print(3) or 4)
    print(max(5, 6))
    return g
print(mad(print)(), 7, print(8))
```

(a) (2.0 pt) What is the last line printed?

- ◯ `1 7 8`
- ◯ `2 7 8`
- ◯ `2 7 None`
- ◯ `4 7 8`
- ◯ `4 7 None`
- ◯ `None 7 8`
- ◯ `None 7 None`

(b) (4.0 pt) Which of these **whole lines** appear **somewhere** in the printed output? Select all that apply.

- ☐ `1`
- ☐ `1 2`
- ☐ `1 2 3`
- ☐ `1 2 3 4`
- ☐ `3`
- ☐ `5`
- ☐ `5 6`
- ☐ `6`
- ☐ `7`
- ☐ `8`
- ☐ `None`
- ☐ `None None`
- ☐ `True`

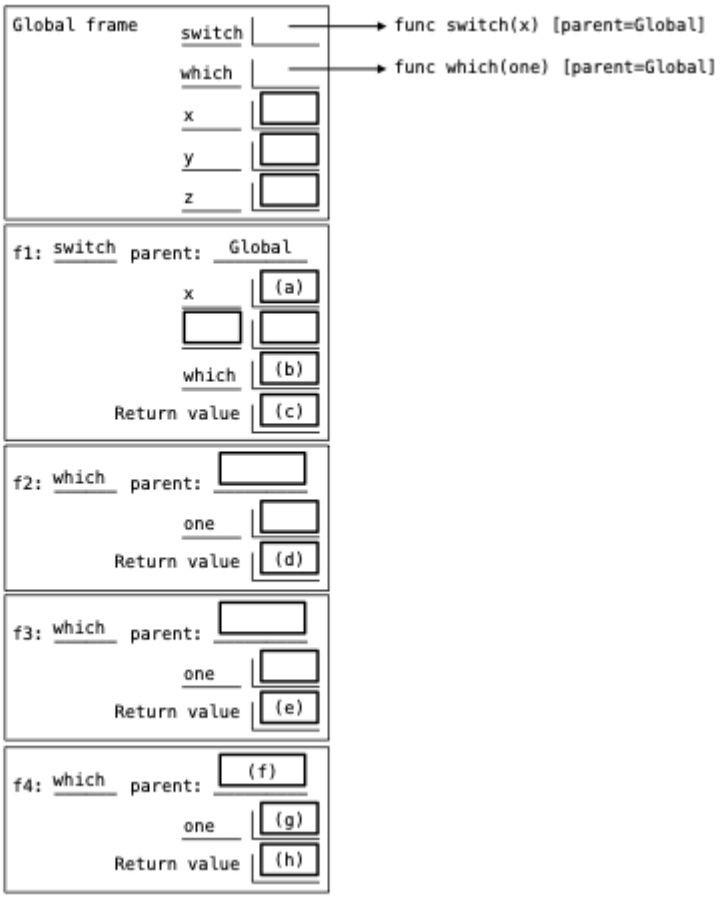
(c) (2.0 pt) What **order** do 1, 6, and 8 appear in the printed output? These numbers may appear as part of longer lines. If a number appears twice, consider just the first occurence.

- ◯ `1 6 8`
- ◯ `1 8 6`
- ◯ `6 1 8`
- ◯ `6 8 1`
- ◯ `8 1 6`
- ◯ `8 6 1`

2. (8.0 points) Which One

Complete the environment diagram below and then answer the questions that follow. There is one question for each labeled blank in the diagram. The blanks with no labels have no questions associated with them and are not scored.

```
 1: x, y, z = 1, 2, 3
 2:
 3: def switch(x):
 4:     z = 4
 5:     if x == y:
 6:         def which(one):
 7:             return x + one
 8:     else:
 9:         def which(one):
10:             return x - one
11:     x = which(x)
12:     which(x)
13:
14: def which(one):
15:     return x * one
16:
17: switch(x+1)
18: x = 5
19: which(z)
```



(a) (1.0 pt) Fill in blank (a).

- ◯ `0`
- ◯ `1`
- ◯ `2`
- ◯ `3`
- ◯ `4`

(b) (1.0 pt) Fill in blank (b).

- ◯ `func which(one) [parent=f1]`
- ◯ `func which(one) [parent=Global]`
- ◯ `func which(one) [parent=switch]`

(c) **(1.0 pt)** Fill in blank (c).

(d) **(1.0 pt)** Fill in blank (d).

- ◯ `0`
- ◯ `1`
- ◯ `2`
- ◯ `3`
- ◯ `4`

(e) **(1.0 pt)** Fill in blank (e).

(f) **(1.0 pt)** Fill in blank (f).

- ◯ `Global`
- ◯ `f1`
- ◯ `f2`
- ◯ `f3`
- ◯ `f4`

(g) **(1.0 pt)** Fill in blank (g).

- ◯ `1`
- ◯ `2`
- ◯ `3`
- ◯ `4`
- ◯ `5`

(h) **(1.0 pt)** Fill in blank (h).

3. (6.0 points) Final Digit

Implement `final_digit`, which takes a non-negative integer `n`. As long as `n` has more than one digit, replace `n` with the sum of the digits of `n`. This process repeats until `n` becomes a single-digit number, which is returned.

```
def final_digit(n):
    """Sum the digits of n repeatedly to reach one digit.

    >>> final_digit(321)       # 3 + 2 + 1 = 6
    6
    >>> final_digit(987)       # 9 + 8 + 7 = 24, and 2 + 4 = 6
    6
    >>> final_digit(989898989) # The digit sum is 77, 7 + 7 = 14, and 1 + 4 = 5
    5
    """
    while n >= 10:
        s = _______
              (a)
        _______:
          (b)
            n, s = n // 10, _______
        _______                 (c)
          (d)
    return n
```

(a) (1.0 pt) Fill in blank (a).

- ◯ `0`
- ◯ `n`
- ◯ `n % 10`

(b) (1.0 pt) Fill in blank (b).

- ◯ `if s`
- ◯ `if n`
- ◯ `while s`
- ◯ `while n`

(c) (2.0 pt) Fill in blank (c).

- ◯ `n`
- ◯ `s // 10`
- ◯ `n % 10`
- ◯ `s + n % 10`
- ◯ `10 * s + n % 10`

(d) (2.0 pt) Fill in blank (d).

4. (8.0 points) Close Enough

Implement `close`, which takes two non-negative integers `m` and `n`. It returns whether `m` can be changed into `n` by either inserting one digit, removing one digit, or changing one digit. If `m` and `n` are the same number, they are *not* `close`.

```
def close(m, n):
    """Return whether m can result from starting with n and adding, removing,
    or changing one digit.

    >>> close(3756, 3456) and close(3456, 346) and close(346, 3456) and close(456, 56)
    True
    >>> close(5, 5) or close(3456, 3546) or close(3456, 36) or close(34, 3456) or close(345, 456)
    False
    """
    if m < n:
        m, n = n, m
    while m or n:
        if _______:
             (a)
            m, n = _______
                     (b)
        else:

            return _______ or _______  # Hint: check here that just one change is enough
                     (c)        (d)
    return _______
             (e)
```

(a) (1.0 pt) Fill in blank (a).

- ◯ `m == n`
- ◯ `m // 10 == n // 10`
- ◯ `m // 10 == n`
- ◯ `m == n // 10`
- ◯ `m % 10 == n % 10`

(b) (2.0 pt) Fill in blank (b).

- ◯ `n, m`
- ◯ `m // 10, n // 10`
- ◯ `m // 10, n`
- ◯ `m, n // 10`
- ◯ `m % 10, n % 10`

(c) (2.0 pt) Fill in blank (c).

- ◯ `m == n`
- ◯ `m - n < 10`
- ◯ `n == 0`
- ◯ `m // 10 == n // 10`
- ◯ `m % 10 == n % 10`
- ◯ `m % 100 == n % 100`

(d) (2.0 pt) Fill in blank (d).

- ◯ `m // 10 == n`
- ◯ `m // 10 - n < 10`
- ◯ `n == 0`
- ◯ `m // 10 == 0`
- ◯ `m // 100 == n // 10`
- ◯ `(m // 10) % 10 == n % 10`

(e) (1.0 pt) Fill in blank (e).

- ◯ `m == n`
- ◯ `m == 0`
- ◯ `n == 0`
- ◯ `True`
- ◯ `False`

5. (10.0 points) Shifty

(a) (4.0 points)

Implement `shift`, which takes a number `k` and a one-argument function `f`. It returns a one-argument function `g` that takes a number `x`. For all numbers `x`, `g(x)` is equal to `f(x + k)`.

```
def shift(k, f):
    """Return a function of x that returns f(x+k).

    >>> square = lambda x: x * x
    >>> g = shift(2, square)
    >>> g(3)     # square(3 + 2)
    25
    """
    return _______
             (a)
```

i. (2.0 pt) Fill in blank (a).

- ◯ `f(x) + k`
- ◯ `f(x + k)`
- ◯ `f(lambda x: x + k)`
- ◯ `f(lambda x: x) + k`
- ◯ `lambda x: f(x) + k`
- ◯ `lambda x: f(x + k)`
- ◯ `(lambda x: f(x))(x + k)`
- ◯ `g(x) + k`
- ◯ `g(x + k)`
- ◯ `g(lambda x: x + k)`
- ◯ `g(lambda x: x) + k`
- ◯ `lambda x: g(x) + k`
- ◯ `lambda x: g(x + k)`
- ◯ `(lambda x: g(x))(x + k)`

ii. (2.0 pt) Provide an alternate solution for blank (a) . This time, your solution **must** call `compose` (defined below), and `f` **may not** be the operator of a call expression. In other words, you can't write `f(` in your answer. You **may not** write `[` or `if`.

```
def compose(f, g):
    """Return a function that takes x and calls f on g of x."""
    return lambda x: f(g(x))
```

(b) (6.0 points)

Implement `sum_range`, which takes positive integers `p` and `q` with `p <= q`, as well as a one-argument function `term`. It returns the sum of the return values of `term` called on each consecutive integer starting with `p` and ending with `q` (including both `p` and `q`). You may call `shift`, `summation`, and `compose`. Assume `shift` is implemented correctly.

```
def summation(n, term):
    """Sum the first n terms of a sequence: term(1) + term(2) + ... + term(n).

    >>> summation(5, lambda x: x*x)  # 1*1 + 2*2 + 3*3 + 4*4 + 5*5
    55
    """
    total, k = 0, 1
    while k <= n:
        total, k = total + term(k), k + 1
    return total

def sum_range(p, q, term):
    """Sum terms p through q of a sequence: term(p) + term(p+1) + ... + term(q).

    >>> sum_range(1, 5, lambda x: x*x)   # 1*1 + 2*2 + 3*3 + 4*4 + 5*5
    55
    >>> sum_range(4, 5, lambda x: x*x)   # 4*4 + 5*5
    41
    >>> sum_range(5, 5, lambda x: x*x)   # 5*5
    25
    """
    assert p <= q
    return _______ ( _______ , _______ )
             (a)       (b)       (c)
```

i. (1.0 pt) Fill in blank (a).

- ◯ `summation`
- ◯ `shift`
- ◯ `compose`
- ◯ `term`

ii. (2.0 pt) Fill in blank (b).

- ◯ `p - q - 1`
- ◯ `p - q`
- ◯ `p - q + 1`
- ◯ `q - p - 1`
- ◯ `q - p`
- ◯ `q - p + 1`

iii. (3.0 pt) Fill in blank (c).

(c) (0.0 points)

This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

The `shifter` function below is a curried version of `shift`.

Implement `unshift`, which takes the result of `shifter(k)` for some number `k`. It returns a function that takes the result of `shifter(k)(f)` for some function `f` and returns a function equivalent to `f`. That is:

`f(x) == unshift(shifter(k))(shifter(k)(f))(x)`

You can write your answer on multiple lines if it's long. You can abbreviate `lambda` using the greek symbol lambda.

Your answer **must** be a call to `shift`. You may also call any of `compose`, `summation`, or `sum_range`.

Hint: If you can compute `k`, then you can `shift` backward by `k` to undo the original shift.

```
def shifter(k):
    def shifted(f):
        return shift(k, f)
    return shifted

def unshift(shifted):
    """Assume shifted is the return value of shifter(k) for some k.

    >>> cubic = lambda x: x*x*x - 5*x*x + 1   # Some complicated function
    >>> cubic(2.0)
    -11.0
    >>> cubic(9.0)
    325.0
    >>> do = shifter(4)
    >>> do(cubic)(2.0)        # same as cubic(6.0)
    37.0
    >>> do(cubic)(9.0)        # same as cubic(13.0)
    1353.0
    >>> undo = unshift(do)
    >>> undo(do(cubic))(2.0)  # same as cubic(2.0)
    -11.0
    >>> undo(do(cubic))(9.0)  # same as cubic(9.0)
    325.0
    """
    return lambda g: _______
```

i. (0.0 pt) Fill in the blank with **a single call to `shift`**.

No more questions.

CS 61A Fall 2024

Structure and Interpretation of Computer Programs

MIDTERM 2

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (5.0 points) What Would Python Display?

Assume the following code has been executed. The `Tree` class appears on the midterm 2 study guide (page 2, left side).

```
scare = Tree(0, [Tree(4), Tree(5, [Tree(10)]), Tree(2)])
crow = Tree(9, [scare, Tree(7, [Tree(6)])])

def climb(t, f):
    if t.is_leaf():
        return [t.label]
    return [t.label] + climb(max(t.branches, key=f), f)

def run(t):
    if t.is_leaf():
        return t.label
    else:
        return max(map(run, t.branches))

def hide(t):
    return t.label

jump = {(2 * x - 1): x for x in range(50)}
```

Write the value of each expression below or *Error* if an error occurs.

(a) (1.0 pt) `crow.branches[0].label`

(b) (1.0 pt) `[hide(b) for b in scare.branches]`

(c) (1.0 pt) `climb(crow, hide)`

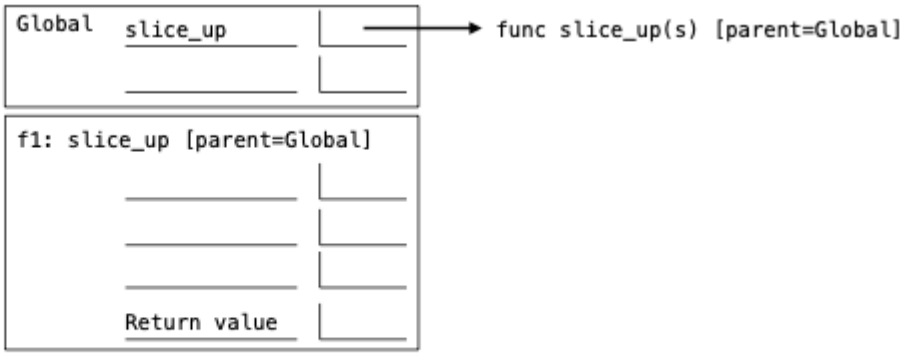
(d) (1.0 pt) `climb(crow, run)`

(e) (1.0 pt) `jump[jump[13]]`

2. (5.0 points) Pizza by the Slice

Answer the questions about the code below. You are encouraged to draw an environment diagram, but the diagram itself will not be scored.

```
 1: def slice_up(s):
 2:     copy = s
 3:     t = [copy]
 4:     while len(s) > 1:
 5:         t.append(s[:len(s)-1])
 6:         s.pop()
 7:         s[0] = len(s)
 8:     return t
 9:
10: t = [3, 4, 5]
11: print(slice_up(t))
12: print(t)
```



(a) (3.0 pt) What would be printed by the expression `print(slice_up(t))` on line 11?

(b) (1.0 pt) What would be printed by the expression `print(t)` on line 12?

- ◯ `[1]`
- ◯ `[2]`
- ◯ `[3]`
- ◯ `[1, 4]`
- ◯ `[2, 4]`
- ◯ `[3, 4]`
- ◯ `[1, 4, 5]`
- ◯ `[2, 4, 5]`
- ◯ `[3, 4, 5]`

(c) (1.0 pt) What is the order of growth of the time it takes to evaluate `list(list(q))` for a list of numbers `q` in terms of the length of `q`. Assume that creating a list with one element takes constant time, and making a copy of a list takes linear time in the length of the list.

- ◯ exponential
- ◯ quadratic
- ◯ linear
- ◯ logarithmic
- ◯ constant

3. (6.0 points) CS 61A Software Store

A `Store` instance has a list of `branches`, and each branch is also a `Store` instance, forming a tree of stores. Each `Store` instance also has a list of programs (strings) called its `inventory`. For a `Store`:

- The `copies(s)` method returns the number of times `s` appears in the inventories of all nodes in its tree of stores.
- The `add_to(k)` method returns a function that takes a string and appends it to the `inventory` of its `k`th branch. Assume that `k` is non-negative and less than the number of branches.

```
class Store:
    """A Store selling programs has branches forming a tree of Stores.
    >>> north, south = Store(['Cats', 'Ants']), Store(['Cats', 'Cats', 'Hog'])
    >>> east, west = Store(['Cats', 'Hog', 'Hog'], [north, south]), Store(['Cats', 'Cats', 'Ants'])
    >>> main = Store(['Ants', 'Ants', 'Hog'], [east, west])
    >>> east.copies('Cats')  # 1 in north, 2 in south, 1 in east
    4
    >>> main.copies('Ants')  # 2 in main, 1 in north, 1 in west
    4
    >>> main.add_to(1)('Ants') # Add 'Ants' to the inventory of west, the branch of main at index 1
    >>> [main.copies('Ants'), west.copies('Ants')]  # increased copies in both main and west
    [5, 2]
    """
    def __init__(self, programs, branches=[]):
        assert all([isinstance(b, Store) for b in branches])
        self.branches = branches
        self.inventory = programs

    def copies(self, s):
        """Return the number of times s (string) appears in all inventories of this tree."""
        return sum([_______ for p in self.inventory if p == s] + [_______ for b in self.branches])
                      (a)                                                    (b)

    def add_to(self, k):
        """Return a function that appends a string to the inventory of the branch at index k."""
        return _______
                 (c)
```

(a) (1.0 pt) Fill in blank (a).

- ◯ `1`
- ◯ `p`
- ◯ `len(p)`
- ◯ `len(self.inventory)`

(b) (2.0 pt) Fill in blank (b).

(c) (3.0 pt) Fill in blank (c).

4. (18.0 points) Almost a Perfect Question

Definition. A *proper divisor* of an integer `n` is an integer `d` less than `n` that evenly divides `n`, and so `n` divided by `d` has no remainder. A *semiperfect* number is a positive integer that is the sum of some (or all) of its proper divisors. For example, $24 = 2 + 4 + 6 + 12$ and so it is semiperfect. The sum of divisors cannot contain repeated numbers.

(a) (6.0 points)

Implement `semiperfect`, which takes a positive integer `n`. It returns `True` if `n` is semiperfect and `False` otherwise.

```
def semiperfect(n):
    """Return whether positive integer n is a sum of some (or all) of its proper divisors.
    >>> [k for k in range(1, 40) if semiperfect(k)]
    [6, 12, 18, 20, 24, 28, 30, 36]
    """
    def f(s, d):
        if _______:
             (a)
            return True
        if d >= n:
            return False
        if _______ and f(_______, d + 1):
             (b)           (c)
            return True
        return _______
                  (d)
    return f(n, 1)
```

i. (1.0 pt) Fill in blank (a).

- ◯ `n == 0`
- ◯ `s == 0`
- ◯ `s == d`
- ◯ `n == d`

ii. (1.0 pt) Fill in blank (b).

- ◯ `n % d == 0`
- ◯ `s % d == 0`
- ◯ `n % d > 0`
- ◯ `s % d > 0`

iii. (2.0 pt) Fill in blank (c).

iv. (2.0 pt) Fill in blank (d).

(b) (6.0 points)

Implement `subsums`, a generator function that takes a list of **unique** (no repeats) positive integers `s` and a positive integer `n`. It yields all sublists of `s` that sum to `n`. A sublist of `s` is a list containing some of the elements of `s` in order. The `subsums` function should not yield the same list twice. The lists may appear in any order.

```
def subsums(s, n):
    """Yield all sublists of s that sum to n.
    >>> list(subsums([1, 2, 3, 6, 9], 18))
    [[1, 2, 6, 9], [3, 6, 9]]
    >>> list(subsums([1, 2, 3, 4, 6], 16))
    [[1, 2, 3, 4, 6]]
    >>> list(subsums([1, 2, 3, 4, 6, 8, 12], 12))
    [[1, 2, 3, 6], [1, 3, 8], [2, 4, 6], [4, 8], [12]]
    """
    if s:
        if _______:
             (e)
            yield [n]
        for t in _______:
                    (f)
            yield _______
                     (g)
        yield from _______
                     (h)
```

i. (1.0 pt) Fill in blank (e).

◯ `n == s`

◯ `n == s[0]`

◯ `n in s`

◯ `[n] == s`

ii. (3.0 pt) Fill in blank (f).

iii. (1.0 pt) Fill in blank (g).

◯ `s + t`

◯ `[s] + t`

◯ `s[1:] + t`

◯ `s[:1] + t`

iv. (1.0 pt) Fill in blank (h).

- ◯ `subsums(s, n)`
- ◯ `subsums(s[1:], n)`
- ◯ `subsums(s, n - 1)`
- ◯ `subsums(s[1:], n - 1)`

(c) (6.0 points)

Implement `semisums`, which takes a positive integer `n`. It returns a list of all lists of unique (no repeats) proper divisors of `n` that sum to `n`. You may call `semiperfect` and `subsums`.

```
def semisums(n):
    """Return a list of all lists (with no repeats) of
    proper divisors of n that sum to n.

    >>> semisums(22)  # 22 is not semiperfect, so there are no sums.
    []
    >>> semisums(30)  # 30 is semiperfect. It has three different sums.
    [[1, 3, 5, 6, 15], [2, 3, 10, 15], [5, 10, 15]]
    """
    return list( _______ ([k for k in range(1, n) if _______ ], _______ ))
                  (i)                                    (j)        (k)
```

i. (1.0 pt) Fill in blank (i).

- ◯ `map`
- ◯ `filter`
- ◯ `sorted`
- ◯ `subsums`
- ◯ `semiperfect`

ii. (1.0 pt) Fill in blank (j).

- ◯ `semiperfect(n)`
- ◯ `semiperfect(k)`
- ◯ `semiperfect(n) or semiperfect(k)`
- ◯ `n % k == 0`

iii. (1.0 pt) Fill in blank (k).

iv. (3.0 pt) Fill in blank (l) of `primitive_semiperfect`, a function that takes a positive integer `n`. It returns `True` if `n` is semiperfect and no smaller semiperfect number is a divisor of `n`. Your answer should have two expressions separated by a comma.

```
def primitive_semiperfect(n):
    """Return whether n is semiperfect and has no semiperfect proper divisors.

    >>> [k for k in range(1, 300) if primitive_semiperfect(k)]
    [6, 20, 28, 88, 104, 272]
    """
    return semiperfect(n) and not any(map(semiperfect, filter(_______)))
                                                                 (l)
```

5. (11.0 points) How Long is this Exam?

(a) (5.0 points)

Implement `longer`, which takes two linked lists of numbers `s` and `t`. It returns the longer of the two. If they have the same length, it returns `s`.

A linked list is either a `Link` instance or `Link.empty`. The `Link` class is on page 2 of the midterm 2 study guide.

```
def longer(s, t):
    """Return the longer linked list, s or t. (Same length? return s.)

    >>> longer(Link(2, Link(3)), Link.empty)
    Link(2, Link(3))
    >>> longer(Link(2, Link(3)), Link(4, Link(5)))
    Link(2, Link(3))
    >>> longer(Link(2, Link(3)), Link(4, Link(5, Link(6, Link(7)))))
    Link(4, Link(5, Link(6, Link(7))))
    >>> longer(Link.empty, Link.empty) is Link.empty
    True
    """
    a, b = s, t

    while b is not Link.empty:

        if _______:
             (a)

            return _______
                      (b)


        _______
          (c)

    return _______
             (d)
```

i. (2.0 pt) Fill in blank (a).

ii. (1.0 pt) Fill in blank (b).

- ◯ `a`
- ◯ `b`
- ◯ `s`
- ◯ `t`
- ◯ `longer(a, b)`
- ◯ `longer(s, t)`

iii. (1.0 pt) Fill in blank (c).

- ◯ `a = a.rest`
- ◯ `a = s.rest`
- ◯ `b = b.rest`
- ◯ `b = t.rest`
- ◯ `a, b = a.rest, b.rest`
- ◯ `a, b = s.rest, t.rest`

iv. (1.0 pt) Fill in blank (d).

- ◯ `a`
- ◯ `b`
- ◯ `s`
- ◯ `t`
- ◯ `longer(a, b)`
- ◯ `longer(s, t)`

(b) (6.0 points)

Implement `longest`, which takes a linked list of postive integers `s` and a positive integer `n`. It returns the longest sublist of `s` with elements that sum to a number less than or equal to `n`. **Do not mutate `s`.** In case of a tie, return any of the longest sublists whose sum is `n` or less. Assume `longer` is implemented correctly.

A sublist of a linked list `s` is a linked list with some (or none or all) of the elements of `s` in order.

Assume the sum of the elements of `Link.empty` is 0. `Link.empty` is a sublist of any linked list.

```
def longest(s, n):
    """Return the longest sublist of s that sums to n or less.

    >>> longest(Link(5, Link(1, Link(3, Link(4, Link(2, Link(7)))))), 7)
    Link(1, Link(3, Link(2)))
    >>> longest(Link(5, Link(1, Link(3, Link(4, Link(2, Link(7)))))), 70)
    Link(5, Link(1, Link(3, Link(4, Link(2, Link(7))))))
    >>> longest(Link(3, Link(4, Link(5))), 2) is Link.empty
    True
    """
    if s is Link.empty:
        return s

    t = _______
          (e)

    if _______:
         (f)

        return longer( _______ , t)
                            (g)
    else:
        return t
```

i. (1.0 pt) Fill in blank (e).

◯ `longest(s, n)`

◯ `longest(s, n - s.first)`

◯ `longest(s.rest, n)`

◯ `longest(s.rest, n - s.first)`

ii. (2.0 pt) Fill in blank (f).

iii. (3.0 pt) Fill in blank (g).

6. (0.0 points) Nice Path!

This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

(a) (0.0 pt) Fill in the blank of `max_path`, which takes a Tree of integers `t` and a function `g`. It returns a list `s` containing the labels along the path from the root of `t` to a leaf for which `g(s)` is as large or larger than for any other path. In case of a tie, return any path with a maximum `g(s)` value.

```
def climb(t, f):
    if t.is_leaf():
        return [t.label]
    return [t.label] + climb(max(t.branches, key=f), f)

def max_path(t, g):
    """Return the path s from the root of t to a leaf for which g(s) is largest.

    >>> scare = Tree(0, [Tree(4), Tree(5, [Tree(10)]), Tree(2)])
    >>> crow = Tree(4, [Tree(5), Tree(9, [scare, Tree(7, [Tree(6)])]), Tree(8)])
    >>> max_path(crow, lambda p: -p[-1])            # The path to the smallest leaf
    [4, 9, 0, 2]
    >>> max_path(crow, len)                         # The longest path
    [4, 9, 0, 5, 10]
    >>> max_path(crow, lambda p: -abs(p[0]-p[-1]))  # To the leaf closest in value to the root
    [4, 9, 0, 4]
    """

    x = [t.label]  # You can use x instead of [t.label] to shorten your answer!

    return climb(t, lambda b: _______ )
```

No more questions.

CS 61A Fall 2025

Structure and Interpretation of Computer Programs

MIDTERM 1

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

You can complete these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (6.0 points) What Would Python Do(ot)?

Assume the code below has been executed, and no errors occurred.

```
about = 6-7
about, face = 6, about * 7

def make_something(f):
    if f(about):
        f = lambda k: about
    def something(about):
        return print(f(about) or print(about))
    return something

f = lambda x: 10 * x
```

Example Question: What would be displayed by evaluating `print(5)`? **Answer**: 5

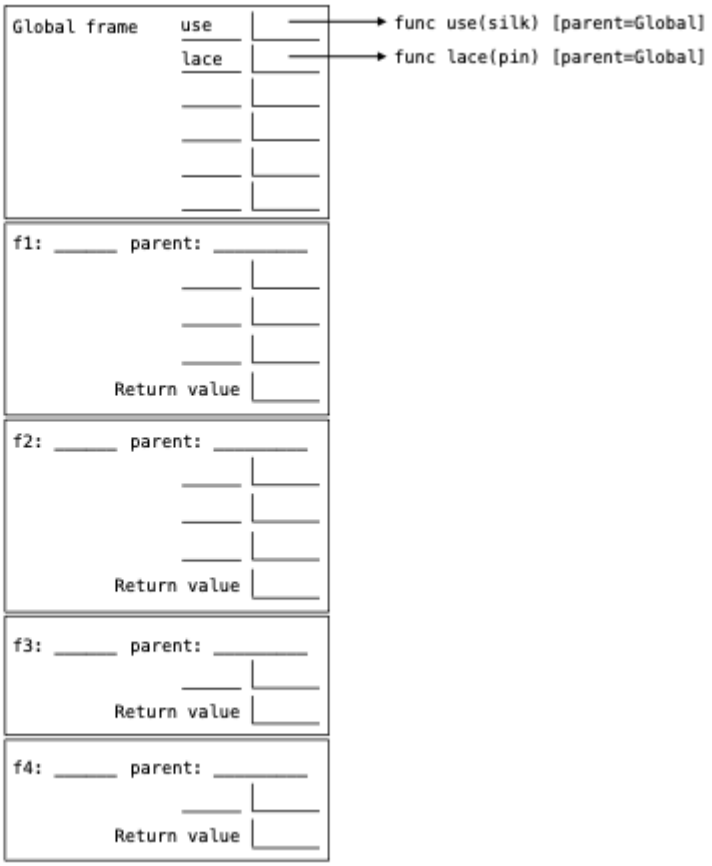
(a) (2.0 pt) What would be displayed by evaluating `print(print(face))`

(b) (4.0 pt) What would be displayed by evaluating `make_something(print)(8)`

2. (5.0 points) Silksong

Complete the environment diagram below to answer the questons about the calls to `print`. Only the questions will be scored, not the diagram. Each call to `print` is evaluated once, and there are no errors caused by running this code.

```
 1: def use(silk):
 2:     a = 2
 3:     print(lace(silk), a)  # (a)
 4:
 5: def lace(pin):
 6:     a = print(pin())      # (b)
 7:     return a
 8:
 9: a = 1
10: thread = lambda x: a + x
11: a = 3
12: needle = lambda: thread(a * 10)
13:
14: use(needle)
15: print(a)                  # (c)
```



(a) (2.0 pt) What is displayed by the call to `print` on **line 3**?

(b) (2.0 pt) What is displayed by the call to `print` on **line 6**?

- ◯ 21
- ◯ 22
- ◯ 23
- ◯ 31
- ◯ 32
- ◯ 33

(c) (1.0 pt) What is displayed by the call to `print` on **line 15**?

◯ 2

◯ 3

◯ None

3. (8.0 points) Saja Boys

Definition. A positive integer is *patterned* if every odd digit has a larger even digit somewhere before it. For example, 4123827 is patterned because the odd digit 7 has 8 (which is even and larger than 7) before it, and the 3 and 1 both have 4 before them.

Implement `patterned`, which takes a positive integer `n`. It returns `True` if `n` is patterned and `False` otherwise. Your implementation should iterate through the digits of `n` just once. The name `odd` in the implementation should always be assigned to an integer from 0 to 9 (inclusive).

```
def patterned(n):
    """Return whether every odd digit of n has a larger even digit somewhere before it.

    >>> patterned(4123827)  # 8 is before 7; 4 is before 1 and 3
    True
    >>> patterned(4412123384137)
    True
    >>> patterned(2468)     # No odd digits
    True

    >>> patterned(1)        # No even digits
    False
    >>> patterned(8192)     # 9 does not have a larger even digit before it (or anywhere)
    False
    >>> patterned(238)      # 3 does not have a larger even digit before it (8 comes after)
    False
    >>> patterned(3888)     # 3 does not have a larger even digit before it (8 comes after)
    False
    >>> patterned(4321587)  # 5 does not have a larger even digit before it (8 comes after)
    False
    """

    odd = 0    # Hint: use the name odd to keep track of a digit you'll need later

    while n:

        n, last = n // 10, n % 10

        if last % 2 == 1:

            -------
              (a)

        elif _______:
               (b)

            -------
              (c)

    return _______
             (d)
```

(a) **(2.0 pt)** Fill in blank (a).

- ◯ `return False`
- ◯ `return max(n) > last`
- ◯ `return n % 10 > last`
- ◯ `return n % 2 == 0 and n % 10 > last`
- ◯ `odd = 0`
- ◯ `odd = last`
- ◯ `odd = n % 10`
- ◯ `odd = max(n % 10, last)`
- ◯ `odd = max(odd, last)`

(b) **(2.0 pt)** Fill in blank (b). You may not use `str` or `len` or `[` or `]`

(c) **(2.0 pt)** Fill in blank (c).

- ◯ `return False`
- ◯ `return max(n) > last`
- ◯ `return n % 10 > last`
- ◯ `return n % 2 == 0 and n % 10 > last`
- ◯ `odd = 0`
- ◯ `odd = last`
- ◯ `odd = n % 10`
- ◯ `odd = max(n % 10, last)`
- ◯ `odd = max(odd, last)`

(d) **(2.0 pt)** Fill in blank (d).

- ◯ `True`
- ◯ `False`
- ◯ `odd == 0`
- ◯ `odd > 0`
- ◯ `last == 0`
- ◯ `last > 0`
- ◯ `last % 2 == 0`
- ◯ `last % 2 == 1`

4. (16.0 points) We're Going Up, Up, Up

Definitions. An *up-sequence* is a sequence of positive integers in which each term is larger than the last. The *next function* `f` for an up-sequence takes a non-negative integer `t` and encodes the sequence as follows: If `t` is any term of the sequence except the last one, then `f(t)` returns the next term in the sequence after `t`. If `t` is the last term of the sequence or not a term in the sequence, `f(t)` returns 0. Finally, when `t` is 0, then `f(0)` returns the first term of the sequence. For example, `fifty_evens` below is a next function for the sequence 2, 4, 6, 8, . . . , 98, 100.

```
def fifty_evens(t):
    "The next function for the finite sequence of the first 50 positive even numbers."
    if t % 2 == 0 and t < 100:
        return t + 2
    return 0
```

(a) (6.0 points)

Implement `sum_sequence`, which takes a next function `f` for a finite up-sequence. It returns the sum of the terms of that sequence.

```
def sum_sequence(f):
    """Return the sum of terms in a finite sequence.

    >>> sum_sequence(fifty_evens)    # 2 + 4 + 6 + 8 + ... + 98 + 100 = 2550
    2550
    """
    t = _______
          (a)

    total = 0
    while t:
        t, total = _______ , _______
                     (b)       (c)
    return total
```

i. (2.0 pt) Fill in blank (a).

ii. (2.0 pt) Fill in blank (b).

iii. (2.0 pt) Fill in blank (c).

- ◯ `total + 1`
- ◯ `total + t`
- ◯ `total + f`
- ◯ `total + fifty_evens`
- ◯ `total + f(t)`
- ◯ `total + fifty_evens(t)`

(b) (6.0 points)

Here is the next function for the **infinite** up-sequence of positive perfect squares: 1, 4, 9, 16, 25, ...

```
def next_square(t):
    """Compute the next perfect square.

    >>> next_square(0)
    1
    >>> next_square(16)
    25
    >>> next_square(17)  # Not a perfect square
    0
    """
    sqrt_t = t ** 0.5                    # The square root of t
    if sqrt_t == round(sqrt_t):          # Make sure t was a perfect square
        return (round(sqrt_t) + 1) ** 2  # Return the next perfect square
    return 0
```

Implement `cap`, which takes a next function `f` for an **infinite** up-sequence and a positive integer `n`. It returns a next function for the finite up-sequence containing the terms of `f` that are less than or equal to `n`. If `n` is a term of the up-sequence for `f`, then it is included in the result's sequence.

```
def cap(f, n):
    """Return the next function for the up-sequence for f up to (and possibly including) n.

    >>> squares_up_to_25 = cap(next_square, 30)  # 30 is not in the next_square sequence
    >>> squares_up_to_25(4)
    9
    >>> squares_up_to_25(16)
    25
    >>> squares_up_to_25(25)
    0
    >>> squares_up_to_25(17)                     # 17 is not in the next_square sequence
    0
    >>> cap(next_square, 81)(64)                 # 81 is in the next_square sequence
    81
    """
    def capped(t):

        if _______:
             (d)


            _______
              (e)

        else:

            return _______
                     (f)

    return capped
```

i. (2.0 pt) Fill in blank (d).

ii. (2.0 pt) Fill in blank (e).

- ◯ `t = 0`
- ◯ `n += 1`
- ◯ `n -= 1`
- ◯ `t += 1`
- ◯ `t -= 1`
- ◯ `t = f(t)`
- ◯ `t = next_square(t)`
- ◯ `return n`
- ◯ `return t`
- ◯ `return 0`

iii. (2.0 pt) Fill in blank (f).

- ◯ `n`
- ◯ `t`
- ◯ `f`
- ◯ `f(t)`
- ◯ `next_square`
- ◯ `next_square(t)`

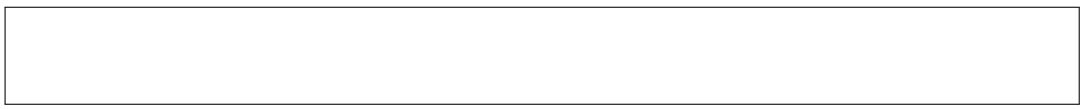
(c) (4.0 points)

Definition. The *previous function* `g` for an up-sequence takes a non-negative integer `t`. If `t` is any other term than the first, `g(t)` is the previous term. If `t` is the first term or not a term at all, `g(t)` returns 0. `g(0)` is the last term.

Implement `reverse`, which takes a next function `f` for a finite up-sequence. It returns the previous function for that same sequence. You may call `max_term`, which is described below, but you don't need to implement `max_term`.

```
def max_term(f):
    """Returns the largest term in the finite up-sequence for next function f.

    >>> max_term(cap(next_square, 20))  # 16 is the largest square less than or equal to 20.
    16
    """
    # Implementation omitted, but you can assume that the function is implemented correctly.

def reverse(f):
    """Return the previous function for the up-sequence encoded by next function f.

    >>> rev_squares = reverse(cap(next_square, 30))  # Goes in reverse through 1, 4, 9, 16, 25
    >>> print(rev_squares(0), rev_squares(25), rev_squares(16), rev_squares(9), rev_squares(4))
    25 16 9 4 1
    >>> rev_squares(1)   # 1 is the first term
    0
    >>> rev_squares(10)  # 10 is not in the sequence
    0
    """

    def previous(t):

        if t == 0:

            return _______
                     (g)
        x = t - 1

        while x > 0 and _______:
                           (h)

            x = x - 1

        return x

    return previous
```

i. (2.0 pt) Fill in blank (g).



ii. (2.0 pt) Fill in blank (h).

(d) (0.0 points)

This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Implement `sum_below`, which takes a next function `f` for an infinite up-sequence and a positive integer `n` that is a term of the sequence. It returns the sum of the terms of the sequence for `f` that are less than (but not including) `n`.

You may **not** use `cap` or `max_term` or `reverse`.

You may **not** use `if` or `else` or `[` or `]`.

```
def sum_below(f, n):
    """Return the sum of the terms of the sequence for f that are below n,
    where n is a term in the sequence for f.

    >>> sum_below(next_square, 25)  # 1 + 4 + 9 + 16
    30
    """
    assert f(n), 'n is not a term of the up-sequence for f'
    return sum_sequence(_______)
                        (i)
```

i. (0.0 pt) Fill in blank (i).

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Spring 2016

TEST 1

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is open book, open notes, closed computer, closed calculator. The official CS 61A midterm 1 study guide will be provided.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
BearFacts email (`@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
I pledge my honor that during this examination I have neither given nor received assistance. **(please sign)**	

1. (12 points) Evaluate This!

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "`Error`". If an expression yields (or prints) a function, write "`<Function>`". No answer requires more than 3 lines. (It's possible that all of them require even fewer.) The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`, plus all values passed to `print`.

(a) (10 pt) Assume that `python3` has executed the statements on the left:

```
y = 7

def b(x):
  return lambda y: x(y)

def c(x):
  return 3

w = b(c)

def c(x):
  return x
```

Expression	Interactive Output
`pow(2, 3)`	8
`print(4, 5) + 1`	4 5 Error
`(3 and abs)(-1)`	
`print(3) or 1/0`	
`print`	
`([1, 2, 3] if y // (y+1) else [4, 5])[1]`	
`w(5)`	

(b) (2 pt) Assume that `python3` has executed the following statements:

```
def d(S):
    def f(k):
        return k < len(S) and (S[k] == S[0] or f(k+1))
    if len(S) == 0:
        return False
    else:
        return f(1) or d(S[1:])
```

Expression	Interactive Output
`print(d([1, 2, 3]), d([0, 1, 2, 1, 0]))`	

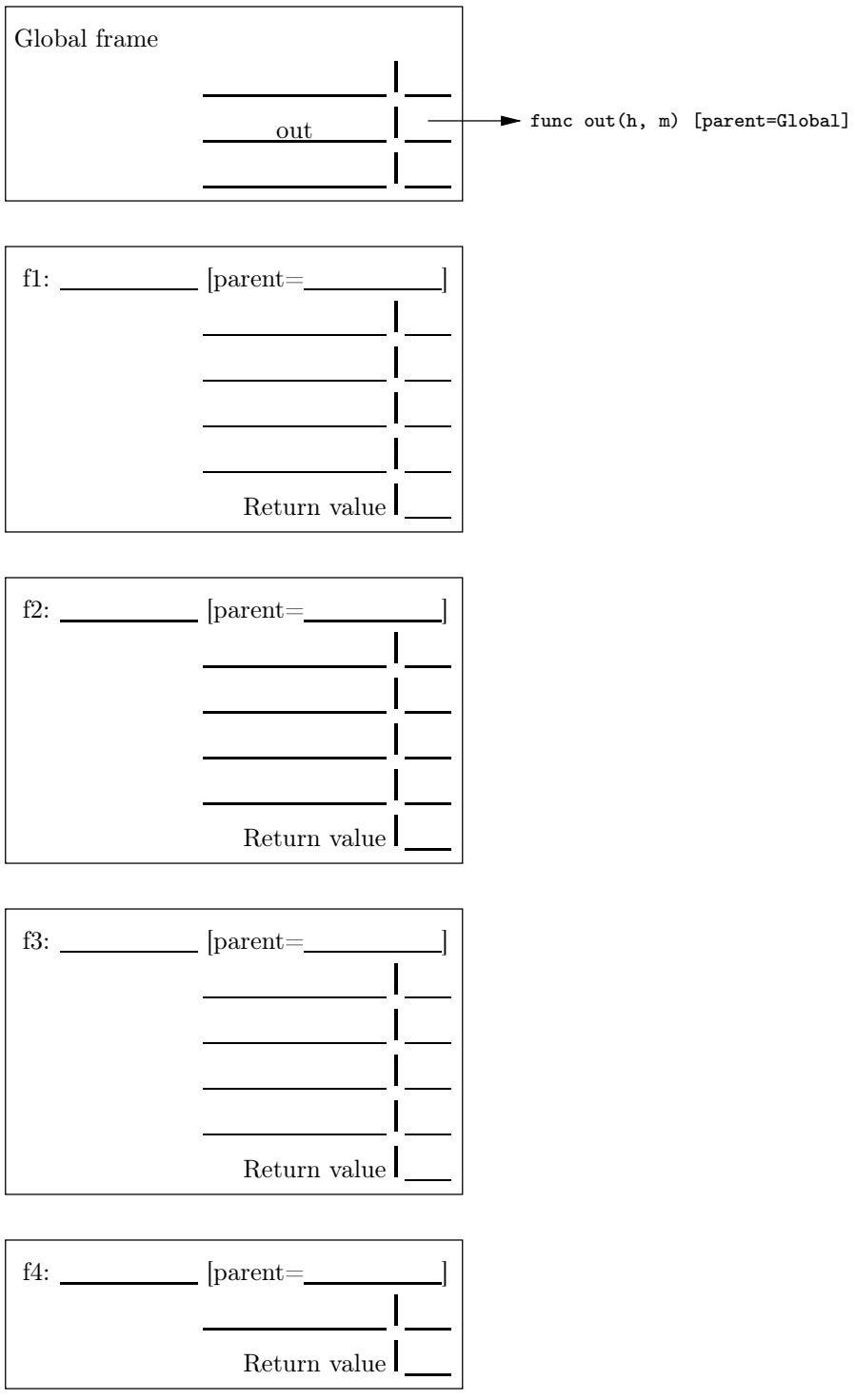
2. (12 points) Environmental Policy

(a) (6 pt) Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*

A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  y = 3
2  def out(h, m):
3      y = 5 * m
4      def inner():
5          return y
6      if m == 0:
7          return h
8      else:
9          return out(inner, m-1)
10 v = out(None, 1)()
```

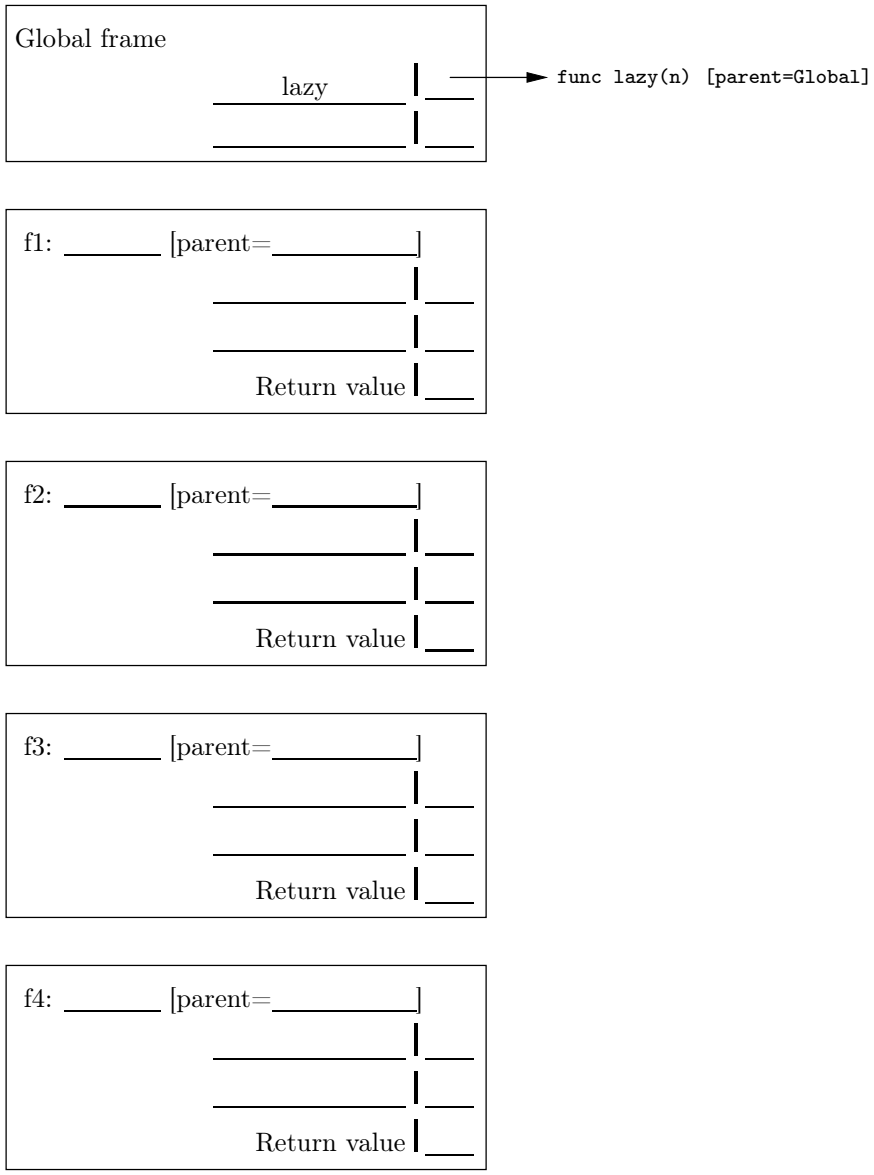


(b) (6 pt) Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.* The `<line ...>` annotation in a lambda value gives the line in the Python source of a lambda expression.

A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Add all missing parents of function values.
- Show the return value for each local frame.

```
1 def lazy(n):
2     return lambda k: (n if k==0 else lazy(n+1))
3 v = lazy(4)(1)(0)
```



3. (1 points) Extra Who described Italy (in translation) as "the land where lemon blossoms blow, / And through dark leaves the golden oranges glow."?

4. (4 points) Going Around in Cycles

Consider a list of integers, `L`, in which each integer is a value in 0 to `len(L)-1` (inclusive). That is, each item in the list could also be used as a valid index into the list. We'll say that `L` has a *cycle of length k* iff for some sequence of index values $i_1, i_2, \ldots, i_k$, we have `L[`i_1`] = `i_2`, L[`i_2`] = `i_3`, ..., L[`i_k`] = `i_1. For example, the list `L = [3, 1, 4, 5, 0, 2]` has a cycle of length 1: $1 \to 1$, and a cycle of length 5: $0 \to 3 \to 5 \to 2 \to 4 \to 0$. For the purposes of this problem, we'll count something as a cycle of length k only if the first k indices in it contain no repetitions (so `L` in the example does not contain a cycle of length 2 even though $1 \to 1 \to 1$.

Write a function `has_cycle(L, k)` that returns a true value iff `L` has a cycle of length k by filling in the template below.

```
def has_cycle(L, k):
    """True iff L has a cycle of length K>=1.  Assumes 0 <= L[i] < len(L)
    for 0 <= i < len(L).
    """
    def cycle_at(s):
        """L has a cycle of length K starting at S."""

        ______________________________________________

        n = 1

        while ________________________________________:


            if ________________________________________:

                return False


            ______________________________________________

            n += 1


        return ________________________________________

    for j in range(len(L)):


        if ________________________________________:


            return ____________________________________


    return ________________________________________
```

5. (4 points) Your Father's Parentheses

Suppose we have a sequence of quantities that we want to multiply together, but can only multiply two at a time. We can express the various ways of doing so by counting the number of different ways to parenthesize the sequence. For example, here are the possibilities for products of 1, 2, 3, 4 and 5 elements:

Product	a	ab	abc	abcd	abcde		
Count	1	1	2	5	14		
Parenthesizations	a	ab	a(bc) (ab)c	a(b(cd)) a((bc)d) (ab)(cd) (a(bc))d ((ab)c)d	a(b(c(de))) a(b((cd)e)) a((bc)(de)) a((b(cd))e) a(((bc)d)e)	(ab)(c(de)) (ab)((cd)e) (a(bc))(de) ((ab)c)(de) (a(b(cd)))e	(a((bc)d))e ((ab)(cd))e ((a(bc))d)e (((ab)c)d)e

Assume, as in the table above, that we don't want to reorder elements.

Define a function `count_groupings` that takes a positive integer n and returns the number of ways of parenthesizing the product of n numbers. (You might not need to use all lines.)

```
def count_groupings(n):
    """For N >= 1, the number of distinct parenthesizations
    of a product of N items.
    >>> count_groupings(1)
    1
    >>> count_groupings(2)
    1
    >>> count_groupings(3)
    2
    >>> count_groupings(4)
    5
    >>> count_groupings(5)
    14
    """
    if n == 1:

        return ____________________________________________________________


    ____________________________________________________________________________


    i = ________________________________________________________________________


    while ____________________________________________________:


        ____________________________________________________________________

        i += 1

    return ____________________________________________________________________
```

6. (8 points) Amazing

In lecture, we did some problems involving finding one's way through a maze, representing the maze as a predicate (boolean function)—the parameter `blocked`. The idea was that `blocked(x, y)` iff the grid square at row `y` and column `x` was blocked. Let's consider a different representation.

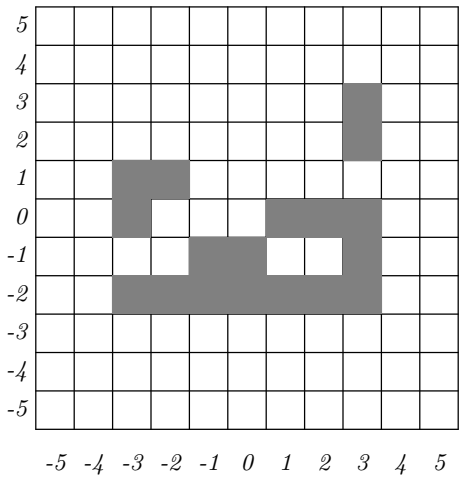
Again, mazes will be represented by functions, but with a different specification. A maze function, say `M`, in this formulation describes the state of the maze from the point of view of a maze runner. `M` accepts a single argument, `direction`, which describes the action the runner tries to take: either the string "south" or "west", indicating that the runner tries to take one step south or one step west, respectively. `M` then returns either:

- Another maze function that represents the state of the runner after taking the indicated step. It again accepts a direction and returns one of these three things;
- The string "exit", signifying that the move causes the runner to successfully reach the exit; or
- The string "dead end", which indicates that the runner was unable to take the requested action because of being blocked by a wall.

(a) (4 pt) Define a function `pred_maze` that returns a maze function (as described above).
If `M = pred_maze`(x, y, P, e), then `M` represents a runner in a maze in which

- The runner is at coordinates (x, y).
- Squares (a, b) for which $a \leq e$ are unblocked and are exit squares.
- A square at (a, b) is otherwise open iff $P(a, b)$.

For example, suppose that `Q` is a function such that `Q(a,b)` is true on all the white squares in the following grid (where, just to be different, the lower-left corner represents position (-5, -5)):



Then we'd have

```
>>> M0 = pred_maze(0, 1, Q, -4)
>>> M1 = M0('west')
>>> M1
<function ...>
>>> M1('west')
'dead end'
>>> M1('south')
<function ...>
>>> M2 = pred_maze(-3, -1, Q, -4)
>>> M2('west')
'exit'
```

Fill in your solution on next page.

```
def pred_maze(x0, y0, open, exit):
    """Return a maze in which the runner is at (X0, Y0), every square
    (a, b) where a <= EXIT is an exit, and otherwise a square (a, b)
    is open iff OPEN(a, b).  It is assumed that (X0, Y0) is open."""

    def maze(____________________________):

        x, y = (x0, y0 - 1) if dir == "south" else (x0 - 1, y0)


        if _____________________________________________:


            return ______________________________________


        elif ____________________________________________:


            return ______________________________________

        else:

            return ______________________________________


    return maze
```

(b) (4 pt) Define a function `path_out` that takes a maze and returns a string describing a path to an exit, or returns `None` iff there is no such path. A call `path_out(M)` will return a string such as

```
'south west south west west'
```

(as in the doctest below) to indicate that the necessary path takes one step south, then a step west, then south, and then two steps west. (Don't worry about extra spaces in the returned string.) When multiple paths exist, return any one. Using the previous value of function `Q`, for example, we'd see:

```
>>> M = pred_maze(-1, 1, Q, -4)
>>> path_out(M)
'south west south west west'
>>> M = pred_maze(2, -1, Q, -4)
>>> path_out(M)   # Returns None
```

FYI: The '+' operator on strings is string concatenation ('+=') also works.)

```
def path_out(M):
    """Given a maze function M in which the runner is not yet out
    of the maze, returns a string of the form "D1 D2 ...", where each
    Di is either south or west, indicating a path from M to an exit,
    or None iff there is no such path."""

    for dir in ["south", "west"]:


        next = ___________________________________________


        if ________________________________________________:


            return ____________________________________


        elif ______________________________________________:


            rest_of_path = _______________________________


            if ________________________________________:


                return ___________________________________

    return None
```

CS 61A Structure and Interpretation of Computer Programs
Spring 2017

TEST 1

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is open book, open notes, closed computer, closed calculator.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
Room in which you are taking exam	
Seat number in the exam room	
I pledge my honor that during this examination I have neither given nor received assistance. **(please sign)**	

Reference Material.

```
# Linked Lists (implementations not shown)

empty = ...  # The empty list

def link(first, rest):
    """A linked list whose first element is FIRST and the linked list
    REST is the rest of the list."""

def first(lnklst):
    """The first item in linked list LNKLST."""

def rest(lnklst):
    """The list following the first item in linked list LNKLST."""

def isempty(lnklst):
    """True if LNKLST is empty."""

def print_link(lnklst):
    """Prints the linked list LNKLST in the format (v0, v1, ...)."""
```

1. (12 points) Evaluate These!

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "`Error`". If an expression yields (or prints) a function, write "`<Function>`". No answer requires more than 3 lines. (It's possible that all of them require even fewer.) The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`, plus all values passed to `print`.

Assume that `python3` has executed the statements on the left:

```
def w(L):
  if len(L) == 0:
    return L
  elif L[0] in L[1:]:
    return w(L[1:])
  else:
    return [L[0]] + w(L[1:])

def reduce(f, L, init):
  if len(L) == 0:
    return init
  else:
    return reduce(f, L[1:],
                  f(init, L[0]))
```

Expression	Interactive Output
`pow(2, 3)`	8
`print(4, 5) + 1`	4 5 Error
`1 + (4 and 6) + (5 or 0) + (0 and 8)`	

`f = lambda x: x`
`g = lambda y: f(g)`
`g(2)(2)`	

`0 and print(2)`	
`range(1,20)[-2]`	
`w([1, 2, 3, 1, 8, 2])`	

`f = lambda x: lambda y: \`
`  lambda z: x+y+z`
`reduce(lambda g, x: g(x),`
`  [1, 2, 3], f)`	

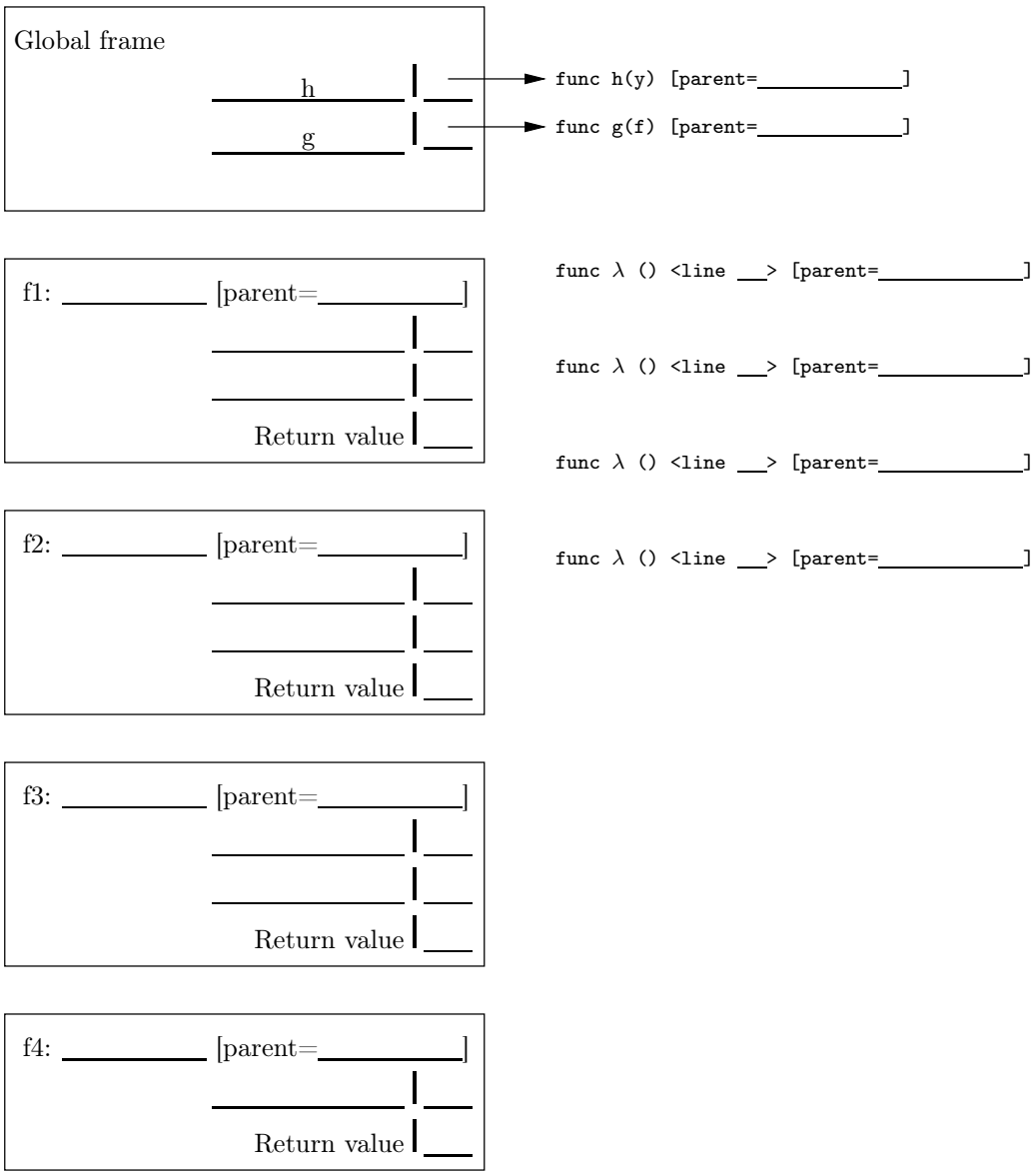
2. (12 points) Environmental Policy

(a) (6 pt) Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces, frames, or blank lambda function values.*

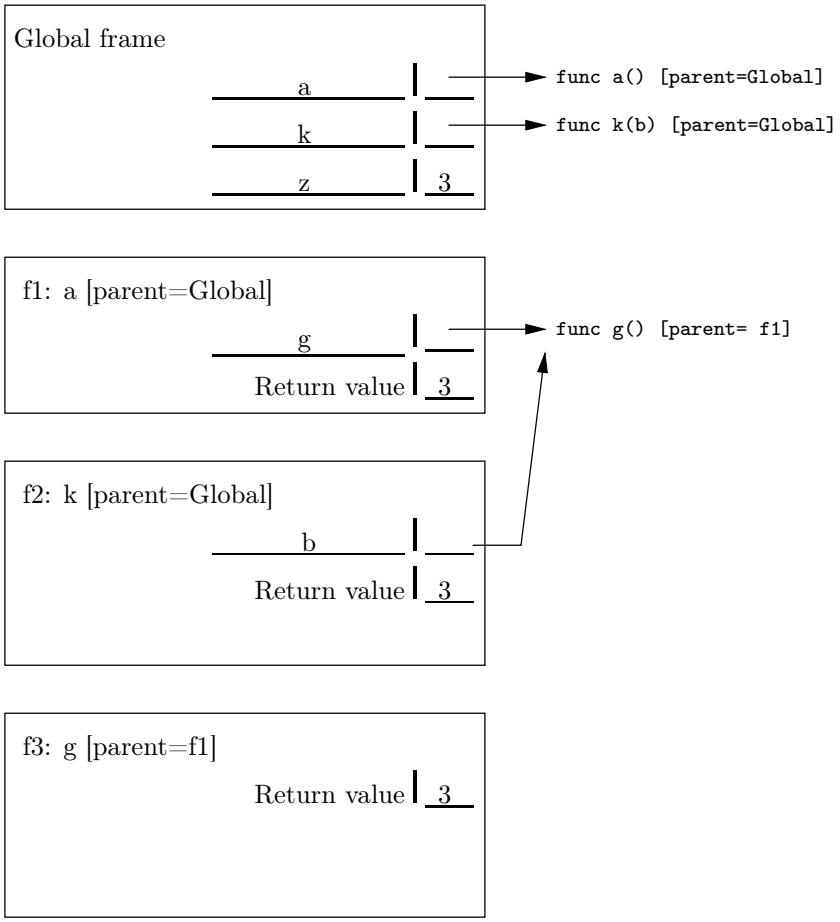
A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1 def h(y):
2     return g(lambda: y)
3 def g(f):
4     return f()()
5 h(lambda: 3)
```



(b) (6 pt) The environment diagram below corresponds to the execution of a certain program. The frames shown were created in top-to-bottom order, and at one point they were all simultaneously active (their functions had not returned). The diagram shows the situation right after all the functions have returned. Write a Python program whose execution corresponds to this environment diagram. Many answers are possible, but as a guideline, fewer than 10 lines are really needed. In any case, your answer must not create extra frames or additional functions.



Write solution here.

3. (4 points) Sequence Checking

Fill in the following function so that it fulfills its comment.

```
def make_checker(relation, start, end):
    """Assumes that START and END are integers and RELATION is a two-argument
    function that returns true/false values.  Returns a function that, given
    a function f as input, returns True if RELATION returns true for
    all adjacent values in the sequence f(START), f(START+1), ... f(END-1).
    For example, eq_chk, below, checks that the values returned by
    its argument function for values 0-4 are all equal, while up_chk checks
    that the values returned by the argument function are in strictly
    increasing order.

    >>> eq_chk = make_checker(lambda x, y: x == y, 0, 5) # Check all equal
    >>> eq_chk(lambda x: 3)
    True
    >>> eq_chk(lambda x: x)
    False
    >>> up_chk = make_checker(lambda x, y: x < y, 0, 5) # Check increasing
    >>> up_chk(lambda x: x)
    True
    >>> up_chk(lambda x: 3)
    False
    """
```

4. (1 points) Extra

In the formula $a = 0.4 + 0.3 \cdot 2^m$ ($m = -\inf, 0, 1, 2, \ldots$), what does a refer to?

5. (4 points) Insert

Fill in the following function to fulfill its comment. The linked-list interface that you should use is given on page **??**. **Warning:** this problem deals with linked lists, *NOT* Python lists or tuples. You cannot use `+`, `len()`, indexing (`L[k]`), or list construction (`[...]`).

```
def link_insert(lnklst, value, before):
    """Return a linked list identical to LNKLST, but with VALUE inserted just
    before the first occurrence of BEFORE in the list, if any.  The returned
    list is identical to LNKLST if BEFORE does not occur in LNKLST.
    The operation is non-destructive.
    >>> L = link(2, link(3, link(7, link(1))))
    >>> print_link(L)
    (2, 3, 7, 1)
    >>> Q = link_insert(L, 19, 7)
    >>> print_link(Q)
    (2, 3, 19, 7, 1)
    >>> print_link(link_insert(L, 19, 20))
    (2, 3, 7, 1)
    """
    if ____________________________________________________________:

        return ____________________________________________________

    elif ____________________________________________________________:

        return ____________________________________________________

    else:

        return ____________________________________________________
```

6. (4 points) Longest Nondecreasing Suffix

Consider a function `up_suffix` that is supposed to return the longest suffix of a Python list of integers such that the suffix consists of nondecreasing values. For example, when applied to

```
[1, 2, 3, 4, 5, 1, 3, 3, 4]
```

it should yield `[1, 3, 3, 4]`.

Fill in the following function to do this:

```
def up_suffix(L):
    """Returns the longest non-descending suffix of Python list L. """

    def longest_suffix_start(L):
        """The index in L of the beginning of the longest nondecreasing suffix
        of L. For the empty list, returns 0.
        >>> longest_suffix_start([1, 2, 3, 4, 5, 1, 3, 3, 4])
        5
        >>> longest_suffix_start([2, 4, 6, 8, 10])
        0
        """


        k = ____________________________________________________________


        while ____________________________________________________________:


            ____________________________________________________________


        return ____________________________________________________________


    return L[ ____________________________________________________ : ]
```

7. (4 points) Post's Problem

Consider two Python lists of strings, where the two lists have equal length, N:

```
A = [ "a", "ab", "bba"]
B = [ "baa", "aa", "bb" ]
```

Is there a sequence of integers—i_1, i_2, ... i_m—where $m > 0$ and $0 \leq i_k < N$ for all k, such that

$$A[i_1] + A[i_2] + \cdots + A[i_m] = B[i_1] + B[i_2] + \cdots + B[i_m]?$$

This is called the *Post Correspondence Problem*. For this `A` and `B`, the answer is yes for $m = 4$: (2, 1, 2, 0), because

```
A[2] + A[1] + A[2] + A[0] == "bba" + "ab" + "bba" + "a"  == "bbaabbbaa"
B[2] + B[1] + B[2] + B[0] == "bb" + "aa" + "bb" + "baa" == "bbaabbbaa"
```

On the other hand, the answer is no if we limit m to 3 (so we cannot add more than three strings), or if we shorten the lists by removing `A[0]` and `B[0]`. Fill in the following function to determine whether there is a solution.

```
def correspond(A, B, M):
    """Assuming A and B are lists of strings with len(A) == len(B), and M
    is an integer, returns true iff there is a sequence of indices into A
    and B, (i1, i2, ..., im), where 1 <= m <= M, such that
        A[i1] + A[i2] + ... + A[im] == B[i1] + B[i2] + ... + B[im].
    """
    N = len(A)

    def can_correspond(sa, sb, M):
        """Return true iff there is some sequence of indices into A and
        B, (i1, i2, ..., im), where 1 <= m <= M, such that
        SA + A[i1] + A[i2] + ... + A[im] == SB + B[i1] + B[i2] + ... + B[im].
        Assumes M is a non-negative integer.
        """

        if ____________________________________________________________:
            return False
        else:

            for i in ___________________________________________________:

                ta = ___________________________________________________

                tb = ___________________________________________________

                if ta == tb:

                    return True

                elif can_correspond(__________, ___________, ___________):

                    return True
            return False

    return can_correspond("", "", M)
```

CS 61A Structure and Interpretation of Computer Programs
Spring 2017

TEST 2 (REVISED)

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is open book, open notes, closed computer, closed calculator.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
Room in which you are taking exam	
Seat number in the exam room	
I pledge my honor that during this examination I have neither given nor received assistance. **(please sign)**	

Reference Material.

```
# Linked Lists

class Link:
    """A linked list cell.
    >>> L = Link(0, Link(1))
    >>> L.first
`   0
    >>> L.rest
    Link(1)
    >>> L.first = 2
    >>> L
    Link(2, Link(1))
    >>> L.rest = Link.empty
    >>> L
    Link(2)
    """
    empty = ()

    def __init__(self, first, rest=empty):
        assert rest is Link.empty or isinstance(rest, Link)
        self.first = first
        self.rest = rest

    def __repr__(self):
        if self.rest is Link.empty:
             return "Link({})".format(self.first)
        else:
             return "Link({}, {})".format(self.first, self.rest)

# Trees

class Tree:
    """A tree node."""

    def __init__(self, label, branches=[]):
        for c in branches:
            assert isinstance(c, Tree)
        self.label = label
        self.branches = branches

    def is_leaf(self):
        return not self.branches
```

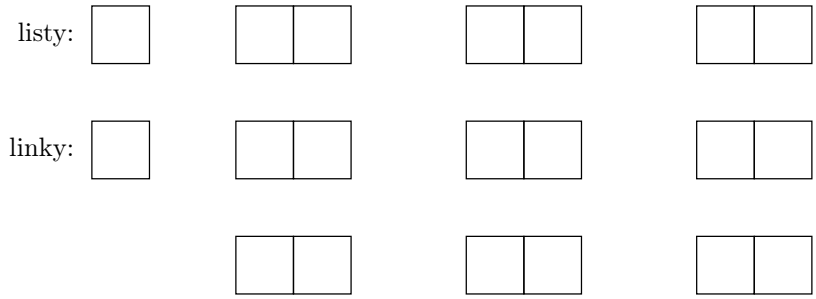
1. (12 points) Pointers

In the following problems, single boxes are variables that contain pointers, and double boxes are `Links` (see the definition of `Link` on page **??**). To show that a box contains a pointer to the empty list, draw the box like this:

In parts (a) and (b), add arrows and values to the object skeletons on the right to show the final state of the program. Not all boxes will be used. (For examples of what kinds of box and pointer diagrams we're looking for, you might look at parts (c) and (d) first.)

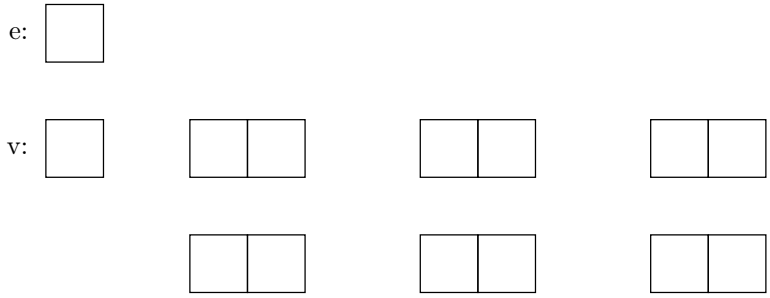
(a) (3 pt)

```
listy = Link(0, Link(1))

def nest(L):
    if L is Link.empty:
        return L
    N = nest(L.rest)
    L.first = Link(L.first, N)
    return L.first

linky = nest(listy)
```

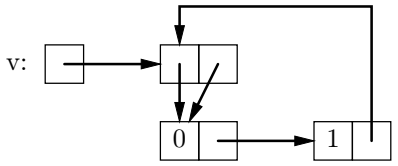


(b) (3 pt)

```
v = Link(0, Link(1, Link(2)))
e = v.rest.rest
e.rest = v.rest
v.rest.rest = v
v.rest = Link.empty
```

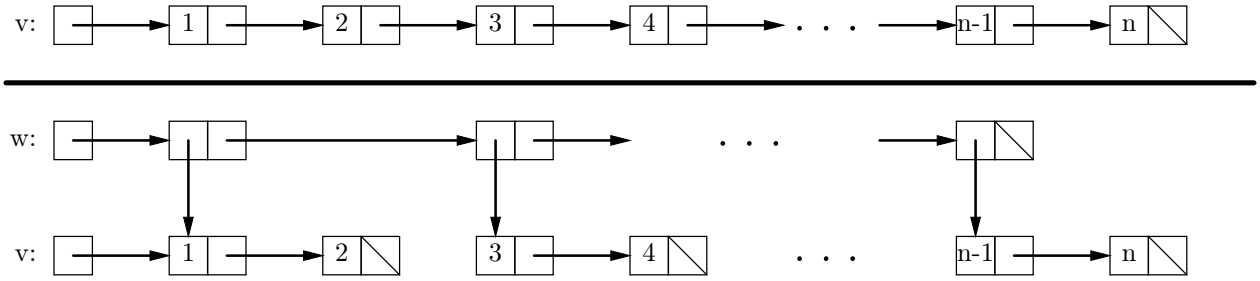


(c) (3 pt) Show Python code that will produce the situation shown in the diagram. (An arrow pointing to a `Link` may be shown as pointing anywhere on the double box for that `Link`.)




```
v = Link(_____________________________________________________________)

v._____________________ = ___________________________________________

v._____________________ = ___________________________________________
```

(d) (3 pt) Show Python code that converts the situation shown above the line into that shown below the line. Assume n is even.




```
def split2(L):
    """Assuming that linked list L has even length, breaks L into
    two-element linked lists, and returns a linked list of those lists."""


    if ____________________________:

           return _________________________
    else:

           result = Link(L, _____________________________________________)

           ______________________________ = Link.empty

           return result

w = split2(v)
```

2. (6 points) Complexity

(a) (1.5 pt) Indicate which of the following assertions are true by circling the letters for those statements. An assertion such as $\Theta(f(n)) \subseteq \Theta(g(n))$ means "any function that is in $\Theta(f(n))$ is also in $\Theta(g(n))$," and $\Theta(f(n)) = \Theta(g(n))$ if and only if $\Theta(f(n)) \subseteq \Theta(g(n))$ and $\Theta(g(n)) \subseteq \Theta(f(n))$.

A. If $f(n) \in \Theta(1)$ and $g(n) \in \Theta(1)$, then $\Theta(|f(n)| + |g(n)|) \in \Theta(1)$.

B. If $\Theta(f(n)) = \Theta(g(n))$, and $g(n) > 0$ everywhere, then $f(n)/g(n) \in \Theta(1)$.

C. $\Theta(x^2) \subseteq \Theta(x^3)$.

D. $\Theta(2^x) \subseteq \Theta(2^x + x^2)$.

E. If $f(n) \in \Theta(1000 \cdot x^3)$, then $f(20) > 800$.

(b) (1.5 pt) Consider the function

```
def num_kinks(L):
    c = 0
    i = 0
    while i < len(L):
        j = i
        while j < len(L):
            while j < len(L):
                if kink(L[i], L[j]):
                    c += 1
                    break
                j += 1
            j += 1
        i += 1
    return c
```

Circle the order of growth that best describes the worst-case execution time (measured by the number of calls to `kink`) of a call to `num_kinks` as a function of N, the length of `L`.

A. $\Theta(\log N)$

B. $\Theta(N)$

C. $\Theta(N^2)$

D. $\Theta(N^3)$

E. $\Theta(2^N)$

(c) (1.5 pt) Consider the following function on `Trees`

```
def count_subtrees(T, p):
    if p(T.label):
        return 1
    else:
        return sum([count_subtrees(child, p) for child in T.branches])
```

Assuming that the maximum number of children of any node is 3, circle the order of growth that best describes the worst-case execution time (measured by the number of calls to `p`) of a call to `count_subtrees` as a function of N, the number of nodes in `T`.

A. $\Theta(\log N)$

B. $\Theta(N)$

C. $\Theta(N^2)$

D. $\Theta(N^3)$

E. $\Theta(3^N)$

(d) (1.5 pt) For the same function as in part (c) above, and again assuming that the maximum number of children of any node is 3, circle the order of growth that best describes the worst-case execution time (measured by the number of calls to `p`) of a call to `count_subtrees` as a function of H, the height of `T`.

A. $\Theta(\log H)$

B. $\Theta(H)$

C. $\Theta(H^2)$

D. $\Theta(H^3)$

E. $\Theta(3^H)$.

3. (1 points) From the Sum of Human Knowledge

This Renaissance composer, famous for his harmonically innovative madrigals, was also infamous for murdering his wife and her lover and thereafter having himself beaten regularly by one of his servants. Who was he?

4. (8 points) OOPs!

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "`ERROR`". No answer requires more than 3 lines. (It's possible that all of them require even fewer.)

```
class Thing:
    id = 0

    def fidget(self, n):
        print(n, "A", self.id)

    def fuss(self, x, n):
        print(n, "B")
        self.fidget(n)
        x.fidget(n)

    def twitch(self, n):
        self.waffle(n)

class Gadget(Thing):
    id = 1

class Whatsit(Gadget):
    def fidget(self, n):
        print(n, "D", self.id)

    def waffle(self, n):
        print(n, "D")

    def fiddle(self, x, n):
        x.waffle(n)

t1 = Thing()
t2 = Gadget()
t3 = Whatsit()
t4 = Whatsit()
t3.id = 2
```

Expression	**Interactive Output**
`t3.fidget(1)`	
`t4.fidget(2)`	
`t4.fuss(t1, 3)`	
`t4.fiddle(t4, 4)`	
`t4.fiddle(t1, 5)`	
`Thing.id = 3` `t1.fidget(6)`	

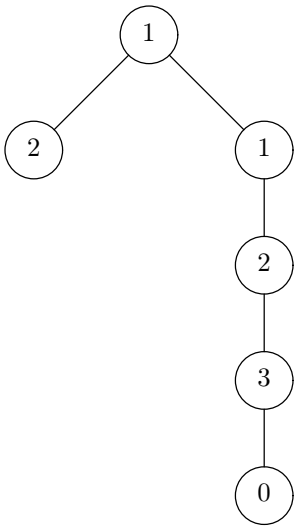
5. (8 points) Inflections

Fill in the definition of class `Wrinkles` to conform to its doc comment. You need not use all the lines shown.

```
class Wrinkles:
    """An object that contains a sequence of items and a predicate (true/false
    function) and that, when iterated over, produces adjacent pairs of items
    in the sequence that satisfy the predicate.
    >>> w = Wrinkles([1, 2, 3, 2, 4, 8, 5, 4], lambda x, y: x > y)
    >>> for p in w:
    ...    print(p)
    (3, 2)
    (8, 5)
    (5, 4)
    """

    def __init__(self, L, wrinkle):
        self._L = L
        self._wrinkle = wrinkle


    ------------------------------------------------------------------------


    ------------------------------------------------------------------------


    ------------------------------------------------------------------------


    ------------------------------------------------------------------------


    ------------------------------------------------------------------------


    ------------------------------------------------------------------------


    ------------------------------------------------------------------------


    ------------------------------------------------------------------------


    ------------------------------------------------------------------------


    ------------------------------------------------------------------------


    ------------------------------------------------------------------------


    ------------------------------------------------------------------------
```

6. (8 points) Tree Paths

Given a tree, t, find the length of the longest downward sequence of node labels in the tree that are increasing consecutive integers. For example, in this tree, the longest such sequence has three labels (1, 2, 3):



As illustrated, the longest sequence can start and end anywhere in the tree, not just the root. (**Hint:** don't forget there's a `max` function.) [The original skeleton was flawed. The original skeleton appears here, and a revised skeleton on the next page.]

```
def longest_seq(t):
    """The length of the longest downward sequence of nodes in T whose
    labels are consecutive integers.
    >>> t = Tree(1, [Tree(2), Tree(1, [Tree(2, [Tree(3, [Tree(0)])])])])
    >>> longest_seq(t) # 1 -> 2 -> 3
    3
    >>> t = Tree(1)
    >>> longest_seq(t)
    1
    """

    if ____________________________________________________________:

        return ____________________________________________________

    max_len = ____________________________________________________

    for ____________________________________________________________:

        if ____________________________________________________:

            ____________________________________________________
        else:

            ____________________________________________________

    return max_len
```

Here is the corrected skeleton.

```
def longest_seq(t):
    """The length of the longest downward sequence of nodes in T whose
    labels are consecutive integers.
    >>> t = Tree(1, [Tree(2), Tree(1, [Tree(2, [Tree(3, [Tree(0)])])])])
    >>> longest_seq(t) # 1 -> 2 -> 3
    3
    >>> t = Tree(1)
    >>> longest_seq(t)
    1
    """
    max_len = 1

    def longest(t):
        """Returns longest downward sequence of nodes starting at T whose
        labels are consecutive integers.  Updates max_len to that length,
        if greater."""

        ____________________________________________________________

        n = 1
        if ____________________________________________________________:

            for ________________________________________________________

                ____________________________________________________

                if ____________________________________________________

                    n = ________________________________________________

            max_len = ____________________________________________________
        return n
    longest(tr)
    return max_len
```

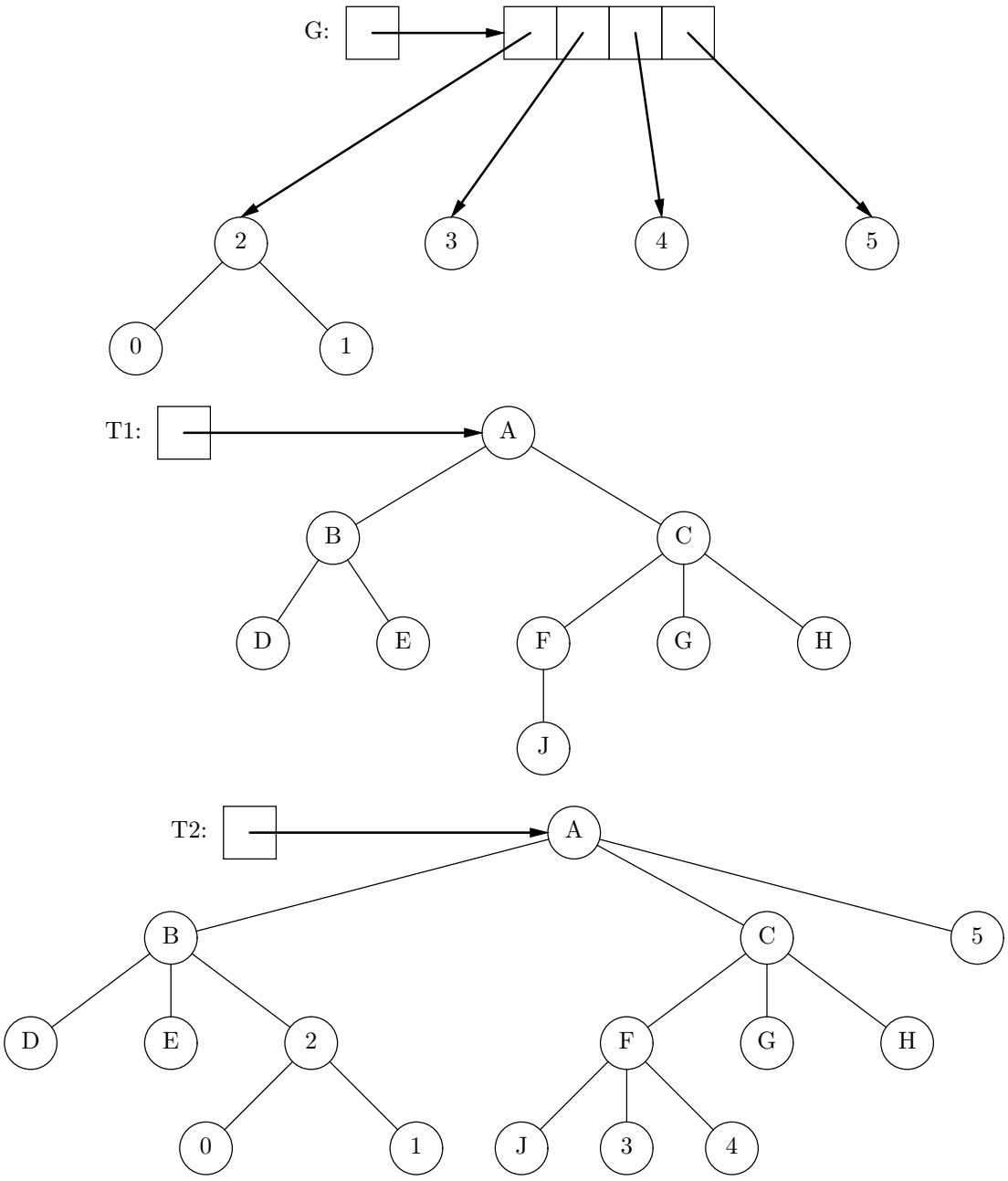
This page deliberately blank.

7. (8 points) Grafting

We want to insert ("graft") branches from a sequence of trees onto a tree in places where a non-leaf node has fewer than K branches, where K is a parameter. For example, given the list of four trees `G` created by

```
G = [ Tree(2, [Tree(0), Tree(1)]), Tree(3), Tree(4), Tree(5) ]
```

and the tree `T1` shown below, we want `T2 = graft(T1, G, 3)` to destructively (and without creating any new tree nodes) turn `T1` into the tree `T2`:



The list of trees (`G` in the example above) will always have enough items to fill all necessary places. Trees are inserted in postorder (that is, bottom to top, left to right).

Fill in the `graft` function here. You need not use all the lines.

```
def graft(T, L, k):
    """Returns the tree created by destructively adding trees from L to
    non-leaf nodes of T with fewer than K branches.  Assume that L has enough
    items to fill all necessary places.  Fill in trees in postorder (bottom to
    top, left to right)."""

    grafts = iter(L)   # Don't have to use this, but it may be useful.

    def do_grafts(tr):


        if ____________________________________________________________________:

            for ____________________________________________________________:

                ____________________________________________________________

            while ____________________________ < ____________________________:

                ____________________________________________________________

                ____________________________________________________________

    do_grafts(T)

    return T
```

CS 61A Structure and Interpretation of Computer Programs
Spring 2018

MIDTERM 1

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 study guide.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, and `abs`. You **may not** use functions defined on your study guide unless clearly specified in the question.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.

1. (12 points) Frame of Thrones

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
from operator import add, sub

def winterfell(a, b):
    a
    b
    return b(a+1, b(a))

da, ny = 20, 18

while da > ny:
    da = ny
    da, ny = ny + 1, da + 3

def tar(gar, yen):
    if print(yen):
        print(yen + 1)
    return gar(yen)

def st(ar, k=None):
    return lambda a, y: ar(y, a)

night = st(sub)
king = st(st(pow))

def jon(sn, ow):
    print(ow)
    jon = sn(ow)
    print(ow)
    return jon

def snow(ow):
    def tarly(snow):
        return ow + snow
    ow += 2
    return tarly
```

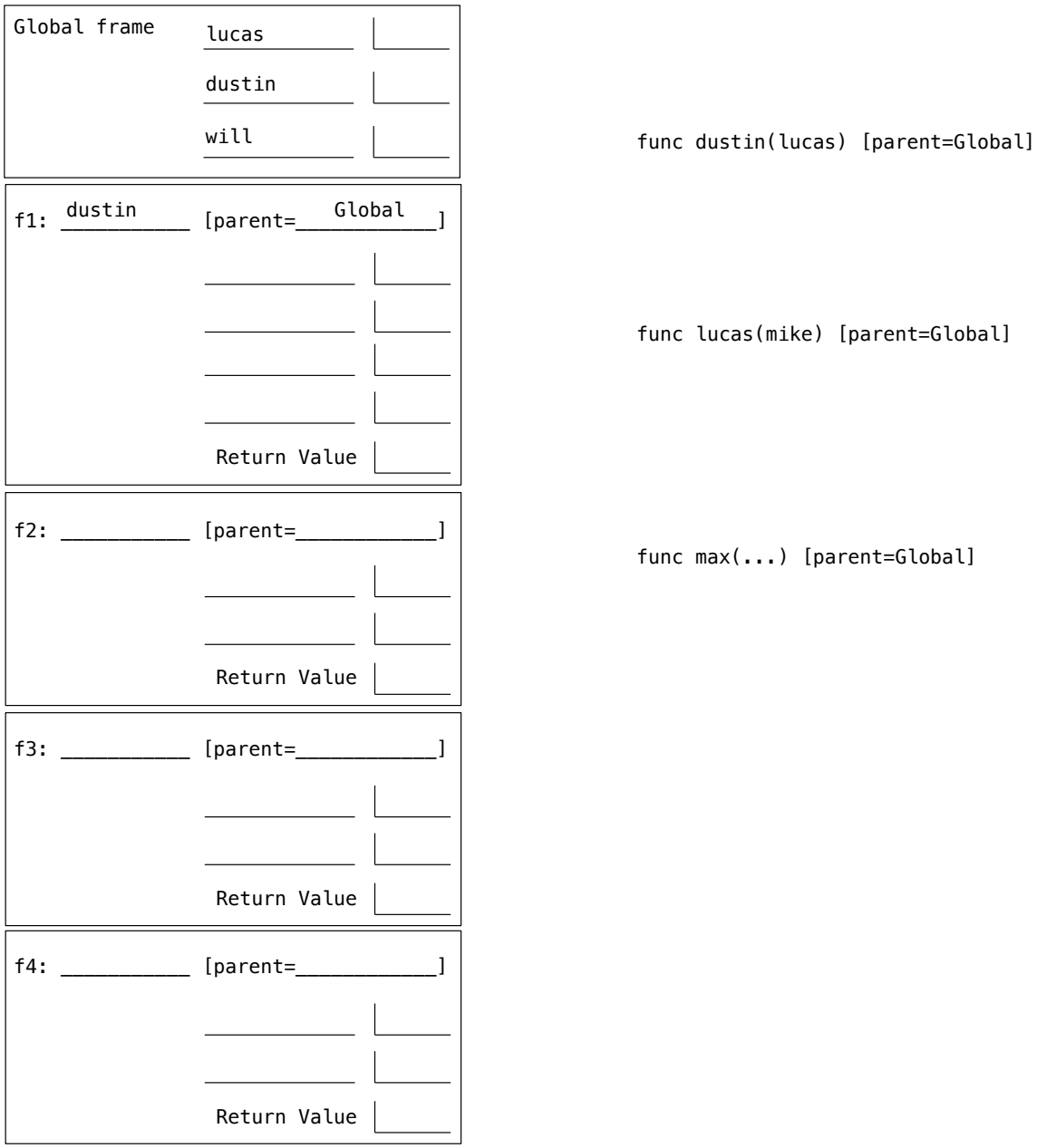
Expression	Interactive Output
`sub(pow(10, 2), 1)`	99
`print(4, 5) + 1`	4 5 Error
`(print(2) or 3) // (0 or 1)`	
`winterfell(2, print)`	
`ny`	
`tar(lambda x: x-7, 8)`	
`night(king(2, 3), 4)`	
`jon(snow(5), 2)`	

2. (8 points) Stranger Frames

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.* A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  def lucas(mike):
2      return will
3
4  def dustin(lucas):
5      will = 1
6      def dustin(mike):
7          will = 2
8          return lucas
9      return lambda mad: dustin(3)(will)
10
11 will = 5 + 6
12 lucas = dustin(lucas)
13 lucas(max)
```



3. (10 points) Choose Wisely

(a) (4 pt) Implement `sum_some`, which takes a non-negative integer `n` and a function `p`. It returns the sum of all the digits `d` for which `p` returns a true value when given `d` as an argument. Assume that the function `p` takes a single digit `d` (from 0 to 9) and returns either `True` or `False`.

```
def sum_some(n, p):
    """Return the sum of the digits of N for which P returns a true value.

    >>> even = lambda d: d % 2 == 0
    >>> big = lambda d: d > 5
    >>> sum_some(124567, even)     # Sum the even digits: 2 + 4 + 6
    12
    >>> sum_some(124567, big)      # Sum the big digits: 6 + 7
    13
    """
    total = 0

    while ____________________________________________________________________________:

        if ____________________________________________________________________________:

            ____________________________________________________________________________

        ____________________________________________________________________________

    return total
```

(b) (4 pt) Implement `sum_largest`, which takes non-negative integers `n` & `k`. It sums the largest `k` digits of `n`.

```
def sum_largest(n, k):
    """Return the sum of the K largest digits of N.

    >>> sum_largest(2018, 2)    # 2 and 8 are the two largest digits (larger than 0 and 1).
    10
    >>> sum_largest(12345, 10)  # There are only five digits, so all are included in the sum.
    15
    """
    if ____________________________________________________________________________:

        return 0

    a = ____________________________________________________________________________

    b = ____________________________________________________________________________

    return ____________________________________________________________________(a, b)
```

(c) (2 pt) Complete the expression below by **only adding parentheses** so that the whole expression evaluates to 2018. Each blank should be filled with one or more parentheses.

```
(lambda a, x: x + (lambda y: lambda z: y+z+1000)(1000 _____ 10 _____ 5, _____ lambda: 8 _____ )
```

4. (10 points) Editor

Definitions. An *edit* is a pure function that takes a non-negative integer and returns a non-negative integer. An *editor* for a non-negative integer `n` is a function that takes an *edit*, applies it to `n`, displays the result, and then returns an editor for the result.

(a) (3 pt) Implement `make_editor`, which takes a non-negative integer `n` and a one-argument function `pr`. It returns an *editor* for `n` that uses `pr` to display the result of each *edit*.

(b) (5 pt) Implement `insert`, which takes a single digit `d` (from 0 to 9) and a non-negative position `k`. It returns an *edit* that inserts `d` into its argument `n` at position `k`, where `k` counts the number of digits from the end of `n`. **Assume that `k` is not larger than the number of digits in `n`. Your solution must be recursive.**

(c) (2 pt) Implement `delete`, which takes a non-negative integer `k` and returns an *edit* that deletes the last `k` digits of its argument `n`. You may use `pow` or `**` in your solution.

```
def make_editor(n, pr):
    """Return an editor for N.

    >>> f = make_editor(2018, lambda n: print('n is now', n))
    >>> f = f(delete(3))     # delete the last 3 digits from the end of 2018
    n is now 2
    >>> f = f(insert(4, 0))  # insert digit 4 at the end of 2 (position 0)
    n is now 24
    >>> f = f(insert(3, 1))  # insert digit 3 in the middle of 24 (position 1)
    n is now 234
    >>> f = f(insert(1, 3))  # insert digit 1 at the start of 234 (position 3)
    n is now 1234
    >>> f = make_editor(123, print)(delete(10))  # delete 10 digits from the end of 123
    0
    """
    def editor(edit):

        result = ____________________________________________________________________________

        ____________________________________________________________________________________

        ____________________________________________________________________________________

    return editor

def insert(d, k):

    def edit(n):

        if ____________________________________________________________________________________:

            return ______________ + 10 * ____________________________________________________

        else:

            return ______________ + 10 * insert ____________________________________________

    return edit

delete = ____________________________________________________________________________________
```

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Spring 2018

MIDTERM 2

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except two hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 and midterm 2 study guides.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, and `abs`.
- You **may not** use example functions defined on your study guides unless clearly specified by the question.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.
- You may use the `Tree`, `Link`, and `BTree` classes defined on Page 2 (left column) of the Midterm 2 Study Guide.

1. (10 points) Buggy Quidditch

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines.

- If an error occurs, write **Error**, but include all output displayed before the error.
- To display a function value, write **Function**.
- If an expression would take forever to evaluate, write **Forever**.

The interactive interpreter displays the contents of the `repr` string of the value of a successfully evaluated expression, unless it is `None`.

Assume that you have started `python3` and executed the code shown on the left first, then you evaluate each expression on the right in the order shown. Expressions evaluated by the interpreter have a cumulative effect.

```
 1  class Ball:
 2      points = 0
 3      time = lambda: 'Draco'
 4
 5      def score(self, who):
 6          print(who, self.points)
 7
 8      def __str__(self):
 9          return 'Magic'
10
11  class Snitch(Ball):
12      points = 100
13      time = lambda: 'Harry'
14
15      def __init__(self):
16          self.points = self.points + 50
17
18      def score(self, p):
19          if not time():
20              print(Ball().score(p))
21          else:
22              Ball.score(self, p)
23
24  def chase(r):
25      r.time = Snitch.time
26      r.points += 1
27      quaffle.points += 10
28      print(r().points)
29
30  quaffle = Ball()
31  quaffle.points = 10
32  chasing = quaffle.score
33  time = lambda: Ball.points
34  malfoy = lambda: Ball.time()
```

Expression	Interactive Output
`Snitch().points`	
`chasing(quaffle)`	
`Snitch().score('Seeker')`	
`chase(Ball)`	
`Snitch().score(malfoy())`	

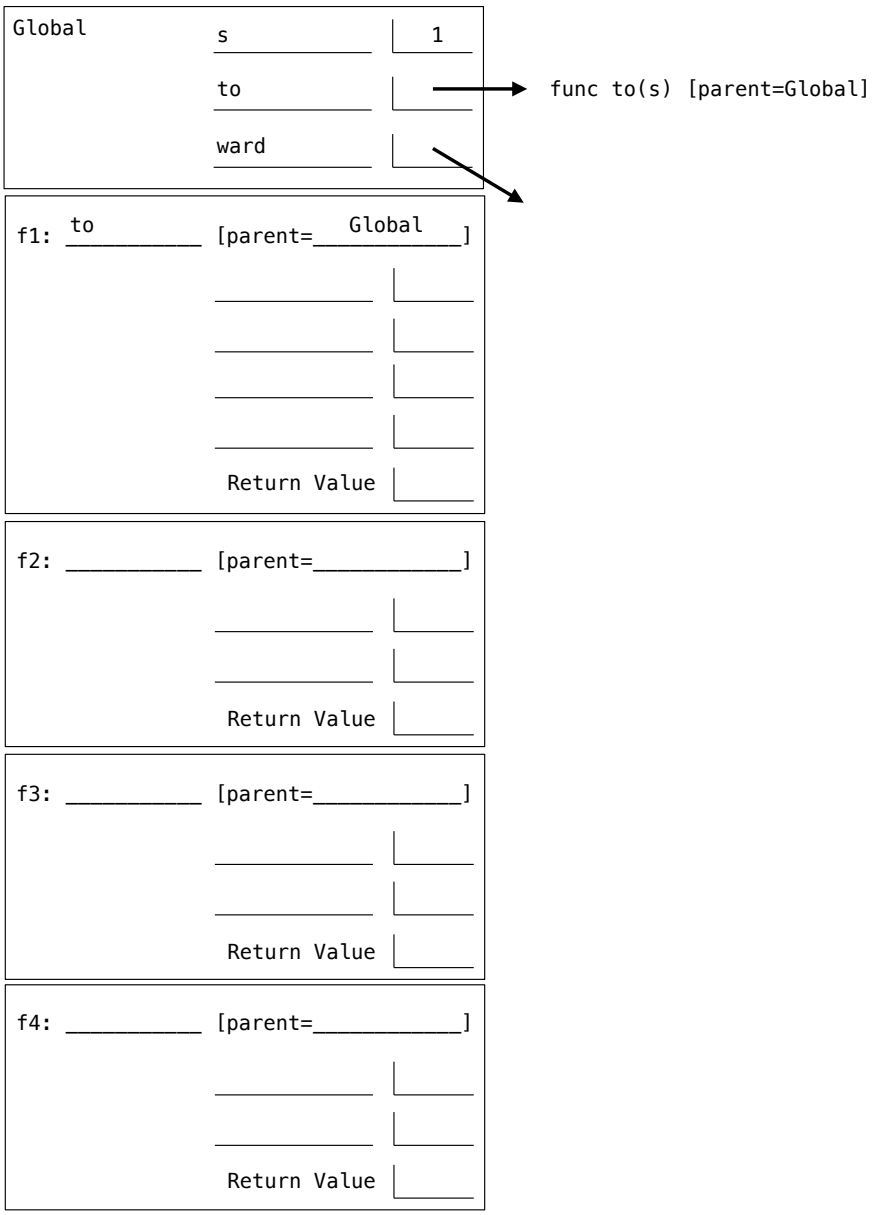
2. (6 points) NVRnment

Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*

A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.
- **Use box-and-pointer diagrams for lists and tuples.**

```
1  s = 1
2  def to(s):
3      ix = s[1:2]
4      ward[1] = 6
5      def re(st):
6          if st is not ward:
7              nonlocal s
8              s = s.extend(ix)
9              return re(ward)
10         else:
11             st.append(ix)
12     return re
13 ward = [3, 4] + list([5])
14 to(ward)(s)
```



3. (12 points) Lists

Definition. A *grid* is a list of lists. Each list in a grid is called a *row*, and all rows must have the same length. `[[1, 2], [3, 4]]` is a grid of integers, but `[[1, 2], [3, 4, 5]]` is not a grid.

(a) (2 pt) Implement *column*, which takes a grid `g` and a non-negative integer `c` that is smaller than the length of a row in `g`. It returns a list containing the element at index `c` of each row in the grid.

```
def column(g, c):
    """Return the column of g at index c.

    >>> column([[3, 4, 5], [6, 7, 8], [9, 10, 11]], 1)
    [4, 7, 10]
    """

    return ____________________________________________________________________________
```

(b) (4 pt) Implement `print_grid`, which takes a grid `g`. It prints one line for each row in `g`. Line k displays each element in row k, separated by the minimum number of spaces needed to align the left edges of the `str` strings for the elements of each column of the grid. At least one space should appear between any two elements. Extra spaces at the end of a line are ok. **You may not use the `column` function above. Assume that `g` has at least one row.**

```
def print_grid(g):
    """Print each row on a separate line with columns aligned.

    >>> print_grid([[1, 234, 50, 4, 5], [67, 8, 90, 0, 500], [3, 4, 5, -500, 7]])
    1  234 50 4    5
    67 8   90 0    500
    3  4   5  -500 7
   """

    cs = ____________________________________________________________________________

    widths = [ __________ ([len(str( __________ [ __________ ])) for row in g]) for c in cs]

    for row in g:

        line = ''

        for c in cs:

            s = str(_____________________________________________________________________)

            line = line + s + ' ' * (_____________________________________________________)

        print(line)
```

(c) (4 pt) Implement `expand`, which takes a grid `g`, a number of rows `h`, a number of columns `w`, and a `fill` value. It mutates the contents of `g` so that `g` has at least `h` rows and `w` columns. Any added values are `fill`.

```
def expand(g, h, w, fill):
    """Expand grid g so that it has at least h rows and w columns.

    >>> g = [[1, 2, 3], [40, 50, 60]]

    >>> print_grid(expand(g, 2, 5, 10))
    1  2  3  10 10
    40 50 60 10 10

    >>> print_grid(expand(g, 5, 6, 0))
    1  2  3  10 10 0
    40 50 60 10 10 0
    0  0  0  0  0  0
    0  0  0  0  0  0
    0  0  0  0  0  0

    >>> print_grid(expand(g, 0, 0, 5))   # expand never reduces the dimensions of g.
    1  2  3  10 10 0
    40 50 60 10 10 0
    0  0  0  0  0  0
    0  0  0  0  0  0
    0  0  0  0  0  0
    """

    for row in g:

        row._____(_____________________________________________________________)

    for k in ____________________________________________________________________________:

        g._____(_____________________________________________________________)

    return g
```

(d) (2 pt) Circle the Θ expression that describes how many new values must be added when a grid with n rows and n columns is expanded to $2 \times n$ rows and $2 \times n$ columns using the `expand` function. Assume that `expand` is implemented correctly.

$\Theta(1)$ $\Theta(\log n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(2^n)$ None of these

4. (12 points) Sequences

(a) (6 pt) Implement `stretch`, which takes a `Link` instance `s` with no cycles. It mutates `s` so that, for each position k in the original `s`, the k^{th} element is repeated k times. **You do not need to use the name `i`.**

```
def stretch(s, repeat=0):
    """Replicate the kth element k times, for all k in s.

    >>> a = Link(3, Link(4, Link(5, Link(6))))
    >>> stretch(a)
    >>> print(a)
    <3, 4, 4, 5, 5, 5, 6, 6, 6, 6>
    """
    if ____________________________________________________________________________________:

        for i in range(repeat):


            ____________ = ____________________________________________________________


            ____________ = ____________________________________________________________


        ____________________________________________________________________________________
```

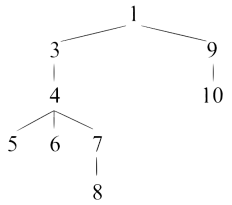
(b) (6 pt) Implement `combo`, which takes two non-negative integers `a` and `b`. It returns the smallest integer that contains all of the digits of `a` in order, as well as all of the digits of `b` in order.

```
def combo(a, b):
    """Return the smallest integer with all of the digits of a and b (in order).

    >>> combo(531, 432)      # 45312 contains both _531_ and 4_3_2.
    45312
    >>> combo(531, 4321)     # 45321 contains both _53_1 and 4_321.
    45321
    >>> combo(1234, 9123)    # 91234 contains both _1234 and 9123_.
    91234
    >>> combo(0, 321)        # The number 0 has no digits, so 0 is not in the result.
    321
    """
    if ____________________________________________________________________________________:

        return a + b

    elif ____________________________________________________________________________________:


        return combo(__________________, _________________)_______________________________


    return ________(_____________________________________________________________________,


                    ____________________________________________________________________)
```

5. (10 points) Trees

Definition. A *sibling* of a node in a tree is another node with the same parent.

(a) (4 pt) Implement `siblings`, which takes a `Tree` instance `t`. It returns a list of the labels of all nodes in `t` that have a sibling. These labels can appear in any order.




```
def siblings(t):
    """Return a list of the labels of all nodes that have siblings in t.

    >>> a = Tree(4, [Tree(5), Tree(6), Tree(7, [Tree(8)])])
    >>> siblings(Tree(1, [Tree(3, [a]), Tree(9, [Tree(10)])]))
    [3, 9, 5, 6, 7]
    """
    result = [_____________________________________________________________________________]

    for b in t.branches:

        _____________________________________________________________________________

    return result
```

(b) (6 pt) Implement the `Sib` class that inherits from `Tree`. In addition to `label` and `branches`, a `Sib` instance `t` has an attribute `siblings` that stores the number of siblings `t` has in `Sib` trees containing `t` as a node. **Assume that the `branches` of a `Sib` instance will never be mutated or re-assigned.**

```
class Sib(Tree):
    """A tree that knows how many siblings it has.

    >>> a = Sib(4, [Sib(5), Sib(6), Sib(7, [Sib(8)])])
    >>> a.label
    4
    >>> a.branches[1].label
    6
    >>> a.siblings
    0
    >>> a.branches[1].siblings
    2
    """
    def __init__(self, label, branches=[]):

        self.siblings = _____________________________________________________________

        _____________________________________________________________________________:

            ___________________________________________________________________

        Tree._____________________________________________________________________
```

CS 61A Structure and Interpretation of Computer Programs
Spring 2019

MIDTERM 1

INSTRUCTIONS

- You have 55 minutes to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 study guide.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, and `abs`.
- You **may not** use example functions defined on your study guides unless a problem clearly states you can.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.

1. (10 points, 15 minutes) What Would Python Display

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
n = 1
f = lambda: n
g = f
n = 2
```

```
x, y, loops = 0, 21, 0
while y:
    loops += 1
    if x <= y:
        if x % 2 == 1:
            x, y = y, x
        elif x == 0:
            x += (2 ** 2)
        else:
            x = x + 3
            y = y // 2
    else:
        y = 0
```

```
def foo(bar, z):
    def bar(w):
        z = w + 2
        return z
    print(bar(z))
    print(z)
    return bar
```

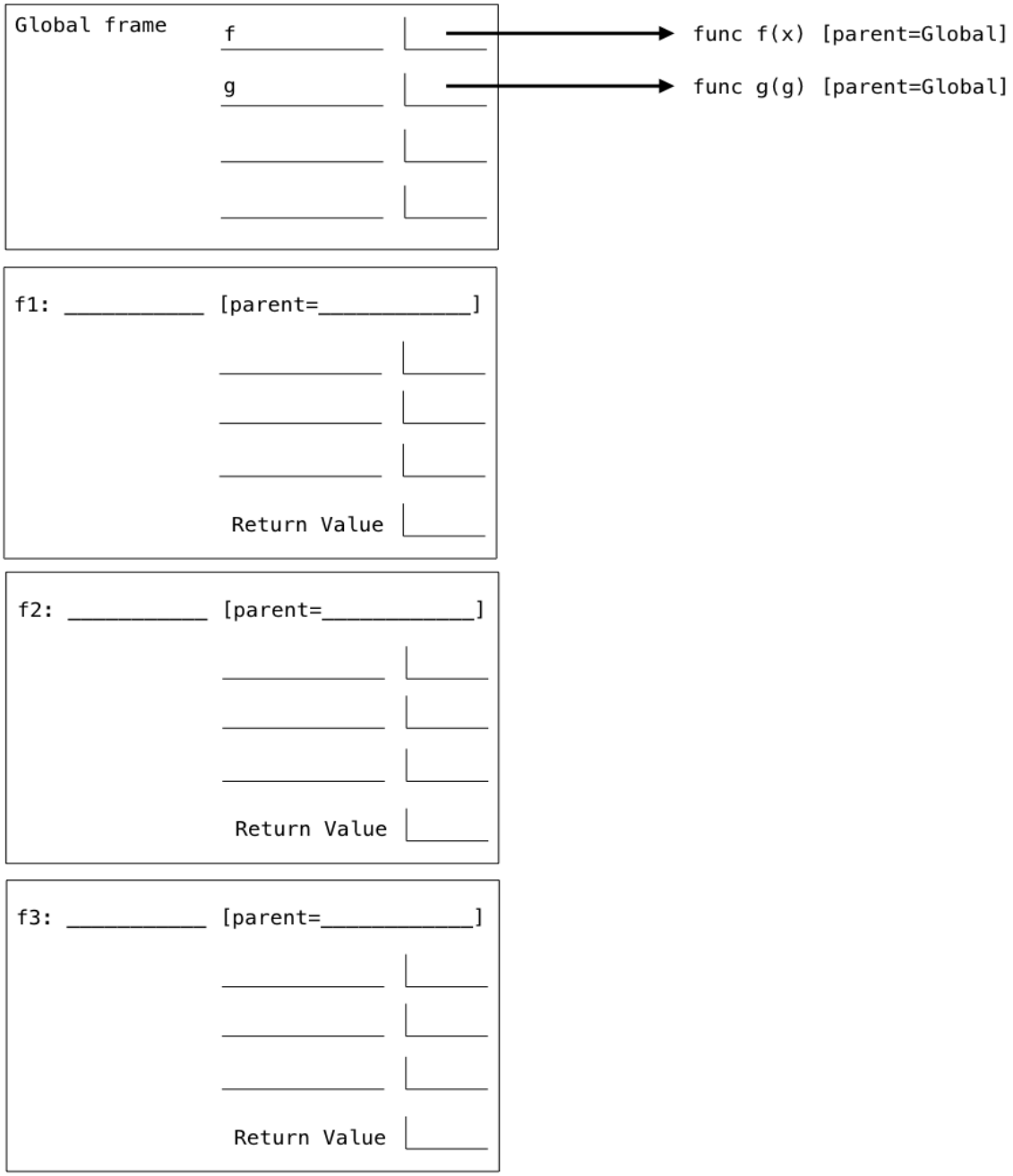
Expression	**Interactive Output**
`pow(10, 2)`	100
`print(4, 5) + 1`	4 5 Error
`print(1, print(print(2), 3 or 4 // 0))`	
`g()`	
`x`	
`loops`	
`foo(lambda m: m + 1, 4)(7)`	

2. (10 points, 15 minutes) Environment Diagram

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.* A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  def f(x):
2    f = 10
3    z = 100
4    x = g(x)(f)
5    def f(x):
6      return x + y
7    return f
8
9  def g(g):
10   return lambda g: g + z
11
12 z = 3
13 f = f(5)
```



3. (10 points, 25 minutes) Zip It

We would like to create a function `make_zipper` that takes two functions `f1(x)` and `f2(x)` and a "zipper sequence", which is a number that contains a series of 1s and 2s. It returns a function that is the equivalent of `f1(f2(f2(...f1(x)...)))` in which the exact sequence of `f1`s and `f2`s is given by the digits of the sequence. As an example, if the sequence were `1211`, that would mean return a function of `x` that is the equivalent to `f1(f2(f1(f1(x))))`. Neither recursion nor containers (lists, dictionaries, sets, etc) are allowed in your solution.

```
def make_zipper(f1, f2, sequence):
    """ Return a function of f1 and f2 composed based on sequence.

    >>> def increment(x):
            return x + 1
    >>> def square(x):
            return x * x
    >>> do_nothing = make_zipper(increment, square, 0)
    >>> do_nothing(2)      # Don't call either f1 or f2, just return your input untouched
    2
    >>> incincsq = make_zipper(increment, square, 112)
    >>> incincsq(2)        # increment(increment(square(2))), so 2 → 4 → 5 → 6
    6
    >>> sqincsqinc = make_zipper(increment, square, 2121)
    >>> sqincsqinc(2)      # square(increment(square(increment(2)))), so 2 → 3 → 9 → 10 → 100
    100
    """

    zipper = _____

    helper = _____

    while _____:

        if _____ == 1:

            zipper = helper(f1, _____ )

        else:

            zipper = helper(f2, _____ )

        sequence = _____

    return zipper
```

CS 61A Spring 2019 — Structure and Interpretation of Computer Programs

MIDTERM 2

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except two hand-written 8.5" × 11" crib sheets of your own creation and the official CS 61A study guides.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, `abs`, `sum`, `next`, `iter`, `list`, `tuple`, `map`, `filter`, `zip`, `all`, and `any`.
- You **may not** use example functions defined on your study guides unless a problem clearly states you can.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.
- You may use the `Tree`, `Link`, and `BTree` classes defined on Page 2 (left column) of the Midterm 2 Study Guide.

1. (12 points) What Would Python Display

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
items, n = [], 2

class Airpods:
    cost, k = 200, 0
    f = lambda self: print(self)

    def __init__(self):
        Airpods.k += 1
        Airpods.f(self)
        items.extend([self])

    def __repr__(self):
        return (Airpods.k < 2 and "lonely") or "pair"

class TwoAirpods(Airpods):
    def __init__(self):
        self.k = 2
        Airpods.__init__(self)
        Airpods.__init__(self)

def discount(a):
    a.cost //= 2

def u(w, u):
    return [print(u) for u in [w, u]]

discount(Airpods)
```

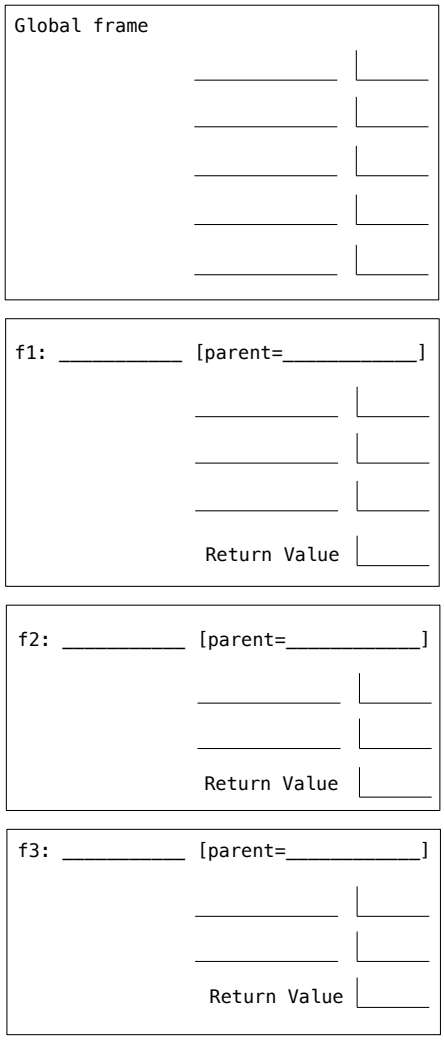
Expression	Interactive Output
`pow(10, 2)`	100
`print(Link(2, Link(3)))`	`<2, 3>`
`TwoAirpods.cost`	
`lost = Airpods()`	
`willbelost = TwoAirpods()`	
`str(lost)`	
`[item.k for item in items]`	
`u(lost, willbelost)`	

2. (10 points) Ultimate

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*
A complete answer will:

- Use box-and-pointer notation for all lists.
- Add missing names and parents to all local frames.
- Add missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  defeated = ['luigi']
2  def start():
3      chal = 'falco'
4      def approach(new_chal):
5          nonlocal chal
6          defeated.append(chal)
7          chal = new_chal
8      def revive():
9          return defeated.pop()
10     return [approach, revive]
11 gameover = start()[:]
12 gameover.pop(0)('marth')
13 restart = gameover[0]()
```



func start() [parent=Global]

func approach(new_chal) [parent= _____]

func revive() [parent= _____]

3. (5 points) Deep lists

Implement `in_nested` which takes in a value `v` and a nested list or an individual value `L` and returns whether the value is contained in the list.

Hint: The built-in function `type` takes an object and returns the type of that object.

```
def in_nested(v, L):
    """
    >>> in_nested(5, [1, 2, [[3], 4]])
    False
    >>> in_nested(9, [[[1], [6, 4, [5, [9]]], 7], 7, 7])
    True
    >>> in_nested(1, 1)
    True
    """
    if _____:

        return _____

    else:

        return _____
```

4. (8 points) A data structure by any other form would smell just as sweet...

(a) (6 pt) Implement `link_to_dict` which takes a linked list encoding a "flattened" dictionary (in which elements are key1 → value1 → key2 → value2 → key3 → value3, etc), **removes all the values**, and returns the equivalent dictionary. The input and returned list **may** include duplicate keys, as in the example below. You may assume the linked list always contains an even number of elements. The `Link` class is provided below.

```
class Link:
   empty = ()

   def __init__(self, first, rest=empty):
       self.first = first
       self.rest = rest

   def __str__(self):
       string = '<'
       while self.rest is not Link.empty:
           string += str(self.first) + ', '
           self = self.rest
       return string + str(self.first) + '>'

def link_to_dict(L):
    """
    >>> L = Link(1, Link(2, Link(3, Link(4, Link(1, Link(5))))))
    >>> print(L)
    <1, 2, 3, 4, 1, 5>
    >>> link_to_dict(L)
    {1: [2, 5], 3: [4]}
    >>> print(L)
    <1, 3, 1>
    """
    D = {}

    while _____:

        key, value = _____

        if _____:

            _____

        else:

            _____

        _____

    return D
```

(b) (2 pt) Circle the Θ expression that describes the number of iterations of the while loop in `link_to_dict` where `n` is the length of the list.

$\Theta(1)$ $\Theta(\log n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(2^n)$ None of these

5. (10 points) I speak for the Trees

Execute each line of code in order. If a line errors, assume we didn't type that line. For each line, indicate whether it was:

- (E) an error
- (D) a Data Abstraction Violation
- (OK) perfectly fine code.

Please fill in the bubbles completely.

```
def tree(label, branches=[]):
    return [label] + list(branches)

def label(tree):
    return tree[0]

def branches(tree):
    return tree[1:]

def print_tree_adt(t, indent=0):
    print('  ' * indent + str(label(t)))
    for b in branches(t):
        print_tree_adt(b, indent + 1)


class Tree:
    def __init__(self, label, branches=[]):
        self.label = label
        self.branches = list(branches)

    def __str__(self):
        return '\n'.join(self.indented())

    def indented(self, k=0):
        ind = []
        for b in self.branches:
            for line in b.indented(k + 1):
                ind.append('  ' + line)
        return [str(self.label)] + ind
```

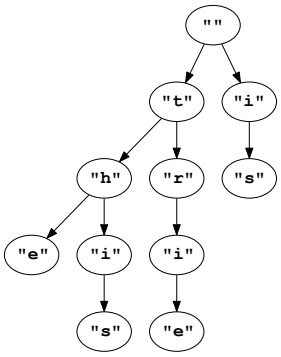
(E)	(D)	(OK)	`t = Tree(3, [Tree(1), Tree(2)])`
(E)	(D)	(OK)	`L = [tree(1)]`
(E)	(D)	(OK)	`L.append(tree(2))`
(E)	(D)	(OK)	`s = tree(3, L)`
(E)	(D)	(OK)	`s`
(E)	(D)	(OK)	`print(s)`
(E)	(D)	(OK)	`t`
(E)	(D)	(OK)	`t.label`
(E)	(D)	(OK)	`print(t)`
(E)	(D)	(OK)	`label(s)`
(E)	(D)	(OK)	`t.label = 4`
(E)	(D)	(OK)	`label(s) = 4`
(E)	(D)	(OK)	`s[0] = 5`
(E)	(D)	(OK)	`print_tree_adt([5])`
(E)	(D)	(OK)	`print_tree_adt(s)`
(E)	(D)	(OK)	`t.branches[0].label = s`
(E)	(D)	(OK)	`print_tree_adt(t.branches[0])`
(E)	(D)	(OK)	`print_tree_adt(t.branches[0].label)`
(E)	(D)	(OK)	`t`
(E)	(D)	(OK)	`print(t)`

6. (15 points) Trie this

A Trie is a Tree where every node in the tree contains a single letter except for the root which is always the empty string. Every path from the root to a leaf forms a word. You may assume no words are substrings of other words in the trie (e.g., "hi" and "him"). The figure below is a trie generated by storing the words ["this", "is", "the", "trie"]. The `Tree` class is defined on Page 2 of the Midterm 2 Study Guide.

(a) (7 pt) Implement `add_word` which takes a Trie and a word and adds the word to the trie.

```
def make_trie(words):
    """ Makes a tree where every node is a letter of a word.
        All words end as a leaf of the tree.
        words is given as a list of strings.
    """
    trie = Tree('')
    for word in words:
        add_word(trie, word)
    return trie
```




```
def add_word(trie, word):

    if _____:
        return

    branch = _____

    for _____:

        if _____:

            branch = _____

    if _____:

        branch = _____

        _____

    _____
```

(b) (8 pt) Implement `get_words`, which takes a Trie and returns a list of all the words the Trie is storing.

```
def get_words(trie):
    """
    >>> get_words(make_trie(['this', 'is', 'the', 'trie']))
    ['this', 'the', 'trie', 'is']
    """
    if _____:

        return _____

    return sum(_____, [])
```

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Spring 2020

MIDTERM 1

INSTRUCTIONS

- You have 1 hour and 50 minutes to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 study guide.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, and `abs`.
- You may not use lists, dictionaries, tuples, sets, or the `:=` operator. These features have not been covered.
- You **may not** use example functions defined on your study guide unless a problem clearly states you can.
- For fill-in-the-blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.

1. (8 points) What Would Python Display

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
def tik(tok):
    tik = lambda: tok + 1
    while tok - tik() < 5:
        tok = tik() + 1
        if tok > 100:
            return tik()
    return tok
```

```
snap = lambda chat: lambda: snap(chat)
snap, chat = print, snap(2020)
```

```
def q(q):
    if print(q, q):
        print(q + 1)
    if q:
        q = q + q
    if q > 0:
        return q
    print(q + 2)
```

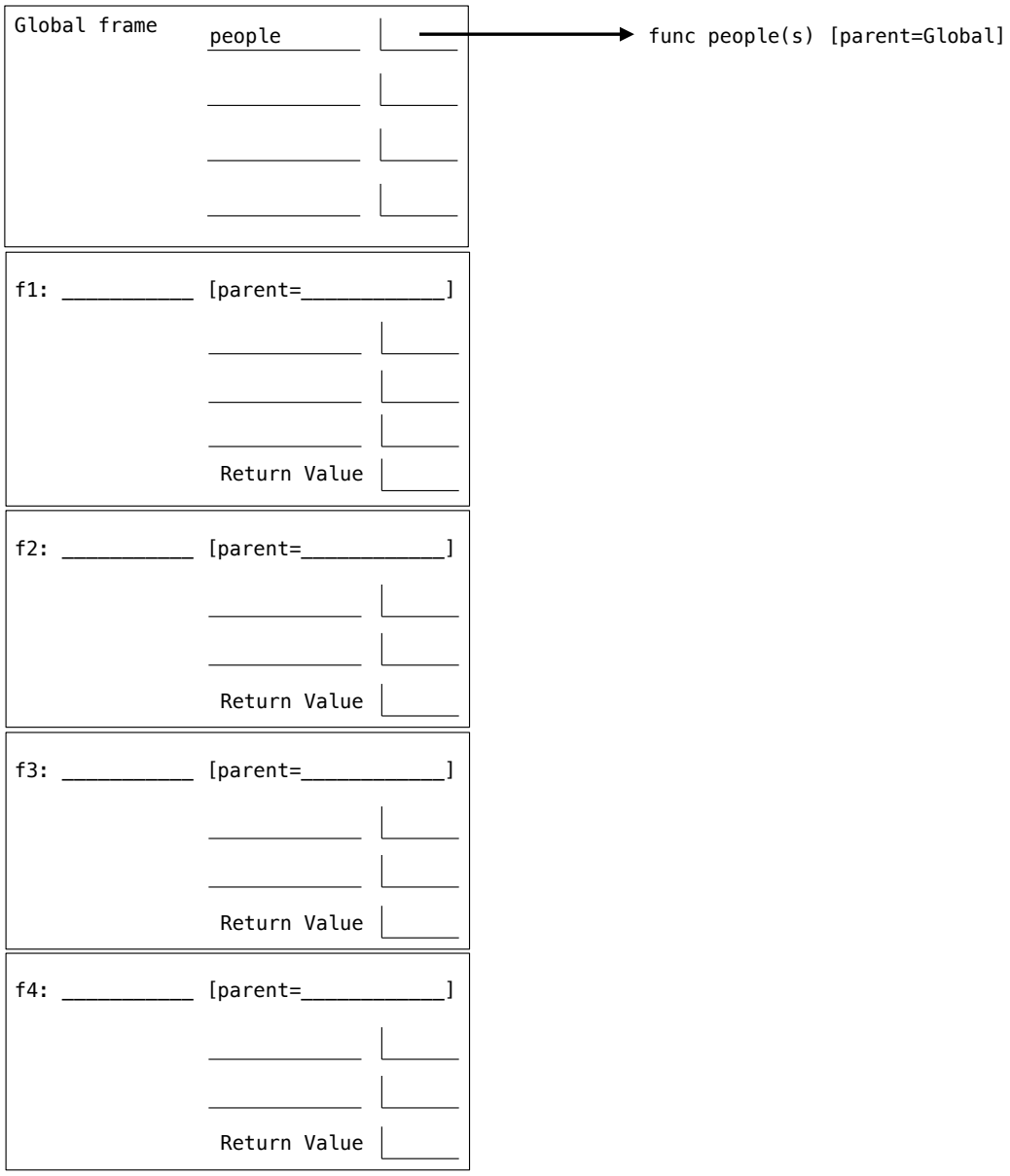
	Expression	Interactive Output
	`pow(10, 2)`	100
	`print(4, 5) + 1`	4 5 Error
(2 pt)	`print(print)(2020)`	
(2 pt)	`tik(50)`	
(1 pt)	`chat(2020)`	
(1 pt)	`chat()`	
(2 pt)	`q(20)`	

2. (8 points) People's Park

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled.
You may not need to use all of the spaces or frames.
Do not include frames for calls to built-in functions.
A complete answer will:

- Add all missing names and parents to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.
- Cross out or erase all arrows and values that are not part of the final diagram.

```
1  def people(s):
2      unit = 2
3      park = s(unit, t) + 1
4      s = (lambda t: park)(s)
5      return lambda: abs(unit)
6
7  unit, t = 1, 3
8  def park(t, park):
9      return unit - 2
10 unit = people(park)
```



3. (16 points) Digit Fidget

(a) (6 pt) Implement `same_digits`, which takes two positive integers. It returns whether they both become the same number after replacing each sequence of a digit repeated consecutively with only one of that digit. For example, in 12222321, the sequence 2222 would be replaced by only 2, leaving 12321.

Restriction: You may only write combinations of the following in the blanks:
`a`, `b`, `end`, `10`, `%`, `if`, `while`, `and`, `or`, `==`, `!=`, `True`, `False`, and `return`. (**No division allowed!**)

```
def same_digits(a, b):
    """Return whether a and b become the same number after removing adjacent repeats.

    >>> same_digits(2002200, 2202000) # Ignoring repeats, both are 2020
    True
    >>> same_digits(21, 12)           # Digits must appear in the same order
    False
    >>> same_digits(12, 2212)         # 12 and 212 are not the same
    False
    >>> same_digits(2020, 20)         # 2020 and 20 are not the same
    False
    """
    assert a > 0 and b > 0
    while a and b:

        if ____________________________________________________________________________:

            end = a % 10

            ____________________________________________________________________________:

                a = a // 10

            ____________________________________________________________________________:

                b = b // 10

        else:

            ____________________________________________________________________________

    ________________________________________________________________________________
```

(b) (3 pt) Implement `no_repeats`, which takes a positive integer `a` and returns the smallest positive integer `b` for which `same_digits(a, b)` returns `True`. Assume `same_digits` is implemented correctly. Watch out for the `assert` statement in the implementation of `same_digits`! You may **not** call `set` or `str`.

```
def no_repeats(a):
    """Remove repeated adjacent digits from a.

    >>> no_repeats(22000200)
    2020
    """
    return search(_____________________________________________________ , _______________)

def search(f, x):
    while not f(x):
        x += 1
    return x
```

(c) (4 pt) Implement `unique_largest`, which takes a positive integer `n`. It returns whether the largest digit in `n` appears only once in `n`. You may assign values to multiple names in an assignment statement.

You may **not** write call expressions or parentheses. You may **not** write `lambda`, `if`, `max`, `set`, `and`, or `or`.

```
def unique_largest(n):
    """Return whether the largest digit in n appears only once.

    >>> unique_largest(132123)  # 3 is largest and appears twice
    False
    >>> unique_largest(1321523) # 5 is largest and appears only once
    True
    >>> unique_largest(5)
    True
    """
    assert n > 0
    top = 0
    while n:

        n, d = n // 10, n % 10


        if ____________________________________________________________________________:


            ______________________________ = ____________________________________________

        elif d == top:

            unique = ___________________________________________________________________

    return unique
```

(d) (3 pt) Implement `transitive`, which takes a two-argument function `p` that returns `True` or `False`. The `transitive` function returns whether it is the case that for every three digits `a`, `b`, `c` for which `p(a, b)` and `p(b, c)` both return `True`, `p(a, c)` also returns `True`. A digit is an integer between 0 and 9, inclusive.

You may **not** write `str`, `[`, or `]`.

```
def transitive(p):
    """Return whether p is transitive over non-negative single digit integers.

    >>> transitive(lambda x, y: x < y)          # if a < b and b < c, then a < c
    True
    >>> transitive(lambda x, y: abs(x-y) == 1)  # E.g., p(3, 4) and p(4, 5), but not p(3, 5)
    False
    """
    abc = 0
    while abc < 1000:

        a, b, c = abc // 100, ____________________________________________________, abc % 10


        if p(a, b) _____________________________________________________________________:

            return False
        abc = abc + 1
    return True
```

4. (8 points) Composition

(a) (4 pt) Implement `compose`, which takes a positive integer `n`. It returns a function that, when called repeatedly on n one-argument functions $f_1, f_2, \ldots, f_n$, returns a one-argument function of `x` that returns $f_1(f_2(\ldots f_n(x)\ldots))$. You may **not** call the `compose1` function from the Midterm 1 Study Guide.

```
def compose(n):
    """Return a function that, when called n times repeatedly on unary
    functions f1, f2, ..., fn, returns a function g(x) equivalent to
    f1(f2( ... fn(x) ... )).

    >>> add1 = lambda y: y + 1
    >>> compose(3)(abs)(add1)(add1)(-4)  # abs(add1(add1(-4)))
    2
    >>> compose(3)(add1)(add1)(abs)(-4)  # add1(add1(abs(-4)))
    6
    >>> compose(1)(abs)(-4)              # abs(-4)
    4
    """
    assert n > 0


    if n == 1:


        return ____________________________________________________________________________


    def call(f):


        def on(g):


            return ______________________________(_______________________________________)


        return on


    return call
```

(b) (4 pt) Complete the final expression below with **only integers and names** so it evaluates to 2020.

```
from operator import add

c = lambda f: lambda x: lambda y: f(x, y)
twice = lambda z: 2 * z


compose(___________)(twice)(___________(___________)(10))(___________(pow)(10))(___________)
```

CS 61A Structure and Interpretation of Computer Programs
Spring 2021

MIDTERM 1

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

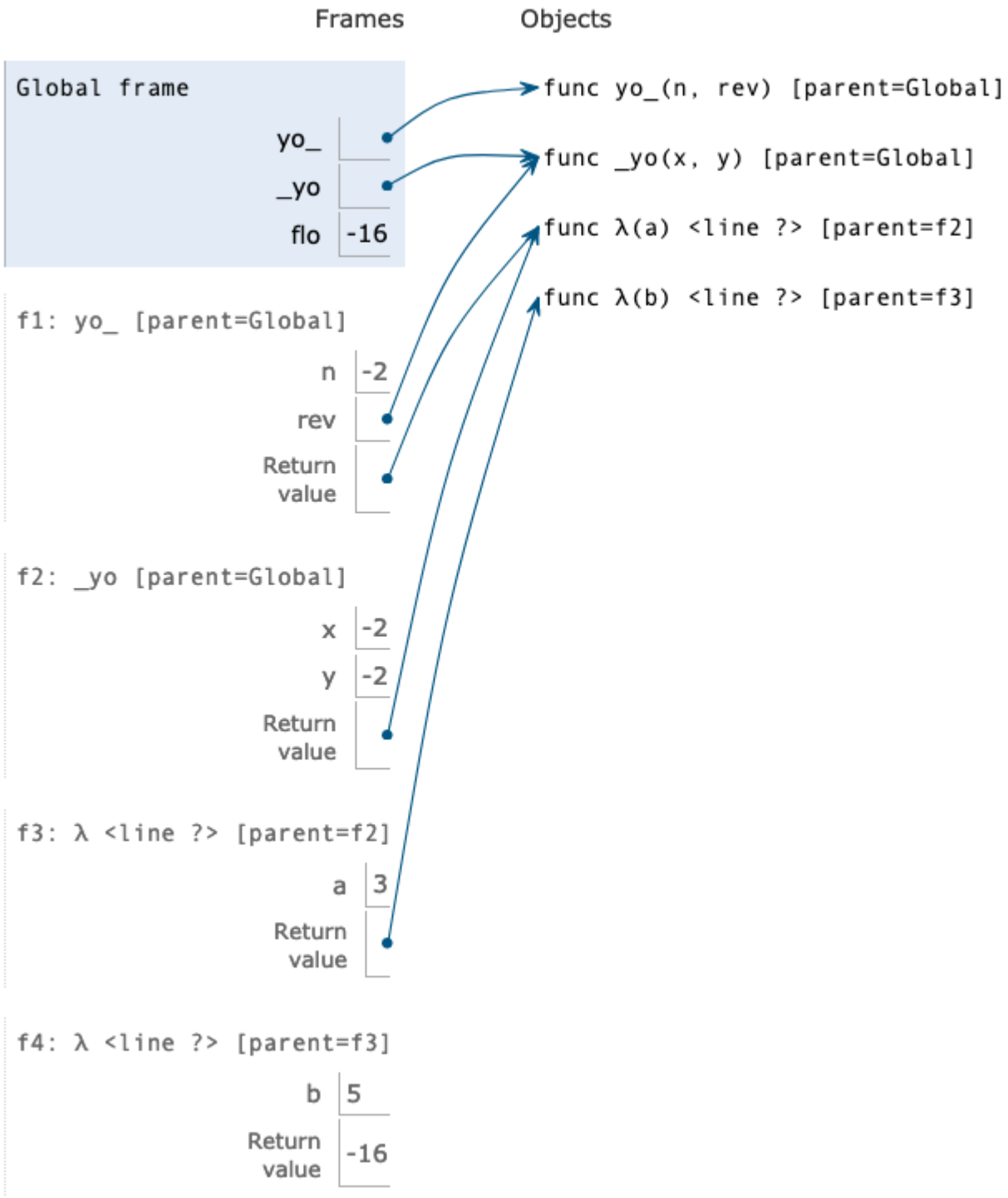
(a) What is your full name?

(b) What is your student ID number?

(c) By writing my name below, I pledge on my honor that I will abide by the rules of this exam and will neither give nor receive assistance. I understand that doing otherwise would be a disservice to my classmates, dishonor me, and could result in me failing the class.

1. (a) (8.0 points) Flow that Yo-Yo

The following environment diagram was generated by a program:



Click here to open the diagram in a new window

In this series of questions, you'll fill in the blanks of the program that follows so that its execution matches the environment diagram. You may want to fill in the blanks in a different order; feel free to answer the questions in whatever order works for you.

```
def yo_(n, rev):
    if n < 0:
        return ______
                 (a)
    elif n == 0:
        return float("inf")
    return n * -2

def _yo(x, y):
    if _______:
         (b)
        y += 1
    if _______:
         (c)
        return lambda a: ______
                              (d)
    return lambda a: lambda b: a + b

flo = yo_(_____, _____)(3)(5)
           (e)    (f)
```

(1.0 pt) Which of these could fill in blank (a)?

- ⓘ `lambda a: a - 3`
- ◯ `lambda a: rev(a, 3)(-1)`
- ◯ `rev`
- ◯ `yo_(rev)`
- ◯ `yo_`
- ◯ `rev(n, -3)`
- ◯ `lambda a: yo_(rev)`
- ◯ `_yo`

ii. (2.0 pt) Which of these could fill in blank (b)? *Select all that apply.*

- ☐ `y < 0`
- ☐ `x <= 0`
- ☐ `x == y`
- ☐ `y <= 0`
- ☐ `y == 0`
- ☐ `x < 0`

iii. (2.0 pt) Which of these could fill in blank (c)? *Select all that apply.*

- ☐ `x > 0`
- ☐ `x == -y`
- ☐ `y == -x`
- ☐ `x == y`
- ☐ `y > 0`
- ☐ `x < 0 and y < 0`

iv. (1.0 pt) Which of these could fill in blank (d)?

- ◯ `x + y`
- ◯ `lambda b: a**x + b**y`
- ◯ `a*x + b*y`
- ◯ `lambda b: a + b`
- ◯ `x * y`
- ◯ `lambda b: a*x * b*y`
- ◯ `a**x + b**y`
- ◯ `lambda b: a*x + b*y`

v. (1.0 pt) Which of these could fill in blank (e)?

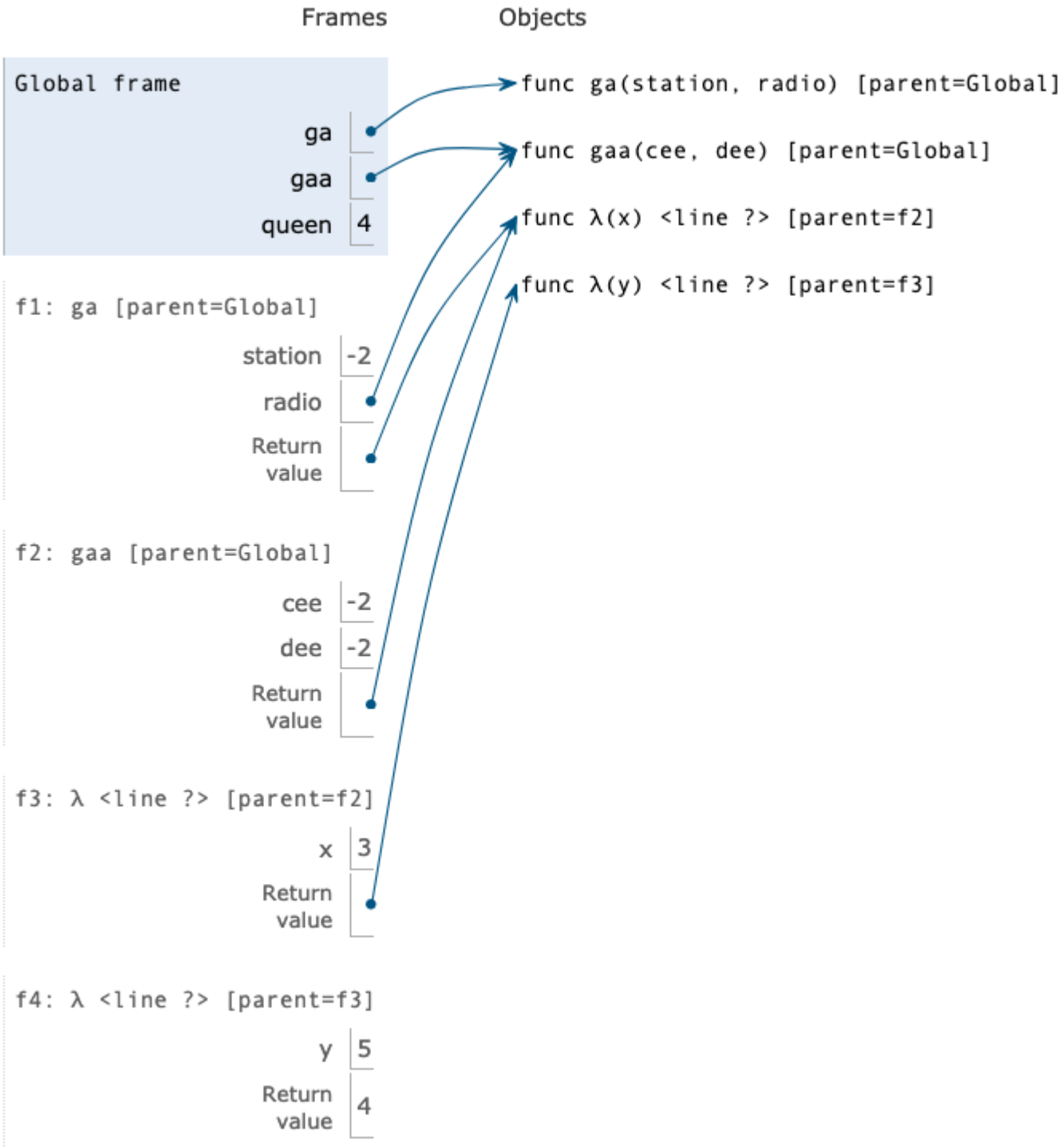
- ◯ `-4`
- ◯ `_yo`
- ◯ `rev`
- ◯ `-2`
- ◯ `yo_`
- ◯ `-3`

vi. (1.0 pt) Which of these could fill in blank (f)?

- ◯ `yo_`
- ◯ `rev`
- ◯ `lambda x: x`
- ◯ `_yo`
- ◯ `lambda x: _yo(x)`
- ◯ `flo`

(8.0 points) Radio Ga-Ga

The following environment diagram was generated by a program:



Click here to open the diagram in a new window

In this series of questions, you'll fill in the blanks of the program that follows so that its execution matches the environment diagram. You may want to fill in the blanks in a different order; feel free to answer the questions in whatever order works for you.

```
def ga(station, radio):
    if station < 0:
        return ______
                 (a)
    elif station == 0:
        return float("inf")
    return station * -98

def gaa(cee, dee):
    if _______:
         (b)
        dee += 1
    if _______:
         (c)
        return lambda x: ______
                              (d)
    return lambda x: lambda y: x * y

queen = ga(_____, _____)(3)(5)
           (e)    (f)
```

(1.0 pt) Which of these could fill in blank (a)?

(b) i.

- ◯ `lambda x: station + x - 3`
- ◯ `lambda x: radio(station, 3)(-1)`
- ◯ `radio`
- ◯ `ga(radio)`
- ◯ `ga`
- ◯ `radio(station, -3)`
- ◯ `lambda x: ga(gaa)`
- ◯ `gaa`

ii. (2.0 pt) Which of these could fill in blank (b)? *Select all that apply.*

- ☐ `cee < 0`
- ☐ `dee <= 0`
- ☐ `cee == dee`
- ☐ `cee <= 0`
- ☐ `cee == 0`
- ☐ `dee < 0`

iii. (2.0 pt) Which of these could fill in blank (c)? *Select all that apply.*

- ☐ `cee > 0`
- ☐ `cee == -dee`
- ☐ `dee == -cee`
- ☐ `cee == dee`
- ☐ `dee > 0`
- ☐ `cee < 0 and dee < 0`

iv. (1.0 pt) Which of these could fill in blank (d)?

- ◯ `cee - dee`
- ◯ `lambda y: x * cee - y * dee`
- ◯ `cee * x - dee * x`
- ◯ `lambda y: cee + y`
- ◯ `cee * dee`
- ◯ `lambda y: cee * x * dee * y`
- ◯ `cee**x + dee**y`
- ◯ `lambda y: x * cee + y * dee`

v. (1.0 pt) Which of these could fill in blank (e)?

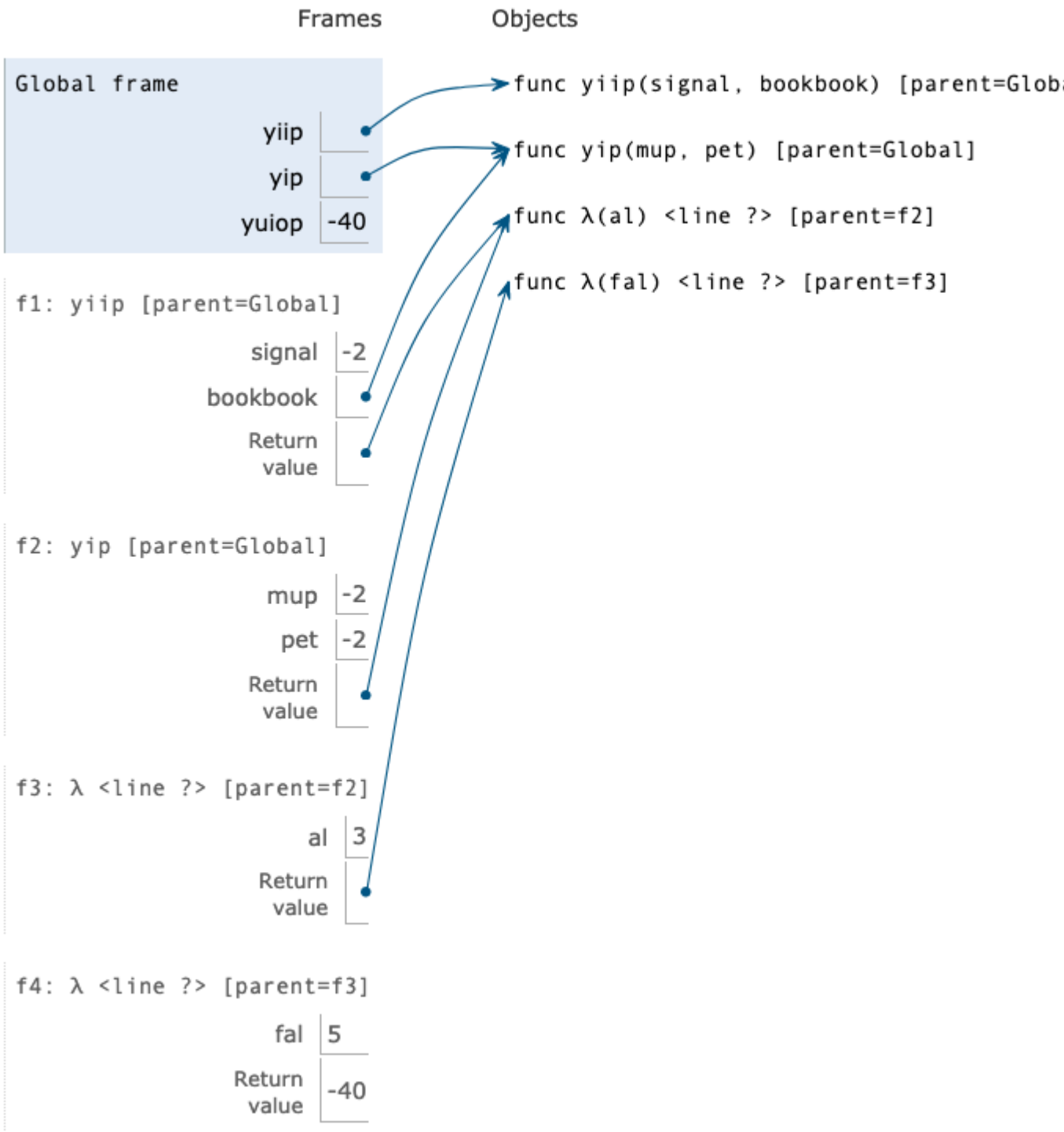
- ◯ `gaa(-2)`
- ◯ `gaa`
- ◯ `radio(ga)`
- ◯ `-2`
- ◯ `ga`
- ◯ `station`

vi. (1.0 pt) Which of these could fill in blank (f)?

- ◯ `ga`
- ◯ `ga()`
- ◯ `lambda n: n`
- ◯ `gaa`
- ◯ `gaa()`
- ◯ `lambda n: gaa(n)`

(8.0 points) YipYip Book

The following environment diagram was generated by a program:



Click here to open the diagram in a new window

In this series of questions, you'll fill in the blanks of the program that follows so that its execution matches the environment diagram. You may want to fill in the blanks in a different order; feel free to answer the questions in whatever order works for you.

```
def yiip(signal, bookbook):
    if signal < 0:
        return ______
                 (a)
    elif signal == 0:
        return float("inf")
    return signal * -98

def yip(mup, pet):
    if _______:
         (b)
        mup += 1
    if _______:
         (c)
        return lambda al: ______
                              (d)
    return lambda al: lambda fal: al - fal

yuiop = yiip(_____, _____)(3)(5)
               (e)    (f)
```

(1.0 pt) Which of these could fill in blank (a)?

(c) i.
- ◯ `lambda al: signal - al * 3`
- ◯ `lambda al: bookbook(signal, 3)(al)`
- ◯ `bookbook`
- ◯ `yiip(bookbook)`
- ◯ `yip`
- ◯ `bookbook(-3, signal)`
- ◯ `bookbook(signal + - 3)`

ii. (2.0 pt) Which of these could fill in blank (b)? *Select all that apply.*

- ☐ `mup < 0`
- ☐ `pet <= 0`
- ☐ `mup == pet`
- ☐ `mup <= 0`
- ☐ `pet == 0`
- ☐ `pet < 0`

iii. (2.0 pt) Which of these could fill in blank (c)? *Select all that apply.*

- ☐ `mup > 0`
- ☐ `mup == -pet`
- ☐ `pet == -mup`
- ☐ `mup == pet`
- ☐ `pet > 0`
- ☐ `mup <= 0 and pet <= 0`

iv. (1.0 pt) Which of these could fill in blank (d)?

- ◯ `al - fal**fal`
- ◯ `lambda fal: mup**al + pet**fal`
- ◯ `mup * al + pet * fal`
- ◯ `lambda fal: mup + pet * fal`
- ◯ `mup * pet`
- ◯ `lambda fal: mup * al * pet * fal`
- ◯ `mup**al + pet**al`
- ◯ `lambda fal: al * mup + fal * pet`

v. (1.0 pt) Which of these could fill in blank (e)?

- ◯ `yiip(2 * -1)`
- ◯ `yiip`
- ◯ `bookbook(yip)`
- ◯ `-2`
- ◯ `yip`
- ◯ `signal - 2`

vi. (1.0 pt) Which of these could fill in blank (f)?

- ◯ `yip`
- ◯ `yip()`
- ◯ `lambda y: y`
- ◯ `yiip`
- ◯ `yiip()`
- ◯ `lambda y: yiip(y)`
- ◯ `-2`

2. (1.0 points) The Case of the Missing Docstring

Consider the following function and its doctests:

```
def mystery(l):
  """
  >>> mystery([1, 2, 3, 4])
  2.5
  >>> mystery([2, 4])
  3.0
  >>> mystery([-5, -2, -9])
  0
  >>> mystery([])
  0
  >>> mystery([345])
  345.0
  """
  s = 0
  t = 0
  for item in l:
    if item > 0:
      s += item
      t += 1
  if t == 0:
    return 0
  return s/t
```

(a) Which of the following docstrings would best describe that function?

◯ Returns the average of all elements in L or returns zero if no elements exist.

◯ Returns the average of elements in L with an odd index or returns zero if no such elements exist.

◯ Returns the average of elements in L that are >= 0 or returns zero if no such elements exist.

◯ Returns the average of positive elements in L or returns zero if no such elements exist.

3. (2.0 points) Magical Test Weaver

Consider the following function signature and docstring:

```
def magic_weave(a, b, c):
    """
    Assuming A and B are positive integers with the same number of base-10 digits
    and C is a positive integer < 10, return the number whose base-10
    representation is the interleaving of digits in A and B (alternating
    first one from A then one from B) from all positions where the
    two digits in A and B at that position are both >= C.  Return 0 if there
    are no such positions. Raises an exception if preconditions are not met.
    """
```

Here is one example of a passing doctest:

```
>>> magic_weave(345, 987, 3)
394857
```

(a) Based on the docstring of that function, which of these would be passing doctests? *Select all that apply.*

☐
```
>>> magic_weave(123, 456, 5)
56
```

☐
```
>>> magic_weave(234, 456, 5)
3546
```

☐
```
>>> magic_weave(456, 567, 5)
5667
```

☐
```
>>> magic_weave(0, 0, 5)
0
```

☐
```
>>> magic_weave(101, 202, 0)
120012
```

☐
```
>>> magic_weave(567, 899, 10)
586979
```

4. (10.0 points) Domain On the Range

The domain of a function is the set of all possible argument values, while the range of a function is the set of values that it can return. In this two part-question, you will implement higher-order functions to restrict the domain and range of other functions.

(a) (4.0 points) restrict_domain

Implement `restrict_domain`, a function that accepts three parameters (`f`, `low_d`, `high_d`) and returns a higher-order function that returns the same thing as `f` when given an argument between `low_d` and `high_d`, inclusive, and otherwise returns `float("-inf")`.

```
def restrict_domain(f, low_d, high_d):
    """Returns a function that restricts the domain of F,
    a function that takes a single argument x.
    If x is not between LOW_D and HIGH_D (inclusive),
    it returns -Infinity, but otherwise returns F(x).

    >>> from math import sqrt
    >>> f = restrict_domain(sqrt, 1, 100)
    >>> f(25)
    5.0
    >>> f(-25)
    -inf
    >>> f(125)
    -inf
    >>> f(1)
    1.0
    >>> f(100)
    10.0
    """
    ---------
    #   (a)
        ---------
        #   (b)
            ---------
            #  (c)
        ---------
        #  (d)
    return wrapper_method_name
```

i. (1.0 pt) Fill in blank (a).

ii. (1.0 pt) Fill in blank (b).

iii. (1.0 pt) Fill in blank (c).

iv. (1.0 pt) Fill in blank (d).

(b) (5.0 points) restrict_range

Implement `restrict_range`, a function that accepts three parameters (`f`, `low_r`, `high_r`) and returns a higher-order function that returns the same thing as `f` when that result is between `low_r` and `high_r` (inclusive), and otherwise returns `float("-inf")`.

```
def restrict_range(f, low_r, high_r):
   """Returns a function that restricts the range of F, a function
   that takes a single argument X. If the return value of F(X)
   is not between LOW_R and HIGH_R (inclusive), it returns -Infinity,
   but otherwise returns F(X).

   >>> cube = lambda x: x * x * x
   >>> f = restrict_range(cube, 1, 1000)
   >>> f(1)
   1
   >>> f(-5)
   -inf
   >>> f(5)
   125
   >>> f(10)
   1000
   >>> f(11)
   -inf
   """
    ---------
    #   (a)
        ---------
        #   (b)
        ---------
        #   (c)
            ---------
            #  (d)
        ---------
        #  (e)
    return wrapper_method_name
```

i. (1.0 pt) Fill in blank (a).

ii. (1.0 pt) Fill in blank (b).

iii. (1.0 pt) Fill in blank (c).

iv. (1.0 pt) Fill in blank (d).

v. (1.0 pt) Fill in blank (e).

(c) (1.0 points) restrict_both

Now that you have those two functions defined, you'll implement `restrict_both`, a higher-order function that accepts a function `f` and four numeric arguments (`low_d`, `high_d`, `low_r`, `high_r`) and returns the result of applying both `restrict_domain` and `restrict_range` on `f`.

```
def restrict_both(f, low_d, high_d, low_r, high_r):
    """
    Returns a version of F with a domain restricted to (LOW_D, HIGH_D)
    and a range restricted to (LOW_R, HIGH_R).

    >>> diva = lambda x: (10000 // x) * 9
    >>> f = enforce_both(diva, 1, 1000, 100, 999)
    >>> f(0)
    -inf
    >>> f(10000)
    -inf
    >>> f(200)
    450
    >>> f(100)
    900
    >>> f(1000)
    -inf
    """
    -------------
```

i. Fill in the blank. You may use one line or multiple lines, as long as the solution is correct. Your solution should use `restrict_domain` and `restrict_range` somehow, and assume that those were implemented correctly.

If you wrote your code in code.cs61a.org, you can paste it in here (do not worry if it seems too indented, as long as the indentation worked there).

5. (8.0 points) Digit replacer

The function `digit_replacer(predicate, transformer)` should return a function that replaces all the digits in a number where `predicate(digit)` is true with the result of `transformer(digit)`. The returned function should accept a single argument, the number `n`, and return the number with the digits replaced.

Here is the function signature and doctests:

```
def digit_replacer(predicate, transformer):
    """Returns a function that accepts a single number N (where N > 0) and
    returns a number where all digits that return true for PREDICATE(DIGIT)
    have been replaced by TRANSFORMER(DIGIT). TRANSFORMER is assumed to always
    return a valid digit >= 0 and <= 9.

    >>> is_even = lambda d: d % 2 == 0
    >>> lt_five = lambda d: d < 5
    >>> always_two = lambda d: 2
    >>> floor_divide_two = lambda d: d // 2
    >>> digit_replacer(is_even, floor_divide_two)(21098)
    11094
    >>> digit_replacer(lt_five, always_two)(1064592)
    2262592
    """
```

(a) (4.0 pt) Use iteration (**without recursion**) to implement `digit_replacer`.

Your solution should only use numbers, arithmetic expressions, and booleans. It should *not* use strings, lists, or other data types, and will earn 0 points if it does. It will also earn 0 points if it uses recursion, since that's tested in the second part of this question.

Here's an approximate structure of a solution, if that helps guide your implementation. It does not necessarily reflect the exact number of lines or indentation.

```
def digit_replacer(predicate, transformer):
    _____
        _____
        _____

        while ______:
            _____
                _____
            _____
                _____
            _____
            _____
       return _____
    ______
```

Remember that you can use code.cs61a.org to try out your code and see if it passes the doctests. You can then paste the code here. Please include the function signature.

(b) (4.0 pt) Use a recursive approach to implement `digit_replacer`.

Your solution should only use numbers, arithmetic expressions, and booleans. It should *not* use strings, lists, or other data types, and will earn 0 points if it does. It will also earn 0 points if it does not use recursion, since that was tested in the first part of this question.

Here's an approximate structure of a solution, if that helps guide your implementation. It does not reflect the exact number of lines or indentation.

```
def digit_replacer(predicate, transformer):
    ------
        ------
            ------
        ------
            ------
        ------
       return ------
    ------
```

Remember that you can use code.cs61a.org to try out your code and see if it passes the doctests. You can then paste the code here. Please include the function signature.

6. (6.0 points) Run checker

Let's use the term "chain function" to mean a function that takes a single numerical argument and returns another chain function (so that if `h` is a chain function, one can call `h(3)(4)(1)(2)(0)...`, which we'll call a "chain of calls.")

The function `run_checker` accepts two functions as arguments (`condition` and `result`) and returns a chain function. Each call in a chain starting with the function returned by `run_checker` first applies `condition` to the two previous arguments in the chain and its own argument. If that returns a false value, it prints "No run!" and otherwise prints the result of `result` applied to the same three arguments. For the first and second calls in the chain, the missing arguments are taken to be -1.

```
def run_checker(condition, result):
    """
    Returns a chain function. Each call in a chain that starts with
    this returned function prints "No run!" if CONDITION returns a false
    value when applied to the previous two arguments and the current argument,
    and otherwise prints the result of applying RESULT to these same
    three arguments.  For calls in the chain where there are fewer than two
    preceding calls in the chain, the missing arguments are taken to be -1.

    >>> f = run_checker(lambda a, b, c: a > b > c and a >= 10, lambda a, b, c: a*(b+c))
    >>> f = f(15)
    No run!
    >>> f = f(10)
    No run!
    >>> f = f(5)
    225
    >>> f = f(2)
    70
    >>> f = f(1)
    No run!
    >>> f = f(11)
    No run!
    >>> f = f(12)
    No run!
    >>> f = f(10)
    No run!
    >>> f = f(2)
    144
    """
    def f(______):
       #   (a)
        def g(______):
           #   (b)

            ------
            ------
            ------
            ------
            ------
            # (c)

        return g
    return f(______)
    #          (d)
```

(a) **(1.0 pt)** Fill in blank (a).

(b) **(1.0 pt)** Fill in blank (b).

(c) **(3.0 pt)** Fill in the (c) blanks. You can use more or fewer lines for your solution than the number suggested by the blanks.

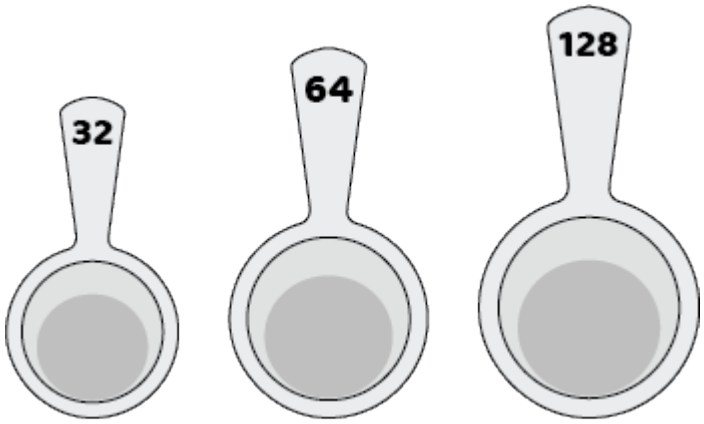
If you use code.cs61a.org to try out your code, you can then paste the code here. Don't worry if it seems too indented, as long as the indentation worked there.

(d) **(1.0 pt)** Fill in blank (d).

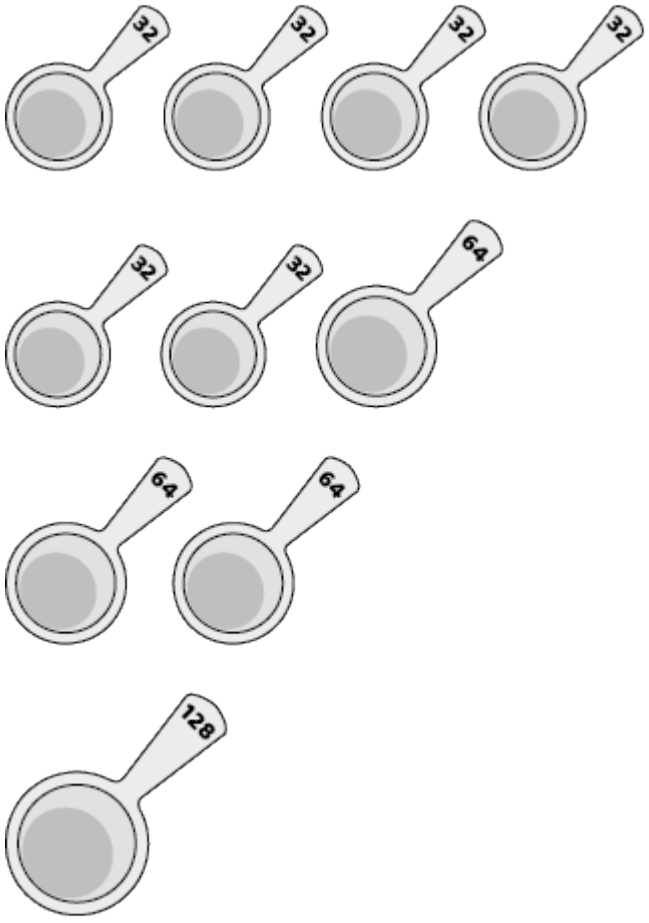
7. (5.0 points) Measure Twice, Cup Once

The function `measure_methods` accepts two arguments (`total_needed`, the total number of grams needed for a recipe, and `cup_sizes`, a list of measuring cup sizes available) and returns the number of possible ways to make exactly `total_needed` using the cup sizes. The `cup_sizes` list is measured in grams, sorted from smallest to largest, and each size is a power of 2. A cup size may be used multiple times in order to come up with `total_needed`.

For example, the list of `[32, 64, 128]` represents these cups:



There are 4 possible ways to make a total of 128 grams from those cups:



Here's the partially defined function:

```
def measure_methods(total_needed, cup_sizes):
```

```
    """Returns the number of ways to make exactly TOTAL_NEEDED with
    the given list of CUP_SIZES (sorted by smallest to largest).

    >>> measure_methods(128, [32, 64, 128])
    4
    >>> measure_methods(256, [32, 64, 128])
    9
    >>> measure_methods(384, [32, 64, 128])
    16
    >>> measure_methods(256, [16, 32, 64])
    25
    >>> measure_methods(125, [32, 64, 128])
    0
    """
    def helper_method_name(______):
        #        (a)
        ------
        ------
        ------
        ------
        ------
        # (b)
    return helper_method_name(______)
    #               (c)
```

(a) (1.0 pt) Fill in blank (a).

(b) (3.0 pt) Fill in the (b) blanks. You can use more or fewer lines for your solution than the number suggested by the blanks, and your code may involve multiple indentation levels.

If you use code.cs61a.org to try out your code, you can then paste the code here. Don't worry if it seems too indented, as long as the indentation worked there.

(c) (1.0 pt) Fill in blank (c).

8. (1.0 points) Extra Point!

(a) (1.0 pt) "Baked Century" (nine letters, two words).

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Spring 2021

MIDTERM 2

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) By writing my name below, I pledge on my honor that I will abide by the rules of this exam and will neither give nor receive assistance. I understand that doing otherwise would be a disservice to my classmates, dishonor me, and could result in me failing the class.

1. (2.0 points) Tuple Trouble

Consider this partially implemented two-line program:

```
greeting_from_staff = ("Good Luck", "You Got This", "Just Breathe")
______ = "Once More with Feeling"
```

The full program results in this error:

```
AttributeError: 'tuple' object has no attribute '__setitem__'
```

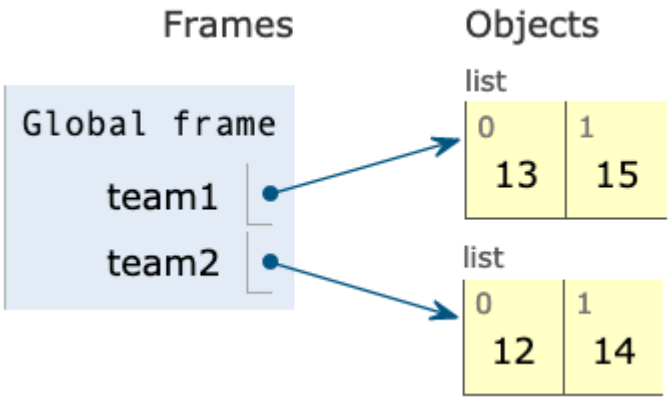
(a) What code could fill in the blank to cause that error?

2. (4.0 points) Teenage Mutant Ninja Lists

A program starts with two list assignments:

```
team1 = [13, 15]
team2 = [12, 14]
```

That code corresponds to this environment diagram:

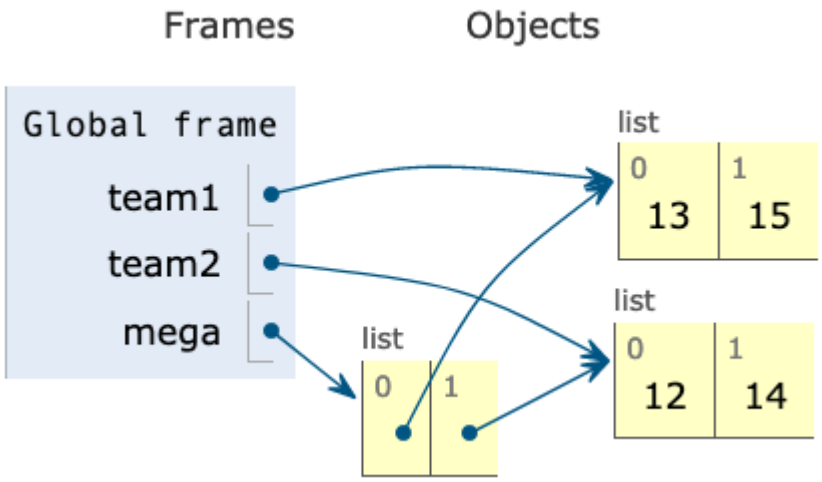


In this question, you'll suggest ways of mutating the lists to result in different environment diagrams.

(a) (1.0 pt)

```
team1 = [13, 15]
team2 = [12, 14]
mega = _____________
```

This modified program corresponds to this diagram:



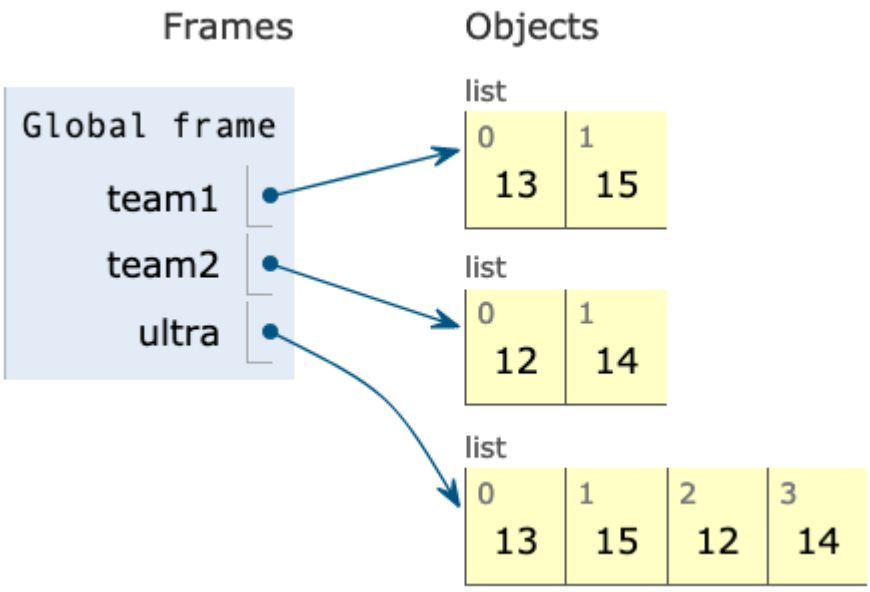
Which of these could fill in the blank? **Select all that apply**

- ☐ `[team1, team2]`
- ☐ `[team1] + [team2]`
- ☐ `[team1[0:], team2[0:]]`
- ☐ `[team1 + team2]`
- ☐ `[list(team1 + team2)]`
- ☐ `team1, team2`
- ☐ `[list(team1), list(team2)]`
- ☐ `team1 + team2`
- ☐ `team1 - team2`
- ☐ `team1 + team2[:]`
- ☐ `team1[:] + team2`
- ☐ `team1[:] + team2[:]`
- ☐ `team1 += team2`

(b) (1.0 pt)

```
team1 = [13, 15]
team2 = [12, 14]
ultra = _____________
```

This modified program corresponds to this diagram:



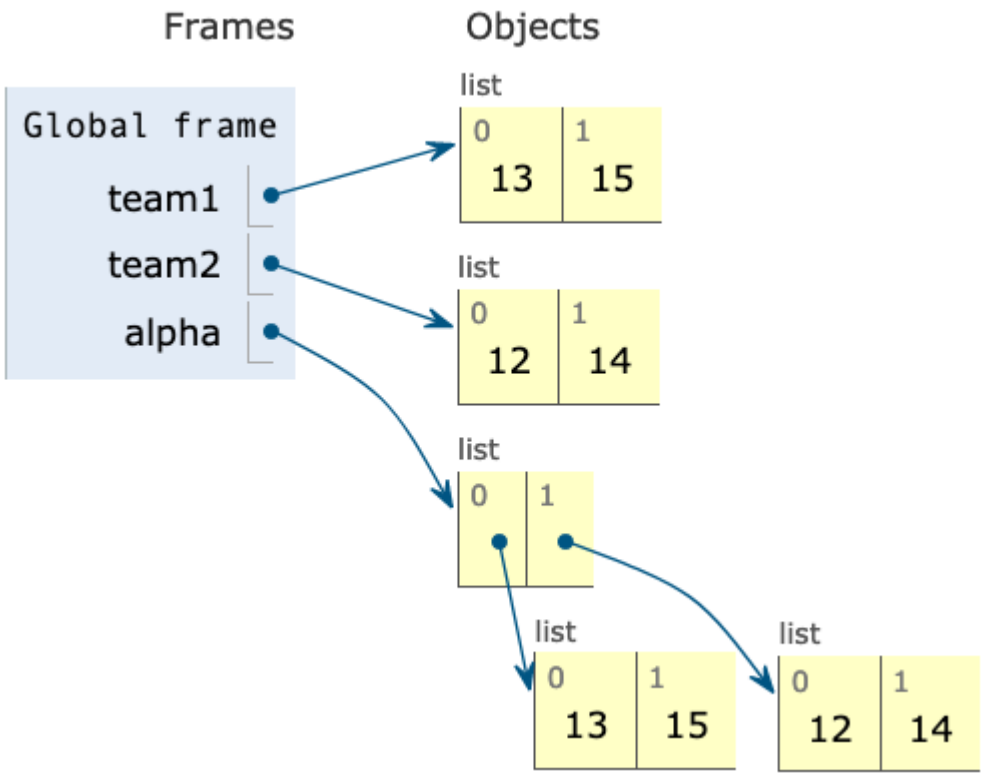
Which of these could fill in the blank? **Select all that apply**

- ☐ `team1 + team2`
- ☐ `team1[:] + team2[:]`
- ☐ `team1 + team2[:]`
- ☐ `team1[:] + team2`
- ☐ `[team1, team2]`
- ☐ `[team1[0:], team2[0:]]`
- ☐ `[team1 + team2]`
- ☐ `[list(team1 + team2)]`
- ☐ `team1, team2`
- ☐ `[list(team1), list(team2)]`
- ☐ `team1 - team2`
- ☐ `[team1] + [team2]`
- ☐ `team1 += team2`

(c) (1.0 pt)

```
team1 = [13, 15]
team2 = [12, 14]
alpha = _____________
```

This modified program corresponds to this diagram:

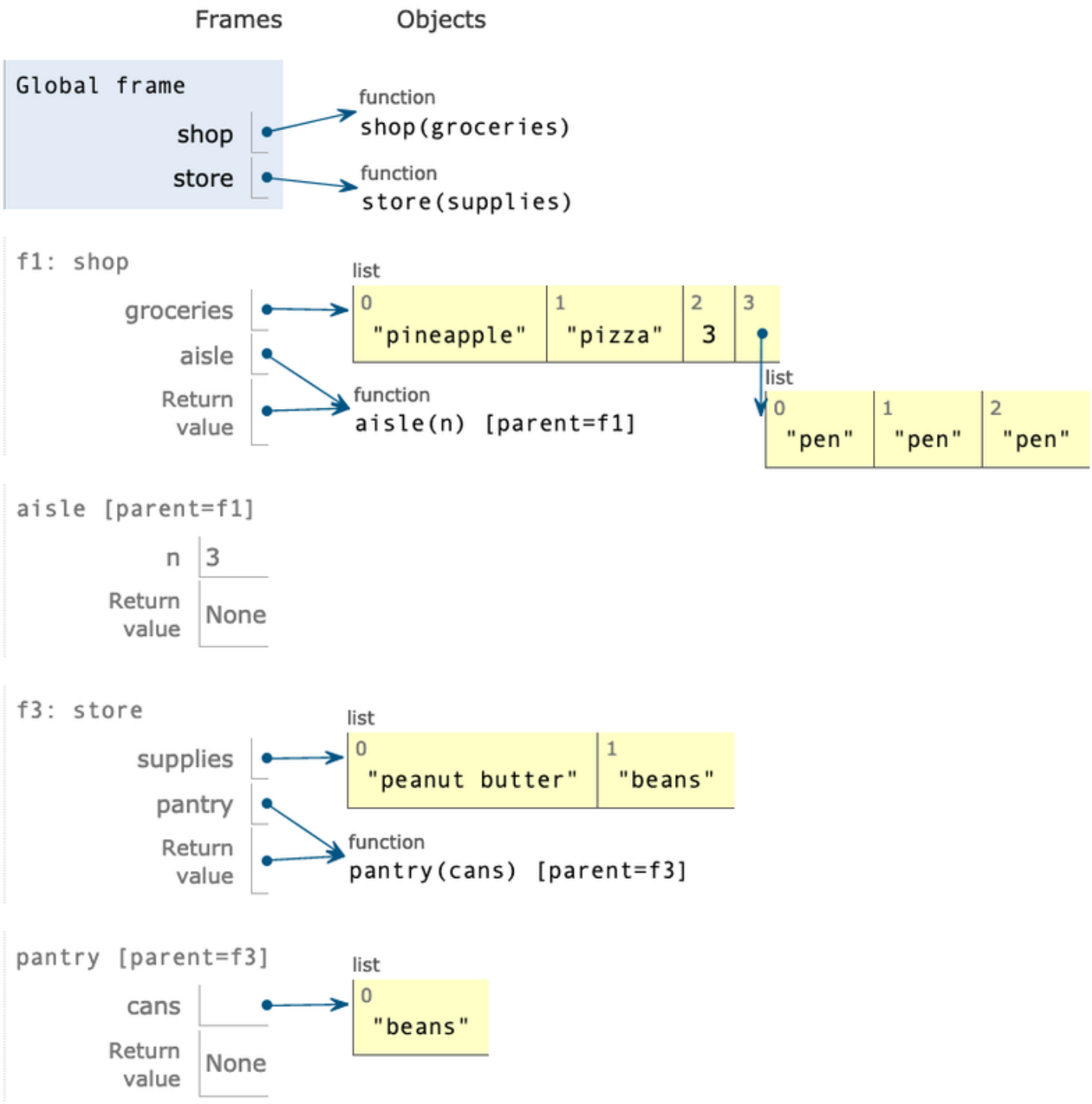


Which of these could fill in the blank? **Select all that apply**

- ☐ `[list(team1), list(team2)]`
- ☐ `[team1[0:], team2[0:]]`
- ☐ `[team1, team2]`
- ☐ `[team1 + team2]`
- ☐ `[list(team1 + team2)]`
- ☐ `team1, team2`
- ☐ `team1 + team2`
- ☐ `team1 - team2`
- ☐ `[team1] + [team2]`
- ☐ `team1 + team2[:]`
- ☐ `team1[:] + team2`
- ☐ `team1[:] + team2[:]`
- ☐ `team1 += team2`

3. (4.0 points) Higher Order Shopping

The following environment diagram was generated by a program:



Click here to open the diagram in a new window

In this series of questions, you'll fill in the blanks of the program that follows so that its execution matches the environment diagram.

```
def shop(groceries):

    def aisle(n):
```

```
        --------
        #  (a)
        --------
        #  (b)

    return aisle

def store(supplies):

    def pantry(cans):
        --------
        #  (c)
        --------
        #  (d)

    return pantry

shop(["pineapple", "pizza"])(3)
store(["peanut butter"])(["beans"])
```

(a) (2.0 pt) Which of these could fill in blank (a)? **Select all that apply!**

- ☐ `groceries.extend([n])`
- ☐ `groceries.append(n)`
- ☐ `nonlocal groceries`
- ☐ `nonlocal n`
- ☐ `nonlocal groceries += n`
- ☐ `nonlocal groceries.extend([n])`
- ☐ `nonlocal groceries.append(n)`
- ☐ `nonlocal groceries.extend(n)`
- ☐ `groceries.extend(n)`
- ☐ `groceries.append([n])`
- ☐ `groceries += n`
- ☐ `nonlocal n, groceries`
- ☐ `nonlocal groceries + n`

(b) (1.0 pt) Which of these could fill in blank (b)?

- ◯ `groceries.append(["pen"] * n)`
- ◯ `groceries.append("pen" * n)`
- ◯ `groceries.extend("pen" * n)`
- ◯ `groceries.extend(["pen" * n])`
- ◯ `groceries += "pen" * n`
- ◯ `groceries += ["pen" for x in n]`
- ◯ `groceries.extend(["pen" for x in n])`
- ◯ `groceries.append(["pen" for x in n])`
- ◯ `groceries[3:n] = ["pen"]`
- ◯ `groceries[:3:n] = ["pen"]`
- ◯ `groceries[0:n] = ["pen"]`

(c) (1.0 pt) Which of these could fill in blank (c)?

- ◯ `nonlocal supplies`
- ◯ `nonlocal store`
- ◯ `nonlocal n`
- ◯ `supplies = list(supplies)`
- ◯ `supplies = supplies[:]`
- ◯ `nonlocal supplies = list(cans)`
- ◯ `nonlocal supplies = list()`
- ◯ `nonlocal list supplies`
- ◯ `nonlocal(supplies)`

(d) (1.0 pt) Which of these could fill in blank (d)?

- ◯ `supplies += cans`
- ◯ `nonlocal supplies += cans`
- ◯ `supplies.append(cans)`
- ◯ `nonlocal supplies.append(cans)`
- ◯ `supplies[:] = cans`
- ◯ `nonlocal supplies[:] = cans`
- ◯ `supplies = cans`
- ◯ `nonlocal supplies = cans`
- ◯ `supplies.extend([cans])`
- ◯ `nonlocal supplies.extend([cans])`

4. (8.0 points) Gotta Keep 'Em Separated

Implement `separate()`, a function that accepts two parameters (`separator` and `lnk`) and returns a new linked list where any two consecutive repeated numbers in `lnk` are separated by a link with the value of `separator`. All values in `lnk` will be numbers.

The function should assume the standard CS61A Link definition, viewable here: code.cs61a.org/link_class

```
def separate(separator, lnk):
    """Returns a new linked list (using Link) that separates
    any two consecutive repeated numbers in non-empty LNK
    by inserting a Link with the value of SEPARATOR.
    All values in LNK will be numbers.

    # Case 1: Separates list that starts with pair
    >>> separate(999, Link(1, Link(1)))
    Link(1, Link(999, Link(1)))

    # Case 2: Separates list with overlapping pairs
    >>> separate(999, Link(1, Link(1, Link(1))))
    Link(1, Link(999, Link(1, Link(999, Link(1)))))

    # Case 3: Doesn't mutate the input list
    >>> link = Link(1, Link(1))
    >>> separate(999, link)
    Link(1, Link(999, Link(1)))
    >>> link
    Link(1, Link(1))

    # Case 4: Only separates pairs, not non-pairs
    >>> separate(-999, Link(2, Link(2, Link(4, Link(5)))))
    Link(2, Link(-999, Link(2, Link(4, Link(5)))))

    # Case 5: Separates pairs at end of list
    >>> separate(-1, Link(2, Link(3, Link(4, Link(4)))))
    Link(2, Link(3, Link(4, Link(-1, Link(4)))))

    # Case 6: Returns same-valued list if no pairs found
    >>> separate(999, Link(1, Link(2)))
    Link(1, Link(2))

    # Case 7: Handles single element lists correctly
    >>> lnk = Link(1)
    >>> lnk2 = separate(999, lnk)
    >>> lnk
    Link(1)
    >>> lnk is not lnk2
    True
    """
```

(a) Here's an approximate structure of a recursive solution, if that helps guide your implementation. It does not reflect the exact number of lines or indentation. **Your solution must use a recursive approach, not an iterative approach. An iterative approach will receive 0 points.**

```
if ______:
  _____________
elif ____:
  _____________
elif ____:
  _____________
_______________
```

You can use code.cs61a.org to try out your code and see if it passes the doctests. You can then paste the code here.

5. (8.0 points) Online Classes

This question is inspired by online education websites that offer a range of materials such as videos, articles, exercises, and quizzes.

We will represent two kinds of educational content as a Python class: `Video` and `Exercise`. Each of those classes inherits from the base class `LearnableContent`.

The partially implemented classes are shown below. You'll fill out the class definitions using inheritance and attributes. **Always use the definition from the base class if it's reasonable to do so.**

```
class LearnableContent:
    """A base class for specific kinds of learnable content.
    All kinds have title and author attributes,
    but each kind may have additional attributes.
    """

    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __str__(self):
        return f"{self.title} by {self.author}"

class Exercise(LearnableContent):
    """
    >>> lambda_calc = Exercise("Lambda Calculus", "Rosie F", 5)
    >>> lambda_calc.title
    'Lambda Calculus'
    >>> lambda_calc.author
    'Rosie F'
    >>> lambda_calc.num_questions
    5
    >>> str(lambda_calc)
    'Lambda Calculus by Rosie F'
    """

    def __init__(self, title, author, num_questions):
        ________________________
        #          (a)
        ________________________
        #          (b)

class Video(LearnableContent):
    """
    >>> vid = Video("The Golden Ratio", "Sal Khan", 881)
    >>> Video.license
    'CC-BY-NC-SA'
    >>> vid.title
    'The Golden Ratio'
    >>> vid.author
    'Sal Khan'
    >>> vid.num_seconds
    881
    >>> str(vid)
    'The Golden Ratio by Sal Khan (881 seconds)'
    """
```

```
____________________
#      (c)

def __init__(self, title, author, num_seconds):
    ________________________
    #          (d)
    ________________________
    #          (e)

def __str__(self):
    ________________________
    #          (f)
```

Please check your work on code.cs61a.org and run the doctests there.

(a) (1.0 pt) What line of code could go in blank (a)? **You only need a single line of code.**

(b) (1.0 pt) What line of code could go in blank (b)? **You only need a single line of code.**

(c) (1.0 pt) What line of code could go in blank (c)? **You only need a single line of code.**

(d) (2.0 pt) What line of code could go in blank (d)? **You only need a single line of code.**

(e) (1.0 pt) What line of code could go in blank (e)? **You only need a single line of code.**

(f) (2.0 pt) What line of code could go in blank (f)? **You only need a single line of code.**

You may use any form of string formatting that you are comfortable with, but your solution must use `super()`.

6. (6.0 points) Gas Composition

This question is inspired by research projects that are tracking the emissions of greenhouse gases into the environment so that they can identify the biggest sources of greenhouse gases and find ways to sustainably reduce them.

We have written two classes to help track gas emissions: `EmissionSource` and `EmissionsTracker`.

The `EmissionSource` class is fully implemented. Each instance stores a name, a dictionary of emission rates per hour per gas type, and the number of hours it's been running (initialized to 0). Read through the implementation below:

```
class EmissionSource:

    def __init__(self, name, co2, ch4, n2o):
        self.name = name
        self.emissions = {
            "carbon dioxide": co2,  # kg CO2/ton/hour
            "methane": ch4,         # g CH4/ton/hour
            "nitrous oxide": n2o,   # g N2O/ton/hour
        }
        self.hours = 0

    def run_for(self, num_hours):
        self.hours += num_hours

    def calc_emissions(self, gas):
        return self.emissions.get(gas, 0) * self.hours
```

The `EmissionsTracker` class is partially implemented, but has doctests that demonstrate how it should work. It's job is to track emission_source_list and calculate statistics about their emissions. Read through the doctests, docstrings, and partial implementation below:

```
class EmissionsTracker:
    """
    >>> tracker = EmissionsTracker()
    >>> pp1 = EmissionSource("Anthracite Coal", 2602, 276, 40)
    >>> pp2 = EmissionSource("Lignite Coal", 1389, 156, 23)
    >>> tracker.add_sources([pp1, pp2])
    >>> pp1.run_for(5)
    >>> pp2.run_for(6)

    # Case 1: Calculate max for 2 added so far
    >>> tracker.calc_emissions("methane")
    [('Anthracite Coal', 1380), ('Lignite Coal', 936)]
    >>> tracker.calc_max_source("methane")
    ('Anthracite Coal', 1380)

    # Case 3: Calculate emissions for all 3
    >>> pp3 = EmissionSource("Plastics", 2850, 1216, 160)
    >>> pp3.run_for(3)
    >>> tracker.add_sources([pp3])
    >>> tracker.calc_emissions("carbon dioxide")
    [('Anthracite Coal', 13010), ('Lignite Coal', 8334), ('Plastics', 8550)]
    >>> tracker.calc_emissions("methane")
    [('Anthracite Coal', 1380), ('Lignite Coal', 936), ('Plastics', 3648)]
    >>> tracker.calc_emissions("nitrous oxide")
    [('Anthracite Coal', 200), ('Lignite Coal', 138), ('Plastics', 480)]
```

```
# Case 4: Calculate max for all 3
>>> tracker.calc_max_source("carbon dioxide")
('Anthracite Coal', 13010)
>>> tracker.calc_max_source("methane")
('Plastics', 3648)
>>> tracker.calc_max_source("nitrous oxide")
('Plastics', 480)
"""
def __init__(self):
    self.emission_source_list = []

def add_sources(self, sources_to_add):
    """ Adds the list of SOURCES_TO_ADD to the end of self.emission_source_list"""
    ____________________________
    #         (a)

def calc_emissions(self, gas):
    """ Returns a list of tuples where the first element is the name of
    the emission_src and the second element is the total amount of emissions
    from that type of GAS."""
    ____________________________
    #         (b)

def calc_max_source(self, gas):
    """ Returns a tuple where the first element is the name of the emission_src
    with the highest emissions and the second element is the total amount
    of emissions from that emission_src for that type of GAS."""
    return max(_______________)
    #               (c)
```

The following questions will ask you to fill in the blanks for `EmissionsTracker`. You can try out the code on code.cs61a.org and run the doctests to verify that your proposed code does indeed work.

(a) **(2.0 pt)** What line of code could go in blank (a)? **You only need a single line of code.**

(b) **(2.0 pt)** What line of code could go in blank (b)? **You only need a single line of code. Your code must use methods from the `EmissionSource` class if it's reasonable to do so.**

(c) **(2.0 pt)** What expression could go in blank (c)? You may want to consult the Python documentation for the max function.

7. (4.0 points) Business as Usual

Modern office supplies companies use software to track their sales inventory and can use object oriented programming to represent the types of products they sell.

We will represent one type of product as a Python class: `PaperReam`.

The class is partially implemented below. Please read through the code and doctests.

```
class PaperReam:
    """
    >>> ream = PaperReam("red", 200)
    >>> ream.color_name
    'red'
    >>> ream.num_sheets
    200
    >>> ream
    PaperReam('red', 200)
    """

    def __init__(self, color_name, num_sheets):
        self.color_name = color_name
        self.num_sheets = num_sheets

    --------------------
    #          (a)
        ----------------
        #      (b)
```

The following questions will ask you to fill in the blanks for `PaperReam`. You can try out the code on code.cs61a.org and run the doctests to verify that your proposed code does indeed work.

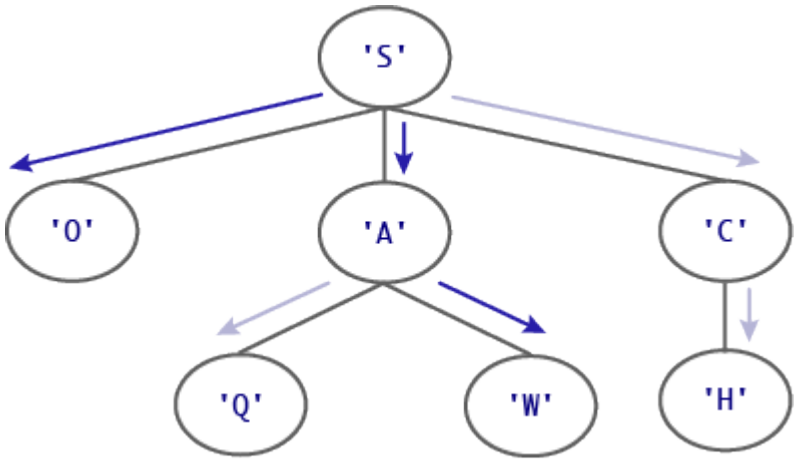
(a) (2.0 pt) What line of code could go in blank (a)? **You only need a single line of code, since it should be a function header.**

(b) (2.0 pt) What line of code could go in blank (b)? **You only need a single line of code.**

8. (7.0 points) The Tree of L-I-F-E

Implement `word_finder`, a generator function that yields each word that can be formed by following a path in a tree from the root to a leaf, where the words are specified in a list.

When given the tree shown in the diagram below and a word list that includes 'SO' and 'SAW', the function should first yield 'SO' and then yield 'SAW'.



Please read through the function header and doctests below. We have provided quite a few doctests to test different situations and demonstrate how the function should work. You can always call `draw(t)` on a particular tree object on code.cs61a.org to help you visualize its structure and understand the results of a doctest.

```
def word_finder(letter_tree, words_list):
    """ Generates each word that can be formed by following a path
    in TREE_OF_LETTERS from the root to a leaf,
    where WORDS_LIST is a list of allowed words (with no duplicates).

    # Case 1: 2 words found
    >>> words = ['SO', 'SAT', 'SAME', 'SAW', 'SOW']
    >>> t = Tree("S", [Tree("O"), Tree("A", [Tree("Q"), Tree("W")]), Tree("C", [Tree("H")])])
    >>> gen = word_finder(t, words)
    >>> next(gen)
    'SO'
    >>> next(gen)
    'SAW'
    >>> list(word_finder(t, words))
    ['SO', 'SAW']

    # Case 2: No words found
    >>> t = Tree("S", [Tree("I"), Tree("A", [Tree("Q"), Tree("E")]), Tree("C", [Tree("H")])])
    >>> list(word_finder(t, words))
    []

    # Case 3: Same word twice
    >>> t = Tree("S", [Tree("O"), Tree("O")] )
    >>> list(word_finder(t, words))
    ['SO', 'SO']

    # Case 4: Words that start the same
    >>> words = ['TAB', 'TAR', 'BAT', 'BAR', 'RAT']
    >>> t = Tree("T", [Tree("A", [Tree("R"), Tree("B")])])
    >>> list(word_finder(t, words))
```

```
['TAR', 'TAB']

# Case 5: Single letter words
>>> words = ['A', 'AN', 'AH']
>>> t = Tree("A")
>>> list(word_finder(t, words))
['A']

# Case 6: Words end in leaf
>>> words = ['A', 'AN', 'AH']
>>> t = Tree("A", [Tree("H"), Tree("N")])
>>> list(word_finder(t, words))
['AH', 'AN']

# Case 7: Words start at root
>>> words = ['GO', 'BEARS', 'GOB', 'EARS']
>>> t = Tree("B", [Tree("E", [Tree("A", [Tree("R", [Tree("S")])])])])
>>> list(word_finder(t, words))
['BEARS']

# Case 8: This special test ensures that your solution does *not*
#  pre-compute all the words before yielding the first one.
#  If done correctly, your solution should error only when it
#  tries to find the second word in this tree.
>>> words = ['SO', 'SAM', 'SAT', 'SAME', 'SAW', 'SOW']
>>> t = Tree("S", [Tree("O"), Tree("A", [Tree("Q"), Tree(1)]), Tree("C", [Tree(1)])])
>>> gen = word_finder(t, words)
>>> next(gen)
'SO'
>>> try:
...     next(gen)
... except TypeError:
...     print("Got a TypeError!")
... else:
...     print("Expected a TypeError!")
Got a TypeError!
"""
```

(a) Here is a skeleton of a correct solution:

```
def word_finder(letter_tree, words_list):
    def _________(____):
        _______________ # Optional
        if ____________:
            yield _______________
        for ____________:
            yield from _________

    yield from __________
```

We highly encourage you to follow the skeleton. You may diverge from the skeleton if your approach is correct and passes the doctests, but a non-working solution that doesn't follow the skeleton will not receive partial credit.

A correct solution should **not** first find all the words and then yield them; it should instead yield whenever it finds a new word. **Your solution may receive 0 points if it pre-computes the words.**

Your function should use the standard CS61A Tree definition, viewable here: code.cs61a.org/tree_class

Please use code.cs61a.org to try out your code. You can then paste the code here.

9. (4.0 points) Light It Up

An individually addressed LED light strip allows programmers to write programs to change the color of each individual LED light.

We could represent a strip of lights using a `SingleLED` class and an `LEDLightStrip` class:

```
class SingleLED:

    def __init__(self, brightness, color):
        self.brightness = brightness
        self.color = color

class LEDLightStrip:

    def __init__(self, leds):
        self.leds = leds
        self.length = len(leds)
        self.is_on = False

    def toggle(self):
        self.is_on = not self.is_on
```

Then a short 3-light strip could be constructed like so:

```
light_strip = LEDLightStrip([SingleLED(0, "blue"), SingleLED(0, "red"), SingleLED(0, "green")])
```

We would like to be able to iterate through the light object, one light at a time, like so:

```
for led in light_strip:
    led.brightness += 20
```

But with our current code, we see this error:

```
TypeError: 'LEDLightStrip' object is not iterable
```

(a) (2.0 pt) We need to define a new method on `LEDLightStrip` to make it iterable. Here's a skeleton of that method:

```
def ______________(self):
    #     (a)
    --------------
    --------------
    #     (b)
```

What method name should go in blank (a)?

(b) (2.0 pt) What code could go in blank (b) such that the for loop works? *You can use a single line of code or multiple lines of code, as long as the code is correct.*

10. (3.0 points) May I Take Your Order?

The following two functions can be used for inserting items into a linked list. The first function `insert_front()` always inserts at the front of the list while the second function `insert_back()` always inserts at the back of the list. Both functions use the standard CS61A Link definition, viewable here: code.cs61a.org/link_class

```
def insert_front(link, new_val):
    """Add NEW_VAL to the front of non-empty LINK, mutating the list.

    >>> link = Link(-1, Link(-3, Link(-5)))
    >>> insert_front(link, -0)
    >>> link
    Link(-0, Link(-1, Link(-3, Link(-5))))
    >>> insert_front(link, -3)
    >>> link
    Link(-3, Link(-0, Link(-1, Link(-3, Link(-5)))))
    """
    original_first = link.first
    link.first = new_val
    link.rest = Link(original_first, link.rest)

def insert_back(link, new_val):
    """Add NEW_VAL to the end of non-empty LINK, mutating the list.

    >>> link = Link(-1, Link(-3, Link(-5)))
    >>> insert_back(link, -0)
    >>> link
    Link(-1, Link(-3, Link(-5, Link(-0))))
    >>> insert_back(link, -3)
    >>> link
    Link(-1, Link(-3, Link(-5, Link(-0, Link(-3)))))
    """
    while link.rest is not Link.empty:
        link = link.rest
    link.rest = Link(new_val)
```

(a) (1.0 pt) What is the order of growth of `insert_front()` as the input list size changes?

- ◯ (1)
- ◯ $(\log_2 n)$
- ◯ (n)
- ◯ $(n \log_2 n)$
- ◯ (n^2)
- ◯ (n^3)
- ◯ (2^n)
- ◯ $(n!)$

(b) (1.0 pt) What is the order of growth of `insert_back()` as the input list size changes?

- ◯ (1)
- ◯ $(\log_2 n)$
- ◯ (n)
- ◯ $(n \log_2 n)$
- ◯ (n^2)
- ◯ (n^3)
- ◯ (2^n)
- ◯ $(n!)$

(c) (1.0 pt) Which of the following correctly describes the two functions?

- ◯ Both functions perform destructive operations on the input parameter, a mutable object.
- ◯ Both functions perform non-destructive operations on the input parameter, a mutable object.
- ◯ The first function performs a non-destructive operation while the second function performs a destructive operation.
- ◯ The first function performs a destructive operation while the second function performs a non-destructive operation.

11. (1.0 points) Extra Point!

(a) (1.0 pt) What phrase might be suggested by "6 = Q. in the S. M."?

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Spring 2022

MIDTERM 1

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

1. (7.0 points) What Would Python Dip?

Assume the following code has been executed.

```
def dipping(dots):
    if print("you dip"):
        return print("i dip")
    else:
        return print(dots) or dots or print("we dip")
```

What would the Python interpreter display? If the interpreter would include a new line, please enter a new line in your answer.

(a) (2.0 pt) `>>> dipping(0)`

(b) (2.0 pt) `>>> dipping(555)`

(c) (1.0 pt) What is the return value of `dipping(0)`?

(d) (1.0 pt) What is the return value of `dipping(-666)`?

(e) (1.0 pt) Rewrite the body of the `dipping` function in a single line of code.

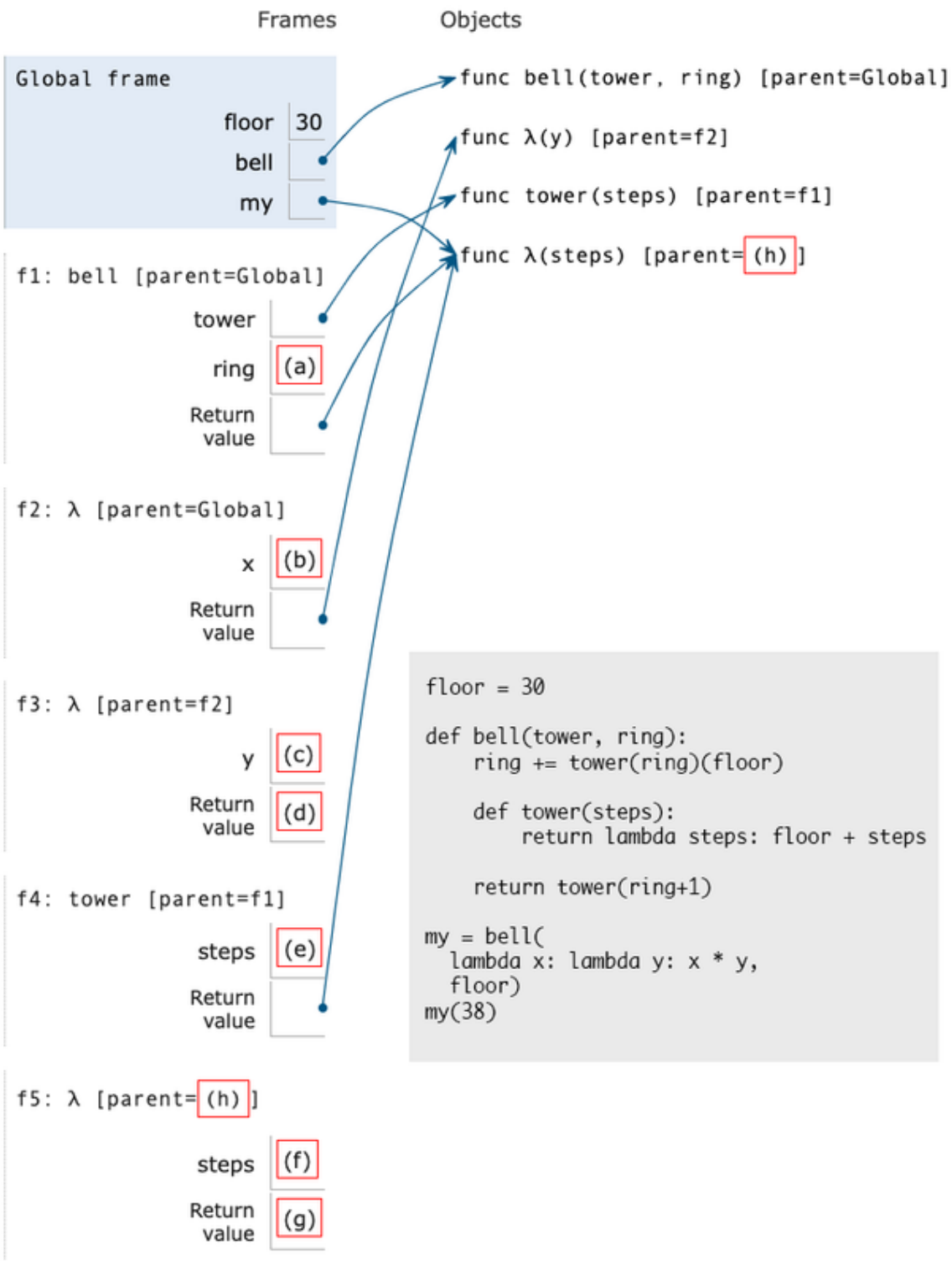
```
def dipping(dots):
    ---------------------
```

2. (8.0 points) Ring My Bell Tower

The following code was used to generate the environment diagram below:

```
floor = 30

def bell(tower, ring):
    ring += tower(ring)(floor)

    def tower(steps):
        return lambda steps: floor + steps

    return tower(ring+1)

my = bell(lambda x: lambda y: x * y, floor)
my(38)
```

The environment diagram below represents the *final* state of the environment. The code is also provided to the right of the diagram, for convenience. Line numbers have been omitted intentionally.



(a) (1.0 pt) Fill in blank (a)

(b) **(1.0 pt)** Fill in blank (b)

(c) **(1.0 pt)** Fill in blank (c)

(d) **(1.0 pt)** Fill in blank (d)

(e) **(1.0 pt)** Fill in blank (e)

(f) **(1.0 pt)** Fill in blank (f)

(g) **(1.0 pt)** Fill in blank (g)

(h) **(1.0 pt)** Fill in blank (h)

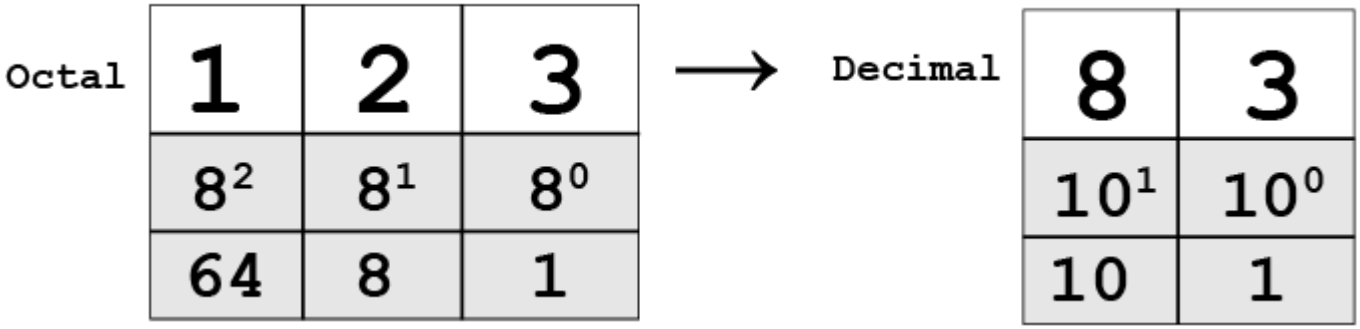
3. (6.0 points) Doctor Octalpus, Reborn

The standard number representation system is the decimal system, where each digit in a number represents a power of ten. The right-most digit is the ones' place, the next digit is the tens' place, etc.

In the octal system, each digit in a number represents a power of eight. The right-most digit is still the 1's place, but the next digit is the 8's place, the next digit is the 64's place, etc. Each digit ranges from 0-7, so octal numbers will never contain the digits 8 or 9.

To convert a number represented in octal to a number represented in decimal, each digit must be multipled by the appropriate power of eight. For example, `123` is actually `(1 * 64) + (2 * 8) + (3 * 1)`, resulting in a decimal representation of 83.

The diagram visualizes the equivalence between the octal and decimal numbers:



Implement `convert_to_decimal`, which takes an octal number and returns the decimal equivalent. The octal number will always start with a non-0 digit, and the number will always be positive.

```
def convert_to_decimal(octal):
    """
    >>> convert_to_decimal(3)   # (8^0 * 3)
    3
    >>> convert_to_decimal(23)  # (8^1 * 2) + (8^0 * 3)
    19
    >>> convert_to_decimal(123) # (8^2 * 1) + (8^1 * 2) + (8^0 * 3)
    83
    """
    decimal = 0
    curr_place = _________
                     (a)
    _________:
       (b)
        curr_digit = _________
                         (c)
        decimal = _________
                      (d)
        curr_place = _________
                         (e)
        octal = _________
                    (f)
    return decimal
```

(a) (1.0 pt) Fill in blank (a).

(b) (1.0 pt) Fill in blank (b).

(c) (1.0 pt) Fill in blank (c).

(d) (1.0 pt) Fill in blank (d).

(e) (1.0 pt) Fill in blank (e).

(f) (1.0 pt) Fill in blank (f).

4. (6.0 points) Forbidden Digits

Implement `forbid_digit`, a higher-order function which takes two arguments, a function `f` and a digit `forbidden`, and returns another function. If the returned function is passed a number where the digit in the 1s place is equal to the forbidden digit, it should return the result of calling the given function on the number without that final digit. Otherwise, it should return the result of calling the given function on the number.

```
def forbid_digit(f, forbidden):
    """
    >>> g = forbid_digit(lambda y: 200 // (y % 10), 0)
    >>> g(11)
    200
    >>> g(10)
    200
    >>> g = forbid_digit(lambda x: f'{x}a', 6)
    >>> g(61)
    '61a'
    >>> g(66)
    '6a'
    >>> g = forbid_digit(g, 3)
    >>> g(43)
    '4a'
    >>> g(63)
    '0a'
    >>> g(44)
    '44a'
    """
    def forbid_wrapper(n):
        if ________:
              (a)
            --------
              (b)
        else:
            --------
              (c)
    --------
       (d)
```

(a) (1.0 pt) Fill in blank (a).

(b) (1.0 pt) Fill in blank (b).

(c) (1.0 pt) Fill in blank (c).

(d) **(1.0 pt)** Fill in blank (d).

(e) **(2.0 pt)** Rewrite the body of the `forbid_digit` function in a single line of code.

```
def forbid_digit(f, forbidden):
    ----------------------
```

5. (7.0 points) The Floor is Lava

Implement `lava_hopper`, a function that "hops" from one number to the next computed number and tries to avoid any number detected as "lava". When it does land on "lava", it steps backwards by one number until it finds a non-lava number and then keeps hopping.

The function takes four arguments: `start_number` (the initial number), `goal_number` (the target number), `next_hop` (a function that computes the next number based on the current), and `is_lava` (a function that returns a boolean indicating if a number is lava), and it returns the minimum number of hops required to get from `start_number` to at least `goal_number`. The number of hops does *not* include steps backwards. If either the `start_number` or `goal_number` spots are lava, it returns the string 'No lava allowed there!'.

For example, consider this call

```
lava_hopper(1, 8, lambda x: x * 2, lambda x: x == 4)
```

The function starts from the number 1 and then hops to the numbers 2, 4, realizes that's lava, steps back to 3, hops to 6, hops to 12, and returns 4 (the number of hops required to get to/past 8).

Note that depending on the functions passed in for `next_hop` and `is_lava`, it is possible for a correct `lava_hopper` implementation to result in an infinite loop.

```
def lava_hopper(start_number, goal_number, next_hop, is_lava):
    """
    >>> # hops from 1->2, 2->4, 4->8
    >>> lava_hopper(1, 8, lambda x: x * 2, lambda x: False)
    3
    >>> # hops from 1->2, 2->4, steps to 3, hops 3->6, hops 6->12
    >>> lava_hopper(1, 8, lambda x: x * 2, lambda x: x == 4)
    4
    >>> # hops from 1->2, 2->4, 4->8, steps to 7, then 6, then 5, hops to 10
    >>> lava_hopper(1, 10, lambda x: x * 2, lambda x: 6 <= x <= 8)
    4
    >>> # hops from 3->6, 6->12, steps to 11, hops 11->22
    >>> lava_hopper(3, 20, lambda x: x * 2, lambda x: x % 10 == 2)
    3
    >>> lava_hopper(1, 8, lambda x: x * 2, lambda x: x == 1)
    'No lava allowed there!'
    >>> lava_hopper(1, 8, lambda x: x * 2, lambda x: x == 8)
    'No lava allowed there!'
    """
    if __________:
           (a)
        return 'No lava allowed there!'
    num_hops = 0
    while __________:
              (b)
        ________________:
              (c)
            ----------
                (d)
        start_number = __________
                    (e)
        ----------
            (f)
    return num_hops
```

(a) (1.0 pt) Fill in blank (a).

(b) (1.0 pt) Fill in blank (b).

(c) (1.0 pt) Fill in blank (c).

(d) (1.0 pt) Fill in blank (d).

(e) (1.0 pt) Fill in blank (e).

(f) (1.0 pt) Fill in blank (f).

(g) (1.0 pt) Write a call to `lava_hopper` that would result in an infinite loop.

```
lava_hopper(______________________)
```

6. (6.0 points) Curry Up Now

The function `order_meal` takes three arguments, `item_price`, `item_quantity`, and `ordered_at`, and either returns the total cost of the meal or reports how many hours should be waited before ordering. Only the doctests are shown below, as the implementation is not necessary for completing the question.

```
def order_meal(item_price, item_quantity, ordered_at):
    """
    >>> order_meal(5.99, 5, 11)
    29.95
    >>> order_meal(9.99, 5, 20)
    49.95
    >>> order_meal(8.99, 5, 7)
    'Wait!'
    """
    # Code intentionally omitted
```

Implement `curry_up_now`, a function that curries `order_meal` into a chain of three functions that each take a single argument. Once the third function is called, it should attempt to order the meal and print out the result. If the meal was successfully ordered during business hours, it should then return another curried function that can re-order the same item with a 50% discount.

```
def curry_up_now(item_price):
    """
    >>> curry_up_now(2.99)(2)(15)
    5.98
    <function <lambda>>
    >>> lunch_special = curry_up_now(8.99)
    >>> lunch_special(5)(11)
    44.95
    <function <lambda>>
    >>> lunch_special(3)(13)(2)(14)
    26.97
    8.99
    >>> no_discount = curry_up_now(10.99)(4)(7)
    Wait!
    >>> print(no_discount)
    None
    """
    def order_quantity(item_quantity):
        def by(ordered_at):
            result =  ________
                        (a)
            ________
               (b)
            ________:
               (c)
                return ________
                          (d)
        return by
    return order_quantity
```

(a) (1.0 pt) Fill in blank (a).

(b) (1.0 pt) Fill in blank (b).

(c) (1.0 pt) Fill in blank (c).

(d) (1.0 pt) Fill in blank (d).

(e) (2.0 pt) Rewrite the body of the inner `by` function in a single line of code, such that it behaves the same and passes the same doctests. Note that two calls to `order_meal` with the same parameters will always have the same result.

```
def curry_up_now(item_price):
    def order_quantity(item_quantity):
        def by(ordered_at):
            ----------------------
        return by
    return order_quantity
```

A large text area is provided to give you ample space to write, but your answer must be a valid *single* line of code.

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Spring 2022

MIDTERM 2

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) Below is a breakdown of the points per question, to help you budget your time.

Question	Points
Q1	5 pts
Q2	3 pts
Q3	2 pts
Q4	6 pts
Q5	4 pts
Q6	3 pts
Q7	6 pts
Q8	21 pts

1. (5.0 points) A Bug's Life

Thank you to our course staff members Richard, Melanie, and Amritansh for donating their bug collection to the midterm.

(a) (1.0 points) Wasp

Richard is trying to write a generator function that maps a given iterable using a one-argument function. Here's what he comes up with:

```
1 def map_gen(fn, iterable):
2     for it in iterable:
3         yield from fn(it)
```

i. (0.5 pt) Richard tests out his function by running the code below. He expected to see `[2, 4, 6, 8]` but instead gets an error when he runs these lines in an interpreter:

```
>>> data = [1, 2, 3, 4]
>>> fn = lambda x: x * 2
>>> list(map_gen(fn, data))
```

Which line of code in the function caused the code to break?

◯ 1

◯ 2

◯ 3

ii. (0.5 pt) Rewrite the buggy line of code you identified above to fix the error.

(b) (1.5 points) Moth

Melanie is trying to write some code to represent a cylinder in Python. This is her code:

```
1  class Cylinder:
2      def __init__(self, radius, height):
3          self.radius = radius
4          self.height = height
5
6      def volume(self):
7          volume = 3.14 * self.radius ** 2 * self.height
8          return volume
9
10     def percent_filled(self, volume_of_water):
11         return (volume_of_water / self.volume) * 100
```

i. (0.5 pt) Melanie runs the following code expecting to see it return 50.0, since 392.5 is a half-empty `cyl` (or is it half-full?).

```
>>> cyl = Cylinder(5, 10)
>>> cyl.percent_filled(392.5)
```

Unfortunately, the code results in this error instead:

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/Users/melanie/cylinder.py", line 11, in percent_filled
    return (volume_of_water / self.volume) * 100
TypeError: unsupported operand type(s) for /: 'int' and 'method'
```

According to this traceback, which method contains an error?

◯ `__init__`

◯ `volume`

◯ `percent_filled`

ii. (1.0 pt) Rewrite the buggy line of code from the method that you identified above to fix the error.

(c) (2.5 points) Bumblebee

Amritansh steps up with a challenge of his own. He's writing some code to flatten lists, but he seems to be running into some errors. Here's his code:

```
1 def flatten(lst):
2     res = []
3     for el in lst:
4         if isinstance(el, list):
5             res += [res.extend(flatten(el))]
6         else:
7             res += [el]
8     return res
```

i. (1.0 pt) He runs this on a nested list, as below:

```
>>> my_data = ["Mialy", ["Daphne", "Jordan"]]
>>> flatten(my_data)
```

The result he expects is `["Mialy", "Daphne", "Jordan"]`. What is the actual result?

ii. (0.5 pt) What line number causes the unexpected output to appear?

iii. (1.0 pt) Rewrite the buggy line you identified above so that Amritansh gets the expected output.

2. (3.0 points) What Would Python Display?

Assume the following code has been run (you may find a box-and-pointer diagram useful):

```
>>> lst1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> number_one = lst1.remove(1)
>>> some_number = lst1.pop()
>>> lst2 = lst1
>>> lst3 = lst2[:]
>>> lst1.insert(4, lst3)
>>> lst3.extend([lst1.pop() for _ in range(3)])
>>> lst2.append(lst1.remove(lst3))
```

For each part below, write the output of running the given code. Assume that none of the lines below will error.

(a) (1.0 pt)

```
>>> lst1[4:] + [number_one, some_number]
```

(b) (1.0 pt)

```
>>> lst3[5:9]
```

(c) (0.5 pt)

```
>>> next(reversed(lst3[:5]))
```

(d) (0.5 pt)

```
>>> lst1 is lst2
```

3. (2.0 points) A Different Kind of Dictionary

The `DictionaryEntry` class stores entries for an English dictionary, using instance variables for the `word` and its `definition`.

```
class DictionaryEntry:
    """
    >>> euph = DictionaryEntry("euphoric", "intensely happy")
    >>> avid = DictionaryEntry("avid", "enthusiastic")
    >>> [euph, avid]
    [DictionaryEntry('euphoric', 'intensely happy'), DictionaryEntry('avid', 'enthusiastic')]
    >>> f'Today we are learning {euph}'
    'Today we are learning euphoric: "intensely happy"'
    """
    def __init__(self, w, d):
        self.word = w
        self.definition = d

    def __repr__(self):
        ----------
           (a)

    def __str__(self):
        ----------
           (b)
```

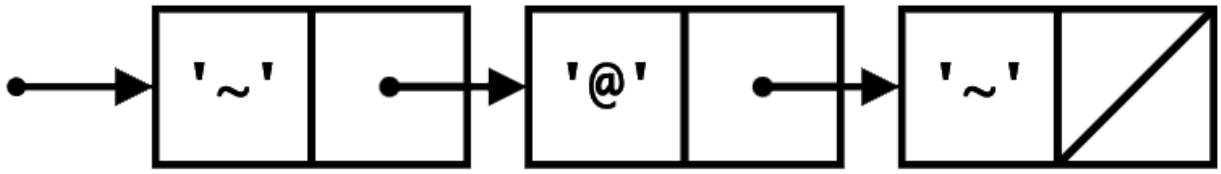
(a) (1.0 pt) Fill in blank (a).

(b) (1.0 pt) Fill in blank (b).

4. (6.0 points) Beadazzled

Implement `make_necklace`, a function that creates a linked list where each value comes from a given list of `beads`, and the beads are repeated in order to make a necklace of a given `length`.

For example, if `make_necklace` is called with `["~", "@"]` and a `length` of 3, then the linked list will contain `'~'`, then `'@'`, then `'~'`. Here's a diagram of that list:



See the docstring and doctests for further details on how the function should behave.

```
def make_necklace(beads, length):
    """
    Returns a linked list where each value is taken from the BEADS list,
    repeating the values from the BEADS list until the linked list has reached
    LENGTH. You can assume that LENGTH is greater than or equal to 1, there is
    at least one bead in BEADS, and all beads are string values.

    >>> wavy_ats = make_necklace(["~", "@"], 3)
    >>> wavy_ats
    Link('~', Link('@', Link('~')))
    >>> print(wavy_ats)
    <~ @ ~>
    >>> wavy_ats2 = make_necklace(["~", "@"], 4)
    >>> print(wavy_ats2)
    <~ @ ~ @>
    >>> curly_os = make_necklace(["}", "O", "{"], 9)
    >>> print(curly_os)
    <} O { } O { } O {>
    """
    if __________:
           (a)
        return __________
                    (b)
    return Link(__________, make_necklace(__________, __________))
                    (c)                        (d)         (e)
```

(a) (1.0 pt) Fill in blank (a).

(b) (1.0 pt) Fill in blank (b).

(c) (1.0 pt) Fill in blank (c).

(d) (2.0 pt) Fill in blank (d).

(e) (1.0 pt) Fill in blank (e).

5. (4.0 points) File Tree-tionaries

We can represent files and folders on a computer with Trees:

```
Tree("C:", [Tree("Documents", [Tree("hw05.py")]), Tree("pwd.txt")])
```

We can also represent the same folder layout with dictionaries:

```
{"C:": {"Documents": {"hw05.py": "FILE"}, "pwd.txt": "FILE"}}
```

Notice that in this model, we still treat files as dictionary keys, but with the value `"FILE"`.

Complete the implementation of `filetree_to_dict` below, which takes in a Tree `t` representing files and folders, and converts it to the dictionary representation.

```
def filetree_to_dict(t):
    """
    >>> filetree_to_dict(Tree("hw05.py"))
    {"hw05.py": "FILE"}
    >>> filetree = Tree("C:", [Tree("Documents", [Tree("hw05.py")]), Tree("pwd.txt")])
    >>> filetree_to_dict(filetree)
    {"C:": {"Documents": {"hw05.py": "FILE"}, "pwd.txt": "FILE"}}
    """
    res = {}
    if t.is_leaf():
        res[t.label] = ______________
                            (a)
    else:
        nested = {}
        for branch in t.branches:
            nested[branch.label] = ______________(branch)[______________]
                                        (b)                    (c)
        ______________ = nested
             (d)
    return res
```

(a) (1.0 pt) Fill in blank (a)

(b) (1.0 pt) Fill in blank (b)

(c) (1.0 pt) Fill in blank (c)

(d) (1.0 pt) Fill in blank (d)

6. (3.0 points) Mapping Time and Space

(a) The goal of the `maplink_to_list` function below is to map a linked list `lnk` into a Python list, applying a provided function `f` to each value in the list along the way. The function is fully written and passes all its doctests.

```
def maplink_to_list1(f, lnk):
    """Returns a Python list that contains f(x) for each x in Link LNK.
    >>> square = lambda x: x * x
    >>> maplink_to_list1(square, Link(3, Link(4, Link(5))))
    [9, 16, 25]
    """
    new_lst = []
    while lnk is not Link.empty:
        new_lst.append(f(lnk.first))
        lnk = lnk.rest
    return new_lst
```

i. (1.0 pt) What is the order of growth of `maplink_to_list1` in respect to the size of the input linked list `lnk`?

- ◯ Constant
- ◯ Logarithmic
- ◯ Linear
- ◯ Quadratic
- ◯ Exponential

(b) The next function, `maplink_to_list2`, serves the same purpose but is implemented slightly differently. This alternative implementation also passes all the doctests.

```
def maplink_to_list2(f, lnk):
    """Returns a Python list that contains f(x) for each x in Link LNK.
    >>> square = lambda x: x * x
    >>> maplink_to_list2(square, Link(3, Link(4, Link(5))))
    [9, 16, 25]
    """
    def map_link(f, lnk):
        if lnk is Link.empty:
            return Link.empty
        return Link(f(lnk.first), map_link(f, lnk.rest))

    mapped_lnk = map_link(f, lnk)
    new_lst = []
    while mapped_lnk is not Link.empty:
        new_lst.append(mapped_lnk.first)
        mapped_lnk = mapped_lnk.rest
    return new_lst
```

i. (1.0 pt) What is the order of growth of the alternative function, `maplink_to_list2`, in respect to the size of the input linked list `lnk`?

- ◯ Constant
- ◯ Logarithmic
- ◯ Linear
- ◯ Quadratic
- ◯ Exponential

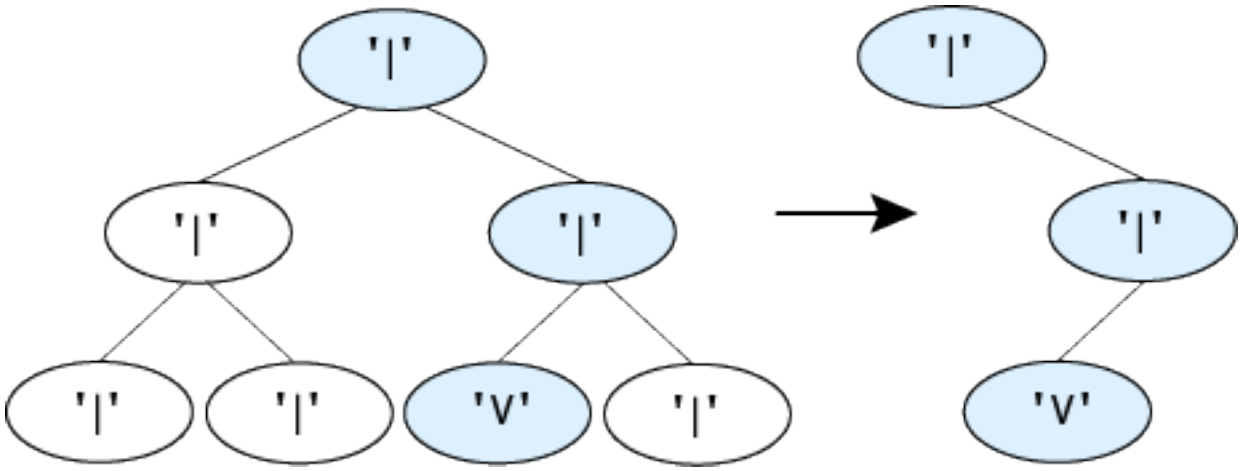
(c) (1.0 pt) Which of the functions requires more **space** to run?

- ◯ `maplink_to_list1`
- ◯ `maplink_to_list2`
- ◯ They both require the same amount of space.

7. (6.0 points) Tulip Mania

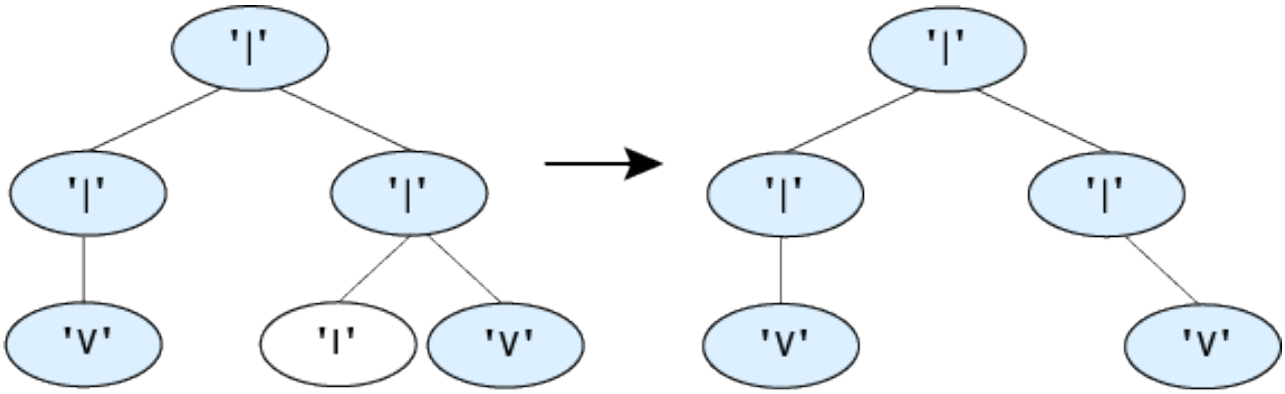
Implement `flower_keeper`, a function that mutates a tree `t` so that the only paths that remain are ones which end in a leaf node with a Tulip flower ('V').

For example, consider this tree where only one path ends in a flower. After calling `flower_keeper`, the tree has only three nodes left, the ones that lead to the flower.

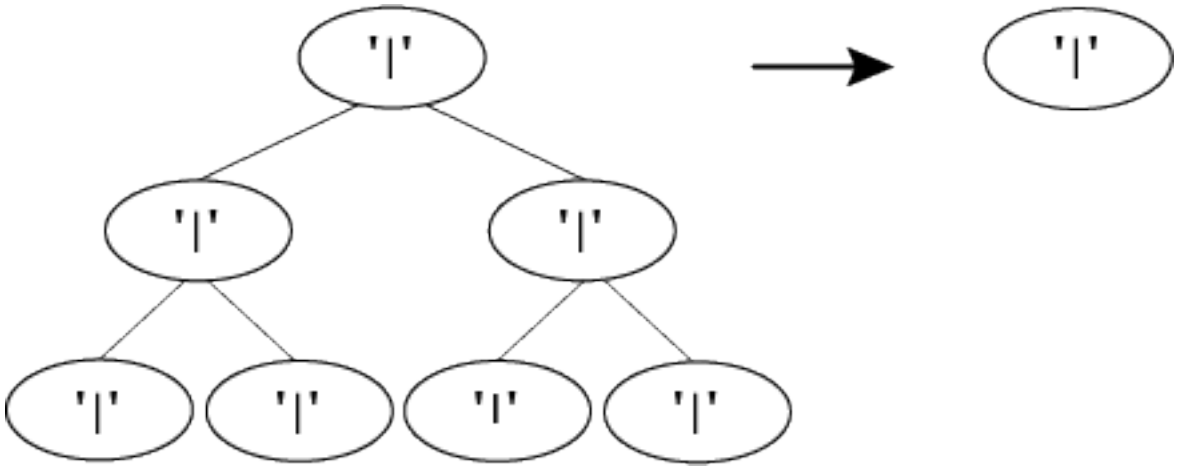


The shaded nodes in the diagram indicate paths that end in flowers.

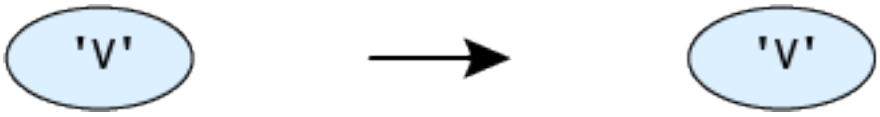
For this tree where two paths end in flowers, the tree keeps both paths that lead to flowers.



For a tree where none of the nodes are flowers, the function removes every branch except the root node.



For a tree with only a single node that is a flower, the function does not remove anything.



Read through the docstring and doctests for additional guidance on completing the implementation of the function. The examples shown above are from the doctests.

```
def flower_keeper(t):
    """
    Mutates the tree T to keep only paths that end in flowers ('V').
    If a path consists entirely of stems ('|'), it must be pruned.
    If T has no paths that end in flowers, the root node is still kept.
    You can assume that a node with a flower will have no branches.

    >>> one_f = Tree('|', [Tree('|', [Tree('|'), Tree('|')]), Tree('|', [Tree('V'), Tree('|')])])
    >>> print(one_f)
    |
      |
        |
        |
      |
        V
        |
    >>> flower_keeper(one_f)
    >>> one_f
    Tree('|', [Tree('|', [Tree('V')])])
    >>> print(one_f)
    |
      |
        V
    >>> no_f = Tree('|', [Tree('|', [Tree('|'), Tree('|')]), Tree('|', [Tree('|'), Tree('|')])])
    >>> flower_keeper(no_f)
    >>> no_f
    Tree('|')
    >>> just_f = Tree('V')
    >>> flower_keeper(just_f)
    >>> just_f
    Tree('V')
    >>> two_f = Tree('|', [Tree('|', [Tree('V')]), Tree('|', [Tree('|'), Tree('V')])])
    >>> flower_keeper(two_f)
    >>> two_f
    Tree('|', [Tree('|', [Tree('V')]), Tree('|', [Tree('V')])])
    """
    for b in __________:
                (a)
        __________
            (b)
    __________ = [__________ for b in __________ if __________]
        (c)           (d)                 (e)           (f)
```

(a) (0.5 pt) Fill in blank (a).

(b) (1.0 pt) Fill in blank (b).

(c) (1.0 pt) Fill in blank (c).

(d) (1.0 pt) Fill in blank (d).

(e) (0.5 pt) Fill in blank (e).

(f) (2.0 pt) Fill in blank (f).

8. (21.0 points) CS 61A Presents The Game of Hoop.

When we introduced the Hog project to you, we designed the rules of the game in a way that would make it possible to implement without using more advanced concepts like OOP and iterators/generators. Now that we've learned these concepts, you can rewrite parts of the game and expand it! The new version, called Hoop, will support *any* number of players greater than or equal to 2.

Assume the following changes from Hog:

- All rules have been taken out except for the Sow Sad rule, which is reiterated below.
- Each player's `strategy` is a function `strategy(own_score, other_scores)` that takes in that player's score and a *list* of the other players' scores, and outputs the number of times that player wishes to roll the dice during their turn. Order doesn't matter for the list of other players' scores.
- You don't have to keep track of the leader, and there is no commentary mechanism.

Sow Sad: If a player rolls any number of 1s in their turn, their score for that set of rolls is a 1 regardless of whatever else they rolled.

To start, take a look at the `HoopPlayer` class, which represents one player in our game:

```
class HoopPlayer:
    def __init__(self, strategy):
        """Initialize a player with STRATEGY, and a starting SCORE of 0. The
        STRATEGY should be a function that takes this player's score and a list
        of other players' scores.
        """
        self.strategy = strategy
        self.score = 0
```

Every game of Hoop will involve rolling a dice as well. Take a look at the `HoopDice` class, which represents a rollable dice that takes a set of values. You'll fill in the blanks in the first part of this question.

```
class HoopDice:
    def __init__(self, values):
        """Initialize a dice with possible values VALUES, and a starting INDEX
        of 0. The INDEX indicates which value from VALUES to return when the
        dice is rolled next.
        """
        self.values = values
        self.index = 0

    def roll(self):
        """Roll this dice. Advance the index to the next step before returning.
        """
        value = ______________
        ______________ = (______________) % ______________
        return value
```

(a) (4.0 points) The Dice: Making it Rollable

Begin by making the `HoopDice` rollable. Fill in the skeleton for the `HoopDice.roll` method below. See above for the rest of `HoopDice`.

```
class HoopDice:
    ...
    def roll(self):
        """Roll this dice. Advance the index to the next step before
        returning.
        >>> five_six = HoopDice([5, 6])
        >>> five_six.roll()
        5
        >>> five_six.index
        1
        >>> five_six.roll()
        6
        >>> five_six.index
        0
        """
        value = _______________
                    (1a)
        _______________ = (_______________) % _______________
            (1b)               (1c)               (1d)
        return value
```

i. (1.0 pt) Fill in blank (1a)

ii. (1.0 pt) Fill in blank (1b)

iii. (1.0 pt) Fill in blank (1c)

iv. (1.0 pt) Fill in blank (1d)

(b) (2.0 points) The Game: Looping Through Players

For the rest of the question, assume that `HoopDice.roll` is implemented correctly, regardless of your answers above.

In the rest of the question, you will work on making the game itself. Take a look at the `HoopGame` class, which takes in the setup for your game and makes it playable. You'll fill in the blanks shortly.

```
class HoopGame:
    def __init__(self, players, dice, goal):
        """Initialize a game with a list of PLAYERS, which contains at least one
        HoopPlayer, a single HoopDice DICE, and a goal score of GOAL.
        """
        self.players = players
        self.dice = dice
        self.goal = goal

    def next_player(self):
        """Infinitely yields the next player in the game, in order."""
        yield from ______________
        yield from ______________

    def get_scores(self):
        """Collects and returns a list of the current scores for all players
        in the same order as the SELF.PLAYERS list.
        """
        # Implementation omitted. Assume this method works correctly.

    def get_scores_except(self, player):
        """Collects and returns a list of the current scores for all players
        except PLAYER.
        """
        return [______________ for pl in ______________ if ______________]

    def roll_dice(self, num_rolls):
        """Simulate rolling SELF.DICE exactly NUM_ROLLS > 0 times. Return sum of
        the outcomes unless any of the outcomes is 1. In that case, return 1.
        """
        outcomes = [______________ for x in ______________]
        ones = [______________ for outcome in outcomes]
        return 1 if ______________(ones) else ______________(outcomes)

    def play(self):
        """Play the game while no player has reached or exceeded the goal score.
        After the game ends, return all players' scores.
        """
        player_gen = self.next_player()
        while max(self.get_scores()) < self.goal:
            player = ______________(player_gen)
            other_scores = self.get_scores_except(player)
            num_rolls = ______________(player.score, other_scores)
            outcome = self.roll_dice(num_rolls)
            ______________ += outcome
        return self.get_scores()
```

To see what the expected behavior of each method is, take a look at its doctests. These will be provided for the relevant methods in each subpart for the rest of the question.

In all following subparts, assume that the following has already been run to set up the game:

```
>>> roll_once_strategy = lambda pl, ops: 1
>>> roll_twice_strategy = lambda pl, ops: 2
>>> always_5 = HoopDice([5])
>>> player1 = HoopPlayer(roll_twice_strategy)
>>> player2 = HoopPlayer(roll_once_strategy)
>>> player3 = HoopPlayer(lambda pl, ops: 6)
>>> game = HoopGame([player1, player2, player3], always_5, 55)
>>> # since we omit the implementation of HoopGame.get_scores, here's what it
>>> # should output:
>>> game.get_scores()
[0, 0, 0]
```

The first `HoopGame` method you'll implement is `HoopGame.next_player`, which gives the game a way to iterate over all of the players playing, in order to give them each a turn in order.

```
class HoopGame:
    ...
    def next_player(self):
        """Infinitely yields the next player in the game, in order.
        >>> player_gen = game.next_player()
        >>> next(player_gen) is player1
        True
        >>> next(player_gen) is player3
        False
        >>> next(player_gen) is player3
        True
        >>> next(player_gen) is player1
        True
        >>> next(player_gen) is player2
        True
        >>> new_player_gen = game.next_player()
        >>> next(new_player_gen) is player1
        True
        >>> next(player_gen) is player3
        True
        """
        yield from ______________
                        (2a)
        yield from ______________
                        (2b)
```

i. (1.0 pt) Fill in blank (2a)

ii. (1.0 pt) Fill in blank (2b)

(c) (3.0 points) The Game: Getting Scores

The `HoopGame.get_scores` method is already fully implemented and provides a way to get a list of the scores of *all* players.

However, each player's strategy relies on the player's *opponents'* scores, so `HoopGame` also needs a method that gives the scores of all players *except* the current one. Fill in the skeleton for `HoopGame.get_scores_except` below. Keep in mind that multiple players may have the same score.

```
class HoopGame:
    ...
    def get_scores_except(self, player):
        """Collects and returns a list of the current scores for all players
        except PLAYER.
        >>> game.get_scores_except(player2)
        [0, 0]
        """
        return [_______________ for pl in _______________ if _______________]
                    (3a)                    (3b)              (3c)
```

As a reminder, here's what the `HoopPlayer` class looks like:

```
class HoopPlayer:
    def __init__(self, strategy):
        """Initialize a player with STRATEGY, and a starting SCORE of 0. The
        STRATEGY should be a function that takes this player's score and a list
        of other players' scores.
        """
        self.strategy = strategy
        self.score = 0
```

i. (1.0 pt) Fill in blank (3a)

ii. (1.0 pt) Fill in blank (3b)

iii. (1.0 pt) Fill in blank (3c)

(d) (5.0 points) The Game: Rolling Dice

Next, give `HoopGame` the ability to roll its dice some given number of times and return the total from rolling all those times. Recall that the dice in the game is just a single dice, represented by the `HoopDice` instance at `self.dice`. Fill in the skeleton for `HoopGame.roll_dice` below:

```
class HoopGame:
    ...
    def roll_dice(self, num_rolls):
        """Simulate rolling SELF.DICE exactly NUM_ROLLS > 0 times. Return sum of
        the outcomes unless any of the outcomes is 1. In that case, return 1.
        >>> game.roll_dice(4)
        20
        """
        outcomes = [_____________ for x in _____________]
                        (4a)                    (4b)
        ones = [_____________ for outcome in outcomes]
                    (4c)
        return 1 if _____________(ones) else _____________(outcomes)
                        (4d)                    (4e)
```

i. (1.0 pt) Fill in blank (4a)

ii. (1.0 pt) Fill in blank (4b)

iii. (1.0 pt) Fill in blank (4c)

iv. (1.0 pt) Fill in blank (4d)

v. (1.0 pt) Fill in blank (4e)

(e) (3.0 points) The Game: Putting it all Together

Finally, it's time to play the game! Recall that a player's strategy depends on two inputs: the player's own score, and a list of the scores of the player's opponents. Using existing attributes wherever possible, fill in the skeleton for `HoopGame.play` below:

```
class HoopGame:
    ...
    def play(self):
        """Play the game while no player has reached or exceeded the goal score.
        After the game ends, return all players' scores.
        >>> game.play()
        [20, 10, 60]
        """
        player_gen = self.next_player()
        while max(self.get_scores()) < self.goal:
            player = _______________(player_gen)
                        (5a)
            other_scores = self.get_scores_except(player)
            num_rolls = _______________(player.score, other_scores)
                           (5b)
            outcome = self.roll_dice(num_rolls)
            _______________ += outcome
                (5c)
        return self.get_scores()
```

i. (1.0 pt) Fill in blank (5a)

ii. (1.0 pt) Fill in blank (5b)

iii. (1.0 pt) Fill in blank (5c)

(f) (4.0 points) The Dice: It's Broken!

Now that you've finished implementing the game, you'll implement a new kind of `BrokenHoopDice`. This dice extends `HoopDice`, but is broken! That is, it alternates between behaving like a normal dice and returning `when_broken` value between turns.

For this part of the question, you may not use `=` or `lambda`.

```
class BrokenHoopDice(HoopDice):
    def __init__(self, values, when_broken):
        ______________(values)
             (6a)
        self.when_broken = when_broken
        self._______________ = False
                  (6b)

    def roll(self):
        """
        >>> broken = BrokenHoopDice([5, 6, 7], 11)
        >>> broken.roll()
        5
        >>> [broken.roll() for _ in range(6)]
        [11, 6, 11, 7, 11, 5]
        """
        if self.is_broken:
            self.is_broken = not self.is_broken
            return _______________
                        (6c)
        else:
            self.is_broken = not self.is_broken
            return _______________()
                        (6d)
```

i. (1.0 pt) Fill in blank (6a)

ii. (1.0 pt) Fill in blank (6b)

iii. (1.0 pt) Fill in blank (6c)

iv. (1.0 pt) Fill in blank (6d)

9. (0.0 points) Just for Fun

Draw some cool dice faces for Hoop! This is not for points and will not be graded.

No more questions.

CS 61A Structure and Interpretation of Computer Programs

Spring 2023 MIDTERM 1

INSTRUCTIONS

You have 1 hour and 50 minutes to complete the exam.

- The exam is closed book, closed notes, closed computer, closed calculator, except one 8.5" × 11" page of your own creation and the provided midterm 1 study guide.
- Mark your answers on the exam itself in the spaces provided. We will not grade answers written on scratch paper or outside the designated answer spaces.
- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `pow`, `len`, `abs`, `bool`, `int`, `float`, `str`, `round`, `max`, and `min`.
- You **may not** use example functions defined on your study guide unless a problem clearly states you can.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.
- You may not use ; to place two statements on the same line.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (10.0 points) What Would Python Display?

Assume the following code has been executed.

```
e = 40
def glob(e):
    print(e)
    e = e + 1
    return e

def mid(nights):
    days = 7
    if days or nights:
        days, nights = (days and days + 1), (nights or days)
    print(days)
    return nights

def special(f, x):
    return f(x) + f(x)
```

For each expression below, write the output **displayed by the interactive Python interpreter** when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. To display a function value, write "Function".

(a) (2.0 pt) `print(0, print(1), 2)`

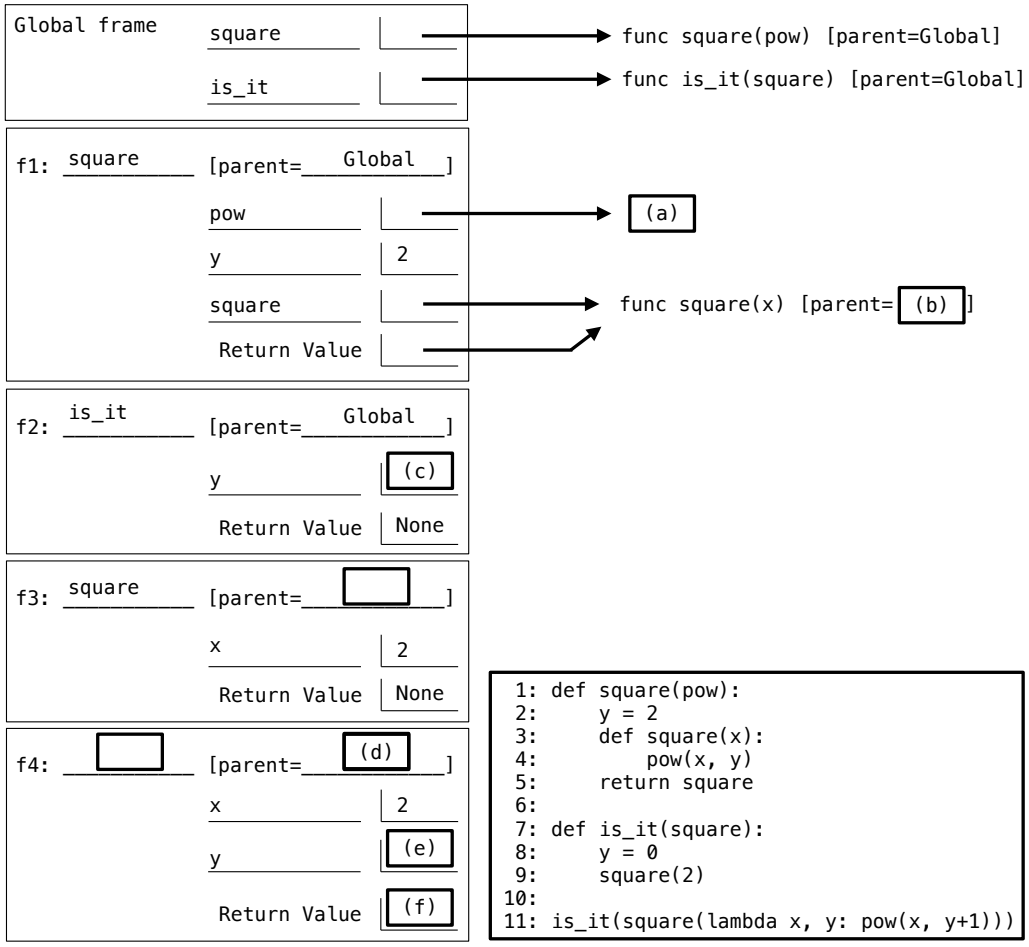
(b) (4.0 pt) `print(glob(e), glob(3), e)`

(c) (2.0 pt) `mid(not 'swift')`

(d) (2.0 pt) `special(str, 10)`

2. (6.0 points) Square the Square

Complete the environment diagram below and then answer the questions that follow. There is a one question for each labeled blank in the diagram. The blanks with no labels have no questions associated with them and are not scored. **If a blank contains an arrow to a function, write the function as it would appear in the diagram. Do not add frames for calls to built-in functions.**



(a) (1.0 pt) Fill in blank (a).

- ◯ `func pow(...) [parent=Global]`
- ◯ `func pow(...) [parent=f1]`
- ◯ `func λ <line 11>(x, y) [parent=Global]`
- ◯ `func λ <line 11>(x, y) [parent=f1]`

(b) (1.0 pt) Fill in blank (b).

- ◯ `f1`
- ◯ `Global`
- ◯ `square`
- ◯ None of these

(c) (1.0 pt) Fill in blank (c).

- ◯ `0`
- ◯ `1`
- ◯ `2`
- ◯ None of these

(d) (1.0 pt) Fill in blank (d).

- ◯ `f1`
- ◯ `Global`
- ◯ `square`
- ◯ None of these

(e) (1.0 pt) Fill in blank (e).

- ◯ `0`
- ◯ `1`
- ◯ `2`
- ◯ `3`
- ◯ None of these

(f) (1.0 pt) Fill in blank (f).

- ◯ `1`
- ◯ `2`
- ◯ `4`
- ◯ `8`
- ◯ None of these

3. (14.0 points) On Repeat

Definition: A positive integer `n` is a *repeating sequence* of positive integer `m` if `n` is written by repeating the digits of `m` one or more times. For example, 616161 is a repeating sequence of 61, but 61616 is not.

Hint: `pow(b, n)` raises `b` to the power of `n`. For example, `pow(10, 3)` is `1000`, and `654321 % pow(10, 3)` is `321` (the last 3 digits).

(a) (8.0 points)

Implement `repeating` which takes positive integers `t` and `n`. It returns whether `n` is a repeating sequence of some `t`-digit integer. **You may not use `str` or `repr` or `[` or `]` or `for`.**

```
def repeating(t, n):
    """Return whether t digits repeat to form positive integer n.

    >>> repeating(1, 6161)
    False
    >>> repeating(2, 6161)  # repeats 61 (2 digits)
    True
    >>> repeating(3, 6161)
    False
    >>> repeating(4, 6161)  # repeats 6161 (4 digits)
    True
    >>> repeating(5, 6161)  # there are only 4 digits
    False
    """
    if _________ > n:  # check that n has at least t digits
          (a)

        return False

    rest = n

    while rest:

        if rest % pow(10, t) != _________:
                                    (b)

            _________
               (c)


        _________
           (d)

    return _________
              (e)
```

i. (2.0 pt) Fill in blank (a).

- ◯ `t`
- ◯ `t - 1`
- ◯ `10 * t`
- ◯ `10 * (t - 1)`
- ◯ `pow(10, t)`
- ◯ `pow(10, t - 1)`

ii. (2.0 pt) Fill in blank (b)

iii. (1.0 pt) Fill in blank (c)

iv. (2.0 pt) Fill in blank (d)

v. (1.0 pt) Fill in blank (e).

- ◯ `True`
- ◯ `False`
- ◯ `rest == n`
- ◯ `rest != n`

(b) (6.0 points)

Implement `digits`, which takes a positive integer `n`. It returns the smallest `m` for which `n` is a repeating sequence of `m`. **You may not use `str` or `repr` or `[` or `]` or `for`.** Assume `repeating` is implemented correctly (and you may use it).

```
def digits(n):
    """Return the smallest m that repeats to form n.

    >>> digits(6161616161)
    61
    >>> digits(33333333)
    3
    >>> digits(12312312)
    12312312
    >>> digits(123123123)
    123
    """
    k = 1

    while True:

        if _________:
              (f)

            return _________
                       (g)
        _________
           (h)
```

i. (3.0 pt) Fill in blank (f).

ii. (2.0 pt) Fill in blank (g).

iii. (1.0 pt) Fill in blank (h).

- ◯ `k = k + 1`
- ◯ `n = n // 10`
- ◯ `return False`
- ◯ `return n`

4. (10.0 points) Perfect Ten

Definitions:

- A *non-negative function* takes one argument and returns a non-negative integer.
- The *nearest* multiple of 10 to a non-negative integer `n` is larger than `n` if the last digit of `n` is 5 or more and less than or equal to `n` otherwise. For example, the nearest multiple of 10 to 25 is 30, but the nearest multiple of 10 to 24 is 20.

(a) (5.0 points)

Implement `nearest_ten`, which takes a non-negative function `f`. The function returned by calling `nearest_ten(f)` takes one argument `n` and returns the nearest multiple of 10 to `f(n)`. (In other words, it rounds `f(n)` to the nearest ten.)

```
def nearest_ten(f):
    """nearest_ten returns a function of n that returns the multiple of 10 nearest to f(n).

    >>> h = nearest_ten(lambda n: n * n)
    >>> h(2)  # The nearest multiple of 10 to 2 * 2 = 4
    0
    >>> h(3)  # The nearest multiple of 10 to 3 * 3 = 9
    10
    >>> h(5)  # The nearest multiple of 10 to 5 * 5 = 25
    30
    >>> h(8)  # The nearest multiple of 10 to 8 * 8 = 64
    60
    >>> h(10)  # The nearest multiple of 10 to 10 * 10 = 100
    100
    """
    def g(n):

        y = _________
              (a)

        rest, ones = y // 10, y % 10

        if _________:
             (b)

            rest = _________
                      (c)

        return _________
                  (d)

    return g
```

i. (2.0 pt) Fill in blank (a). **You may not use `round` or `int` or `//` for this blank.**

ii. **(1.0 pt)** Fill in blank (b).

- ◯ `rest > 5`
- ◯ `rest >= 5`
- ◯ `ones > 5`
- ◯ `ones >= 5`
- ◯ `y > 5`
- ◯ `y >= 5`

iii. **(1.0 pt)** Fill in blank (c).

- ◯ `rest + ones`
- ◯ `rest * 10 + ones`
- ◯ `rest // 10`
- ◯ `rest % 10`
- ◯ `rest + 1`

iv. **(1.0 pt)** Fill in blank (d).

- ◯ `rest`
- ◯ `rest * 10`
- ◯ `rest + ones`
- ◯ `rest * 10 + ones`
- ◯ `rest + round(ones)`
- ◯ `rest * 10 + round(ones)`
- ◯ `round(rest + ones)`
- ◯ `round(rest * 10 + ones)`

(b) (5.0 points)

Implement `distance`, which takes two arguments: a positive integer function `f` and a positive integer `n`. It returns the absolute difference (a non-negative integer) between `f(n)` and the nearest multiple of 10 to `f(n)`. **You may not call `round` or `int` or use `+` or `-`.** Assume `nearest_ten` is implemented correctly (and you may use it).

```
def distance(f, n):
    """Return the absolute value of the difference between f(n)
    and the nearest multiple of 10 to f(n).

    >>> distance(lambda n: n * n, 4)  # 16 is 4 away from 20
    4
    >>> distance(lambda n: n * n, 9)  # 81 is 1 away from 80
    1
    """
    return abs(_________ - _________)
                  (e)         (f)
```

i. (2.0 pt) Fill in blank (e).

- ◯ `f`
- ◯ `f(n)`
- ◯ `n`
- ◯ `n * n`
- ◯ `lambda n: n`
- ◯ `lambda n: f(n)`
- ◯ `lambda n: n * n`
- ◯ `(lambda n: n * n)(n)`

ii. (3.0 pt) Fill in blank (f).

(c) This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

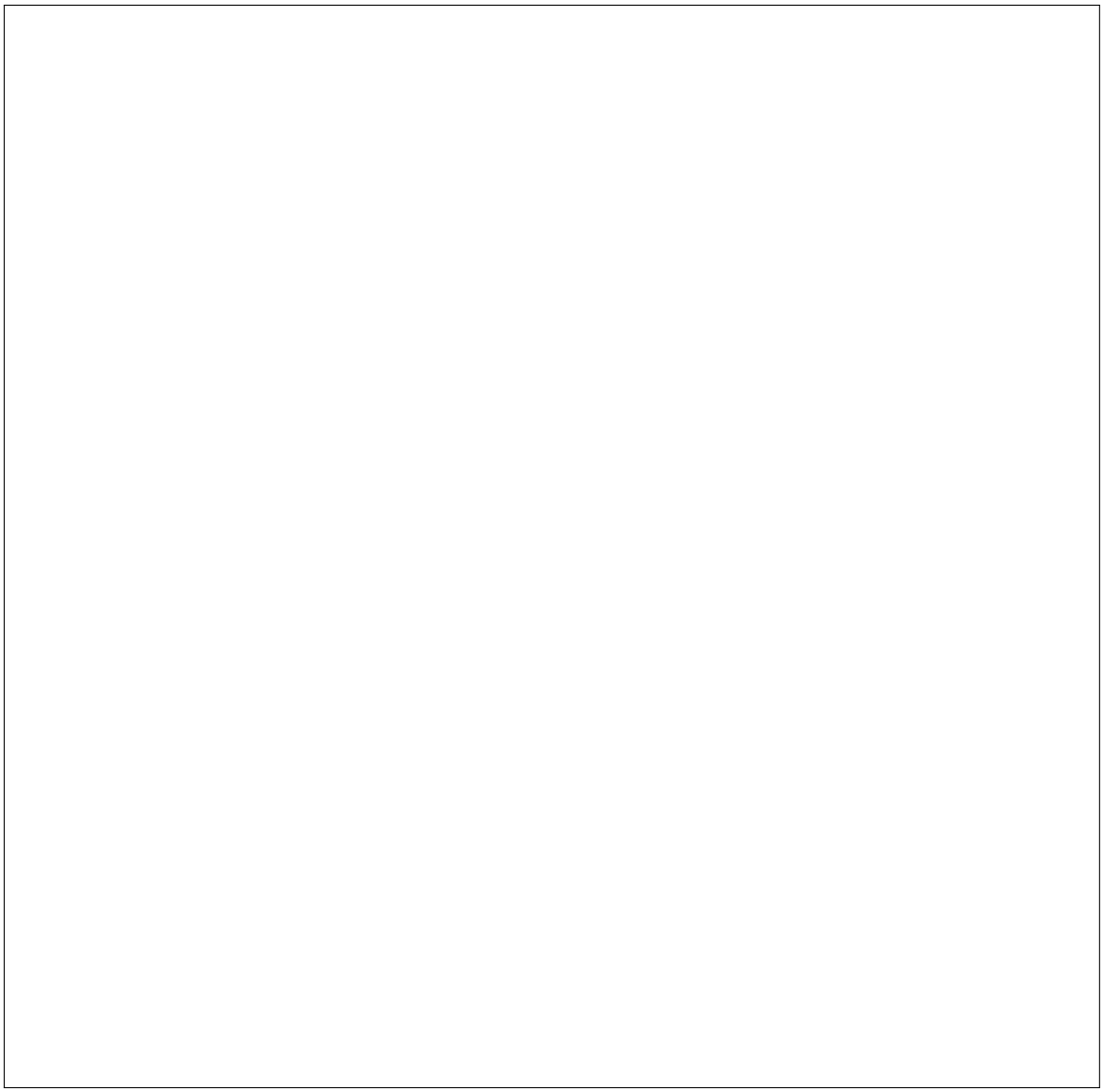
Use `nearest_ten` to implement `nearest_hundred`, which takes a positive integer function `f`. The function returned by calling `nearest_hundred(f)` takes a positive integer `n` and returns the nearest multiple of 100 to `f(n)`. **Note:** The nearest multiple of 100 to `n` is larger than `n` if and only if the tens digit of `n` is 5 or more, so 300 is nearest to 250, but 200 is nearest to 249. **You may not call `round` or `int`. You may not use `str` or `repr` or `%`.** Assume `nearest_ten` is implemented correctly, and you **must** use it.

```
def nearest_hundred(f):
    """nearest_hundred(f)(n) is the multiple of 100 nearest to f(n).

    >>> h = nearest_hundred(lambda n: n * n)
    >>> h(8)  # The nearest multiple of 100 to 8 * 8 = 64
    100
    >>> h(20)  # The nearest multiple of 100 to 20 * 20 = 400
    400
    >>> h(25)  # The nearest multiple of 100 to 25 * 25 = 625
    600
    """
    return _________
```

i. Fill in the blank.

5. Just for Fun

(a) **Optional:** If CS 61A were a sport, what would it look like? (Feel free to draw.)



CS 61A Structure and Interpretation of Computer Programs
Spring 2023

MIDTERM 2

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (6.0 points) What Would Python Display?

Assume the following code has been executed.

```
def chain(s):
    return [s[0], s[1:]]

silver = [2, chain([3, 4, 5])]
gold = [silver[0], silver[1].pop()]
silver[0] = 1
platinum = chain(chain([6, 7, 8]))
```

Write the output **displayed by the interactive Python interpreter** when each expression below is evaluated.

Hint: Try drawing an environment diagram!

Reminder: `s.pop()` removes and returns the last item in list `s`.

(a) (2.0 pt) `silver`

(b) (2.0 pt) `gold`

(c) (2.0 pt) `platinum`

2. (10.0 points) Letter Grade

(a) (4.0 points)

Implement the `Letter` class. A `Letter` has two instance attributes: `contents` (a `str`) and `sent` (a `bool`). Each `Letter` can only be sent once. The `send` method prints whether the letter was sent, and if it was, returns the reply, which is a new `Letter` instance with the same `contents`, but in all caps. *Hint*: `'hi'.upper()` evaluates to `'HI'`.

```
class Letter:
    """A letter receives an all-caps reply when sent successfully.

    >>> hi = Letter('Hello, World!')
    >>> hi.send()
    Hello, World! has been sent.
    HELLO, WORLD!
    >>> hi.send()
    Hello, World! was already sent.
    >>> Letter('Hey').send().send()
    Hey has been sent.
    HEY has been sent.
    HEY
    """
    def __init__(self, contents):
        self.contents = contents
        _________
           (a)
    def send(self):
        if self.sent:
            print(self, 'was already sent.')
        else:
            print(self, 'has been sent.')
            _________
               (b)
            return _________
                       (c)
    def __repr__(self):
        # Note: since no __str__ method is defined, the repr and str strings are the same.
        return self.contents
```

i. (1.0 pt) Fill in blank (a).

ii. (1.0 pt) Fill in blank (b)

iii. (2.0 pt) Fill in blank (c)

(b) (6.0 points)

Implement the `Numbered` class. A `Numbered` letter has a `number` attribute equal to how many numbered letters have previously been constructed. This number appears in its `repr` string. Assume `Letter` is implemented correctly.

```
class Numbered(Letter):
    """A numbered letter has a different repr method that shows its number.

    >>> hey = Numbered('Hello, World!')
    >>> hey.send()
    #0 has been sent.
    HELLO, WORLD!
    >>> Numbered('Hi!').send()
    #1 has been sent.
    HI!
    >>> hey
    #0
    """
    number = 0

    def __init__(self, contents):
        super().__init__(contents)
        _________
           (d)
        _________
           (e)

    def __repr__(self):
        return '#' + _________
                        (f)
```

i. (2.0 pt) Fill in blank (d).

- ◯ `Numbered.number = 0`
- ◯ `Numbered.number = number`
- ◯ `Numbered.number = self.number`
- ◯ `self.number = 0`
- ◯ `self.number = number`
- ◯ `self.number = Numbered.number`

ii. (2.0 pt) Fill in blank (e).

iii. (2.0 pt) Fill in blank (f).

3. (9.0 points) Prefixes

Definition. A *prefix sum* of a sequence of numbers is the sum of the first `n` elements for some positive length `n`.

(a) (4.0 points)

Implement `prefix`, which takes a list of numbers `s` and returns a list of the prefix sums of `s` in increasing order of the length of the prefix.

```
def prefix(s):
    """Return a list of all prefix sums of list s.

    >>> prefix([1, 2, 3, 0, 4, 5])
    [1, 3, 6, 6, 10, 15]
    >>> prefix([2, 2, 2, 0, -5, 5])
    [2, 4, 6, 6, 1, 6]
    """
    return [_________ for k in _________]
               (a)                (b)
```

i. (2.0 pt) Fill in blank (a).

ii. (1.0 pt) Fill in blank (b).

- ◯ `s`
- ◯ `[s]`
- ◯ `s[1:]`
- ◯ `range(s)`
- ◯ `range(len(s))`

iii. (1.0 pt) What is the order of growth of the time to run this alternate implentation in terms of the length of `s`?

Assume that the `append` method of a list takes just 1 step (constant time).

```
def prefix(s):
    """Return a list of all prefix sums of list s."""
    t = 0
    result = []
    for x in s:
        t = t + x
        result.append(t)
    return result
```

- ◯ Constant
- ◯ Logarithmic
- ◯ Linear
- ◯ Quadratic
- ◯ Exponential

(b) (5.0 points)

Implement `tens`, which takes a **non-empty** linked list of numbers `s` represented as a `Link` instance. It prints all of the prefix sums of `s` that are multiples of 10 in increasing order of the length of the prefix.

The `Link` class appears on the midterm 2 study guide.

```
def tens(s):
    """Print all prefix sums of Link s that are multiples of ten.

    >>> tens(Link(3, Link(9, Link(8, Link(10, Link(0, Link(14, Link(6)))))))
    20
    30
    30
    50
    """

    def f(suffix, total):

        if total % 10 == 0:

            print(total)

        if _________:
              (d)

            _________
               (e)

    _________
       (f)
```

i. (1.0 pt) Fill in blank (d).

- ◯ `s is not Link.empty`
- ◯ `s.rest is not Link.empty`
- ◯ `s.rest.rest is not Link.empty`
- ◯ `suffix is not Link.empty`
- ◯ `suffix.rest is not Link.empty`
- ◯ `suffix.rest.rest is not Link.empty`

ii. (2.0 pt) Fill in blank (e).

iii. (2.0 pt) Fill in blank (f).

4. (10.0 points) Tree Trimming

(a) (7.0 points)

Implement `exclude`, which takes a `Tree` instance `t` and a value `x`. It returns a `Tree` containing the root node of `t` as well as each non-root node of `t` with a label not equal to `x`. The parent of a node in the result is its nearest ancestor node that is not excluded. The input `t` must not be modified.

The `Tree` class appears on the midterm 2 study guide.

```
def exclude(t, x):
    """Return a Tree with the non-root nodes of t whose labels are not equal to x.

    >>> t = Tree(1, [Tree(2, [Tree(2), Tree(3)]), Tree(4, [Tree(1)])])
    >>> exclude(t, 2)
    Tree(1, [Tree(3), Tree(4, [Tree(1)])])
    >>> t              # t is not changed
    Tree(1, [Tree(2, [Tree(2), Tree(3)]), Tree(4, [Tree(1)])])
    >>> exclude(t, 1)  # The root node cannot be excluded
    Tree(1, [Tree(2, [Tree(2), Tree(3)]), Tree(4)])
    """

    filtered_branches = map(lambda y: _________, t.branches)
                                          (a)

    bs = []

    for b in filtered_branches:

        if _________:
              (b)

            bs._________(_________)
                  (c)       (d)

        else:

            bs.append(b)

    return Tree(t.label, bs)
```

i. (2.0 pt) Fill in blank (a).

ii. (2.0 pt) Fill in blank (b).

iii. (1.0 pt) Fill in blank (c).

- ◯ `remove`
- ◯ `pop`
- ◯ `append`
- ◯ `extend`

iv. (2.0 pt) Fill in blank (d).

- ◯ `b`
- ◯ `b.branches`
- ◯ `exclude(b, x)`
- ◯ `Tree(b.label, [exclude(y, x) for y in b.branches])`

(b) (3.0 points)

Implement `remove`, which takes a `Tree` instance `t` and a value `x`. It removes all non-root nodes from `t` that have a label equal to `x`, then returns `t`. The parent of a node in `t` is its nearest ancestor that is not removed. Assume that `exclude` is implemented correctly.

```
def remove(t, x):
    """Remove all non-root nodes labeled x from t and return t.

    >>> t = Tree(1, [Tree(2, [Tree(2), Tree(3)]), Tree(4)])
    >>> remove(t, 2)
    Tree(1, [Tree(3), Tree(4)])
    >>> remove(t, 3)  # t was changed, so both 2 and 3 are now removed.
    Tree(1, [Tree(4)])
    """

    ---------
       (e)

    return t
```

i. (3.0 pt) Fill in blank (e).

5. (10.0 points) Parking

Definition. When parking vehicles in a row, a *motorcycle* takes up 1 parking spot and a *car* takes up 2 adjacent parking spots. A string of length `n` can represent `n` adjacent parking spots using `%` for a motorcycle, `<>` for a car, and `.` for an empty spot. For example `'.%%.<><>'` represents an empty spot, then two spots containing motorcycles, then another empty spot, then four spots containing two cars.

Thanks to the Berkeley Math Circle for introducing this question.

(a) (4.0 points)

Implement `count_park`, which returns the number of ways that vehicles can be parked in `n` adjacent parking spots for positive integer `n`. Some or all spots can be empty.

```
def count_park(n):
    """Count the ways to park cars and motorcycles in n adjacent spots.

    >>> count_park(1)  # '.' or '%'
    2
    >>> count_park(2)  # '..', '.%', '%.', '%%', or '<>'
    5
    >>> count_park(4)  # some examples: '<><>', '.%%.', '%<>%', '%.<>'
    29
    """

    if n < 0:

        return 0

    if n == 0:

        return __________
                   (a)

    return __________
              (b)
```

i. (1.0 pt) Fill in blank (a).

ii. (3.0 pt) Fill in blank (b).

(b) (6.0 points)

Implement `park`, a generator function that takes a non-negative integer `n`. It yields (in any order) all possible strings representing `n` adjacent parking spots.

```
def park(n):
    """Yield the ways to park cars and motorcycles in n adjacent spots.

    >>> sorted(park(1))
    ['%', '.']
    >>> sorted(park(2))
    ['%%', '%.', '.%', '..', '<>']
    >>> sorted(park(3))
    ['%%%', '%%.', '%.%', '%..', '%<>', '.%%', '.%.', '..%', '...', '.<>', '<>%', '<>.']
    >>> len(list(park(4)))
    29
    """
    if n == 0:
        yield ''
    elif n > 0:
        for s in park(n-1):
            ---------
               (c)
            ---------
               (d)

        for s in _________:
                     (e)
            ---------
               (f)
```

i. (3.0 pt) Fill in blanks (c) and (d) with two lines of code. You may **not** use `yield from`.

ii. (1.0 pt) Fill in blank (e).

iii. (2.0 pt) Fill in blank (f).

(c) This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Implement `pack`, a generator function that takes a non-negative integer `n`. It yields (in any order) all possible strings representing `n` adjacent parking spots in which an equal number of cars and motorcycles are parked, no motorcycle is adjacent to a car, and no empty spot is adjacent to another empty spot.

```
def pack(n):
    """Yield the ways to park an equal number of cars and motorcycles
    in n adjacent spots with no motorcycle adjacent to a car and no
    empty spot adjacent to another empty spot.

    >>> sorted(pack(4))
    ['%.<>', '<>.%']
    >>> sorted(pack(7))
    ['%%.<><>', '<><>.%%']
    >>> sorted(pack(8))
    ['%%.<>.<>', '%%.<><>.', '%.%.<><>', '%.<><>.%', '.%%.<><>',
     '.<><>.%%', '<>.%%.<>', '<>.<>.%%', '<><>.%%.', '<><>.%.%']
    """
    def f(n, k):
        if n == 0 and k == 0:
            yield ''
        elif n > 0:
            yield from g(n-1, k-1, '<', '%' )
            yield from g(n-1, k,   '.', '.' )
            yield from g(n-2, k+1, '%', '<>')

    def g(n, k, no, yes):

        yield from _________

    yield from f(n, 0)
```

i. Fill in the blank.

No more questions.

CS 61A Structure and Interpretation of Computer Programs

Spring 2024

Midterm 1

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (8.0 points) What Would Python Display?

Assume the following code has been executed already.

```
one = 1

def choose(one):
    if big(one):
        print('A')
        if huge(one):
            print('B')
    elif big(one) or huge(one):
        print('C')
    if big(one) or print('D'):
        print('E')
    else:
        print('F')

big = lambda x: x >= one
huge = lambda x: x > one

def which():
    one = 3
    def this():
        return one
        return one + 1
    return this
    one = 4
```

(a) (6.0 pt) Which lines are displayed by the interactive Python interpreter after evaluating `choose(one + one)`? **Select all that apply.**

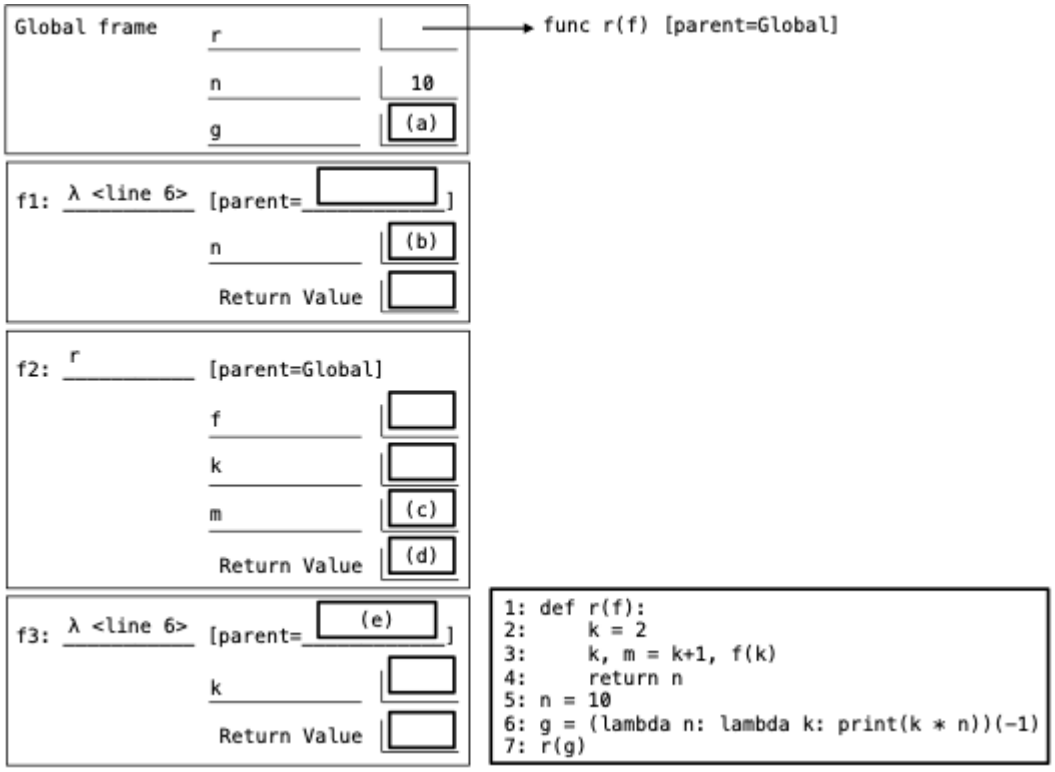
- ☐ `A`
- ☐ `B`
- ☐ `C`
- ☐ `D`
- ☐ `E`
- ☐ `None`
- ☐ None of the above

(b) (2.0 pt) What is displayed by the interactive Python interpreter after evaluating `which()()`?

- ◯ 2
- ◯ 3
- ◯ 4
- ◯ 5
- ◯ A function
- ◯ An error occurs before anything is displayed

2. (8.0 points) Silence of the Lambda

Complete the environment diagram below and then answer the questions that follow. There is one question for each labeled blank in the diagram. The blanks with no labels have no questions associated with them and are not scored. If a blank contains an arrow to a function, write the function as it would appear in the diagram.



(a) (2.0 pt) Fill in blank (a). The line number has been omitted for simplicity.

- ◯ -1
- ◯ 1
- ◯ -10
- ◯ `func lambda(n) [parent = Global]`
- ◯ `func lambda(k) [parent = Global]`
- ◯ `func lambda(n) [parent = f1]`
- ◯ `func lambda(k) [parent = f1]`

(b) (1.0 pt) Fill in blank (b).

- ◯ `-1`
- ◯ `2`
- ◯ `3`
- ◯ `10`

(c) (1.0 pt) Fill in blank (c).

- ◯ `-3`
- ◯ `-2`
- ◯ `20`
- ◯ `30`
- ◯ `None`

(d) (1.0 pt) Fill in blank (d).

- ◯ `-1`
- ◯ `2`
- ◯ `3`
- ◯ `10`

(e) (1.0 pt) Fill in blank (e).

- ◯ `Global`
- ◯ `f1`
- ◯ `f2`
- ◯ `f3`

(f) (2.0 pt) What is printed by the call to `print` on line 6?

- ◯ `-3`
- ◯ `-2`
- ◯ `20`
- ◯ `30`
- ◯ `None`

3. (8.0 points) Nearly Square

Implement `near_square`, which takes positive integer `n` and non-negative integer `k`. It returns the largest integer less than or equal to `n` which is the product of two positive integers that differ by `k` or less. You may use `solve`, which is provided.

```
def near_square(n, k):
    """Return the largest integer that is less than or equal to n and
    equals a * b for some positive integers a and b where abs(a - b) <= k.

    >>> near_square(125, 0)  # 11 * 11 = 121 and abs(11 - 11) = 0
    121
    >>> near_square(120, 3)  # 10 * 12 = 120 and abs(10 - 12) = 2
    120
    >>> near_square(120, 1)  # 10 * 11 = 110 and abs(10 - 11) = 1
    110
    """
    while True:

        gap = k

        while _______:
                (a)
            x = _______
                  (b)
            if _______:  # Check if x is a whole number
                 (c)

                return _______
                         (d)

            _______
              (e)

        _______
          (f)

def solve(b, c):
    """Returns the largest x for which x * (x + b) = c

    >>> solve(2, 120)  # x=10 solves x * (x + 2) = 120
    10.0
    >>> solve(2, 121)  # x=10.045... solves x * (x + 2) = 121
    10.045361017187261
    """
    return (b*b/4 + c) ** 0.5 - b/2
```

(a) (2.0 pt) Fill in blank (a). Select **all** that apply.

- ☐ `gap`
- ☐ `gap != 0`
- ☐ `gap > 0`
- ☐ `gap >= 0`

(b) (2.0 pt) Fill in blank (b).

(c) (1.0 pt) Fill in blank (c). The `round` function is demonstrated on the Midterm 1 Study Guide (page 2 righthand side).

- ◯ `round(x)`
- ◯ `x % 10`
- ◯ `x == round(x)`
- ◯ `x == x % 10`

(d) (1.0 pt) Fill in blank (d).

- ◯ `n`
- ◯ `k`
- ◯ `gap`
- ◯ `True`

(e) (1.0 pt) Fill in blank (e).

- ◯ `n = n - 1`
- ◯ `n = k - 1`
- ◯ `n = n - k`
- ◯ `gap = gap - 1`
- ◯ `gap = k - 1`
- ◯ `k = k - 1`

(f) (1.0 pt) Fill in blank (f).

- ◯ `n = n - 1`
- ◯ `n = k - 1`
- ◯ `n = n - k`
- ◯ `gap = gap - 1`
- ◯ `gap = k - 1`
- ◯ `k = k - 1`

4. (16.0 points) Nice Dice

A **dice integer** is a positive integer whose digits are all greater than or equal to 1 and all less than or equal to 6.

(a) (8.0 points)

Implement `streak`, which takes a positive integer `n`. It returns `True` if `n` is a **dice integer** whose digits are all the same and `False` otherwise.

```
def streak(n):
    """Return whether positive n is a dice integer in which all the digits are the same.

    >>> streak(22222)
    True
    >>> streak(4)
    True
    >>> streak(22322)  # 2 and 3 are different digits.
    False
    >>> streak(99999)  # 9 is not allowed in a dice integer.
    False
    """
    d = n % 10

    if _______:
         (a)

        return False

    while n:

        if _______:
             (b)

            _______
              (c)

        n = n // 10

    return _______
             (d)
```

i. (2.0 pt) Fill in blank (a).

◯ `d < 1 and d > 6`

◯ `d < 1 or d > 6`

◯ `n < 1 and n > 6`

◯ `n < 1 or n > 6`

◯ `min(n) < 1 and max(n) > 6`

◯ `min(n) < 1 or max(n) > 6`

ii. (2.0 pt) Fill in blank (b).

iii. (1.0 pt) Fill in blank (c).

- ◯ `d = n % 10`
- ◯ `d = (n // 10) % 10`
- ◯ `result = False`
- ◯ `return False`

iv. (1.0 pt) Fill in blank (d).

- ◯ `True`
- ◯ `result`
- ◯ `d == n`
- ◯ `d == n % 10`

v. (2.0 pt) Fill in the blank of `changes`, which takes a dice number `n`. It returns the number of digits of `n` that are different from the digit to their left.

```
def changes(n):
    """Return the number of adjacent digits that are different in dice number n.

    >>> changes(22222)
    0
    >>> changes(222255)
    1
    >>> changes(22322)
    2
    >>> changes(22366666622)
    3
    >>> changes(52431)
    4
    """
    count = 0
    while n >= 10:

        if _______:

            count = count + 1
        n = n // 10
    return count
```

◯ `n != n % 100`

◯ `n != n // 10`

◯ `n != (n // 10) % 10`

◯ `n != (n // 10) % 100`

◯ `n % 10 != n % 100`

◯ `n % 10 != n // 10`

◯ `n % 10 != (n // 10) % 10`

◯ `n % 10 != (n // 10) % 100`

(b) (2.0 pt) Fill in the blank of `count_if`, which takes a one-argument function `f`. It returns a function that takes a positive integer `n` and returns the count of its digits for which `f` returns a true value.

```
def count_if(f):
    """Return a function that takes a positive integer n and returns
    the count of its digits for which f returns a true value.

    >>> is_three = lambda d: d == 3
    >>> count_threes = count_if(is_three)  # count_threes returns the number of threes in n
    >>> count_threes(431663334231)         # 3 appears 5 times
    5
    """
    def g(n):
        count = 0
        while n:
            if _______:
                count += 1
            n = n // 10
        return count
    return g
```

(c) (2.0 pt) Fill in the blank of `count_at_least`, which takes a positive integer `k`. It returns a function that takes a positive integer `n` and returns the count of its digits that are greater than or equal to `k`.

```
def count_at_least(k):
    """Return a function that returns how many of the digits of its argument are at least k.

    >>> above_3 = count_at_least(3)  # above_3 returns the # of digits greater than or equal to 3
    >>> above_3(431663334231)
    9
    """
    return count_if(_______)
```

- ◯ `lambda: n % 10 >= k`
- ◯ `lambda: n >= k`
- ◯ `lambda: d >= k`
- ◯ `lambda: d >= n % 10`
- ◯ `lambda d: n % 10 >= k`
- ◯ `lambda d: n >= k`
- ◯ `lambda d: d >= k`
- ◯ `lambda d: d >= n % 10`
- ◯ `lambda d: lambda k: n % 10 >= k`
- ◯ `lambda d: lambda k: n >= k`
- ◯ `lambda d: lambda k: d >= k`
- ◯ `lambda d: lambda k: d >= n % 10`

(d) (4.0 pt) Fill in the blank of `times`, which takes a non-negative integer `k`, a positive integer `n`, and a digit `d`. It returns `True` if `n` contains digit `d` exactly `k` times and `False` otherwise. You may use `count_if`.

```
def times(k, n, d):
    """Returns whether n contains digit d exactly k times.

    >>> times(3, 6132344561, 4)   # 4 only appears 2 times
    False
    >>> times(3, 61323445614, 4)  # 4 appears exactly 3 times
    True
    """
    return _______
```

(e) (0.0 pt) This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Fill in the blank to implement `streak` using `count_if`. You may not write `else` or `*` or `**`.

The expression is long, so you may use multiple lines to write it.

```
def streak(n):
    """Return whether positive n is a dice integer in which all the digits are the same.

    >>> streak(22222)
    True
    >>> streak(4)
    True
    >>> streak(22322)  # 2 and 3 are different digits.
    False
    >>> streak(99999)  # 9 is not allowed in a dice integer.
    False
    """
    return ______
```

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Spring 2024

MIDTERM 2

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (7.0 points) What Would Python Display?

Assume the following code has been executed. The `Link` class appears on the midterm 2 study guide (page 2, left side).

```
def shake(it):
    if it is not Link.empty and it.rest is not Link.empty:
        if it.first + 1 < it.rest.first:
            it.rest = Link(it.rest.first-1, it.rest)
            shake(it)
        else:
            shake(it.rest)
it = Link(2, Link(5, Link(7)))
off = Link(1, it.rest)
shake(it)

def cruel(summer):
    while summer is not Link.empty:
        yield summer.first
        summer = summer.rest
        if summer is not Link.empty:
            summer = summer.rest
summer = Link(1, Link(2, Link(3, Link(4))))
```

Write the output printed for each expression below or *Error* if an error occurs.

(a) (2.0 pt) `print(it)`

- ◯ `<2 5 7>`
- ◯ `<2 4 5 7>`
- ◯ `<2 4 5 6 7>`
- ◯ `<2 3 4 5 7>`
- ◯ `<2 4 3 5 7>`
- ◯ `<2 3 4 5 6 7>`
- ◯ `<2 4 3 5 6 7>`

(b) (2.0 pt) `print(off)`

(c) (2.0 pt) `print([x*x for x in cruel(summer)])`

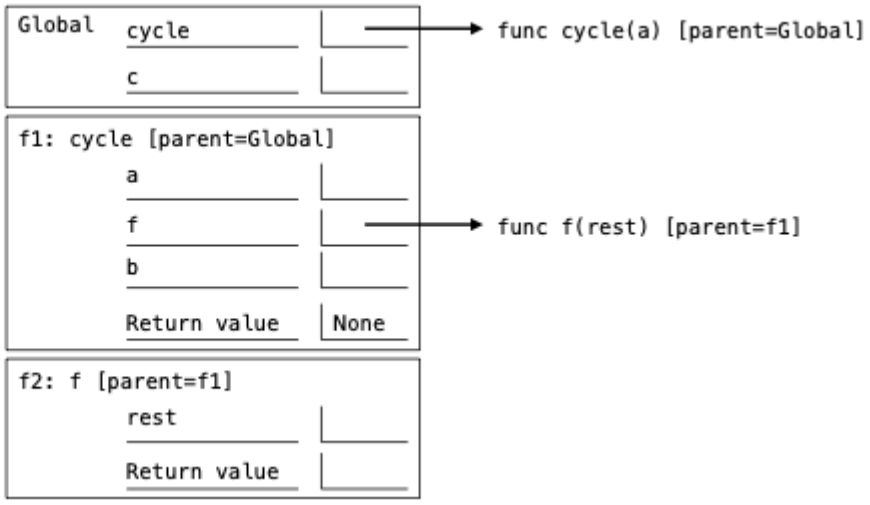
(d) (1.0 pt) What is the order of growth of the time it takes to evaluate `shake(Link(1, Link(n)))` in terms of `n`?

- ◯ exponential
- ◯ quadratic
- ◯ linear
- ◯ constant

2. (6.0 points) Spin Cycle

Complete the environment diagram below and then answer the questions that follow. Do not add frames for calls to built-in functions (such as `print`).

```
 1: def cycle(a):
 2:     def f(rest):
 3:         rest.append(a.pop())
 4:         return [rest.append(a)]
 5:     b = []
 6:     a = f(b)
 7:     b = b + [5]
 8:     print(a)
 9:     print(b)
10:
11: c = [2, 3, 4]
12: cycle(c)
13: print(c)
```



(a) (2.0 pt) What would be printed by the expression `print(a)` on line 8?

(b) (2.0 pt) What would be printed by the expression `print(b)` on line 9?

(c) (2.0 pt) What would be printed by the expression `print(c)` on line 13?

3. (8.0 points) Fearless

If you sing lyrics into a mic, every connected speaker repeats them.

A `Mic` instance has a dictionary `speakers` containing `Speaker` instances as values, each with its location (`str`) as its key. Its `sing` method takes a string `lyrics` and invokes the `repeat` method of each `Speaker` instance connected to it.

A `Speaker` takes a `transform` function that takes and returns a string. To `connect` a `Speaker` instance to `m` (`Mic`) in a `location` (`str`), add that instance to the `speakers` dictionary of `m` in that `location`. To `repeat` a signal `s` (`str`), return the result of calling the speaker's `transform` function on `s`.

Every `Mic` starts connected to a `Speaker` in the `Front` location that repeats the exact same signal it receives.

Implement the `Mic` and `Speaker` classes to match the doctests. The `str.lower` and `str.upper` functions return lowercased and uppercased versions of a string, respectively.

```
class Mic:
    """A microphone connected to speakers.

    >>> m = Mic()  # Front is connected automatically
    >>> m.sing('Is this thing on?')
    Front - Is this thing on?
    >>> Speaker(str.lower).connect(m, 'Left Side')
    >>> Speaker(str.upper).connect(m, 'Right Side')
    >>> m.sing("You belong with me.")
    Front - You belong with me.
    Left Side - you belong with me.
    Right Side - YOU BELONG WITH ME.
    """
    def __init__(self):
        self.speakers = _______
                          (a)

    def sing(self, lyrics):
        for k in self.speakers.keys():  # iterate over the keys of a dictionary
            print(k, '-', _______.repeat(lyrics))
                            (b)

class Speaker:
    def __init__(self, transform):
        self.transform = transform

    def connect(self, m, location):
        _______
          (c)

    def repeat(self, s):
        return _______
                 (d)
```

(a) (3.0 pt)

(b) (2.0 pt)

- ◯ `self`
- ◯ `self.speaker`
- ◯ `self.speakers`
- ◯ `self.speakers[k]`
- ◯ `self.speakers[k].self`
- ◯ `speaker`
- ◯ `speakers`
- ◯ `speakers[k]`
- ◯ `speakers[k].self`

(c) (2.0 pt)

(d) (1.0 pt)

- ◯ `s`
- ◯ `lyrics`
- ◯ `transform`
- ◯ `transform(s)`
- ◯ `transform(lyrics)`
- ◯ `self.transform`
- ◯ `self.transform(s)`
- ◯ `self.transform(lyrics)`

4. (29.0 points) Who's counting?

Definition. A *strip* is a list of integers in which each integer is one more than the last. For example, `[3, 4, 5, 6]` is a strip. Empty and one-element lists are strips.

(a) (4.0 points)

Implement `is_strip`, which takes a list of integers `s` and returns whether it is a strip.

```
def is_strip(s):
    """Return whether list s is a strip.

    >>> is_strip([3, 4, 5, 6])
    True
    >>> is_strip([3, 3, 3])        # 3 after 3
    False
    >>> is_strip([3, 4, 5, 4, 6])  # 4 after 5
    False
    >>> is_strip([3, 4, 5, 5, 6])  # 5 after 5
    False
    >>> is_strip([3, 4, 5, 6, 8])  # 8 after 6
    False
    >>> is_strip([5])
    True
    >>> is_strip([])
    True
    """
    assert type(s) == list
    return _______ or s == list ( range ( _______ , _______ ))
             (a)                              (b)       (c)
```

i. (1.0 pt) Fill in blank (a).

- ◯ `len(s) == 0`
- ◯ `s[0] + 1 == s[1]`
- ◯ `s[0] + 1 in s`
- ◯ `s[0] + 1 in s[1:]`

ii. (1.0 pt) Fill in blank (b).

iii. (2.0 pt) Fill in blank (c).

- ◯ `s`
- ◯ `s[0]`
- ◯ `s[1]`
- ◯ `s[0] + 1`
- ◯ `s[0] + len(s)`
- ◯ `s[len(s) - 1]`
- ◯ `s[1] - s[0]`

(b) (7.0 points)

Implement `drip`, which takes two non-empty lists of integers `s` and `t`. It returns `True` if there is a strip containing all and only the elements of `s` and `t` **starting with** `s[0]` in which the elements of `s` appear in order and the elements of `t` appear in order. It returns `False` otherwise.

```
def drip(s, t):
    """Return whether there is a strip made out of interleaving s and t.

    >>> drip([1, 3, 5], [2, 4, 6])    # 1 2 3 4 5 6
    True
    >>> drip([1, 4, 5], [2, 3, 6])    # 1 2 3 4 5 6
    True
    >>> drip([1, 2, 3], [4, 5, 6])    # 1 2 3 4 5 6
    True
    >>> drip([2, 4, 5], [1, 3, 6])    # No strip starting with 2 can contain 1
    False
    >>> drip([1, 2, 4, 5], [1, 3, 6]) # No strip can contain 1 and 1
    False
    >>> drip([1, 4, 5], [2, 3, 7])    # No strip can contain 5 and 7 but no 6
    False
    >>> drip([1, 5, 4], [2, 3, 6])    # No strip can contain 5 before 4
    False
    >>> drip([2], [3, 4, 5])          # 2 3 4 5
    True
    >>> drip([1], [2, 3, 5])          # No strip can contain 3 and 5 but no 4
    False
    """
    while s and t:
        if s[0] + 1 == t[0]:
            s, t = _______  # The next element of the strip is in t
                     (a)
        elif _______:
               (b)
            s = s[1:]       # The next element of the strip is in s
        else:
            return _______
                     (c)
    return _______
             (d)
```

i. (1.0 pt) Fill in blank (a).

◯ `t, s`

◯ `t, s[1:]`

◯ `t[1:], s`

ii. (3.0 pt) Fill in blank (b).

iii. (1.0 pt) Fill in blank (c).

- ◯ `False`
- ◯ `s[0] != t[0]`
- ◯ `is_strip(s) or is_strip(t)`
- ◯ `not (is_strip(s) or is_strip(t))`
- ◯ `is_strip(s) and is_strip(t)`
- ◯ `not (is_strip(s) and is_strip(t))`

iv. (2.0 pt) Fill in blank (d).

- ◯ `True`
- ◯ `is_strip(s)`
- ◯ `is_strip(t)`
- ◯ `is_strip(s) or is_strip(t)`
- ◯ `not (is_strip(s) or is_strip(t))`
- ◯ `is_strip(s) and is_strip(t)`
- ◯ `not (is_strip(s) and is_strip(t))`

(c) (6.0 points)

Implement `longest`, which takes a list of integers `s`. It returns the longest strip whose elements appear in s in order. If two such strips exist, return the one that starts earlier in `s`. **Hint:** `s[-1]` is the last element in list `s`.

```
def longest(s):
    """Return the longest strip whose elements appear in s in order.
    >>> longest([4, 2, 3, 5, 6, 4, 6, 5])  # 2 3 4 5 is the longest strip in s
    [2, 3, 4, 5]
    >>> longest([4, 2, 3, 5, 6, 4])  # 4 5 6 is as long as 2 3 4 and is earlier
    [4, 5, 6]
    >>> longest([2, 4, 6])
    [2]
    """
    if len(s) == 0:
        return []
    return max([longest_with_s0(s), _______ ], key= _______ )
                                       (a)             (b)
def longest_with_s0(s):
    """Return the longest strip in s that starts with s[0]."""
    result = _______
               (c)
    for k in s[1:]:
        if _______:
             (d)
            result.append(k)
    return result
```

i. (2.0 pt) Fill in blank (a).

ii. (1.0 pt) Fill in blank (b).

- ◯ `len`
- ◯ `len(s)`
- ◯ `s[0]`
- ◯ `lambda i: s[i]`

iii. (1.0 pt) Fill in blank (c).

- ◯ `[]`
- ◯ `[s]`
- ◯ `s[0]`
- ◯ `s[:1]`

iv. (2.0 pt) Fill in blank (d).

(d) (5.0 points)

Implement `has_strip`, which takes a `Tree` of integers `t`. It returns whether there is a strip containing the labels along some path from the root to a leaf of `t`. The `Tree` class is on the midterm 2 study guide (page 2 left side).

```
def has_strip(t):
    """Return whether the elements of some root-to-leaf path form a strip.

    >>> has_strip(Tree(1, [Tree(3, [Tree(4)]), Tree(2, [Tree(2), Tree(3)])])) # 1, 2, 3
    True
    >>> has_strip(Tree(1, [Tree(3, [Tree(4)]), Tree(2, [Tree(2, [Tree(3)])])]))
    False
    >>> has_strip(Tree(1, [Tree(3, [Tree(4)]), Tree(2, [Tree(2)])]))
    False
    >>> has_strip(Tree(1, [Tree(2, [Tree(4)]), Tree(3, [Tree(3)])]))
    False
    """
    if t.is_leaf():
        return _______
                 (a)
    for b in t.branches:
        if _______
             (b)
            return True
    return _______
             (c)
```

i. (1.0 pt) Fill in blank (a).

- ◯ `True`
- ◯ `False`
- ◯ `t`
- ◯ `t.label`
- ◯ `t.label == t + 1`
- ◯ `t + 1 == t.label`

ii. (3.0 pt) Fill in blank (b).

iii. (1.0 pt) Fill in blank (c).

- ◯ `True`
- ◯ `False`
- ◯ `t`
- ◯ `t.label`
- ◯ `t.label == t + 1`
- ◯ `t + 1 == t.label`

(e) (7.0 points)

Implement `strips`, a generator function that takes a `Tree` of integers `t`. It yields all strips that contain the labels along a path from the root to a leaf of `t`.

```
def strips(t):
    """Yield the paths from the root to a leaf of Tree t that form strips.

    >>> list(strips(Tree(1, [Tree(3, [Tree(4)]), Tree(2, [Tree(2), Tree(3)])])))
    [[1, 2, 3]]
    >>> list(strips(Tree(1, [Tree(2), Tree(2, [Tree(2), Tree(3, [Tree(4)]), Tree(3)])])))
    [[1, 2], [1, 2, 3, 4], [1, 2, 3]]
    """
    if t.is_leaf():
        yield _______
                (a)

    for b in t.branches:
        if _______:
             (b)

            for s in _______:
                       (c)

                yield _______
                        (d)
```

i. (2.0 pt) Fill in blank (a).

- ◯ `None`
- ◯ `t`
- ◯ `t.label`
- ◯ `[t]`
- ◯ `[t.label]`

ii. (1.0 pt) Fill in blank (b).

- ◯ `t.label == b.label + 1`
- ◯ `b.label == t.label + 1`
- ◯ `t.label == b.label + 1 and is_strip(b)`
- ◯ `b.label == t.label + 1 and is_strip(b)`

iii. (2.0 pt) Fill in blank (c).

- ◯ `strips(t)`
- ◯ `strips(b)`
- ◯ `t.branches`
- ◯ `b.branches`
- ◯ `range(len(t.branches))`
- ◯ `range(len(b.branches))`

iv. (2.0 pt) Fill in blank (d).

◯ `s`

◯ `t`

◯ `t + s`

◯ `t.label + s`

◯ `[t] + s`

◯ `[t.label] + s`

◯ `[t] + [s]`

◯ `[t.label] + [s]`

v. (0.0 pt) This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Fill in the blank of `only_strips`, which takes a Tree of integers `t`. It removes all nodes that are **not** on a *strip path*. A *strip path* is a path from the root to a leaf whose labels form a strip. You may use functions implemented earlier in this question.

```
def test_only_strips(t):
    """Call only_strips(t) and then return t. Assume has_strip(t) is True.

    >>> test_only_strips(Tree(1, [Tree(2), Tree(2, [Tree(2), Tree(3, [Tree(4)]), Tree(3)])]))
    Tree(1, [Tree(2), Tree(2, [Tree(3, [Tree(4)]), Tree(3)])])
    >>> test_only_strips(Tree(5, [Tree(6), Tree(6, [Tree(7, [Tree(9)])]), Tree(7)]))
    Tree(5, [Tree(6)])
    """
    only_strips(t)
    return t

def only_strips(t):
    """Remove all nodes of Tree t that are not on a path from the root to a leaf
    whose labels form a strip. Assume has_strip(t) is True.
    """
    t.branches = [b for b in t.branches if _______]
```

No more questions.

CS 61A Structure and Interpretation of Computer Programs

Spring 2025

MIDTERM 1

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (6.0 points) What Would Python Print?

Answer the questions about this code. No errors occur while it is executed.

```
def double(x):
    return 2 * x

def square(f):
    return lambda x: f(x) * f(x)

def inc(f):
    return lambda x: f(x + 1)

def triple(f):
    return lambda x: f(f(f(x)))

def put(x):
    return lambda f: f(x)

one = put(1)
triple(print)(5)
```

(a) (2.0 pt) What is displayed by the last line?

(b) (2.0 pt) What would be displayed by evaluating `print(one(square(double)))`

(c) (2.0 pt) What would be displayed by evaluating `print(inc(put)(1)(triple(double)))`

(d) (0.0 pt) This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

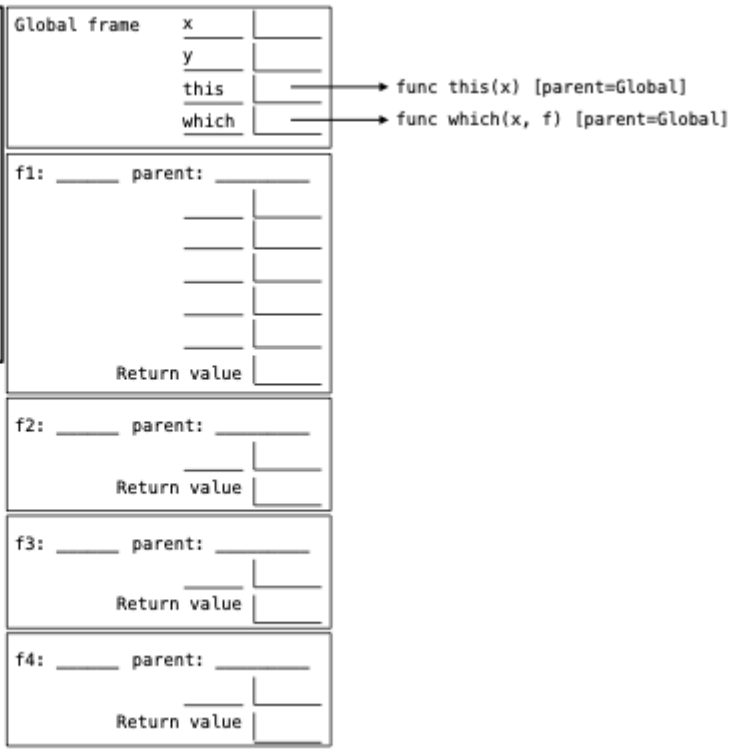
Write one line of code just after `triple(print)(5)` that would double and add one to all numbers in the answers to parts (b) and (c) above. For example, for an answer containing 100, it would now contain 201 instead.

You **may not** use any numbers or arithmetic symbols (such as `+` or `*`) or arithmetic functions (such as `add` or `mul`). You **may not** reassign `triple` or `square`. You may use multiple assignment: `x, y = ___, ___`

2. (8.0 points) Which One

Complete the environment diagram below to answer the questons about the calls to `print`. Only the questions will be scored, not the diagram. Each call to `print` is evaluated once, and there are no errors caused by running this code.

```
 1: x, y = 1, 2
 2:
 3: def this(x):
 4:     x = 3
 5:     print(y)
 6:     return 4
 7:
 8: def which(x, f):
 9:     def this(x):
10:         x = 5
11:         return y
12:     y = 6
13:     y = f(x)
14:     print(this(y))
15:     return lambda: (print(x) or y)
16:
17: print(which(7, this)())
```



(a) (2.0 pt) What is displayed by the call to `print` on line 5?

(b) (2.0 pt) What is displayed by the call to `print` on line 14?

(c) (2.0 pt) What is displayed by the call to `print` on line 15?

(d) (2.0 pt) What is displayed by the call to `print` on line 17?

3. (13.0 points) Legit Digit

(a) (4.0 points)

Implement `all_digits`, which takes a positive integer `n` and one-argument function `cond` that always returns `True` or `False`. It returns `True` if `cond(d)` returns `True` when called on every digit `d` in `n`, and `False` otherwise.

```
def all_digits(n, cond):
    """Return whether cond returns true for every digit of positive n.

    >> odd = lambda d: d % 2 == 1
    >>> all_digits(123, odd)  # not all digits are odd
    False
    >>> all_digits(357, odd)  # all digits are odd
    True
    """

    while n:

        if _______:
             (a)

            _______
              (b)

        n = n // 10

    return _______
             (c)
```

i. (2.0 pt) Fill in blank (a).

ii. (1.0 pt) Fill in blank (b).

- ◯ `n = n % 10`
- ◯ `n = n * 10`
- ◯ `return True`
- ◯ `return False`
- ◯ `return odd`
- ◯ `return cond`

iii. (1.0 pt) Fill in blank (c).

- ◯ `n`
- ◯ `True`
- ◯ `False`
- ◯ `odd(n)`
- ◯ `cond(n)`

(b) (3.0 points)

Definition. A *prefix* of a positive integer `n` is the value of `n//pow(10, p)` for some non-negative integer `p`. For example, 3456 has prefixes 3456, 345, 34, 3, and 0.

Implement `prefix_digits`, which takes a positive integer `n` and a one-argument function `cond` that always returns `True` or `False`. It returns the largest prefix of `n` for which `cond` returns `True` when called on every digit of the prefix. You may call `all_digits` and `process`.

```
def process(n, check):
    """A function to help implement prefix_digits."""
    while n:
        if check(n):
            return n
        n = n // 10
    return 0

def prefix_digits(n, cond):
    """Return the largest prefix of positive n for which cond returns true for every digit.

    >>> odd = lambda d: d % 2 == 1
    >>> prefix_digits(94720, odd)
    9
    >>> prefix_digits(919321, odd)
    9193
    >>> prefix_digits(2025, odd)
    0
    >>> prefix_digits(20252025, lambda d: d < 4)
    202
    >>> prefix_digits(20252025, lambda d: True)
    20252025
    """
    return process(n, lambda k: _______ )
                                  (d)
```

i. (3.0 pt) Fill in blank (d).

- ◯ `all_digits(n, cond)`
- ◯ `all_digits(k, cond)`
- ◯ `all_digits(n, cond(n))`
- ◯ `all_digits(k, cond(k))`
- ◯ `lambda cond: all_digits(n, cond)`
- ◯ `lambda cond: all_digits(k, cond)`
- ◯ `lambda cond: all_digits(n, cond(n))`
- ◯ `lambda cond: all_digits(k, cond(k))`
- ◯ `all_digits(n, lambda n: cond)`
- ◯ `all_digits(k, lambda n: cond)`
- ◯ `all_digits(n, cond(lambda n: n))`
- ◯ `all_digits(k, cond(lambda n: k))`

(c) (6.0 points)

Re-implement `prefix_digits`, which takes a positive integer `n` and a one-argument function `cond` that always returns `True` or `False`. It returns the largest prefix of `n` for which `cond` returns `True` when called on every digit of the prefix. You **may not** call `all_digits` or `process`.

```
def prefix_digits(n, cond):
    """Return the largest prefix of positive n for which cond returns true for every digit.

    >>> odd = lambda d: d % 2 == 1
    >>> prefix_digits(94720, odd)
    9
    >>> prefix_digits(919321, odd)
    9193
    >>> prefix_digits(2025, odd)
    0
    >>> prefix_digits(20252025, lambda d: d < 4)
    202
    """
    k = 0
    while n >= ________:
                  (e)
        if cond(_______):
                  (f)
            k += 1
        else:
            n = n // 10
            -------
              (g)
    return n
```

i. (2.0 pt) Fill in blank (e).

- ◯ `0`
- ◯ `k`
- ◯ `k // 10`
- ◯ `pow(10, k)`

ii. (2.0 pt) Fill in blank (f).

- ◯ `n % 10`
- ◯ `n // 10 % 10`
- ◯ `n % pow(10, k)`
- ◯ `n // pow(10, k)`
- ◯ `n // 10 % pow(10, k)`
- ◯ `n // pow(10, k) % 10`

iii. (2.0 pt) Fill in blank (g). You may not call `all_digits` or `process`.

4. (4.0 points) Hailstone Returns

Definition. A *hailstone sequence* is formed by picking a positive integer `n` as the start then repeatedly updating `n` until it is 1 using this rule: if `n` is even, divide it by 2; if `n` is odd, multiply it by 3 and add 1.

Implement `hailstone`, which prints each update in a hailstone sequence by displaying: the update number (counting up from 1), the current `n`, an `->` arrow symbol, and the updated `n`.

This implementation may not be possible using the template. If that's the case, respond *Impossible* to the following questions.

```
def hailstone(n):
    """Print numbered updates in the hailstone sequence.

    >>> hailstone(10)
    1 10 -> 5
    2 5 -> 16
    3 16 -> 8
    4 8 -> 4
    5 4 -> 2
    6 2 -> 1
    """
    def f():
        if n % 2 == 1:
            m = 3 * n + 1
        else:
            m = n // 2
        -------
          (a)
        -------
          (b)
    k = 1
    while n > 1:
        k, n = k + 1, f()
```

(a) (2.0 pt) Fill in blank (a).

- ◯ `print(k, n, ->, m)`
- ◯ `print(k, n, '->', m)`
- ◯ `print(k - 1, n, ->, m)`
- ◯ `print(k - 1, n, '->', m)`
- ◯ *Impossible*

(b) (2.0 pt) Fill in blank (b). If the problem is not possible using the template, write *Impossible*.

5. (4.0 points) It Takes Two

Implement `at_least_two`, which takes three arguments. It returns `True` if at least two of them are true values and `False` otherwise. The built-in `bool` function takes one argument and returns `True` if it is a true value and `False` if it is a false value.

```
def at_least_two(x, y, z):
    """Returns whether at least two of the arguments are true values.
    >>> at_least_two(1 + 1, 3 + 3, 5 + 5)
    True
    >>> at_least_two(1 + 1, 3 - 3, 5 + 5)
    True
    >>> at_least_two(1 - 1, 3 + 3, 5 + 5)
    True
    >>> at_least_two(1 - 1, 3 + 3, 0)
    False
    >>> at_least_two(1 - 1, 0, 0)
    False
    """
    if _______:
         (a)
        return _______
    else:        (b)
        return _______
                 (c)
```

(a) (2.0 pt) Fill in blank (a).

- ◯ `x`
- ◯ `x == True`
- ◯ `x and y`
- ◯ `x == True and y == True`

(b) (1.0 pt) Fill in blank (b). *Select all correct answers.*

- ☐ `y or z`
- ☐ `y and z`
- ☐ `bool(y or z)`
- ☐ `bool(y and z)`
- ☐ `not(y or z)`
- ☐ `not(y and z)`

(c) (1.0 pt) Fill in blank (c). *Select all correct answers.*

- ☐ `y or z`
- ☐ `y and z`
- ☐ `bool(y or z)`
- ☐ `bool(y and z)`
- ☐ `not(y or z)`
- ☐ `not(y and z)`

No more questions.

CS 61A Spring 2025

Structure and Interpretation of Computer Programs

MIDTERM 2

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (6.0 points) What Would Python Display?

Assume the following code has been executed. The `pop` method of a list removes and returns its last element.

```
s = [2, [5, 8], 11]
t = [1, 3, 5, 7, 9]

def add_t(x):
    if isinstance(x, list):
        return [add_t(y) for y in x]
    else:
        return x + t.pop()

m = map(add_t, s)
```

Write the value of each expression below or *Error* if an error occurs. Assume that each expression is evaluated in order and sequentially, so evaluating the first could affect the value of the second. Write *Map* for a map object and *Function* for a function.

(a) (2.0 pt) `[x[1] for x in s if isinstance(x, list)]`

(b) (1.0 pt) `next(m)`

- ◯ 3
- ◯ 5
- ◯ 7
- ◯ 9
- ◯ 11

(c) (1.0 pt) `t.pop()`

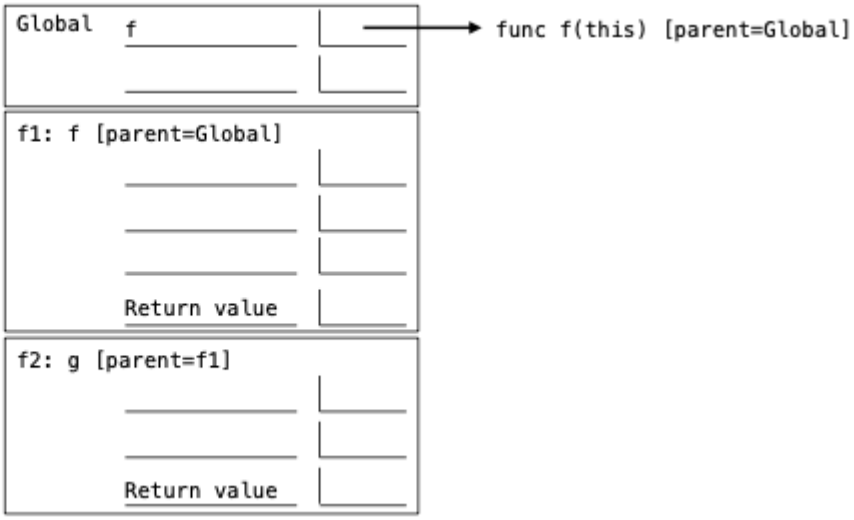
- ◯ 1
- ◯ 3
- ◯ 5
- ◯ 7
- ◯ 9

(d) (2.0 pt) `[next(m) for z in range(2)]`

2. (7.0 points) This or That

Answer the questions about the code below. Draw an environment diagram, but the diagram itself will not be scored.

```
 1: def f(this):
 2:     that = this[1]
 3:     def g(copy):
 4:         that[0] = 4
 5:         copy[0] = 5
 6:         copy[1].append(6)
 7:         print(copy)
 8:     g(list(this))
 9:     this.append(that)
10:     that.append(7)
11:
12: t = [1, [2], 3]
13: f(t)
14: print(t)
```



(a) (3.0 pt) What is displayed by the call to `print` on line 7?

(b) (3.0 pt) What is displayed by the call to `print` on line 14?

(c) (1.0 pt) What is the order of growth of the time it takes to execute `result = nest(n)` in terms of positive integer `n`? Assume that `append` always takes one step (constant time).

```
def nest(n):
    """Create a nested list.
    >>> nest(3)
    [[[[], []], [[], []]], [[[], []], [[], []]]]
    """
    s = []
    first = s
    for x in range(n):
        next_s = []
        s.append(next_s)
        s.append(next_s)
        s = next_s
    return first
```

◯ exponential

◯ quadratic

◯ linear

◯ logarithmic

◯ constant

3. (10.0 points) Berkeley Time

(a) (5.0 points)

An `Event` instance has `start` and `end` attributes that are positive integers representing the number of minutes after midnight that the `Event` begins and concludes.

Implement the `Event` class. The `split` method takes an integer `k` that **evenly divides** the difference between the `end` and `start` times. It is a generator function that yields `k` sequential events of equal length that together span the same time interval as the original event. Calling `split` **does not** change the event.

```
class Event:
    """An Event has a start and end time and can be split into equal-length shorter events.
    >>> calapalooza = Event(980, 1180)
    >>> list(calapalooza.split(4))  # Split it into 4 smaller events that go from 980 to 1180.
    [Event(980, 1030), Event(1030, 1080), Event(1080, 1130), Event(1130, 1180)]
    """
    def __init__(self, start, end):
        self.start, self.end = start, end
    def __repr__(self):
        return f'Event({self.start}, {self.end})'
    def split(self, k):
        assert (self.end - self.start) % k == 0  # k evenly divides the duration of the event.
        length = (self.end - self.start) // k

        start = _______
                  (a)
        for i in range(k):

            yield _______
                    (b)

            _______
              (c)
```

i. (1.0 pt) Fill in blank (a).

- ◯ `0`
- ◯ `None`
- ◯ `self`
- ◯ `start`
- ◯ `self.start`

ii. (3.0 pt) Fill in blank (b).

iii. (1.0 pt) Fill in blank (c).

- ◯ `start = start + length`
- ◯ `start = self.start + length`
- ◯ `self.start = start + length`
- ◯ `self.start = self.start + length`

(b) (5.0 points)

A `BerkeleyEvent` is an `Event` with a modified `split` method. The first `Event` yielded by `split` starts at the same time as the original event, but all events after the first one are 10 minutes shorter and start 10 minutes after the previous event ends. Again, assume `k` evenly divides the difference between the `end` and `start` attributes.

```
class BerkeleyEvent(Event):
    """When a BerkeleyEvent is split, there is time between events.
    >>> list(BerkeleyEvent(840, 930).split(3))
    [Event(840, 870), Event(880, 900), Event(910, 930)]
    """
    time = 10  # How much time between events
    def split(self, k):
        first = True
        for e in _______:
                    (d)
            if first:
                first = False
            else:
                _______.start = _______ + _______
                  (e)             (f)       (g)
            yield e
```

i. (2.0 pt) Fill in blank (d).

ii. (1.0 pt) Fill in blank (e).

- ◯ `e`
- ◯ `self`
- ◯ `self.e`

iii. (1.0 pt) Fill in blank (f).

- ◯ `self`
- ◯ `self.start`
- ◯ `self.end`
- ◯ `e`
- ◯ `e.start`
- ◯ `e.end`

iv. (1.0 pt) Fill in blank (g). **Select all that apply.**

- ☐ `time`
- ☐ `self.time`
- ☐ `self.BerkeleyEvent.time`
- ☐ `BerkeleyEvent.time`
- ☐ `Event.time`

4. (11.0 points) DeepSeek

Definition. The *depth* of the root node of a tree is 0. The depth of each other node is one plus the depth of its parent.

(a) (5.0 points)

Implement `deepest`, which takes a `Tree` instance `t` and a value `x`. It returns the largest depth of a node labeled `x` in `t`. If there are no nodes labeled `x` in `t`, then `deepest(t, x)` returns `None`.

The `Tree` class appears on the Midterm 2 study guide (page 2, left column).

```
def deepest(t, x):
    """Return the maximum depth of a node labeled x in Tree t.
    Return None if x is not a label in t.

    >>> example = Tree(30, [Tree(20), Tree(20, [Tree(30, [Tree(20)])])])
    >>> deepest(example, 30)
    2
    >>> deepest(example, 20)
    3
    >>> print(deepest(example, 10))
    None
    """
    below = [deepest(b, x) for b in t.branches]
    depths = [ _______ ]
                (a)
    if _______:
         (b)
        depths._______
                (c)
    if depths:
        return max(depths)
    else:
        return None
```

i. (3.0 pt) Fill in blank (a). Include `for` and `in` and `if` in your answer.

ii. (1.0 pt) Fill in blank (b).

- ◯ `x in depths`
- ◯ `depths`
- ◯ `not depths`
- ◯ `t == x`
- ◯ `t.label == x`
- ◯ `t.is_leaf()`

iii. (1.0 pt) Fill in blank (c).

(b) (6.0 points)

Implement `deep`, which takes a `Tree` instance `t`. It returns a dictionary containing each unique node label of `t` as a key. The value for each key `k` is the largest depth of any node in `t` labeled `k`.

IMPORTANT: You may not call `deepest` in your implementation.

Hint: For a dictionary `d`, `d.get(1, 2)` returns the value for the key `1` if that key appears in `d` and 2 otherwise.

```
def deep(t):
    """Return a dictionary containing each unique label of t as a key
    and the maximum depth of a node labeled with that key as its value.

    >>> deep(Tree(30, [Tree(20), Tree(20, [Tree(30, [Tree(20)])])]))
    {30: 2, 20: 3}
    """
    result = {}
    def visit(node, depth):
        _______ = max(_______, result.get(_______, 0))
          (d)           (e)                 (f)
        for b in node.branches:
            -------
              (g)
    visit(t, 0)
    return result
```

i. (2.0 pt) Fill in blank (d).

ii. (1.0 pt) Fill in blank (e).

- ◯ 0
- ◯ 1
- ◯ `depth`
- ◯ `t.label`
- ◯ `node.label`

iii. (1.0 pt) Fill in blank (f).

- ◯ `depth`
- ◯ `t`
- ◯ `t.label`
- ◯ `node`
- ◯ `node.label`

iv. (2.0 pt) Fill in blank (g).

5. (16.0 points) Just Add and Multiply

Definition. A plus-times-expression for a list (or linked list) of numbers inserts either `+` or `*` between each adjacent pair of numbers. For example, `2*3+4+5` is one possible plus-times-expression for `[2, 3, 4, 5]`.

(a) (5.0 points)

Implement `muladd`, which takes a **linked list** of numbers `s`. It returns the value of the plus-times-expression that starts with `*` and then alternates between `+` and `*`. If `s` has only one element, `muladd` returns that element.

The `Link` class is defined on the left column of page 2 of the Midterm 2 Study Guide.

```
def muladd(s):
    """Combine the numbers in linked list s by alternately multiplying and adding.

    >>> example = Link(9, Link(4, Link(7, Link(2, Link(0)))))  # <9 4 7 2 0>
    >>> muladd(example)           # 9 * 4 + 7 * 2 + 0, not 9 * (4 + 7 * (2 + 0))
    50
    >>> muladd(Link(2, example))  # 2 * 9 + 4 * 7 + 2 * 0
    46
    >>> muladd(Link(2))
    2
    """
    if s is Link.empty:

        return 0

    elif s.rest is Link.empty:

        return _______
                 (a)
    else:

        return _______
                 (b)
```

i. (1.0 pt) Fill in blank (a).

- ◯ `0`
- ◯ `1`
- ◯ `s.first`
- ◯ `s.rest`
- ◯ `s.rest.first`

ii. (4.0 pt) Fill in blank (b). If your answer is too long, you can continue on another line.

(b) (5.0 points)

Implement `ways`, which takes positive integers `k` and `n`. It returns the number of possible plus-times-expressions for a list of length `n` that have at most `k` `*`-symbols in a row. For example, `2+3*4*5*6+7*8*9+10+11*12*13*14+15*16` has 3 in a row `3*4*5*6`, then 2 in a row `7*8*9`, then 3 in a row `11*12*13*14`, then 1 in a row `15*16`.

```
def ways(k, n):
    """Return the number of plus-times-expressions with at most k consecutive *'s for n numbers.

    >>> ways(1, 4) # For [2, 3, 4, 5], these 5 ways: 2+3+4+5, 2+3+4*5, 2+3*4+5, 2*3+4+5, 2*3+4*5
    5
    >>> ways(2, 4) # For [2, 3, 4, 5], the 7 ways would also include 2+3*4*5 and 2*3*4+5
    7
    >>> ways(2, 5) # Some examples for [2, 3, 4, 5, 6]: 2+3*4+5*6 & 2*3*4+5*6 (but not 2+3*4*5*6)
    13
    """
    def f(left, n):

        if _______:
             (c)

            return 1

        result = f( _______ , n - 1)
                      (d)
        if left:

            result += _______
                        (e)
        return result

    return f(k, n)
```

i. (1.0 pt) Fill in blank (c).

- ◯ `left == 0`
- ◯ `left == 1`
- ◯ `n == 0`
- ◯ `n == 1`

ii. (1.0 pt) Fill in blank (d).

- ◯ `left`
- ◯ `left - 1`
- ◯ `k`
- ◯ `k - 1`

iii. (3.0 pt) Fill in blank (e).

(c) (6.0 points)

Implement `close`, which takes a list of numbers `s` and a number `x`. It returns the number closest (in absolute value) to `x` that is the value of some plus-times-expression for `s`.

Hint: A slice that starts past the end of a list is empty. For example, `[2, 3, 4][3:]` evalautes to `[]`.

The `product` function used to implement `close` is defined below.

```
def close(s, x):
    """Return the value of a plus-times-expression for s that is closest to x.

    >>> print(close([1, 2, 3, 4], 10))  # 1 + 2 + 3 + 4 = 10
    10
    >>> print(close([1, 2, 3, 4], 20))  # 1 * 2 * 3 * 4 = 24 (4 away from 20)
    24
    >>> print(close([1, 2, 3, 4], 30))  # 1 + 2 * 3 * 4 = 25 (5 away from 30)
    25
    >>> print(close([3, 7, 5, 4, 2, 6], 66))  # 3 * 7 + 5 * 4 * 2 + 6 = 67 (1 away from 66)
    67
    """
    if not s:
        return 0
    choices = [product(s[:i]) + _______ for i in range(1, len(s) + 1)]
                                  (f)

    return min(choices, key=_______)
                             (g)

def product(s):
    """Return the result of multiplying together the elements of a non-empty list of numbers s."""
    if len(s) == 1:
        return s[0]
    return s[0] * product(s[1:])
```

i. (4.0 pt) Fill in blank (f).

ii. (2.0 pt) Fill in blank (g). **Select all that apply.** Assume `sub` has been imported from the `operator` module. The `sub` function takes two arguments and subtracts the second from the first.

- ☐ `abs`
- ☐ `sub`
- ☐ `abs(sub)`
- ☐ `sub(abs)`
- ☐ `lambda y: abs(x - y)`
- ☐ `lambda x, y: abs(x - y)`

(d) (0.0 points)

This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Fill in the blank of `close_exp`, which takes a list of numbers `s` and a number `x`. It returns a string containing a plus-times-expression for `s` that evaluates to a number that is as close (in absolute value) as possible to `x`. (If there is more than one such expression, return any of them.)

The built-in `eval` function takes a string containing an expression and returns its value. For example, `eval('2+2')` evaluates to 4.

```
def close_exp(s, x):
    """Return the plus-times-expression of s that has a value closest to x.

    >>> print(close_exp([1, 2, 3, 4], 10))  # 1 + 2 + 3 + 4 = 10
    1+2+3+4
    >>> print(close_exp([1, 2, 3, 4], 20))  # 1 * 2 * 3 * 4 = 24 (4 away from 20)
    1*2*3*4
    >>> print(close_exp([1, 2, 3, 4], 30))  # 1 + 2 * 3 * 4 = 25 (5 away from 30)
    1+2*3*4
    >>> print(close_exp([3, 7, 5, 4, 2, 6], 66))  # 3 * 7 + 5 * 4 * 2 + 6 = 67 (1 away from 66)
    3*7+5*4*2+6
    """
    goal = close(s, x)
    term = ''
    for i in range(len(s)):
        if term:
            term += '*'
        term += str(s[i])
        candidate = term
        if i < len(s) - 1:
            candidate += _________________________
        if eval(candidate) == goal:
            return candidate
```

i. (0.0 pt) Fill in the blank. If your answer is too long, you can continue on another line.

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Summer 2019 Midterm

INSTRUCTIONS

- You have 3 hours to complete the exam individually.
- The exam is closed book, closed notes, closed computer, and closed calculator, except for two hand-written 8.5" × 11" crib sheet of your own creation.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last (Family) Name	
First (Given) Name	
Student ID Number	
Berkeley Email	
Teaching Assistant	
Exam Room	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, and `abs`.
- You **may not** use example functions defined on your study guides unless a problem clearly states you can.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.

1. (10 points) WWPD

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. Each expression has at least one line of output.

- If an error occurs, write **Error**, but include all output displayed before the error.
- To display a function value, write **Function**.
- If an expression would take forever to evaluate, write **Forever**.
- If nothing is displayed, write **Nothing**

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have started `python3` and executed the code shown on the left first, then you evaluate each expression on the right in the order shown. Expressions evaluated by the interpreter have a cumulative effect.

```
1  d = {}
2  x = 2
3  y = 9
4  p = lambda : print('crikey')
5
6  def f(a, b):
7      if b or print(a):
8          print(a, b)
9      if b > a:
10         return a + b
11     else:
12         return "less"
13
14 def g(f, x, y):
15     def m(n):
16         if f(n):
17             m = f
18             print('bonk')
19         else:
20             m = p
21             print('ers')
22         return m(n - 1)
23     return m(x) and f(x)
24
25 def h(x):
26     if x not in d:
27         d[x] = 'yes'
28         return False
29     else:
30         return True
31
32 def m(x):
33     d = {}
34
35 def e(x):
36     print(h(x))
37     m(x)
38     print(h(x))
```

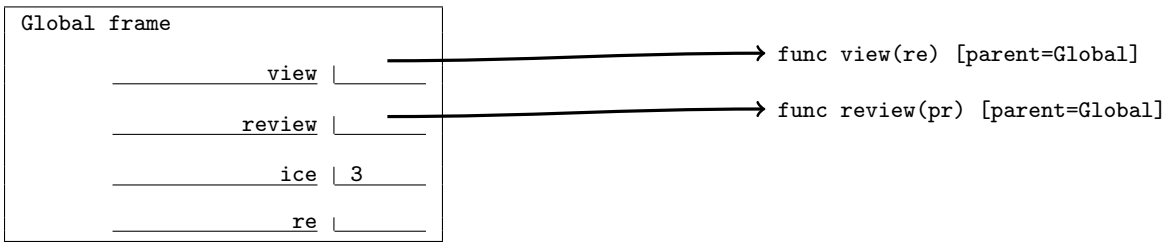
Expression	**Interactive Output**
`print(4, 5) + 1`	4 5 **Error**
`(p() and p()) or p()`	
`print(e(5))`	
`f(x, y)`	
`g(h, 4, 10)`	
`g(lambda x: x % 2, y, x)`	

2. (10 points) Environment Diagram

Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*

A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.



f1: _____ [parent: _____]

Return Value

f2: _____ [parent: _____]

Return Value

f3: _____ [parent: _____]

Return Value

f4: _____ [parent: _____]

Return Value

```
1  def view(re):
2      ice = 4
3      def k(means):
4          return re(means)
5      return k
6
7  def review(pr):
8      return re(pr * 2)
9
10 ice = 3
11 re = view(lambda pr: pr * ice)
12 review(1)
```

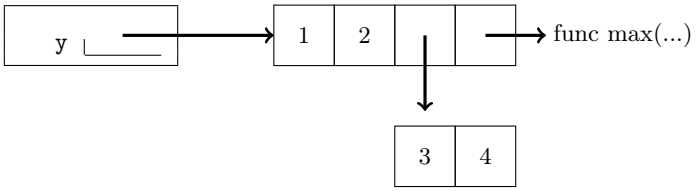
3. (8 points) Boxing Day

Draw box-and-pointer diagrams for the state of the lists after executing each block of code below. Each box should contain a value or a pointer to an object, as in an environment diagram.

You don't need to write index numbers or the word `list`. Please erase or cross out any boxes or pointers that are not part of a final diagram.

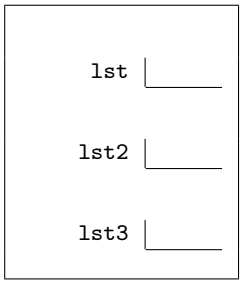
An example is given below:

```
1  y = [1, 2, [3, 4], max]
```



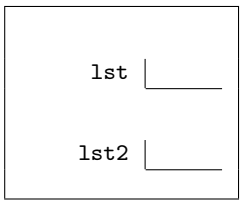
(a) (3 pt)

```
1  lst = [2, 4, lambda: lst]
2  lst2 = lst
3  lst = lst[2:]
4  lst3 = lst[0]()
```



(b) (3 pt)

```
1  lst = [[5], 2, 4, 10]
2  lst2 = lst[1:3] + lst[:3]
3  for n in lst2[:2]:
4      lst.append(lst2[n])
```



(c) (2 pt)

```
1  lst = ['goodbye', 0, None, 8, 'hello', 1]
2  while lst.pop():
3      x = lst.pop()
4      if x:
5          lst.pop()
6      else:
7          lst.append('three')
```

4. (2 points) Conceptual Questions

There are three problems here, which cover topics from this week's lecture. Each problem is worth 1 point, but you can only earn a maximum of 2 points on this problem, so you only need to know two answers.

Please make sure to fill in the bubble completely when answering. Each question has only one right answer.

(a) (1 pt) Mutable Functions

Declaring a variable nonlocal allows you to modify a binding for that variable in:

◯ Any parent frame ◯ The global frame ◯ Any parent frame other than global ◯ The current frame

(b) (1 pt) Growth

Out of these options, what's the best way to measure the performance of a program?

◯ Run the program many times and calculate the average time it takes to run
◯ Count the number of operations executed by the program for different inputs
◯ Count how many while and for loops the code contains

(c) (1 pt) Iterators and Generators

Select the statement that is ***not*** true. A generator function is a function that...

◯ is an iterator ◯ returns an iterator ◯ returns a generator ◯ contains the "yield" keyword

5. (4 points) Negative Feedback

Write a function `clear_negatives` that takes in a list containing integers, and returns a list which is identical to the original, but in each case where a negative integer appears, delete a number of elements from the list equivalent to the magnitude of the negative integer.

For example, if -4 occurs in the list, remove a total of four elements: -4 and the next three elements. See the doctests for examples.

Your solution ***must*** use recursion, and should not modify the input list.

```
def clear_negatives(lst):
    """
    >>> clear_negatives([1, 0, 6])
    [1, 0, 6]
    >>> clear_negatives([3, -2, 1, 6, 2])
    [3, 6, 2]
    >>> lst = [-3, -4, 5, 5]
    >>> clear_negatives(lst)
    [5]
    >>> lst
    [-3, -4, 5, 5]
    """

    if ____________________________________________________________________________:

        return []

    elif ____________________________________________________________________________:

        return ____________________________________________________________________

    else:

        return ____________________________________________________________________
```

6. (6 points) Your Guess Is As Good As Mine

Let's play a guessing game! In order to do this, we'll use higher order functions.

Write a function, `make_guess`, which takes in a number that we want someone to try to guess, and returns a guessing function.

A *guessing function* is a one-argument function which takes in a number. If the number passed in is the number we wanted to guess, it will return the number of incorrect guesses made prior to the correct guess. Otherwise, it returns another guessing function, which keeps track of the fact that we've made an incorrect guess.

Solutions which use lists, object mutation, nonlocal, or global will receive **no credit.**

```
def make_guess(n):
    """
    >>> guesser = make_guess(10)
    >>> guess1 = guesser(6)
    >>> guess2 = guess1(7)
    >>> guess3 = guess2(8)
    >>> guess3(10)
    3
    >>> guess2(10)
    2
    >>> a = make_guess(5)(1)(2)(3)(4)(5)
    >>> a
    4
    """
    def update_guess(num_incorrect):

        def new_guess(x):

            if ____________________________________________________________:

                ____________________________________________________________

            else:

                ____________________________________________________________

        ____________________________________________________________________

    return ____________________________________________________________________
```

7. (12 points) Scrabbled Eggs

In this problem, we will build (part of) the ultimate Scrabble AI! Scrabble is a game where, given a collection of letters that each have a point value assigned, you try to make the word with the greatest score.

For example, if "a" and "s" are both worth 1 point, and "b" and "c" are both worth 2 points, the word "cab" would get 5 points, and "sass" would get 4 points.

(a) (8 pt) Implement the function `is_subseq` which takes two strings, `w1` and `w2`, and returns `True` if `w1` is a subsequence of `w2`.

Recall from Lab 5 that a string is a subsequence of another string if all the letters in the first string appear in the second string in the same order, although not necessarily directly next to each other. Put another way, you can transform the first string to the second string just by adding letters at any position in the first string.

For example, "put" and "cot" are both subsequences of "computer", but "car" and "rot" are not.

```
def is_subseq(w1, w2):
    """ Returns True if w1 is a subsequence of w2 and False otherwise.

    >>> is_subseq("word", "word")
    True
    >>> is_subseq("compute", "computer")
    True
    >>> is_subseq("put", "computer")
    True
    >>> is_subseq("computer", "put")
    False
    >>> is_subseq("sin", "science")
    True
    >>> is_subseq("nice", "science")
    False
    is_subseq("boot", "bottle")
    False
    """

    if ____________________________________________________________________________:

        return ____________________________________________________________________

    elif ____________________________________________________________________________

        return ____________________________________________________________________

    else:

        with_elem = _____________________________________ and _____________________________________

        without_elem = ____________________________________________________________

        return ____________________________________________________________________
```

(b) (4 pt) Now, implemment `scrabbler` which takes in a string `chars`, a list of strings `words`, and a dictionary `values` which maps letters to point values. It should return a dictionary where each key is a word in `words` which can be formed from the letters in `chars` and each value is the point value of that word.

For the purposes of this problem, we will only consider words we can form by using letters in `chars` in the same order they appear in the string. Assume `values` contains all the letters across valid words as keys. You may additionally assume `is_subseq` from the previous part has been implemented correctly.

```
def scrabbler(chars, words, values):
    """ Given a list of words and point values for letters, returns a
    dictionary mapping each word that can be formed from letters in chars
    to their point value. You may not need all lines

    >>> words = ["easy", "as", "pie"]
    >>> values = {"e": 2, "a": 2, "s": 1, "p": 3, "i": 2, "y": 4}
    >>> scrabbler("heuaiosby", words, values)
    {'easy': 9, 'as': 3}
    >>> scrabbler("piayse", words, values)
    {'pie': 7, 'as': 3}
    """

    result = ____________________________________________________________

    for ____________________________________________________________:

        if ____________________________________________________________:

            ____________________________________________________________

            ____________________________________________________________

            ____________________________________________________________

            ____________________________________________________________

    return result
```

8. (8 points) Leaf It To Me

Write a function `max_path` which takes in a tree `t` with positive labels and a number `k`, and returns the path with length at most `k` for which the sum of the labels in the path is greatest. If there are multiple paths with the greatest sum, return the leftmost one.

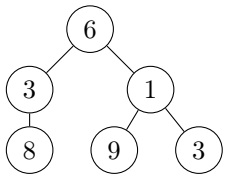
The path does *not* have to start at the root of the tree - the path can contain any top-to-bottom sequence of nodes.

```
def tree(label, branches=[]):
    return [label] + list(branches)

def label(t):
    return t[0]

def is_leaf(t):
    return not branches(t)

def branches(t):
    return t[1:]
```

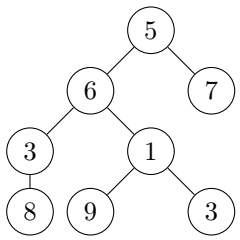
The tree data abstraction is provided here for your reference. Do not violate the abstraction barrier!

```
def max_path(t, k):
    """ Return a list of the labels on any path in tree t of length at most k with the greatest sum

    >>> t1 = tree(6, [tree(3, [tree(8)]), tree(1, [tree(9), tree(3)])])
    >>> max_path(t1, 3)
    [6, 3, 8]
    >>> max_path(t1, 2)
    [3, 8]
    >>> t2 = tree(5, [t1, tree(7)])
    >>> max_path(t2, 1)
    [9]
    >>> max_path(t2, 2)
    [5, 7]
    >>> max_path(t2, 3)
    [6, 3, 8]
    """
    def helper(t, k, on_path):

        if ____________________________________________________________________:

            return []

        elif ____________________________________________________________________:

            return [label(t)]

        a =  ____________________________________________________

        if ____________________________________________________________________:

            return ____________(____________________, key = ________________________)

        else:

            b = ____________________________________________________

            return _____________(____________________, key = ________________________)

    return helper(t, k, False)
```

t1



t2



9. (0 points) The Prisoner's Dilemma

In this extra credit problem, you may choose one of two options.

- Mark the choice to "Betray" and write a positive integer in the blank below. The one student who writes the *smallest, unique positive integer* will receive *two* (2) extra credit points but only if fewer than 90% of students choose the next option.
- Mark the choice to "Work Together". If at least 90% of students choose this option, all students who chose this option will receive *one* (1) extra credit point and those who marked the choice to "Betray" will receive zero (0) extra credit points.

Will you *work together*? Or will you *betray* your fellow students? It is up to you.

◯ Betray _____

◯ Work Together

10. (0 points) The End

(a) (0 pt) Any feedback for us on how this exam went / how the course is going so far?

(b) (0 pt) Use this space to draw a picture, write a note, or otherwise express yourself.

CS 61A Structure and Interpretation of Computer Programs
Summer 2021

DIAGNOSTIC

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number? A regex restricts inputs to numerical responses only.

1. (1.0 points) Applications are Closed

Write a higher order function `apply_until_n` that takes in `f`, a one-argument function, and a positive integer `n` > 0. `apply_until_n` should return a function that takes in a number `x` and applies `f` onto `x` as many times as possible before the result exceeds `n`.

This function should return the latest result that doesn't exceed `n`. You may assume that the return value of `f` is always strictly greater than the input.

```
def apply_until_n(f, n):
    """
    >>> square = lambda x: x * x
    >>> add_one = lambda x: x + 1
    >>> apply_until_n(add_one, 5)(3)
    5
    >>> apply_until_n(square, 10)(2)
    4
    >>> apply_until_n(square, 20)(2)
    16
    """
    def applier(x):
        while ________:
        #        (a)
            ____________
            #    (b)
        return x
    return ________
    #         (c)
```

(a) Fill in blank (a)?

(b) Fill in blank (b)?

(c) Fill in blank (c)?

2. (1.0 points) Catch that Bug!

Your classmates are trying to complete a coding assignment but are struggling. The assignment is to write a function `digit_counter`, which takes in `f`, a one-argument function, and `n`, a non-negative integer, and returns the number of digits in `n` for which `f(digit)` returns `True`.

Here's a doctest showing the intended behavior:

```
>>> is_even = lambda x: (x % 2) == 0
>>> digit_counter(is_even, 1112)
1
>>> digit_counter(is_even, 9843)
2
>>> greater_than_three = lambda x: x > 3
>>> digit_counter(greater_than_three, 123456789)
6
```

In all below parts of this problem, you don't need to indent your answer when writing code. Each solution can be made correct by changing exactly one line.

(a) Albert has decided to use a while loop to complete the assignment, but it's not working!

```
1 def digit_counter(f, n):
2     counter = 0
3     while n >= 0:
4         if f(n % 10):
5             counter += 1
6         n = n // 10
7     return counter
```

Which line number is Albert's error on?

(b) What should that line be replaced with?

(c) Alex has decided to use recursion instead, but it's also not working!

```
1 def digit_counter(f, n):
2     if n < 10 and f(n):
3         return n
4     if f(n % 10):
5         return 1 + digit_counter(f, n // 10)
6     return digit_counter(f, n // 10)
```

Which line number is Alex's error on?

(d) What should that line be replaced with?

(e) Catherine has taken an unconventional approach and has decided to use a helper function – and it's still wrong!

```
1 def digit_counter(f, n):
2     def helper(x, sofar):
3         if x > n:
4             return sofar
5         last = (n // x) % 10
6         return helper(x * 10, sofar + f(last))
7     return helper(0, 0)
```

Which line number is Catherine's error on?

(f) What should that line be replaced with?

3. (1.0 points) Oh, Camel!

Definition: A *camel sequence* is an integer in which each digit is either strictly less than or strictly greater than both of its adjacent digits. Write a function `is_camel_sequence` that takes in a nonnegative integer `n` and returns whether `n` is a *camel sequence*.

Note: Any single digit integer is a valid camel sequence.

Restrictions: You may not use `int`, `str`, `[` or `]` in your solution.

```
def is_camel_sequence(n):
    """
    >>> is_camel_sequence(15263) # 1 < 5, 5 > 2, 2 < 6, 6 > 3
    True
    >>> is_camel_sequence(98989)
    True
    >>> is_camel_sequence(123) # 1 < 2, but 2 is not greater than 3.
    False
    >>> is_camel_sequence(4114) # 1 is not strictly less than 1
    False
    >>> is_camel_sequence(1)
    True
    >>> is_camel_sequence(12)
    True
    >>> is_camel_sequence(11)
    False
    """
    def helper(n, incr):
        if _____________:
        #      (a)
            return True
        elif incr:
            return _______________ and helper(_____________)
            #            (b)                       (c)
        else:
            return _______________ and helper(_____________)
            #            (d)                       (e)
    return ____________ or _____________
    #          (f)             (g)
```

(a) Fill in blank (a)?

(b) Which of the following could fill in blank (b)?

- ◯ `n % 10 < n // 10 % 10`
- ◯ `n % 10 <= n // 10 % 10`
- ◯ `n % 10 < n // 10`
- ◯ `n % 10 <= n // 10`
- ◯ `n % 10 < n % 100`
- ◯ `n % 10 <= n % 100`
- ◯ `n % 10 < n % 10 // 10`
- ◯ `n % 10 <= n % 10 // 10`

(c) Fill in blank (c)?

(d) Which of the following could fill in blank (d)?

- ◯ `n % 10 > n // 10 % 10`
- ◯ `n % 10 >= n // 10 % 10`
- ◯ `n % 10 > n // 10`
- ◯ `n % 10 >= n // 10`
- ◯ `n % 10 > n % 100`
- ◯ `n % 10 >= n % 100`
- ◯ `n % 10 > n % 10 // 10`
- ◯ `n % 10 >= n % 10 // 10`

(e) Fill in blank (e)?

(f) Fill in blank (f)?

(g) Fill in blank (g)?

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Summer 2021

MIDTERM

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

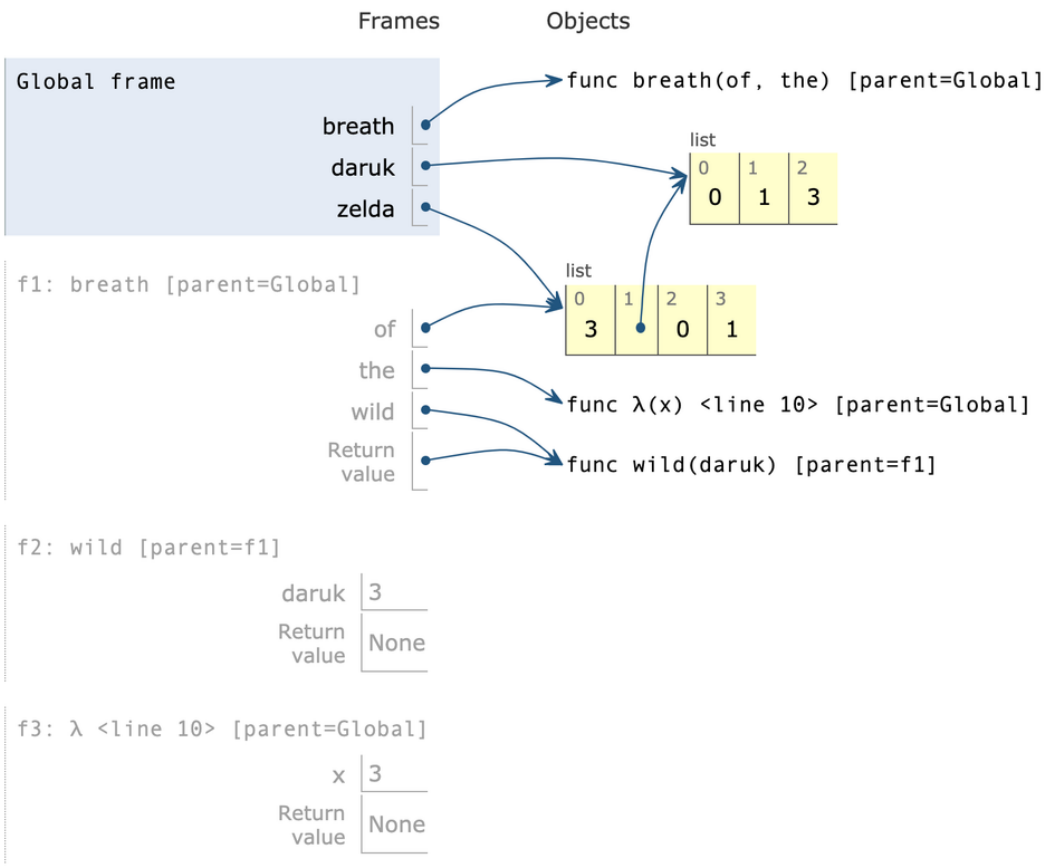
(a) What is your full name?

(b) What is your student ID number?

(c) By writing my name below, I pledge on my honor that I will abide by the rules of this exam and will neither give nor receive assistance. I understand that doing otherwise would be a disservice to my classmates, dishonor me, and could result in me failing the class.

1. Breath of the Environment

(a) The following environment diagram was generated by a program:



Click here to open the diagram in a new window

In this series of questions, you'll fill in the blanks of the program that follows so that its execution matches the environment diagram.

```
def breath(of, the):
    def wild(daruk):
        of.extend(______(a)______)
        ____(b)_____(____(c)_______)
    return wild


daruk = [0, 1]
zelda = [3, daruk]
breath(zelda, _________(d)_________)(3)
```

i. Which of these could fill in blank (a)? **Select all that apply!**

- ☐ `zelda[1]`
- ☐ `daruk`
- ☐ `daruk[1]`
- ☐ `of[1]`
- ☐ `of[1:]`
- ☐ `[1, 3]`
- ☐ `zelda`

ii. Which of these could fill in blank (b)?

- ◯ `breath`
- ◯ `(lambda x: zelda[1].extend([daruk]))`
- ◯ `daruk`
- ◯ `wild`
- ◯ `the`
- ◯ `zelda`
- ◯ `(lambda x: x)(3)`

iii. Which of these could fill in blank (c)? **Select all that apply!**

- ☐ `3`
- ☐ `(lambda x: x)(3)`
- ☐ `zelda[0]`
- ☐ `zelda.pop(0)`
- ☐ `zelda.append(3)`
- ☐ `of[0]`
- ☐ `daruk`

iv. Which of these could fill in blank (d)?

- ◯ `lambda x: None`
- ◯ `[z + 1 for z in daruk]`
- ◯ `lambda x: zelda[1].append(x)`
- ◯ `lambda x: zelda[1].extend([daruk])`
- ◯ `the(daruk)`
- ◯ `lambda x: zelda[1] + [3]`

2. LIST-en Up!

(a) Order in the Court!

Write a function `order_order` which takes a non-empty list of 2-argument functions `operators` and returns a new 2-argument function. When called, this returned function should print the result of applying the first function in operators to the two parameters passed in. It should then return another function that applies the second function in `operators` to the parameters, and so on. When the returned function has called the last function in the `operators` list, it should cycle back to the beginning of the list and use the first function again on the next call.

See the doctest for an example.

```
def order_order(operators):
    """
    >>> from operator import add, mul, sub
    >>> ops = [add, mul, sub]
    >>> order = order_order(ops)
    >>> order = order(1, 2) # applies add and returns mul
    3
    >>> order = order(1, 2) # applies mul and returns sub
    2
    >>> order = order(1, 2) # applies sub and cycles back to return add
    -1
    >>> order = order(1, 2) # cycles back to applying add
    3
    >>> order = order(1, 2)
    2
    >>> order = order(1, 2)
    -1
    """
    def apply(x, y):
        print(_________________)
        #           (a)
        return order_order(_________________)
        #                       (b)
    return ___________________
    #              (c)
```

i. What line of code could go in blank (a)?

ii. What line of code could go in blank (b)?

iii. What line of code could go in blank (c)?

(b) Skipping Around

A skip list is defined as a sublist of a list such that each element in the sublist is non adjacent in the original list. For original list `[5, 6, 8, 2]`, the lists `[5, 8]`, `[5, 2]`, `[6, 2]`, `[5]`, `[6]`, `[8]`, `[2]`, `[]` are all skip lists of the original list. The empty list is always a skip list of any list.

Given a list `int_lst` of unique integers, return a list of all unique skip lists of `int_lst` where **each skip list contains integers in strictly increasing order**. The order in which the skip lists are returned does not matter.

```
def list_skipper(int_lst):
    """
    >>> list_skipper([5,6,8,2])
    [[5, 8], [5], [6], [8], [2], []]
    >>> list_skipper([1,2,3,4,5])
    [[1, 3, 5], [1, 3], [1, 4], [1, 5], [1], [2, 4], [2, 5], [2], [3, 5], [3], [4], [5], []]
    >>> list_skipper([])
    [[]]
    """
    if len(int_lst) == 0:
        return _________________
                    #  (a)
    with_first = ____________________
                       #  (b)
    without_first = ____________________
                         #   (c)
    with_first = [ _________ for x in with_first if x == [] or _____________]
                    # (d)                                              (e)
    return with_first + without_first
```

i. What line of code could go in blank (a)?

ii. Which of these could fill in blank (b)?

- ◯ `list_skipper(int_lst)`
- ◯ `list_skipper(int_lst[1:])`
- ◯ `list_skipper(int_lst[2:])`
- ◯ `list_skipper(int_lst[0])`
- ◯ `list_skipper(int_lst[:-1])`

iii. What line of code could go in blank (c)?

iv. Which of these could fill in blank (d)?

- ◯ `x`
- ◯ `x[0]`
- ◯ `[x[0]] + int_lst`
- ◯ `[int_lst[0]] + x`
- ◯ `int_lst.append(x)`
- ◯ `x.append(int_lst)`

v. Which of these could fill in blank (e)?

- ◯ `x[0] < int_lst[0]`
- ◯ `x < int_lst[0]`
- ◯ `x[0] > int_lst[0]`
- ◯ `x > int_lst[0]`
- ◯ `x[0] == int_lst[1]`
- ◯ `len(x) < len(int_lst)`

3. Growing Up

(a) Examine the `mystery1` function below which takes in a single positive integer parameter `n`:

```
def mystery1(n):
    i = 1
    total = 0
    while i <= n:
        if len([j for j in range(2, n + 1) if i % j == 0]) > 1:
            total = total + 1
        i = i + 1
    return total
```

i. Select the option indicating the *worst-case* order of growth of the runtime of `mystery1`.

◯ Constant: $O(1)$

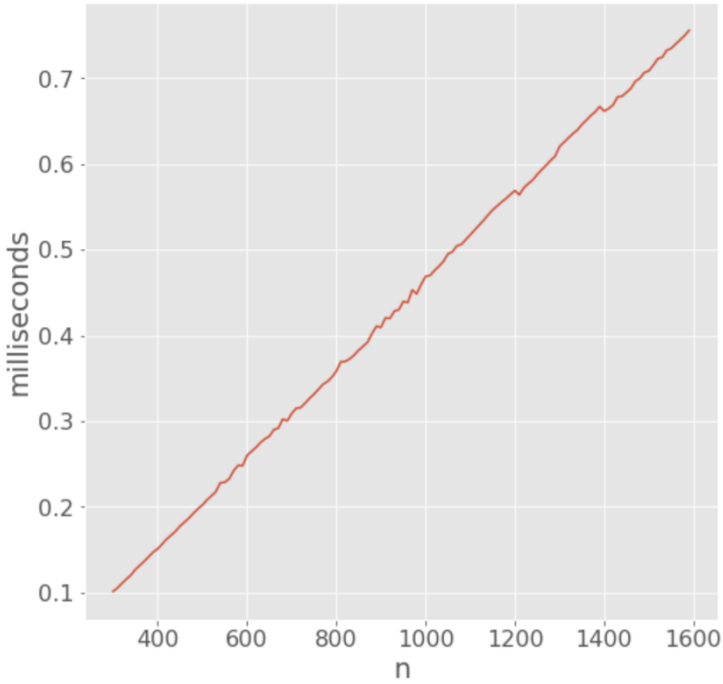
◯ Logarithmic: $O(log_2 n)$

◯ Linear: $O(n)$

◯ Quadratic: $O(n^2)$

◯ Exponential: $O(2^n)$

(b) Assume that there exists a `mystery2` function which takes in a single positive integer parameter `n`. The below graph compares `n` to the *worst-case* time it takes to evaluate `mystery2(n)`.



You may assume that `mystery2(n)`'s runtime follows a similar trend for all values of `n`.

i. Select the option best matching the *worst-case* order of growth of the runtime of `mystery2`.

- ◯ Constant: $O(1)$
- ◯ Logarithmic: $O(log_2 n)$
- ◯ Linear: $O(n)$
- ◯ Quadratic: $O(n^2)$
- ◯ Exponential: $O(2^n)$

4. Maximum Exponen-tree-ation

(a) For a list of integers `values`, define the **maximum exponentiation** of `values` to be the largest single number that can be achieved by repeatedly combining pairs of consecutive elements from `values` through exponentiation. Two possible exponentiations for the list `[3, 1, 2, 3]` are shown below:

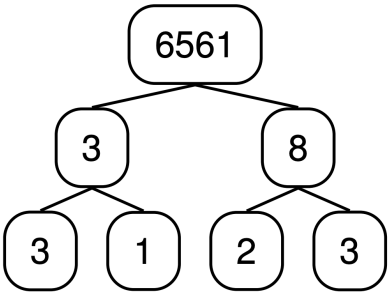
```
(3 ** 1) ** (2 ** 3) = 6561
(3 ** (1 ** 2)) ** 3 = 27
```

Of all the exponentiations of `[3, 1, 2, 3]`, the maximum exponentiation is `6561`, which can be calculated through the below steps:

```
[3, 1, 2, 3]
[3 ** 1, 2, 3]
[3, 2, 3]
[3, 2 ** 3]
[3, 8]
[3 ** 8]
6561
```

Implement `exp_tree`, which accepts a non-empty list of integers `values` and returns a tree whose label is the maximum exponentiation of `values`. Given that the maximum exponentiation is `b ** n` for some two integers `b` and `n`, the two branches of this tree should be the `exp_tree`s of the sublists of `values` used to exponentiate to `b` and `n`, respectively. You may assume that `values` has a unique exponentiation tree.

For example, below is a drawing of the `exp_tree` for `values = [3, 1, 2, 3]`:



The function should assume the standard CS61A Tree ADT definition, which is already loaded into code.cs61a.org, and is viewable here: code.cs61a.org/tree_adt

Note: Exponentials in Python are expressed as `base ** exponent`.

```
def exp_tree(values):
    """
    Returns the exponential tree that can be made from VALUES
    with the greatest possible root label

    >>> print_tree(exp_tree([5]))
    5
    >>> print_tree(exp_tree([3, 2]))
    9
      3
      2
    >>> print_tree(exp_tree([2, 3, 2]))
    512
      2
      9
        3
        2
    >>> lst = [3, 1, 2, 3]
```

```
>>> print_tree(exp_tree(lst))
6561
  3
    3
    1
  8
    2
    3
"""
if __________:
    #  (a)
   return __________
            # (b)
else:
    def tree_at_split(i):
        base = exp_tree(__________)
                        #  (c)
        exponent = exp_tree(__________)
                           #  (d)
        return tree(__________, [base, exponent])
                    #  (e)
    trees = [tree_at_split(i) for i in range(1, len(values))]
    return max(trees, key=__________)
                          #  (f)
```

i. Which of these could fill in blank (a)?

- ◯ `len(values) == 0`
- ◯ `len(values) == 1`
- ◯ `values`
- ◯ `len(values) == 2`
- ◯ `values[0] < values[1]`
- ◯ `len(values) <= 2`
- ◯ `len(values) <= 3`

ii. Which of these could fill in blank (b)?

- ◯ `0`
- ◯ `tree(0)`
- ◯ `values[0]`
- ◯ `tree(values[0])`
- ◯ `values[1]`
- ◯ `tree(values[1])`
- ◯ `values[0] ** values[1]`
- ◯ `tree(values[0] ** values[1])`
- ◯ `values[1] ** values[0]`
- ◯ `tree(values[1] ** values[0])`

iii. What line of code could go in blank (c)?

iv. What line of code could go in blank (d)?

v. What line of code could go in blank (e)?

vi. What line of code could go in blank (f)?

5. You've Seen it All

(a) Wait 'til You See What's in Store

Implement `memory_store`, a function that will return two functions: `add_val` and `times_seen`. When called on a number `var1`, `times_seen` will return the number of times `add_val` has been called on that particular value `var1`.

```
def memory_store():
    """
    >>> add_val, times_seen = memory_store()
    >>> add_val(4)
    >>> for _ in range(3):
    ...     add_val(3)
    >>> times_seen(3)
    3
    >>> times_seen(4)
    1
    >>> times_seen(2)
    0
    >>> add_val(3)
    >>> times_seen(3)
    4
    """
    memory = {}
    def add_val(var1):
        if var1 in memory:
            ----------------
            #      (a)
        else:
            ------------------
            #      (b)
    def times_seen(var1):
        return ------------------
        #             (c)
    return add_val, times_seen
```

i. Which of these could fill in blank (a)?

- ◯ `memory[var1] = 0`
- ◯ `memory[var1] = var1`
- ◯ `memory.append(var1)`
- ◯ `memory[var1] = memory[var1] + 1`
- ◯ `return memory[var1]`
- ◯ `return memory[0]`
- ◯ `return memory.get(var1, 0)`

ii. What line of code could go in blank (b)?

iii. What line of code could go in blank (c)?

(b) XKSeenD

Definition: A repeatable function is a function that returns another repeatable function, which also returns a repeatable function, and so on infinitely.

Implement the function `add_seen_d`, which takes in an integer `d` and returns a repeatable function `inner`.

When `inner` is called on an integer `innervar`, it prints the sum of all integers that have been passed to `inner` at least `d` times so far among the repeated calls.

You will need to implement the helper function `make_seen_function`, which returns a seen function. A seen function is a function that takes in a value and returns the number of times the number has been seen already.

You may not use lists, dictionaries, or the function defined in 4.1 in this part.

```
initial_seen = lambda x: 0
def add_seen_d(d, seen_fn=initial_seen, total=0):
    """
    >>> a = add_seen_d(2)
    >>> a = a(3)
    0
    >>> a = a(4)
    0
    >>> a = a(3) # 3 seen for the 2nd time, thus added
    3
    >>> a = a(3) # 3 seen for the 3rd time, and won't be added again
    3
    >>> a = a(4) # 4 seen for the 2nd time, thus added
    7
    >>> a = a(5) # 5 is seen for the 1st time, so not added
    7
    """
    def inner(innervar):
        new_seen = ____________________________
                            #  (a)
        if new_seen(innervar) == d:
            print(____________________________)
                        #  (b)
            return ____________________________
                          #  (c)
        else:
            print(____________________________)
                        #  (d)
            return add_seen_d(d, new_seen, total)
    return inner

def make_seen_function(f, msvar):
    def seen(seenvar):
        if msvar == seenvar:
            return ____________________________
                      #  (e)
        return ____________________________
                   #  (f)
    return seen
```

i. Which of these could fill in blank (a)?

- ◯ `seen_fn`
- ◯ `lambda x: seen_fn(x) + 1`
- ◯ `make_seen_function(seen_fn, d)`
- ◯ `make_seen_function(seen_fn, innervar)`
- ◯ `make_seen_function(lambda x: seen_fn(x) + 1, d)`
- ◯ `make_seen_function(lambda x: seen_fn(x) + 1, innervar)`

ii. What line of code could go in blank (b)?

(You may not use lists, dictionaries, or the function defined in 4.1 in this part)

iii. What line of code could go in blank (c)?

(You may not use lists, dictionaries, or the function defined in 4.1 in this part)

iv. What line of code could go in blank (d)?

(You may not use lists, dictionaries, or the function defined in 4.1 in this part)

v. Which of these could fill in blank (e)? **Select all that apply!**

- ☐ `f(msvar)`
- ☐ `f(seenvar)`
- ☐ `1 + f(msvar)`
- ☐ `1 + f(seenvar)`
- ☐ `f(msvar) + f(seenvar)`
- ☐ `f(seenvar) - msvar`

vi. What line of code could go in blank (f)?

(You may not use lists, dictionaries, or the function defined in 4.1 in this part)

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Summer 2022

MIDTERM

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

1. (7.0 points) What Would Python Display?

For each of the expressions below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines.

- If an error occurs, write `Error`, but include all output displayed before the error.
- If evaluation would run forever, write `Forever`.
- To display a function value, write `Function`.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`. Assume that you have first started `python3` and executed the below statements.

```
def f(x):
    return not x

def my_pow(x, n):
    print(x, n)
    if f(n):
        return 1
    elif n < 0:
        return 1 // my_pow(x, -n)
    elif n % 2:
        return x * my_pow(x, n - 1)
    return my_pow(x * x, n // 2)


def hero(spider):
    def man(home):
        def marvel(home):
            return None
        print(spider)
        print(marvel)
        return spider - home
    return man

goat = lambda m: lambda n: m - n
bleat = (lambda a, b, c, d: b or a(d)(c))(goat, 5 == 6, 7, 4)
```

(a) (2.0 pt) `my_pow(2, -1)`

(b) (1.0 pt) `hero(1, 2, 3)`

(c) (1.0 pt) `hero(1)(2)(3)`

(d) (2.0 pt) `print(1, print(4, goat(5)(4 // 2)))`

(e) (1.0 pt) `bleat`

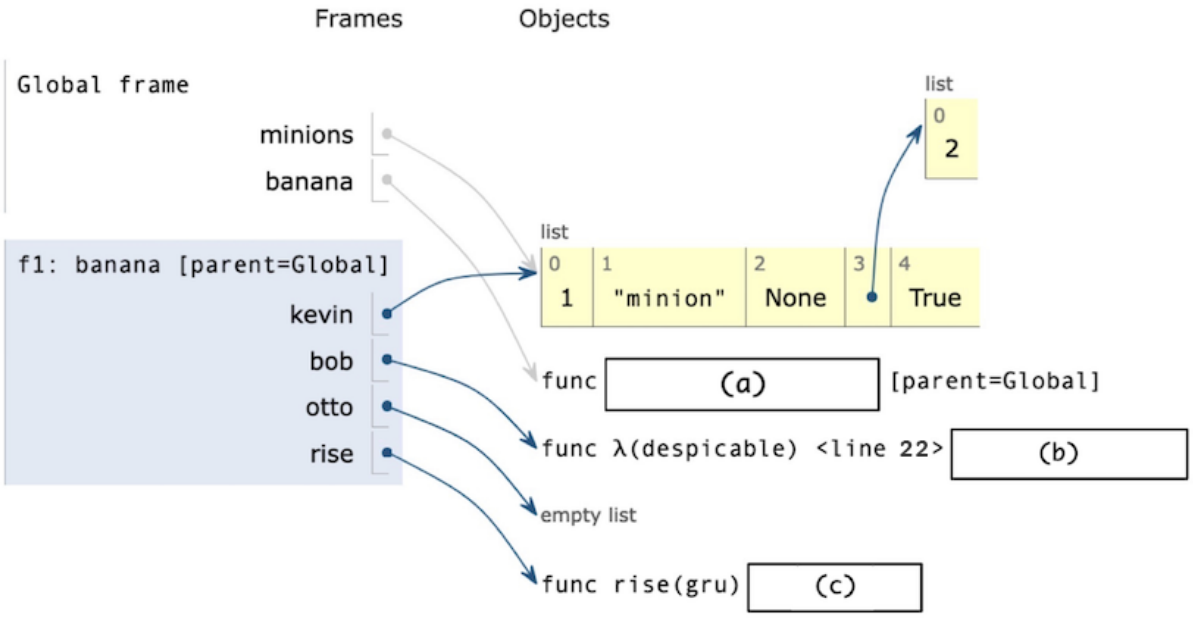
2. (6.0 points) Minions

Answer the following questions to fill in the blanks in the environment diagram, and answer what is printed when the function is run. Line numbers are included for convenience.

```
 1| minions = [1, "minion", None, [2], True]
 2|
 3| def banana(kevin, bob):
 4|     otto = []
 5|
 6|     def rise(gru):
 7|         gru.extend([gru.append([gru[1]])])
 8|         print(gru[2])
 9|         return gru[-1]
10|
11|     # STOP EXECUTION HERE FOR PART I
12|
13|     while kevin.pop():
14|         stuart = kevin.pop(0)
15|         otto.append(bob(stuart))
16|
17|     print(otto)
18|     print(minions)
19|
20|     print(rise(otto))
21|
22| banana(minions, lambda despicable: despicable * 2)
```

(a) (1.5 points) Part I

The following environment diagram shows the execution of the program until, but not including, the `while` loop beginning on line 13.



i. (0.5 pt) Fill in blank (a)

ii. (0.5 pt) Fill in blank (b)

iii. (0.5 pt) Fill in blank (c)

(b) (4.0 points) Part II

Answer the following questions assuming the remaining code has been executed.

i. (1.5 pt) What will be printed to the terminal as a result of executing `print(otto)` on line 17?

ii. (1.0 pt) What will be printed to the terminal as a result of executing `print(minions)` on line 18?

iii. (1.0 pt) What will be printed to the terminal as a result of executing `print(gru[2])` on line 8?

iv. (1.0 pt) What will be printed to the terminal as a result of executing `print(rise(otto))` on line 20?

3. (3.0 points) Bite-Size HOFs

(a) (1.5 points) Inverse Checker

Implement `inverse_checker`, a function that takes in two functions `f` and `g` and returns a function that returns True if `g` is the inverse function of `f` on input `n`. That is, `g` undoes the effect of `f` called on `n`.

```
def inverse_checker(f, g):
    """
    >>> checker0 = inverse_checker(lambda x: x + 1, lambda x: x - 1)
    >>> all([checker0(n) for n in range(100)])
    True
    >>> # `g` is the inverse `f`, but `f` is not the inverse of `g`
    >>> checker1 = inverse_checker(lambda x: x * 2, lambda x: x // 2)
    >>> all([checker1(n) for n in range(100)])
    True
    >>> checker2 = inverse_checker(lambda x: x ** 3, lambda x: x ** -3)
    >>> all([checker2(n) for n in range(1, 100)])
    False
    """
    def checker(n):

        return __________
                    (a)
    return __________
               (b)
```

i. (1.0 pt) Fill in blank (a)

ii. (0.5 pt) Fill in blank (b)

(b) (1.5 points) Force Truthy

Implement `force_truthy`, a function that takes in a function `f` and returns a function that returns the same thing as `f` when given an argument `n` such that `f(n)` outputs a truthy value, and otherwise returns `True`.

```
def force_truthy(f):
    """
    >>> truthy = force_truthy(lambda x: x // 10)
    >>> all([truthy(n) for n in range(10)])
    True
    >>> truthy(9)
    True
    >>> truthy(10)
    1
    >>> truthy(20)
    2
    """
    def truthy(n):

        return __________
                    (a)
    return __________
               (b)
```

i. (1.0 pt) Fill in blank (a)

ii. (0.5 pt) Fill in blank (b)

4. (6.0 points) Least Resistance

Fill in the definition of the function `least_resistance`, which takes in three parameters, `m`, `n`, and `f`. `m` and `n` are integers which specify a coordinates position on a grid, and `f` is a two-argument function that takes in coordinates and returns a number. Your goal is to find the path of "least resistance" from the position `(m, n)` to the position `(0, 0)` on the grid, relative to `f`, which defines the resistance of each square, and return the total resistance met along that path.

A path is a series of consecutive steps from a coordinate position on the grid to `(0, 0)`, where at each step you may either take one step down, or one step to the left. The total resistance of a path is defined as the sum of `f` called on each coordinate position visited. For example, the below graphic visualizes the paths and of least resistance, and total resistance met, for the first two doctests.

5	25	26	29	34	41	50
4	16	17	20	25	32	41
3	9	10	13	18	25	34
2	4	5	8	13	20	29
1	1	2	5	10	17	26
0	0	1	4	9	16	25
	0	1	2	3	4	5

```
f = lambda x, y: x ** 2 + y ** 2
```

```
50 + 41 + 32 + 25 + 18 + 13 +
  8 + 5 + 2 + 1 + 0 = 195
```

5	5	5	5	5	5	5
4	4	4	4	4	4	4
3	3	3	3	3	3	3
2	2	2	2	2	2	2
1	1	1	1	1	1	1
0	0	0	0	0	0	0
	0	1	2	3	4	5

```
g = lambda x, y: y
```

```
5 + 4 + 3 + 2 + 1 + 0 +
0 + 0 + 0 + 0 + 0 = 15
```

Note: In the skeleton, you are provided a line that uses `float('inf')`. This will return the Python equivalent of infinity. That is, for any number `n`, `float('inf') > n` will be `True`, no matter the value of `n`.

```
def least_resistance(m, n, f):
    """
    >>> f = lambda x, y: x ** 2 + y ** 2
    >>> least_resistance(5, 5, f)
    195
    >>> g = lambda x, y: y
    >>> least_resistance(5, 5, g)
    15
    """
    if __________:
          (a)
        return __________
                  (b)
    elif __________:
            (c)
        return float('inf')
    else:
        r1 = least_resistance(_________________________________)
                                              (d)
        r2 = least_resistance(_________________________________)
                                              (e)
        return __________(r1, r2) + __________
                  (f)                   (g)
```

(a) (1.0 pt) Fill in blank (a)

(b) (0.5 pt) Fill in blank (b)

(c) (1.0 pt) Fill in blank (c)

(d) (1.0 pt) Fill in blank (d)

(e) (1.0 pt) Fill in blank (e)

(f) (0.5 pt) Fill in blank (f)

(g) (1.0 pt) Fill in blank (g)

5. (7.0 points) Conditional Curry

Implement `cond_curry`, a function that takes in two functions, `f` and `cond`. `f` is a function that takes in two arguments, and `cond` is a predicate function that will take in a single argument and return either `True` or `False`. `cond_curry` returns a curried version of `f` that only "accepts" an argument `x` if calling `cond(x)` would return True. Otherwise, `x` is not accepted as an argument. Once the curried function has accepted two arguments, it will behave exactly as `f` would when called on those arguments.

```
def cond_curry(f, cond):
    """
    >>> from operator import add
    >>> curried = cond_curry(add, is_prime) # assume `is_prime` is implemented
    >>> curried(11)(13) # 11 + 13 = 24
    24
    >>> curried(10)(11)(12)(13) # 10 and 12 are not prime, and so are ignored
    24
    >>> curried(7)(4)(4)(4)(4)(4)(4)(4)(4)(4)(7)
    14
    """
    __________:
        (a)
        __________:
            (b)
            __________:
                (c)
                return __________
                            (d)
            return __________
                        (e)
        __________:
            (f)
            return __________
                        (g)
        return __________
                    (h)
    return __________
                (i)
```

(a) (0.5 pt) Fill in blank (a)

- ◯ `if cond(f)`
- ◯ `def g(x)`

(b) (1.0 pt) Fill in blank (b)

- ◯ `if cond(f)`
- ◯ `def h(y)`
- ◯ `if cond(x)`
- ◯ `if f(x, cond)`
- ◯ `if f(cond, x)`

(c) **(0.75 pt)** Fill in blank (c)

- ◯ `if cond(x)`
- ◯ `if cond(y)`
- ◯ `if cond(f)`
- ◯ `def i(z)`

(d) **(1.0 pt)** Fill in blank (d)

- ◯ `f(x, y)`
- ◯ `f(x, z)`
- ◯ `f(y, z)`
- ◯ `h`
- ◯ `i`

(e) **(0.75 pt)** Fill in blank (e)

- ◯ `f(x, y)`
- ◯ `g`
- ◯ `h`
- ◯ `i`

(f) **(1.0 pt)** Fill in blank (f)

- ◯ `if cond(x)`
- ◯ `if cond(f)`
- ◯ `def h(y)`

(g) **(0.75 pt)** Fill in blank (g)

- ◯ `f(x, y)`
- ◯ `cond(x)`
- ◯ `cond(y)`
- ◯ `h`
- ◯ `g`

(h) **(0.75 pt)** Fill in blank (h)

- ◯ `g`
- ◯ `h`
- ◯ `f(x, cond)`
- ◯ `cond(x)`

(i) (0.5 pt) Fill in blank (i)

◯ `cond(f)`

◯ `g`

6. (9.0 points) Blob Sum

(a) (2.0 points) Count Digits

Implement `count_digits`, a function that takes in a number `n` and returns the number of digits `n` contains. You should treat the number 0 as having no digits.

```
def count_digits(n):
    """
    >>> count_digits(0) # 0 has no digits
    0
    >>> count_digits(618)
    3
    >>> count_digits(2022)
    4
    """
    if __________:
          (a)
        return __________
                    (b)
    return __________
              (c)
```

i. (0.5 pt) Fill in blank (a)

ii. (0.5 pt) Fill in blank (b)

iii. (1.0 pt) Fill in blank (c)

(b) (7.0 points) Blob Sum

Implement `blob_sum`, a function that takes in two positive integers, `n` and `k`, and returns `True` if there exists a way to add together the digits of `n` to equal `k`, where every digit of `n` is used exactly once. However, in `blob_sum`, multiple consecutive digits can be considered as a single multi-digit number (a blob), or as multiple individual digits.

Digits are read left-to-right. For example, `123` can `blob_sum` to `15` (`= 12 + 3`) but not `33` (`= 1 + 32`).

You may assume `count_digits` is implemented correctly.

```
def blob_sum(n, k):
    """
    >>> blob_sum(123, 15) # 12 + 3 = 15
    True
    >>> blob_sum(123, 6) # 1 + 2 + 3 = 6
    True
    >>> blob_sum(123, 33) # digits of `n` must be read left-to-right
    False
    >>> blob_sum(123, 24) # 1 + 23 = 24
    True
    >>> blob_sum(123, 12) # every digit of `n` must be used
    False
    >>> blob_sum(123, 35) # every digit of `n` can only be used once
    False
    """
    def helper(n, k, blob):
        if __________:
              (a)
            return __________
                        (b)
        if __________:
              (c)
            return False
        rest, last = __________
                       (d)
        new_blob = __________
                        (e)
        return __________
                    (f)
    return helper(_______________________________)
                               (g)
```

i. (0.5 pt) Fill in blank (a)

ii. (1.0 pt) Fill in blank (b)

iii. **(0.5 pt)** Fill in blank (c)

iv. **(0.5 pt)** Fill in blank (d)

v. **(2.0 pt)** Fill in blank (e)

vi. **(2.0 pt)** Fill in blank (f)

vii. **(0.5 pt)** Fill in blank (g)

7. (6.0 points) Even Out

Fill in the definition for the function `even_out`. `even_out` takes in two parameters: `lst`, which is a list containing only the numbers `1` and `0` as elements, and `d`, which is a non-negative integer. `even_out` mutates `lst` such that exactly `d` instances of `0` occur between each instance of `1`. It also returns two values: the number of zeros it had to add to accomplish this, and the number of zeros it had to remove. You may assume that the first and last elements of `lst` will always be `1`.

```
def even_out(lst, d):
    """
    >>> lst = [1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1]
    >>> a, r = even_out(lst, 2)
    >>> lst
    [1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1]
    >>> a
    3
    >>> r
    3
    >>> a, r = even_out(lst, 0)
    >>> lst
    [1, 1, 1, 1, 1]
    >>> a
    0
    >>> r
    8
    """
    i = 0
    count = d
    added, removed = 0, 0
    while i < len(lst):
        if __________:
              (a)
            i += 1
            count = 0
        elif __________:
                (b)
            __________
               (c)
            i += 1
            count += 1
            added += 1
        elif __________:
                (d)
            __________
               (e)
            removed += 1
        else:
            i += 1
            count += 1
    return added, removed
```

(a) (1.0 pt) Fill in blank (a)

(b) (1.0 pt) Fill in blank (b)

(c) (1.5 pt) Fill in blank (c)

(d) (1.0 pt) Fill in blank (d)

(e) (1.5 pt) Fill in blank (e)

8. (20.0 points) Boba Cafe OOPerations

(a) (9.0 points) Part I: Makin' Drinks

Tyler and Chris want to open a new restaurant, but they disagree on what style of restaurant would be most successful in Berkeley. Tyler believes that a boba restaurant will attract more sales, while Chris argues that a cafe will attract more customers. As a compromise, they've decided to open a combined boba cafe that will sell a variety of different drinks and food.

Tyler and Chris have decided to represent their restaurant's sales with different Food classes in order to track their sales.

```
class Item:

    all_items = {}

    def __init__(self, name, cost, in_stock=True):
        self.name = name
        self.cost = cost
        self.in_stock = in_stock
```

Complete the implementations for the class Boba and the class Coffee. Boba and Coffee are items in the restaurant. Whenever a new item is created, that item's name should be added to the class attribute dictionary `all_items` with a value of 1 in the class `Item`. If the name is already in the dictionary, the value should increase by 1. `Boba` should have a list `all_boba` that stores all the Boba items that have been sold. `Coffee` should have a list `all_coffee` that stores all Coffee items that have been sold.

Boba should have an additional attribute called `topping` that keeps track of what topping that drink had, if any.

Coffee should have an additional attribute called `temp` that defaults to 'hot' and a method called `add_ice` that sets the temp to 'cold' when it is called.

```
class Boba(_____):
            (a)
    all_boba = ________
                (b)

    def __init__(self, name, cost, in_stock, topping):

        ________________________
                   (c)
        __________ = _________
            (d)          (e)
        ________________________
                   (f)
        if ___________ not in ____________________:
                (g)                    (h)
            ___________________[_________] = 1
                   (i)           (j)
        else:
            ___________________[_________] += 1
                   (i)           (j)


class Coffee(_____):
              (a)
    all_coffee = ________
                   (b)

    def __init__(self, name, cost, in_stock):

        ________________________
                   (c)
        _________ = 'hot'
            (k)
        ________________________
                   (l)
        if ___________ not in ____________________:
                (g)                    (h)
            ___________________[_________] = 1
                   (i)           (j)
        else:
            ___________________[_________] += 1
                   (i)           (j)

    def add_ice(__________):
                   (m)
        __________ = 'cold'
            (n)
```

i. (0.5 pt) Fill in blank (a)

ii. **(0.5 pt)** Fill in blanks labeled (b)

iii. **(1.5 pt)** Fill in blanks labeled (c)

iv. **(0.5 pt)** Fill in blank (d)

v. **(0.5 pt)** Fill in blank (e)

vi. **(0.75 pt)** Fill in blank (f)

vii. **(0.5 pt)** Fill in blanks labeled (g)

viii. **(1.0 pt)** Fill in blanks labeled (h)

ix. **(0.5 pt)** Fill in blanks labeled (i)

x. **(0.5 pt)** Fill in blanks labeled (j)

xi. **(0.5 pt)** Fill in blank (k)

xii. **(0.75 pt)** Fill in blank (l)

xiii. **(0.5 pt)** Fill in blank (m)

xiv. **(0.5 pt)** Fill in blank (n)

(b) (5.0 points) Part II: Who Wins?

Tyler is feeling a bit competitive still and wants to prove to Chris that the boba part of their restaurant is the more successful part. Write a method called `more_sold` that returns whether more Boba or more Coffee has been sold. Remember Item stores a dictionary with keys as names of drinks and values as the number of that drink sold. If sales are equal, then make sure neither Tyler nor Chris think they are selling more than the other.

```
def more_sold():
        """
        >>> bubble_tea = Boba('Bubble Tea', 4, True, 'Tapioca pearls')
        >>> black_coffee = Coffee('Black', 2, True)
        >>> latte = Coffee('Latte', 4, True)
        >>> Item.more_sold()
        'Coffee'
        """
        boba_names = [__________ for boba in ____________________]
                           (a)                        (b)
        coffee_names = [__________ for coffee in ____________________]
                             (c)                          (d)
        boba_total = [_________________ for name in _____________________________]
                             (e)                                  (f)
        coffee_total = [_________________ for name in _____________________________]
                             (e)                                    (g)
        if _____________________:
                    (h)
            return 'Boba'
        elif _____________________:
                     (i)
            return 'Coffee'
        else:
            return 'Neither'
```

i. (0.5 pt) Fill in blank (a)

ii. (0.5 pt) Fill in blank (b)

iii. (0.5 pt) Fill in blank (c)

iv. (0.5 pt) Fill in blank (d)

v. (0.5 pt) Fill in blanks labeled (e)

vi. (0.75 pt) Which of these could fill blanks labeled (f)? Check all that apply.

- ☐ `Item.all_items.keys() if name in boba_names`
- ☐ `Item.all_items.values() if name in boba_names`
- ☐ `Item.all_items.items() if name in boba_names`
- ☐ `Item.all_items if name`
- ☐ `Item.all_items if boba_names`
- ☐ `Item.all_items if name in boba_names`
- ☐ `Item.all_items[]`
- ☐ `boba_names`
- ☐ `boba_names if name in coffee_names`
- ☐ `name`

vii. (0.75 pt) Which of these could fill blanks labeled (g)? Check all that apply.

- ☐ `Item.all_items.keys() if name in coffee_names`
- ☐ `Item.all_items.values() if name in coffee_names`
- ☐ `Item.all_items.items() if name in coffee_names`
- ☐ `Item.all_items if name`
- ☐ `Item.all_items if coffee_names`
- ☐ `Item.all_items if name in coffee_names`
- ☐ `Item.all_items[]`
- ☐ `coffee_names`
- ☐ `coffee_names if name in boba_names`
- ☐ `name`

viii. (0.5 pt) Fill in blank (h)

ix. (0.5 pt) Fill in blank (i)

(c) (6.0 points) Part III: Optimize Cost

This question was removed.

9. (1.0 points) Extra Credit

Here are three questions about lecture. You must get all three correct to earn one point of extra credit.

(a) Songs from which artist were put into Richard's playlist in his Sequences lecture (lecture 8)?

(b) Which algorithm was demoed in Laryn's Recursion lecture (lecture 6) as a method of verifying credit card numbers?

(c) What did Cooper say was Richard's favorite Taylor Swift album in the Objects lecture (lecture 10)?

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Summer 2023

MIDTERM

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

> Oski the Bear

(b) What is your student ID number?

> 123456789

(c) What is your @berkeley.edu email address?

> oski@berkeley.edu

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

> Oski Bear

1. (8.0 points) What Would Python Display

For each expression below, choose the correct option or write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines.

- If an error occurs, write "Error", but include all output displayed before the error.
- If evaluation would run forever, write "Forever".
- To display a function value, write "Function".
- If the evaluated expression wouldn't display anything, write "Nothing". The interactive interpreter displays the value of a successfully evaluated expression, unless it is None.

(a) i. (0.5 pt)

```
>>> (1 or 2) and (3 or 4)
```

◯ `True`

◯ `False`

◯ `0`

◯ `1`

◯ `2`

◯ `3`

◯ `4`

ii. (0.5 pt)

```
>>> ('Barbie' or 'Ken') * 2
```

◯ `'Barbie'`

◯ `'BarbieBarbie'`

◯ `'Ken'`

◯ `'KenKen'`

◯ `'BarbieKen'`

◯ `'KenBarbie'`

◯ `'BarbieBarbieKenKen'`

◯ `2`

◯ `Error`

◯ `Nothing`

iii. (0.5 pt) For the rest of this question, assume the following code has been executed.

```
>>> it = iter([2, ['a', 'b'], 'c', 3])
>>> next(it)
```

iv. (0.5 pt)

```
>>> next(it)
```

v. (1.0 pt)

```
>>> list(it)
```

vi. (1.0 pt)

```
>>> next(it)
```

vii. (1.0 pt) For the rest of this question, assume the following code has been executed.

```
dreamhouse = 101
beach = lambda pink: pink(dreamhouse)
dreamhouse = 102

def is_even(n):
    return n % 2 == 0
```

```
>>> beach(is_even)
```

◯ True

◯ False

◯ 101

◯ 102

◯ 1

◯ 0

◯ Function

◯ Error

◯ Nothing

viii. (1.0 pt)

```
>>> beach(dreamhouse)
```

◯ True

◯ False

◯ 101

◯ 1

◯ 0

◯ Function

◯ Error

◯ Nothing

ix. (1.0 pt)

```
>>> pink = beach(print)
```

- ◯ `True`
- ◯ `False`
- ◯ `101`
- ◯ `102`
- ◯ `1`
- ◯ `0`
- ◯ `Function`
- ◯ `Error`
- ◯ `Nothing`

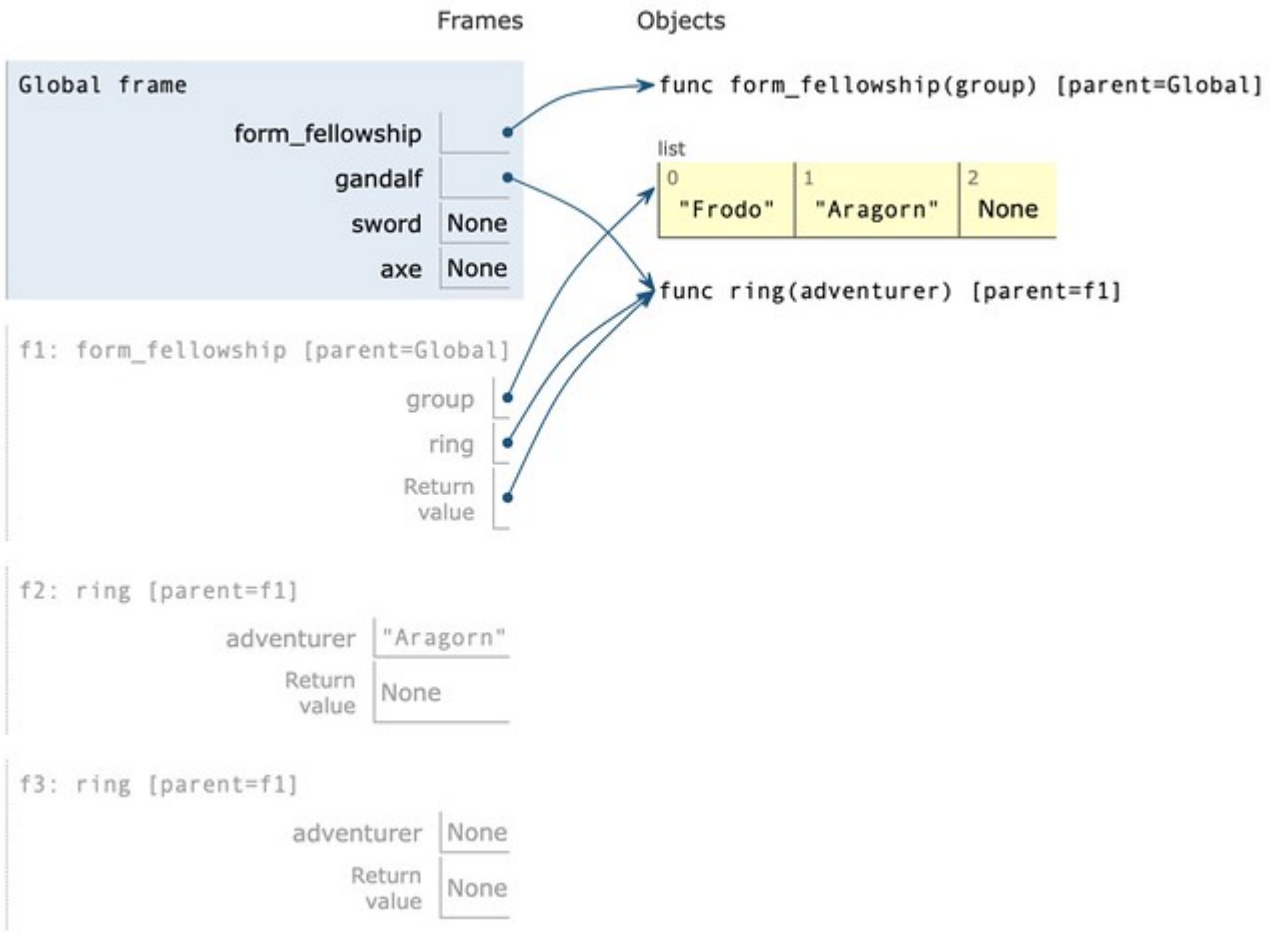
x. (1.0 pt)

```
>>> pink
```

- ◯ `True`
- ◯ `False`
- ◯ `101`
- ◯ `102`
- ◯ `1`
- ◯ `0`
- ◯ `Function`
- ◯ `Error`
- ◯ `Nothing`

2. (10.0 points) The Fellowship of the List

Given the environment diagram and skeleton code, answer the questions below.




```
def form_fellowship(__1__):
    def ring(__2__):
        return ____3____
    return ____4____

gandalf = form_fellowship(____5____)
sword = _____6_____
_____7_____ = _____8_____
```

(a) i. (1.0 pt) Fill in blank (1)

ii. (1.0 pt) Fill in blank (2)

iii. **(1.5 pt)** Which of these could go in 3? (select all that apply)

- ☐ `group + adventurer`
- ☐ `adventurer + group`
- ☐ `group.extend(adventurer)`
- ☐ `group.extend([adventurer])`
- ☐ `adventurer.extend(group)`
- ☐ `adventurer.extend([group])`
- ☐ `group.append(adventurer)`
- ☐ `group.append([adventurer])`
- ☐ `adventurer.append(group)`
- ☐ `adventurer.append([group])`
- ☐ `group + adventurer[:]`
- ☐ `group[:] + adventurer`

iv. **(1.0 pt)** Fill in blank (4)

v. **(1.5 pt)** Which of these could go in 5? (select one)

- ◯ `['Frodo']`
- ◯ `'Frodo'`
- ◯ `['Frodo', 'Aragorn']`
- ◯ `'Aragorn'`
- ◯ `['Frodo', 'Aragorn', None]`
- ◯ `None`

vi. **(2.0 pt)** Fill in blank (6). You may only make one call expression and may not use `or`, `and`, `if`, or `else`.

vii. **(1.0 pt)** Fill in blank (7)

viii. (1.0 pt) Which of these could go in 8? (select all that apply)

- ☐ `gandalf(sword)`
- ☐ `gandalf(sword.append(None))`
- ☐ `gandalf(group)`
- ☐ `gandalf(adventurer)`
- ☐ `gandalf(None)`
- ☐ `sword('Aragorn')`
- ☐ `sword(None)`

3. (13.0 points) Digit Widgets

In this question, we will be testing your understanding of both iterative and recursive solutions.

Hint: `pow(b, n)` raises `b` to the power of `n`. For example, `pow(10, 3)` is `1000`.

(a) (5.0 points)

Implement the recursive version of `digit_widget` which takes digit `d`. It returns a function with positive integer parameter `n` that removes all instances of `d` from `n`. **You may not use `str` or `repr` or `[` or `]` or `for`.**

```
def digit_widget(d):
    """
    Given a digit D, returns a function which accepts positive integer N that removes
    all instances of D from N. If there are no digits remaining in N, return 0.
    >>> remove_5s = digit_widget(5)
    >>> remove_5s(1234512345)
    12341234
    >>> remove_5s(55555)
    0
    >>> remove_0s = digit_widget(0)
    >>> remove_0s(102001)
    121
    >>> remove_0s(900)
    9
    >>> remove_0s(0)
    0
    """
    def remove_digit(n):
        if _________:
              (a)
            return _________
                       (b)
        if n % 10 == _________:
                         (c)
            return _________
                       (d)
        return _________
                  (e)
    return _________
              (f)
```

i. (0.5 pt) Fill in blank (a).

◯ `n == d`

◯ `n == 0`

◯ `n % 10 == 0`

◯ `n > 0`

◯ `n >= 0`

◯ `n < 0`

ii. (0.5 pt) Fill in blank (b)

- ◯ `0`
- ◯ `1`
- ◯ `d`
- ◯ `n - 1`
- ◯ `pow(10, d)`
- ◯ `pow(10, n)`

iii. (0.5 pt) Fill in blank (c)

- ◯ `0`
- ◯ `1`
- ◯ `d`
- ◯ `n - 1`
- ◯ `pow(10, d)`
- ◯ `pow(10, n)`

iv. (1.0 pt) Fill in blank (d)

- ◯ `remove_digit(n % d)`
- ◯ `remove_digit(n // 10)`
- ◯ `remove_digit(n % 10)`
- ◯ `remove_digit(n - 1)`
- ◯ `remove_digit(n % pow(10, d))`
- ◯ `remove_digit(n // pow(10, d))`

v. (2.0 pt) Fill in blank (e).

vi. (0.5 pt) Fill in blank (f).

(b) (5.0 points)

Implement the iterative version of `digit_widget`, `digit_widget_iter`, which takes digit `d`. It returns a function with positive integer parameter `n` that removes all instances of `d` from `n`. **You may not use `str` or `repr` or `[` or `]` or `for`.**

```
def digit_widget_iter(d):
    """
    Given a digit D, returns a function which accepts positive integer N that removes
    all instances of D from N. If there are no digits remaining in N, return 0.
    >>> remove_5s = digit_widget_iter(5)
    >>> remove_5s(1234512345)
    12341234
    >>> remove_5s(55555)
    0
    >>> remove_0s = digit_widget_iter(0)
    >>> remove_0s(102001)
    121
    >>> remove_0s(900)
    9
    >>> remove_0s(0)
    0
    """
    def remove_digit(n):
        result, i = 0, 0
        while _________:
                 (a)
            if _________:
                   (b)
                result += _________
                               (c)

                _________
                   (d)

            _________
               (e)
        return result
    return _________
              (f)
```

i. (0.5 pt) Fill in blank (a).

- ◯ `n == d`
- ◯ `n == 0`
- ◯ `n % 10 == 0`
- ◯ `n > 0`
- ◯ `n >= 0`
- ◯ `n < 0`

ii. (0.5 pt) Fill in blank (b)

- ◯ `n % 10 == 0`
- ◯ `n // 10 != 0`
- ◯ `n % 10 == d`
- ◯ `n % 10 != d`
- ◯ `n % pow(10, d) == 0`
- ◯ `n // 10 == pow(10, d)`

iii. (2.0 pt) Fill in blank (c)

iv. (0.5 pt) Fill in blank (d).

- ◯ `n = n % pow(10, i)`
- ◯ `i = pow(10, i)`
- ◯ `i += 1`
- ◯ `n = n % 10`
- ◯ `n = n // 10`
- ◯ `n -= 1`

v. (1.0 pt) Fill in blank (e).

- ◯ `n = n % pow(10, i)`
- ◯ `i = pow(10, i)`
- ◯ `i += 1`
- ◯ `n = n % 10`
- ◯ `n = n // 10`
- ◯ `n -= 1`

vi. (0.5 pt) Fill in blank (f).

(c) (3.0 points)

Implement `digit_machine`, which takes a positive integer `n`. It returns `n` where all instances of 4 and 8 have been removed. **You may not use `str` or `repr` or `[` or `]` or `for`.** Assume `digit_widget` is implemented correctly (and you may use it).

```
def digit_machine(n):
    """
    Given an integer N, return a modified N such that all instances of 4 and 8 have been removed.
    >>> digit_machine(484848)
    0
    >>> digit_machine(123456789)
    1235679
    >>> digit_machine(208)
    20
    """
    return _________
              (a)
```

i. (3.0 pt) Fill in blank (a).

4. (8.0 points) Goatda

This question will test the debugging skills you've learned in the class so far. We have included an implementation of the `goatda` function which accepts a single argument function `lamb` and a non-negative integer `n`. It returns a single argument function that applies `lamb` to the argument `n` times. However, **the included implementation is buggy**!

```
def goatda(lamb, n):
    """
    Implement goatda which accepts LAMB, a single argument function, and N the number of times
    to apply LAMB. Return a single argument function that applies LAMB to the argument N times.
    >>> add3 = goatda(lambda x: x+1, 3)
    >>> add3(10) # (((10 + 1) + 1) + 1)
    13
    >>> add3(2) # (((2 + 1) + 1) + 1)
    5
    >>> print2 = goatda(print, 2)
    >>> print2("hi") # print(print("hi"))
    hi
    None
    >>> identity = goatda(lambda x: x+3, 0)  # The function is applied 0 times
    >>> identity(0)
    0
    """
    if n == 0:                                # line 1
        return lambda x: x                    # line 2
    return lambda x: lamb(goatda(lamb, n-1)) # line 3
```

(a) After implementing `goatda`, we execute the following lines of code in the terminal.

```
>>> add3 = goatda(lambda x: x+1, 3)
>>> add3(10)
```

The call to `add3(10)` results in an error!

i. (2.0 pt) What kind of error does this result in?

- ◯ `RecursionError: too much recursion`
- ◯ `TypeError: unsupported operand type(s)`
- ◯ `TypeError: 'NoneType' object is not callable`
- ◯ `IndentationError`
- ◯ `IndexError`
- ◯ `NameError`

ii. (2.0 pt) Let's modify the code, so that it no longer errors. Select how to replace the 3 lines, so that the function passes the doctests and no longer errors. Select `No change` if you do not want to modify the line. Indentation will remain the same.

How would you modify line 1?

- ◯ `if n > 0:`
- ◯ `if n < 0:`
- ◯ `if n >= 0:`
- ◯ `if n % 10 > 0:`
- ◯ `if n % 10 >= 0:`
- ◯ `if lamb(n) == 0:`
- ◯ `if lamb(n):`
- ◯ `No change`

iii. (2.0 pt) How would you modify line 2?

- ◯ `return lamb(n)`
- ◯ `return n`
- ◯ `return lambda x: n`
- ◯ `return lambda x: 0`
- ◯ `return lambda x: lamb(x)`
- ◯ `return lambda x: lambda y: lamb(y)`
- ◯ `No change`

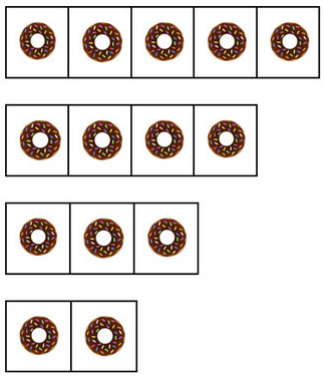
iv. (2.0 pt) How would you modify line 3?

- ◯ `return goatda(lamb, lamb(n-1))`
- ◯ `return lamb(goatda(lamb, n-1))`
- ◯ `return lambda x: lamb(x) + goatda(lamb, n-1)`
- ◯ `return lambda x: goatda(lamb(x), n-1)`
- ◯ `return lambda x: lamb(goatda(lamb, n-1)(x))`
- ◯ `return lambda x: goatda(lamb, n-1)(n)`
- ◯ `No change`

5. (13.0 points) Sweetness Overload

(a) (3.0 points)

Implement `donut_tower` which takes in the number of donuts `n` and positive integer `k`. It returns the number of layers one can fill using `n` donuts, given the smallest layer consists of `k` donuts and each subsequent layers consists of one more donut than the previous one. For instance, if `n` is 16 and `k` is 2, then you can create 4 layers (as shown below) using 14 donuts.



```
def donut_tower(n, k):
    """
    Given non-zero numbers N and K, returns the number of donut layers
    that can be formed using N donuts and K-donut smallest layer
    >>> donut_tower(0, 3)
    0
    >>> donut_tower(1, 1)
    1
    >>> donut_tower(10, 2) # 2 + 3 + 4 = 9 donuts
    3
    >>> donut_tower(20, 3)  # 3 + 4 + 5 + 6 = 18 donuts
    4
    """
    if _________:
          (a)
       return _________
                  (b)
    else:
       return 1 + _________
                     (c)
```

i. (1.0 pt) Fill in blank (a)

- ◯ `n < k`
- ◯ `n >= k`
- ◯ `n > k`
- ◯ `n == 1`
- ◯ `n == 0`
- ◯ `n < 1`

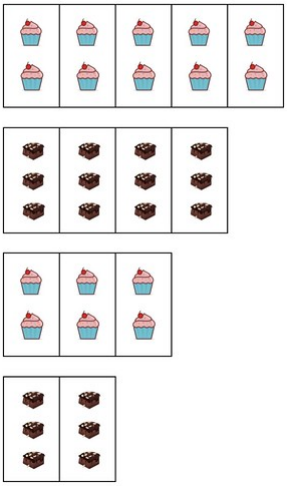
ii. (1.0 pt) Fill in blank (b)

iii. (1.0 pt) Fill in blank (c)

- ◯ `max(donut_tower(n, k + 1), donut_tower(n - k, k))`
- ◯ `donut_tower(n, k + 1)`
- ◯ `donut_tower(n - k, k + 1)`
- ◯ `donut_tower(n - k, k)`
- ◯ `donut_tower(n - k + 1, k)`
- ◯ `donut_tower(n - 1, k)`

(b) (4.0 points)

Implement `alternating_tower`, which takes a positive integer `c`, the number of cupcakes, positive integer `b`, the number of brownies, and `k`, the size of the smallest layer that can hold `k` donuts. It returns the maximum numbers of dessert layers that can be formed using brownies and cupcakes. Since those items are smaller than donuts, each k-sized donut layer can hold twice as many cupcakes and thrice as many brownies. Also, a dessert tower has alternating layers of brownies and cupcakes, but **the first layer must always consist of brownies.** For instance, if `b` is 20, `c` is 20 and `k` is 2, then you can create a maximum of 4 layers (as shown below) using 18 brownies and 16 cupcakes.



```
def alternating_tower(b, c, k):
    """
    >>> alternating_tower(5, 4, 1) # 1st layer has 3 brownies, 2nd layer has 4 cupcakes
    2
    >>> alternating_tower(10, 15, 3) # 1st layer has 9 brownies, 2nd layer has 8 cupcakes
    2
    >>> alternating_tower(16, 6, 5) # 1st layer has 15 brownies
    1
    """
    def helper(b_item, c_item, curr_layer, is_cupcakes):
        if _________:
              (a)
            return 0
        elif is_cupcakes:
            return _________
                      (b)
        else:
            return 1 + helper(_________, c_item, curr_layer + 1, not is_cupcakes)
                                    (c)
    return helper(_________, c, k + 1, True)
                     (d)
```

i. (1.0 pt) Fill in blank (a)

- ◯ `b_item < 0 or c_item < 0`
- ◯ `b_item`
- ◯ `b_item == 0 or c_item == 0`
- ◯ `b_item > 0 or c_item > 0`
- ◯ `c_item`
- ◯ `b_item < 0 and c_item < 0`
- ◯ `c_item * 2 == k or b_item * 3 == k`
- ◯ `b_item * 3 <= k`

ii. (1.0 pt) Fill in blank (b)

- ◯ `helper(b_item - 3 * curr_layer, c_item, curr_layer + 1, is_cupcakes) - 1`
- ◯ `1 + helper(b_item - 3 * curr_layer, c_item - 2 * curr_layer, curr_layer - 1, not is_cupcake`
- ◯ `1 + helper(b_item, c_item - 2 * curr_layer, curr_layer + 1, not is_cupcakes)`
- ◯ `helper(b_item - 3 * curr_layer, c_item - 2 * curr_layer, curr_layer - 1, is_cupcakes)`
- ◯ `helper(b_item, b_item - 2 * curr_layer, curr_layer + 1, not is_cupcakes)`
- ◯ `helper(b_item - 3 * curr_layer, c_item, curr_layer + 1, not is_cupcakes) + helper(b_item, c_item - 2 * curr_layer, curr_layer + 1, is_cupcakes)`

iii. (1.0 pt) Fill in blank (c)

- ◯ `b_item`
- ◯ `b_item - 3 * (curr_layer + 1)`
- ◯ `b_item - 3 * curr_layer`
- ◯ `3 * b_item`
- ◯ `b_item + c_item * 3 * curr_layer`
- ◯ `b_item // 3 * (curr_layer + 1)`
- ◯ `b_item // 3 * curr_layer`
- ◯ `b_item - 3 // curr_layer`

iv. (1.0 pt) Fill in blank (d)

(c) (6.0 points)

Implement `sweetest_tower`, which takes a positive integer `c`, the number of cupcakes, positive integer `b`, the number of brownies, and `k`, the size of the smallest layer that can hold `k` donuts. It returns the maximum number of dessert layers that can be formed using brownies and cupcakes. As previously stated, since those items are smaller than donuts, each k-sized donut layer can hold twice as many cupcakes and thrice as many brownies. **Unlike `alternating_tower`, layers can be in any order.**

```
def sweetest_tower(b, c, k):
    """
    >>> sweetest_tower(0, 2, 1) # 1st layer with 2 cupcakes
    1
    >>> sweetest_tower(5, 4, 4)
    0
    >>> sweetest_tower(17, 19, 3) # layer 1 has 6 cupcakes, 2nd has 8 cupcakes, 3rd has 15 brownie
    3
    >>> sweetest_tower(30, 8, 4)    # 1st layer has 12 brownies, 2nd has 15 brownies
    2
    """
    with_brownies, with_cupcakes = 0,0
    if _________:
          (a)
        with_brownies = _________
                            (b)
    if _________:
          (c)
        with_cupcakes = _________
                            (d)
    return _________(with_brownies, with_cupcakes)
              (e)
```

i. (1.0 pt) Fill in blank (a)

- ◯ `b >= 3 * k`
- ◯ `b > 3 * k`
- ◯ `b`
- ◯ `b == 3 * k`
- ◯ `b > c`
- ◯ `b // 3 < k`
- ◯ `b // 3 == k`
- ◯ `b >= 0`

ii. **(1.0 pt)** Fill in blank (b)

- ◯ `1 + sweetest_tower(b // 3 - k, c, k - 1)`
- ◯ `sweetest_tower(b - 3 * k, c, k + 1)`
- ◯ `1 + sweetest_tower(b - 3 * k, c, k + 1)`
- ◯ `sweetest_tower(b - 3 * k, c, k - 1)`
- ◯ `3 * sweetest_tower(b, c - 2 * k, k + 1)`
- ◯ `3 + sweetest_tower(b, c - 2 * k, k + 1)`
- ◯ `sweetest_tower(b - 3 // k, c - 2 * k, k + 1)`

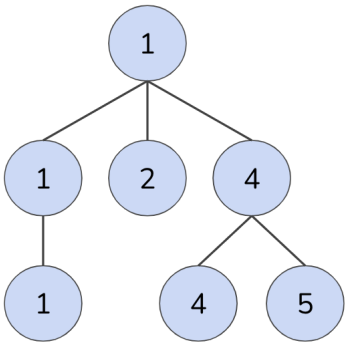
iii. **(1.0 pt)** Fill in blank (c)

iv. **(1.0 pt)** Fill in blank (d)

v. **(1.0 pt)** Fill in blank (e)

6. (12.0 points) All Treeils Lead to Rome

Definition: A **Treeil** is a tree where *exactly* one of the child node(s) of each non-leaf node has a label equal to the label of the node. Alternatively, a treeil is a tree where every node has a singular path from the node to a leaf consisting entirely of nodes with the same label. Here is an example of a valid treeil:



(a) (7.0 points) Treeil Trial

Implement `is_treeil` which takes in a tree `t` and determines whether `t` is a treeil.

```
def is_treeil(t):
    """
    Returns whether a tree is a treeil. A treeil is a tree where exactly one of
    the branches of each non-leaf node has a label equal to the label of the node.
    >>> t1 = tree(1)
    >>> is_treeil(t1)
    True
    >>> t2 = tree(1, [tree(2), tree(3)])
    >>> is_treeil(t2)
    False
    >>> t3 = tree(1, [tree(1, [tree(1), tree(3)]), tree(2)])
    >>> is_treeil(t3)
    True
    >>> t4 = tree(1, [tree(1, [tree(1), tree(1)]), tree(2)])
    >>> is_treeil(t4)
    False
    >>> t5 = tree(2, [tree(3, [tree(1)]), tree(2, [tree(2)])])
    >>> is_treeil(t5)
    False
    """
    if _________:
          (a)
        return True
    else:
        match_one = _________([b for b in branches(t) if _________]) == 1
                       (b)                                  (c)
        return _________ and _________([_________ for b in branches(t)])
                  (d)           (e)        (f)
```

i. **(1.0 pt)** Fill in blank (a). Select **all** that apply.

☐ `True`

☐ `False`

☐ `is_leaf(t)`

☐ `branches(t)`

☐ `not branches(t)`

☐ `label(t)`

☐ `not label(t)`

☐ `branches(t) == []`

ii. **(1.0 pt)** Fill in blank (b).

◯ `sum`

◯ `max`

◯ `min`

◯ `len`

◯ `any`

◯ `all`

iii. **(2.0 pt)** Fill in blank (c).

iv. **(1.0 pt)** Fill in blank (d).

v. **(1.0 pt)** Fill in blank (e).

◯ `sum`

◯ `max`

◯ `min`

◯ `len`

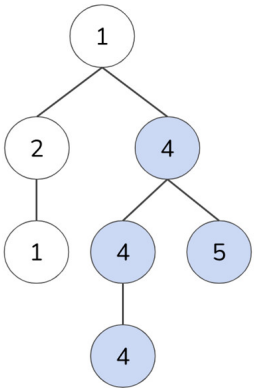
◯ `any`

◯ `all`

vi. **(1.0 pt)** Fill in blank (f).

(b) (5.0 points) Treeil Trail

Implement `max_treeil` which takes in a tree `t` and determines the highest number of nodes in a treeil that is a subtree of `t`. You may assume `is_treeil` is implemented correctly from the previous question. The following diagram displays a tree `t`, which is not a treeil. The treeil subtree of `t` with the highest number of nodes is highlighted. Calling `max_treeil` on this tree `t` should give a result of `4`.




```
def max_treeil(t):
    """
    Given a tree (that is not necessarily a treeil), returns
    the highest number of nodes in a treeil in the tree.
    >>> t1 = tree(1)
    >>> max_treeil(t1)
    1
    >>> t2 = tree(1, [tree(2), tree(3)])
    >>> max_treeil(t2)
    1
    >>> t3 = tree(1, [tree(1, [tree(1), tree(3)]), tree(2)])
    >>> max_treeil(t3)
    5
    >>> t4 = tree(1, [tree(1, [tree(1), tree(1)]), tree(2)])
    >>> max_treeil(t4)
    1
    >>> t5 = tree(2, [tree(3, [tree(1)]), tree(2, [tree(2)])])
    >>> max_treeil(t5)
    2
    """
    if _________:
          (a)
        return 1
    elif _________:
            (b)
        return 1 + _________
                      (c)
    else:
        return max([_________ for b in branches(t)])
                       (d)
```

i. **(1.0 pt)** Fill in blank (a).

- ◯ `is_treeil(t)`
- ◯ `not is_treeil(t)`
- ◯ `is_leaf(t)`
- ◯ `is_treeil(t) or is_leaf(t)`
- ◯ `not is_treeil(t) and is_leaf(t)`

ii. **(1.5 pt)** Fill in blank (b).

iii. **(1.0 pt)** Fill in blank (c).

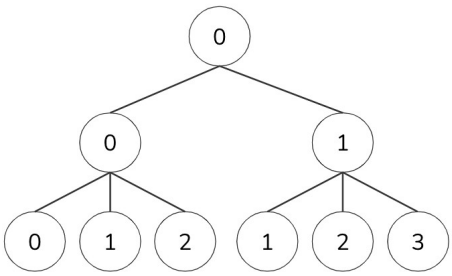
- ◯ `max([max_treeil(b) for b in branches(t)])`
- ◯ `len([max_treeil(b) for b in branches(t)])`
- ◯ `sum([max_treeil(b) for b in branches(t)])`
- ◯ `sum([b for b in branches(t) if is_treeil(b)])`
- ◯ `len([b for b in branches(t) if is_treeil(b)])`

iv. **(1.5 pt)** Fill in blank (d).

(c) (0.0 points) Treeil Treeatment (A+ Question)

This A+ question is not worth any points. This can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Implement `make_treeil` which takes in a non-negative integer `n` and returns a treeil of depth `n`. Each node of the treeil has exactly one more child than its parent node. The first branch of a node will have value equal to its parent. Subsequent branches will have a label of exactly one more than the previous branch. The root has a label of 0 and has exactly two children for a tree of depth 1. Here's an example of a treeil of depth 2.




```
def make_treeil(n):
    """
    Returns a treeil of depth n where each node has one more child than its parent.
    The first branch of a node will have label equal to its parent with subsequent
    branches having label of exactly one more than the previous branch.
    >>> t2 = make_treeil(2)
    >>> print_tree(t2)
    0
      0
        0
        1
        2
      1
        1
        2
        3
    """
    def treeil_tracker(k, parent):
        if n == k:
            return tree(parent)
        return _________
                   (a)
    return treeil_tracker(0, 0)
```

i. (0.0 pt) Fill in blank (a).

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Summer 2024

MIDTERM

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign your name to confirm that all work on this exam will be your own.

The penalty for academic misconduct on an exam is an F in the course.

1. (12.0 points) Generiterator

Assume the following code has been executed. No error occurs when executing this code block.

```
def f(x):
    yield from map(lambda x: x[::-1], x)

def next_next(i):
    print(next(i))
    return next(i)

def generiterator1(s):
    yield f(s)
    yield from f(s)
    print('warmed up!')

def generiterator2(s):
    while s:
        yield f(s)
        yield iter(s[0])
        s = s[1:]
    print('generiterating complete!')

g_str = generiterator1(['i', '<3', '61a', '!'])
g_num = generiterator2([[1], [1, 2], [1, 2, 3]])
```

`What Would Python Display`? Write the output displayed by evaluating each expression below. If an error occurs, write `"Error"`, but include all output displayed before the error. If evaluation would run forever, write `"Forever"`. To display an iterator object, write `"Iterator"`. To display a generator object, write `"Generator"`.

Assume the expressions are evaluated in order in the same interactive session, and so evaluating an earlier expression may affect the result of a later one.

Hint 1: Draw it out!

Hint 2: When a string is passed into `print`, no quotation marks are displayed. When a string is the value of an expression evaluated by the interpreter, quotation marks *are* displayed. A list of strings always displays quotation marks.

(a) (5.0 points) Jester

i. (1.0 pt) `next(g_str)`

ii. (1.0 pt) `next(g_str)`

iii. (1.0 pt) `next_next(g_str)`

iv. (2.0 pt) `next_next(g_str)`

(b) (7.0 points) Genome

i. (1.0 pt) `list(next(g_num))`

ii. (2.0 pt) `next_next(next(g_num))`

iii. (2.0 pt) `next_next(next(g_num))`

iv. (2.0 pt) `len(list(g_num))`

2. (8.0 points) Conveyor Belt

Draw the environment diagram for the code below using box and pointer notation and then answer the questions that follow. Your diagram will not be graded.

Hint: If you pass an argument into `pop`, it uses the argument as the index of the element to remove. If you do not pass an argument into `pop`, it defaults to popping the last element.

```
def conveyor_belt(s):
    k = -1
    while s[k]:
        k = s[0].pop(chain_pop(s[0]))
        s = s[k:]
        if s[0] == box:
            s.append(box or s.append(box))
        else:
            s.append(s.extend(s[:1]))
    return s

def chain_pop(s):
    return s.pop(s.pop(s.pop()))

box = [1, 1, 0, 0]

result = conveyor_belt([box, box[:]])
```

Blank Space for Diagram:

(a) (4.0 pt) What would be displayed by evaluating `print(result)` in the Global frame?

(b) (0.5 pt) What is the result of evaluating `box in result` in the Global frame?

- ◯ True
- ◯ False

(c) (0.5 pt) What is the result of evaluating `[] in result` in the Global frame?

- ◯ True
- ◯ False

(d) (0.5 pt) What is the result of evaluating `None in result` in the Global frame?

- ◯ True
- ◯ False

(e) (0.5 pt) What is the result of evaluating `result[0] == result[1]` in the Global frame?

- ◯ True
- ◯ False
- ◯ Error

(f) (0.5 pt) What is the result of evaluating `result[0] is result[1]` in the Global frame?

- ◯ True
- ◯ False
- ◯ Error

(g) (0.5 pt) What is the result of evaluating `result[0] == result[2]` in the Global frame?

- ◯ True
- ◯ False
- ◯ Error

(h) (0.5 pt) What is the result of evaluating `result[0] is result[2]` in the Global frame?

- ◯ True
- ◯ False
- ◯ Error

(i) (0.5 pt) What is the result of evaluating `not result[0]` in the Global frame?

- ◯ True
- ◯ False
- ◯ Error

3. (4.0 points) Tree Sum

Laryn, Charlotte, and Raymond are working together on a CS 61A problem, but they can't agree on a solution. Help them determine which implementation(s) are correct, if any.

`tree_sum` is a function that takes in a tree `t` and a one-argument function `cond` and returns the sum of all the labels, `x`, in `t` for which `cond(x)` returns `True`.

```
>>> t2 = tree(5, [tree(6), tree(7)])
>>> t1 = tree(3, [tree(4), t2])
>>> tree_sum(t1, lambda x: x >= 4)
22
>>> tree_sum(t1, lambda x: x >= 6)
13
>>> tree_sum(t2, lambda x: x >= 6)
13
```

```
# Attempt 1
def tree_sum1(t, cond):
    if cond(label(t)):
        total = label(t)
    else:
        total = 0
    for b in branches(t):
        if cond(label(b)):
            total += tree_sum1(b, cond)
    return total
```

```
########################################
```

```
# Attempt 2
def tree_sum2(t, cond):
    if cond(label(t)):
        total = label(t)
    else:
        total = 0
    for b in branches(t):
        total += tree_sum2(b, cond)
    return total
```

```
########################################
```

```
# Attempt 3
def tree_sum3(t, cond):
    total = label(t)
    for b in branches(t):
        if cond(label(b)):
            total += tree_sum3(b, cond)
    return total
```

(a) (4.0 pt) Which implementation(s) are correct, if any? **Select all** that apply.

- ☐ `tree_sum1` is correct
- ☐ `tree_sum2` is correct
- ☐ `tree_sum3` is correct
- ☐ `tree_sum1`, `tree_sum2`, and `tree_sum3` are all incorrect

4. (17.0 points) Add Consecutive

Implement `add_consecutive`, a function that takes in a positive integer `n` and returns a list of integers. Each element of the returned list is the sum of adjacent consecutive digits in `n`. Two digits are adjacent if they are directly beside each other. Two digits are consecutive if the absolute difference between them is exactly 1. Two of the same digit are **not** considered consecutive.

You may not use `str` or `repr` or `[` or `]` or `for`.

(a) (9.0 points) Iterative Add Consecutive

Implement `add_consecutive` iteratively.

```
def add_consecutive(n):
    """
    >>> add_consecutive(123456789)
    [45]
    >>> add_consecutive(567231) # [5 + 6 + 7, 2 + 3, 1]
    [18, 5, 1]
    >>> add_consecutive(111) # repeated digits are not consecutive
    [1, 1, 1]
    >>> add_consecutive(1235689)
    [6, 11, 17]
    >>> add_consecutive(3216598)
    [6, 11, 17]
    >>> add_consecutive(13579)
    [1, 3, 5, 7, 9]
    >>> add_consecutive(12321) # [1 + 2 + 3 + 2 + 1]
    [9]
    >>> add_consecutive(4)
    [4]
    >>> add_consecutive(105)
    [1, 5]
    >>> add_consecutive(135797531)
    [1, 3, 5, 7, 9, 7, 5, 3, 1]
    """
    result = []

    subtotal = 0

    while __________:
              (a)

        rest, last = n // 10, n % 10

        subtotal = __________
                       (b)
        if __________:
               (c)
            result = __________
                          (d)
            subtotal = __________
                             (e)
        n = rest

    result = __________
                 (f)
    return result
```

i. (2.0 pt) Select all of the expressions below that could fill in blank (a).

- ☐ `n`
- ☐ `n > 0`
- ☐ `n >= 0`
- ☐ `n != 0`
- ☐ `n > 10`
- ☐ `n >= 10`
- ☐ `n > 9`
- ☐ `n >= 9`
- ☐ `n // 10`
- ☐ `n % 10`

ii. (1.0 pt) Fill in blank (b).

iii. (2.0 pt) Fill in blank (c).

iv. (1.0 pt) Fill in blank (d).

v. (1.0 pt) Select all of the expressions below that could fill in blank (e).

- ☐ `0`
- ☐ `rest`
- ☐ `last`
- ☐ `n`
- ☐ `subtotal`
- ☐ `subtotal + rest`
- ☐ `subtotal + last`
- ☐ `subtotal + n`

vi. (2.0 pt) Fill in blank (f).

(b) (8.0 points) Recursive Add Consecutive

Implement `add_consecutive` recursively.

```
def add_consecutive(n):
    """
    >>> add_consecutive(123456789)
    [45]
    >>> add_consecutive(567231) # [5 + 6 + 7, 2 + 3, 1]
    [18, 5, 1]
    >>> add_consecutive(111) # repeated digits are not consecutive
    [1, 1, 1]
    >>> add_consecutive(1235689)
    [6, 11, 17]
    >>> add_consecutive(3216598)
    [6, 11, 17]
    >>> add_consecutive(13579)
    [1, 3, 5, 7, 9]
    >>> add_consecutive(12321) # [1 + 2 + 3 + 2 + 1]
    [9]
    >>> add_consecutive(4)
    [4]
    >>> add_consecutive(105)
    [1, 5]
    >>> add_consecutive(135797531)
    [1, 3, 5, 7, 9, 7, 5, 3, 1]
    """
    def helper(n, subtotal):
        rest, last = n // 10, n % 10

        subtotal = __________
                       (a)
        if __________:
               (b)
            return __________
                       (c)
        if __________:
               (d)
            return helper(__________)
                              (e)
        else:
            return __________
                       (f)
    return helper(n, __________)
                          (g)
```

i. (1.0 pt) Select all of the expressions below that could fill in blank (a).

- ☐ `0`
- ☐ `rest`
- ☐ `last`
- ☐ `n`
- ☐ `subtotal`
- ☐ `subtotal + rest`
- ☐ `subtotal + last`
- ☐ `subtotal + n`

ii. (1.0 pt) Fill in blank (b).

iii. (1.0 pt) Fill in blank (c).

iv. (1.0 pt) Fill in blank (d).

v. (1.0 pt) Fill in blank (e).

vi. (2.0 pt) Fill in blank (f).

vii. (1.0 pt) Fill in blank (g).

5. (7.0 points) Combine Tree

Implement `combine_tree`, a function that takes in a tree `t` and a two-argument function `f` and returns a new tree where the label of any node `b` is the result of calling `f` on the labels of all the nodes in the subtree rooted at `b` (including the label of `b` itself).

You may assume that `f` is an associative function. That is, `f(x, y) == f(y, x)` for all `x` and `y`.

```
def combine_tree(t, f):
    """
    >>> from operator import add, mul
    >>> t = tree(1, [tree(2), tree(3, [tree(4), tree(5), tree(6)])])
    >>> sum_tree = combine_tree(t, add)
    >>> print_tree(sum_tree)
    21
        2
        18
            4
            5
            6
    >>> product_tree = combine_tree(t, mul)
    >>> print_tree(product_tree)
    720
        2
        360
            4
            5
            6
    >>> max_tree = combine_tree(t, max)
    >>> print_tree(max_tree)
    6
        2
        6
            4
            5
            6
    """
    if is_leaf(t):

        return __________
                   (a)
    total = __________
              (b)
    new_branches = []

    for b in branches(t):

        new_b = __________
                    (c)
        new_branches.append(new_b)

        total = __________
                    (d)
    return tree(total, new_branches)
```

(a) (1.0 pt) Fill in blank (a).

- ◯ `t`
- ◯ `label(t)`
- ◯ `0`
- ◯ `1`

(b) (1.0 pt) Select all of the expressions below that could fill in blank (b).

- ☐ `t`
- ☐ `label(t)`
- ☐ `0`
- ☐ `1`
- ☐ `f(label(t))`
- ☐ `f(label(t), label(t))`
- ☐ `f(0, label(t))`
- ☐ `f(1, label(t))`
- ☐ `combine_tree(t, f)`
- ☐ `f(sum([combine_tree(b, f) for b in branches(t)]), label(t))`
- ☐ `sum(map(f, [combine_tree(b, f) for b in branches(t)]))`

(c) (2.0 pt) Fill in blank (c).

- ◯ `t`
- ◯ `b`
- ◯ `f(t)`
- ◯ `f(b)`
- ◯ `f(label(t))`
- ◯ `f(label(b))`
- ◯ `f(label(t), label(b))`
- ◯ `f(label(b), label(t))`
- ◯ `combine_tree(t, f)`
- ◯ `combine_tree(b, f)`

(d) (3.0 pt) Fill in blank (d).

6. (11.0 points) Multi Compose

Implement `multi_compose`, a function that takes in a list of functions and two integers `x` and `y`. It returns a composite function that on input `x` outputs `y` by applying a subsequence of functions in the input list. Functions can only be applied in the order in which they appear in the list. That is, the ith-indexed function must be applied before the i+1th-indexed function. Each function can be used 0 or 1 times.

If no such composite function exists, return `None`. If more than one such composite function exists, return any one of them.

You may assume `x != y`. Your solution must use `safe_compose`, which composes two functions together when called with two functions as arguments and returns `None` when called with `None` as an argument.

```
def safe_compose(f, g):
    """
    >>> composed = safe_compose(lambda x: x + 1, lambda x: x * 2)
    >>> composed(5)
    11
    >>> safe_compose(lambda x: x, None) is None and safe_compose(None, lambda x: x) is None
    True
    """
    if f is None or g is None:
        return None
    def composed(x):
        return f(g(x))
    return composed


def multi_compose(funcs, x, y):
    """
    >>> add_one, double = lambda x: x + 1, lambda x: x * 2
    >>> sub_three, square = lambda x: x - 3, lambda x: x ** 2
    >>> list_of_funcs = [add_one, double, sub_three, square]
    >>> double_then_square = multi_compose(list_of_funcs, 3, 36)
    >>> double_then_square
    Function
    >>> double_then_square(1) # (1 * 2) ** 2 --> 4
    4
    >>> square_then_double = multi_compose(list_of_funcs, 3, 18)
    >>> square_then_double # None
    >>> all_funcs = multi_compose(list_of_funcs, 3, 25)
    >>> all_funcs(1) # (((1 + 1) * 2) - 3) ** 2 --> 1
    1
    >>> double = multi_compose(list_of_funcs, 50, 100)
    >>> double(1) # 1 * 2 --> 2
    2
    >>> sub_two = multi_compose(list_of_funcs, 50, 48)
    >>> sub_two(2) # 2 - 2 --> 0
    0
    >>> double_then_sub = multi_compose(list_of_funcs, 50, 97)
    >>> double_then_sub(1) # (1 * 2) - 3 --> -1
    -1
    >>> negate = multi_compose(list_of_funcs, 100, -100)
    >>> negate # None
    """
```

```
if x == y:
    return lambda x: __________
                         (a)
if __________:
       (b)
    return None

return __________ or multi_compose(__________)
           (c)                         (d)
```

(a) **(2.0 pt)** Fill in blank (a).

(b) **(1.0 pt)** Fill in blank (b).

(c) **(6.0 pt)** Fill in blank (c).

(d) **(2.0 pt)** Fill in blank (d).

7. (5.0 points) Memoized Fibonacci Tree

Recall the Fibonacci sequence from lecture. It is defined as follows:

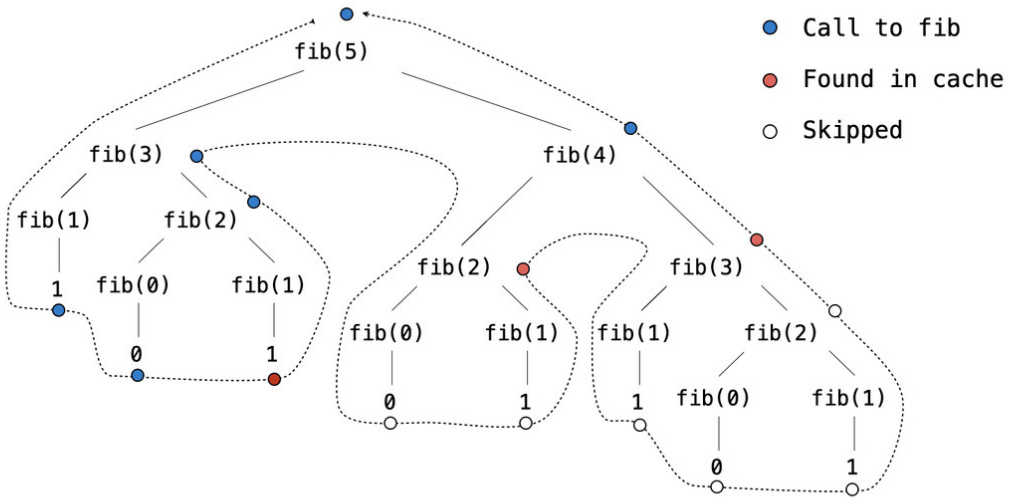
```
         0 if n == 0
fib(n) = 1 if n == 1
         fib(n - 1) + fib(n - 2) else
```

A **Fibonacci Tree** is a tree where each label is a Fibonacci number and each non-leaf node has exactly two children: the two Fibonacci numbers that appear directly before it in the Fibonacci sequence.

The `fib_tree` function below takes in a nonnegative integer `n` and returns a Fibonacci Tree that has root label `fib(n)`. Implement `fib_tree_memo`, a memoized version of `fib_tree`.

```
def fib_tree(n):
    """
    >>> print_tree(fib_tree(5))
    5
        2
            1
            1
                0
                1
        3
            1
                0
                1
            2
                1
                1
                    0
                    1
    """
    # IMPLEMENTATION OMITTED
```

Here is a visual indicating how the call diagram of `fib_tree` should change once you memoize it in `fib_tree_memo`.



```
def fib_tree_memo(n):
    """
    >>> print_tree(fib_tree_memo(5))
    5
        2
            1
            1
                0
                1
        3
            1
            2
    """
    def helper(n, cache):
        if __________:
               (a)
            return tree(__________)
                              (b)
        if n <= 1:

            return tree(n)

        b0, b1 = helper(n - 2, cache), helper(n - 1, cache)

        fib_num = __________
                      (d)
        __________ = fib_num
            (e)
        return tree(fib_num, [b0, b1])

    return helper(n, {})
```

(a) **(1.0 pt)** Fill in blank (a).

(b) **(1.0 pt)** Fill in blank (b).

(c) **(1.0 pt)** Fill in blank (d).

(d) **(1.0 pt)** Fill in blank (e).

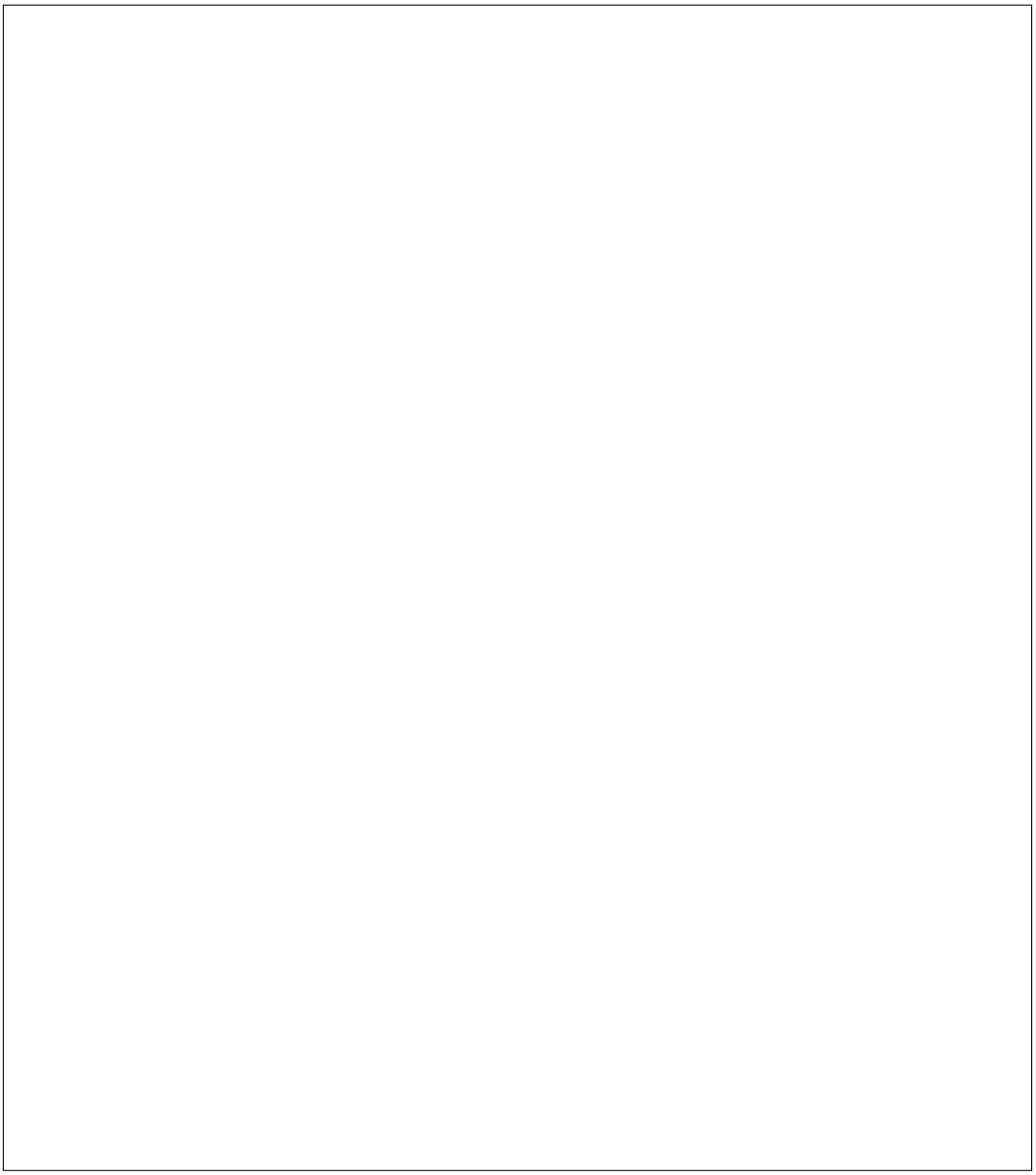
(e) (1.0 pt) What is the order of growth of the run time of `fib_tree_memo` with respect to `n`?

- ◯ Constant, $\Theta(1)$, $O(1)$
- ◯ Logarithmic, $\Theta(\log n)$, $O(\log n)$
- ◯ Linear, $\Theta(n)$, $O(n)$
- ◯ Quadratic, $\Theta(n^2)$, $O(n^2)$
- ◯ Exponential, $\Theta(b^n)$, $O(b^n)$

8. (0.0 points) Just for Fun

This is not for points and will not be graded.

(a) **Optional**: Draw your favorite spot on campus!



No more questions.

CS 61A Structure and Interpretation of Computer Programs

Summer 2025 Midterm

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (5.0 points) What Would Python Display?

```
1: a = [[13], 8]
2: b = a + [[2]]
3: b[1] -= 2
4: a[1] -= b[1]
5:
6: print(a)
7:
8: a[0] += [7]
9: b[b[2][0]] += [a.pop(1)]
10:
11: print(b)
12:
13: c = b[-1]
14: c.append(a[0])
15: c.append(b[0].extend([c[0]]))
16:
17: print(c)
```

Assume the expressions are evaluated in order in the same interactive session, and so evaluating an earlier expression may affect the result of a later one.

Hint: Draw it out!

(a) (1.0 pt) What is displayed by the call to `print` in line 6?

(b) (2.0 pt) What is displayed by the call to `print` in line 11?

- ◯ `[[13], 2, [2]]`
- ◯ `[[13, 7], 6, [2, 2]]`
- ◯ `[[13], 6, [2]]`
- ◯ `[[13, 7], 2, [2]]`
- ◯ `[[13, 7], 6, [4]]`
- ◯ `[[13, 7], 6, [2, [2]]]`
- ◯ `[[13], 6, [2, 2]]`

(c) (2.0 pt) What is displayed by the call to `print` in line 17?

- ◯ `[2, 2, [13, 7], None]`
- ◯ `[2, 2, [13], [2, 2]]`
- ◯ `[2, 2, [13, 2], [13]]`
- ◯ `[2, 2, [13], 2]`
- ◯ `[2, 2, [13, 7, 2], None]`
- ◯ `[2, 2, [13, 7, 2], [None]]`

2. (10.0 points) Surf's Up!

Answer the questions about the code below. Use the free space or scratch paper to draw the diagram to help you answer the questions, however any drawn diagrams will not be graded.

```
1: def wave(twin):
2:     n = 3
3:     def twin():
4:         print(wave, n)
5:         return print('Shred') or n
6:         print('Wipeout')
7:     return twin()
8:
9: def twin():
10:     def twin(wave):
11:         while wave(twin):
12:             return print('Riptide')
13:         print('Tide')
14:     return twin
15:
16: twin = twin()(lambda twin: print('Twin Wave') or wave(twin))
```

If an error occurs, write `"Error"`. If a function is printed, write `"Function"`.

(a) (4.0 pt) What is printed after evaluating the above code?

(b) (1.0 pt) How many frames are opened (not including the Global Frame)?

- ◯ `3`
- ◯ `4`
- ◯ `5`
- ◯ `6`

(c) (2.0 pt) What is the parent frame of `lambda twin:` in line 16?

- ◯ `Global`
- ◯ `f1`
- ◯ `f2`
- ◯ `None`

(d) (1.0 pt) How many calls are made to the function `wave` defined in line 1?

- ◯ `0`
- ◯ `1`
- ◯ `2`
- ◯ `3`
- ◯ `None`

(e) (2.0 pt) What is `twin` assigned to in the Global Frame?

3. (17.0 points) Python in python?

Definition. A **python** is a sequence of digits where the following are true:

- All digits are equal to some digit `d`
- All digits are two places apart (difference between indices equals 2)

In other words, a python is a sequence of digits where a digit `d` occurs in every other digit place, and the length of the python is the number of times `d` appears consecutively.

(a) (9.0 points)

Implement `python`, which takes in a number `n` and a digit `d`, and returns the length of the longest **python** in `n` where all digits equal `d`

Assume `n > 0`

```
def python(n, d):
    """
    Return the length of the longest python of digit d in integer n.
    >>> python(2125262, 2)      # positions of 2s: [0,2,4,6] -> all 2 apart -> length 4
    4
    >>> python(2020211202, 2)   # positions of 2s: [0,2,4,7,9] -> longest python is 3 (at 0,2,4)
    3
    >>> python(20211120202, 2)  # positions of 2s: [0,2,6,8,10] -> longest python is 3 (at 6,8,10)
    3
    >>> python(1010, 0)
    2
    >>> python(123456, 7)       # no 7 exists within n
    0
    """
    lengths = []
    _______:
      (a)
        copied_n = _______
                     (b)
        length = 0
        _______ and copied_n > 0:
          (c)
            length += 1
            _______
              (d)
        lengths._______
                  (e)
        _______
          (f)
    return max(lengths)
```

i. (1.0 pt) Fill in blank (a). Select all that apply.

☐ `while n // 10 < 0:`

☐ `while n > 0:`

☐ `while n >= 10:`

☐ `while d == n:`

ii. **(1.0 pt)** Fill in blank (b).

iii. **(3.0 pt)** Fill in blank (c).

iv. **(1.0 pt)** Fill in blank (d).

- ◯ `n = n // 100`
- ◯ `n = copied_n // 100`
- ◯ `copied_n = copied_n // 100`
- ◯ `copied_n = n // 100`

v. **(2.0 pt)** Fill in blank (e). Select all that apply.

- ☐ `insert(0,length)`
- ☐ `append([length])`
- ☐ `extend(list(length))`
- ☐ `extend([length])`

vi. **(1.0 pt)** Fill in blank (f).

(b) (8.0 points)

Implement `longest_python`, which takes in a number n and returns the digit and length for the digit with the longest **python** in n. This function explicity returns the digit, followed by its corresponding length.

If there is a tie for longest python, return the numerically greater digit.

Assume `python` from part (3a) is implemented correctly.

```
def longest_python(n):
    """
    Return the digit and length of the longest python for any digit d
    If there is a tie, return the larger digit.
    >>> longest_python(2324252)   # 2 has len 4, others have len 1
    (2, 4)
    >>> longest_python(1212121)   # 1 has len 4, 2 has len 3,
    (1, 4)
    >>> longest_python(56565656)  # 5 has len 4, 6 has len 4, 6 > 5
    (6, 4)
    >>> longest_python(67676565)  # 6 has len 4, 7 has len 2, 5 has len 2
    (6, 4)
    >>> longest_python(12345)     # all have len 1, return largest digit
    (5, 1)
    """
    d = {}
    for x in _______:
                (g)

        _______
          (h)
    key = _______
            (i)
    return d[key], _______
                     (j)
```

i. (1.0 pt) Fill in blank (g).

- ◯ `n`
- ◯ `range(n)`
- ◯ `range(len(n))`
- ◯ `range(10)`

ii. (3.0 pt) Fill in blank (h). You may not use `for`, `in`, `if`, or `lambda` in your solution.

iii. (2.0 pt) Fill in blank (i). You may not use `for`, `in`, `if`, or `lambda` in your solution.

iv. (2.0 pt) Fill in blank (j).

4. (17.0 points) Valid Number

Definition. A number is valid if it satisfies either of the following critera:

- It has 2 or fewer digits, or
- Excluding the last two digits, for every digit, there is another digit 1 or 2 places to the right with the same **parity** (even or odd)

For example, 3412 is valid as 3 and 1 have the same parity, and 4 and 2 have the same parity

(a) (7.0 points)

Implement `is_valid`, which takes in a number `n` and returns True if `n` is valid, `False` otherwise.

Assume `n > 0`

```
def is_valid(n):
    """
    >>> is_valid(3412)   # 1 is two digits to the right of 3, 2 is two digits to the right of 4.
    True
    >>> is_valid(43)     # 2 or fewer digits
    True
    >>> is_valid(3443)   # No odd number within two digits of leftmost 3.
    False
    >>> is_valid(213)    # No even number to the right of 2.
    False
    """
    def helper(last, second, n):
        if _______:
             (a)
            return True
        curr = n % 10
        if _______ and _______:
             (b)          (c)
            return False
        return helper(________)
                        (d)

    return helper(n % 10, n // 10 % 10, n // 100)
```

i. (1.0 pt) Fill in blank (a).

◯ `n == 0`

◯ `n >= 0`

◯ `n < 0`

◯ `n == 1`

ii. (1.5 pt) Fill in blank (b).

◯ `curr % 2 == 0`

◯ `last % 2 == 0`

◯ `curr % 2 != last % 2`

◯ `(curr + last) % 2 == 0`

iii. (1.5 pt) Fill in blank (c).

iv. (3.0 pt) Fill in blank (d).

(b) (10.0 points)

Implement `valid_subseq`, which takes in some number `n`, and returns the number of subsequences of n that are valid numbers.

A subsequence of a number is a combination of digits whose relative order remains the same as in the original number. In other words, a subsequence of a number is the number with some (or no) digits removed.

For example, 213 has the following subsequences: 2, 1, 3, 21, 23, 13, 213

Assume `is_valid` from part (4a) is implemented correctly.

Additionally, we have provided you with a function `reverse`, which takes a number and reverses its digits.

Hint True + True == 2

```
def valid_subseq(n):
    """
    Return the number of valid subsequences of n.
    >>> valid_subseq(12)    # possible subseqs: 1, 2, 12 - all are valid
    3
    >>> valid_subseq(123)
    7
    >>> valid_subseq(213)   # possible subseqs: 2, 1, 3, 21, 23, 13, 213 - 6 of which are valid.
    6
    >>> valid_subseq(3412)
    14
    """

    def reverse(x):
        """
        This function gives the reverse of a number
        >>> reverse(3412)
        2143
        """
        # IMPLEMENTATION NOT SHOWN

    def helper(num, rest):
        if _______ and rest == 0:
             (e)
            return _______
                      (f)
        elif _______:
               (g)
            return _______
                      (h)
        return helper(_______, _______) + helper(_______,_______)
                        (i)      (j)               (k)     (l)
    return helper(0, reverse(n))
```

i. (1.0 pt) Fill in blank (e).

- ◯ `n >= 0`
- ◯ `num > 0`
- ◯ `num == 0`
- ◯ `n < 0`

ii. (1.0 pt) Fill in blank (f).

- ◯ `0`
- ◯ `1`
- ◯ `num`
- ◯ `rest`

iii. (1.0 pt) Fill in blank (g).

iv. (2.0 pt) Fill in blank (h).

- ◯ `0`
- ◯ `1`
- ◯ `is_valid(rest)`
- ◯ `is_valid(num)`

v. (2.0 pt) Fill in blank (i).

vi. (1.0 pt) Fill in blank (j).

vii. (1.0 pt) Fill in blank (k).

- ◯ `n`
- ◯ `num`
- ◯ `rest`
- ◯ `num // 10`

viii. (1.0 pt) Fill in blank (l).

5. (15.0 points) Family Trees

(a) (7.0 points)

Implement `descendants`, which takes in a tree `t` and depth `d`, and returns a list of labels for all children at least depth `d` away from the root.

```
def descendants(t, d):
    """Return a list of all labels of descendants at least d levels away from the root node.

    >>> t1 = tree(1, [tree(2, [tree(3), tree(4)]), tree(5, [tree(6), tree(7)])])
    >>> print(t1)
    1
        2
            3
            4
        5
            6
            7
    >>> descendants(t1, 1)
    [2, 3, 4, 5, 6, 7]
    >>> descendants(t1, 2)
    [3, 4, 6, 7]
    >>> descendants(t1, 3)
    []
    """
    lst = []
    if _______:
         (a)
        _______
          (b)
    for b in branches(t):
        lst = _______
                (c)
    return _______
             (d)
```

i. (1.0 pt) Fill in blank (a).

◯ `d == 0`

◯ `d >= 0`

◯ `d < 0`

◯ `d <= 0`

ii. (2.0 pt) Fill in blank (b). Select all that apply.

☐ `lst + label(t)`

☐ `lst.append(t)`

☐ `lst += [t]`

☐ `lst.extend([label(t)])`

iii. (3.0 pt) Fill in blank (c).

iv. (1.0 pt) Fill in blank (d)

(b) (8.0 points)

Definition. A tree is an ancestor if all paths from the root node to a depth `d` have strictly decreasing labels.

Implement `is_ancestor`, which takes in a tree `t` and a depth `d` and returns `True` if the tree is an ancestor, `False` otherwise.

```
def is_ancestor(t, d):
    """
    Return True if t is an ancestor with respect to a depth d, False otherwise.
    >>> t1 = tree(2, [tree(1)])
    >>> print_tree(t1)
    2
        1

    >>> is_ancestor(t1, 0)
    True
    >>> is_ancestor(t1, 1)    # 2 -> 1: strictly decreasing
    True

    >>> t2 = tree(4, [t1, tree(3, [tree(5), tree(3)])])
    >>> print_tree(t2)
    4
        2
            1
        3
            5
            3
    >>> is_ancestor(t2, 0)
    True     # 4
    >>> is_ancestor(t2, 1)    # 4 -> 2 and 4 -> 3 are both strictly decreasing
    True
    >>> is_ancestor(t2, 2)    # 4 -> 3 -> 5 and 4 -> 3 -> 3 are not strictly decreasing
    False
    """
    if _______:
         (e)
        return True
    for b in branches(t):
        if _______ _______ _______:
             (f)     (g)     (h)
            return _______
                     (i)
    return _______
             (j)
```

i. (1.0 pt) Fill in blank (e).

◯ `d == 0`

◯ `d < 0`

◯ `d == 1`

◯ `d >= 0`

ii. (2.0 pt) Fill in blank (f). You may not use `and`, `or`, or `not`.

iii. (1.0 pt) Fill in blank (g).

- ◯ `and`
- ◯ `or`
- ◯ `not`
- ◯ `==`

iv. (2.0 pt) Fill in blank (h).

v. (1.0 pt) Fill in blank (i).

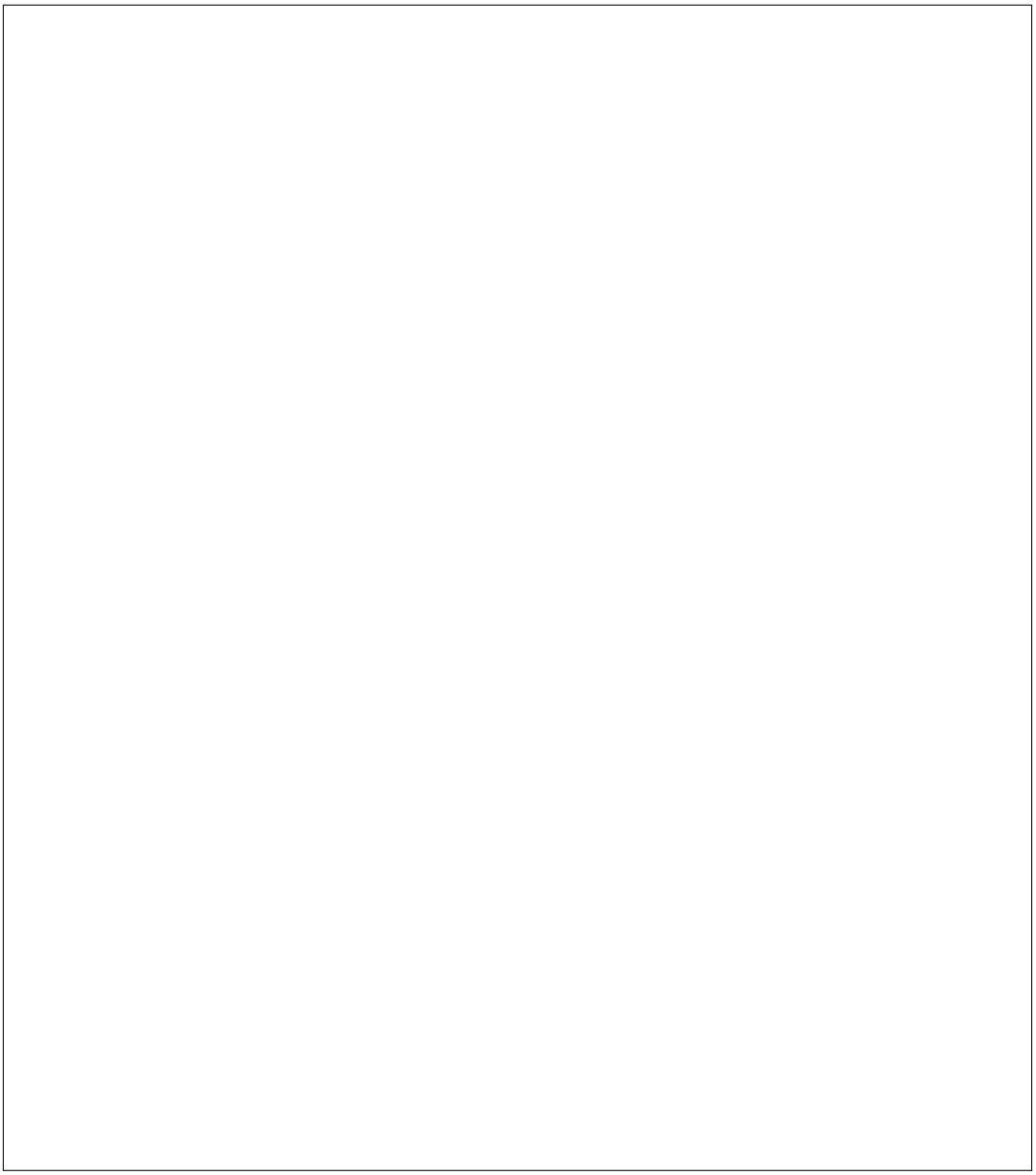
- ◯ `not b`
- ◯ `True`
- ◯ `is_ancestor(b, d-1)`
- ◯ `False`

vi. (1.0 pt) Fill in blank (j).

6. (0.0 points) Just for Fun

This is not for points and will not be graded.

(a) Optional: Draw how this exam made you feel!



No more questions.

CS 61A Fall 2016

Structure and Interpretation of Computer Programs

MIDTERM 1 SOLUTIONS

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 study guide.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

1. (10 points) Exeggcute

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If a function value is displayed, write "Function".

The first two rows have been provided as examples.

Recall: The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have started `python3` and executed the following statements:

```
equals = lambda a, b: a == b
nemo = lambda n: lambda: print(n)
ray = nemo(1)

def if_function(f, g, h):
    if h:
        f()
    elif h:
        f(f())
    else:
        print(5 or 6)
    g()

def dory():
    print('fish')
    return lambda: 1/0
```

Expression	**Interactive Output**
`pow(2, 3)`	8
`print(4, 5) + 1`	4 5 Error
`equals(3==4, equals(5, equals(5, 5)))`	True
`print(print(print(2)), print(3))`	2 None 3 None None
`print(nemo(print(5))())`	5 None None
`if_function(nemo(3), dory, 2)`	3 fish
`if_function(dory, nemo(2), ray())`	1 5 2

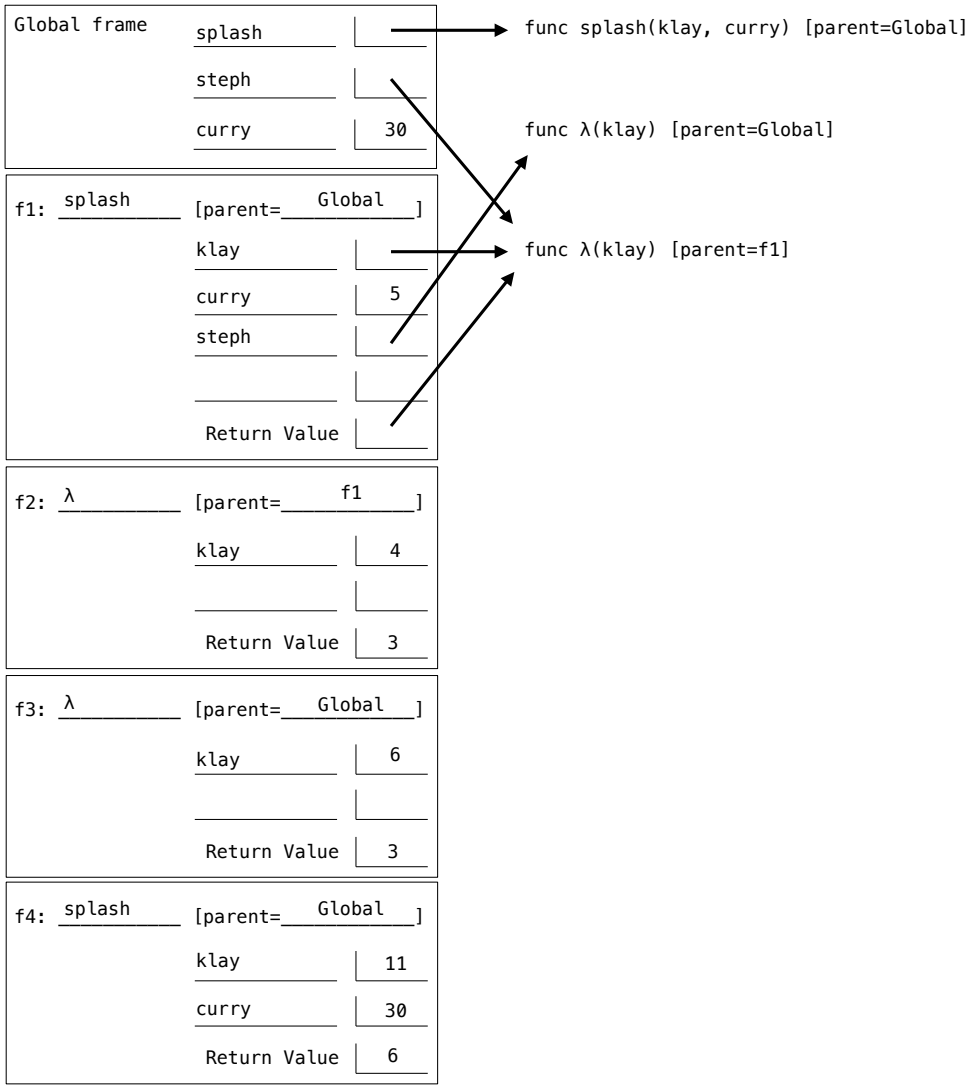
2. (8 points) Goldeen State

Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*

A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  def splash(klay, curry):
2    while curry == 3:
3      steph = klay
4      klay = lambda klay: steph(curry + 1)
5      curry = curry + 2
6      return klay
7    return curry // 5
8
9  steph = lambda klay: splash(11, curry)-3
10 steph, curry = splash(steph, 3), 30
11 steph(4)
```



3. (9 points) Countizard

(a) (6 pt) Implement `counter`, which takes a non-negative single-digit integer `d`. It returns a function `count` that takes a non-negative integer `n` and returns the number of times that `d` appears as a digit in `n`. You may not use recursive calls or any features of Python not yet covered in the course.

```
def counter(d):
    """Return a function of N that returns the number of times D appears in N.

    >>> counter(8)(8018)
    2
    >>> counter(0)(2016)
    1
    >>> counter(0)(0)
    0
    """
    def count(n):

        k = 0

        while n > 0:


            n, last = n // 10, n % 10


            if last == d:


                k += 1


        return k


    return count
```

(b) (3 pt) Implement `significant`, which takes positive integers `n` and `k`. It returns the `k` most significant digits of `n` as an integer. These are the first `k` digits of `n`, starting from the left. If `n` has fewer than `k` digits, it returns `n`. You may not use `round`, `int`, `str`, or any functions from the `math` module. You may use `pow`, which raises its first argument to the power of its second: `pow(9, 2)` is 81 and `pow(9, 0.5)` is 3.0.

```
def significant(n, k):
    """Return the K most significant digits of N.

    >>> significant(12345, 3)
    123
    >>> significant(12345, 7)
    12345
    """
    if n < pow(10, k):

        return n

    return significant(n // 10, k)
```

4. (6 points) Caterepeat

(a) (4 pt) Implement `repeat_sum`, which takes a one-argument function `f`, a value `x`, and a non-negative integer `n`. It returns the sum of $n+1$ terms. Each term, indexed by `k` starting at 0, is the result of applying `f` to `x` repeatedly `k` times. You may assign to only one name in each of the three assignment statements. You may not use recursive calls or any features of Python not yet covered in the course.

```
def repeat_sum(f, x, n):
    """Compute the following summation of N+1 terms, where the last term
    calls F N times: x + f(x) + f(f(x)) + f(f(f(x))) + ... + f(f(...f(x)))

    >>> repeat_sum(lambda x: x*x, 3, 0) # 3
    3
    >>> repeat_sum(lambda x: x*x, 3, 1) # 3 + 9
    12
    >>> repeat_sum(lambda x: x+2, 3, 4) # 3 + 5 + 7 + 9 + 11
    35
    """
    total, k = 0, 0


    while k <= n:


        total = total + x


        x = f(x)


        k = k + 1


    return total
```

(b) (2 pt) Implement `sum_squares`, which takes a non-negative integer `n` and uses `repeat_sum` to return the sum of the squares of the first `n` positive integers. Assume `repeat_sum` is implemented correctly. You may use `pow`, which raises its first argument to the power of its second: `pow(9, 2)` is 81 and `pow(9, 0.5)` is 3.0.

```
from repeat_sol import *

def sum_squares(n):
    """Return the sum of the first N perfect squares.

    >>> sum_squares(0)
    0
    >>> sum_squares(3) # 1**2 + 2**2 + 3**2
    14
    >>> sum_squares(5) # 1**2 + 2**2 + 3**2 + 4**2 + 5**2
    55
    """

    f = lambda x: pow(round(pow(x, 0.5) + 1), 2)

    return repeat_sum(f, 0, n)
```

5. (7 points) Multikarp

Terminology. An *order 1 numeric function* is a function that takes a number and returns a number. An order 2 numeric function is a function that takes a number and returns an order 1 numeric function. Likewise, an order n numeric function is a function that takes a number and returns an order $n-1$ numeric function.

The *argument sequence* of a nested call expression is the sequence of all arguments in all subexpressions, in the order they appear. For example, the expression `f(3)(4)(5)(6)(7)` has the argument sequence 3, 4, 5, 6, 7.

(a) (4 pt) Implement `multiadder`, which takes a positive integer `n` and returns an order n numeric function that sums an argument sequence of length n.

```
def multiadder(n):
    """Return a function that takes N arguments, one at a time, and adds them.

    >>> f = multiadder(3)
    >>> f(5)(6)(7)                # 5 + 6 + 7
    18
    >>> multiadder(1)(5)
    5
    >>> multiadder(2)(5)(6)       # 5 + 6
    11
    >>> multiadder(4)(5)(6)(7)(8) # 5 + 6 + 7 + 8
    26
    """

    assert n > 0



    if n == 1:



        return lambda x: x



    else:



        return lambda a: lambda b: multiadder(n-1)(a+b)
```

(b) (3 pt) Complete the expression below by writing one integer in each blank so that the whole expression evaluates to 2016. The `compose1` function appears on your midterm 1 study guide in the middle of the left column of page 2. Assume `multiadder` is implemented correctly.

```
compose1( multiadder(4)(1000), multiadder(3)(10)(1000))(1)(2)(3)
```

CS 61A Fall 2016

Structure and Interpretation of Computer Programs

MIDTERM 2 SOLUTIONS

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A study guides.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

1. (8 points) Halloween

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. The `Link` class appears on page 2 of the midterm 2 study guide.

The first two rows have been provided as examples.

Recall: The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have started `python3` and executed the following statements:

```
class Party:
    guests = Link.empty

    def __init__(self, time):
        Party.guests = Link(time+1, Party.guests)

    def attend(self):
        self.guests.rest = Link(self.guests.rest)
        return self.guests

class Costume(Party):
    def __init__(self, bow, tie):
        Party.guests.rest = Link(bow)
        self.ie = Link(self)

    def attend(self):
        print(repr(self.ie))
        Party.attend = lambda self: Party(9).guests

    def __repr__(self):
        print('Nice')
        return 'Costume'
```

Expression	**Interactive Output**
`Link(1, Link.empty)`	Link(1)
`Link(1, Link(2))`	Link(1, Link(2))
`Party(1).guests`	`Link(2)`
`Party(3).attend()`	`Link(4, Link(Link(2)))`
`Costume(5, 6).attend()`	`Nice` `Link(Costume)`
`Party(7).attend()`	`Link(10, Link(8, Link(4, Link(5))))`

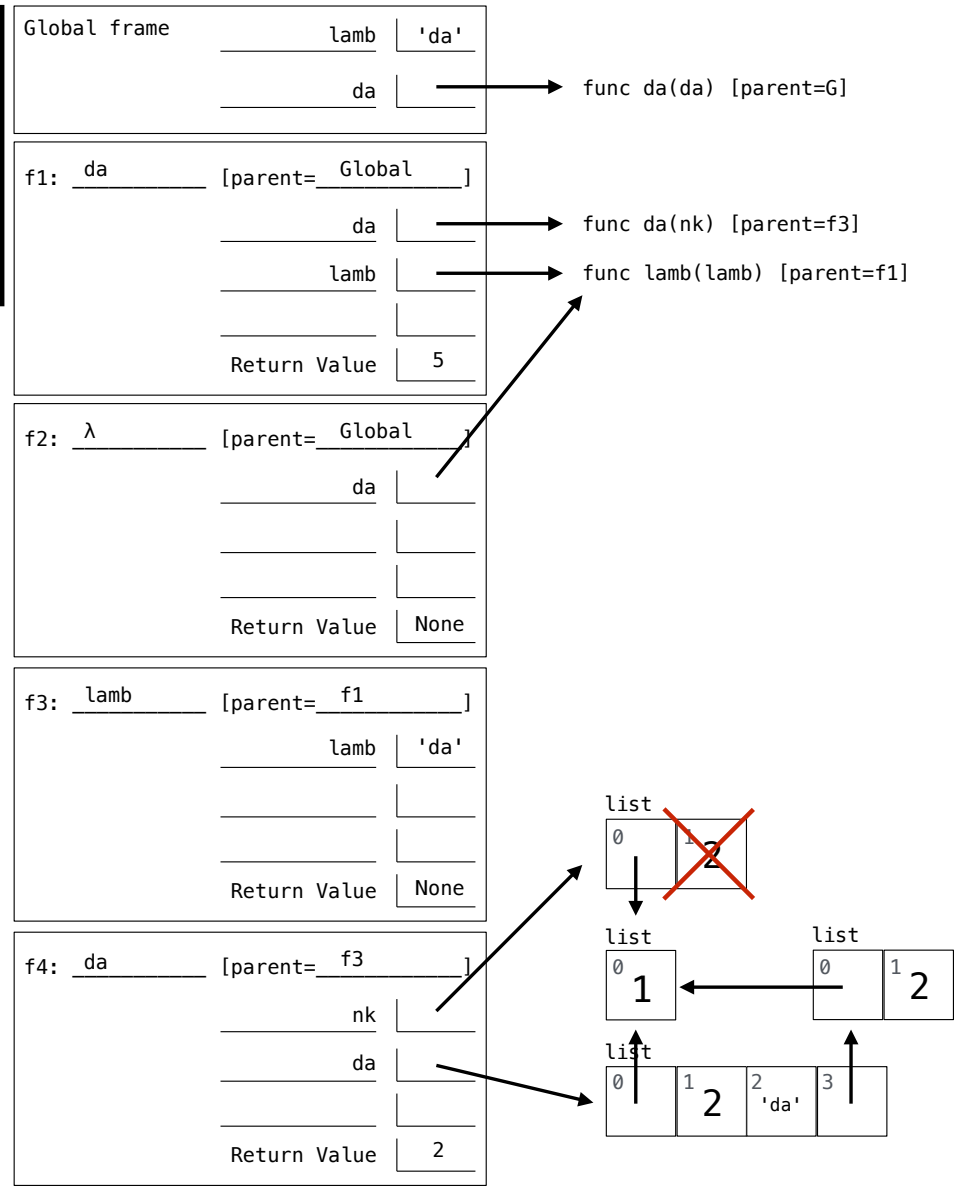
2. (8 points) A List with a Twist

Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*

A complete answer will:

- Add all missing names and parent annotations to frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.
- Use box-and-pointer notation for list values. You do not need to write index numbers or the word "list".

```
1  lamb = 'da'
2  def da(da):
3    def lamb(lamb):
4      nonlocal da
5      def da(nk):
6        da = nk + ['da']
7        da.append(nk[0:2])
8        return nk.pop()
9    da(lamb)
10   return da([[1], 2]) + 3
11
12 da(lambda da: da(lamb))
```



3. (6 points) Return Policy

Implement `quota`, which takes a one-argument function `f` and a non-negative integer `limit`. The function it returns has the same behavior as `f`, except that each value is only returned up to `limit` times. After that, the function returns an "Over quota" message instead, and the `limit` is decreased by 1 for future calls.

```
def quota(f, limit):
    """A decorator that limits the number of times a value can be returned.

    >>> square = lambda x: x * x
    >>> square = quota(square, 3)
    >>> square(6)                     # 1st call with return value 36
    36
    >>> [square(5) for x in range(3)] # 3 calls when the limit is 3
    [25, 25, 25]
    >>> square(5)                     # 4th call with return value 25
    'Over quota! Limit is now 2'
    >>> square(-6)                    # 2nd call with return value 36
    36
    >>> square(-6)                    # 3rd call when the limit is 2
    'Over quota! Limit is now 1'
    >>> square(7)                     # 1st call when the limit is 1
    49
    >>> square(5)                     # 5th call with return value 25
    'Over quota! Limit is now 0'
    """

    values = []

    def limited(n):

        nonlocal limit


        y = f(n)


        count = len([x for x in values if x == y])

        values.append(y)


        if count < limit:


            return y


        limit = limit - 1


        return 'Over quota! Limit is now ' + str(limit)

    return limited
```

4. (6 points) A Classy Election

Implement the `VotingMachine` and `Ballot` classes based on the doctest below. The voting machine must determine which choice has the most votes (the `winner`) and detect if a ballot is used more than once. In case of a tie, the `winner` among choices with maximal votes is the one that most recently received a vote. `Ballot.vote` takes a string, and a `VotingMachine` must handle an arbitrary number of choices.

```
class VotingMachine:
    """A machine that creates and records ballots.

    >>> machine = VotingMachine(4)
    >>> a, b, c, d = machine.ballots
    >>> d.vote('Bruin')
    'Bruin is winning'
    >>> b.vote('Bruin')
    'Bruin is winning'
    >>> c.vote('Bear')
    'Bear is losing'
    >>> a.vote('Bear')
    'Bear is winning'
    >>> c.vote('Tree')
    'Fraud: multiple votes from the same ballot!'
    >>> machine.winner
    'Bear'
    """
    def __init__(self, k):
        self.ballots =   [Ballot(self) for i in range(k)]
        self.votes = {}

    def record(self, ballot, choice):
        if ballot.used:
            return 'Fraud: multiple votes from the same ballot!'


        ballot.used = True


        self.votes[choice] = self.votes.get(choice, 0) + 1


        if max(self.votes.values()) > self.votes[choice]:
            return choice + ' is losing'
        else:

            self.winner = choice
            return choice + ' is winning'
class Ballot:

    used = False

    def __init__(self, machine):
        self.machine = machine
    def vote(self, x):

        return self.machine.record(self, x)
```

5. (6 points) Trick or Tree

Implement `path`, which takes a linked list `s` and a `Tree` instance `t`. It returns whether `s` is a path from the root of `t` to some leaf. The `Tree` and `Link` classes are on page 2 of the midterm 2 study guide.

Restrictions:

- You may not call the built-in `len` function on a linked list or invoke its `__len__` method.
- You may not apply element selection (e.g., `s[2]`) on a linked list or invoke its `__getitem__` method.

```
from tree import *
from link import *

def path(s, t):
    """Return whether Link S is a path from the root to a leaf in Tree T.

    >>> t = Tree(1, [Tree(2), Tree(3, [Tree(4), Tree(5)]), Tree(6)])
    >>> a = Link(1, Link(3, Link(4)))  # A full path
    >>> path(a, t)
    True
    >>> b = Link(1, Link(3))           # A partial path
    >>> path(b, t)
    False
    >>> c = Link(1, Link(2, Link(7)))  # A path and an extra value
    >>> path(c, t)
    False
    >>> d = Link(3, Link(4))           # A path of a branch
    >>> path(d, t)
    False
    """

    if s is Link.empty or t.root != s.first:


        return False


    if t.is_leaf() and s.rest is Link.empty:


        return True


    return any([path(s.rest, b) for b in t.branches])
```

6. (6 points) Left it Right There

(a) (4 pt) Implement `binary`, which takes a list `s` of unique integers. It returns a *binary search tree* containing all of those integers, represented as a `BTree` instance or `BTree.empty`. The values in any path of this tree must appear in the same order as they did in `s`. The `BTree` class is on page 2 of the midterm 2 study guide.

```
from tree import *

def binary(s):
    """Construct a binary search tree from S for which all paths are in order.

    >>> binary([3, 5, 1])
    BTree(3, BTree(1), BTree(5))
    >>> binary([4, 3, 7, 6, 2, 9, 8])
    BTree(4, BTree(3, BTree(2)), BTree(7, BTree(6), BTree(9, BTree(8))))
    """
    assert len(s) == len(set(s)), 'All elements of s should be unique'

    if not s:

        return BTree.empty

    root = s[0]

    left = [x for x in s if x < root]

    right = [x for x in s if x > root]

    return BTree(root, binary(left), binary(right))
```

(b) (1 pt) Circle the Θ expression that describes the **smallest** possible height of the tree returned by `binary(s)` for a list `s` of length n. The height of a tree is the length of the longest path from its root to a leaf.

$\Theta(1)$ $\Theta(\log n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(2^n)$ None of these

(c) (1 pt) Circle the Θ expression that describes the **largest** possible height of the tree returned by `binary(s)` for a list `s` of length n. The height of a tree is the length of the longest path from its root to a leaf.

$\Theta(1)$ $\Theta(\log n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(2^n)$ None of these

7. (10 points) Summer Camp

(a) (6 pt) Implement `sums`, which takes two positive integers `n` and `k`. It returns a list of lists containing all the ways that a list of `k` positive integers can sum to `n`. Results can appear in any order.

```
def sums(n, k):
    """Return the ways in which K positive integers can sum to N.

    >>> sums(2, 2)
    [[1, 1]]
    >>> sums(2, 3)
    []
    >>> sums(4, 2)
    [[3, 1], [2, 2], [1, 3]]
    >>> sums(5, 3)
    [[3, 1, 1], [2, 2, 1], [1, 3, 1], [2, 1, 2], [1, 2, 2], [1, 1, 3]]
    """
    if k == 1:
        return [[n]]
    y = []
    for x in range(1, n): # range(1, n-k+2) is OK
        y.extend([s + [x] for s in sums(n-x, k-1)])
    return y

    OR

    if n == 0 and k == 0:
        return [[]]
    y = []
    for x in range(1, n+1):
        y.extend([s + [x] for s in sums(n-x, k-1)])
    return y
```

(b) (4 pt) *Why so many lines?* Implement `f` and `g` for this alternative version of the `sums` function.

```
f = lambda x, y: (x and [[x] + z for z in y] + f(x-1, y)) or []

def sums(n, k):
    """Return the ways in which K positive integers can sum to N.

    >>> sums(2, 2)
    [[1, 1]]
    >>> sums(4, 2)
    [[3, 1], [2, 2], [1, 3]]
    >>> sums(5, 3)
    [[3, 1, 1], [2, 2, 1], [2, 1, 2], [1, 3, 1], [1, 2, 2], [1, 1, 3]]
    """
    g = lambda w: (w and f(n, g(w-1))) or [[]]
    return [v for v in g(k) if sum(v) == n]
```

CS 61A Fall 2017

Structure and Interpretation of Computer Programs

MIDTERM 1 SOLUTIONS

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 study guide.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

1. (10 points) I Wonder What Python Would Display

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
aaron, burr = 2, 5
aaron, burr = 4, aaron + 1

hamil = 10

def alex(hamil):
    def g(w):
        hamil = 2 * w
        print(hamil, w)
        w = hamil
        return hamil
    w = 5
    alex = g(w + 1)
    print(w, alex, hamil)

def el(i, za):
    def angelica():
        return i + 1
    if i > 10:
        return za()
    elif i > 4:
        print(angelica())
        return el(i * i, za)
    else:
        return el(i * i,
                  angelica)

K = lambda x: lambda y: x

def pr(x):
    print(x)
    return x
```

Expression	Interactive Output
`pow(2, 3)`	8
`print(4, 5) + 1`	4 5 Error
`print(aaron, burr)`	4 3
`alex(3)`	12 6 5 12 3
`el(3, el)`	10 4
`K`	Function
`K(3)`	Function
`K(3)(2)`	3
`pr(True) and pr(0) and pr(1)`	True 0 0

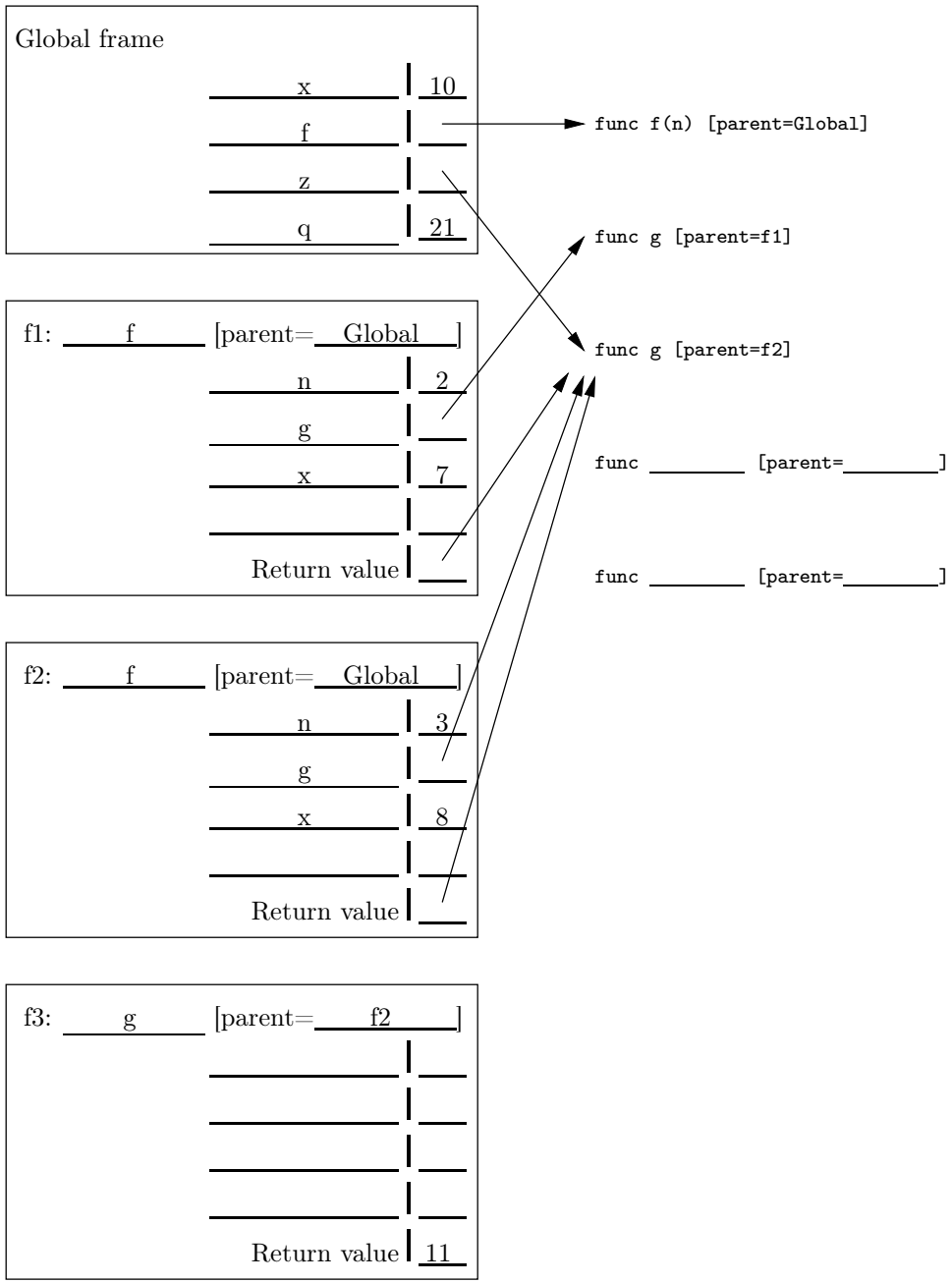
2. (8 points) Environmental Influences

Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces, frames, or function values.*

A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  x = 1
2  def f(n):
3      def g():
4          return n + x
5      x = n + 5
6      if n % 3 == 0:
7          return g
8      else:
9          return f(n + 1)
10 x = 10
11 z = f(2)
12 q = x + z()
```



3. (3 points) Triangulate

It's easy to see that in any triangle, each side must be shorter than the sum of the other two. Implement `triangle`, which takes three positive numbers, `a`, `b`, and `c`, and returns whether these three numbers could possibly be the lengths of the three sides of a triangle.

```
def triangle(a, b, c):
    """Return whether a, b, and c could be the legs of a triangle.

    >>> triangle(3, 4, 5)
    True
    >>> triangle(3, 4, 6)
    True
    >>> triangle(6, 3, 4)
    True
    >>> triangle(3, 6, 4)
    True
    >>> triangle(9, 2, 2)
    False
    >>> triangle(2, 4, 2)
    False
    """




    longest = max(a, b, c)



    sum_of_others = a + b + c - longest  # or min(a+b, a+c, b+c)



    return longest < sum_of_others
```

4. (9 points) Digital

(a) (3 pt) Implement `collapse`, which takes a non-negative integer, and returns the result of removing all digits from it that duplicate the digit immediately to their right.

```
def collapse(n):
    """For non-negative N, the result of removing all digits that are equal
    to the digit on their right, so that no adjacent digits are the same.

    >>> collapse(1234)
    1234
    >>> collapse(12234441)
    12341
    >>> collapse(0)
    0
    >>> collapse(3)
    3
    >>> collapse(11200000013333)
    12013
    """



    left, last = n // 10, n % 10



    if left == 0:



        return last



    elif last == left % 10:



        return collapse(left)



    else:



        return collapse(left) * 10 + last
```

(b) (6 pt) Implement `find_pair`, which takes a two-argument function, `p`, as input and returns another function. The returned function takes a non-negative integer `n`; it returns `True` if and only if `p` returns a true value when called on at least one pair of adjacent digits in `n`, and `False` otherwise.

```
def find_pair(p):
    """Given a two-argument function P, return a function that takes a
    non-negative integer and returns True if and only if two adjacent digits
    in that integer satisfy P (that is, cause P to return a true value).

    >>> z = find_pair(lambda a, b: a == b)   # Adjacent equal digits
    >>> z(1313)
    False
    >>> z(12334)
    True
    >>> z = find_pair(lambda a, b: a > b)
    >>> z(1234)
    False
    >>> z(123412)
    True
    >>> find_pair(lambda a, b: a <= b)(9753)
    False
    >>> find_pair(lambda a, b: a == 1)(1)    # Only one digit; no pairs.
    False
    """

    def find(n):



        while n >= 10:



            if p((n // 10) % 10, n % 10):



                return True



            else:



                n = n // 10



        return False



    return find
```

5. (10 points) Please Confirm

Definition. A *confirming function* for a sequence of digits, called a *code*, takes a single digit as its only argument. If the digit does not match the first (left-most) digit of the code to be confirmed, it returns `False`. If the digit does match, then the confirming function returns `True` if the code has only one digit, or another confirming function for the rest of the code if there are more digits to confirm.

(a) (5 pt) Implement `confirmer` so that when `confirmer` takes a positive integer `code`, it returns a confirming function for the digits of that `code`.

```
def confirmer(code):
    """Return a confirming function for CODE.

    >>> confirmer(204)(2)(0)(4) # The digits of 204 are 2, then 0, then 4.
    True
    >>> confirmer(204)(2)(0)(0) # The third digit of 204 is not 0.
    False
    >>> confirmer(204)(2)(1)    # The second digit of 204 is not 1.
    False
    >>> confirmer(204)(20)      # The first digit of 204 is not 20.
    False
    """
    def confirm1(d, t):
        def result(digit):
            if d == digit:
                return t
            else:
                return False
        return result

    def extend(prefix, rest):
        """Return a confirming function that returns REST when given the digits of PREFIX.
        For example, if c = extend(12, confirmer(34)), then c(1)(2) returns confirmer(34),
        so that c is a confirming function for 1234."""

        left, last = prefix // 10, prefix % 10



        if prefix < 10:



            return confirm1(prefix, rest)



        else:



            return extend(left, confirm1(last, rest))  # or extend(left, extend(last, rest))



    return extend(code, True)
```

(b) (5 pt) Given a confirming function, one can find the code it confirms, one digit at a time. Implement `decode`, which takes a confirming function `f` and returns the code that it confirms.

```
from confirm_soln import confirmer

def decode(f, y=0):
    """Return the code for a confirming function f.

    >>> decode(confirmer(12001))
    12001
    >>> decode(confirmer(56789))
    56789
    """


    d = 0


    while d < 10:


        x, code = f(d), 10 * y + d


        if x == True:


            return code


        elif x == False:


            d = d + 1



        else:



            return decode(x, code)
```

CS 61A Fall 2017

Structure and Interpretation of Computer Programs

MIDTERM 2 SOLUTIONS

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except two hand-written 8.5" × 11" crib sheets of your own creation and the two official CS 61A midterm study guides.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- Before asking a question, read the announcements on the screen/board. We will not answer your question directly. If we decide to respond, we'll add our response to the screen/board so everyone can see the clarification.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.

1. (12 points) By Any Other Name

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. The interactive interpreter displays the repr string of the value of a successfully evaluated expression, unless it is `None`. If an error occurs, write "Error", but include all output displayed before the error. The first row has been provided as an example.

Assume that you have started `python3` and executed the code shown on the left first, then you evaluate each expression on the right in order. Statements and expressions sent to the interpreter have a cumulative effect.

```
class Plant:
    k = 1
    kind = "green"

    def __init__(self):
        self.k = Plant.k
        Plant.k = self.k + 1
        if self.k > 3:
            Plant.name = lambda t: "tree"
            Plant.k = 6

    def name(self):
        return kind

    def __repr__(self):
        s = self.name() + " "
        return s + str(self.k)

class Flower(Plant):
    kind = "pretty"

    def __repr__(self):
        s = self.smell() + " "
        return s + Plant.__repr__(self)

    def smell(self):
        return "bad"

class Rose(Flower):
    def name(self):
        return "rose"

    def smell(self):
        return "nice"

class Garden:
    def __init__(self, kind):
        self.name = kind
        self.smell = kind().smell

    def smell(self):
        return self.name.kind

f1 = Flower()
f2 = Flower()
```

Expression	**Interactive Output**
`[2, 3]`	[2, 3]
(1 pt) `f1.name()`	`Error`
(1 pt) `f1.k`	`1`
(1 pt) `Plant().k`	`3`
(1 pt) `Rose.k`	`4`
(2 pt) `Plant()`	`tree 4`
(2 pt) `Rose()`	`nice rose 6`
(2 pt) `Garden(Flower).smell()`	`'bad'`
(2 pt) `Garden(Flower).name()`	`bad tree 6`

2. (8 points) Buy Local

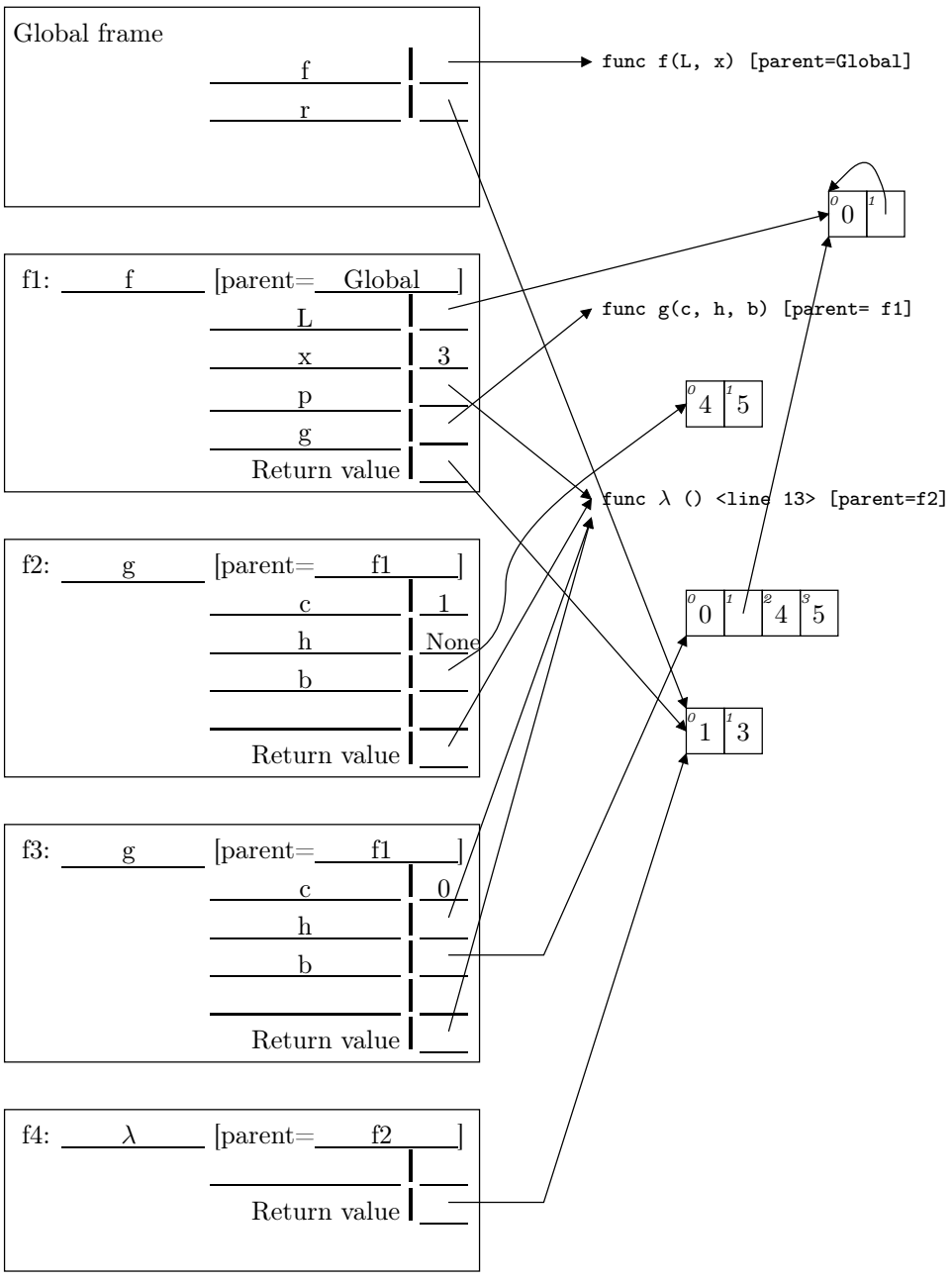
Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*

A complete answer will:

- Add all missing names and parent annotations to frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.
- Use box-and-pointer notation for list values. You do not need to write index numbers or the word "list".

Important: The slash on line 11 means that the return expression continues on the next line.

```
1  def f(L, x):
2    L[1] = L
3    def g(c, h, b):
4      nonlocal x
5      x = c
6      if c == 0:
7        b.append(5)
8        b = L + b
9        return h
10     else:
11       return \
12         g(c-1,
13           lambda: [c, x],
14           b)
15   p = g(1, None, [4])
16   x += 3
17   return p()
18 r = f([0, 0], 1)
```



3. (10 points) Pumpkin Splice Latte

(a) (2 pt) Implement `splice`, which takes two lists `a` and `b` and a non-negative integer `k` that is less than or equal to the length of `a`. It returns the result of *splicing* `b` into `a` at `k`. That is, it returns a new list containing the first `k` elements of `a`, then all elements of `b`, then the remaining elements of `a`.

```
def splice(a, b, k):
    """Return a list of the first k elements of a, then all of b, then the rest of a.

    >>> splice([2, 3, 4, 5], [6, 7], 2)
    [2, 3, 6, 7, 4, 5]
    """

    return a[:k] + b + a[k:]
```

(b) (3 pt) Implement `all_splice`, which returns a list of all the non-negative integers `k` such that splicing list `b` into list `a` at `k` creates a list with the same contents as `c`. Assume that `splice` is implemented correctly.

```
def all_splice(a, b, c):
    """Return a list of all k such that splicing b into a at position k gives c.

    >>> all_splice([1, 2], [3, 4], [1, 3, 4, 2])
    [1]
    >>> all_splice([1, 2, 1, 2], [1, 2], [1, 2, 1, 2, 1, 2])
    [0, 2, 4]
    """

    return [k for k in range(len(a) + 1) if splice(a, b, k) == c]
```

(c) (5 pt) Implement `splink`, which takes two `Link` instances `a` and `b` and a non-negative integer `k` that is less than or equal to the length of `a`. It returns a `Link` instance containing the first `k` elements of `a`, then all elements of `b`, then the remaining elements of `a`. The `Link` class is defined on the midterm 2 study guide. **Important**: You may **not** use `len`, `in`, `for`, `list`, slicing, element selection, addition, or list comprehensions.

```
def splink(a, b, k):
    """Return a Link containing the first k elements of a, then all of b, then the rest of a.

    >>> splink(Link(2, Link(3, Link(4, Link(5)))), Link(6, Link(7)), 2)
    Link(2, Link(3, Link(6, Link(7, Link(4, Link(5))))))
    """

    if b is Link.empty:


        return a


    elif k == 0:


        return Link(b.first, splink(a, b.rest, k))


    return Link(a.first, splink(a.rest, b, k-1))
```

4. (11 points) Both Ways

(a) (4 pt) Implement `both`, which takes two *sorted* linked lists composed of `Link` objects and returns whether some value is in both of them. The `Link` class is defined on the midterm 2 study guide.
Important: You may **not** use `len`, `in`, `for`, `list`, slicing, element selection, addition, or list comprehensions.

```
def both(a, b):
    """Return whether there is any value that appears in both a and b, two sorted Link instances.

    >>> both(Link(1, Link(3, Link(5, Link(7)))), Link(2, Link(4, Link(6))))
    False
    >>> both(Link(1, Link(3, Link(5, Link(7)))), Link(2, Link(7, Link(9))))  # both have 7
    True
    >>> both(Link(1, Link(4, Link(5, Link(7)))), Link(2, Link(4, Link(5))))  # both have 4 and 5
    True
    """
    if a is Link.empty or b is Link.empty:

        return False

    if a.first > b.first:

        a, b = b, a

    return a.first == b.first or both(a.rest, b)
```

(b) (2 pt) Circle the Θ expression that describes the minimum number of comparisons (e.g., `<`, `>`, `<=`,`==`, or `>=` expressions) required to verify that two sorted lists of length n contain no values in common.

$\Theta(1)$ $\Theta(\log n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(2^n)$ None of these

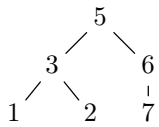
(c) (5 pt) Implement `ways`, which takes two values *start* and *end*, a non-negative integer k, and a list of one-argument functions `actions`. It returns the number of ways of choosing functions f_1, f_2, ..., f_j from `actions`, such that $f_1(f_2(...(f_j(start))))$ equals *end* and $j \leq k$. The same action function can be chosen multiple times. If a sequence of actions reaches *end*, then no further actions can be applied (see the first example below).

```
def ways(start, end, k, actions):
    """Return the number of ways of reaching end from start by taking up to k actions.

    >>> ways(-1, 1, 5, [abs, lambda x: x+2])   # abs(-1) or -1+2, but not abs(abs(-1))
    2
    >>> ways(1, 10, 5, [lambda x: x+1, lambda x: x+4])  # 1+1+4+4, 1+4+4+1, or 1+4+1+4
    3
    >>> ways(1, 20, 5, [lambda x: x+1, lambda x: x+4])
    0
    >>> ways([3], [2, 3, 2, 3], 4, [lambda x: [2]+x, lambda x: 2*x, lambda x: x[:-1]])
    3
    """
    if start == end:

        return 1

    elif k == 0:

        return 0

    return sum([ways(f(start), end, k - 1, actions) for f in actions])
```

5. (9 points) Autumn Leaves

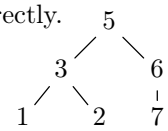
Definition. A *pile* (of leaves) for a tree t with no repeated leaf labels is a dictionary in which the label for each leaf of t is a key, and its value is the path from that leaf to the root. Each path from a node to the root is either an empty tuple, if the node is the root, or a two-element tuple containing the label of the node's parent and the rest of the path (i.e., the path to the root from the node's parent).

(a) (5 pt) Implement `pile`, which takes a tree constructed using the `tree` data abstraction. It returns a *pile* for that tree. You may use the `tree`, `label`, `branches`, and `is_leaf` functions from the midterm 2 study guide.




```
def pile(t):
    """Return a dict that contains every path from a leaf to the root of tree t.

    >>> pile(tree(5, [tree(3, [tree(1), tree(2)]), tree(6, [tree(7)])]))
    {1: (3, (5, ())), 2: (3, (5, ())), 7: (6, (5, ()))}
    """
    p = {}

    def gather(u, parent):

        if is_leaf(u):


            p[label(u)] = parent

        for b in branches(u):


            gather(b, (label(u), parent))


    gather(t, ())

    return p
```

(b) (4 pt) Implement `Path`, a class whose constructor takes a tree `t` constructed by `tree` and a `leaf_label`. Assume all leaf labels of `t` are unique. When a `Path` is printed, labels in the path from the root to the leaf of `t` with label `leaf_label` are displayed, separated by dashes. Assume `pile` is implemented correctly.




```
class Path:
    """A path through a tree from the root to a leaf, identified by its leaf label.

    >>> a = tree(5, [tree(3, [tree(1), tree(2)]), tree(6, [tree(7)])])
    >>> print(Path(a, 7), Path(a, 2))
    5-6-7 5-3-2
    """
    def __init__(self, t, leaf_label):
        self.pile, self.end = pile(t), leaf_label

    def __str__(self):

        path, s = self.pile[self.end], str(self.end)

        while path:

            path, s = path[1], str(path[0]) + '-' + s

        return s
```

CS 61A Fall 2018

Structure and Interpretation of Computer Programs

MIDTERM 1 SOLUTIONS

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 study guide.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, and `abs`.
- You **may not** use example functions defined on your study guides unless a problem clearly states you can.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.

1. (10 points) What Would Python Display

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
identity = lambda x: x
increment = lambda x: x + 1


def fif(c, t, f, x):
    if c(x):
        return t(x)
    else:
        return f(x)


def bounce(x, y):
    while x < y:
        if x <= (y and x):
            print('a')
        if x > 0:
            print('b')
        elif x > -5:
            print('c')
        x, y = -y, increment(x - y)
        print(y)


crazy = lambda rich: 100 * rich
crazy = lambda rich: crazy(crazy(rich))


def ok(py):
    def h(w):
        print(py // 10)
        return ok(py)
    return lambda h: h(py)
```

Expression	Interactive Output
`pow(10, 2)`	100
`print(4, 5) + 1`	4 5 Error
`print(None, print(1, 2))`	1 2 None None
`fif(abs, print, print, -2)`	-2
`bounce(1, 2)`	a b 0 a c -1
`crazy(88)`	Error/Forever
`ok(314)(identity)`	314

2. (4 points) Implement `factorial`, which computes the factorial of a positive integer n. You may use any of the functions defined above. **You may <u>not</u> write `if`, `else`, `and`, `or`, or `lambda` in your solution.**

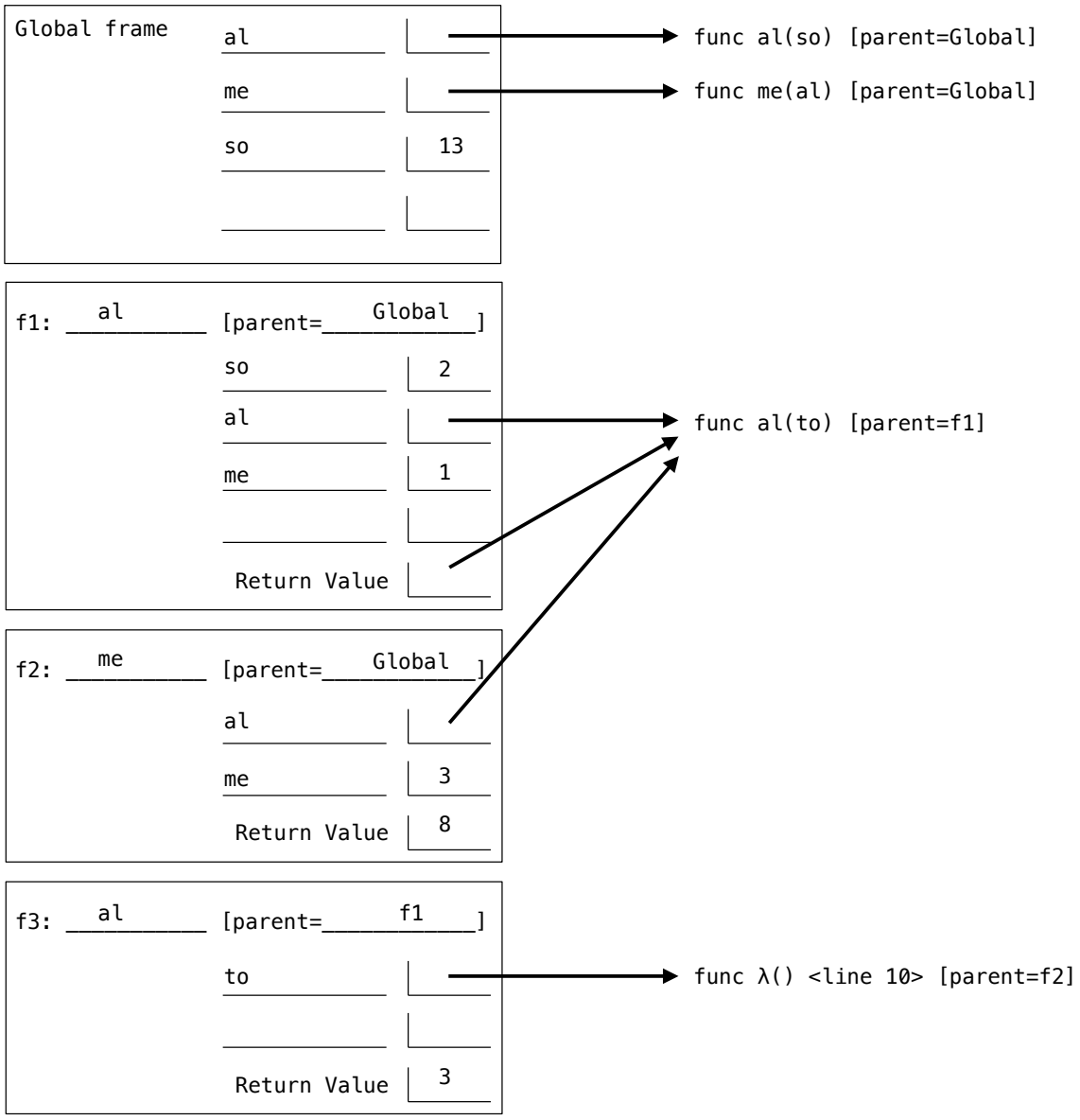
```
def factorial(n):
    """The factorial of positive n: n * (n-1) * (n-2) * ... * 1

    >>> factorial(4)  # 4 * 3 * 2 * 1
    24
    >>> factorial(1)
    1
    """
    return n * fif(identity, factorial, increment, n-1)
```

3. (6 points) Also Some Meal

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.* A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  def al(so):
2      me = 1
3      def al(to):
4          return so + me
5      so = 2
6      return al
7
8  def me(al):
9      me = 3
10     return al(lambda: 4) + so
11
12 so = 5
13 so = me(al(6)) + so
```



4. (5 points) Getting Rect

Implement `rect`, which takes two positive integer arguments, `perimeter` and `area`. It returns the **integer** length of the longest side of a rectangle with integer side lengths ℓ and h which has the given perimeter and area. If no such rectangle exists, it returns `False`.

The perimeter of a rectangle with sides ℓ and h is $2\ell + 2h$. The area is $\ell \cdot h$.

Hint: The built-in function `round` takes a number as its argument and returns the nearest integer. For example, `round(2.0)` evaluates to `2`, and `round(2.5)` evaluates to `3`.

```
def rect(area, perimeter):
    """Return the longest side of a rectangle with AREA and PERIMETER that has integer sides.

    >>> rect(10, 14)  # A 2 x 5 rectangle
    5
    >>> rect(5, 12)   # A 1 x 5 rectangle
    5
    >>> rect(25, 20)  # A 5 x 5 rectangle
    5
    >>> rect(25, 25)  # A 2.5 x 10 rectangle doesn't count because sides are not integers
    False
    >>> rect(25, 29)  # A 2 x 12.5 rectangle doesn't count because sides are not integers
    False
    >>> rect(100, 50) # A 5 x 20 rectangle
    20
    >>> rect(5, 11)
    False
    >>> rect(4, 11)
    False
    """


    side = 1


    while side * side <= area:


        other = round(perimeter / 2 - side)


        if side * other == area and 2 * (side + other) == perimeter:


            return other


        side = side + 1


    return False
```

5. (6 points) Dig It

Implement `sequence`, which takes a positive integer `n` and a function `term`. It returns an integer whose digits show the n elements of the sequence `term(1)`, `term(2)`, ..., `term(n)` in order. Assume the `term` function takes a positive integer argument and returns a positive integer.

Important: You may not use `pow`, `**`, `log`, `str`, or `len` in your solution.

```
def sequence(n, term):
    """Return the first n terms of a sequence as an integer.

    >>> sequence(6, abs)                  # Terms are 1, 2, 3, 4, 5, 6
    123456
    >>> sequence(5, lambda k: k+8)        # Terms are 9, 10, 11, 12, 13
    910111213
    >>> sequence(4, lambda k: pow(10, k)) # Terms are 10, 100, 1000, 10000
    10100100010000
    """


    t, k = 0, 1


    while k <= n:


        m = 1


        x = term(k)


        while m <= x:


            m *= 10


        t = t * m + x


        k = k + 1


    return t
```

6. (9 points) This Again?

Definitions. A *repeatable integer* function takes an integer argument and returns a repeatable integer function.

(a) (6 pt) Implement `repeat`, which is a repeatable integer function that detects repeated arguments. As a side effect of repeated calls, it prints each argument that has been used before in a sequence of repeated calls. Therefore, if an argument appears n times, it is printed $n - 1$ times in total, each time other than the first. The `detector` function is part of the implementation of `repeat`; you must determine how it is used.

Important: You may <u>not</u> use a list, set, or any other data type not covered yet in the course.

```
def repeat(k):
    """When called repeatedly, print each repeated argument.

    >>> f = repeat(1)(7)(7)(3)(4)(2)(5)(1)(6)(5)(1)
    7
    1
    5
    1
    """

    return detector(lambda n: False)(k)


def detector(f):

    def g(i):


        if f(i):


            print(i)


        return detector(lambda n: n == i or f(n))


    return g
```

(b) (3 pt) Implement `repeat_digits`, which takes a non-negative integer `n`. For the digits of `n` from right to left, it prints each digit that also appears somewhere to its right. Assume `repeat` is implemented correctly.

```
def repeat_digits(n):
    """Print the repeated digits of non-negative integer n.

    >>> repeat_digits(581002821)
    2
    0
    1
    8
    """

    f = repeat

    while n:

        f, n = f(n % 10), n // 10
```

CS 61A Fall 2018

Structure and Interpretation of Computer Programs

MIDTERM 2 SOLUTIONS

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except two hand-written 8.5" × 11" crib sheets of your own creation and the official CS 61A study guides.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, `abs`, `sum`, `next`, `iter`, `list`, `tuple`, `map`, `filter`, `zip`, `all`, and `any`.
- You **may not** use example functions defined on your study guides unless a problem clearly states you can.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.
- You may use the `Tree`, `Link`, and `BTree` classes defined on Page 2 (left column) of the Midterm 2 Study Guide.

1. (12 points) A/B Test

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
x = [1, 2]

class A:
    x = 3
    y = 4

    def __init__(self, y):
        self.a = y
        self.x = b
        self.__str__ = lambda: str(y)

    def __str__(self):
        return 'BB'

class B(A):
    x = [5, 6]

    def __init__(self, y):
        self.a = x[1]
        y[1] = 8

b = B(x)
a = A(6)

def dash(x):
    return print(self.x)

elastigirl = Link(7, Link(8))
elastigirl.first = elastigirl.rest
```

Expression	Interactive Output
`pow(10, 2)`	100
`print(Link(2, Link(3)))`	`<2, 3>`
`[c.a for c in [a, b]]`	`[6, 2]`
`print(a.x, b.x)`	`BB [5, 6]`
`print(b.y, x)`	`4 [1, 8]`
`a.__str__()`	`'6'`
`dash(b)`	`Error`
`print(elastigirl)`	`<<8>, 8>`

2. (3 points) Implement `lowest`, which takes a list of numbers `s` and returns a list of only the elements of `s` with the smallest absolute value. **You may only write a single name in each blank.**

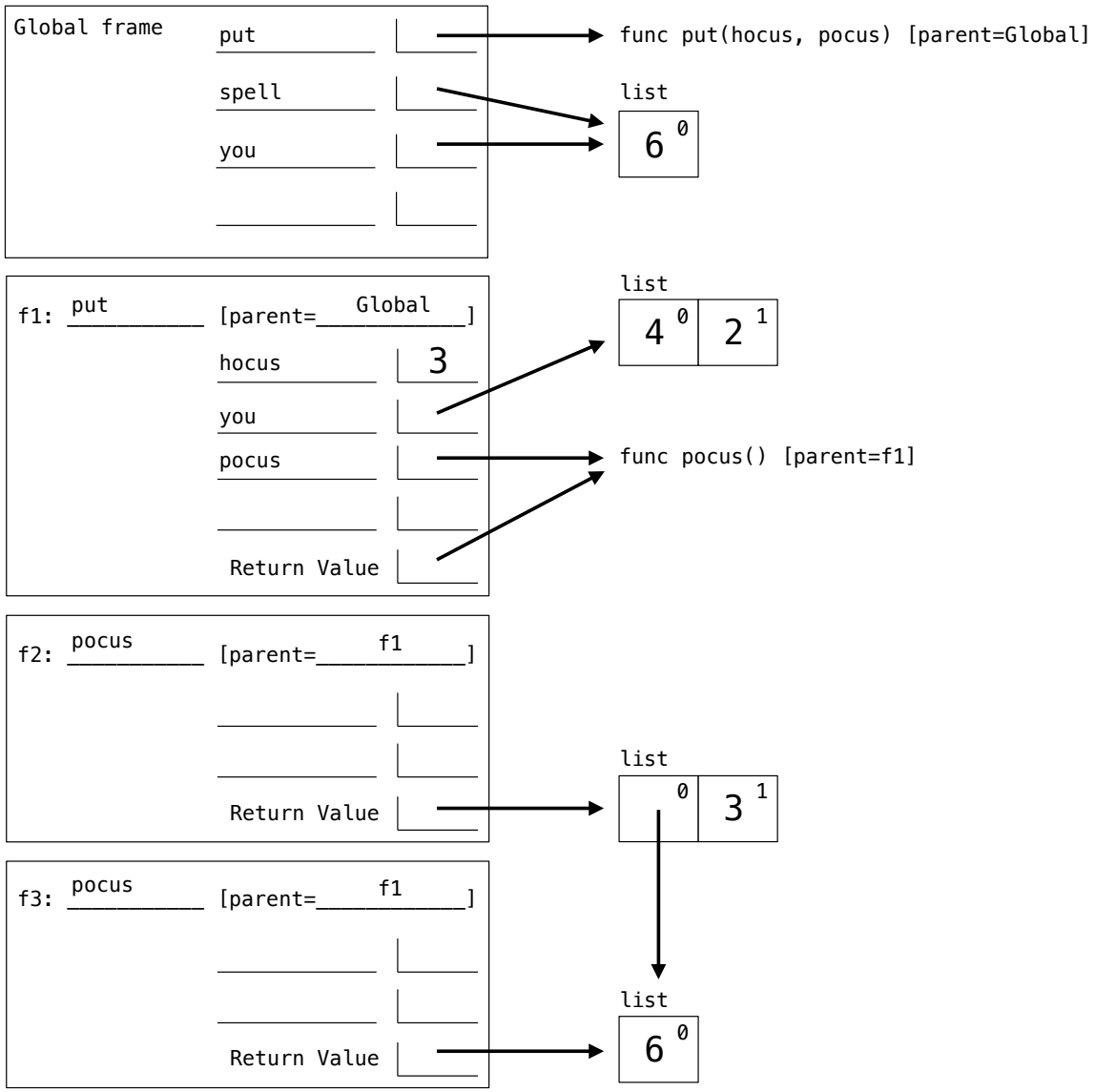
```
def lowest(s):
    """Return a list of the elements in s with the smallest absolute value.

    >>> lowest([3, -2, 2, -3, -4, 2, 3, 4])
    [-2, 2, 2]
    >>> lowest(range(-5, 5))
    [0]
    """
    return (lambda y: [x for x in s if abs(x) == y])(abs(min(s, key=abs)))
```

3. (8 points) Hocus Pocus

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*
A complete answer will:

- Use box-and-pointer notation for all lists.
- Add missing names and parents to all local frames.
- Add missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  def put(hocus, pocus):
2      hocus = 2
3      you = pocus[hocus]
4      def pocus():
5          nonlocal hocus
6          if type(spell.pop()) == list:
7              you.append(hocus)
8              return [pocus(), hocus]
9          else:
10             hocus = 3
11             return spell[0:1]
12     return pocus
13 spell = [6, 5, [4]]
14 you = spell
15 put(spell, you)()
```



4. (10 points) Nonplussed

Definition. A *plus expression* for a non-negative integer `n` is made by inserting `+` symbols in between digits of `n`, such that there are **never more than two consecutive digits** in the resulting expression. For example, one plus expression for 2018 is `20+1+8`, and its value is 29. Assume that a two-digit number starting with 0 evaluates to its one's digit. For example, another plus expression for 2018 is `2+01+8`, and its value is 11.

(a) (3 pt) Implement `plus`, which takes a non-negative integer `n`. It returns the largest value of any plus expression for `n`.

```
def plus(n):
    """Return the largest sum that results from inserting +'s into n.

    >>> plus(123456)     # 12 + 34 + 56 = 102
    102
    >>> plus(1604)       # 1 + 60 + 4 = 65
    65
    >>> plus(160450)     # 1 + 60 + 4 + 50 = 115
    115
    """
    if n:

        return max(n % 10 + plus(n // 10), n % 100 + plus(n // 100))

    return 0
```

(b) (5 pt) Implement `plusses`, which takes non-negative integers `n` and `cap`. It returns the number of plus expressions for `n` that have a value less than `cap`.

```
def plusses(n, cap):
    """Return the number of plus expressions for n with values below cap.

    >>> plusses(123, 16)  # 1+2+3=6 and 12+3=15, but 1+23=24 isn't below cap.
    2
    >>> plusses(2018, 38) # 2+0+1+8, 20+1+8, 2+0+18, and 2+01+8, but not 20+18.
    4
    >>> plusses(1, 2)
    1
    """
    if n < 10 and n < cap:

        return 1

    elif cap <= 0:

        return 0

    else:

        return plusses(n // 10, cap - n % 10) + plusses(n // 100, cap - n % 100)
```

(c) (2 pt) Circle the Θ expression that describes how many addition operations are required to evaluate a plus expression for a positive integer n.

$\Theta(1)$ $\Theta(\log n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(2^n)$ $\Theta(10^n)$ None of these

5. (8 points) Midterm Elections

(a) (6 pt) Implement the `Poll` class and the `tally` function, which takes a choice `c` and returns a list describing the number of votes for `c`. This list contains pairs, each with a name and the number of times `vote` was called on that `choice` at the `Poll` with that name. Pairs can be in any order. Assume all `Poll` instances have distinct names. *Hint*: the dictionary `get(key, default)` method (MT 2 guide, page 1 top-right) returns the value for a `key` if it appears in the dictionary and `default` otherwise.

```
class Poll:
    s = []
    def __init__(self, n):

        self.name = n

        self.votes = {}

        Poll.s.append(self)

    def vote(self, choice):

        self.votes[choice] = self.votes.get(choice, 0) + 1

def tally(c):
    """Tally all votes for a choice c as a list of (poll name, vote count) pairs.

    >>> a, b, c = Poll('A'), Poll('B'), Poll('C')
    >>> c.vote('dog')
    >>> a.vote('dog')
    >>> a.vote('cat')
    >>> b.vote('cat')
    >>> a.vote('dog')
    >>> tally('dog')
    [('A', 2), ('C', 1)]
    >>> tally('cat')
    [('A', 1), ('B', 1)]
    """

    return [(p.name, p.votes[c]) for p in Poll.s if c in p.votes]
```

(b) (2 pt) Implement the `vote` method of the `Crooked` class, which only records every other `vote` call for each `Crooked` instance. Only odd numbered calls to `vote` are recorded, e.g., first, third, fifth, etc.

```
class Crooked(Poll):
    """A poll that ignores every other call to vote.

    >>> d = Crooked('D')
    >>> for s in ['dog', 'cat', 'dog', 'cat', 'cat']:
    ...     d.vote(s)
    >>> d.votes
    {'dog': 2, 'cat': 1}
    """
    record = True
    def vote(self, choice):
        if self.record:

            Poll.vote(self, choice)

        self.record = not self.record
```

6. (4 points) Dr. Frankenlink

Implement `replace`, which takes two non-empty linked lists `s` and `t`, as well as **positive** integers `i` and `j` with `i < j`. It mutates `s` by removing elements with indices from `i` to `j` (removing element i but not removing element `j`) and replacing them with `t`. Afterward, `s` contains all of the objects in `t`, so a change to `t` would be reflected in `s` as well. `t` may change as a result of calling `replace`. Assume `s` has at least `j` elements.

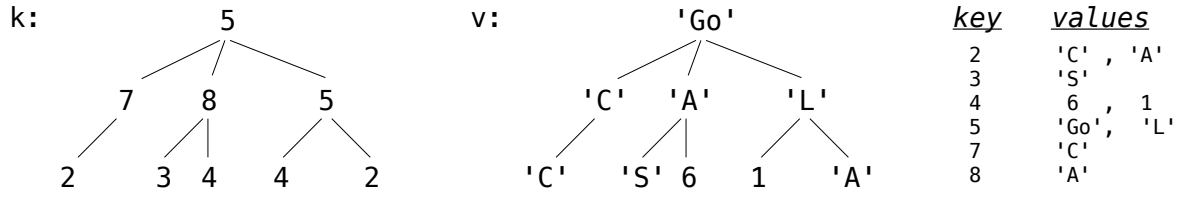
```
def replace(s, t, i, j):
    """Replace the slice of s from i to j with t.

    >>> s, t = Link(3, Link(4, Link(5, Link(6, Link(7))))), Link(0, Link(1, Link(2)))
    >>> replace(s, t, 2, 4)
    >>> print(s)
    <3, 4, 0, 1, 2, 7>
    >>> t.rest.first = 8
    >>> print(s)
    <3, 4, 0, 8, 2, 7>
    """

    assert s is not Link.empty and t is not Link.empty and i > 0 and i < j


    if i > 1:


        replace(s.rest, t, i - 1, j - 1)


    else:


        for k in range(j - i):


            s.rest = s.rest.rest


        end = t


        while end.rest is not Link.empty:


            end = end.rest


        s.rest, end.rest = t, s.rest
```

7. (5 points) Trictionary or Treat

Definitions. A *trictionary* is a pair of `Tree` instances `k` and `v` that have identical structure: each node in `k` has a corresponding node in `v`. The labels in `k` are called *keys*. Each key may be the label for multiple nodes in `k`, and the *values* for that key are the labels of all the corresponding nodes in `v`.

A *lookup function* returns one of the values for a key. Specifically, a lookup function for a node in `k` is a function that takes `v` as an argument and returns the label for the corresponding node in `v`.

Implement the generator function `lookups`, which takes a `Tree` instance `k` and some `key`. It yields all *lookup functions* for nodes in `k` that have `key` as their label. The `new_lookup` function is part of the implementation.




```
k = Tree(5, [Tree(7, [Tree(2)]), Tree(8, [Tree(3), Tree(4)]), Tree(5, [Tree(4), Tree(2)])])
v = Tree('Go', [Tree('C', [Tree('C')]), Tree('A', [Tree('S'), Tree(6)]), Tree('L', [Tree(1), Tree('A')])])

def lookups(k, key):
    """Yield one lookup function for each node of k that has the label key.

    >>> [f(v) for f in lookups(k, 2)]
    ['C', 'A']
    >>> [f(v) for f in lookups(k, 3)]
    ['S']
    >>> [f(v) for f in lookups(k, 6)]
    []
    """

    if k.label == key:


        yield lambda v: v.label


    for i in range(len(k.branches)):


        for lookup in lookups(k.branches[i], key):


            yield new_lookup(i, lookup)


def new_lookup(i, f):


    def g(v):


        return f(v.branches[i])


    return g
```

No more questions.

CS 61A Fall 2019

Structure and Interpretation of Computer Programs

MIDTERM 1 SOLUTIONS

INSTRUCTIONS

- You have 1 hour and 50 minutes to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 study guide.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, and `abs`.
- You **may not** use example functions defined on your study guide unless a problem clearly states you can.
- For fill-in-the-blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.

1. (12 points) What Would Python Display

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
def mint(y):
    return print(-2)

def snooze(e, f):
    if e and f():
        print(e)
    if e or f():
        print(f)
    if not e:
        print('naughty')

def lose():
    return -1

def alarm():
    print('Midterm')
    1 / 0
    print('Time')

def sim(b, a):
    while a > 1:
        def sc(ar):
            a = b + 4
            return b
        a, b = a // 2, b - a
        print(a)
    print(sc(b - 1), a)

pumbaa = lambda f: lambda x: f(f(x))
pumbaa = pumbaa(pumbaa)
rafiki = 1
timon = lambda y: y + rafiki
rafiki = -1
```

	Expression	Interactive Output
	`pow(10, 2)`	100
	`print(4, 5) + 1`	4 5 Error
(2 pt)	`print(mint(print))`	-2 None
(3 pt)	`print(snooze(1, lose))`	1 Function None
(3 pt)	`snooze(print(1), alarm)`	1 Midterm Error
(2 pt)	`sim(3, 3)`	1 0 1
(2 pt)	`pumbaa(timon)(5)`	1

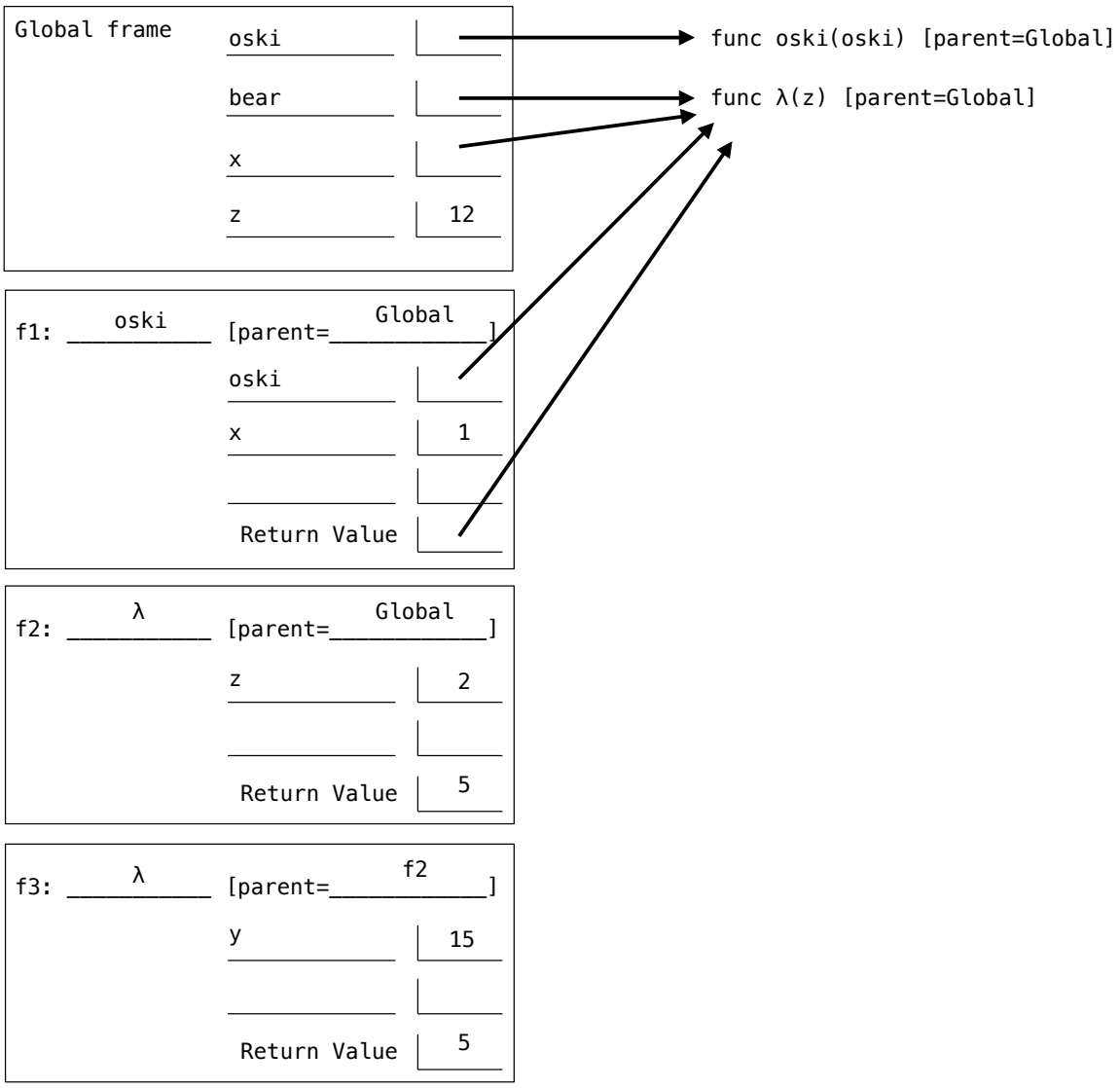
2. (6 points) Environmental Studies

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*

A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1 def oski(oski):
2     x = 1
3     if oski(2) > x:
4         return oski
5 bear = lambda z: (lambda y: z+3)(x+4)
6 x, z = 11, 12
7 x = oski(bear)
```



3. (5 points) You Again

Implement `again`, which takes a function `f` as an argument. The `again` function returns the smallest non-negative integer `n` for which `f(n)` is equal to `f(m)` for some non-negative `m` that is less than `n`. Assume that `f` takes non-negative integers and returns the same value for at least two different non-negative arguments.

Constraints:

- Lines numbered 2, 4, and 5 must begin with either `while` or `if`.
- Lines numbered 6 and 7 must contain either `return` or `=`.

```
def parabola(x):
    """A parabola function (for testing the again function)."""
    return (x-3) * (x-6)

def vee(x):
    """A V-shaped function (for testing the again function)."""
    return abs(x-2)

def again(f):
    """Return the smallest non-negative integer n such that f(n) == f(m) for some m < n.

    >>> again(parabola)  # parabola(4) == parabola(5)
    5
    >>> again(vee)       # vee(1) == vee(3)
    3
    """
    n = 1
    while True:
        m = 0
        while m < n:
            if f(n) == f(m):
                return n
            m = m + 1
        n = n + 1
```

4. (17 points) Ups and Downs

Definition. Two adjacent digits in a non-negative integer are an *increase* if the left digit is smaller than the right digit, and a *decrease* if the left digit is larger than the right digit.

For example, 61127 has 2 increases ($1 \to 2$ and $2 \to 7$) and 1 decrease ($6 \to 1$).

You may use the `sign` function defined below in all parts of this question.

```
def sign(x):
    if x > 0:
        return 1
    elif x < 0:
        return -1
    else:
        return 0
```

(a) (5 pt) Implement `ramp`, which takes a non-negative integer `n` and returns whether it has more *increases* than *decreases* when reading its digits from left to right (see the definition above).

```
from sign import *

def ramp(n):
    """Return whether non-negative integer N has more increases than decreases.

    >>> ramp(123)   # 2 increases (1 -> 2, 2 -> 3) and 0 decreases
    True
    >>> ramp(1315)  # 2 increases (1 -> 3, 1 -> 5) and 1 decrease (3 -> 1)
    True
    >>> ramp(176)   # 1 increase (1 -> 7) and 1 decrease (7 -> 6)
    False
    >>> ramp(5)     # 0 increases and 0 decreases
    False
    """

    n, last, tally = n // 10, n % 10, 0

    while n:

        n, last, tally = n // 10, n % 10, tally + sign(last - n % 10)


    return tally > 0
```

(b) (3 pt) Implement `over_under`, which takes a number `y` and returns a function that takes a number `x`. This function returns 1 if `x` is greater than `y`, 0 if `x` equals `y`, and -1 if `x` is less than `y`.

You may not use `if`, `and`, or `or`.

```
from sign import *

def over_under(y):
    """Return a function that takes X and returns:
         -1 if X is less than Y
          0 if X is equal to Y
          1 if X is greater than Y

    >>> over_under(5)(3)  # 3 < 5
    -1
    >>> over_under(5)(5)  # 5 == 5
    0
    >>> over_under(5)(7)  # 7 > 5
    1
    """
    return lambda x: sign(x-y)
```

Read this first. The `process` function below uses `tally` and `result` functions to analyze all adjacent pairs of digits in a non-negative integer `n`. A `tally` function is called on each pair of adjacent digits.

```
def process(n, tally, result):
    """Process all pairs of adjacent digits in N using functions TALLY and RESULT."""
    while n >= 10:
        tally, result = tally(n % 100 // 10, n % 10)
        n = n // 10
    return result()
```

(c) (6 pt) Implement `ups`, which returns two functions that can be passed as `tally` and `result` arguments to `process`, so that `process` computes whether a non-negative integer `n` has exactly `k` *increases*.

Hint: You can use `sign` from the previous page and the built-in `max` and `min` functions.

```
from sign import *
from process import *

def ups(k):
    """Return tally and result functions that compute whether N has exactly K increases.

    >>> f, g = ups(3)
    >>> process(1200849, f, g)    # Exactly 3 increases: 1 -> 2, 0 -> 8, 4 -> 9
    True
    >>> process(94004, f, g)      # 1 increase: 0 -> 4
    False
    >>> process(122333445, f, g)  # 4 increases: 1 -> 2, 2 -> 3, 3 -> 4, 4 -> 5
    False
    >>> process(0, f, g)          # 0 increases
    False
    """
    def f(left, right):


        return ups(min(k, k + sign(left - right)))


    return f, lambda: k == 0
```

(d) (3 pt) Implement `at_most`, which returns `True` if the number of *increases* in a non-negative integer `n` is less than or equal to `k`, and `False` otherwise. Assume `ups` is implemented correctly. You may use any of the functions from previous parts of this question: `sign`, `ramp`, `over_under`, and `process`.

```
from ups_sol import *

def at_most(n, k):
    """Return whether non-negative integer N has at most K increases.

    >>> at_most(567, 3)
    True
    >>> at_most(567, 2)
    True
    >>> at_most(567, 1)
    False
    """
    result = False

    while k >= 0:

        a, b = ups(k)

        k, result = k - 1, result or process(n, a, b)

    return result
```

CS 61A Fall 2019

Structure and Interpretation of Computer Programs

MIDTERM 2 SOLUTIONS

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except two hand-written 8.5" × 11" crib sheet of your own creation and the provided CS 61A study guides.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, `abs`, `sum`, `next`, `iter`, `list`, `sorted`, `reversed`, `tuple`, `map`, `filter`, `zip`, `all`, and `any`.
- You **may not** use example functions defined on your study guide unless a problem clearly states you can.
- For fill-in-the-blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.
- You may use the `Tree` and `Link` classes defined on Page 2 (left column) of the Midterm 2 Study Guide.

1. (12 points) What Would Python Display

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the contents of the `repr` string of the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left, which cause no errors. Assume that expressions on the right are executed in the order shown. Expressions evaluated by the interpreter have a cumulative effect.

```
class C:
    x = 'e'
    def f(self, y):
        self.x = self.x + y
        return self

    def __str__(self):
        return 'go'

class Big(C):
    x = 'u'
    def f(self, y):
        C.x = C.x + y
        return C.f(self, 'm')

    def __repr__(self):
        return '<bears>'

m = C().f('i')
n = Big().f('o')

def g(y):
    def h(x):
        nonlocal h
        k = 2
        if len(x) == 1:
            h = lambda x: y
            k = 1
        return [-x[0]] + h(x[k:])
    return h(y)

py = {3: {5: [7]}}
thon = {5: 6, 7: py}
thon[7][3] = thon

q = range(3, 5)
r = iter(q)
code = [next(r), next(iter(q)), list(r)]
```

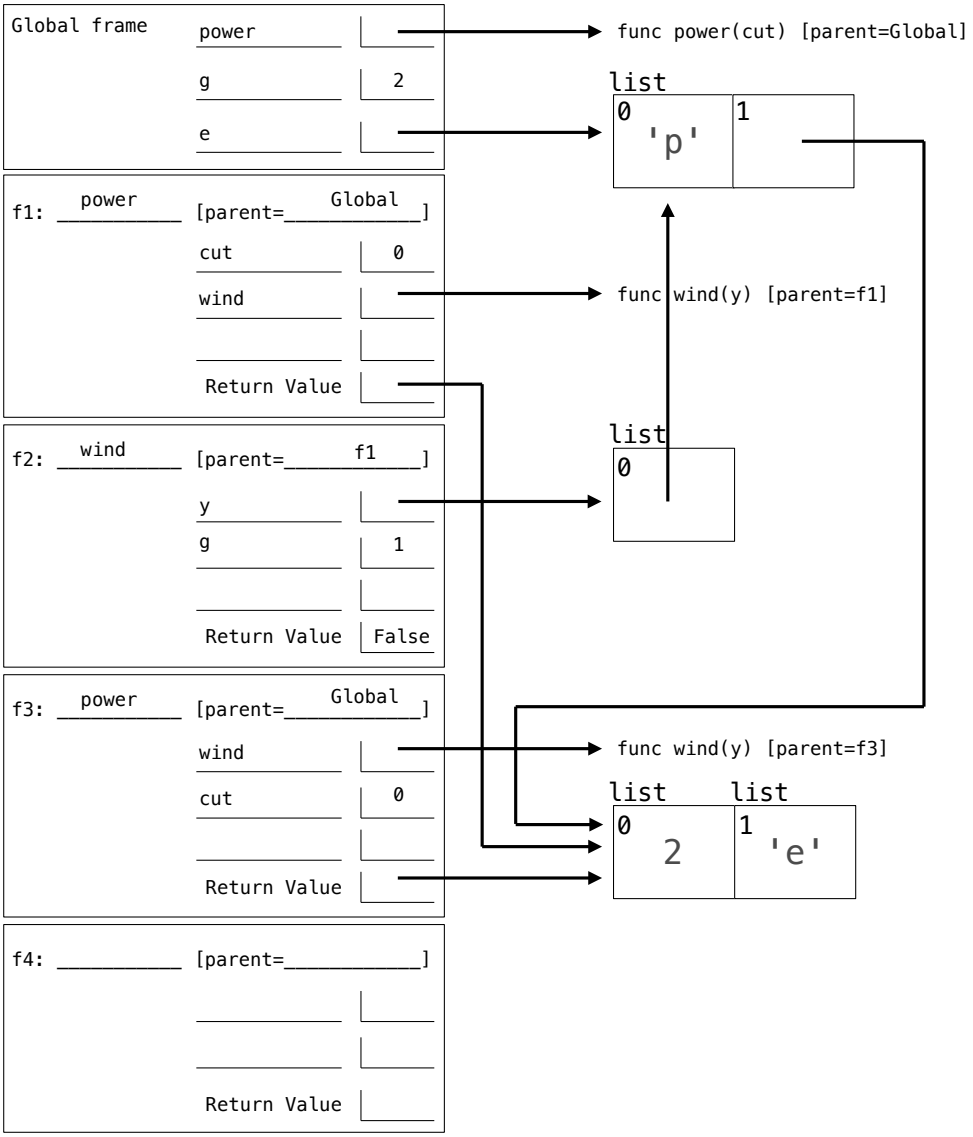
	Expression	Interactive Output
	`pow(10, 2)`	100
	`print(4, 5) + 1`	4 5 Error
(2 pt)	`[m.x, n.x]`	`['ei', 'um']`
(2 pt)	`[C.f(n, 'a').x, C().x]`	`['uma', 'eo']`
(2 pt)	`print(m, n)`	`go go`
(1 pt)	`n`	`<bears>`
(2 pt)	`g([3, 4, 5])`	`[-3, -5, 3, 4, 5]`
(1 pt)	`py[3][5]`	6
(2 pt)	`code`	`[3, 3, [4]]`

2. (8 points) Environmental Studies

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*
A complete answer will:

- Use box-and-pointer notation for lists.
- Add all missing names and parent annotations.
- Add all missing values created or referenced.
- Show the return value for each local frame.

```
1  def power(cut):
2      def wind(y):
3          nonlocal cut
4          cut, g = cut - 1, cut
5          y = [e]
6          return y is e
7      if cut and not wind('y'):
8          return power(cut)
9      else:
10         return [g, 'e']
11 g = 'p'
12 e = [g]
13 g = 2
14 e.append(power(1))
```



3. (6 points) Max Tree

(a) (4 pt) Implement `max_tree`, which takes a `Tree` instance `t` and a function `key` that takes one argument and returns a number. The `max_tree` function returns the label `n` of `t` for which `key(n)` is largest. If there is more than one label for which `key` returns the same largest value, `max_tree` can return any of those labels. The `Tree` class is defined on the Midterm 2 Study Guide. You may **not** call `min` or `max`.

```
def max_tree(t, key):
    """Return the label n of t for which key(n) returns the largest value.

    >>> t = Tree(6, [Tree(3, [Tree(5)]), Tree(2), Tree(4, [Tree(7)])])
    >>> max_tree(t, key=lambda x: x)
    7
    >>> max_tree(t, key=lambda x: -x)
    2
    >>> max_tree(t, key=lambda x: -abs(x - 4))
    4
    """
    if t.is_leaf():


        return t.label


    x = t.label


    for b in t.branches:


        m = max_tree(b, key)


        if key(m) > key(x):


            x = m


    return x
```

(b) (2 pt) Now implement `max_tree` in one line using a call to the built-in `max` function.
You may **not** include any additional calls to `max` beyond the one included in the template below.

```
def max_tree(t, key):
    "Return the label n of t for which key(n) returns the largest value."


    return max([t.label] + [max_tree(b, key) for b in t.branches], key=key)
```

4. (4 points) Seek Once

Implement `stable`, which takes a list of numbers `s`, a positive integer `k`, and a non-negative number `n`. It returns whether all pairs of values in `s` with indices that differ by at most `k` have an absolute difference in value of at most `n`.

You may **not** use `lambda`, `if`, `and`, or `or` in your solution.

Note: The "functions that aggregate iterable arguments" on the Midterm 2 Study Guide have the following behavior when called on an empty list:

- `sum([])` evaluates to 0.
- `max([])` causes a `ValueError`.
- `min([])` causes a `ValueError`.
- `all([])` evaluates to `True`.
- `any([])` evaluates to `False`.

```
def stable(s, k, n):
    """Return whether all pairs of elements of S within distance K differ by at most N.

    >>> stable([1, 2, 3, 5, 6], 1, 2)  # All adjacent values differ by at most 2.
    True
    >>> stable([1, 2, 3, 5, 6], 2, 2)  # abs(5-2) is a difference of 3.
    False
    >>> stable([1, 5, 1, 5, 1], 2, 2)  # abs(5-1) is a difference of 4.
    False
    """

    for i in range(len(s)):


        near = range(max(0, i-k), i)


        if any([abs(s[i] - s[j]) > n for j in near]):


            return False


    return True
```

5. (7 points) Do You Yield?

(a) (6 pt) Implement `partitions`, which is a generator function that takes positive integers `n` and `m`. It yields strings describing all partitions of `n` using parts up to size `m`. Each partition is a sum of non-increasing positive integers less than or equal to `m` that totals `n`. The `partitions` function yields a string for each partition exactly once.

You may **not** use `lambda`, `if`, `and`, `or`, lists, tuples, or dictionaries in your solution (other than what already appears in the template).

```
def partitions(n, m):
    """Yield all partitions of N using parts up to size M.

    >>> list(partitions(1, 1))
    ['1']
    >>> list(partitions(2, 2))
    ['2', '1 + 1']
    >>> list(partitions(4, 2))
    ['2 + 2', '2 + 1 + 1', '1 + 1 + 1 + 1']
    >>> for p in partitions(6, 4):
    ...     print(p)
    4 + 2
    4 + 1 + 1
    3 + 3
    3 + 2 + 1
    3 + 1 + 1 + 1
    2 + 2 + 2
    2 + 2 + 1 + 1
    2 + 1 + 1 + 1 + 1
    1 + 1 + 1 + 1 + 1 + 1
    """
    if n == m:

        yield str(m)

    if n > 0 and m > 0:

        for p in partitions(n - m, m):

            yield str(m) + ' + ' + p

        yield from partitions(n, m - 1)
```

(b) (1 pt) Circle the order of growth of the value of `len(list(partitions(n, 2)))` as a function of `n`. For example, the value of `len(list(partitions(3, 2)))` is 2, because there are 2 partitions of 3 using parts up to size 2: $2+1$ and $1+1+1$. Assume `partitions` is implemented correctly.

Constant Logarithmic Linear Quadratic Exponential None of these

6. (4 points) Best of Both

Definition. A *switch list* `r` for two source lists `s` and `t`, both of length n, is a list where each element `r[i]` for $0 \leq i < n$ is either `s[i]` or `t[i]`. The *switch count* for `r` is the number of indices `i` for which `r[i]` and `r[i-1]` come from different lists. As a special case, index 0 contributes 0 to the switch count if `r[0]` comes from `s` and 1 if it comes from `t`.

Implement `switch`, which takes two lists of numbers `s` and `t` that have the same length, and a non-negative integer `k`. It returns the *switch list* for `s` and `t` that has the largest sum and has a *switch count* of at most `k`.

Constraints:

- The blanks on line 4 must be filled with one of the following combinations:
  - `s, t`
  - `t, s`
  - `s[1:], t`
  - `t, s[1:]`
  - `s[1:], t[1:]`
  - `t[1:], s[1:]`
- The blank on line 5 must use one of the following templates:
  - `_______ + switch(_______, _______, k - 1)`
  - `_______ + switch(_______, _______, k)`
  - `switch(_______, _______, k - 1)`
  - `switch(_______, _______, k)`

```
def switch(s, t, k):
    """Return the list with the largest sum built by switching between S and T at most K times.

    >>> switch([1, 2, 7], [3, 4, 5], 0)
    [1, 2, 7]
    >>> switch([1, 2, 7], [3, 4, 5], 1)
    [3, 4, 5]
    >>> switch([1, 2, 7], [3, 4, 5], 2)
    [3, 4, 7]
    >>> switch([1, 2, 7], [3, 4, 5], 3)
    [3, 4, 7]
    """
    if k == 0 or len(s) == 0:
        return s
    else:
        a = switch(t, s, k-1)
        b = s[:1] + switch(s[1:], t[1:], k)
        return max(a, b, key=sum)
```

7. (5 points) Version 2.0

Implement the `Version`, `Insert`, and `Delete` classes as follows:

- A `Version` represents a string that has been edited. A `Version` instance has a `previous` value (which may be a `str` or `Version` instance) and an `edit` (which may be an instance of `Insert` or `Delete`).
- Calling `str` on a `Version` instance should return the string that results from applying its `edit` to the string represented by `previous`. Assume that string is long enough to perform the `edit`.
- An `edit` object represents a change to a string parameterized by some value `c` applied at an index `i`.
- The `apply` method for `Insert` inserts the string `c` into string `t` starting at index `i`.
- The `apply` method for `Delete` removes `c` (a positive `int`) characters from string `t` starting at index `i`.

```
class Version:
    """A version of a string after an edit.

    >>> s = Version('No power?', Delete(3, 6))
    >>> print(Version(s, Insert(3, 'class!')))
    No class!
    >>> t = Version('Beary', Insert(4, 'kele'))
    >>> print(t)
    Bearkeley
    >>> print(Version(t, Delete(2, 1)))
    Berkeley
    >>> print(Version(t, Delete(4, 5)))
    Bear
    """
    def __init__(self, previous, edit):
        self.previous, self.edit = previous, edit
    def __str__(self):


        return self.edit.apply(str(self.previous))

class Edit:
    def __init__(self, i, c):
        self.i, self.c = i, c

class Insert(Edit):
    def apply(self, t):
        "Return a new string by inserting string c into t starting at position i."


        return t[:self.i] + self.c + t[self.i:]

class Delete(Edit):
    def apply(self, t):
        "Return a new string by deleting c characters from t starting at position i."


        return t[:self.i] + t[self.i + self.c:]
```

8. (4 points) Their Mascot is a Tree?!?

Definitions. The *depth* of the root node of a tree is 0, and the depth of a child is 1 more than the depth of its parent. A linked list is either a `Link` instance or `Link.empty`.

Implement `layer`, which takes a `Tree` instance `t` and a non-negative integer `d`. It returns a linked list containing the labels of all nodes in `t` that have depth `d`. Labels in the result should appear in branch order. The `Tree` and `Link` classes appear on the Midterm 2 Study Guide.

You may **not** use `lambda`, `if`, `and`, or `or` in your solution (other than what already appears in the template).

Note: The built-in `reversed` function returns a reverse iterator over the values of a sequence. For example, `list(reversed([1, 2, 3]))` evaluates to `[3, 2, 1]`.

```
def layer(t, d):
    """Return a linked list containing all labels of Tree T at depth D.

    >>> a_tree = Tree(1, [Tree('b', [Tree('mas')]),
    ...                   Tree('a', [Tree('co')]),
    ...                   Tree('d', [Tree('t', [Tree('!')])])])
    >>> print(layer(a_tree, 0))
    <1>
    >>> print(layer(a_tree, 1))
    <b a d>
    >>> print(layer(a_tree, 2))
    <mas co t>
    >>> print(layer(a_tree, 3))
    <!>
    """
    return helper(t, d, Link.empty)


def helper(t, d, s):


    if d == 0:


        return Link(t.label, s)


    else:


        for b in reversed(t.branches):


            s = helper(b, d - 1, s)


        return s
```

CS 61A Fall 2020

Structure and Interpretation of Computer Programs

Midterm 1 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

1. (14.0 points) Down for the Count

Definition. A *digit* is a non-negative integer less than 10. Integers contain digits.

Examples.

- The integer 21 contains the digits 1 and 2.
- The integer 474 contains the digit 4 twice and the digit 7 once.
- The integer 400 contains the digit 4 once and the digit 0 twice.
- The integer -77 contains the digit 7 twice.
- The integer 0 is a 0-digit number that contains no digits.

Reminders.

- You may call built-in functions that do not require import, such as `min`, `max`, `abs`, and `pow`.
- You may call functions defined in earlier parts of the question in your implementation for later parts, and you may assume that the functions you call are implemented correctly.

RESTRICTION. You may not call `str` or `repr` or use `[` or `]` in any part of this question.

(a) (4.0 points)

Implement `count`, which takes a digit `element` and an integer `box`. It returns the number of times that `element` appears in `box`.

Warning: `n % d` and `n // d` may not behave as you expect for negative `n`. For example, `-123 % 10` evaluates to 7. `-1 // 10` evaluates to -1. You do not need to know how these operators apply to negative `n` in order to solve this problem.

```
def count(element, box):
    """Count how many times digit element appears in integer box.

    >>> count(2, 222122)
    5
    >>> count(0, -2020)
    2
    >>> count(0, 0)  # 0 has no digits
    0
    """
    assert element >= 0 and element < 10


    _________
       (a)

    total = 0

    while box > 0:

        if _________:
              (b)

            total = _________
                       (c)

        box = box // 10

    return total
```

i. (2.0 pt) Fill in blank (a).

```
box = abs(box)
```

ii. (1.0 pt) Which of these could fill in blank (b)?

- ○ `box == element`
- ● `box % 10 == element`
- ○ `box % element == 0`
- ○ `box % element > 0`

iii. (1.0 pt) Which of these could fill in blank (c)?

- ● `total + 1`
- ○ `element`
- ○ `total + element`
- ○ `box % 10`
- ○ `total + box % 10`

(b) (5.0 points)

Implement `count_nine`, which takes a digit `element` and a non-negative integer `box`. It returns the number of times that `element` appears in `box` and is not adjacent to a 9.

```
def count_nine(element, box):
    """Count how many times digit element appears in the non-negative integer
    box in a place that is not next to a 9.

    >>> count_nine(2, 222122)
    5
    >>> count_nine(1, 1911191) # Only the middle 1 is not next to a 9
    1
    >>> count_nine(9, 9)
    1
    >>> count_nine(9, 99)
    0
    >>> count_nine(3, 314159265359)
    2
    >>> count_nine(5, 314159265359)
    1
    >>> count_nine(9, 314159265359)
    2
    >>> count_nine(0, 0) # No digits are in 0
    0
    """
    assert element >= 0 and element < 10
    assert box >= 0

    nine, total = False, 0

    while box > 0:

        if _________ and not (nine or _________):
              (a)                       (b)

            total = _________
                      (c)

        nine = _________ == 9
                  (d)

        box = box // 10

    return total
```

i. (1.0 pt) Which of these could fill in blank (a)?

- ○ `box == element`
- ● `box % 10 == element`
- ○ `box % element == 0`
- ○ `box % element > 0`

ii. **(2.0 pt)** Fill in blank (b).

(box // 10) % 10 == 9

iii. **(1.0 pt)** Which of these could fill in blank (c)?

- (●) `total + 1`
- ( ) `element`
- ( ) `total + element`
- ( ) `box % 10`
- ( ) `total + box % 10`

iv. **(1.0 pt)** Fill in blank (d).

box % 10

(c) (5.0 points)

Implement `fit`, which takes two non-negative integers `pegs` and `holes`. It returns whether every digit in `pegs` appears at least as many times in `holes` as it does in `pegs`.

```
def fit(pegs, holes):
    """Return whether every digit in pegs appears at least as many times in
    holes as it does in pegs.

    >>> fit(123, 321)    # Each digit appears once in pegs and in holes.
    True
    >>> fit(1213, 33221) # 1 appears twice in pegs, but only once in holes.
    False
    >>> fit(12, 22)      # 1 appears once in pegs, but not at all in holes.
    False
    >>> fit(314159, 112233456789)
    True
    """
    i = 0

    while i <= _________:
                  (a)

        if _________:
              (b)

            _________
               (c)

        i = i + 1

    return _________
              (d)
```

i. (1.0 pt) Fill in blank (a).

9

ii. (2.0 pt) Fill in blank (b).

count(i, pegs) > count(i, holes)

iii. (1.0 pt) Fill in blank (c).

return False

iv. (1.0 pt) Which of these could fill in blank (d)?

- (●) `True`
- ( ) `False`
- ( ) `holes > pegs`
- ( ) `pegs > holes`
- ( ) `holes >= pegs`
- ( ) `pegs >= holes`

2. (8.0 points) Mystery Function

Assume `mystery` is a deterministic pure function that takes one integer argument, returns an integer, and never errors.

```
def mystery(n):
    ...
```

Assume the following functions are also defined:

```
def add_two(y):
    return y + 2

def two(y):
    return 2

def constant(k):
    def ignore(x):
        return k
    return ignore

def diff(f, g):
    return lambda z: abs(f(z) - g(z))
```

Definition. Two functions `f` and `g` have identical behavior if `f(x)` and `g(x)` return equal values or return functions with identical behavior, for every `x` that does not cause an error.

Complete each statement below so that it is true for all possible deterministic pure `mystery` functions.

(a) (2.0 pt) The result of evaluating `constant(2)` has identical behavior to the result of evaluating the expression...

- ◯ ... `add_two`
- ◯ ... `add_two(0)`
- ◯ ... `add_two(2)`
- ● ... `two`
- ◯ ... `two(0)`
- ◯ ... `two(2)`
- ◯ None of these

(b) (2.0 pt) The result of evaluating `diff(constant(1), constant(-1))` has identical behavior to the result of evaluating the expression...

- ◯ ... `constant`
- ◯ ... `constant(0)`
- ● ... `constant(2)`
- ◯ ... `diff(constant, constant)`
- ◯ None of these

(c) (2.0 pt) The result of evaluating `diff(mystery, mystery)` has identical behavior to the result of evaluating the expression...

- ◯ ... `constant`
- ● ... `constant(0)`
- ◯ ... `constant(2)`
- ◯ ... `diff(constant, constant)`
- ◯ ... `constant(mystery)`
- ◯ ... `mystery`
- ◯ None of these

(d) (2.0 pt) The result of evaluating `diff(mystery, diff(mystery, mystery))` has identical behavior to the result of evaluating the expression...

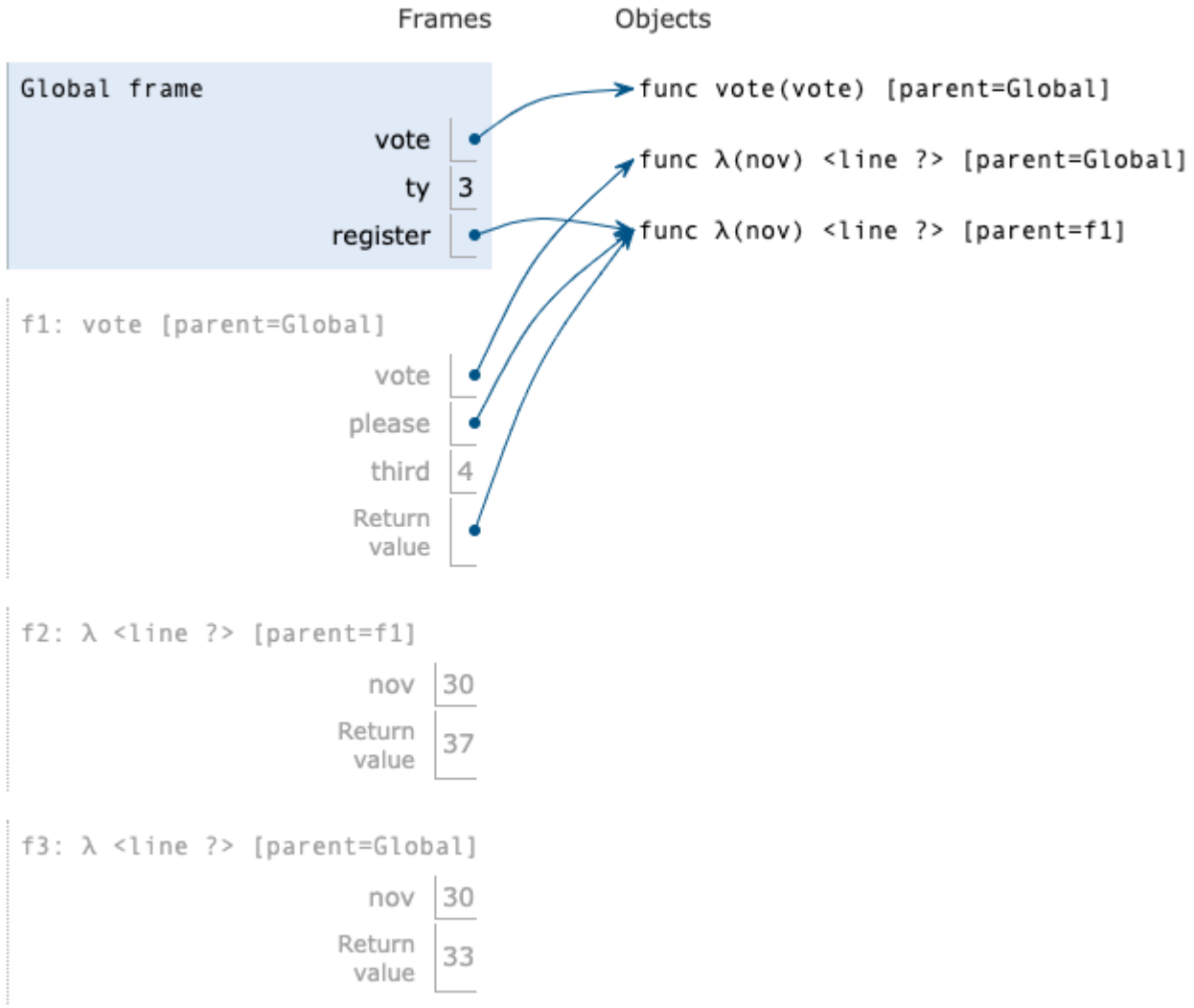
- ◯ ... `mystery`
- ◯ ... `abs(mystery)`
- ● ... `lambda y: abs(mystery(y))`
- ◯ ... `lambda y: mystery(abs(y))`
- ◯ ... `lambda y: lambda z: mystery(abs(y)) - mystery(abs(z))`
- ◯ ... `lambda y: lambda z: abs(mystery(y)) - abs(mystery(z))`
- ◯ ... `lambda y: lambda z: abs(mystery(y) - mystery(z))`
- ◯ None of these

3. (8.0 points) Please Register to Vote

Fill in each blank in the code example below so that its environment diagram is the following.

RESTRICTIONS. You must use all of the blanks. Each blank can only include one statement or expression.

Click here to open the diagram in a new window




```
def vote(vote):

    please = _________
                (a)

    _________ = ty + 3
       (b)

    return please

ty = 1

register = _________(lambda nov: nov + ty)
                (c)
```

```
---------
   (d)

register(_________)
            (e)
```

(a) (2.0 pt) Which of these could fill in blank (a)?

- ○ `vote(ty)`
- ○ `vote(30)`
- ○ `vote`
- ● `lambda nov: vote(nov) + third`
- ○ `lambda nov: vote(nov + third)`
- ○ `lambda nov: vote(nov) + ty`
- ○ `lambda nov: vote(nov + ty)`

(b) (1.0 pt) Which of these could fill in blank (b)?

- ● `third`
- ○ `ty`
- ○ `please`
- ○ `vote`

(c) (1.0 pt) Which of these could fill in blank (c)?

- ○ `third`
- ○ `ty`
- ○ `please`
- ● `vote`

(d) (2.0 pt) Fill in blank (d).

ty = 3

(e) (2.0 pt) Which of these could fill blank (e)? **Check all that apply.**

- ■ `ty * 10`
- ☐ `ty - 1 + 30`
- ■ `30`
- ☐ `third + 26`
- ☐ `(lambda x: x + x)(15)`

4. (10.0 points) Amazing Job Growth

Definition. A *repeatable* function is a function that returns a repeatable function.

Reminder. You may call built-in functions that do not require import, such as `min`, `max`, `abs`, and `pow`.

(a) (4.0 points)

Implement `growth`, which takes a number `baseline` and returns a repeatable function `increase`. When `increase` is called on a number `observed`, it prints the difference between `observed` and the smallest argument passed to `growth` or `increase` so far among the repeated calls.

```
def growth(baseline):
    """Return a function that can be called repeatedly on numbers and prints
    the difference between its argument and the smallest argument used so far
    (including baseline).

    >>> job = growth(148)(149)(150)(130)(133)(139)(137)
    1
    2
    0
    3
    9
    7
    """
    def increase(observed):

        under = _________
                  (a)

        print(observed - under)

        return _________
                  (b)

    return increase
```

i. (2.0 pt) Fill in blank (a).

min(observed, baseline)

ii. (2.0 pt) Which of these could fill in blank (b)?

- ○ `increase`
- ○ `increase(under)`
- ○ `increase(observed)`
- ○ `increase(baseline)`
- ○ `growth`
- ● `growth(under)`
- ○ `growth(observed)`
- ○ `growth(baseline)`

(b) (6.0 points)

Implement `maxer`, a higher-order function that takes a function `smoke`, which takes a number and returns a number. The `maxer` function returns a repeatable function `fire` that takes a number `y` and prints the largest result of calling `smoke` on any value of `y` passed to `fire` so far among the repeated calls.

Assume that `smoke` is a deterministic pure function.

```
def square(x):
    return x * x

def maxer(smoke):
    """Return a repeatable function fire(y) that prints the largest smoke(y) so far.

    >>> g = maxer(square)
    >>> h = g(2)(1)(3)(2)(-4)  # print the largest square(y) so far
    4
    4
    9
    9
    16
    >>> h = maxer(abs)(2)(1)(3)(2)(-4)  # print the largest abs(y) so far
    2
    2
    3
    3
    4
    """

    def fire(y):

        _________
           (a)

        def haze(z):

            if _________:
                  (b)

                z = y

            return _________
                      (c)

        return haze

    return fire
```

i. (2.0 pt) Fill in blank (a). You may not write a `return` statement for this blank.

print(smoke(y))

ii. (2.0 pt) Fill in blank (b).

smoke(y) > smoke(z)

iii. (2.0 pt) Which of these could fill in blank (c)?

- ○ `y`
- ○ `smoke(y)`
- ○ `fire(y)`
- ○ `fire(smoke(y))`
- ○ `haze`
- ○ `haze(y)`
- ○ `haze(smoke(y))`
- ○ `z`
- ○ `smoke(z)`
- ● `fire(z)`
- ○ `fire(smoke(z))`
- ○ `haze(z)`
- ○ `haze(smoke(z))`

No more questions.

CS 61A Fall 2020

Structure and Interpretation of Computer Programs

Midterm 2 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

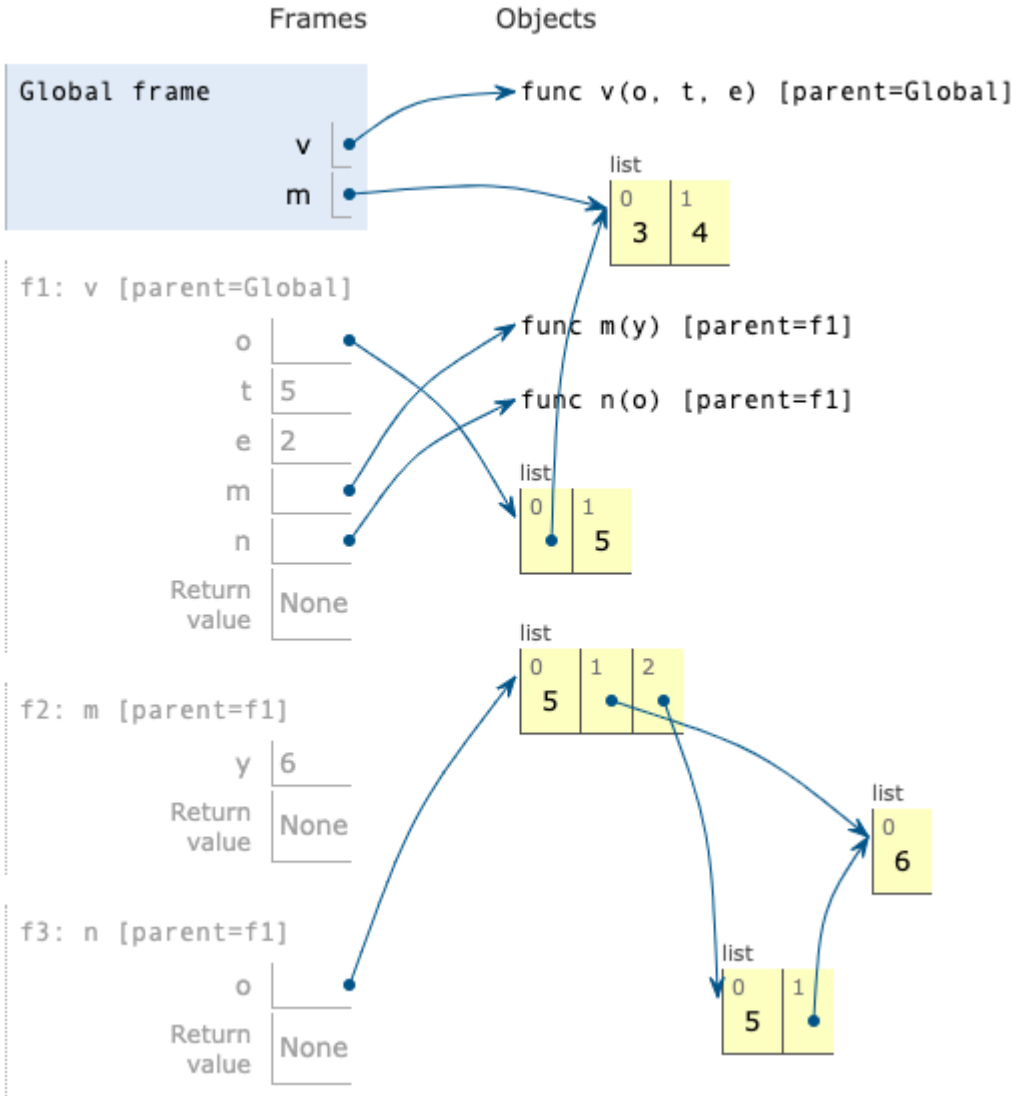
(b) What is your student ID number?

1. (8.0 points) Political Environment

Fill in each blank in the code example below so that executing it would generate the following environment diagram.

RESTRICTIONS. You must use all of the blanks. Each blank can only include one statement or expression.

Click here to open the diagram in a new window/tab




```
def v(o, t, e):
    def m(y):

        ---------
           (a)

        ---------
           (b)

    def n(o):

        o.append(_________)
```

```
                    (c)

        o.append(_________)
                    (d)

    m(e)
    n([t])
    e = 2

m = [3, 4]

v(m, 5, 6)
```

(a) **(2.0 pt)** Fill in blank (a). **You may not write any numbers or arithmetic operators (+, -, *, /, //, **) in your solution.**

nonlocal o

(b) **(2.0 pt)** Fill in blank (b). **You may not write any numbers or arithmetic operators (+, -, *, /, //, **) in your solution.**

o = [o, t]

(c) **(2.0 pt)** Fill in blank (c). **You may not write any numbers or arithmetic operators (+, -, *, /, //, **) in your solution.**

[e]

(d) **(2.0 pt)** Which of these could fill in blank (d)? **Check all that apply.**

- ☐ `o`
- ☐ `[o]`
- ■ `list(o)`
- ☐ `[list(o)]`
- ☐ `list([o])`
- ■ `o + []`
- ■ `[o[0], o[1]]`
- ■ `o[:]`

2. (10.0 points) Yield, Fibonacci!

(a) (4.0 points)

Implement `fibs`, a generator function that takes a one-argument pure function `f` and yields all Fibonacci numbers `x` for which `f(x)` returns a true value.

The Fibonacci numbers begin with 0 and then 1. Each subsequent Fibonacci number is the sum of the previous two. Yield the Fibonacci numbers in order.

```
def fibs(f):
    """Yield all Fibonacci numbers x for which f(x) is a true value.

    >>> odds = fibs(lambda x: x % 2 == 1)
    >>> [next(odds) for i in range(10)]
    [1, 1, 3, 5, 13, 21, 55, 89, 233, 377]
    >>> bigs = fibs(lambda x: x > 20)
    >>> [next(bigs) for i in range(10)]
    [21, 34, 55, 89, 144, 233, 377, 610, 987, 1597]
    >>> evens = fibs(lambda x: x % 2 == 0)
    >>> [next(evens) for i in range(10)]
    [0, 2, 8, 34, 144, 610, 2584, 10946, 46368, 196418]
    """

    n, m = 0, 1

    while _________:
             (a)

        if _________:
              (b)

            _________
               (c)

        _________
           (d)
```

i. (1.0 pt) Which of these could fill in blank (a)?

- ○ `f(n)`
- ○ `f(m)`
- ○ `f(n) or f(m)`
- ○ `f(n) and f(m)`
- ● `True`
- ○ `False`

ii. **(1.0 pt)** Which of these could fill in blank (b)?

- (●) `f(n)`
- ( ) `f(m)`
- ( ) `f(n) or f(m)`
- ( ) `f(n) and f(m)`
- ( ) `True`
- ( ) `False`

iii. **(1.0 pt)** Fill in blank (c).

yield n

iv. **(1.0 pt)** Fill in blank (d).

n, m = m, n + m

(b) (6.0 points)

Definition. For a linked list `s`, the *index* of an element is the number of times `rest` appears in the smallest dot expression containing only `s`, `rest`, and `first` that evaluates to that element. For example, in the linked list `s = Link(5, Link(7, Link(9, Link(11))))`,

- The index of 5 (`s.first`) is 0.
- The index of 7 (`s.rest.first`) is 1.
- The index of 11 (`s.rest.rest.rest.first`) is 3.

Implement `filter_index`, a function that takes a one-argument pure function `f` and a `Link` instance `s`. It returns a `Link` containing all elements of `s` that have an **index** `i` for which `f(i)` returns a true value.

Assume that `s` is a finite linked list of numbers that contains no repeated elements. The `Link` class appears on Page 2 (left column) of the Midterm 2 Study Guide.

```
def filter_index(f, s):
    """Return a Link containing the elements of Link s that have an index i for
    which f(i) is a true value.

    >>> powers = Link(1, Link(2, Link(4, Link(8, Link(16, Link(32))))))
    >>> filter_index(lambda x: x < 4, powers)
    Link(1, Link(2, Link(4, Link(8))))
    >>> filter_index(lambda x: x % 2 == 1, powers)
    Link(2, Link(8, Link(32)))
    """

    def helper(i, s):

        if s is Link.empty:

            return s

        filtered_rest = _________
                            (a)
        if _________:
              (b)

            return _________
                       (c)

        else:

            return filtered_rest

    return _________
              (d)
```

i. **(1.0 pt)** Which of these could fill in blank (a)?

- (●) `helper(i + 1, s.rest)`
- ( ) `helper(i + 1, s.rest.rest)`
- ( ) `filter_index(f, s.rest)`
- ( ) `filter_index(f, s.rest.rest)`
- ( ) `Link(helper(i + 1, s.rest))`
- ( ) `Link(helper(i + 1, s.rest.rest))`
- ( ) `Link(filter_index(f, s.rest))`
- ( ) `Link(filter_index(f, s.rest.rest))`

ii. **(1.0 pt)** Fill in blank (b).

f(i)

iii. **(2.0 pt)** Fill in blank (c).

Link(s.first, filtered_rest)

iv. **(2.0 pt)** Fill in blank (d).

helper(0, s)

3. (12.0 points) Sparse Lists

The `most_common` function returns the most common element in a non-empty list. You do not need to implement this function. Assume that it is implemented for you.

```
def most_common(s):
    """Return the most common element in non-empty list s. In case of a tie,
    return the most common element that appears first in s.

    >>> most_common([3, 1, 4, 1, 5, 9])
    1
    >>> most_common([3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5])
    5
    >>> most_common([2, 7, 1, 8, 2, 8, 1, 8, 2, 8])
    8
    >>> most_common([3, 5, 7, 7, 7, 5, 5])
    5
    >>> most_common([3, 7, 5, 5, 7, 7])
    7
    """
```

Implement the `SparseList` class. A `SparseList` instance represents a non-empty list `s`.

- Its `common` attribute is the most common value in `s`.
- Its `others` dictionary has a value for every element in `s` that is not `common`. The corresponding key is the **index** for that value in `s`.
- Its `item` method takes a non-negative integer `i` and returns `s[i]` or the string `'out of range'` if `i` is not smaller than the length of `s`.
- Its `items` method returns a list with the same elements as `s` in the same order as `s`.

```
class SparseList:
    """Represent a non-empty list as a most common value and a dictionary from
    indices to values that contains only values that are not the most common.

    >>> pi = SparseList([3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5])
    >>> pi.common
    5
    >>> pi.others
    {0: 3, 1: 1, 2: 4, 3: 1, 5: 9, 6: 2, 7: 6, 9: 3}
    >>> [pi.item(0), pi.item(1), pi.item(2), pi.item(3), pi.item(4)]
    [3, 1, 4, 1, 5]
    >>> pi.item(10)
    5
    >>> pi.item(11)
    'out of range'
    >>> pi.items()
    [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
    """
    def __init__(self, s):
        ...
    def item(self, i):
        ...
    def items(self):
        ...
```

(a) (5.0 points)

Implement the `__init__` method, which takes a list `s`.

```
def __init__(self, s):

    assert s, 's cannot be empty'

    self.n = len(s)

    self.common = most_common(_________)
                              (a)

    self.others = { _________: _________ for i in range(_________) if _________ }
                    (b)        (c)                      (d)           (e)
```

i. **(1.0 pt)** Fill in blank (a).

> **s**

ii. **(1.0 pt)** Which of these could fill in blank (b)?

- (●) `i`
- ( ) `self.i`
- ( ) `n`
- ( ) `self.n`
- ( ) `s[i]`
- ( ) `s`

iii. **(1.0 pt)** Fill in blank (c).

> **s[i]**

iv. **(1.0 pt)** Which of these could fill in blank (d)? **Check all that apply.**

- [ ] `s`
- [■] `len(s)`
- [ ] `self.s`
- [ ] `len(self.s)`
- [ ] `n`
- [ ] `len(n)`
- [■] `self.n`
- [ ] `len(self.n)`

v. **(1.0 pt)** Fill in blank (e).

> **s[i] != self.common**

(b) (3.0 points)

Implement the `item` method, which takes a non-negative integer `i`.

```
def item(self, i):
    """Return s[i] or 'out of range' if i is not smaller than the length of s."""

    assert i >= 0, 'index i must be non-negative'

    if _________:
          (a)

        return 'out of range'

    elif _________:
            (b)

        return _________
                  (c)

    else:

        return self.common
```

i. (1.0 pt) Fill in blank (a).

i >= self.n

ii. (1.0 pt) Fill in blank (b).

i in self.others

iii. (1.0 pt) Which of these could fill in blank (c)?

- ◯ `others[i]`
- ● `self.others[i]`
- ◯ `others[self.i]`
- ◯ `self.others[self.i]`
- ◯ `others`
- ◯ `self.others`

(c) (4.0 points)

Implement the `items` method.

```
def items(self):

    """Return a list with the same elements as s in the same order as s."""

    return [_________ for i in _________]
               (a)                (b)
```

i. (2.0 pt) Fill in blank (a). You may not use `and`, `or`, `if`, `else`, `[`, `]`, or `get`.

Hint: Don't repeat yourself.

self.item(i)

ii. (2.0 pt) Fill in blank (b).

range(self.n)

4. (20.0 points) Fork It

The *tree* data abstraction, which is implemented by the constructor `tree`, selectors `branches` and `label`, and helper functions `is_leaf` and `is_tree` appear on Page 2 (left column) of the Midterm 2 Study Guide. You may call these functions. Do not violate the abstraction barriers of the tree data abstraction.

(a) (4.0 points)

Implement `max_path`, which takes a tree `t` whose labels are **all positive** numbers and returns the largest sum of the labels along a path from the root of `t` to one of its leaves.

You may call `tree`, `label`, `branches`, `is_leaf`, `is_tree`, and `max_path`.

```
def max_path(t):
    """Return the largest sum of labels along any path from the root to a leaf
    of tree t, which has positive numbers as labels.

    >>> a = tree(1, [tree(2), tree(3), tree(4, [tree(5)])])
    >>> max_path(a) # 1 + 4 + 5
    10
    >>> b = tree(6, [a, a, a])
    >>> max_path(b) # 6 + 1 + 4 + 5
    16
    """

    return _________ + max(_________ + _________)
              (a)              (b)          (c)
```

i. (1.0 pt) Which of the following could fill in blank (a)?

- ○ `t`
- ● `label(t)`
- ○ `[t]`
- ○ `[label(t)]`
- ○ `[0]`
- ○ `sum([b for b in branches(t)])`
- ○ `sum([label(b) for b in branches(t)])`

ii. (1.0 pt) Which of the following could fill in blank (b)?

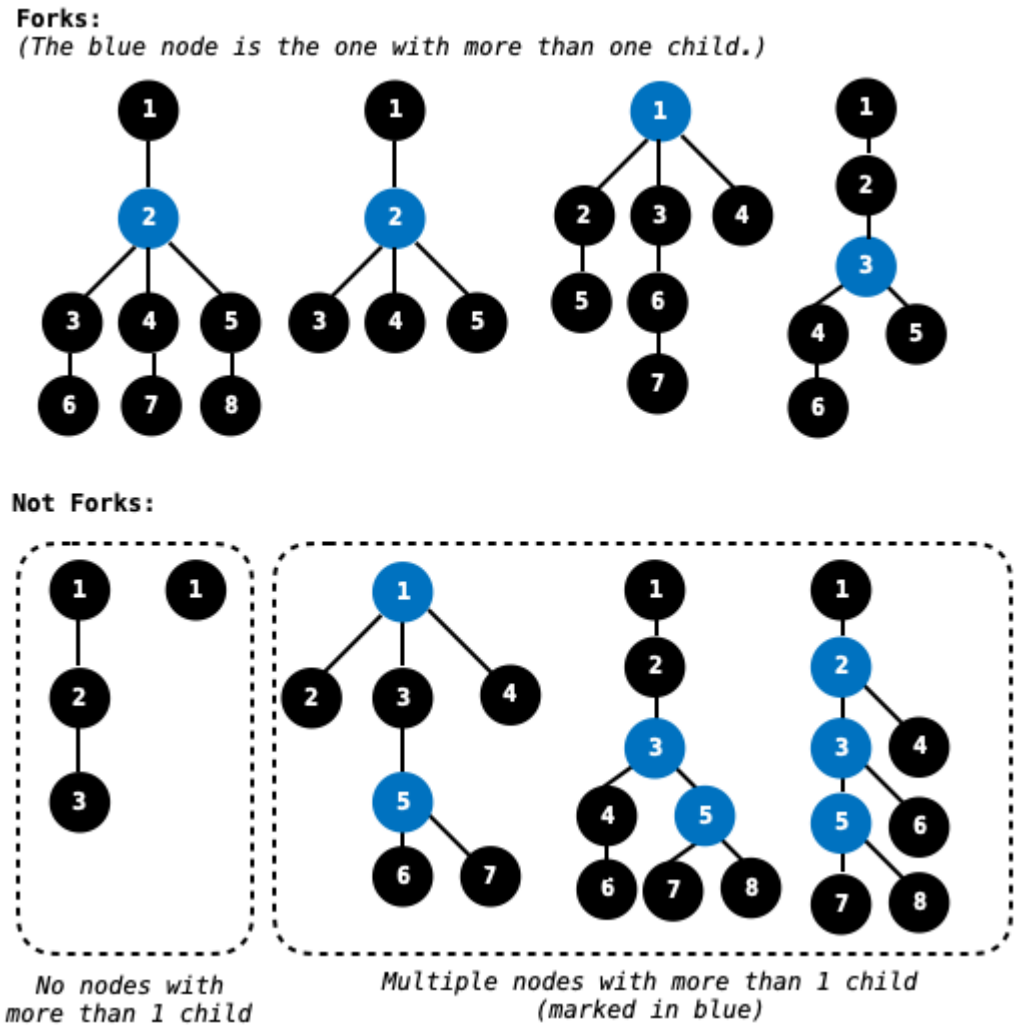
- ○ `t`
- ○ `label(t)`
- ○ `[t]`
- ○ `[label(t)]`
- ● `[0]`
- ○ `[label(b) for b in branches(t)]`

iii. (2.0 pt) Fill in blank (c). You may not use the word `default`.

[max_path(b) for b in branches(t)]

(b) (8.0 points)

Definition. A *fork* is a tree in which **exactly one node** has more than one child.



Implement `is_fork` and its helper function `slide`. The `is_fork` function takes a *tree* `t` and returns `True` if `t` is a fork and `False` otherwise.

You may call `tree`, `label`, `branches`, `is_leaf`, `is_tree`, `max_path`, `is_fork`, and `slide`.

```
def is_fork(t):
    """Return whether tree t is a fork.

    >>> is_fork(tree(1, [tree(2, [tree(3), tree(4), tree(5)])]))
    True
    >>> is_fork(tree(1, [tree(2, [tree(3)]), tree(4)]))
    True
    >>> is_fork(tree(1, [tree(2), tree(3), tree(4)]))
    True
    >>> is_fork(tree(1, [tree(2, [tree(3, [tree(5)]), tree(4, [tree(6)])])]))
    True
    >>> is_fork(tree(1))
    False
    >>> is_fork(tree(1, [tree(2, [tree(3)])]))
    False
```

```
    >>> is_fork(tree(1, [tree(2, [tree(3)]), tree(4, [tree(5), tree(6)])]))
    False
    >>> is_fork(tree(1, [tree(2, [tree(3, [tree(5, [tree(7), tree(8)]), tree(6)]), tree(4)])]))
    False
    """

    neck = slide(t)

    if is_leaf(neck):

        _________
           (a)

    return _________([_________ for b in branches(_________)])
              (b)        (c)                          (d)

def slide(t):
    """Return the deepest node within tree t whose ancestors all have exactly one child.

    Definition: The ancestors of a node include its parent and the parents of all its ancestors.

    >>> deepest = slide(tree(1, [tree(2, [tree(3)])]))
    >>> label(deepest)
    3
    >>> label(slide(tree(1, [tree(2, [tree(3), tree(4)])])))
    2
    """

    while _________:
             (e)

        t = _________
               (f)

    return t
```

i. **(1.0 pt)** Fill in blank (a).

return False

ii. **(1.0 pt)** Which of the following could fill in blank (b)?

- ● `all`
- ○ `any`
- ○ `list`
- ○ `tree`
- ○ `branches`
- ○ `label`

iii. **(2.0 pt)** Fill in blank (c).

is_leaf(slide(b))

iv. **(1.0 pt)** Which of the following could fill in blank (d)?

- ○ `t`
- ○ `tree`
- ● `neck`
- ○ `label(t)`
- ○ `label(tree)`
- ○ `label(neck)`

v. **(1.0 pt)** Which of the following could fill in blank (e)?

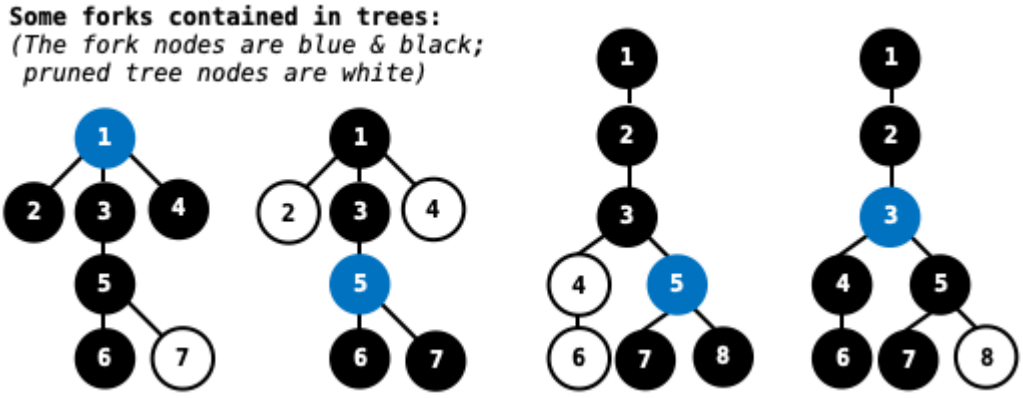
- ● `len(branches(t)) == 1`
- ○ `len(branches(t)) > 1`
- ○ `len(branches(t)) != 1`
- ○ `branches(t) == 1`
- ○ `branches(t) > 1`
- ○ `branches(t) != 1`

vi. **(2.0 pt)** Fill in blank (f).

branches(t)[0]

(c) (8.0 points)

Definition. A tree `t` *contains* a fork `u` if `u` is the result of pruning zero or more nodes from `t`.



Implement `max_fork`, which takes a tree `t` whose labels are all positive integers. It returns the largest sum of the labels of a fork that is contained in `t`. If `t` does not contain any forks, then `max_fork` returns 0.

You may call `tree`, `label`, `branches`, `is_leaf`, `is_tree`, `max_path`, `is_fork`, `slide`, and `max_fork`.

```
def max_fork(t):
    """Return the largest sum of the labels in any fork contained in tree t,
    which has positive numbers as labels. If t contains no forks, return 0.


    >>> a = tree(1, [tree(2), tree(3), tree(4, [tree(5)])])
    >>> max_fork(a) # 1 + 2 + 3 + 4 + 5
    15
    >>> b = tree(6, [a, a, a])
    >>> max_fork(b)  #          6 + (1 + 4 + 5) + (1 + 4 + 5) + (1 + 4 + 5)
    36
    >>> c = tree(7, [tree(8), b, tree(9)])
    >>> max_fork(c)  #     7 + (6 + (1 + 4 + 5) + (1 + 4 + 5) + (1 + 4 + 5))
    43
    >>> d = tree(9, [c])
    >>> max_fork(d)  # 9 + 7 + (6 + (1 + 4 + 5) + (1 + 4 + 5) + (1 + 4 + 5))
    52
    >>> max_fork(tree(1, [tree(2, [tree(3)])])) # No forks here!
    0
    """

    n = len(branches(t))

    if n == 0:

        return 0

    elif n == 1:

        below = _________
                  (a)

        if _________:
              (b)
```

```
            return _________ + below
                       (c)

        else:

            return 0

    else:

        here = sum([_________ for b in branches(t)])
                        (d)

        there = max([_________ for b in branches(t)])
                         (e)

        return label(t) + max(here, there)
```

i. **(2.0 pt)** Fill in blank (a).

max_fork(branches(t)[0])

ii. **(1.0 pt)** Which of the these could fill in blank (b)?

- (●) `below > 0`
- ( ) `label(t) > 0`
- ( ) `max_fork(t) > 0`
- ( ) `True`
- ( ) `max_fork(t) > max_path(t)`
- ( ) `is_fork(t)`

iii. **(1.0 pt)** Which of these could fill in blank (c)?

- ( ) `1`
- (●) `label(t)`
- ( ) `max_path(t)`
- ( ) `label(slide(t))`
- ( ) `max([label(b) for b in branches(t)])`
- ( ) `label(branches(t)[0])`

iv. **(2.0 pt)** Fill in blank (d).

max_path(b)

v. (2.0 pt) Fill in blank (e).

max_fork(b)

No more questions.

CS 61A Fall 2021

Structure and Interpretation of Computer Programs

Midterm 1 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

1. (8.0 points) What Would Python Display?

(a) (4.0 points)

Assume the following code has been executed.

```
def os(ki):
    t = -1

i = 5
t = 7
while i > 3:
    t = i - 4 and i + 4
    os(i - 2)
    i, j = i - 1, i * 2
    s = i
```

i. (1.0 pt) What value is bound to `i` in the global frame?

3

ii. (1.0 pt) What value is bound to `j` in the global frame?

8

iii. (1.0 pt) What value is bound to `s` in the global frame?

3

iv. (1.0 pt) What value is bound to `t` in the global frame?

0

(b) (4.0 points)

The function `tik` takes an argument `tok` and returns a function `insta` that takes an argument `gram`. The `insta` function prints `tok` and `gram` on the same line separated by a space and has no return statement. Its implementation has been omitted intentionally.

```
def tik(tok):
    """Returns a function that takes gram and prints tok and gram on a line.

    >>> tik(5)(6)
    5 6
    """
    def insta(gram):
        ... # The implementation of this function has been omitted.
    return insta
```

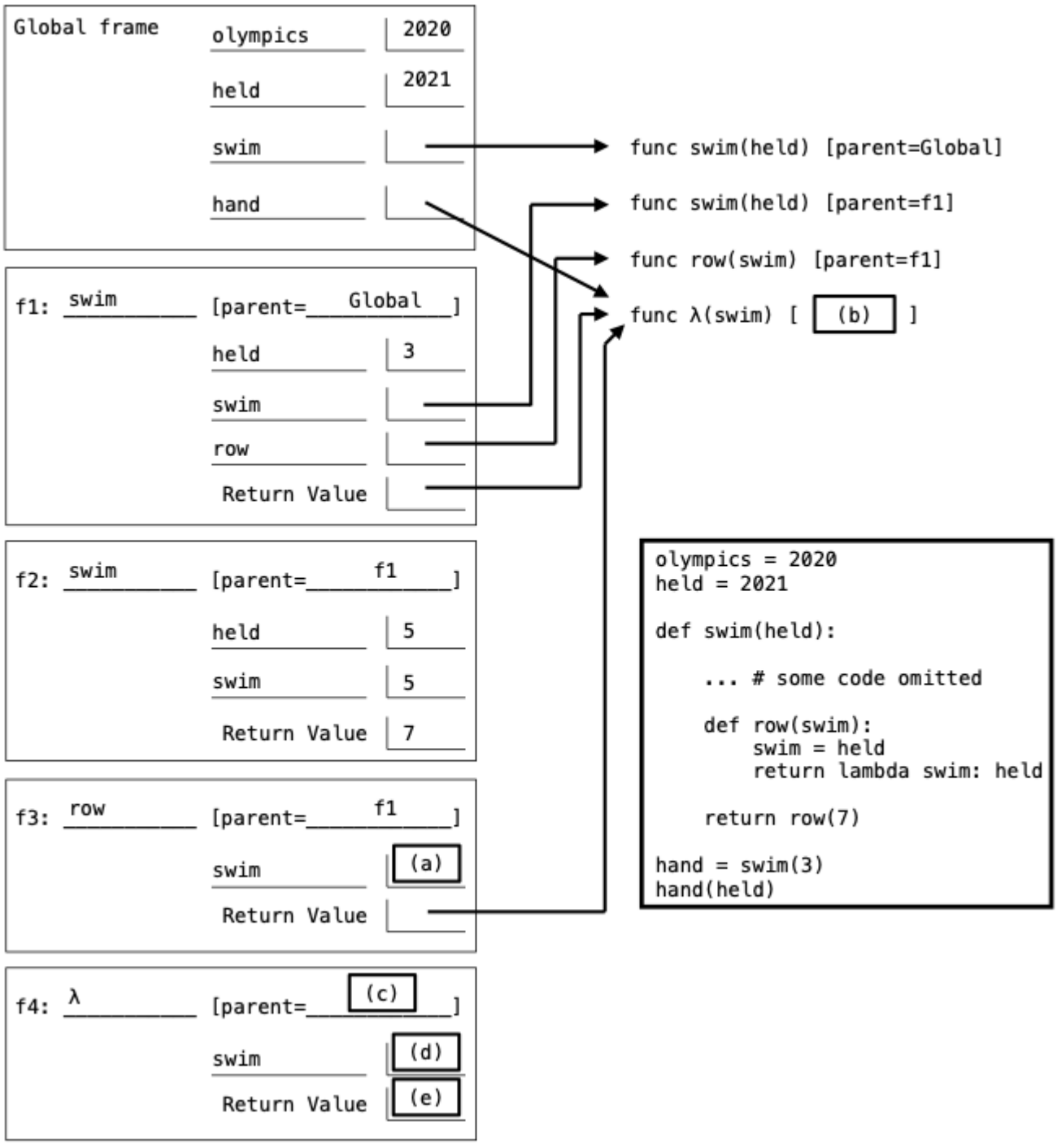
i. (4.0 pt) What would the interactive Python interpreter display upon evaluating the expression:

```
tik(tik(5)(print(6)))(print(7))
```

6
5 None
7
None None

2. (5.0 points) 31 Cal Olympians

The environment diagram below was generated by code that is only partially provided to the right of the diagram. All of the relevant code needed to fill in the blanks in the environment diagram is shown. Line numbers have been omitted intentionally.



(a) (1.0 pt) Which of these could fill in blank (a)?

- [x] `3`
- [ ] `5`
- [ ] `7`
- [ ] `2021`

(b) (1.0 pt) Which of these could fill in blank (b)? (The parent of the function returned in `f3`.)

- [ ] `parent=Global`
- [ ] `parent=f1`
- [ ] `parent=f2`
- [x] `parent=f3`

(c) (1.0 pt) Which of these could fill in blank (c)?

- [ ] `parent=Global`
- [ ] `parent=f1`
- [ ] `parent=f2`
- [x] `parent=f3`

(d) (1.0 pt) Which of these could fill in blank (d)?

- [ ] `3`
- [ ] `5`
- [ ] `7`
- [x] `2021`

(e) (1.0 pt) Which of these could fill in blank (e)?

- [x] `3`
- [ ] `5`
- [ ] `7`
- [ ] `2021`

3. (27.0 points) All Hail the Stone

Definition: A *hailstone sequence* begins with a positive integer `n`. If `n` is even, divide it by 2. If `n` is odd, triple it and add 1. Repeat until 1 is reached. For example, the hailstone sequence starting at 3 is 3, 10, 5, 16, 8, 4, 2, 1. Assume all hailstone sequences are finite and end with 1.

Assume the following code has been executed.

```
from operator import add, mul

def next_hail(k):
    """Return the next element in a hailstone sequence."""
    assert k > 1
    if k % 2 == 0:
        return k // 2
    else:
        return 3 * k + 1
```

(a) (8.0 points)

Implement `hail_min`, which takes a positive integer `n` and a one-argument function `measure`. It returns the element of the hailstone sequence starting with `n` for which calling `measure` on the element returns the smallest value.

If more than one element of the sequence has the smallest `measure` value, return the earliest one.

```
def hail_min(n, measure):
    """Return the element k of the hailstone sequence starting with n for which
    measure(k) is smallest. In case of a tie, return the earliest element.

    >>> hail_min(5, lambda k: -k)          # Among 5, 16, 8, 4, 2, 1; 16 is largest
    16
    >>> hail_min(8, lambda k: -k)          # Among 8, 4, 2, 1; 8 is largest
    8
    >>> hail_min(3, lambda k: abs(k - 7))  # Among 3, 10, 5, 16, 8, 4, 2, 1; 8 is closest to 7
    8
    >>> hail_min(9, lambda k: abs(k - 7))  # Among 9, 28, 14, 7, 22, ...; 7 is closest to 7
    7
    >>> hail_min(8, lambda k: abs(k - 3))  # 4 and 2 are both close to 3, but 4 is earliest
    4
    """

    apple = _________
              (a)

    while n > 1:

        n = next_hail(n)

        if _________:
              (b)

            _________
              (c)

    return _________
              (d)
```

i. **(1.0 pt)** Fill in blank (a).

n

ii. **(3.0 pt)** Fill in blank (b).

measure(n) < measure(apple)

iii. **(1.0 pt)** Fill in blank (c).

apple = n

iv. **(1.0 pt)** Which of these could fill in blank (d)? Check all that apply.

- ☐ `n`
- ■ `apple`
- ☐ `measure(n)`
- ☐ `measure(apple)`
- ☐ `min(n, apple)`
- ☐ `min(measure(n), measure(apple))`

v. **(2.0 pt)** What element of the hailstone sequence starting with `n` larger than 1 is returned by the expression:

```
hail_min(n, lambda k: 1 - k % 2)
```

- ◯ Always `n` (the first element)
- ◯ Always 1 (the last element)
- ◯ The largest odd element
- ◯ The largest even element
- ◯ The smallest odd element
- ◯ The smallest even element
- ● The first odd element
- ◯ The first even element

(b) (6.0 points)

Definition: An *accumulator function* is a function that takes two integers and returns an integer. It can serve as the second argument to `hail_tally` below.

```
def hail_tally(n, f):
    """Accumulate the elements of the hailstone sequence starting with n using
    accumulator function f.

    >>> hail_tally(3, add)   # 3 + 10 + 5 + 16 + 8 + 4 + 2 + 1 = 49
    49
    >>> hail_tally(10, max)  # Largest of 10, 5, 16, 8, 4, 2, 1
    16
    """
    total = 0
    while n > 1:
        total = f(total, n)
        n = next_hail(n)
    return f(total, 1)
```

Implement `sum_some`, which takes a one-argument function `select` and returns an *accumulator function* `f`. For positive integer `n`, the call `hail_tally(n, f)` returns the sum of the elements of the hailstone sequence starting with `n` for which calling `select` on the element returns a true value.

```
def sum_some(select):
    """Return an accumulator function that sums all k for which select(k) is a true value.

    >>> def below_ten(k):
    ...     return k < 10
    >>> sum_below_ten = sum_some(below_ten)
    >>> hail_tally(3, sum_below_ten)  # [3] + 10 + [5] + 16 + [8] + [4] + [2] + [1]
    23
    """
    def f(total, k):

        if _________:
              (a)

            return _________(total, k)
                       (b)

        return _________
                  (c)


    _________
       (d)
```

i. (2.0 pt) Fill in blank (a).

select(k)

ii. **(1.0 pt)** Which of these could fill in blank (b)? Check all that apply.

- [x] `add`
- [ ] `mul`
- [ ] `sum_some`
- [ ] `total`
- [ ] `f`
- [ ] `hail_tally`

iii. **(1.0 pt)** Which of these could fill in blank (c)?

- ○ `k`
- ● `total`
- ○ `total + k`
- ○ `hail_tally(k, sum_some)`
- ○ `hail_tally(total, sum_some)`
- ○ `hail_tally(total + k, sum_some)`

iv. **(2.0 pt)** Fill in blank (d).

```
return f
```

(c) (5.0 points)

Implement `hail_odd_sum`, a function that takes a positive integer `n` and returns the sum of all odd elements in the hailstone sequences starting with `n`.

Important: Your solution **must** include `sum_some`.

You may also call other functions defined previously in this question.

```
def hail_odd_sum(n):
    """Sum the odd elements of the hailstone sequence starting with n.

    >>> hail_odd_sum(3)   # [3], 10, [5], 16, 8, 4, [1]; 3 + 5 + 1 = 9
    9
    >>> hail_odd_sum(34)  # 34, [17], 52, 26, [13], 40, 20, 10, [5], ..., [1]
    36
    """
    return _________(_________, _________)
              (a)        (b)         (c)
```

i. (1.0 pt) Fill in blank (a).

hail_tally

ii. (1.0 pt) Fill in blank (b).

n

iii. (3.0 pt) Fill in blank (c).

sum_some(lambda k: k % 2 == 1)

(d) (8.0 points)

Definition: To call a function `f` *repeatedly* on a sequence of values `x`, `y`, `z` means to use `f` in a nested call expression in which each element of the sequence is passed in as a single argument in order: `f(x)(y)(z)`.

Implement `hail`, which takes an integer `n` greater than 1. It returns a function that returns `True` when called repeatedly on all of the remaining elements of the hailstone sequence starting with `n`. It returns `False` when called repeatedly on any sequence that differs from the hailstone sequence starting with `n`.

Hint: You may call `next_hail` (and other functions defined previously in this question).

```
def hail(n):
    """Return a function that returns True if called repeatedly on the remaining
    elements of the hailstone sequence starting with n and False otherwise.

    >>> hail(3)(10)(5)(16)(8)(4)(2)(1)
    True
    >>> hail(3)(4)                  # The next element should have been 10.
    False
    >>> hail(3)(10)(5)(16)(8)(1)    # The next element should have been 4.
    False
    """

    assert n > 1

    def check(k):

        if _________:
              (a)

            return True

        if _________:
              (b)

            return False

        return _________
                  (c)

    return _________
              (d)
```

i. (2.0 pt) Fill in blank (a).

n == 2 and k == 1

ii. (2.0 pt) Fill in blank (b).

next_hail(n) != k

iii. (2.0 pt) Fill in blank (c).

hail(k)

iv. (2.0 pt) Fill in blank (d).

check

No more questions.

CS 61A Fall 2021

Structure and Interpretation of Computer Programs

Midterm 2 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

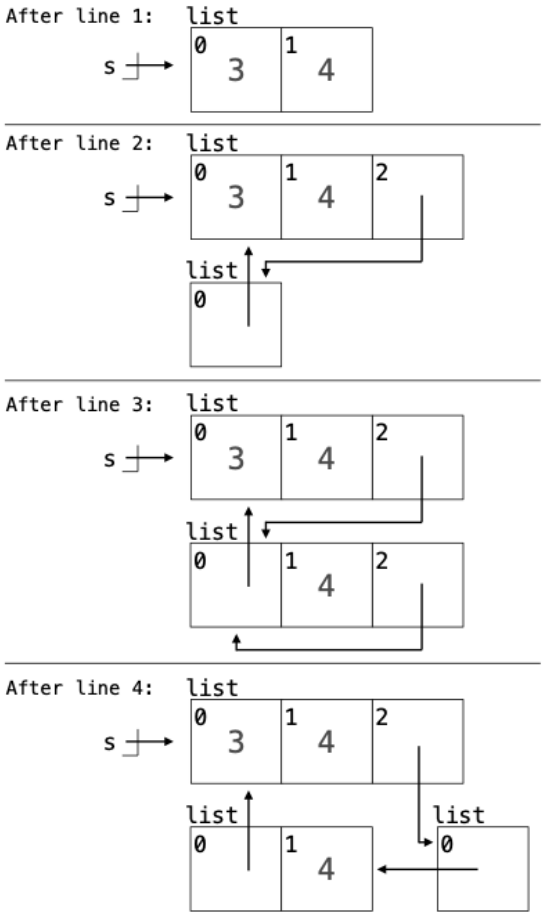
(a) What is your full name?

(b) What is your student ID number? A regex restricts inputs to numerical responses only.

1. (7.0 points) Hawkeye

The box-and-pointer diagram for the value of `s` is shown after each line of the program below. Fill in the blanks to match the diagrams.

```
s = [3, 4]

s.append(_________)
            (a)

_________.________(_________)
   (b)       (c)       (d)

s[2] = _________
           (e)
```



(a) (2.0 pt) Which of these could fill in blank (a)? Check all that apply.

- ☐ `s`
- ☐ `s[:]`
- ☐ `s[0]`
- ■ `[s]`
- ☐ `[s[:]]`
- ☐ `[s[0]]`
- ☐ `list(s)`
- ☐ `list(s[:])`
- ☐ `list(s[0])`

(b) (1.0 pt) Fill in blank (b).

```
s[2]
```

(c) (1.0 pt) Fill in blank (c).

```
extend
```

(d) (1.0 pt) Which of these could fill in blank (d)? (Select one.)

- ○ `s[1]`
- ● `s[1:]`
- ○ `s[1:2]`
- ○ `[s[1]]`
- ○ `[s[1:]]`
- ○ `[s[1:2]]`
- ○ `[s[1], s]`
- ○ `[s[1]] + s`

(e) (2.0 pt) Fill in blank (e).

```
[s[2].pop()]
```

2. (11.0 points) Doctor Change

(a) (6.0 points)

Implement `change`, a function that takes an integer `n` and a list of positive integers `coins`. It returns whether there is a subset of the values in `coins` that sums to `n`. As a side effect, `change` modifies `coins`.

```
def change(n, coins):
    """Return whether a subset of coins adds up to n.

    >>> change(10, [2, 7, 1, 8, 2])  # e.g., 2 + 8
    True
    >>> change(20, [2, 7, 1, 8, 2])  # e.g., 2 + 7 + 1 + 8 + 2
    True
    >>> change(6, [2, 7, 1, 8, 2])   # Impossible; only two 2's in coins
    False
    """
    if n == 0:
        return True
    elif _________:
            (a)
        return False
    coin = coins.pop()  # remove the end of coins and name it "coin"
    return change(n, _________) _________ change(_________, _________)
                        (b)        (c)              (d)        (e)
```

i. (2.0 pt) Fill in blank (a).

```
n < 0 or not coins or just not coins
```

ii. (1.0 pt) Fill in blank (b).

```
list(coins)
```
At least one of blank (b) and blank (e) must copy the `coins` list.

iii. (1.0 pt) Fill in blank (c).

```
or
```

iv. (1.0 pt) Which of these could fill in blank (d)? (Select one.)

- (●) `n - coin`
- ( ) `n - coins[0]`
- ( ) `n - coins[1]`
- ( ) `n - coins.pop()`

v. (1.0 pt) Fill in blank (e).

```
list(coins)
```

(b) (5.0 points)

Implement `amounts`, which takes a list of positive integers `coins`. It returns a sorted list of all unique non-negative integers `n` for which `change(n, coins)` returns `True`. You **may not** call `change`.

```
def amounts(coins):
    """List all unique n such that change(n, coins) returns True (in sorted order).

    >>> amounts([2, 5, 3])
    [0, 2, 3, 5, 7, 8, 10]
    >>> amounts([2, 7, 1, 8, 2])
    [0, 1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 13, 15, 16, 17, 18, 19, 20]
    """
    if not coins:
        return _________
                  (a)
    coin = coins[0]
    rest = amounts(_________)
                      (b)
    return sorted(_________ + [k + coin for _________])
                    (c)                        (d)
```

i. (1.0 pt) Fill in blank (a).

```
[0]
```

ii. (1.0 pt) Fill in blank (b).

```
coins[1:]
```

iii. (1.0 pt) Which of these could fill in blank (c)? (Select one.)

- (●) `rest`
- ( ) `rest[1:]`
- ( ) `amounts(rest)`
- ( ) `amounts(rest[1:])`
- ( ) `[k + coin for k in rest]`
- ( ) `[k + coin for k in rest[1:]]`

iv. (2.0 pt) Fill in blank (d).

```
k in rest if k + coin not in rest
```

3. (13.0 points) Shang-Chi

(a) (6.0 points)

Implement the `Valet` and `Garage` classes.

A `Valet` instance has two instance attributes: their total `tips` and the `garage` where they work (a `Garage` instance).

- The `park` method takes a string `car` and parks it in the `Garage` where they work.
- The `wash` method takes a string `car` that has been parked in their garage and a number `tip`. The `Valet` who washed the car and the `Valet` who **most recently** parked the car split the tip.

The `Garage` constructor takes a list of the `Valet`s who work in that `Garage`. Assume that `park` and `wash` are only invoked on a `Valet` that already has a `Garage`.

```
class Valet:
    """A valet is tipped after they wash a car,
    or after one of their parked cars is washed.

    >>> shaun = Valet()
    >>> katy = Valet()
    >>> g = Garage([shaun, katy])
    >>> shaun.park('Benz')
    >>> katy.park('BMW')
    >>> shaun.wash('Benz', 1)   # $1.0 to Shaun
    >>> katy.wash('Benz', 2)    # $1.0 to Katy, $1.0 to Shaun

    >>> shaun.park('Rolls')
    >>> katy.park('Rolls')
    >>> katy.wash('BMW', 2)     # $2.0 to Katy
    >>> shaun.wash('Rolls', 2)  # $1.0 to Shaun, $1.0 to Katy
    >>> [shaun.tips, katy.tips]
    [3.0, 4.0]
    """
    def __init__(self):
        self.tips = 0
        self.garage = None

    def park(self, car):
        _________
           (a)

    def wash(self, car, tip):
        self.tips += tip / 2
        _________ += tip / 2
           (b)

class Garage:
    """A garage holds cars parked by the valets who work there."""
    def __init__(self, valets):
        self.cars = {}
        for valet in valets:
            _________ = _________
               (c)         (d)
```

i. (2.0 pt) Fill in blank (a).

```
self.garage.cars[car] = self
```

ii. (2.0 pt) Fill in blank (b).

```
self.garage.cars[car].tips
```

iii. (1.0 pt) Fill in blank (c).

```
valet.garage
```

iv. (1.0 pt) Which of these could fill in blank (d)? (Select one.)

- ○ `Garage`
- ○ `valet`
- ○ `valets`
- ○ `garage`
- ○ `self.valet`
- ○ `self.valets`
- ○ `self.garage`
- ● `self`

(b) (2.0 points)

Definition. An *infinite* iterator `t` is one for which `next(t)` can be called any number of times and always returns a value.

Implement `ring`, a generator function that takes a non-empty list `s`. It returns an infinite generator that repeatedly yields the values of `s` in the order they appear in `s`.

```
def ring(s):
    """Yield all values of non-empty s in order, repeatedly.

    >>> t = ring([2, 5, 3])
    >>> [next(t), next(t), next(t), next(t), next(t), next(t), next(t)]
    [2, 5, 3, 2, 5, 3, 2]
    """
    _________:
       (a)

        _________
           (b)
```

i. (1.0 pt) Which of these could fill in blank (a)? (Select one.)

- ● `while True`
- ○ `while s`
- ○ `for x in s`
- ○ `for x in ring(s)`

ii. (1.0 pt) Fill in blank (b).

```
yield from s
```

(c) (5.0 points)

Implement `fork`, a function that takes an infinite iterator `t`. It returns two infinite iterators that each iterate over the contents of `t`.

```
def fork(t):
    """Return two iterators with the same contents as infinite iterator t.

    >>> a, b = fork(ring([1, 2, 3]))
    >>> [next(a), [next(b), next(b), next(b)], next(a), [next(b), next(b), next(b)], next(a)]
    [1, [1, 2, 3], 2, [1, 2, 3], 3]
    """
    s = []
    def copy():
        i = 0
        while True:
            if _________:
                  (a)
               s.append(_________)
                            (b)
            yield _________
                      (c)
            i += 1
    return _________
              (d)
```

i. (1.0 pt) Fill in blank (a).

```
len(s) == i
```

ii. (1.0 pt) Fill in blank (b).

```
next(t)
```

iii. (1.0 pt) Which of these could fill in blank (c)? (Select one.)

- ○ `from s`
- ○ `from t`
- ○ `s[0]`
- ● `s[i]`
- ○ `next(t)`

iv. (2.0 pt) Fill in blank (d).

```
copy(), copy()
```

4. (10.0 points) Thanos

Hint: you may call built-in sequence functions: `sum`, `max`, `min`, `all`, `any`, `map`, `filter`, `zip`, and `reversed`.

(a) (4.0 points)

Implement `snap`, which takes a one-argument function `f`, a one-argument function `g`, and a sequence `s`. It returns a list of `(x, f(x))` pairs (two-element tuples) for all `x` in `s` for which `g(f(x))` is a true value. The implementation of `snap` only calls `f` once per element of `s`; never twice.

Important: For full credit, your implementation may only call `f` **once** on each element of `s`.

```
def snap(f, g, s):
    """Return a list of (x, f(x)) pairs for each x in s such that g(f(x)) is a true value.

    >>> snap(lambda x: x * x, lambda x: x < 10, range(5))
    [(0, 0), (1, 1), (2, 4), (3, 9)]
    >>> snap(lambda x: x * x, lambda x: x > 10, range(5))
    [(4, 16)]
    >>> snap(lambda x: x * x, lambda x: x and x - 9, range(5))
    [(1, 1), (2, 4), (4, 16)]
    """
    return [(x, _________) for _________ in _________ if _________]
                  (a)            (b)          (c)          (d)
```

i. (1.0 pt) Fill in blank (a).

```
y
```

ii. (1.0 pt) Which of these could fill in blank (b)? (Select one.)

- ○ `x`
- ● `x, y`
- ○ `y`
- ○ `y, z`

iii. (1.0 pt) Fill in blank (c).

```
zip(s, map(f, s)) or [(z, f(z)) for z in s]
```

iv. (1.0 pt) Fill in blank (d).

```
g(y)
```

(b) (4.0 points)

Implement `max_diff`, which takes a non-empty sequence `s` and a one-argument function `f`. It returns a pair of elements `(v, w)` in `s` for which `f(v) - f(w)` is largest. `v` and `w` may be the same or different elements of `s`. Also, describe the order of growth of the run time of `max_diff`.

```
def max_diff(s, f):
    """Return two elements (v, w) of s for which f(v) - f(w) is largest.

    >>> max_diff(range(-7, 4), lambda x: x * x)  # (-7 * -7) - (0 * 0) = 49
    (-7, 0)
    >>> max_diff(['what', 'a', 'great', 'film'], len)  # len('great') - len('a')
    ('great', 'a')
    """
    assert s, 's cannot be empty'
    v, w = None, None
    for x in s:
        for y in s:
            if v is None or _________:
                              (a)

                ---------
                   (b)
    return v, w
```

i. (1.0 pt) Fill in blank (a).

```
f(x) - f(y) > f(v) - f(w)
```

ii. (1.0 pt) Fill in blank (b).

```
v, w = x, y
```

iii. (2.0 pt) What is the order of growth of the time required to evaluate `max_diff(s, f)` for a sequence `s` of length n and a function `f` that requires constant time to evaluate.

- ○ Exponential, $\Theta(b^n)$
- ● Quadratic, $\Theta(n^2)$
- ○ Linear, $\Theta(n)$
- ○ Logarithmic, $\Theta(\log_2 n)$
- ○ Constant, $\Theta(1)$

(c) (2.0 points)

Implement `max_diff_fast`, which has the same signature and behavior as `max_diff`, but has a faster order of growth of its run time.

Important. You may **not** use a list comprehension in your solution.

```
def max_diff_fast(s, f):

    return _________ , _________
              (a)          (b)
```

i. (1.0 pt) Fill in blank (a), but do not use a comprehension.

```
max(s, key=f)
```

ii. (1.0 pt) Fill in blank (b), but do not use a comprehension.

```
min(s, key=f)
```

5. (9.0 points) Groot

Definition. A *twig* is a tree that is not a leaf but whose branches are all leaves.

The `Tree` and `Link` classes appear on your midterm 2 study guide. Assume they are defined.

(a) (4.0 points)

Implement `twig`, which takes a `Tree` instance t. It returns `True` if `t` is a twig and `False` otherwise.

```
def twig(t):
    """Return True if Tree t is a twig and False otherwise.

    >>> twig(Tree(1))
    False
    >>> twig(Tree(1, [Tree(2), Tree(3)]))
    True
    >>> twig(Tree(1, [Tree(2), Tree(3, [Tree(4)])]))
    False
    """
    return _________ and _________
              (a)           (b)
```

i. (2.0 pt) Which of these could fill in blank (a)? Check all that apply. *Hint*: The `bool` function returns `True` for a true value and `False` for a false value.

- ☐ `t`
- ☐ `bool(t)`
- ☐ `t.is_leaf()`
- ■ `not t.is_leaf()`
- ☐ `t.branches`
- ■ `bool(t.branches)`
- ☐ `len(t.branches)`
- ■ `len(t.branches) > 0`
- ☐ `len(t.branches) >= 0`

ii. (2.0 pt) Fill in blank (b).

```
all([b.is_leaf() for b in t.branches])
```

(b) (5.0 points)

Implement `twigs`, which takes a `Tree` instance `t`. It returns a linked list (either a `Link` instance or `Link.empty`) containing all of the labels of the twigs in `t`. Labels should be in the same order as they appear in `repr(t)`.

```
def twigs(t):
    """Return a linked list of the labels of the twigs in t.

    >>> t = Tree(1, [Tree(2), Tree(3, [Tree(4)]), Tree(5, [Tree(6, [Tree(7), Tree(8)])])])
    >>> print(twigs(t))
    <3 6>
    >>> print(twigs(Tree(0, [t, t, t])))
    <3 6 3 6 3 6>
    >>> twigs(Tree(0)) is Link.empty
    True
    """

    def add_twigs(t, rest):

        if twig(t):

            return _________
                     (a)

        for b in reversed(t.branches):

            rest = _________
                     (b)

        return rest

    return add_twigs(t, _________)
                           (c)
```

i. (2.0 pt) Fill in blank (a).

```
Link(t.label, rest)
```

ii. (2.0 pt) Fill in blank (b).

```
add_twigs(b, rest)
```

iii. (1.0 pt) Fill in blank (c).

```
Link.empty
```

No more questions.

CS 61A Structure and Interpretation of Computer Programs

Fall 2022 MIDTERM 1

INSTRUCTIONS

You have 1 hour and 50 minutes to complete the exam.

- The exam is closed book, closed notes, closed computer, closed calculator, except one 8.5" × 11" page of your own creation and the provided midterm 1 study guide.
- Mark your answers on the exam itself in the spaces provided. We will not grade answers written on scratch paper or outside the designated answer spaces.
- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `pow`, `len`, `abs`, `bool`, `int`, `float`, `str`, `max`, and `min`.
- You **may not** use example functions defined on your study guide unless a problem clearly states you can.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.
- You may not use ; to place two statements on the same line.

For questions with **circular bubbles**, you should select exactly *one* choice.

◯ You must choose either this option

◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

☐ You could select this choice.

☐ You could select this one too!

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

1. (4.0 points) What Would Python Display?

Assume the following code has been executed.

```
bear = -1
oski = lambda print: print(bear)
bear = -2

s = "Knock"
```

For each expression below, write the output **displayed by the interactive Python interpreter** when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

(a) (1.0 pt) `(3 and 4) - 5`

- ○ -2
- ● -1
- ○ 1
- ○ 2
- ○ Error

The `and` operator returns the first falsey value (a false value, but not necessarily `False`), or the last value if none are falsey. Since 3 is truthy (a true value, even though it's not `True`), `3 and 4` evaluates to 4. `4 - 5` evaluates to -1.

(b) (1.0 pt) `oski(abs)`

- ○ -2
- ○ -1
- ○ 1
- ● 2
- ○ Function
- ○ Error

The lambda function bound to `oski` takes a function and calls it on `bear`. Thus, `oski(abs)` evaluates `abs(bear)`. Since `bear` isn't defined in the lambda function, we take the global value of `bear`, which is -2. `abs(-2)` is 2.

(c) (2.0 pt) `print(print(print(s, s) or print("Who's There?")), "Who?")`

Knock Knock
Who's There?
None
None Who?

Python evaluation order evaluates the operator expression (to a function), then the operands from left to right (to the arguments), then calls the function on its arguments.

Below, we substitue operands with argument values and describe the side effects of doing so.

`print(print(print(s, s) or print("Who's There?")), "Who?")`

`print(print(None or print("Who's There?")), "Who?")` ; Calls print on "Knock Knock"

`print(print(print("Who's There?")), "Who?")` ; Since `None` is a false value, `None or print(...)` evaluates the `print` call.

`print(print(None), "Who?")` ; Calls print on "Who's There?"

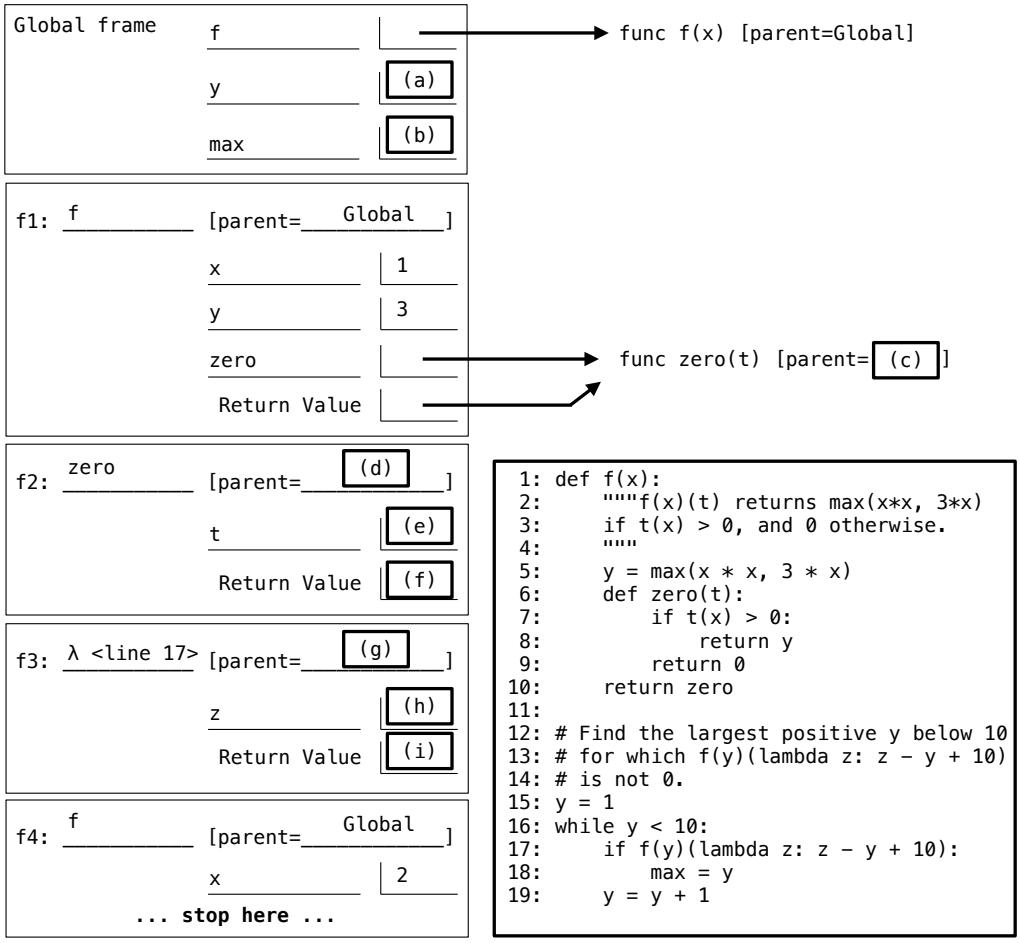
`print(None, "Who?")` ; Calls print on None

`None` ; Calls print on two arguments and displays "None Who?"

Since the value of the whole expression is None, that final None is not displayed.

2. (12.0 points) Don't Call Me, I'll Call You

When working on a large project, you get the error `'int' object is not callable` in the code below during the second call to `f`. Complete the environment diagram until the error occurs to identify the problem with the code. **If a blank contains an arrow to a function, write the function as it would appear in the diagram.**




```
 1: def f(x):
 2:     """f(x)(t) returns max(x*x, 3*x)
 3:     if t(x) > 0, and 0 otherwise.
 4:     """
 5:     y = max(x * x, 3 * x)
 6:     def zero(t):
 7:         if t(x) > 0:
 8:             return y
 9:         return 0
10:     return zero
11:
12: # Find the largest positive y below 10
13: # for which f(y)(lambda z: z - y + 10)
14: # is not 0.
15: y = 1
16: while y < 10:
17:     if f(y)(lambda z: z - y + 10):
18:         max = y
19:     y = y + 1
```

(a) (1.0 pt) Fill in blank (a).

2

(b) (1.0 pt) Fill in blank (b).

1

(c) (1.0 pt) Which of these could fill in blank (c)?

- ◯ `Global`
- ◯ `f`
- `f1`

(d) (1.0 pt) Which of these could fill in blank (d)?

- ◯ `Global`
- ◯ `f`

`f1`

(e) (2.0 pt) Fill in blank (e).

func λ(z) <line 17> [parent=Global]

(f) (1.0 pt) Fill in blank (f).

3

(g) (1.0 pt) Which of these could fill in blank (g)?

`Global`

- ◯ `f`
- ◯ `zero`
- ◯ `f1`
- ◯ `f2`

(h) (1.0 pt) Fill in blank (h).

1

(i) (1.0 pt) Fill in blank (i).

10

(j) (2.0 pt) Explain in 10 words or less why the `'int' object is not callable` error occurred.

Example answer: "Global `max` was reassigned to an integer"

The first iteration of the while loop works as intended; `f(y)` returns the function `zero` with `x` set to `1` and `y` set to 3. We then call `zero` on `lambda z: z - y + 10`. This evaluates `(lambda z: z - y + 10)(1) == 1 - 1 + 10 == 10`. Since `10 > 0`, we return `y`, which is equal to `3`. This is truthy, so we set `max` to `1`.

On the second iteration of the loop, we run until line 5. At this point, we evaluate a call expression with `max` as the operator. However, `max` was set to 1 in the previous iteration, overwriting its original assignment to the built-in `max` function. Thus, Python tries to call `1` as a function; since `1` is not a function, Python errors, stating that `'int' object is not callable`.

3. (10.0 points) It's Perfect

Definitions. A *perfect* number is a positive integer `n` whose *proper factors* (the factors of `n` below `n`) sum to exactly `n`. A number `n` is *abundant* if the sum of `n`'s proper factors is greater than `n` and *deficient* if that sum is less than `n`.

(a) (4.0 points)

Implement `classify`, a function that takes an integer `n` **greater than** 1. It returns the string `'deficient'`, `'perfect'`, or `'abundant'` that correctly describes `n`.

```
def classify(n):
    """Return whether n > 1 is 'deficient', 'perfect', or 'abundant'.

    >>> classify(6)   # Proper factors 1, 2 and 3 sum to exactly 6.
    'perfect'
    >>> classify(24)  # Proper factors 1, 2, 3, 4, 6, 8, and 12 sum to 36.
    'abundant'
    >>> classify(23)  # Proper factor 1 sums to 1.
    'deficient'
    """
    total, k = 0, 1
    while __<BLANK 1>__:
        if __<BLANK 2>__:
            __<BLANK 3>__
        k = k + 1
    if total == n:
        return 'perfect'
    elif __<BLANK 4>__:
        return 'deficient'
    else:
        return 'abundant'
```

i. (1.0 pt) BLANK 1

- (●) `k < n`
- ( ) `k <= n`
- ( ) `k < total`
- ( ) `k <= total`

ii. (1.0 pt) BLANK 2

n % k == 0

iii. (1.0 pt) BLANK 3

total += k

iv. (1.0 pt) `BLANK 4`

total < n

In order to classify a number, we divide the code roughly into two steps:

- Compute the sum of all the proper factors of `n`
- Compare that sum to `n` to determine what classification to return

The first step is done with the `while` loop; we iterate over all values of `k` from 1 to `n` (not including `n`), and check if each one is a factor of `n`. This can be done by running `n % k`; if this is zero, then `k` divides `n` evenly, so `k` is a factor of `n`. If `k` is a factor of `n`, we add its value to the running total.

The second step is done by checking if `total` is either equal to, less than, or greater than `n`. This is done in the last `if/elif/else` clause; since we want to return `'deficient'` when `total < n`, we pick that to fill blank 4.

(b) (6.0 points)

Implement `nearest_perfect`, which takes an integer `n` above 5. It returns the nearest perfect number to `n`. If two perfect numbers are equally close to `n`, return the larger one. A number `a` is nearer to `n` than another number `b` if `abs(a - n) < abs(b - n)`. Assume `classify` is implemented correctly.

```
def nearest_perfect(n):
    """Return the nearest perfect number to n. In a tie, return the larger one.

    >>> nearest_perfect(8)   # 6 is perfect and 2 away from 8.
    6
    >>> nearest_perfect(20)  # 28 is perfect and 8 away from 20.
    28
    >>> nearest_perfect(17)  # Both 6 and 28 are 11 away from 17.
    28
    >>> nearest_perfect(6)   # 6 is perfect and 0 away from 6.
    6
    """
    k = 0
    while True:
        if __<BLANK 5>__:
            __<BLANK 6>__
        if __<BLANK 7>__:
            k = -k
        else:
            __<BLANK 8>__
```

i. (2.0 pt) `BLANK 5`

classify(n+k) == "perfect"

ii. (1.0 pt) `BLANK 6`

- ○ `n = n + k`
- ○ `n = n + 1`
- ○ `n = n - k`
- ○ `n = n - 1`
- ● `return n + k`
- ○ `return n`
- ○ `return k`

iii. (1.0 pt) `BLANK 7`

k > 0; Alternate solution: classify(n-k)=="perfect"

iv. (2.0 pt) `BLANK 8`

k = -k + 1; Alternate solution: k = k+1

The key observation here is that we can determine if a number `k` is perfect by checking if `classify(k)` returns `'perfect'`.

We can further note that according to the rules for returning, we should return the first perfect number that appears in the sequence `n, n+1, n-1, n+2, n-2, ...`

For example, if we try to get `nearest_perfect(17)`, we should check, in order, `17, 18, 16, 19, 15, 20, ..., 28, 6, ...`. Among these values, `28` is the first perfect number, so it is the nearest perfect number to 17. Note that we check `17 + 11 == 28` before we check `17 - 11 == 6`, so we return the larger perfect number if `n` is equally close to two perfect numbers.

Thus, we can fill blanks 5 and 6 so check if `n+k` is a perfect number, and return `n+k`, respectively. For blanks 7 and 8, we want to make it so that `k` goes `0, 1, -1, 2, -2, ...`. We note that after every positive number `k` in this sequence, the next number is `-k`, so we set blank 7 to check if `k > 0`. For all other numbers, we go from `0 -> 1`, `-1 -> 2`, `-2 -> 3`, and so on. This can be done by negating `k` and adding 1; that is, `1 == -0 + 1`, `2 == -(-1)+1`, etc. Thus we fill blank 8 with `k = -k + 1`.

An alternative solution is to check if `classify(n-k)=='perfect'` in line 7. If this is the case, then k gets set to -k, and we return the appropriate perfect number in the next iteration of the while loop. Using this, line 8 needs only to increment k, so we can do `k += 1` on that line.

4. (4.0 points) Super Powers

Definition. A *function power*, written $f^n(x)$, describes repeated application of a function f to x. $f^2(x) = f(f(x))$, $f^5(x) = f(f(f(f(x))))$, and so on.

(a) (2.0 pt) Choose **all** correct implementations of `funsquare`, a function that takes a one-argument function `f`. It returns a one-argument function `f2` such that `f2(x)` has the same behavior as `f(f(x))` for all `x`.

```
>>> triple = lambda x: 3 * x
>>> funsquare(triple)(5)  # Equivalent to triple(triple(5))
45
```

```
A:  def funsquare(f):
        return f(f)
```

```
B:  def funsquare(f):
        return lambda: f(f)
```

```
C:  def funsquare(f, x):
        def g(x):
            return f(f(x))
        return g
```

```
D:  def funsquare(f):
        return lambda x: f(f(x))
```

```
E:  def funsquare(f, x):
        return f(f(x))
```

```
F:  def funsquare(f):
        def g(x):
            return f(f(x))
        return g
```

☐ A

☐ B

☐ C

■ D

☐ E

■ F

C and E receive two inputs, while `funsquare` is defined to only receive one input; thus, these two are incorrect.

For A and B, we end up calling `f` with a function as an argument, even if `f` expects to receive an integer as an argument. Thus, these two cannot work.

D and F both work; we can verify this by trying to run these functions on the doctests.

(b) (1.0 pt) Fill in the blank to assign `quadruple` to a function that takes a number x and returns $4x$.

```
>>> quadruple = funsquare(__<BLANK 1>__)
>>> quadruple(3)  # 4 * 3
12
```

BLANK 1

lambda x: 2 * x

`quadruple` is the result of doubling a number twice, so we should fill blank 1 with a lambda function that doubles its input.

(c) (1.0 pt) Fill in the blank to assign `funfourth` to a function that takes a one-argument function $f(x)$ and returns a one-argument function $f^4(x)$. **You may not use a lambda expression.**

```
>>> funfourth = __<BLANK 2>__
>>> funfourth(triple)(10)  # 10 * 3 * 3 * 3 * 3 (triple is defined near the top of the page)
810
```

BLANK 2

funsquare(funsquare)

$f^4(x)$ is $f(f(f(f(x))))$, which is also $f^2(f^2(x))$. To apply f^2 twice, we call `funsquare(funsquare)`, which returns a function that takes `f` and calls `funsquare(funsquare(f))`.

5. (0.0 points) A+ Question 1: Busy Beaver

This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

To receive an A+ in CS 61A, you need at least 300 points in the course overall and must solve 2 A+ questions. To receive credit for this A+ question, you must answer all parts of it entirely correctly.

Introduced by Tibor Radó in 1962, the *Busy Beaver Game* aims to find the terminating program of a given length that produces the most possible output. Here, we'll explore one approach to generating a lot of output.

Implement `b`, which takes a zero-argument function `f` that returns `None`. The `b` function calls its argument `f` 64 times and returns `None`.

- You may not use any numbers or strings or any expressions that evaluate to numbers or strings.
- You may not import any functions, but may use built-in functions that do not need to be imported.
- You may not use lists, tuples, or dictionaries.
- Your answer for blank 2 must be no longer than 50 characters, ignoring whitespace.

```
def b(f):
    """Evaluate f() 64 times.

    >>> a = lambda: print('A', end='')
    >>> b(a)
    AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
    """
    (lambda g: g(g(g(g(g(g(g(__<BLANK 1>__)))))))(__<BLANK 2>__)__<BLANK 3>__
```

(a) (0.0 pt) `BLANK 1`

- (●) `f`
- ( ) `g`
- ( ) `f()`
- ( ) `g()`

(b) (0.0 pt) `BLANK 2` (**Your answer must not be longer than 50 characters, ignoring whitespace.**)

lambda f: lambda: f() or f()

(c) (0.0 pt) `BLANK 3`

- ● `()`
- ○ `(f)`
- ○ `(g)`

To solve this, we note that $64 = 2^6$, and that there are exactly 6 `g`s in the lambda. Thus, our strategy to call `f` 64 times is as follows:

- Blank 2 should be a function that receives as input a function `h` and returns a new function that calls `h` twice. Since `f` is guaranteed to return None, we can use `h() or h()` to call `h` twice (and return None).
- The first lambda will take in Blank 2 as input `g`, and run g^6 on some input. Since each layer of Blank 2 doubles the number of function calls, g^6 will call its input function $2^6 = 64$ times. This suggests that we should set Blank 1 to `f`.
- Therefore, `(lambda g: g(g(g(g(g(g(__<BLANK 1>__)))))))(__<BLANK 2>__)` creates a function that calls `f()` 64 times. We still need to call it, so Blank 3 should call the newly created function. This can be done by setting Blank 3 to `()`.

CS 61A Fall 2022

Structure and Interpretation of Computer Programs

Midterm 2 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

1. (7.0 points) What would Python Python?

Assume the code below has been executed.

```
class Snake:
    legs = 0
    def __init__(self, name):
        self.name = name
    def __str__(self):
        return self.name
    def run(self, s):
        print("Snakes don't run")
        return self.crawl()
    def crawl(self):
        print(f"{self} crawled")
    def eat(self, s):
        self.run(s)
        print("Nom nom")

class Python(Snake):
    def run(self, s):
        print(eval(s))  # eval(s) evaluates string s as a Python expression

snek = Snake("atari")
solidsnake = Snake("David")
solidsnake.legs = 2
solidsnake.run = lambda s: print("He ran")
python = Python("pypy")
```

For each expression below, write the output **displayed by the interactive Python interpreter** when the expression is evaluated.

- If an output has multiple lines, write each line separately.
- If an error occurs, write "Error", but include all output displayed before the error.
- If evaluation would run forever, write "Forever".
- To display a function value, write "Function".

Each expression below should be evaluated independently of previously-evaluated expressions.

(a) (1.0 pt) `[snek.legs, Snake.legs]`

[0, 0]

(b) (2.0 pt) `solidsnake.eat("python")`

He ran
Nom nom

(c) **(2.0 pt)** `python.eat("snek")`

atari
Nom nom

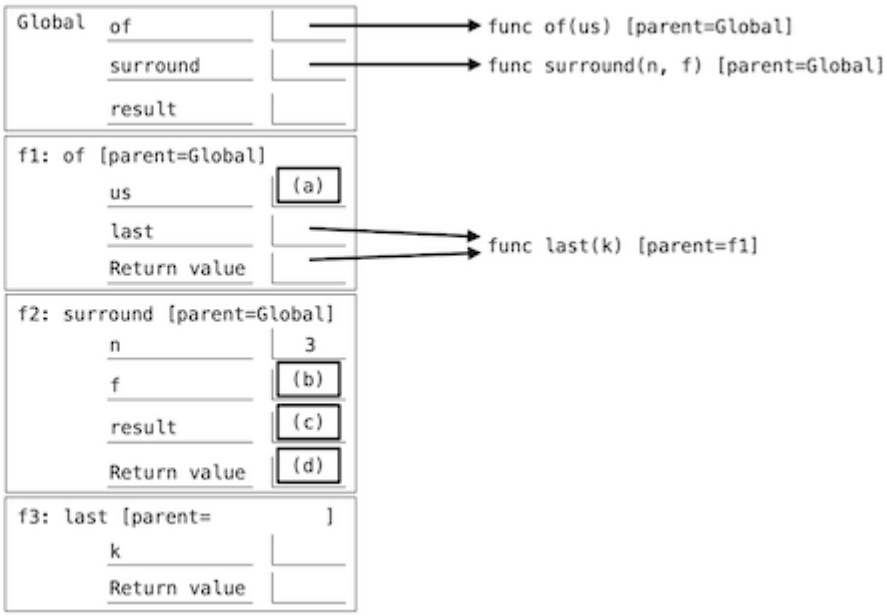
(d) **(2.0 pt)** `Snake.run(python, python)`

Snakes don't run
pypy crawled

2. (6.0 points) Environmental Disaster

Your friend asks you to help debug their program below, which doesn't print what they expect. Complete the environment diagram, then answer questions about it. The diagram itself will not be scored; only the questions based on it.

```
def of(us):
    def last(k):
        "The last k items of us"
        while k > 0:
            result.append(us.pop())
            k = k - 1
        return result
    return last

def surround(n, f):
    "n is the first and last item of f(2)"
    result = [n]
    result = f(2)
    result[0] = [n]
    return result.append(n)

result = [1]
surround(3, of([4, 5, 6]))
print(result)
```



(a) (1.0 pt) *Blank (a)*: What is the name `us` bound to in f1?

- ○ A list `[4, 5, 6]`
- ○ A list `[4, 5]`
- ● A list `[4]`

(b) (1.0 pt) *Blank (b)*: What is the name `f` bound to in frame `f2`?

- ○ `func of(us) [parent=Global]`
- ● `func last(k) [parent=f1]`
- ○ None of the above

(c) (1.0 pt) *Blank (c)*: What is the name `result` bound to in frame `f2`?

- ● The same list bound to the name `result` in the Global frame
- ○ A list that is **not** bound to any name in the Global frame

(d) (1.0 pt) *Blank (d)*: What is the return value of frame `f2`?

- ○ The same list bound to the name `result` in `f2`
- ○ A list that is one element longer than the one bound to the name `result` in `f2`
- ● None of the above

(e) (2.0 pt) What is displayed by the last line, `print(result)`?

```
[[3], 6, 5, 3]
```

3. (8.0 points) Hog Revisited

In Project 1, we created a simulator for the game of Hog without using lists or iterators. Let's revise the implementation, but instead of representing dice as zero-argument functions, **we'll represent dice as iterators over outcomes**.

(a) (3.0 points)

Implement `make_test_dice`, which takes a list of `outcomes`. It returns an infinite iterator that repeatedly cycles through the outcomes.

```
def make_test_dice(outcomes):
    """Return an infinite iterator that cycles through the elements of outcomes.

    >>> dice = make_test_dice([1, 2, 3])
    >>> next(dice)
    1
    >>> next(dice)
    2
    >>> next(dice)
    3
    >>> next(dice)
    1
    >>> next(dice)
    2
    >>> next(dice)
    3
    >>> next(dice)
    1
    >>> next(dice)
    2
    """
    outcomes = list(outcomes)
    while True:
        for i in ___(a)___:
            ___(b)___
```

i. (1.0 pt) Which of these could fill in blank (a)? Select **all** that apply.

- [x] `outcomes`
- [ ] `outcomes + make_test_dice(outcomes)`
- [ ] `outcomes.extend(make_test_dice(outcomes))`
- [ ] `outcomes[0] + make_test_dice(outcomes[1:])`
- [ ] `make_test_dice(outcomes) + make_test_dice(outcomes)`
- [ ] `make_test_dice(outcomes[0]) + make_test_dice(outcomes[1:])`

ii. (2.0 pt) Fill in blank (b).

```
yield i
```

(b) (5.0 points)

Implement `roll_dice`, which takes two arguments: a positive integer called `num_rolls` giving the number of dice to roll and a `dice` iterator that provides outcomes. It returns the number of points scored by rolling the `dice` `num_rolls` times in a turn. The points scored are either the sum of the dice outcomes or 1 (Sow Sad).

- Sow Sad. If any of the dice outcomes is a 1, the current player's score for the turn is 1.

```
def roll_dice(num_rolls, dice):
    """Return the number of points scored by rolling dice num_rolls times.

    >>> dice = make_test_dice([6, 6, 6, 6, 6, 1])
    >>> roll_dice(5, dice) # 6, 6, 6, 6, 6
    30
    >>> roll_dice(5, dice) # 1, 6, 6, 6, 6
    1
    >>> roll_dice(2, dice) # 6, 1
    1
    >>> roll_dice(10,  make_test_dice([2, 4, 3])) # 2, 4, 3, 2, 4, 3, 2, 4, 3, 2
    29
    """
    rolls = ___(a)___
    if ___(b)___:
        return 1
    ___(c)___
```

i. (2.0 pt) Fill in blank (a).

```
[next(dice) for _ in range(num_rolls)]
```

ii. (1.0 pt) Fill in blank (b).

```
1 in rolls
```

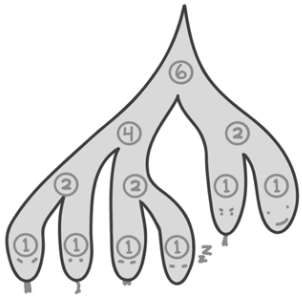
iii. (2.0 pt) Fill in blank (c).

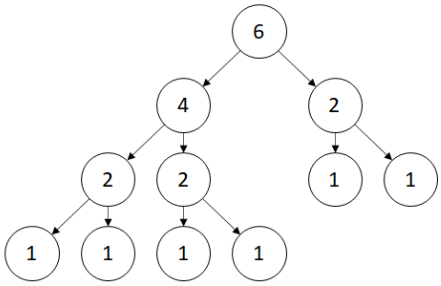
```
return sum(rolls)
```

4. (9.0 points) The Lambdanean Hydra

Heracles is fighting the Lambdanean Hydra, a many-headed monster, where two heads regrow when one head gets chopped off. A hydra is represented by a tree, each head is represented by a leaf labeled 1, and each non-leaf node has two branches and is labeled with the number of leaves among its descendants. Here's a six-headed hydra named `lerna`:

```
lerna=Tree(6,[Tree(4,[Tree(2,[Tree(1),Tree(1)]),Tree(2,[Tree(1),Tree(1)])]),Tree(2,[Tree(1),Tree(1)])])
```





(a) (4.0 points)

Implement `is_hydra`, which takes in as input a Tree instance `t` and determines if `t` represents a valid hydra.

```
def is_hydra(t):
    """Return True if Tree instance t properly represents a hydra and False otherwise.

    >>> is_hydra(Tree(1))
    True
    >>> is_hydra(Tree(2, [Tree(1), Tree(1)]))
    True
    >>> is_hydra(Tree(3, [Tree(1), Tree(2)]))  # Wrong leaf label
    False
    >>> is_hydra(Tree(3, [Tree(1), Tree(1)]))  # Wrong root label
    False
    >>> is_hydra(Tree(3, [Tree(3, [Tree(1), Tree(1)]), Tree(1)]))  # Wrong node label below root
    False
    >>>is_hydra(lerna)  # lerna the six-headed hydra is defined above
    True
    """
    if t.is_leaf():
        return t.label == 1
    if len(t.branches) != 2:
        return False
    return (t.label == ___(a)___) and ___(b)___
```

i. (2.0 pt) Fill in blank (a).

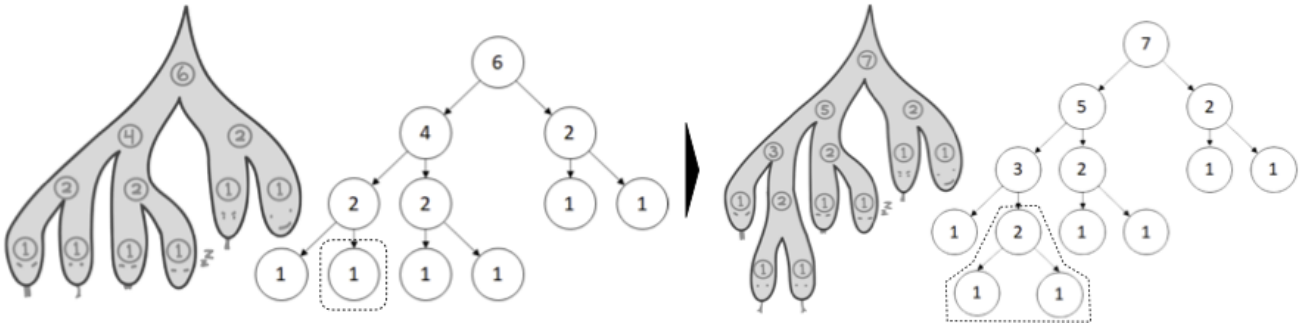
```
sum([b.label for b in t.branches])
```

ii. (2.0 pt) Fill in blank (b).

```
is_hydra(t.branches[0]) and is_hydra(t.branches[1])
```

(b) (5.0 points)

If Heracles cuts off the second head from the left, two new heads appear there, and all ancestor labels are updated.



Implement `chop_head(hydra, n)`, which takes as input a Tree instance `hydra` representing a hydra and a positive integer `n`. It mutates `hydra` by chopping off the `n`th head from the left (adding two new adjacent heads in its place).

```
def chop_head(hydra, n):
    """
    >>> lerna = Tree(1)
    >>> chop_head(lerna, 1)  # Note that n is 1-indexed
    >>> chop_head(lerna, 1)
    >>> chop_head(lerna, 3)
    >>> chop_head(lerna, 1)
    >>> chop_head(lerna, 3)
    >>> lerna                # Now lerna is a six-headed hydra (above left)
    Tree(6,[Tree(4,[Tree(2,[Tree(1),Tree(1)]),Tree(2,[Tree(1),Tree(1)])]),
            Tree(2,[Tree(1),Tree(1)])])
    >>> chop_head(lerna, 2)
    >>> lerna                # The mutated lerna now has seven heads (above right)
    Tree(7,[Tree(5,[Tree(3,[Tree(1),Tree(2,[Tree(1),Tree(1)])]),Tree(2,[Tree(1),Tree(1)])]),
            Tree(2,[Tree(1),Tree(1)])])
    """
    assert is_hydra(hydra)
    assert n > 0 and n <= hydra.label
    if hydra.is_leaf():
        ___(a)___  # This blank may be filled with multiple lines of code
        return
    ___(b)___
    left, right = hydra.branches
    if ___(c)___:
        chop_head(right, ___(d)___)
    else:
        chop_head(left, n)
```

i. **(2.0 pt)** Select **all** options that could fit in blank (a), which may have multiple lines.

```
Option 1:  hydra = Tree(2,[Tree(1), Tree(1)])

Option 2:  for _ in range(2):
               hydra.branches.append(Tree(1))
               hydra.label = 2

Option 3:  hydra.label = Tree(2)
           hydra.branches = [Tree(1),Tree(1)]

Option 4:  hydra.label = 2
           hydra.branches = [Tree(1), Tree(1)]

Option 5:  hydra.label = 2
           hydra.branches = [Tree(1)] * 2
```

- ☐ Option 1
- ■ Option 2
- ☐ Option 3
- ■ Option 4
- ☐ Option 5

ii. **(1.0 pt)** Fill in blank (b).

```
hydra.label += 1
```

iii. **(1.0 pt)** Fill in blank (c).

```
n > left.label
```

iv. **(1.0 pt)** Fill in blank (d).

```
n - left.label
```

5. (7.0 points) Aim for 100

The function `count_subsets` takes as input a list of positive integers `s`. It returns the number of lists that sum to 100 and contain a subset of the elements of `s` in order.

```
def count_subsets(s):
    """
    >>> count_subsets([25, 50, 75, 100, 125, 150])  # [25, 75], [100]
    2
    >>> count_subsets([25, 50, 25, 75])  # [25, 75] (first 25), [25, 75] (second 25), [25, 50, 25]
    3
    >>> count_subsets(list(range(1,10000)))
    444793
    """
```

(a) (5.0 points)

Complete the following implementation of `count_subsets`.

```
def count_subsets(s):
    def helper(sum_so_far, index):
        if ___(a)___:
            if ___(b)___:
                return 1
            return 0
        return ___(c)___ + ___(d)___
    return helper(0,0)
```

i. (1.0 pt) Fill in blank (a).

- ○ `index == s`
- ● `index == len(s)`
- ○ `sum_so_far == 100`
- ○ `sum_so_far != 100`

ii. (1.0 pt) Fill in blank (b).

- ○ `index == s`
- ○ `index == len(s)`
- ● `sum_so_far == 100`
- ○ `sum_so_far != 100`

iii. (1.0 pt) Fill in blank (c).

- ○ `count_subsets(s[index:])`
- ○ `count_subsets(s[1:])`
- ○ `helper(sum_so_far, index)`
- ● `helper(sum_so_far, index + 1)`

iv. (2.0 pt) Fill in blank (d).

```
helper(sum_so_far+s[index], index + 1)
```

(b) (2.0 points)

i. (1.0 pt) What is the order of growth of the time required to evaluate `count_subsets`, in terms of the length of `s`?

- (●) Exponential
- ( ) Quadratic
- ( ) Linear
- ( ) Logarithmic
- ( ) Constant

ii. (1.0 pt) We decide to rewrite `count_subsets`, following a different approach:

```
def count_subsets(s):
    values = [1]+[0]*100
    for i in s:
        for j in reversed(range(100-i+1)):
            values[j+i]+=values[j]
    return values[100]
```

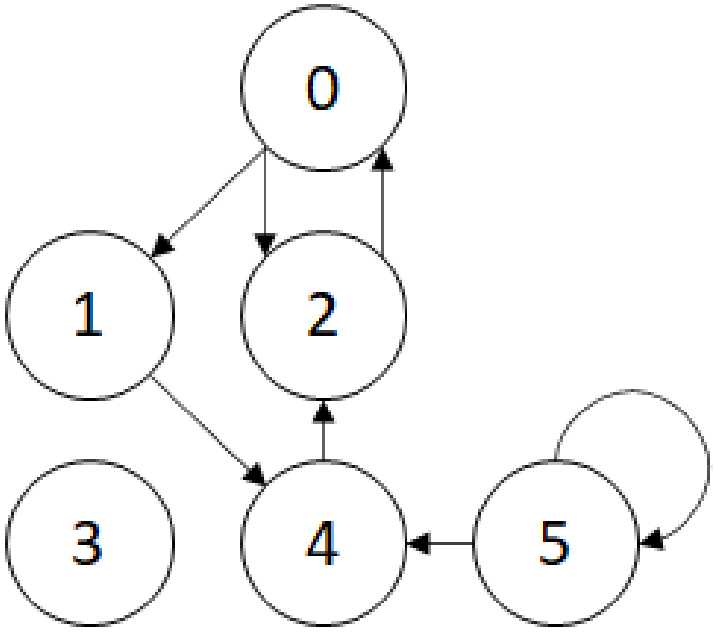
What is the order of growth of the time required to evaluate the new version of `count_subsets`, in terms of the length of `s`?

- ( ) Exponential
- ( ) Quadratic
- (●) Linear
- ( ) Logarithmic
- ( ) Constant

6. (8.0 points) Point A to Point B

A *graph* is a structure consisting of two parts:

- A set of n nodes, which are labeled from 0 to n-1.
- A set of directed edges, which go from one node (the source node) to another node (the destination).



The above is an example of a graph with 6 nodes, with edges represented as arrows. Note that a graph can have:

- Nodes that have no edges to/from them (Node 3)
- Two edges between the same two nodes, with source and destination reversed (The edges between Nodes 0 and 2)
- An edge from a node to itself (Node 5)

However, a graph cannot have two distinct edges from one node to another. For example, we cannot have a graph where two edges go from Node 0 to Node 1.

We say that a Node j is a **destination** of Node i if there is a directed edge from Node i to Node j. Thus, in the above example, Node 0 has two destinations (Nodes 1 and 2), and Node 1 has one destination (Node 4). Note that Node 0 is not considered a destination of Node 1.

We say that there is a **path** from Node i to Node j if either `i==j`, or we can go from Node i to Node j by following a sequence of edges in the graph. For example, we can get from Node 1 to Node 0 by following the edges `1->4`, then `4->2`, then `2->0`.

Graphs are often used to represent transportation maps, where nodes represent individual locations, and edges represent a path you can take to go from one node to another.

A `Graph` instance is constructed from a number of nodes and begins with no edges. Edges are added, and then the `destinations` method lists destinations for a node and the `has_path` method determines if a path exists between two nodes.

```
>>> g = Graph(6)       # A graph with 6 nodes and no edges.
>>> g.add_edge(0, 2)   # Add an edge from node 0 (source) to node 2 (destination)
>>> g.add_edge(2, 0)   # Add an edge from node 2 (source) to node 0 (destination)
>>> g.add_edge(0, 1)
>>> g.add_edge(1, 4)
>>> g.add_edge(4, 2)
>>> g.add_edge(5, 4)
>>> g.add_edge(5, 5)
```

```
>>> g.destinations(0)
[1, 2]
>>> g.destinations(5)
[4, 5]
>>> g.destinations(3)
[]
>>> g.has_path(0,2)  # 0 -> 2
True
>>> g.has_path(5,1)  # 5 -> 4 -> 2 -> 0 -> 1
True
>>> g.has_path(3,3)  # 3
True
>>> g.has_path(1,5)  # No path exists
False
```

(a) (2.0 points)

Implement the `destinations` method of the `Graph` class.

```
class Graph:
    "A graph."

    def __init__(self, nodes):
        self.nodes = nodes
        self.edges = []

    def add_edge(self, source, destination):
        "Add an edge from source to destination."
        if source >= 0 and source < self.nodes:
            if destination >= 0 and destination < self.nodes:
                if (source, destination) not in self.edges:
                    self.edges.append((source, destination))

    def destinations(self, source):
        "Returns a list of all destinations of the given node."
        assert source >= 0 and source < self.nodes
        return ___(a)___

    def has_path(self, source, destination):
        "Returns True if a path exists from source to destination, and False otherwise."
        ...  # The implementation of this method is omitted, but assume it works.
```

i. (2.0 pt) Fill in blank (a).

```
[i[1] for i in self.edges if i[0] == source]
```

(b) (6.0 points)

The function `compose_path` takes in a list of functions `funcs` and three non-negative integers, `x`, `y`, and `maxval`. It returns whether there exists a list of numbers `s` such that:

- `s` begins with `x` and ends with `y`.
- `s[i+1] = f(s[i])` for some `f` in `funcs`, for every adjacent pair of elements `s[i]` and `s[i+1]`.
- `0 <= s[i]` and `s[i] < maxval` for all elements `s[i]`.

Assume `x` and `y` are less than `maxval` and that all elements of `funcs` are one-argument functions that take in an integer and output an integer. Complete the implementation of `compose_path`. (**Hint**: Treat each function call f(x) as "moving" from x to f(x).)

```
def compose_path(funcs, x, y, maxval):
    """
    >>> f, g, h = lambda x: 2*x-1, lambda x: x*x+1, lambda x: x-4
    >>> #3->10->6->11->122->118->114->110->106->102->98
    >>> #  g   h  f   g    h    h    h    h    h    h
    >>> compose_path([f, g, h], 3, 98, 1000)
    True
    >>> #The above path hits 122, and no other path exists to get from 3 to 98
    >>> compose_path([f, g, h], 3, 98, 122)
    False
    >>> [i for i in range(100) if compose_path([h], 10, i, 100)]
    [2, 6, 10]
    """
    g = Graph(___(a)___)
    for f in funcs:
        ___(b)___:
            ___(c)___
    ___(d)___
```

i. (1.0 pt) Fill in blank (a).

```
maxval
```

ii. (1.0 pt) Fill in blank (b).

```
for j in range(maxval)
```

iii. (2.0 pt) Fill in blank (c).

```
g.add_edge(j, f(j))
```

iv. (1.0 pt) Fill in blank (d).

`return g.has_path(x, y)` **The graph contains all allowed transitions j -> f(j) after iterating through all `f` and all `j`. Therefore, `has_path` returns whether a sequence of allowed transitions exists.**

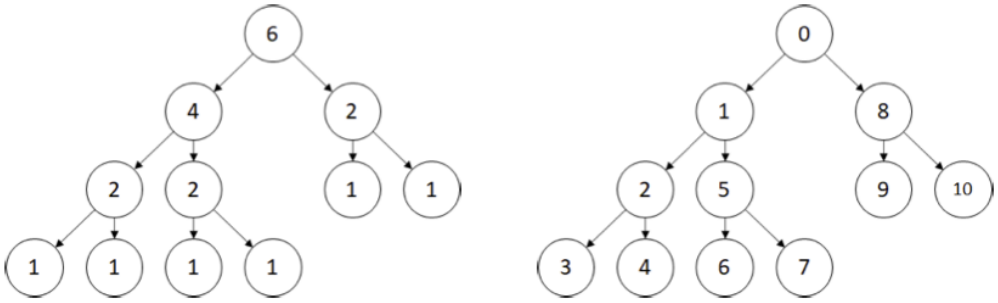
7. (0.0 points) A+ Question 2: Deforestation

This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Implement the function `tree_to_graph`, which takes as input a Tree instance. It returns a Graph instance (defined in the previous question) corresponding to the tree, with nodes ordered in pre-order starting with 0. Pre-order is defined as:

- The root node is assigned the lowest value unassigned.
- After this, the first branch of the tree is assigned values according to pre-order.
- Then, the second branch of the tree is assigned values, and so on.

For example, calling `tree_to_graph` on this `Tree` instance to the left should create this `Graph` instance to the right.




```
def tree_to_graph(tree):
    def helper(tree, op1, op2):
        a = op1()
        return op2(a, [helper(b, op1, op2) for b in tree.branches]) or a
    num_nodes = helper(tree, ___(a)___, ___(b)___)
    g = Graph(num_nodes)
    b = iter(range(num_nodes))
    helper(tree, ___(c)___,___(d)___)
    return g
```

(a) (0.0 pt) Fill in blank (a).

```
lambda: 1
```

(b) (0.0 pt) Fill in blank (b).

```
lambda a, b: a+sum(b)
```

(c) (0.0 pt) Fill in blank (c).

```
lambda: next(b)
```

(d) (0.0 pt) Fill in blank (d).

```
lambda a, b: any([g.add_edge(a,i) for i in b])
```

See the following page for an explanation.

8. (0.0 points) OPTIONAL

(a) (0.0 pt) Draw a picture related to CS 61A or write a program about CS 61A.

A+ question explanation: There are three parts to this question:

- **Determining the number of nodes in the tree (blank a and b)**
- **Creating a graph with the given number of nodes (blank c)**
- **Adding the appropriate edges to the graph (blank d and e)**

Step 2 can be done fairly easily by calling `Graph(numnodes)`
For Steps 1 and 3, the tricky part is that the requested behavior must be achieved solely through the two lambda function inputs of the helper.
Step 1 can be done by using `op1` as a lambda that returns 1 and `op2` as a function to add the results of all branches
Step 3 is trickier, but can be done if we make `op1` return the ID of the graph node corresponding to the current tree node. As it turns out, the pre-order definition perfectly matches assigning IDs in numeric order (using the traversal order given in the helper function), so we can use the iterator `b` to perform this. `op2` is then set to run `add_edge` for each element of the branches, and return a falsey value so that we output a. Note that the solution provides must use `any` instead of `all`, in order to return False on leaves.

No more questions.

CS 61A Fall 2023

Structure and Interpretation of Computer Programs

Midterm 1 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (6.0 points) What Would Python Display?

Assume the following code has been executed already.

```
cat = 3
def dog():
    return cat
def hog():
    cat = 4
    print(cat, dog())
cat = 5
```

For each expression below, write the output **displayed by the interactive Python interpreter** when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. To display a function value, write "Function".

(a) (1.0 pt) `dog(cat)`

- ○ 3
- ○ 4
- ○ 5
- ○ Function
- ● Error

The function `dog` is defined to take no arguments, but this call expression passes an argument.

(b) (1.0 pt) `dog()`

- ○ 3
- ○ 4
- ● 5
- ○ Function
- ○ Error

The name cat is in the global frame. It is assigned to 5, which overwrites the assignment to 3.

Some might answer 3, as cat is initially bound to 3. But since `dog()` is being called after `cat` is bound to 5 and the body of `dog` is only evaluated when the function is called, it will return 5.

(c) (2.0 pt) `print((lambda dog: print(dog()))(lambda: 6))`

```
6
None
```

The entire expression is a call expression. The first lambda expression defines a function with an argument named `dog` and prints the result of calling dog with no arguments. That function is then called on a function defined by the second lambda which takes no arguments and always returns 6.

Thus, the result of calling the function `lambda: 6` is printed.

(d) (2.0 pt) `hog()`

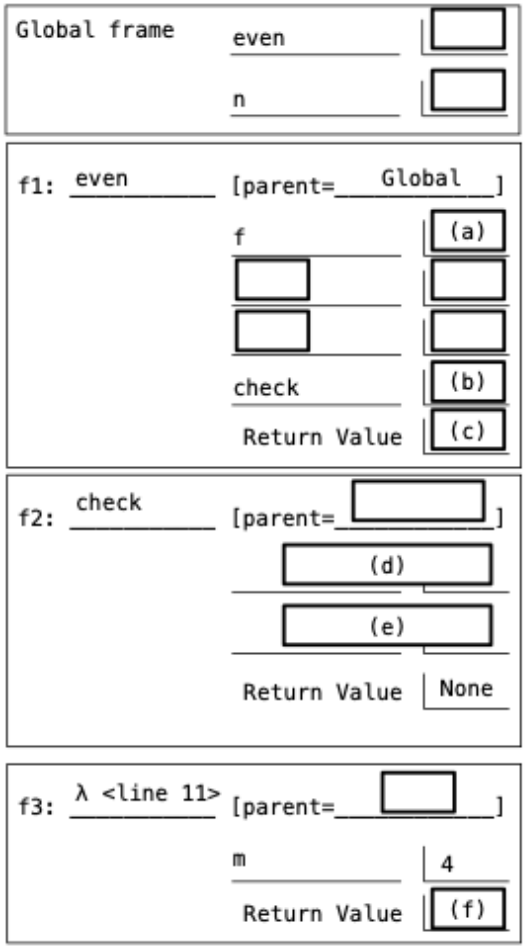
```
4 5
```

Two values are passed to `print`, the value of `cat` which appears in the local frame and is 4, as well as the result of calling `dog()` which is 5.

A key thing to note is that the environment of the call to dog() does not include the frame from the call to hog().

2. (8.0 points) An Odd Implementation of Even

Complete the environment diagram below and then answer the questions that follow. There is one question for each labeled blank in the diagram. The blanks with no labels have no questions associated with them and are not scored. **Some blanks may be empty or unused. If a blank contains an arrow to a function, write the function as it would appear in the diagram. Do not add frames for calls to built-in functions.**




```
 1: def even(f, n):
 2:     "Return whether f(n) is even."
 3:     even = False
 4:     def check(h):
 5:         if h(n) % 2 == 0:
 6:             even = True
 7:     check(f)
 8:     return even
 9:
10: n = 2
11: even(lambda m: m + n, 4)
```

(a) (1.0 pt) Fill in blank (a).

- ● `func lambda(m) <line 11> [parent=Global]`
- ○ `func lambda(m) <line 11> [parent=f1]`
- ○ `func lambda(m, n) <line 11> [parent=Global]`
- ○ `func lambda(m, n) <line 11> [parent=f1]`

`lambda m: m + n` is not defined within another function definition. It is evaluated in the global frame.

(b) (1.0 pt) Fill in blank (b).

- ○ `func check(f) [parent=Global]`
- ○ `func check(f) [parent=f1]`
- ○ `func check(h) [parent=Global]`
- ● `func check(h) [parent=f1]`

The function `check` is defined in the call to `even`, and thus its parent frame is the `even` frame.

(c) (1.0 pt) Fill in blank (c).

- ○ `None`
- ○ A function named `check`
- ○ `True`
- ● `False`
- ○ Blank: an error occurs before a value is returned.

The name `even` is still bound to `False` because the assignment statement in line 6 makes a change in the `check` frame (f2), not the `even` frame (f1).

(d) (1.0 pt) Fill in blank (d).

- ○ `f` is bound to a lambda function.
- ● `h` is bound to a lambda function.
- ○ `f` is bound to a number.
- ○ `h` is bound to a number.

(e) (1.0 pt) Fill in blank (e).

- ○ `n` is bound to a number.
- ○ `m` is bound to a number.
- ○ `check` is bound to `True`
- ○ `check` is bound to `False`
- ● `even` is bound to `True`
- ○ `even` is bound to `False`
- ○ Blank: there is no name-value binding here.

Assignment always changes the current frame, and so line 6 adds a binding from `even` to `True` here.

(f) (1.0 pt) Fill in blank (f).

- ○ 2
- ○ 4
- ● 6
- ○ 8
- ○ Blank: an error occurs before a value is returned.

The lambda frame's parent is the lambda function's parent, which is the global frame. Therefore, `n` evaluates to 2.

(g) (2.0 pt) What problems are there in the `even` function's implementation? **Select all that apply.**

- ☐ It does not call `f` on `n` for its original arguments `f` and `n`.
- ☐ It does not correctly test whether `f(n)` is an even number.
- ☐ It always stops without returning due to an error.
- ■ It always returns `False` regardless of the value of `f(n)`.
- ☐ None of the above, but there is some other error in the implementation.
- ☐ None of the above because there is no error in the implementation.

Since `even` in the `even` frame is never reassigned, the `even` function will always return `False`.

3. (11.0 points) In Your Prime

Definition: The *prime gap* of a prime number `p` is the difference `q-p` between `p` and the next larger prime `q`.

Assume a function `is_prime` is implemented that takes a positive integer and returns `True` if it is prime and `False` otherwise. You may call `is_prime` in your implementations below.

```
def is_prime(n):
    """Return whether positive integer n is a prime number."""
```

(a) (6.0 points)

Implement `smallest_gap`, which takes a positive integer `k`. It returns the smallest prime number that has a prime gap greater than or equal to `k`.

```
def smallest_gap(k):
    """Return the smallest prime number with prime gap greater than or equal to k.

    >>> smallest_gap(2) # 5 - 3 = 2
    3
    >>> smallest_gap(6) # 29 - 23 = 6
    23
    >>> smallest_gap(9) # 127 - 113 = 14, and no prime below 113 has a gap above 8
    113
    """
    p, q = 2, 3  # The two smallest prime numbers
    while _______:
            (a)
        p, q = _______
                 (b)
        while _______:
                (c)
            q = q + 1
    return p
```

i. (2.0 pt) Fill in blank (a).

- ○ `q - p == k`
- ○ `q - p != k`
- ○ `q - p >= k`
- ○ `q - p <= k`
- ○ `q - p > k`
- ● `q - p < k`

The while condition expresses when to keep looking.

ii. (2.0 pt) Fill in blank (b).

```
q, q + 1
```

This sets p to be the next prime q, and then initializes q to be the next number so that we can search for the next prime in the following while loop.

iii. (2.0 pt) Fill in blank (c). **You may not use `and` or `or`.**

```
not is_prime(q)
```

Keep incrementing q until it is prime.

(b) (5.0 points)

Implement `plus_one`, a function that takes two different positive integers `a` and `b`. If the larger one is prime, it returns one more than the smaller one. Otherwise, it returns one more than the average of `a` and `b`.

```
def plus_one(a, b):
    """Return one more than either the smaller (if larger is prime) or average of a and b.

    >>> plus_one(5, 7) # 7 is prime, so return 1 more than 5
    6
    >>> plus_one(15, 7) # 15 is not prime, so return 1 more than the average of 15 and 7
    12.0
    """
    assert a > 0 and b > 0 and a != b

    if _______:
         (d)

        f = _______
              (e)

    else:

        f = _______
              (f)

    return f(a, b) + 1
```

i. (2.0 pt) Fill in blank (d).

```
is_prime(max(a, b))
```

ii. (1.0 pt) Fill in blank (e).

- ○ `max`
- ● `min`
- ○ `max()`
- ○ `min()`
- ○ `max(a, b)`
- ○ `min(a, b)`

Since the return statement calls the function f, one should assign a function to f. It should be min in this case according to the spec as the return statement adds 1 to the min of a,b if f is min.

iii. (2.0 pt) Fill in blank (f).

```
lambda a, b: (a+b)/2
```

Since there is no built-in averaging function, one must be defined.

4. (15.0 points) Choose Wisely

Definition: A *digit test* is a function that takes a non-negative integer less than 10 and returns `True` or `False`.

(a) (9.0 points)

Implement `only` which takes a non-negative integer `n` and a *digit test* `t`. It returns a non-negative integer containing only the digits of `n` for which `t` returns `True` when called on each of those digits. The digits should appear in the same order as they did in `n`. The number 0 has no digits. **You may not use `str` or `repr` or `[` or `]` or `for`.**

```
def only(n, t):
    """Return only the digits of n for which t returns True when called on each digit.

    >>> only(23344567, lambda d: d % 2 == 0)
    2446
    >>> only(987654349675, lambda d: d < 7)
    6543465
    >>> only(2023, lambda d: False)
    0
    """
    keep = 0
    while n:
        n, d = n // 10, n % 10

        if _______:
             (a)

            keep = 10 * keep + _______
                                  (b)
    _______:
      (c)

        _______
          (d)
    return n
```

i. (2.0 pt) Fill in blank (a).

```
t(d)
```

Calls the digit test on the current digit to see if one should include that digit.

ii. (1.0 pt) Fill in blank (b).

- (●) `d`
- ( ) `d // 10`
- ( ) `n`
- ( ) `n % 10`
- ( ) `keep`
- ( ) `keep % 10`
- ( ) `keep // 10`

iii. (2.0 pt) Fill in blank (c)

- ○ `if True`
- ○ `if n`
- ○ `if keep`
- ○ `if n > keep`
- ○ `if keep > n`
- ○ `while True`
- ○ `while n`
- ● `while keep`
- ○ `while n > keep`
- ○ `while keep > n`

iv. (4.0 pt) Fill in blank (d) **with an assignment statement**. You are allowed to assign to multiple names.

```
n, keep = n * 10 + keep % 10, keep // 10
```

(b) (6.0 points)

Implement `every` which takes a *digit test* `t` and returns a function `digit` that takes a positive integer `n`. The `digit` function returns whether `t` returns True for every digit of `n`.

```
def every(t):
    """Return a function of n that returns whether t returns True for every digit of n.

    >>> f = every(lambda d: d % 2 == 1)
    >>> f(37511)  # every digit is odd
    True
    >>> f(2023)   # Not every digit is odd
    False
    """
    def digit(n):
        assert n > 0
        while n:
            if _______:
                 (e)
                _______
                  (f)
            n = n // 10
        return _______
                  (g)
    return _______
             (h)
```

i. (3.0 pt) Fill in blank (e).

```
not t(n % 10)
```

ii. (1.0 pt) Fill in blank (f).

```
return False
```

iii. (1.0 pt) Fill in blank (g).

- ○ `every`
- ○ `digit`
- ● `True`
- ○ `False`
- ○ `n > 0`

iv. (1.0 pt) Fill in blank (h).

- ● `digit`
- ○ `digit()`
- ○ `digit(n)`
- ○ `lambda n: digit`

(c) This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Implement `even_odd`, which takes a positive integer `n` that has both even and odd digits. It returns `True` if all of the odd digits of `n` are larger than the last (right-most) even digit of `n`. Otherwise, it returns `False`. You may call `only` and `every`. **You may not use `str` or `repr` or `[` or `]` or `for`.**

```
def even_odd(n):
    """Return whether all odd digits of n are larger than n's last even digit.

    >>> even_odd(8023) # 3 (the only odd digit) is larger than 2 (the last even digit)
    True
    >>> even_odd(5237679) # 3 (an odd digit) is smaller than 6 (the last even digit)
    False
    >>> even_odd(7297679) # All odd digits (7 & 9) are larger than 6 (the last even digit)
    True
    """
    return _______
```

i. Fill in the blank. You may only write one return expression, but if it's long you can use multiple lines.

```
every(lambda d: d % 2 == 0 or d > only(n, lambda d: d % 2 == 0) % 10)(n)
every(lambda d: d > only(n, lambda d: d % 2 == 0) % 10)(only(n, lambda
d: d % 2 == 1))
```

No more questions.

CS 61A Fall 2023

Structure and Interpretation of Computer Programs

Midterm 2 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (4.0 points) What Would Python Display?

Assume the following code has been executed.

```
s = range(3, 7)
t = iter(s)
u = map(lambda x: 2 * x, t)
v = [next(t), next(t)]      # This line does not cause an error
```

Choose the output **displayed by the interactive Python interpreter** when each expression below is evaluated or *Error* if an error occurs. These expressions are evaluated **in order** and the value of later expressions may be affected by evaluating previous expressions.

(a) (1.0 pt) `[-k for k in v if k in s]`

- ○ `[]`
- ○ `[3, 4]`
- ● `[-3, -4]`
- ○ *Error*

`v` contains 3 and 4 (the first two elements of the range 3 through 7), and both are in the range.

(b) (1.0 pt) `tuple(u)`

- ○ `(3, 4, 5, 6)`
- ○ `(3, 4, 5, 6, 7)`
- ○ `(6, 8, 10, 12)`
- ○ `(6, 8, 10, 12, 14)`
- ● `(10, 12)`
- ○ `(10, 12, 14)`
- ○ *Error*

The map object does not apply its function until `tuple` is called. Making `v` already iterated over 3 and 4. Thus, the map will begin at 5. The iterator `t` stops after 6.

(c) (1.0 pt) `next(t)`

- ○ `3`
- ○ `5`
- ○ `range(3, 7)`
- ○ `range(5, 7)`
- ● *Error*

The `tuple` call used up the values in `t`. `StopIteration` occurs.

(d) (1.0 pt) `next(iter(s)) + next(iter(s))`

- (●) `6`
- ( ) `7`
- ( ) `10`
- ( ) `11`
- ( ) *Error*

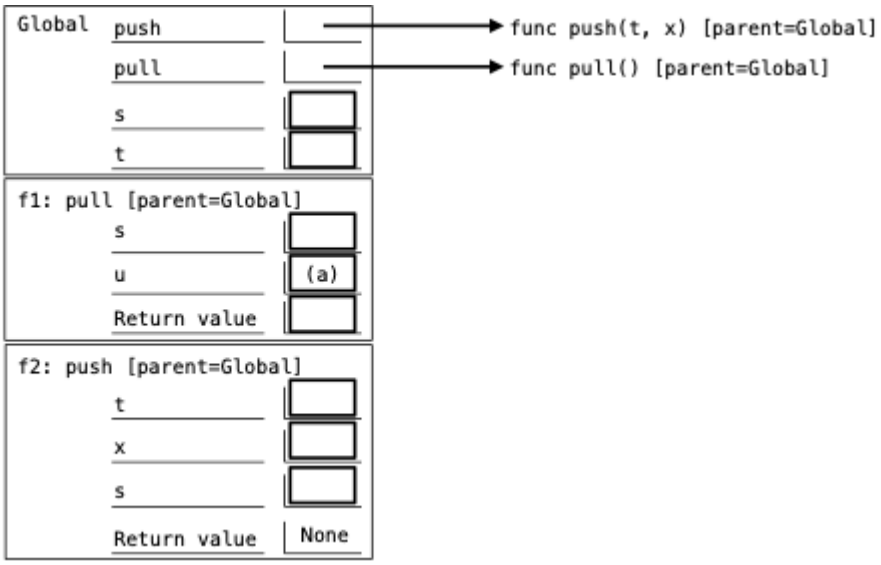
One makes a new iterator out of the iterable, range(3,7) in each call, and thus each call to next yields a 3.

2. (6.0 points) Making a List, Checking it Twice

Complete the environment diagram below and then answer the questions that follow. There is one question for each labeled blank in the diagram. The blanks with no labels have no questions associated with them and are not scored. **Some blanks may be empty or unused. If a blank contains an arrow to a function, write the function as it would appear in the diagram. Do not add frames for calls to built-in functions.**

Reminder: `list(s)` creates a new list with the same elements as `s`; `s.pop()` removes and returns the last element of `s`.

```
def push(t, x):
    s = list(t)
    s[1].append(x)

def pull():
    s = list(t)
    u = s.pop()
    return u[1][0]

s = [7, [8, 9]]
t = [6, s]

push(s, pull())
print(t)
```



(a) (3.0 pt) What value fills blank (a)? Select **all** that apply.

- ☐ `None`
- ■ A list
- ☐ A number
- ■ The object bound to global `s`
- ☐ The object bound to global `t`
- ☐ The same object that `s[0]` would evaluate to in the global frame
- ☐ The same object that `s[1]` would evaluate to in the global frame
- ☐ The same object that `t[0]` would evaluate to in the global frame
- ■ The same object that `t[1]` would evaluate to in the global frame

(b) (3.0 pt) What would be printed by the expression `print(t)` at the end?

```
[6, [7, [8, 9, 8]]]
```

3. (6.0 points) 24-Hour Library

Anyone can check out a book from a library and then bring that book back. Implement the `Library` and `Book` classes. A `Library` takes a list of unique strings called `titles` and constructs a `books` dictionary that has strings as keys and `Book` objects as values. Its `checkout` method takes a string `title` and returns the corresponding `Book` object if that `title` is not checked out, or prints a message. After `bring_back` is invoked on that book, it can be checked out again.

```
class Library:
    """A library with one copy of each title that can be checked out.

    >>> cs = Library(['Composing Programs', 'Python Docs', 'Berkeley Academic Guide'])
    >>> bs = [cs.checkout('Composing Programs'), cs.checkout('Python Docs')]
    >>> cs.checkout('Composing Programs')        # This time, no Book is returned
    Composing Programs is checked out
    >>> bs[0].bring_back()
    >>> cs.checkout('Composing Programs').title  # This time, a Book is returned
    'Composing Programs'
    """
    def __init__(self, titles):
        self.books = {t: Book(t, _______) for t in titles}
                                   (a)
        self.out = []  # A list of Book objects

    def checkout(self, title):
        assert title in self.books, title + " isn't in this library's collection"
        book = _______
                 (b)
        if book not in self.out:
            _______
              (c)
            return book
        else:
            print(book, 'is checked out')

class Book:
    def __init__(self, title, library):
        self.title = title      # a string
        self.library = library  # a Library object

    def bring_back(self):
        _______.remove(_______)
          (d)            (e)

    def __str__(self):
        return _______
                 (f)
```

(a) **(1.0 pt)** Fill in blank (a).

- (●) `self`
- ( ) `self.books`
- ( ) `Library`
- ( ) `Library()`
- ( ) `Library(titles)`

The book takes a library. The libary that the book is in is the ‘self“ object that is being created with this dictionary of books.

(b) **(1.0 pt)** Fill in blank (b).

```
self.books[title]
```

Get the relevant book from the dictionary of books.

(c) **(1.0 pt)** Fill in blank (c).

- ( ) `out.append(book)`
- ( ) `out.extend(book)`
- ( ) `Library.append(book)`
- ( ) `Library.extend(book)`
- (●) `self.out.append(book)`
- ( ) `self.out.extend(book)`

We wish to add the book to the list of checked out books. Append adds an element to a list. (Extend extends a list with another list.)

(d) **(1.0 pt)** Fill in blank (d).

- ( ) `self.`
- ( ) `self.out`
- ( ) `library`
- ( ) `library.out`
- (●) `self.library.out`

Remove the book from the list of checked out books.

(e) **(1.0 pt)** Fill in blank (e).

- (●) `self`
- ( ) `title`
- ( ) `library`
- ( ) `self.title`
- ( ) `self.library`

The name `self` is bound to the book we’re removing.

(f) (1.0 pt) Fill in blank (f).

- ○ `self`
- ○ `title`
- ● `self.title`
- ○ `repr(self)`
- ○ `repr(title)`
- ○ `repr(self.title)`

One needs to return a string with the title in it. The repr would return a string with quotes in it.

4. (16.0 points) A Perfect Question

Definition. A *perfect square* is `k*k` for some integer `k`.

(a) (7.0 points)

Implement `fit`, which takes **positive** integers `total` and `n`. It returns `True` or `False` indicating whether there are `n` **positive** perfect squares that sum to `total`. The perfect squares need not be unique.

```
def fit(total, n):
    """Return whether there are n positive perfect squares that sums to total.

    >>> [fit(4, 1), fit(4, 2), fit(4, 3), fit(4, 4)]  # 1*(2*2) for n=1; 4*(1*1) for n=4
    [True, False, False, True]
    >>> [fit(12, n) for n in range(3, 8)]  # 3*(2*2), 3*(1*1)+3*3, 4*(1*1)+2*(2*2)
    [True, True, False, True, False]
    >>> [fit(32, 2), fit(32, 3), fit(32, 4), fit(32, 5)] # 2*(4*4), 3*(1*1)+2*2+5*5
    [True, False, False, True]
    """
    def f(total, n, k):

        if _______:
             (a)
            return True

        elif _______:
               (b)
            return False
        else:
            return _______
                     (c)

    return f(total, n, 1)
```

i. (2.0 pt) Select **all** of the options that could fill blank (a).

- ☐ `total == 0`
- ☐ `n == k`
- ■ `total == 0 and n == 0`
- ☐ `total == k * k`
- ■ `total == k * k and n == 1`
- ■ `total == n * k * k`

The final condition is ideal, but the other two would still provide the correct answer after additional recursive calls.

ii. (2.0 pt) Fill in blank (b).

```
total < k * k
```

At this point, one can only use terms that are larger than `k*k` and thus there can't be a solution if total is smaller than that.

iii. (3.0 pt) Fill in blank (c) with an expression of the form `f(___, ___, ___) ___ f(___, ___, ___)`.

```
f(total - k * k, n - 1, k) or f(total, n, k + 1)
```

You either use `k*k` at least once or you only use larger perfect squares.

(b) (9.0 points)

Implement the generator function `squares`, which takes **positive** integers `total` and `k`. It yields all lists of perfect squares greater or equal to `k*k` that sum to `total`. Each list is in non-increasing order (large to small).

```
def squares(total, k):
    """Yield the ways in which perfect squares greater or equal to k*k sum to total.

    >>> list(squares(10, 1))  # All lists of perfect squares that sum to 10
    [[1, 1, 1, 1, 1, 1, 1, 1, 1, 1], [4, 1, 1, 1, 1, 1, 1], [4, 4, 1, 1], [9, 1]]
    >>> list(squares(20, 2))  # Only use perfect squares greater or equal to 4 (2*2).
    [[4, 4, 4, 4, 4], [16, 4]]
    """
    assert total > 0 and k > 0
    if total == k * k:

        yield _______
                (d)
    elif total > k * k:

        for s in _______:
                    (e)

            yield _______
                    (f)
        yield from squares(total, k + 1)
```

i. (2.0 pt) Fill in blank (d).

```
[k * k]
```

ii. (3.0 pt) Fill in blank (e).

```
squares(total - k * k, k)
```

This gives a generator that yields solutions whose total area is `total - k*k`. And adding one more square of `k*k` gives a solution for `total`.

iii. (1.0 pt) Select **all** of the options that could fill in blank (f).

- ☐ `s + k*k`
- ☐ `s.append(k*k)`
- ■ `s + [k*k]`
- ☐ `[s] + [k*k]`

Each `s` is a list whose elements add to `total - k*k`, and adding `k*k` gives a desired solution.

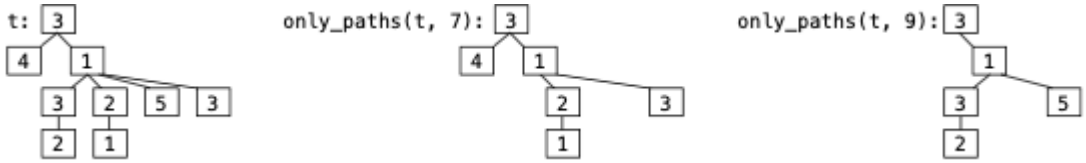
iv. (3.0 pt) Write an expression containing `a` that evaluates to the **shortest** list of perfect squares that sum to an integer `a`. For example, if `a = 32`, your expression should evaluate to `[16, 16]` (not `[25, 4, 1, 1, 1]`). If there are two or more such lists that are both shortest, it can evaluate to any of them. Assume `squares` is implemented correctly.

```
min(squares(a, 1), key=len)
```

5. (7.0 points) Only Paths

Implement `only_paths`, which takes a Tree of numbers `t` and a number `n`. It returns a new Tree with only the nodes of `t` on a path from the root to a leaf with labels that sum to `n`, or `None` if no path sums to `n`. Do not mutate `t`.

The Tree class appears on the midterm 2 study guide. Here is an illustration of the doctest examples involving `t`.




```
def only_paths(t, n):
    """Return a Tree with only the nodes of t along paths from the root to a leaf of t
    for which the node labels of the path sum to n. If no paths sum to n, return None.

    >>> print(only_paths(Tree(5, [Tree(2), Tree(1, [Tree(2)]), Tree(1, [Tree(1)])]), 7))
    5
      2
      1
        1
    >>> t = Tree(3, [Tree(4), Tree(1, [Tree(3, [Tree(2)]), Tree(2, [Tree(1)]), Tree(5), Tree(3)])])
    >>> print(only_paths(t, 7))
    3
      4
      1
        2
          1
        3
    >>> print(only_paths(t, 9))
    3
      1
        3
          2
        5
    >>> print(only_paths(t, 3))
    None
    """
    if _______:
         (a)

        return t

    new_branches = [_______ for b in t.branches]
                       (b)

    if _______(new_branches):
         (c)

        return Tree(t.label, [b for b in new_branches if _______])
                                                           (d)
```

(a) **(2.0 pt)** Fill in blank (a).

```
t.is_leaf() and n == t.label
```

This is a tree with a path to a leaf with sum of labels being n as specified. And no recursive call is needed.

(b) **(3.0 pt)** Fill in blank (b).

```
only_paths(b, n - t.label)
```

The sum of labels along a path to any leaf is the sum in a branch plus the label. Thus, we wish to find a solution for each branch with sum `n-t.label`.

(c) **(1.0 pt)** Fill in blank (c).

- ◯ `all`
- ● `any`
- ◯ `has`
- ◯ `only_paths`

The function returns `None` if a branch has no valid path. Thus, if all the entries of the list of solutions for branches are `None`, the procedure is done. Otherwise it should return a tree.

(d) **(1.0 pt)** Fill in blank (d).

- ◯ `only_paths(b, n)`
- ◯ `b.label == n`
- ◯ `b.label != n`
- ◯ `b is None`
- ● `b is not None`

Only keep the results that are trees, not `None`.

(e) **This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!** Fill in the blank to implement `only_long_paths`, which takes a Tree of numbers `t` and a number `n`. It returns a new Tree containing only the nodes of `t` that lie on a path from the root to a leaf for which the sum of the labels **plus the length of the path** is `n`. Do not mutate `t`. You may not write `and`, `or`, `if`, `[` or `]`. Assume `only_paths` is implemented correctly.

```
only_paths = ______________

def only_long_paths(t, n):
    """Return a Tree with only the nodes of t along paths from the root to a leaf of t
    for which the sum of node labels plus the length of the path is n.

    >>> example = Tree(5, [Tree(3), Tree(1, [Tree(2)]), Tree(1, [Tree(1)])])
    >>> only_long_paths(example, 10)  # Result has paths 5-3 (length 2) and 5-1-1 (length 3)
    Tree(5, [Tree(3), Tree(1, [Tree(1)])])
    """
    return only_paths(t, n)
```

```
(lambda f: lambda t, n: f(t, n-1))(only_paths)
```

Like `memo` from lecture, this lambda function takes a function and returns a similar function. This similar function calls the original `only_paths` on `n-1` rather than `n` to account for the length of the path.

6. (6.0 points) After Party

Implement `after`, which takes a linked list `s` and values `a` and `b`. It returns whether an element of `s` equal to `b` appears after an element of `s` equal to `a`. The `Link` class appears on the Midterm 2 study guide.

```
def after(s, a, b):
    """Return whether b comes after a in linked list s.

    >>> t = Link(3, Link(6, Link(5, Link(4))))
    >>> after(t, 6, 4)
    True
    >>> after(t, 4, 6)
    False
    >>> after(t, 6, 6)
    False
    """
    def find(s, n, f):

        if s == Link.empty:

            return _______
                     (a)

        elif s.first == n:
            return f(s.rest)
        else:
            return find(s.rest, n, f)

    return find(s, a, lambda rest: _______)
                                     (b)
```

(a) (1.0 pt) Fill in blank (a).

- ○ `n`
- ○ True
- ● False
- ○ `a < b`
- ○ `f(Link.empty)`

You have not found n, which is a, yet, and the list is done.

(b) (4.0 pt) Fill in blank (b). **You may not use `or`, `and`, `if`, `[`, or `]`.**

```
find(rest, b, lambda rest: True)
```

At this point, you have found a in the list, so you should see if b is in the rest of the list.

(c) (1.0 pt) What is the order of growth of the run time of an efficient implementation of `after` in terms of the length of `s`? Here, *efficient* means that you have filled in the blanks to give the function the fastest order of growth.

- ◯ Constant
- ◯ Logarithmic
- ● Linear
- ◯ Quadratic
- ◯ Exponential

Each element is considered once since each recursive function call is called on the rest of the current list, giving linear time. In the best case, `a` and `b` could appear at the beginning of the list, which would be found in constant time, so that answer was accepted as well.

No more questions.

CS 61A Fall 2024

Structure and Interpretation of Computer Programs

Midterm 1 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (8.0 points) What Would Python Display?

Answer the following questions about the output printed by this code.

```
def mad(max):
    g = lambda: (print(1) or 2) or (print(3) or 4)
    print(max(5, 6))
    return g
print(mad(print)(), 7, print(8))
```

(a) (2.0 pt) What is the last line printed?

- ○ `1 7 8`
- ○ `2 7 8`
- ● `2 7 None`
- ○ `4 7 8`
- ○ `4 7 None`
- ○ `None 7 8`
- ○ `None 7 None`

(b) (4.0 pt) Which of these **whole lines** appear **somewhere** in the printed output? Select all that apply.

- ■ `1`
- ☐ `1 2`
- ☐ `1 2 3`
- ☐ `1 2 3 4`
- ☐ `3`
- ☐ `5`
- ■ `5 6`
- ☐ `6`
- ☐ `7`
- ■ `8`
- ■ `None`
- ☐ `None None`
- ☐ `True`

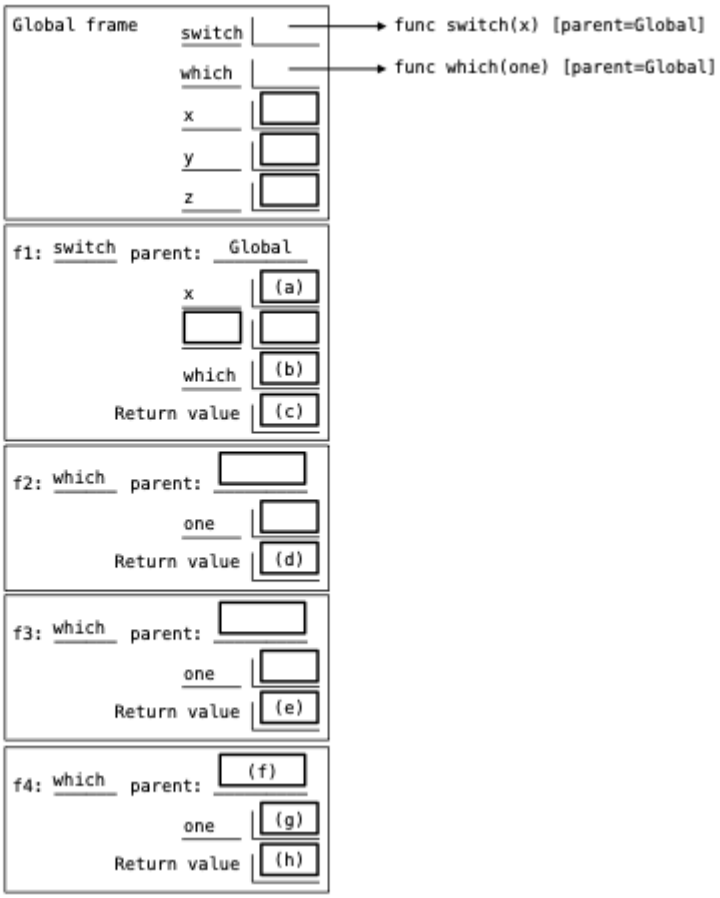
(c) (2.0 pt) What **order** do 1, 6, and 8 appear in the printed output? These numbers may appear as part of longer lines. If a number appears twice, consider just the first occurence.

- ○ `1 6 8`
- ○ `1 8 6`
- ● `6 1 8`
- ○ `6 8 1`
- ○ `8 1 6`
- ○ `8 6 1`

2. (8.0 points) Which One

Complete the environment diagram below and then answer the questions that follow. There is one question for each labeled blank in the diagram. The blanks with no labels have no questions associated with them and are not scored.

```
 1: x, y, z = 1, 2, 3
 2:
 3: def switch(x):
 4:     z = 4
 5:     if x == y:
 6:         def which(one):
 7:             return x + one
 8:     else:
 9:         def which(one):
10:             return x - one
11:     x = which(x)
12:     which(x)
13:
14: def which(one):
15:     return x * one
16:
17: switch(x+1)
18: x = 5
19: which(z)
```



(a) (1.0 pt) Fill in blank (a).

- ○ 0
- ○ 1
- ○ 2
- ○ 3
- ● 4

(b) (1.0 pt) Fill in blank (b).

- ● `func which(one) [parent=f1]`
- ○ `func which(one) [parent=Global]`
- ○ `func which(one) [parent=switch]`

(c) (1.0 pt) Fill in blank (c).

None

(d) (1.0 pt) Fill in blank (d).

- ○ 0
- ○ 1
- ○ 2
- ○ 3
- ● 4

(e) (1.0 pt) Fill in blank (e).

8

(f) (1.0 pt) Fill in blank (f).

- ● Global
- ○ f1
- ○ f2
- ○ f3
- ○ f4

(g) (1.0 pt) Fill in blank (g).

- ○ 1
- ○ 2
- ● 3
- ○ 4
- ○ 5

(h) (1.0 pt) Fill in blank (h).

15

3. (6.0 points) Final Digit

Implement `final_digit`, which takes a non-negative integer `n`. As long as `n` has more than one digit, replace `n` with the sum of the digits of `n`. This process repeats until `n` becomes a single-digit number, which is returned.

```
def final_digit(n):
    """Sum the digits of n repeatedly to reach one digit.

    >>> final_digit(321)       # 3 + 2 + 1 = 6
    6
    >>> final_digit(987)       # 9 + 8 + 7 = 24, and 2 + 4 = 6
    6
    >>> final_digit(989898989) # The digit sum is 77, 7 + 7 = 14, and 1 + 4 = 5
    5
    """
    while n >= 10:
        s = _______
              (a)
        _______:
          (b)
            n, s = n // 10, _______
        _______                 (c)
          (d)
    return n
```

(a) (1.0 pt) Fill in blank (a).

- (●) `0`
- ( ) `n`
- ( ) `n % 10`

(b) (1.0 pt) Fill in blank (b).

- ( ) `if s`
- ( ) `if n`
- ( ) `while s`
- (●) `while n`

(c) (2.0 pt) Fill in blank (c).

- ( ) `n`
- ( ) `s // 10`
- ( ) `n % 10`
- (●) `s + n % 10`
- ( ) `10 * s + n % 10`

(d) (2.0 pt) Fill in blank (d).

```
n = s
```

4. (8.0 points) Close Enough

Implement `close`, which takes two non-negative integers `m` and `n`. It returns whether `m` can be changed into `n` by either inserting one digit, removing one digit, or changing one digit. If `m` and `n` are the same number, they are *not* `close`.

```
def close(m, n):
    """Return whether m can result from starting with n and adding, removing,
    or changing one digit.

    >>> close(3756, 3456) and close(3456, 346) and close(346, 3456) and close(456, 56)
    True
    >>> close(5, 5) or close(3456, 3546) or close(3456, 36) or close(34, 3456) or close(345, 456)
    False
    """
    if m < n:
        m, n = n, m
    while m or n:
        if _______:
             (a)
            m, n = _______
                     (b)
        else:

            return _______ or _______  # Hint: check here that just one change is enough
                     (c)        (d)
    return _______
             (e)
```

(a) (1.0 pt) Fill in blank (a).

- ○ `m == n`
- ○ `m // 10 == n // 10`
- ○ `m // 10 == n`
- ○ `m == n // 10`
- ● `m % 10 == n % 10`

(b) (2.0 pt) Fill in blank (b).

- ○ `n, m`
- ● `m // 10, n // 10`
- ○ `m // 10, n`
- ○ `m, n // 10`
- ○ `m % 10, n % 10`

(c) (2.0 pt) Fill in blank (c).

- ○ `m == n`
- ○ `m - n < 10`
- ○ `n == 0`
- ● `m // 10 == n // 10`
- ○ `m % 10 == n % 10`
- ○ `m % 100 == n % 100`

(d) (2.0 pt) Fill in blank (d).

- ● `m // 10 == n`
- ○ `m // 10 - n < 10`
- ○ `n == 0`
- ○ `m // 10 == 0`
- ○ `m // 100 == n // 10`
- ○ `(m // 10) % 10 == n % 10`

(e) (1.0 pt) Fill in blank (e).

- ○ `m == n`
- ○ `m == 0`
- ○ `n == 0`
- ○ `True`
- ● `False`

5. (10.0 points) Shifty

(a) (4.0 points)

Implement `shift`, which takes a number `k` and a one-argument function `f`. It returns a one-argument function `g` that takes a number `x`. For all numbers `x`, `g(x)` is equal to `f(x + k)`.

```
def shift(k, f):
    """Return a function of x that returns f(x+k).

    >>> square = lambda x: x * x
    >>> g = shift(2, square)
    >>> g(3)     # square(3 + 2)
    25
    """
    return _______
             (a)
```

i. (2.0 pt) Fill in blank (a).

- ○ `f(x) + k`
- ○ `f(x + k)`
- ○ `f(lambda x: x + k)`
- ○ `f(lambda x: x) + k`
- ○ `lambda x: f(x) + k`
- ● `lambda x: f(x + k)`
- ○ `(lambda x: f(x))(x + k)`
- ○ `g(x) + k`
- ○ `g(x + k)`
- ○ `g(lambda x: x + k)`
- ○ `g(lambda x: x) + k`
- ○ `lambda x: g(x) + k`
- ○ `lambda x: g(x + k)`
- ○ `(lambda x: g(x))(x + k)`

ii. (2.0 pt) Provide an alternate solution for blank (a) . This time, your solution **must** call `compose` (defined below), and `f` **may not** be the operator of a call expression. In other words, you can't write `f(` in your answer. You **may not** write `[` or `if`.

```
def compose(f, g):
    """Return a function that takes x and calls f on g of x."""
    return lambda x: f(g(x))
```

```
compose(f, lambda x: x + k)
```

(b) (6.0 points)

Implement `sum_range`, which takes positive integers `p` and `q` with `p <= q`, as well as a one-argument function `term`. It returns the sum of the return values of `term` called on each consecutive integer starting with `p` and ending with `q` (including both `p` and `q`). You may call `shift`, `summation`, and `compose`. Assume `shift` is implemented correctly.

```
def summation(n, term):
    """Sum the first n terms of a sequence: term(1) + term(2) + ... + term(n).

    >>> summation(5, lambda x: x*x)  # 1*1 + 2*2 + 3*3 + 4*4 + 5*5
    55
    """
    total, k = 0, 1
    while k <= n:
        total, k = total + term(k), k + 1
    return total

def sum_range(p, q, term):
    """Sum terms p through q of a sequence: term(p) + term(p+1) + ... + term(q).

    >>> sum_range(1, 5, lambda x: x*x)   # 1*1 + 2*2 + 3*3 + 4*4 + 5*5
    55
    >>> sum_range(4, 5, lambda x: x*x)   # 4*4 + 5*5
    41
    >>> sum_range(5, 5, lambda x: x*x)   # 5*5
    25
    """
    assert p <= q
    return _______ ( _______ , _______ )
             (a)       (b)       (c)
```

i. (1.0 pt) Fill in blank (a).

- (●) `summation`
- ( ) `shift`
- ( ) `compose`
- ( ) `term`

ii. (2.0 pt) Fill in blank (b).

- ( ) `p - q - 1`
- ( ) `p - q`
- ( ) `p - q + 1`
- ( ) `q - p - 1`
- ( ) `q - p`
- (●) `q - p + 1`

iii. (3.0 pt) Fill in blank (c).

```
shift(p - 1, term)
```

(c) (0.0 points)

This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

The `shifter` function below is a curried version of `shift`.

Implement `unshift`, which takes the result of `shifter(k)` for some number `k`. It returns a function that takes the result of `shifter(k)(f)` for some function `f` and returns a function equivalent to `f`. That is:

`f(x) == unshift(shifter(k))(shifter(k)(f))(x)`

You can write your answer on multiple lines if it's long. You can abbreviate `lambda` using the greek symbol lambda.

Your answer **must** be a call to `shift`. You may also call any of `compose`, `summation`, or `sum_range`.

Hint: If you can compute `k`, then you can `shift` backward by `k` to undo the original shift.

```
def shifter(k):
    def shifted(f):
        return shift(k, f)
    return shifted

def unshift(shifted):
    """Assume shifted is the return value of shifter(k) for some k.

    >>> cubic = lambda x: x*x*x - 5*x*x + 1   # Some complicated function
    >>> cubic(2.0)
    -11.0
    >>> cubic(9.0)
    325.0
    >>> do = shifter(4)
    >>> do(cubic)(2.0)        # same as cubic(6.0)
    37.0
    >>> do(cubic)(9.0)        # same as cubic(13.0)
    1353.0
    >>> undo = unshift(do)
    >>> undo(do(cubic))(2.0)  # same as cubic(2.0)
    -11.0
    >>> undo(do(cubic))(9.0)  # same as cubic(9.0)
    325.0
    """
    return lambda g: _______
```

i. (0.0 pt) Fill in the blank with **a single call to** `shift`.

`shift(-shifted(lambda x: x)(0), g)` **-OR-** `shifter(-shifted(lambda x: x)(0))(g)`

No more questions.

CS 61A Fall 2024

Structure and Interpretation of Computer Programs

Midterm 2 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (5.0 points) What Would Python Display?

Assume the following code has been executed. The `Tree` class appears on the midterm 2 study guide (page 2, left side).

```
scare = Tree(0, [Tree(4), Tree(5, [Tree(10)]), Tree(2)])
crow = Tree(9, [scare, Tree(7, [Tree(6)])])

def climb(t, f):
    if t.is_leaf():
        return [t.label]
    return [t.label] + climb(max(t.branches, key=f), f)

def run(t):
    if t.is_leaf():
        return t.label
    else:
        return max(map(run, t.branches))

def hide(t):
    return t.label

jump = {(2 * x - 1): x for x in range(50)}
```

Write the value of each expression below or *Error* if an error occurs.

(a) (1.0 pt) `crow.branches[0].label`

0

(b) (1.0 pt) `[hide(b) for b in scare.branches]`

`[4, 5, 2]`

(c) (1.0 pt) `climb(crow, hide)`

`[9, 7, 6]`

(d) (1.0 pt) `climb(crow, run)`

`[9, 0, 5, 10]`

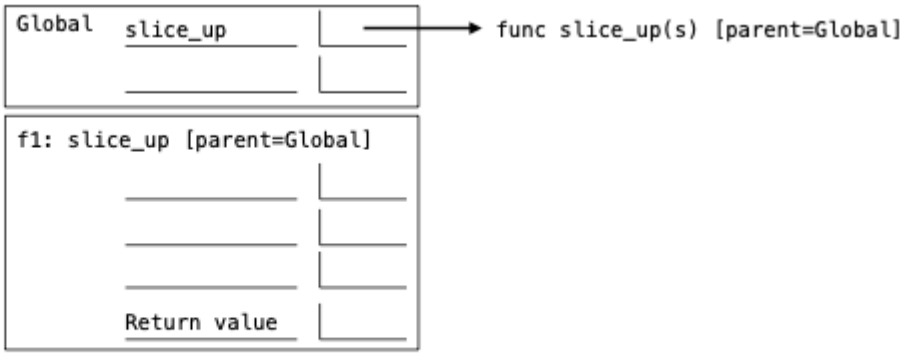
(e) (1.0 pt) `jump[jump[13]]`

4

2. (5.0 points) Pizza by the Slice

Answer the questions about the code below. You are encouraged to draw an environment diagram, but the diagram itself will not be scored.

```
 1: def slice_up(s):
 2:     copy = s
 3:     t = [copy]
 4:     while len(s) > 1:
 5:         t.append(s[:len(s)-1])
 6:         s.pop()
 7:         s[0] = len(s)
 8:     return t
 9:
10: t = [3, 4, 5]
11: print(slice_up(t))
12: print(t)
```



(a) (3.0 pt) What would be printed by the expression `print(slice_up(t))` on line 11?

```
[[1], [3, 4], [2]]
```

(b) (1.0 pt) What would be printed by the expression `print(t)` on line 12?

- (●) `[1]`
- ( ) `[2]`
- ( ) `[3]`
- ( ) `[1, 4]`
- ( ) `[2, 4]`
- ( ) `[3, 4]`
- ( ) `[1, 4, 5]`
- ( ) `[2, 4, 5]`
- ( ) `[3, 4, 5]`

(c) (1.0 pt) What is the order of growth of the time it takes to evaluate `list(list(q))` for a list of numbers `q` in terms of the length of `q`. Assume that creating a list with one element takes constant time, and making a copy of a list takes linear time in the length of the list.

- ( ) exponential
- ( ) quadratic
- (●) linear
- ( ) logarithmic
- ( ) constant

3. (6.0 points) CS 61A Software Store

A `Store` instance has a list of `branches`, and each branch is also a `Store` instance, forming a tree of stores. Each `Store` instance also has a list of programs (strings) called its `inventory`. For a `Store`:

- The `copies(s)` method returns the number of times `s` appears in the inventories of all nodes in its tree of stores.
- The `add_to(k)` method returns a function that takes a string and appends it to the `inventory` of its `k`th branch. Assume that `k` is non-negative and less than the number of branches.

```
class Store:
    """A Store selling programs has branches forming a tree of Stores.
    >>> north, south = Store(['Cats', 'Ants']), Store(['Cats', 'Cats', 'Hog'])
    >>> east, west = Store(['Cats', 'Hog', 'Hog'], [north, south]), Store(['Cats', 'Cats', 'Ants'])
    >>> main = Store(['Ants', 'Ants', 'Hog'], [east, west])
    >>> east.copies('Cats')  # 1 in north, 2 in south, 1 in east
    4
    >>> main.copies('Ants')  # 2 in main, 1 in north, 1 in west
    4
    >>> main.add_to(1)('Ants') # Add 'Ants' to the inventory of west, the branch of main at index 1
    >>> [main.copies('Ants'), west.copies('Ants')]  # increased copies in both main and west
    [5, 2]
    """
    def __init__(self, programs, branches=[]):
        assert all([isinstance(b, Store) for b in branches])
        self.branches = branches
        self.inventory = programs

    def copies(self, s):
        """Return the number of times s (string) appears in all inventories of this tree."""
        return sum([_______ for p in self.inventory if p == s] + [_______ for b in self.branches])
                      (a)                                                    (b)

    def add_to(self, k):
        """Return a function that appends a string to the inventory of the branch at index k."""
        return _______
                 (c)
```

(a) (1.0 pt) Fill in blank (a).

- (●) `1`
- ( ) `p`
- ( ) `len(p)`
- ( ) `len(self.inventory)`

(b) (2.0 pt) Fill in blank (b).

`b.copies(s)`

(c) (3.0 pt) Fill in blank (c).

```
self.branches[k].inventory.append
```

4. (18.0 points) Almost a Perfect Question

Definition. A *proper divisor* of an integer `n` is an integer `d` less than `n` that evenly divides `n`, and so `n` divided by `d` has no remainder. A *semiperfect* number is a positive integer that is the sum of some (or all) of its proper divisors. For example, $24 = 2 + 4 + 6 + 12$ and so it is semiperfect. The sum of divisors cannot contain repeated numbers.

(a) (6.0 points)

Implement `semiperfect`, which takes a positive integer `n`. It returns `True` if `n` is semiperfect and `False` otherwise.

```
def semiperfect(n):
    """Return whether positive integer n is a sum of some (or all) of its proper divisors.
    >>> [k for k in range(1, 40) if semiperfect(k)]
    [6, 12, 18, 20, 24, 28, 30, 36]
    """
    def f(s, d):
        if _______:
             (a)
            return True
        if d >= n:
            return False
        if _______ and f(_______, d + 1):
             (b)           (c)
            return True
        return _______
                  (d)
    return f(n, 1)
```

i. (1.0 pt) Fill in blank (a).

- ◯ `n == 0`
- ● `s == 0`
- ◯ `s == d`
- ◯ `n == d`

ii. (1.0 pt) Fill in blank (b).

- ● `n % d == 0`
- ◯ `s % d == 0`
- ◯ `n % d > 0`
- ◯ `s % d > 0`

iii. (2.0 pt) Fill in blank (c).

```
s - d
```

iv. (2.0 pt) Fill in blank (d).

```
f(s, d + 1)
```

(b) (6.0 points)

Implement `subsums`, a generator function that takes a list of **unique** (no repeats) positive integers `s` and a positive integer `n`. It yields all sublists of `s` that sum to `n`. A sublist of `s` is a list containing some of the elements of `s` in order. The `subsums` function should not yield the same list twice. The lists may appear in any order.

```
def subsums(s, n):
    """Yield all sublists of s that sum to n.
    >>> list(subsums([1, 2, 3, 6, 9], 18))
    [[1, 2, 6, 9], [3, 6, 9]]
    >>> list(subsums([1, 2, 3, 4, 6], 16))
    [[1, 2, 3, 4, 6]]
    >>> list(subsums([1, 2, 3, 4, 6, 8, 12], 12))
    [[1, 2, 3, 6], [1, 3, 8], [2, 4, 6], [4, 8], [12]]
    """
    if s:
        if _______:
             (e)
            yield [n]
        for t in _______:
                    (f)
            yield _______
                     (g)
        yield from _______
                     (h)
```

i. (1.0 pt) Fill in blank (e).

- ◯ `n == s`
- ● `n == s[0]`
- ◯ `n in s`
- ◯ `[n] == s`

ii. (3.0 pt) Fill in blank (f).

```
subsums(s[1:], n-s[0])
```

iii. (1.0 pt) Fill in blank (g).

- ◯ `s + t`
- ◯ `[s] + t`
- ◯ `s[1:] + t`
- ● `s[:1] + t`

iv. (1.0 pt) Fill in blank (h).

- ○ `subsums(s, n)`
- ● `subsums(s[1:], n)`
- ○ `subsums(s, n - 1)`
- ○ `subsums(s[1:], n - 1)`

(c) (6.0 points)

Implement `semisums`, which takes a positive integer `n`. It returns a list of all lists of unique (no repeats) proper divisors of `n` that sum to `n`. You may call `semiperfect` and `subsums`.

```
def semisums(n):
    """Return a list of all lists (with no repeats) of
    proper divisors of n that sum to n.

    >>> semisums(22)  # 22 is not semiperfect, so there are no sums.
    []
    >>> semisums(30)  # 30 is semiperfect. It has three different sums.
    [[1, 3, 5, 6, 15], [2, 3, 10, 15], [5, 10, 15]]
    """
    return list( _______ ([k for k in range(1, n) if _______ ], _______ ))
                  (i)                                   (j)        (k)
```

i. (1.0 pt) Fill in blank (i).

- ○ `map`
- ○ `filter`
- ○ `sorted`
- ● `subsums`
- ○ `semiperfect`

ii. (1.0 pt) Fill in blank (j).

- ○ `semiperfect(n)`
- ○ `semiperfect(k)`
- ○ `semiperfect(n) or semiperfect(k)`
- ● `n % k == 0`

iii. (1.0 pt) Fill in blank (k).

```
n
```

iv. (3.0 pt) Fill in blank (l) of `primitive_semiperfect`, a function that takes a positive integer `n`. It returns `True` if `n` is semiperfect and no smaller semiperfect number is a divisor of `n`. Your answer should have two expressions separated by a comma.

```
def primitive_semiperfect(n):
    """Return whether n is semiperfect and has no semiperfect proper divisors.

    >>> [k for k in range(1, 300) if primitive_semiperfect(k)]
    [6, 20, 28, 88, 104, 272]
    """
    return semiperfect(n) and not any(map(semiperfect, filter(_______)))
                                                              (l)
```

```
lambda k: n % k == 0, range(1, n)
```

5. (11.0 points) How Long is this Exam?

(a) (5.0 points)

Implement `longer`, which takes two linked lists of numbers `s` and `t`. It returns the longer of the two. If they have the same length, it returns `s`.

A linked list is either a `Link` instance or `Link.empty`. The `Link` class is on page 2 of the midterm 2 study guide.

```
def longer(s, t):
    """Return the longer linked list, s or t. (Same length? return s.)

    >>> longer(Link(2, Link(3)), Link.empty)
    Link(2, Link(3))
    >>> longer(Link(2, Link(3)), Link(4, Link(5)))
    Link(2, Link(3))
    >>> longer(Link(2, Link(3)), Link(4, Link(5, Link(6, Link(7)))))
    Link(4, Link(5, Link(6, Link(7))))
    >>> longer(Link.empty, Link.empty) is Link.empty
    True
    """
    a, b = s, t

    while b is not Link.empty:

        if _______:
             (a)

            return _______
                     (b)


        _______
          (c)

    return _______
             (d)
```

i. (2.0 pt) Fill in blank (a).

```
a is Link.empty
```

ii. (1.0 pt) Fill in blank (b).

- ○ `a`
- ○ `b`
- ○ `s`
- ● `t`
- ○ `longer(a, b)`
- ○ `longer(s, t)`

iii. **(1.0 pt)** Fill in blank (c).

- ◯ `a = a.rest`
- ◯ `a = s.rest`
- ◯ `b = b.rest`
- ◯ `b = t.rest`
- ● `a, b = a.rest, b.rest`
- ◯ `a, b = s.rest, t.rest`

iv. **(1.0 pt)** Fill in blank (d).

- ◯ `a`
- ◯ `b`
- ● `s`
- ◯ `t`
- ◯ `longer(a, b)`
- ◯ `longer(s, t)`

(b) (6.0 points)

Implement `longest`, which takes a linked list of postive integers `s` and a positive integer `n`. It returns the longest sublist of `s` with elements that sum to a number less than or equal to `n`. **Do not mutate `s`.** In case of a tie, return any of the longest sublists whose sum is `n` or less. Assume `longer` is implemented correctly.

A sublist of a linked list `s` is a linked list with some (or none or all) of the elements of `s` in order.

Assume the sum of the elements of `Link.empty` is 0. `Link.empty` is a sublist of any linked list.

```
def longest(s, n):
    """Return the longest sublist of s that sums to n or less.

    >>> longest(Link(5, Link(1, Link(3, Link(4, Link(2, Link(7)))))), 7)
    Link(1, Link(3, Link(2)))
    >>> longest(Link(5, Link(1, Link(3, Link(4, Link(2, Link(7)))))), 70)
    Link(5, Link(1, Link(3, Link(4, Link(2, Link(7))))))
    >>> longest(Link(3, Link(4, Link(5))), 2) is Link.empty
    True
    """
    if s is Link.empty:
        return s

    t = _______
          (e)

    if _______:
         (f)

        return longer( _______ , t)
                           (g)
    else:
        return t
```

i. (1.0 pt) Fill in blank (e).

- ○ `longest(s, n)`
- ○ `longest(s, n - s.first)`
- ● `longest(s.rest, n)`
- ○ `longest(s.rest, n - s.first)`

ii. (2.0 pt) Fill in blank (f).

```
s.first <= n
```

iii. (3.0 pt) Fill in blank (g).

```
Link(s.first, longest(s.rest, n - s.first))
```

6. (0.0 points) Nice Path!

This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

(a) (0.0 pt) Fill in the blank of `max_path`, which takes a Tree of integers `t` and a function `g`. It returns a list `s` containing the labels along the path from the root of `t` to a leaf for which `g(s)` is as large or larger than for any other path. In case of a tie, return any path with a maximum `g(s)` value.

```
def climb(t, f):
    if t.is_leaf():
        return [t.label]
    return [t.label] + climb(max(t.branches, key=f), f)

def max_path(t, g):
    """Return the path s from the root of t to a leaf for which g(s) is largest.

    >>> scare = Tree(0, [Tree(4), Tree(5, [Tree(10)]), Tree(2)])
    >>> crow = Tree(4, [Tree(5), Tree(9, [scare, Tree(7, [Tree(6)])]), Tree(8)])
    >>> max_path(crow, lambda p: -p[-1])            # The path to the smallest leaf
    [4, 9, 0, 2]
    >>> max_path(crow, len)                         # The longest path
    [4, 9, 0, 5, 10]
    >>> max_path(crow, lambda p: -abs(p[0]-p[-1]))  # To the leaf closest in value to the root
    [4, 9, 0, 4]
    """

    x = [t.label]  # You can use x instead of [t.label] to shorten your answer!

    return climb(t, lambda b: _______ )
```

```
g(x + max_path(b, lambda p: g(x + p)))
```

No more questions.

CS 61A Fall 2025

Structure and Interpretation of Computer Programs

Midterm 1 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

You can complete these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (6.0 points) What Would Python Do(ot)?

Assume the code below has been executed, and no errors occurred.

```
about = 6-7
about, face = 6, about * 7

def make_something(f):
    if f(about):
        f = lambda k: about
    def something(about):
        return print(f(about) or print(about))
    return something

f = lambda x: 10 * x
```

Example Question: What would be displayed by evaluating `print(5)`? **Answer**: 5

(a) (2.0 pt) What would be displayed by evaluating `print(print(face))`

```
-7 None
```

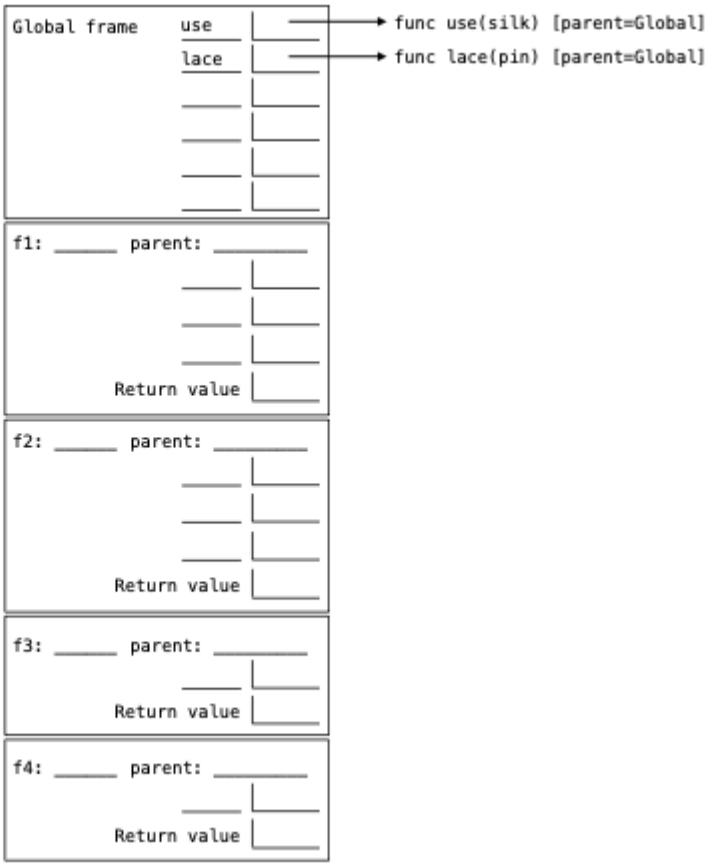
(b) (4.0 pt) What would be displayed by evaluating `make_something(print)(8)`

```
6 8 8 None
```

2. (5.0 points) Silksong

Complete the environment diagram below to answer the questons about the calls to `print`. Only the questions will be scored, not the diagram. Each call to `print` is evaluated once, and there are no errors caused by running this code.

```
 1: def use(silk):
 2:     a = 2
 3:     print(lace(silk), a)  # (a)
 4:
 5: def lace(pin):
 6:     a = print(pin())      # (b)
 7:     return a
 8:
 9: a = 1
10: thread = lambda x: a + x
11: a = 3
12: needle = lambda: thread(a * 10)
13:
14: use(needle)
15: print(a)                  # (c)
```



(a) (2.0 pt) What is displayed by the call to `print` on **line 3**?

```
None 2
```

(b) (2.0 pt) What is displayed by the call to `print` on **line 6**?

- ○ 21
- ○ 22
- ○ 23
- ○ 31
- ○ 32
- ● 33

(c) (1.0 pt) What is displayed by the call to `print` on **line 15**?

- ○ 2
- ● 3
- ○ None

3. (8.0 points) Saja Boys

Definition. A positive integer is *patterned* if every odd digit has a larger even digit somewhere before it. For example, 4123827 is patterned because the odd digit 7 has 8 (which is even and larger than 7) before it, and the 3 and 1 both have 4 before them.

Implement `patterned`, which takes a positive integer `n`. It returns `True` if `n` is patterned and `False` otherwise. Your implementation should iterate through the digits of `n` just once. The name `odd` in the implementation should always be assigned to an integer from 0 to 9 (inclusive).

```
def patterned(n):
    """Return whether every odd digit of n has a larger even digit somewhere before it.

    >>> patterned(4123827)  # 8 is before 7; 4 is before 1 and 3
    True
    >>> patterned(4412123384137)
    True
    >>> patterned(2468)     # No odd digits
    True

    >>> patterned(1)        # No even digits
    False
    >>> patterned(8192)     # 9 does not have a larger even digit before it (or anywhere)
    False
    >>> patterned(238)      # 3 does not have a larger even digit before it (8 comes after)
    False
    >>> patterned(3888)     # 3 does not have a larger even digit before it (8 comes after)
    False
    >>> patterned(4321587)  # 5 does not have a larger even digit before it (8 comes after)
    False
    """

    odd = 0    # Hint: use the name odd to keep track of a digit you'll need later

    while n:

        n, last = n // 10, n % 10

        if last % 2 == 1:

            -------
              (a)

        elif _______:
               (b)

            -------
              (c)

    return _______
             (d)
```

(a) (2.0 pt) Fill in blank (a).

- ○ `return False`
- ○ `return max(n) > last`
- ○ `return n % 10 > last`
- ○ `return n % 2 == 0 and n % 10 > last`
- ○ `odd = 0`
- ○ `odd = last`
- ○ `odd = n % 10`
- ○ `odd = max(n % 10, last)`
- ● `odd = max(odd, last)`

(b) (2.0 pt) Fill in blank (b). You may not use `str` or `len` or `[` or `]`

```
last > odd
```

(c) (2.0 pt) Fill in blank (c).

- ○ `return False`
- ○ `return max(n) > last`
- ○ `return n % 10 > last`
- ○ `return n % 2 == 0 and n % 10 > last`
- ● `odd = 0`
- ○ `odd = last`
- ○ `odd = n % 10`
- ○ `odd = max(n % 10, last)`
- ○ `odd = max(odd, last)`

(d) (2.0 pt) Fill in blank (d).

- ○ `True`
- ○ `False`
- ● `odd == 0`
- ○ `odd > 0`
- ○ `last == 0`
- ○ `last > 0`
- ○ `last % 2 == 0`
- ○ `last % 2 == 1`

4. (16.0 points) We're Going Up, Up, Up

Definitions. An *up-sequence* is a sequence of positive integers in which each term is larger than the last. The *next function* `f` for an up-sequence takes a non-negative integer `t` and encodes the sequence as follows: If `t` is any term of the sequence except the last one, then `f(t)` returns the next term in the sequence after `t`. If `t` is the last term of the sequence or not a term in the sequence, `f(t)` returns 0. Finally, when `t` is 0, then `f(0)` returns the first term of the sequence. For example, `fifty_evens` below is a next function for the sequence 2, 4, 6, 8, . . . , 98, 100.

```
def fifty_evens(t):
    "The next function for the finite sequence of the first 50 positive even numbers."
    if t % 2 == 0 and t < 100:
        return t + 2
    return 0
```

(a) (6.0 points)

Implement `sum_sequence`, which takes a next function `f` for a finite up-sequence. It returns the sum of the terms of that sequence.

```
def sum_sequence(f):
    """Return the sum of terms in a finite sequence.

    >>> sum_sequence(fifty_evens)    # 2 + 4 + 6 + 8 + ... + 98 + 100 = 2550
    2550
    """
    t = _______
          (a)

    total = 0
    while t:
        t, total = _______ , _______
                     (b)       (c)
    return total
```

i. (2.0 pt) Fill in blank (a).

```
f(0)
```

ii. (2.0 pt) Fill in blank (b).

```
f(t)
```

iii. (2.0 pt) Fill in blank (c).

- ◯ `total + 1`
- ● `total + t`
- ◯ `total + f`
- ◯ `total + fifty_evens`
- ◯ `total + f(t)`
- ◯ `total + fifty_evens(t)`

(b) (6.0 points)

Here is the next function for the **infinite** up-sequence of positive perfect squares: 1, 4, 9, 16, 25, ...

```
def next_square(t):
    """Compute the next perfect square.

    >>> next_square(0)
    1
    >>> next_square(16)
    25
    >>> next_square(17)  # Not a perfect square
    0
    """
    sqrt_t = t ** 0.5                    # The square root of t
    if sqrt_t == round(sqrt_t):          # Make sure t was a perfect square
        return (round(sqrt_t) + 1) ** 2  # Return the next perfect square
    return 0
```

Implement `cap`, which takes a next function `f` for an **infinite** up-sequence and a positive integer `n`. It returns a next function for the finite up-sequence containing the terms of `f` that are less than or equal to `n`. If `n` is a term of the up-sequence for `f`, then it is included in the result's sequence.

```
def cap(f, n):
    """Return the next function for the up-sequence for f up to (and possibly including) n.

    >>> squares_up_to_25 = cap(next_square, 30)  # 30 is not in the next_square sequence
    >>> squares_up_to_25(4)
    9
    >>> squares_up_to_25(16)
    25
    >>> squares_up_to_25(25)
    0
    >>> squares_up_to_25(17)                     # 17 is not in the next_square sequence
    0
    >>> cap(next_square, 81)(64)                 # 81 is in the next_square sequence
    81
    """
    def capped(t):

        if _______:
             (d)


            _______
              (e)

        else:

            return _______
                     (f)

    return capped
```

i. **(2.0 pt)** Fill in blank (d).

```
f(t) > n
```

ii. **(2.0 pt)** Fill in blank (e).

- ◯ `t = 0`
- ◯ `n += 1`
- ◯ `n -= 1`
- ◯ `t += 1`
- ◯ `t -= 1`
- ◯ `t = f(t)`
- ◯ `t = next_square(t)`
- ◯ `return n`
- ◯ `return t`
- ● `return 0`

iii. **(2.0 pt)** Fill in blank (f).

- ◯ `n`
- ◯ `t`
- ◯ `f`
- ● `f(t)`
- ◯ `next_square`
- ◯ `next_square(t)`

(c) (4.0 points)

Definition. The *previous function* `g` for an up-sequence takes a non-negative integer `t`. If `t` is any other term than the first, `g(t)` is the previous term. If `t` is the first term or not a term at all, `g(t)` returns 0. `g(0)` is the last term.

Implement `reverse`, which takes a next function `f` for a finite up-sequence. It returns the previous function for that same sequence. You may call `max_term`, which is described below, but you don't need to implement `max_term`.

```
def max_term(f):
    """Returns the largest term in the finite up-sequence for next function f.

    >>> max_term(cap(next_square, 20))  # 16 is the largest square less than or equal to 20.
    16
    """
    # Implementation omitted, but you can assume that the function is implemented correctly.

def reverse(f):
    """Return the previous function for the up-sequence encoded by next function f.

    >>> rev_squares = reverse(cap(next_square, 30))  # Goes in reverse through 1, 4, 9, 16, 25
    >>> print(rev_squares(0), rev_squares(25), rev_squares(16), rev_squares(9), rev_squares(4))
    25 16 9 4 1
    >>> rev_squares(1)   # 1 is the first term
    0
    >>> rev_squares(10)  # 10 is not in the sequence
    0
    """

    def previous(t):

        if t == 0:

            return _______
                     (g)
        x = t - 1

        while x > 0 and _______:
                           (h)

            x = x - 1

        return x

    return previous
```

i. (2.0 pt) Fill in blank (g).

$\texttt{max}_t erm(f)$

ii. (2.0 pt) Fill in blank (h).

```
f(x) != t
```

(d) (0.0 points)

This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Implement `sum_below`, which takes a next function `f` for an infinite up-sequence and a positive integer `n` that is a term of the sequence. It returns the sum of the terms of the sequence for `f` that are less than (but not including) `n`.

You may **not** use `cap` or `max_term` or `reverse`.

You may **not** use `if` or `else` or `[` or `]`.

```
def sum_below(f, n):
    """Return the sum of the terms of the sequence for f that are below n,
    where n is a term in the sequence for f.

    >>> sum_below(next_square, 25)  # 1 + 4 + 9 + 16
    30
    """
    assert f(n), 'n is not a term of the up-sequence for f'
    return sum_sequence(_______)
                        (i)
```

i. (0.0 pt) Fill in blank (i).

```
lambda u:  f(u) - n and f(u) lambda u:  (f(u) < n and f(u)) or 0 lambda
t:  int(f(t) < n)*f(t)
```

No more questions.

CS 61A Spring 2016 — Structure and Interpretation of Computer Programs

Test 1 Solutions

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is open book, open notes, closed computer, closed calculator. The official CS 61A midterm 1 study guide will be provided.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
BearFacts email (`@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
I pledge my honor that during this examination I have neither given nor received assistance. **(please sign)**	

1. (12 points) Evaluate This!

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "`Error`". If an expression yields (or prints) a function, write "`<Function>`". No answer requires more than 3 lines. (It's possible that all of them require even fewer.) The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`, plus all values passed to `print`.

(a) (10 pt) Assume that `python3` has executed the statements on the left:

```
y = 7

def b(x):
  return lambda y: x(y)

def c(x):
  return 3

w = b(c)

def c(x):
  return x
```

Expression	Interactive Output
`pow(2, 3)`	8
`print(4, 5) + 1`	4 5 Error
`(3 and abs)(-1)`	1
`print(3) or 1/0`	3 Error
`print`	Function
`([1, 2, 3] if y // (y+1) else [4, 5])[1]`	5
`w(5)`	3

(b) (2 pt) Assume that `python3` has executed the following statements:

```
def d(S):
    def f(k):
        return k < len(S) and (S[k] == S[0] or f(k+1))
    if len(S) == 0:
        return False
    else:
        return f(1) or d(S[1:])
```

Expression	Interactive Output
`print(d([1, 2, 3]), d([0, 1, 2, 1, 0]))`	False True

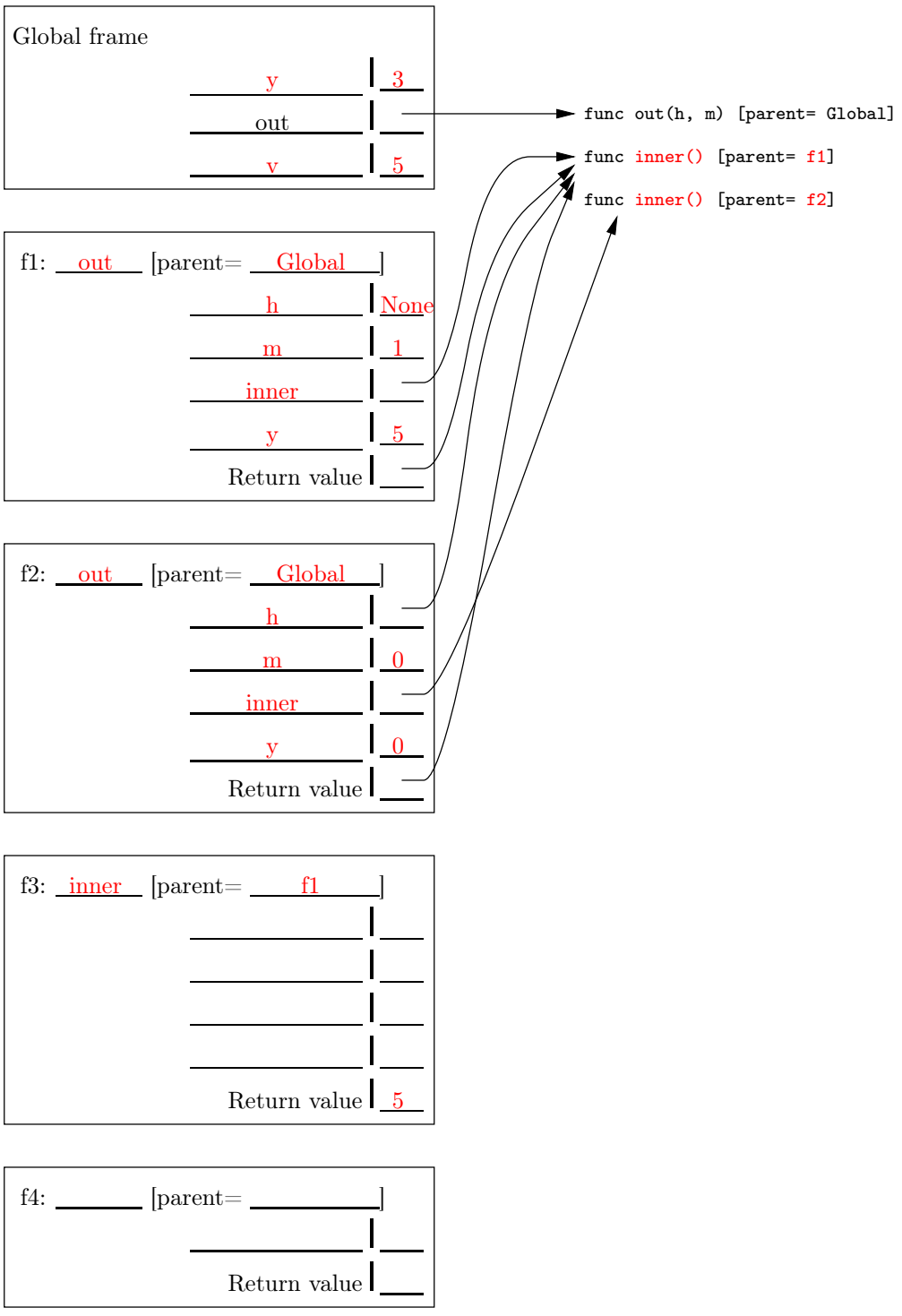
2. (12 points) Environmental Policy

(a) (6 pt) Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*

A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  y = 3
2  def out(h, m):
3      y = 5 * m
4      def inner():
5          return y
6      if m == 0:
7          return h
8      else:
9          return out(inner, m-1)
10 v = out(None, 1)()
```

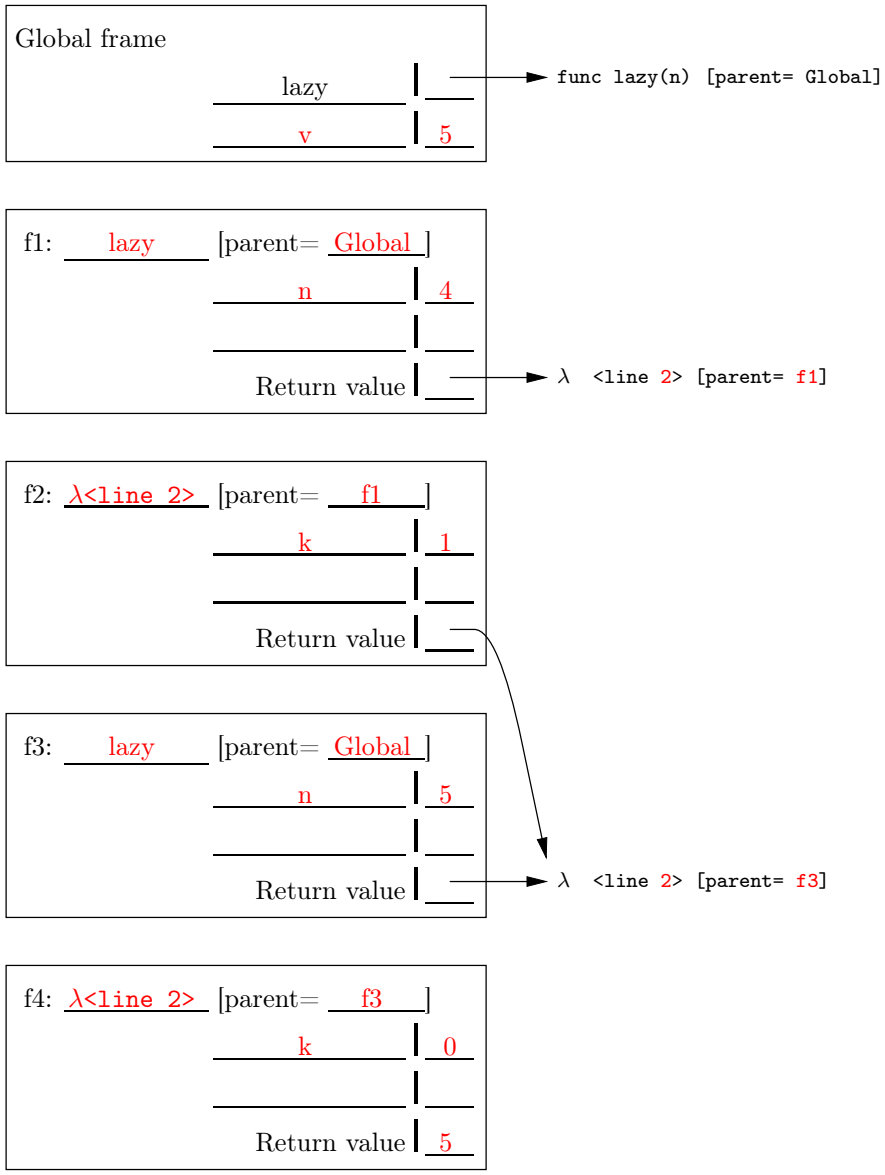


(b) (6 pt) Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.* The `<line ...>` annotation in a lambda value gives the line in the Python source of a lambda expression.

A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Add all missing parents of function values.
- Show the return value for each local frame.

```
1 def lazy(n):
2     return lambda k: n if k==0 else lazy(n+1)
3 v = lazy(4)(1)(0)
```



3. (1 points) Extra Who described Italy (in translation) as "the land where lemon blossoms blow, / And through dark leaves the golden oranges glow."?

4. (4 points) Going Around in Cycles

Consider a list of integers, `L`, in which each integer is a value in 0 to `len(L)-1` (inclusive). That is, each item in the list could also be used as a valid index into the list. We'll say that `L` has a *cycle of length k* iff for some sequence of index values $i_1, i_2, \ldots, i_k$, we have `L[`i_1`] =` i_2`, L[`i_2`] =` i_3`, ..., L[`i_k`] =` i_1. For example, the list `L = [3, 1, 4, 5, 0, 2]` has a cycle of length 1: $1 \to 1$, and a cycle of length 5: $0 \to 3 \to 5 \to 2 \to 4 \to 0$. For the purposes of this problem, we'll count something as a cycle of length k only if the first k indices in it contain no repetitions (so `L` in the example does not contain a cycle of length 2 even though $1 \to 1 \to 1$.

Write a function `has_cycle(L, k)` that returns a true value iff `L` has a cycle of length k by filling in the template below.

```
def has_cycle(L, k):
    """True iff L has a cycle of length K>=1.  Assumes 0 <= L[i] < len(L)
    for 0 <= i < len(L).
    """
    def cycle_at(s):
        """L has a cycle of length K starting at S."""
        p = L[s]
        n = 1
        while n < k:
            if p == s:
                return False
            p = L[p]
            n += 1
        return p == s

    for j in range(len(L)):
        if cycle_at(j):
            return True
    return False
```

5. (4 points) Your Father's Parentheses

Suppose we have a sequence of quantities that we want to multiply together, but can only multiply two at a time. We can express the various ways of doing so by counting the number of different ways to parenthesize the sequence. For example, here are the possibilities for products of 1, 2, 3, 4 and 5 elements:

Product	a	ab	abc	abcd	abcde		
Count	1	1	2	5	14		
Parenthesizations	a	ab	a(bc) (ab)c	a(b(cd)) a((bc)d) (ab)(cd) (a(bc))d ((ab)c)d	a(b(c(de))) a(b((cd)e)) a((bc)(de)) a((b(cd))e) a(((bc)d)e)	(ab)(c(de)) (ab)((cd)e) (a(bc))(de) ((ab)c)(de) (a(b(cd)))e	(a((bc)d))e ((ab)(cd))e ((a(bc))d)e (((ab)c)d)e

Assume, as in the table above, that we don't want to reorder elements.

Define a function `count_groupings` that takes a positive integer n and returns the number of ways of parenthesizing the product of n numbers. (You might not need to use all lines.)

```
def count_groupings(n):
    """For N >= 1, the number of distinct parenthesizations
    of a product of N items.
    >>> count_groupings(1)
    1
    >>> count_groupings(2)
    1
    >>> count_groupings(3)
    2
    >>> count_groupings(4)
    5
    >>> count_groupings(5)
    14
    """

    if n == 1:

        return 1

    total = 0

    i = 1

    while i < n:

        total += count_groupings(i) * count_groupings(n - i)

        i += 1

    return total
```

6. (8 points) Amazing

In lecture, we did some problems involving finding one's way through a maze, representing the maze as a predicate (boolean function)—the parameter `blocked`. The idea was that `blocked(x, y)` iff the grid square at row `y` and column `x` was blocked. Let's consider a different representation.

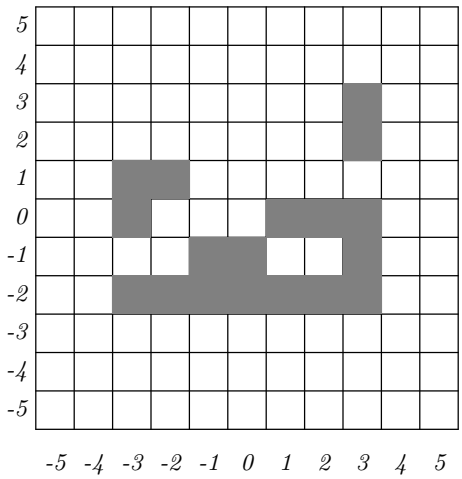
Again, mazes will be represented by functions, but with a different specification. A maze function, say `M`, in this formulation describes the state of the maze from the point of view of a maze runner. `M` accepts a single argument, `direction`, which describes the action the runner tries to take: either the string "south" or "west", indicating that the runner tries to take one step south or one step west, respectively. `M` then returns either:

- Another maze function that represents the state of the runner after taking the indicated step. It again accepts a direction and returns one of these three things;
- The string "exit", signifying that the move causes the runner to successfully reach the exit; or
- The string "dead end", which indicates that the runner was unable to take the requested action because of being blocked by a wall.

(a) (4 pt) Define a function `pred_maze` that returns a maze function (as described above).
If `M = pred_maze`(x, y, P, e), then `M` represents a runner in a maze in which

- The runner is at coordinates (x, y).
- Squares (a, b) for which $a \leq e$ are unblocked and are exit squares.
- A square at (a, b) is otherwise open iff $P(a, b)$.

For example, suppose that `Q` is a function such that `Q(a,b)` is true on all the white squares in the following grid (where, just to be different, the lower-left corner represents position (-5, -5)):



Then we'd have

```
>>> M0 = pred_maze(0, 1, Q, -4)
>>> M1 = M0('west')
>>> M1
<function ...>
>>> M1('west')
'dead end'
>>> M1('south')
<function ...>
>>> M2 = pred_maze(-3, -1, Q, -4)
>>> M2('west')
'exit'
```

Fill in your solution on next page.

```
def pred_maze(x0, y0, open, exit):
    """Return a maze in which the runner is at (X0, Y0), every square
    (a, b) where a <= EXIT is an exit, and otherwise a square (a, b)
    is open iff OPEN(a, b).  It is assumed that (X0, Y0) is open."""

    def maze(dir):

        x, y = (x0, y0 - 1) if dir == "south" else (x0 - 1, y0)

        if x <= exit:

            return "exit"

        elif open(x, y):

            return pred_maze(x, y, open, exit)

        else:

            return "dead end"


    return maze
```

(b) (4 pt) Define a function `path_out` that takes a maze and returns a string describing a path to an exit, or returns `None` iff there is no such path. A call `path_out(M)` will return a string such as

```
'south west south west west'
```

(as in the doctest below) to indicate that the necessary path takes one step south, then a step west, then south, and then two steps west. (Don't worry about extra spaces in the returned string.) When multiple paths exist, return any one. Using the previous value of function `Q`, for example, we'd see:

```
>>> M = pred_maze(-1, 1, Q, -4)
>>> path_out(M)
'south west south west west'
>>> M = pred_maze(2, -1, Q, -4)
>>> path_out(M)   # Returns None
```

FYI: The '+' operator on strings is string concatenation ('+=') also works.)

```
def path_out(M):
    """Given a maze function M in which the runner is not yet out
    of the maze, returns a string of the form "D1 D2 ...", where each
    Di is either south or west, indicating a path from M to an exit,
    or None iff there is no such path."""

    for dir in ["south", "west"]:

        next = M(dir)

        if next == "exit":

            return dir + " "

        elif next != "dead end":

            rest_of_path = path_out(next)

            if rest_of_path:

                return dir + " " + rest_of_path

    return None
```

CS 61A Spring 2017

Structure and Interpretation of Computer Programs

Test 1 Solutions

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is open book, open notes, closed computer, closed calculator.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
Room in which you are taking exam	
Seat number in the exam room	
I pledge my honor that during this examination I have neither given nor received assistance. **(please sign)**	

Reference Material.

```
# Linked Lists (implementations not shown)

empty = ...  # The empty list

def link(first, rest):
    """A linked list whose first element is FIRST and the linked list
    REST is the rest of the list."""

def first(lnklst):
    """The first item in linked list LNKLST."""

def rest(lnklst):
    """The list following the first item in linked list LNKLST."""

def isempty(lnklst):
    """True if LNKLST is empty."""

def print_link(lnklst):
    """Prints the linked list LNKLST in the format (v0, v1, ...)."""
```

1. (12 points) Evaluate These!

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "`Error`". If an expression yields (or prints) a function, write "`<Function>`". No answer requires more than 3 lines. (It's possible that all of them require even fewer.) The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`, plus all values passed to `print`.

Assume that `python3` has executed the statements on the left:

```
def w(L):
  if len(L) == 0:
    return L
  elif L[0] in L[1:]:
    return w(L[1:])
  else:
    return [L[0]] + w(L[1:])

def reduce(f, L, init):
  if len(L) == 0:
    return init
  else:
    return reduce(f, L[1:],
                  f(init, L[0]))
```

Expression	**Interactive Output**
`pow(2, 3)`	8
`print(4, 5) + 1`	4 5 Error
`1 + (4 and 6) + (5 or 0) + (0 and 8)`	12

`f = lambda x: x`
`g = lambda y: f(g)`
`g(2)(2)`
	`<Function>`

`0 and print(2)`	0
`range(1,20)[-2]`	18
`w([1, 2, 3, 1, 8, 2])`	[3,1,8 2]

`f = lambda x: lambda y: \`
`  lambda z: x+y+z`
`reduce(lambda g, x: g(x),`
`  [1, 2, 3], f)`
	6

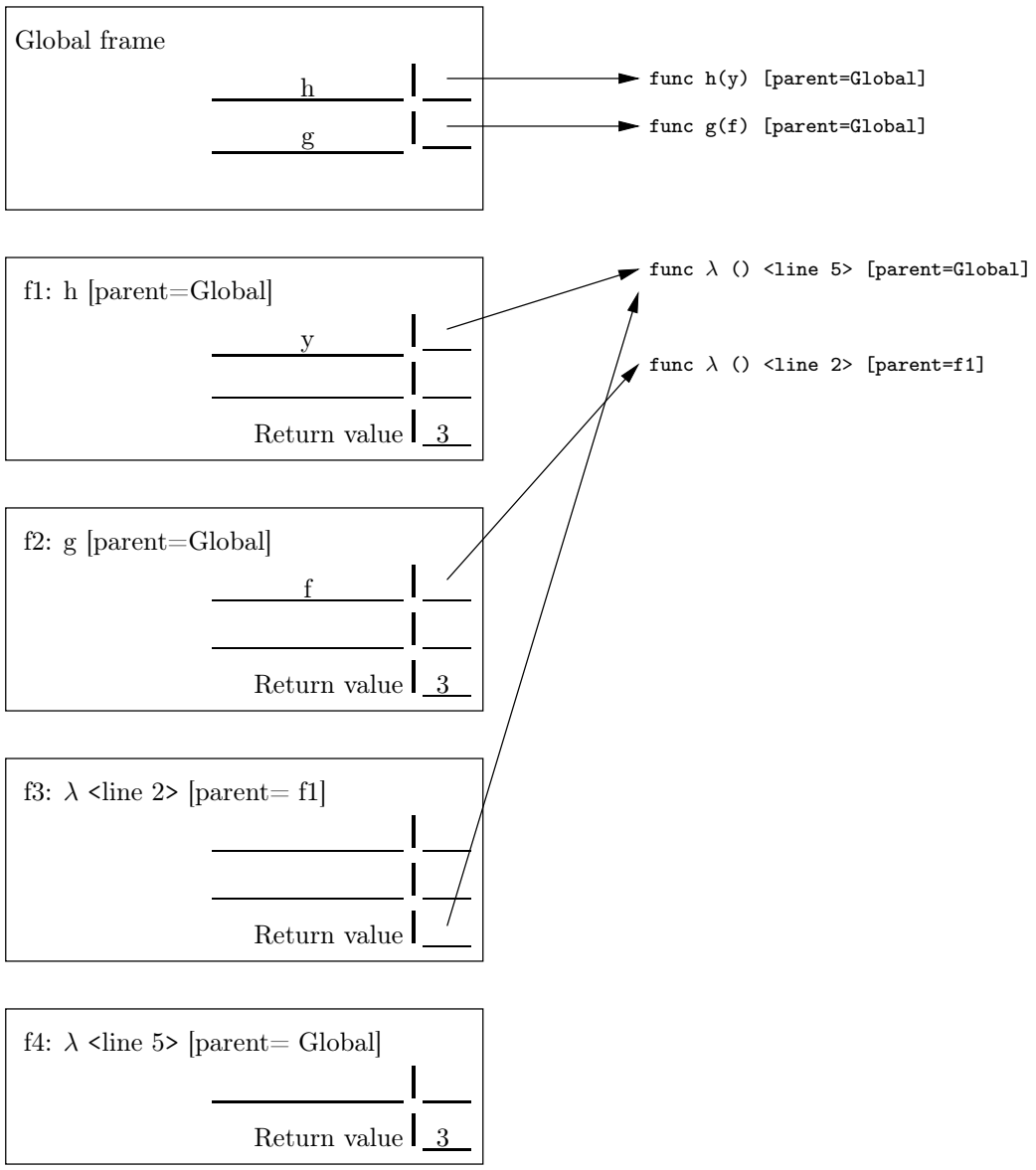
2. (12 points) Environmental Policy

(a) (6 pt) Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces, frames, or blank lambda function values.*

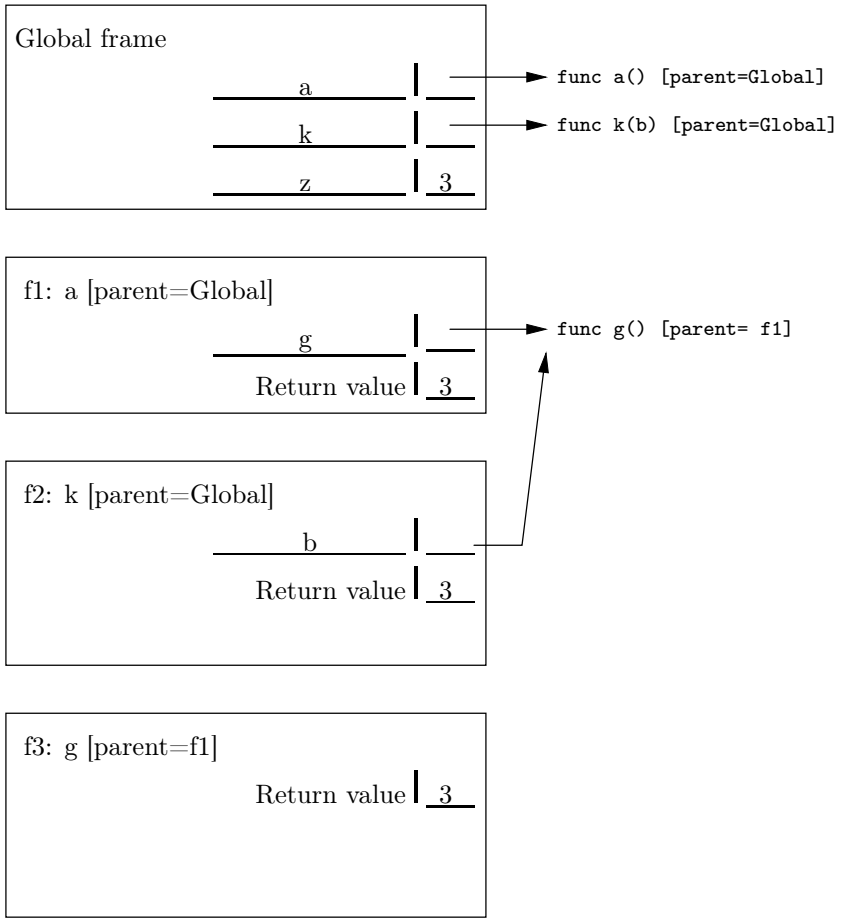
A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1 def h(y):
2     return g(lambda: y)
3 def g(f):
4     return f()()
5 h(lambda: 3)
```



(b) (6 pt) The environment diagram below corresponds to the execution of a certain program. The frames shown were created in top-to-bottom order, and at one point they were all simultaneously active (their functions had not returned). The diagram shows the situation right after all the functions have returned. Write a Python program whose execution corresponds to this environment diagram. Many answers are possible, but as a guideline, fewer than 10 lines are really needed. In any case, your answer must not create extra frames or additional functions.



Write solution here.

```
def a():
    def g():
        return 3
    return k(g)

def k(b):
    return b()

z = a()
```

3. (4 points) Sequence Checking

Fill in the following function so that it fulfills its comment.

```
def make_checker(relation, start, end):
    """Assumes that START and END are integers and RELATION is a two-argument
    function that returns true/false values.  Returns a function that, given
    a function f as input, returns True if RELATION returns true for
    all adjacent values in the sequence f(START), f(START+1), ... f(END-1).
    For example, eq_chk, below, checks that the values returned by
    its argument function for values 0-4 are all equal, while up_chk checks
    that the values returned by the argument function are in strictly
    increasing order.

    >>> eq_chk = make_checker(lambda x, y: x == y, 0, 5) # Check all equal
    >>> eq_chk(lambda x: 3)
    True
    >>> eq_chk(lambda x: x)
    False
    >>> up_chk = make_checker(lambda x, y: x < y, 0, 5) # Check increasing
    >>> up_chk(lambda x: x)
    True
    >>> up_chk(lambda x: 3)
    False
    """

    def checker(f):
        for k in range(start + 1, end):
            if not relation(f(k-1), f(k)):
                return False
        return True
    return checker
```

4. (1 points) Extra

In the formula $a = 0.4 + 0.3 \cdot 2^m$ ($m = -\inf, 0, 1, 2, \ldots$), what does a refer to?

The approximate positions of planets (in AU). This is known as Bode's Law.

5. (4 points) Insert

Fill in the following function to fulfill its comment. The linked-list interface that you should use is given on page **??**. **Warning:** this problem deals with linked lists, *NOT* Python lists or tuples. You cannot use `+`, `len()`, indexing (`L[k]`), or list construction (`[...]`).

```
def link_insert(lnklst, value, before):
    """Return a linked list identical to LNKLST, but with VALUE inserted just
    before the first occurrence of BEFORE in the list, if any.  The returned
    list is identical to LNKLST if BEFORE does not occur in LNKLST.
    The operation is non-destructive.
    >>> L = link(2, link(3, link(7, link(1))))
    >>> print_link(L)
    (2, 3, 7, 1)
    >>> Q = link_insert(L, 19, 7)
    >>> print_link(Q)
    (2, 3, 19, 7, 1)
    >>> print_link(link_insert(L, 19, 20))
    (2, 3, 7, 1)
    """

    if isempty(lnklst):
        return lnklst
    elif first(lnklst) == before:
        return link(value, lnklst)
    else:
        return link(first(lnklst), link_insert(rest(lnklst), value, before))
```

6. (4 points) Longest Nondecreasing Suffix

Consider a function `up_suffix` that is supposed to return the longest suffix of a Python list of integers such that the suffix consists of nondecreasing values. For example, when applied to

```
[1, 2, 3, 4, 5, 1, 3, 3, 4]
```

it should yield `[1, 3, 3, 4]`.

Fill in the following function to do this:

```
def up_suffix(L):
    """Returns the longest non-descending suffix of Python list L. """

    def longest_suffix_start(L):
        """The index in L of the beginning of the longest nondecreasing suffix
        of L. For the empty list, returns 0.
        >>> longest_suffix_start([1, 2, 3, 4, 5, 1, 3, 3, 4])
        5
        >>> longest_suffix_start([2, 4, 6, 8, 10])
        0
        """

        k = len(L) - 1

        while k > 0 and L[k-1] <= L[k]:
            k -= 1
        return max(k, 0)
        # We were not fussy about people who returned -1 for the empty list,
        # so just return k was all right as well.

    return L[longest_suffix_start(L):]
```

7. (4 points) Post's Problem

Consider two Python lists of strings, where the two lists have equal length, N:

```
A = [ "a", "ab", "bba"]
B = [ "baa", "aa", "bb" ]
```

Is there a sequence of integers—i_1, i_2, ... i_m—where $m > 0$ and $0 \leq i_k < N$ for all k, such that

$$A[i_1] + A[i_2] + \cdots + A[i_m] = B[i_1] + B[i_2] + \cdots + B[i_m]?$$

This is called the *Post Correspondence Problem*. For this `A` and `B`, the answer is yes for $m = 4$: (2, 1, 2, 0), because

```
A[2] + A[1] + A[2] + A[0] == "bba" + "ab" + "bba" + "a"  == "bbaabbbaa"
B[2] + B[1] + B[2] + B[0] == "bb" + "aa" + "bb" + "baa" == "bbaabbbaa"
```

On the other hand, the answer is no if we limit m to 3 (so we cannot add more than three strings), or if we shorten the lists by removing `A[0]` and `B[0]`. Fill in the following function to determine whether there is a solution.

```
def correspond(A, B, M):
    """Assuming A and B are lists of strings with len(A) == len(B), and M
    is an integer, returns true iff there is a sequence of indices into A
    and B, (i1, i2, ..., im), where 1 <= m <= M, such that
        A[i1] + A[i2] + ... + A[im] == B[i1] + B[i2] + ... + B[im].
    """
    N = len(A)

    def can_correspond(sa, sb, M):
        """Return true iff there is some sequence of indices into A and
        B, (i1, i2, ..., im), where 1 <= m <= M, such that
        SA + A[i1] + A[i2] + ... + A[im] == SB + B[i1] + B[i2] + ... + B[im].
        Assumes M is a non-negative integer.
        """

        if M <= 0:
            return False
        else:
            for i in range(N):
                ta = sa + A[i]
                tb = sb + B[i]
                if ta == tb:
                    return True
                elif can_correspond(ta, tb, M - 1):
                    return True
            return False

    return can_correspond("", "", M)
```

CS 61A Spring 2017 — Structure and Interpretation of Computer Programs

TEST 2 (REVISED) SOLUTIONS

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is open book, open notes, closed computer, closed calculator.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
Room in which you are taking exam	
Seat number in the exam room	
I pledge my honor that during this examination I have neither given nor received assistance. **(please sign)**	

Reference Material.

```
# Linked Lists

class Link:
    """A linked list cell.
    >>> L = Link(0, Link(1))
    >>> L.first
`   0
    >>> L.rest
    Link(1)
    >>> L.first = 2
    >>> L
    Link(2, Link(1))
    >>> L.rest = Link.empty
    >>> L
    Link(2)
    """
    empty = ()

    def __init__(self, first, rest=empty):
        assert rest is Link.empty or isinstance(rest, Link)
        self.first = first
        self.rest = rest

    def __repr__(self):
        if self.rest is Link.empty:
             return "Link({})".format(self.first)
        else:
             return "Link({}, {})".format(self.first, self.rest)

# Trees

class Tree:
    """A tree node."""

    def __init__(self, label, branches=[]):
        for c in branches:
            assert isinstance(c, Tree)
        self.label = label
        self.branches = branches

    def is_leaf(self):
        return not self.branches
```

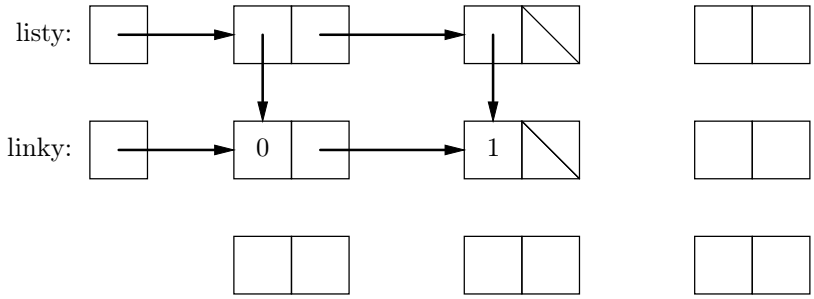
1. (12 points) Pointers

In the following problems, single boxes are variables that contain pointers, and double boxes are `Links` (see the definition of `Link` on page **??**). To show that a box contains a pointer to the empty list, draw the box like this:

In parts (a) and (b), add arrows and values to the object skeletons on the right to show the final state of the program. Not all boxes will be used. (For examples of what kinds of box and pointer diagrams we're looking for, you might look at parts (c) and (d) first.)

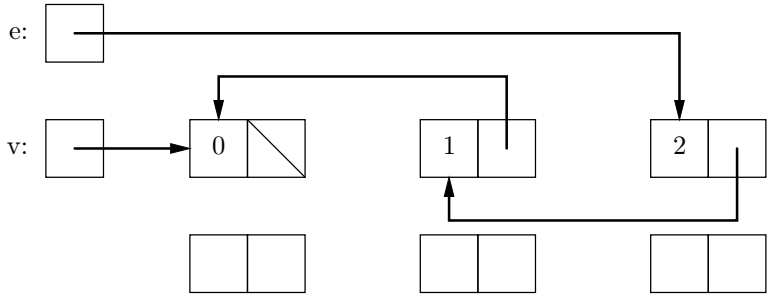
(a) (3 pt)

```
listy = Link(0, Link(1))

def nest(L):
    if L is Link.empty:
        return L
    N = nest(L.rest)
    L.first = Link(L.first, N)
    return L.first

linky = nest(listy)
```

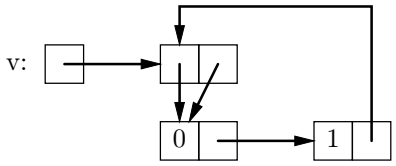


(b) (3 pt)

```
v = Link(0, Link(1, Link(2)))
e = v.rest.rest
e.rest = v.rest
v.rest.rest = v
v.rest = Link.empty
```



(c) (3 pt) Show Python code that will produce the situation shown in the diagram. (An arrow pointing to a `Link` may be shown as pointing anywhere on the double box for that `Link`.)

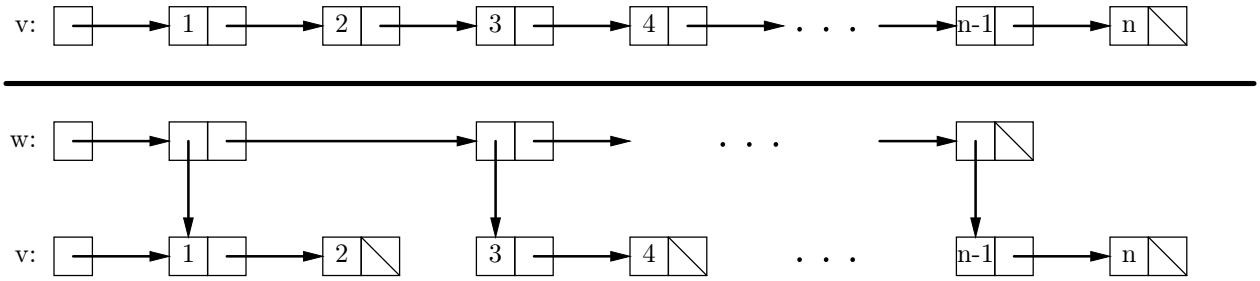



```
v = Link(None, Link(0, Link(1)))
```

```
v.first = v.rest
```

```
v.rest.rest.rest = v
```

(d) (3 pt) Show Python code that converts the situation shown above the line into that shown below the line. Assume n is even.




```
def split2(L):
    """Assuming that linked list L has even length, breaks L into
    two-element linked lists, and returns a linked list of those lists."""

    if L is Link.empty:

           return Link.empty
    else:

           result = Link(L, split2(L.rest.rest))

           L.rest.rest = Link.empty

           return result
w = split2(v)
```

2. (6 points) Complexity

(a) (1.5 pt) Indicate which of the following assertions are true by circling the letters for those statements. An assertion such as $\Theta(f(n)) \subseteq \Theta(g(n))$ means "any function that is in $\Theta(f(n))$ is also in $\Theta(g(n))$," and $\Theta(f(n)) = \Theta(g(n))$ if and only if $\Theta(f(n)) \subseteq \Theta(g(n))$ and $\Theta(g(n)) \subseteq \Theta(f(n))$.

A. If $f(n) \in \Theta(1)$ and $g(n) \in \Theta(1)$, then $\Theta(|f(n)| + |g(n)|) \in \Theta(1)$.

B. If $\Theta(f(n)) = \Theta(g(n))$, and $g(n) > 0$ everywhere, then $f(n)/g(n) \in \Theta(1)$.

C. $\Theta(x^2) \subseteq \Theta(x^3)$.

D. $\Theta(2^x) \subseteq \Theta(2^x + x^2)$.

E. If $f(n) \in \Theta(1000 \cdot x^3)$, then $f(20) > 800$.

(b) (1.5 pt) Consider the function

```
def num_kinks(L):
    c = 0
    i = 0
    while i < len(L):
        j = i
        while j < len(L):
            while j < len(L):
                if kink(L[i], L[j]):
                    c += 1
                    break
                j += 1
            j += 1
        i += 1
    return c
```

Circle the order of growth that best describes the worst-case execution time (measured by the number of calls to `kink`) of a call to `num_kinks` as a function of N, the length of `L`.

A. $\Theta(\log N)$

B. $\Theta(N)$

C. $\Theta(N^2)$

D. $\Theta(N^3)$

E. $\Theta(2^N)$

(c) (1.5 pt) Consider the following function on `Trees`

```
def count_subtrees(T, p):
    if p(T.label):
        return 1
    else:
        return sum([count_subtrees(child, p) for child in T.branches])
```

Assuming that the maximum number of children of any node is 3, circle the order of growth that best describes the worst-case execution time (measured by the number of calls to `p`) of a call to `count_subtrees` as a function of N, the number of nodes in `T`.

A. $\Theta(\log N)$

B. $\Theta(N)$

C. $\Theta(N^2)$

D. $\Theta(N^3)$

E. $\Theta(3^N)$

(d) (1.5 pt) For the same function as in part (c) above, and again assuming that the maximum number of children of any node is 3, circle the order of growth that best describes the worst-case execution time (measured by the number of calls to `p`) of a call to `count_subtrees` as a function of H, the height of `T`.

A. $\Theta(\log H)$

B. $\Theta(H)$

C. $\Theta(H^2)$

D. $\Theta(H^3)$

E. $\Theta(3^H)$.

3. (1 points) From the Sum of Human Knowledge

This Renaissance composer, famous for his harmonically innovative madrigals, was also infamous for murdering his wife and her lover and thereafter having himself beaten regularly by one of his servants. Who was he?

Carlo Gesualdo

4. (8 points) OOPs!

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "`ERROR`". No answer requires more than 3 lines. (It's possible that all of them require even fewer.)

```
class Thing:
    id = 0

    def fidget(self, n):
        print(n, "A", self.id)

    def fuss(self, x, n):
        print(n, "B")
        self.fidget(n)
        x.fidget(n)

    def twitch(self, n):
        self.waffle(n)

class Gadget(Thing):
    id = 1

class Whatsit(Gadget):
    def fidget(self, n):
        print(n, "D", self.id)

    def waffle(self, n):
        print(n, "D")

    def fiddle(self, x, n):
        x.waffle(n)

t1 = Thing()
t2 = Gadget()
t3 = Whatsit()
t4 = Whatsit()
t3.id = 2
```

Expression	Interactive Output
`t3.fidget(1)`	`1 D 2`
`t4.fidget(2)`	`2 D 1`

`t4.fuss(t1, 3)`	`3 B`
	`3 D 1`
	`3 A 0`

`t4.fiddle(t4, 4)`	`4 D`
`t4.fiddle(t1, 5)`	ERROR
`Thing.id = 3` `t1.fidget(6)`	`6 A 3`

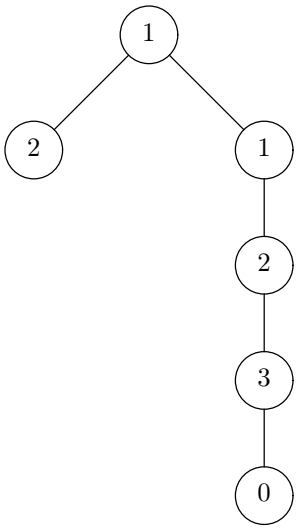
5. (8 points) Inflections

Fill in the definition of class `Wrinkles` to conform to its doc comment. You need not use all the lines shown.

```
class Wrinkles:
    """An object that contains a sequence of items and a predicate (true/false
    function) and that, when iterated over, produces adjacent pairs of items
    in the sequence that satisfy the predicate.
    >>> w = Wrinkles([1, 2, 3, 2, 4, 8, 5, 4], lambda x, y: x > y)
    >>> for p in w:
    ...    print(p)
    (3, 2)
    (8, 5)
    (5, 4)
    """

    def __init__(self, L, wrinkle):
        self._L = L
        self._wrinkle = wrinkle


    def __iter__(self):

        for k in range(1, len(self._L)):

            if self._wrinkle(self._L[k - 1], self._[k]):
                yield (self._L[k - 1], self._[k])
```

6. (8 points) Tree Paths

Given a tree, t, find the length of the longest downward sequence of node labels in the tree that are increasing consecutive integers. For example, in this tree, the longest such sequence has three labels (1, 2, 3):



As illustrated, the longest sequence can start and end anywhere in the tree, not just the root. (**Hint:** don't forget there's a `max` function.) [The original skeleton was flawed. [The "solution" below shows what we were expecting, but does not properly take care of all cases where the longest path does not start at the root of the tree. The next page shows a complete solution, based on a different solution.]

```
def longest_seq(t):
    """The length of the longest downward sequence of nodes in T whose
    labels are consecutive integers.
    >>> t = Tree(1, [Tree(2), Tree(1, [Tree(2, [Tree(3, [Tree(0)])])])])
    >>> longest_seq(t) # 1 -> 2 -> 3
    3
    >>> t = Tree(1)
    >>> longest_seq(t)
    1
    """

    if t.is_leaf():

        return 1

    max_len = 1

    for b in t.branches:

        if b.label == t.label + 1:

            max_len = max(max_len, 1 + longest_seq(b))

        else:

            max_len = max(max_len, longest_seq(b))

    return max_len
```

Here is a proper solution, based on the corrected skeleton

```
def longest_seq(tr):
    """The length of the longest downward sequence of nodes in TR whose
    labels are consecutive integers.
    >>> t = Tree(1, [Tree(2), Tree(1, [Tree(2, [Tree(3, [Tree(0)])])])])
    >>> longest_seq(t) # 1 -> 2 -> 3
    3
    >>> t = Tree(1)
    >>> longest_seq(t)
    1
    """
    max_len = 1

    def longest(t):
        """Returns longest downward sequence of nodes starting at T whose
        labels are consecutive integers.  Updates max_len to that length,
        if greater."""

        nonlocal max_len

        n = 1
        if not t.is_leaf():

            for b in t.branches:

                L = longest(b)

                if b.label == t.label + 1:

                    n = max(n, L + 1)

            max_len = max(n, max_len)
        return n
    longest(tr)
    return max_len
```

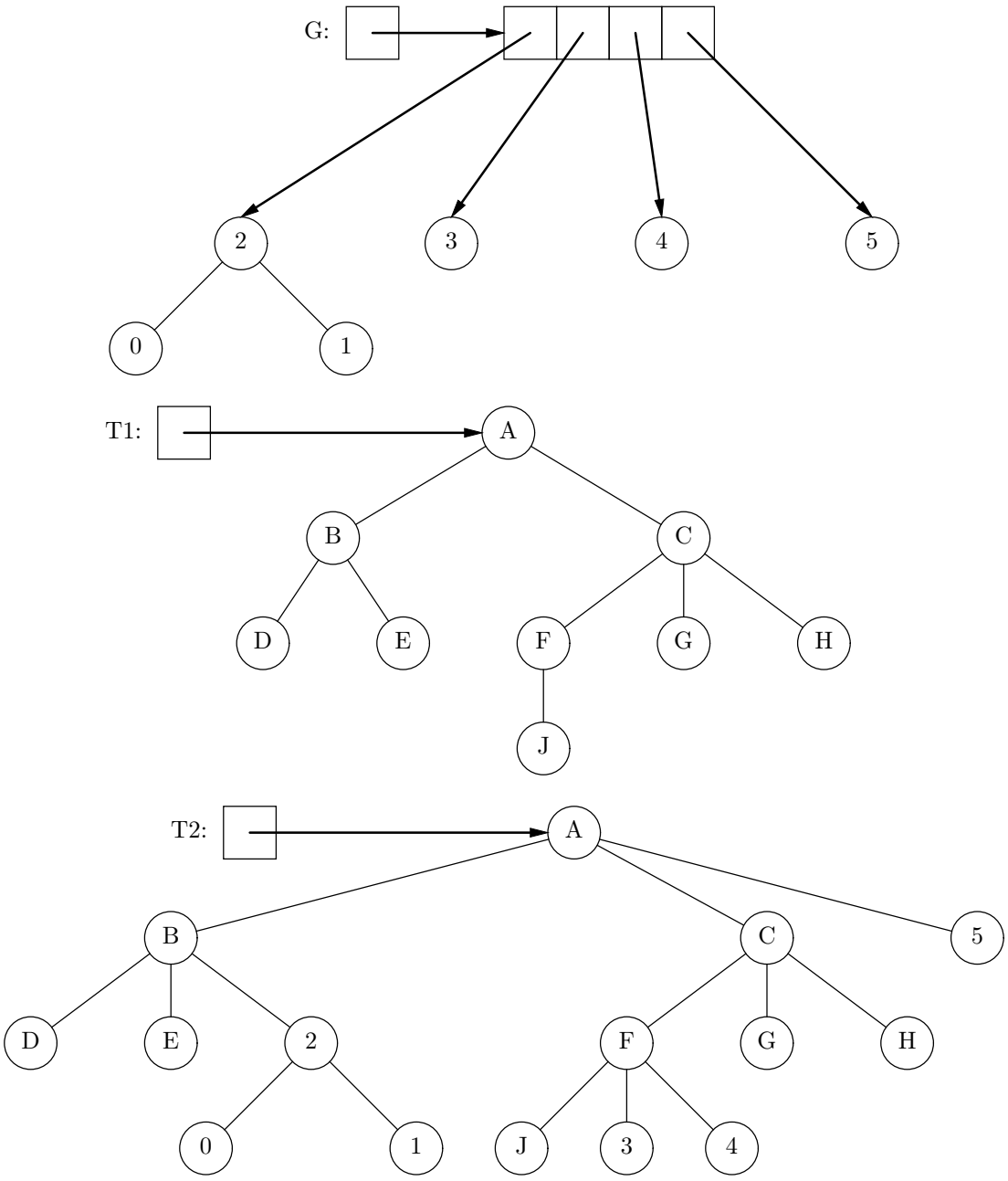
This page deliberately blank.

7. (8 points) Grafting

We want to insert ("graft") branches from a sequence of trees onto a tree in places where a non-leaf node has fewer than K branches, where K is a parameter. For example, given the list of four trees `G` created by

```
G = [ Tree(2, [Tree(0), Tree(1)]), Tree(3), Tree(4), Tree(5) ]
```

and the tree `T1` shown below, we want `T2 = graft(T1, G, 3)` to destructively (and without creating any new tree nodes) turn `T1` into the tree `T2`:



The list of trees (`G` in the example above) will always have enough items to fill all necessary places. Trees are inserted in postorder (that is, bottom to top, left to right).

Fill in the `graft` function here. You need not use all the lines.

```
def graft(T, L, k):
    """Returns the Tree created by destructively adding trees from L to
    non-leaf nodes of Tree T with fewer than K branches.  Assume that L has enough
    items to fill all necessary places.  Fill in trees in postorder (bottom to
    top, left to right)."""

    grafts = iter(L)

    def do_grafts(tr):


        if not tr.is_leaf():


            for b in tr.branches:


                do_grafts(b)


            while len(tr.branches) < k:


                tr.branches.append(next(grafts))


    do_grafts(T)

    return T
```

CS 61A Spring 2018 — Structure and Interpretation of Computer Programs

MIDTERM 1 SOLUTIONS

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 study guide.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, and `abs`. You **may not** use functions defined on your study guide unless clearly specified in the question.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.

1. (12 points) Frame of Thrones

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
from operator import add, sub

def winterfell(a, b):
    a
    b
    return b(a+1, b(a))

da, ny = 20, 18

while da > ny:
    da = ny
    da, ny = ny + 1, da + 3

def tar(gar, yen):
    if print(yen):
        print(yen + 1)
    return gar(yen)

def st(ar, k=None):
    return lambda a, y: ar(y, a)

night = st(sub)
king = st(st(pow))

def jon(sn, ow):
    print(ow)
    jon = sn(ow)
    print(ow)
    return jon

def snow(ow):
    def tarly(snow):
        return ow + snow
    ow += 2
    return tarly
```

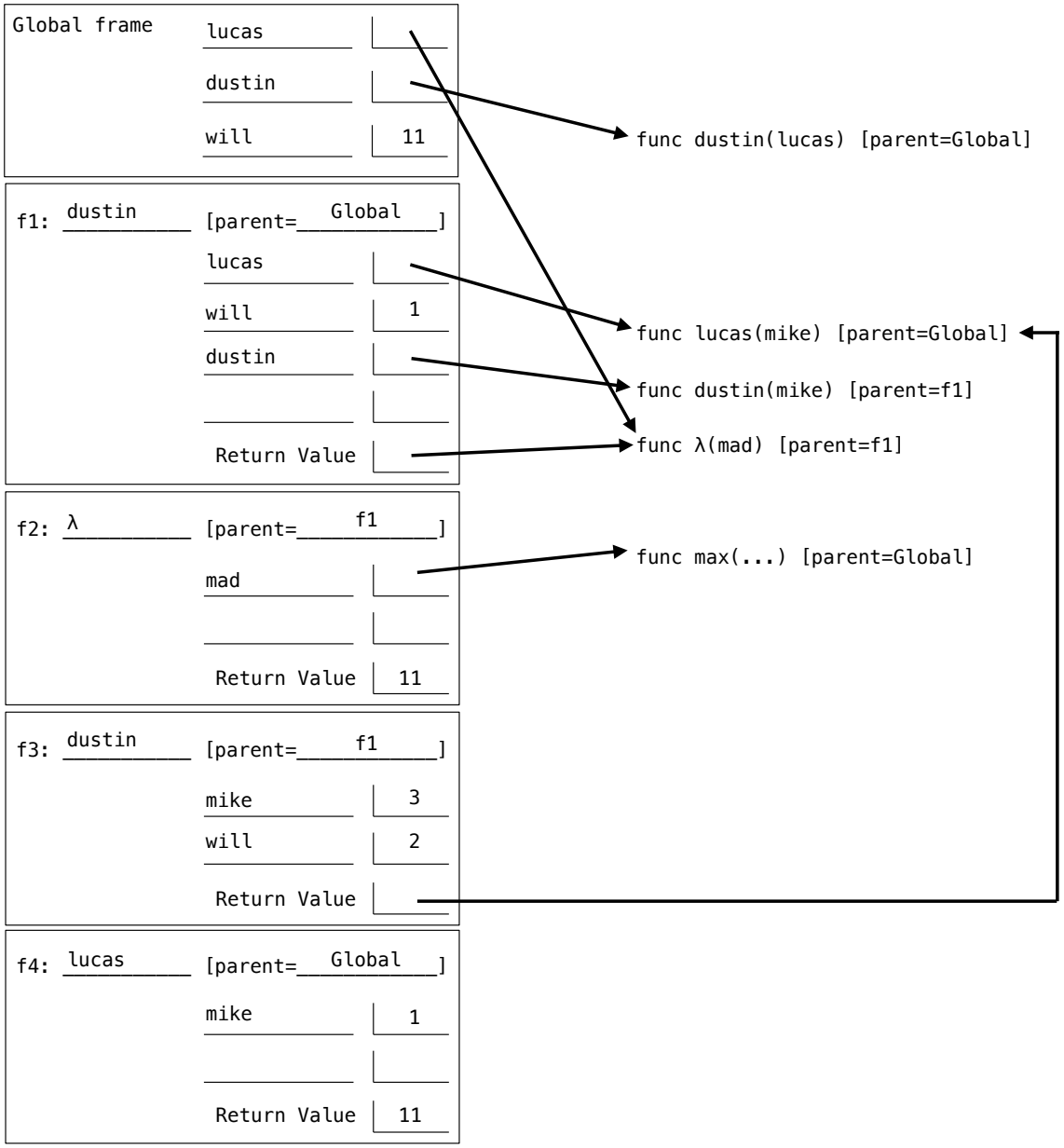
Expression	Interactive Output
`sub(pow(10, 2), 1)`	99
`print(4, 5) + 1`	4 5 Error
`(print(2) or 3) // (0 or 1)`	2 3
`winterfell(2, print)`	2 3 None
`ny`	21
`tar(lambda x: x-7, 8)`	8 1
`night(king(2, 3), 4)`	-4
`jon(snow(5), 2)`	2 2 9

2. (8 points) Stranger Frames

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.* A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  def lucas(mike):
2      return will
3
4  def dustin(lucas):
5      will = 1
6      def dustin(mike):
7          will = 2
8          return lucas
9      return lambda mad: dustin(3)(will)
10
11 will = 5 + 6
12 lucas = dustin(lucas)
13 lucas(max)
```



3. (10 points) Choose Wisely

(a) (4 pt) Implement `sum_some`, which takes a non-negative integer `n` and a function `p`. It returns the sum of all the digits `d` for which `p` returns a true value when given `d` as an argument. Assume that the function `p` takes a single digit `d` (from 0 to 9) and returns either `True` or `False`.

```
def sum_some(n, p):
    """Return the sum of the digits of N for which P returns a true value.

    >>> even = lambda d: d % 2 == 0
    >>> big = lambda d: d > 5
    >>> sum_some(124567, even)     # Sum the even digits: 2 + 4 + 6
    12
    >>> sum_some(124567, big)      # Sum the big digits: 6 + 7
    13
    """
    total = 0

    while n:

        if p(n % 10):

            total += n % 10

        n = n // 10

    return total
```

(b) (4 pt) Implement `sum_largest`, which takes non-negative integers `n` & `k`. It sums the largest `k` digits of `n`.

```
def sum_largest(n, k):
    """Return the sum of the K largest digits of N.

    >>> sum_largest(2018, 2)    # 2 and 8 are the two largest digits (larger than 0 and 1).
    10
    >>> sum_largest(12345, 10)  # There are only five digits, so all are included in the sum.
    15
    """
    if n == 0 or k == 0:

        return 0

    a = n % 10 + sum_largest(n // 10, k - 1)

    b = sum_largest(n // 10, k)

    return max(a, b)
```

(c) (2 pt) Complete the expression below by **only adding parentheses** so that the whole expression evaluates to 2018. Each blank should be filled with one or more parentheses.

```
(lambda a, x: x + (lambda y: lambda z: y+z+1000)(1000 _)(_ 10 _))(_ 5, _(_ lambda: 8 _)()_ )
```

4. (10 points) Editor

Definitions. An *edit* is a pure function that takes a non-negative integer and returns a non-negative integer. An *editor* for a non-negative integer `n` is a function that takes an *edit*, applies it to `n`, displays the result, and then returns an editor for the result.

(a) (3 pt) Implement `make_editor`, which takes a non-negative integer `n` and a one-argument function `pr`. It returns an *editor* for `n` that uses `pr` to display the result of each *edit*.

(b) (5 pt) Implement `insert`, which takes a single digit `d` (from 0 to 9) and a non-negative position `k`. It returns an *edit* that inserts `d` into its argument `n` at position `k`, where `k` counts the number of digits from the end of `n`. **Assume that `k` is not larger than the number of digits in `n`. Your solution must be recursive.**

(c) (2 pt) Implement `delete`, which takes a non-negative integer `k` and returns an *edit* that deletes the last `k` digits of its argument `n`. You may use `pow` or `**` in your solution.

```
def make_editor(n, pr):
    """Return an editor for N.

    >>> f = make_editor(2018, lambda n: print('n is now', n))
    >>> f = f(delete(3))     # delete the last 3 digits from the end of 2018
    n is now 2
    >>> f = f(insert(4, 0))  # insert digit 4 at the end of 2 (position 0)
    n is now 24
    >>> f = f(insert(3, 1))  # insert digit 3 in the middle of 24 (position 1)
    n is now 234
    >>> f = f(insert(1, 3))  # insert digit 1 at the start of 234 (position 3)
    n is now 1234
    >>> f = make_editor(123, print)(delete(10))  # delete 10 digits from the end of 123
    0
    """
    def editor(edit):

        result = edit(n)

        pr(result)

        return make_editor(result, pr)

    return editor

def insert(d, k):

    def edit(n):

        if k == 0:

            return d + 10 * n

        else:

            return n % 10 + 10 * insert(d, k-1)(n//10)

    return edit

delete = lambda k: lambda n: n // 10 ** k
```

No more questions.

CS 61A Spring 2018 — Structure and Interpretation of Computer Programs

MIDTERM 2 SOLUTIONS

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except two hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 and midterm 2 study guides.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, and `abs`.
- You **may not** use example functions defined on your study guides unless clearly specified by the question.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.
- You may use the `Tree`, `Link`, and `BTree` classes defined on Page 2 (left column) of the Midterm 2 Study Guide.

1. (10 points) Buggy Quidditch

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines.

- If an error occurs, write **Error**, but include all output displayed before the error.
- To display a function value, write **Function**.
- If an expression would take forever to evaluate, write **Forever**.

The interactive interpreter displays the contents of the `repr` string of the value of a successfully evaluated expression, unless it is `None`.

Assume that you have started `python3` and executed the code shown on the left first, then you evaluate each expression on the right in the order shown. Expressions evaluated by the interpreter have a cumulative effect.

```
1  class Ball:
2      points = 0
3      time = lambda: 'Draco'
4
5      def score(self, who):
6          print(who, self.points)
7
8      def __str__(self):
9          return 'Magic'
10
11 class Snitch(Ball):
12     points = 100
13     time = lambda: 'Harry'
14
15     def __init__(self):
16         self.points = self.points + 50
17
18     def score(self, p):
19         if not time():
20             print(Ball().score(p))
21         else:
22             Ball.score(self, p)
23
24 def chase(r):
25     r.time = Snitch.time
26     r.points += 1
27     quaffle.points += 10
28     print(r().points)
29
30 quaffle = Ball()
31 quaffle.points = 10
32 chasing = quaffle.score
33 time = lambda: Ball.points
34 malfoy = lambda: Ball.time()
```

Expression	Interactive Output
`Snitch().points`	150
`chasing(quaffle)`	Magic 10
`Snitch().score('Seeker')`	Seeker 0 None
`chase(Ball)`	1
`Snitch().score(malfoy())`	Harry 150

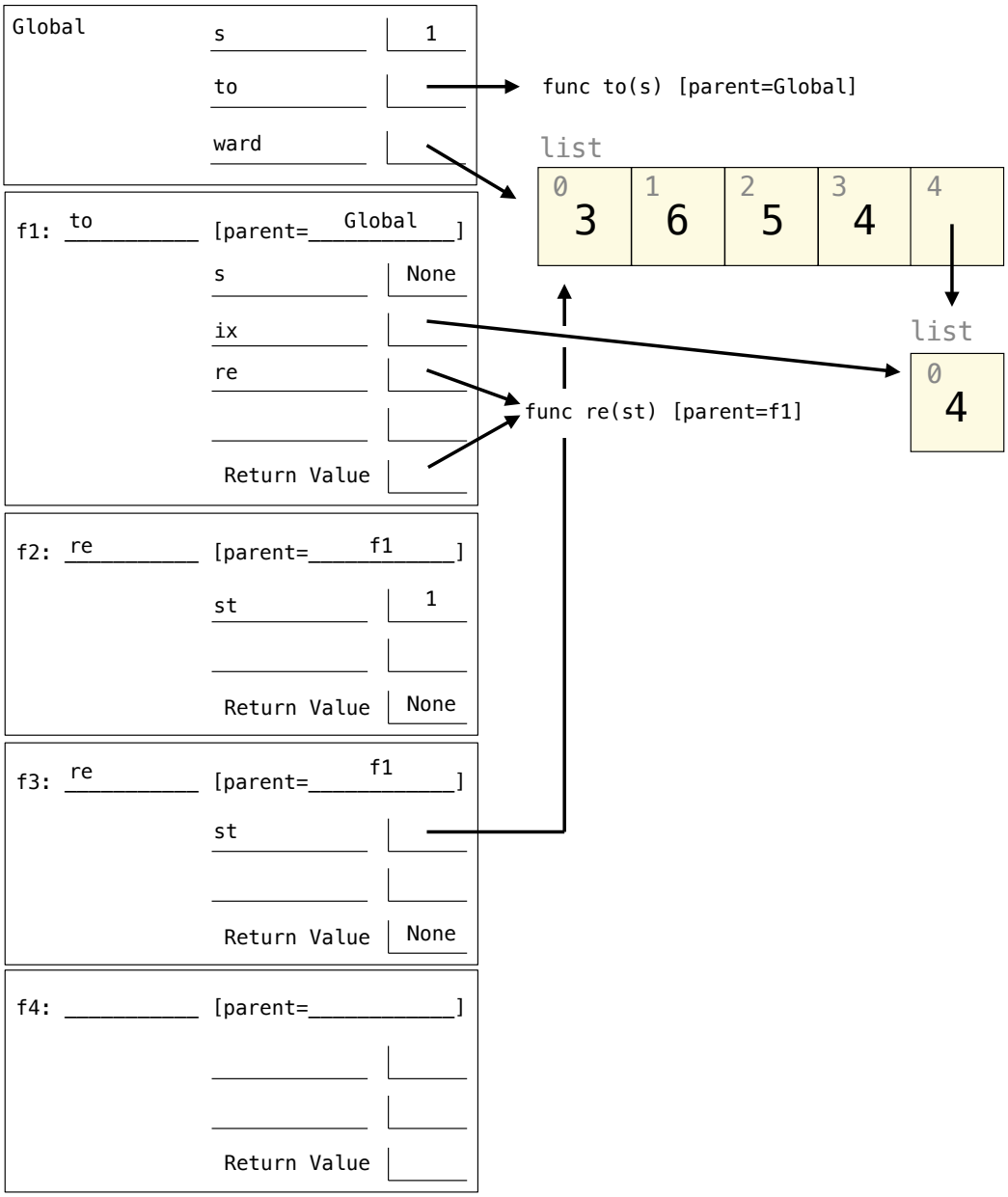
2. (6 points) NVRnment

Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*

A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.
- **Use box-and-pointer diagrams for lists and tuples.**

```
1  s = 1
2  def to(s):
3      ix = s[1:2]
4      ward[1] = 6
5      def re(st):
6          if st is not ward:
7              nonlocal s
8              s = s.extend(ix)
9              return re(ward)
10         else:
11             st.append(ix)
12     return re
13 ward = [3, 4] + list([5])
14 to(ward)(s)
```



3. (12 points) Lists

Definition. A *grid* is a list of lists. Each list in a grid is called a *row*, and all rows must have the same length. `[[1, 2], [3, 4]]` is a grid of integers, but `[[1, 2], [3, 4, 5]]` is not a grid.

(a) (2 pt) Implement *column*, which takes a grid `g` and a non-negative integer `c` that is smaller than the length of a row in `g`. It returns a list containing the element at index `c` of each row in the grid.

```
def column(g, c):
    """Return the column of g at index c.

    >>> column([[3, 4, 5], [6, 7, 8], [9, 10, 11]], 1)
    [4, 7, 10]
    """

    return [row[c] for row in g]
```

(b) (4 pt) Implement `print_grid`, which takes a grid `g`. It prints one line for each row in `g`. Line k displays each element in row k, separated by the minimum number of spaces needed to align the left edges of the `str` strings for the elements of each column of the grid. At least one space should appear between any two elements. Extra spaces at the end of a line are ok. **You may not use the `column` function above. Assume that `g` has at least one row.**

```
def print_grid(g):
    """Print each row on a separate line with columns aligned.

    >>> print_grid([[1, 234, 50, 4, 5], [67, 8, 90, 0, 500], [3, 4, 5, -500, 7]])
    1  234 50 4    5
    67 8   90 0    500
    3  4   5  -500 7
    """

    cs = range(len(g[0]))


    widths = [max([len(str(row[c])) for row in g]) for c in cs]


    for row in g:


        line = ''


        for c in cs:


            s = str(row[c])


            line = line + s + ' ' * (widths[c] - len(s) + 1)


        print(line)
```

(c) (4 pt) Implement `expand`, which takes a grid `g`, a number of rows `h`, a number of columns `w`, and a `fill` value. It mutates the contents of `g` so that `g` has at least `h` rows and `w` columns. Any added values are `fill`.

```
def expand(g, h, w, fill):
    """Expand grid g so that it has at least h rows and w columns.

    >>> g = [[1, 2, 3], [40, 50, 60]]

    >>> print_grid(expand(g, 2, 5, 10))
    1  2  3  10 10
    40 50 60 10 10

    >>> print_grid(expand(g, 5, 6, 0))
    1  2  3  10 10 0
    40 50 60 10 10 0
    0  0  0  0  0  0
    0  0  0  0  0  0
    0  0  0  0  0  0

    >>> print_grid(expand(g, 0, 0, 5))
    1  2  3  10 10 0
    40 50 60 10 10 0
    0  0  0  0  0  0
    0  0  0  0  0  0
    0  0  0  0  0  0
    """
    for row in g:

        row.extend([fill] * (w - len(row)))
        # Alternate correct answer: row.extend([fill] * max(0, (w - len(row))))

    for k in range(h - len(g)):
        # Alternate correct answer: range(max(0, h - len(g)))

        g.append([fill] * w)
        # Note: This solution fails if w < len(g[0]) but h > len(g),
        #       a case that was never demonstrated in the doctests.
        #       So, it was given full credit.
        #       A full-credit solution that works for all non-empty g is:
        #           g.append([fill] * len(g[0]))
        #       A correct solution that works for all g is more complicated:
        #           g.append([fill] * ((g and len(g[0])) or w))

    return g
```

(d) (2 pt) Circle the Θ expression that describes how many new values must be added when a grid with n rows and n columns is expanded to $2 \times n$ rows and $2 \times n$ columns using the `expand` function. Assume that `expand` is implemented correctly.

$\Theta(1)$ $\Theta(\log n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(2^n)$ None of these

4. (12 points) Sequences

(a) (6 pt) Implement `stretch`, which takes a `Link` instance `s` with no cycles. It mutates `s` so that, for each position k in the original `s`, the k^{th} element is repeated k times. **You do not need to use the name `i`.**

```
def stretch(s, repeat=0):
    """Replicate the kth element k times, for all k in s.

    >>> a = Link(3, Link(4, Link(5, Link(6))))
    >>> stretch(a)
    >>> print(a)
    <3, 4, 4, 5, 5, 5, 6, 6, 6, 6>
    """
    if s is not Link.empty:

        for i in range(repeat):


            s.rest = Link(s.first, s.rest)


            s = s.rest


        stretch(s.rest, repeat + 1)
```

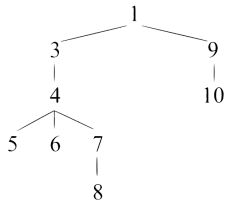
(b) (6 pt) Implement `combo`, which takes two non-negative integers `a` and `b`. It returns the smallest integer that contains all of the digits of `a` in order, as well as all of the digits of `b` in order.

```
def combo(a, b):
    """Return the smallest integer with all of the digits of a and b (in order).

    >>> combo(531, 432)      # 45312 contains both _531_ and 4_3_2.
    45312
    >>> combo(531, 4321)     # 45321 contains both _53_1 and 4_321.
    45321
    >>> combo(1234, 9123)    # 91234 contains both _1234 and 9123_.
    91234
    >>> combo(0, 321)        # The number 0 has no digits, so 0 is not in the result.
    321
    """
    if min(a, b) == 0:

        return a + b

    elif a % 10 == b % 10:

        return combo(a // 10, b // 10) * 10 + a % 10

    else:

        return min(combo(a // 10, b) * 10 + a % 10,

                   combo(a, b // 10) * 10 + b % 10)
```

5. (10 points) Trees

Definition. A *sibling* of a node in a tree is another node with the same parent.

(a) (4 pt) Implement `siblings`, which takes a `Tree` instance `t`. It returns a list of the labels of all nodes in `t` that have a sibling. These labels can appear in any order.




```
def siblings(t):
    """Return a list of the labels of all nodes that have siblings in t.

    >>> a = Tree(4, [Tree(5), Tree(6), Tree(7, [Tree(8)])])
    >>> siblings(Tree(1, [Tree(3, [a]), Tree(9, [Tree(10)])]))
    [3, 9, 5, 6, 7]
    """

    result = [b.label for b in t.branches if len(t.branches) > 1]

    for b in t.branches:

        result.extend(siblings(b))

    return result
```

(b) (6 pt) Implement the `Sib` class that inherits from `Tree`. In addition to `label` and `branches`, a `Sib` instance `t` has an attribute `siblings` that stores the number of siblings `t` has in `Sib` trees containing `t` as a node. **Assume that the `branches` of a `Sib` instance will never be mutated or re-assigned.**

```
class Sib(Tree):
    """A tree that knows how many siblings it has.

    >>> a = Sib(4, [Sib(5), Sib(6), Sib(7, [Sib(8)])])
    >>> a.siblings
    0
    >>> a.branches[1].siblings
    2
    """
    def __init__(self, label, branches=[]):

        self.siblings = 0

        for b in branches:

            b.siblings += len(branches) - 1

        Tree.__init__(self, label, branches)
```

CS 61A Spring 2019 — Structure and Interpretation of Computer Programs

MIDTERM 1 SOLUTIONS

INSTRUCTIONS

- You have 55 minutes to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 study guide.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, and `abs`.
- You **may not** use example functions defined on your study guides unless a problem clearly states you can.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.

1. (10 points, 15 minutes) What Would Python Display

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
n = 1
f = lambda: n
g = f
n = 2
```

```
x, y, loops = 0, 21, 0
while y:
    loops += 1
    if x <= y:
        if x % 2 == 1:
            x, y = y, x
        elif x == 0:
            x += (2 ** 2)
        else:
            x = x + 3
            y = y // 2
    else:
        y = 0
```

```
def foo(bar, z):
    def bar(w):
        z = w + 2
        return z
    print(bar(z))
    print(z)
    return bar
```

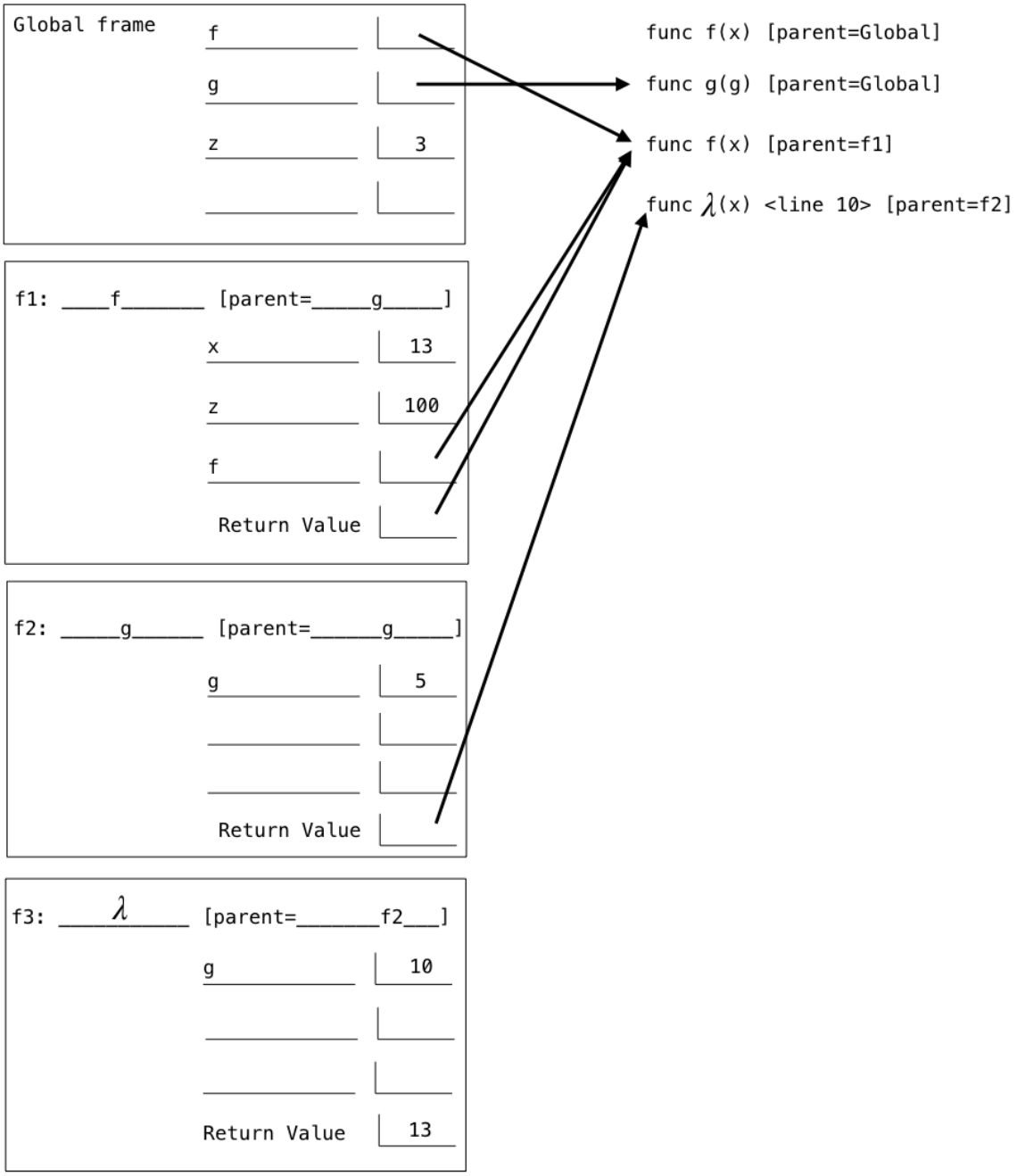
Expression	Interactive Output
`pow(10, 2)`	100
`print(4, 5) + 1`	4 5 Error
`print(1, print(print(2), 3 or 4 // 0))`	2 None 3 1 None
`g()`	2
`x`	10
`loops`	4
`foo(lambda m: m + 1, 4)(7)`	6 4 9

2. (10 points, 15 minutes) Environment Diagram

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.* A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  def f(x):
2    f = 10
3    z = 100
4    x = g(x)(f)
5    def f(x):
6      return x + y
7    return f
8
9  def g(g):
10   return lambda g: g + z
11
12 z = 3
13 f = f(5)
```



3. (10 points, 25 minutes) Zip It

We would like to create a function `make_zipper` that takes two functions `f1(x)` and `f2(x)` and a "zipper sequence", which is a number that contains a series of 1s and 2s. It returns a function that is the equivalent of `f1(f2(f2(...f1(x)...)))` in which the exact sequence of `f1`s and `f2`s is given by the digits of the sequence. As an example, if the sequence were `1211`, that would mean return a function of `x` that is the equivalent to `f1(f2(f1(f1(x))))`. Neither recursion nor containers (lists, dictionaries, sets, etc) are allowed in your solution.

```
def make_zipper(f1, f2, sequence):
    """ Return a function of f1 and f2 composed based on sequence.

    >>> def increment(x):
            return x + 1
    >>> def square(x):
            return x * x
    >>> do_nothing = make_zipper(increment, square, 0)
    >>> do_nothing(2)      # Don't call either f1 or f2, just return your input untouched
    2
    >>> incincsq = make_zipper(increment, square, 112)
    >>> incincsq(2)        # increment(increment(square(2))), so 2 -> 4 -> 5 -> 6
    6
    >>> sqincsqinc = make_zipper(increment, square, 2121)
    >>> sqincsqinc(2)      # square(increment(square(increment(2)))), so 2 -> 3 -> 9 -> 10 -> 100
    100
    """


    zipper = lambda x: x


    helper = lambda f, g: lambda x: f(g(x))


    while sequence != 0:


        if sequence % 10 == 1:


            zipper = helper(f1, zipper)


        else:


            zipper = helper(f2, zipper)


        sequence = sequence // 10


    return zipper
```

CS 61A Spring 2019 — Structure and Interpretation of Computer Programs

MIDTERM 2 SOLUTIONS

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except two hand-written 8.5" × 11" crib sheets of your own creation and the official CS 61A study guides.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, `abs`, `sum`, `next`, `iter`, `list`, `tuple`, `map`, `filter`, `zip`, `all`, and `any`.
- You **may not** use example functions defined on your study guides unless a problem clearly states you can.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.
- You may use the `Tree`, `Link`, and `BTree` classes defined on Page 2 (left column) of the Midterm 2 Study Guide.

1. (12 points) What Would Python Display

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
items, n = [], 2

class Airpods:
    cost, k = 200, 0
    f = lambda self: print(self)

    def __init__(self):
        Airpods.k += 1
        Airpods.f(self)
        items.extend([self])

    def __repr__(self):
        return (Airpods.k < 2 and "lonely") or "pair"

class TwoAirpods(Airpods):
    def __init__(self):
        self.k = 2
        Airpods.__init__(self)
        Airpods.__init__(self)

def discount(a):
    a.cost //= 2

def u(w, u):
    return [print(u) for u in [w, u]]

discount(Airpods)
```

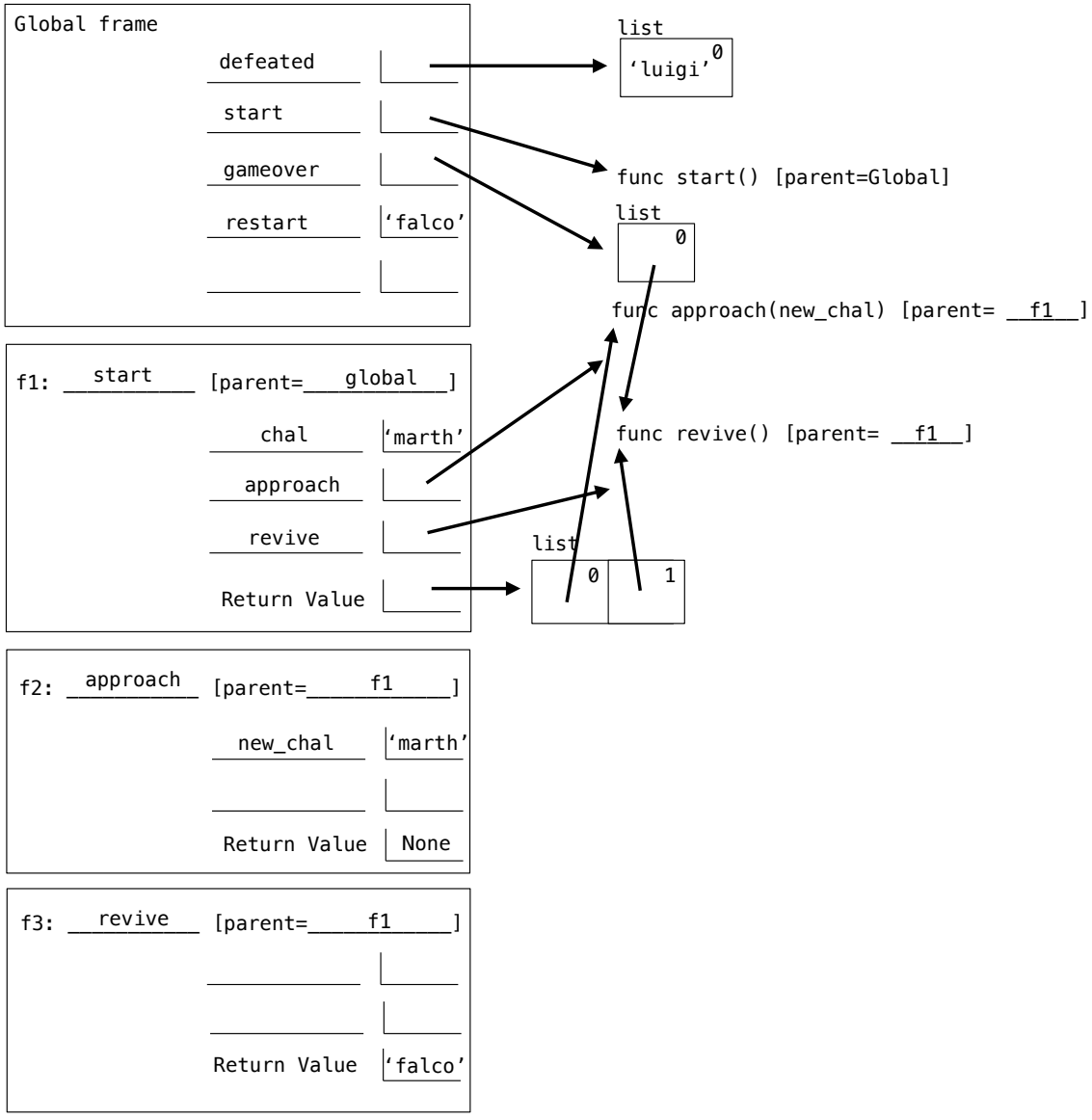
Expression	Interactive Output
`pow(10, 2)`	100
`print(Link(2, Link(3)))`	`<2, 3>`
`TwoAirpods.cost`	100
`lost = Airpods()`	lonely
`willbelost = TwoAirpods()`	pair pair
`str(lost)`	`'pair'`
`[item.k for item in items]`	`[3, 2, 2]`
`u(lost, willbelost)`	pair pair `[None, None]`

2. (10 points) Ultimate

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*
A complete answer will:

- Use box-and-pointer notation for all lists.
- Add missing names and parents to all local frames.
- Add missing values created or referenced during execution.
- Show the return value for each local frame.

```
1  defeated = ['luigi']
2  def start():
3      chal = 'falco'
4      def approach(new_chal):
5          nonlocal chal
6          defeated.append(chal)
7          chal = new_chal
8      def revive():
9          return defeated.pop()
10     return [approach, revive]
11 gameover = start()[:]
12 gameover.pop(0)('marth')
13 restart = gameover[0]()
```



3. (5 points) Deep lists

Implement `in_nested` which takes in a value `v` and a nested list or an individual value `L` and returns whether the value is contained in the list.

Hint: The built-in function `type` takes an object and returns the type of that object.

```
# Solution 1
def in_nested(v, L):
    """
    >>> find(5, [1, 2, [[3], 4]])
    False
    >>> in_nested(9, [[[1], [6, 4, [5, [9]]], 7], 7, 7])
    True
    >>> find(1, 1)
    True
    """
    if type(L) != list:

        return L == v

    else:

        return any([in_nested(v, sub) for sub in L])



# Solution 2
def in_nested(v, L):
    """
    >>> find(5, [1, 2, [[3], 4]])
    False
    >>> in_nested(9, [[[1], [6, 4, [5, [9]]], 7], 7, 7])
    True
    >>> find(1, 1)
    True
    """
    if type(L) == type(v) or L == []:

        return L == v

    else:

        return in_nested(v, L[0]) or in_nested(v, L[1:])
```

4. (8 points) A data structure by any other form would smell just as sweet...

(a) (6 pt) Implement `link_to_dict` which takes a linked list encoding a "flattened" dictionary (in which elements are key1 → value1 → key2 → value2 → key3 → value3, etc), **removes all the values**, and returns the equivalent dictionary. The input and returned list **may** include duplicate keys, as in the example below. You may assume the linked list always contains an even number of elements. The `Link` class is provided below.

```
class Link:
   empty = ()

   def __init__(self, first, rest=empty):
       self.first = first
       self.rest = rest

   def __str__(self):
       string = '<'
       while self.rest is not Link.empty:
           string += str(self.first) + ', '
           self = self.rest
       return string + str(self.first) + '>'

def link_to_dict(L):
    """
    >>> L = Link(1, Link(2, Link(3, Link(4, Link(1, Link(5))))))
    >>> print(L)
    <1, 2, 3, 4, 1, 5>
    >>> link_to_dict(L)
    {1: [2, 5], 3: [4]}
    >>> print(L)
    <1, 3, 1>
    """
    D = {}

    while L is not Link.empty:

        key, value =  L.first, L.rest.first

        if key not in D:

            D[key] = [value]

        else:

            D[key].append(value)

        L.rest, L = L.rest.rest, L.rest.rest

    return D
```

(b) (2 pt) Circle the Θ expression that describes the number of iterations of the while loop in `link_to_dict` where `n` is the length of the list.

$\Theta(1)$ $\Theta(\log n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(2^n)$ None of these

5. (10 points) I speak for the Trees

Execute each line of code in order. If a line errors, assume we didn't type that line. For each line, indicate whether it was:

- (E) an error
- (D) a Data Abstraction Violation
- (OK) perfectly fine code.

Please fill in the bubbles completely.

```
def tree(label, branches=[]):
    return [label] + list(branches)

def label(tree):
    return tree[0]

def branches(tree):
    return tree[1:]

def print_tree_adt(t, indent=0):
    print('  ' * indent + str(label(t)))
    for b in branches(t):
        print_tree_adt(b, indent + 1)


class Tree:
    def __init__(self, label, branches=[]):
        self.label = label
        self.branches = list(branches)

    def __str__(self):
        return '\n'.join(self.indented())

    def indented(self, k=0):
        ind = []
        for b in self.branches:
            for line in b.indented(k + 1):
                ind.append('  ' + line)
        return [str(self.label)] + ind
```

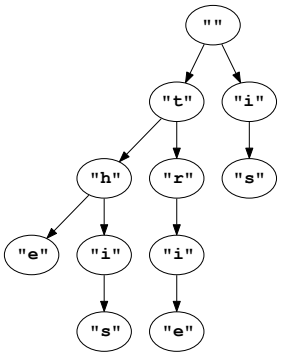
E	D	**OK**	`t = Tree(3, [Tree(1), Tree(2)])`
E	D	**OK**	`L = [tree(1)]`
E	D	**OK**	`L.append(tree(2))`
E	D	**OK**	`s = tree(3, L)`
E	**D**	OK	`s`
E	**D**	OK	`print(s)`
E	D	**OK**	`t`
E	D	**OK**	`t.label`
E	D	**OK**	`print(t)`
E	D	**OK**	`label(s)`
E	D	**OK**	`t.label = 4`
E	D	OK	`label(s) = 4`
E	**D**	OK	`s[0] = 5`
E	**D**	OK	`print_tree_adt([5])`
E	D	**OK**	`print_tree_adt(s)`
E	D	**OK**	`t.branches[0].label = s`
E	D	OK	`print_tree_adt(t.branches[0])`
E	D	**OK**	`print_tree_adt(t.branches[0].label)`
E	D	**OK**	`t`
E	**D**	OK	`print(t)`

6. (15 points) Trie this

A Trie is a Tree where every node in the tree contains a single letter except for the root which is always the empty string. Every path from the root to a leaf forms a word. You may assume no words are substrings of other words in the trie (e.g., "hi" and "him"). The figure below is a trie generated by storing the words ["this", "is", "the", "trie"]. The `Tree` class is defined on Page 2 of the Midterm 2 Study Guide.

(a) (7 pt) Implement `add_word` which takes a Trie and a word and adds the word to the trie.

```
def make_trie(words):
    """ Makes a tree where every node is a letter of a word.
        All words end as a leaf of the tree.
        words is given as a list of strings.
    """
    trie = Tree('')
    for word in words:
        add_word(trie, word)
    return trie
```




```
def add_word(trie, word):

    if word == '':
        return

    branch = None

    for b in trie.branches:

        if b.label == word[0]:

            branch = b

    if not branch:

        branch = Tree(word[0])

        trie.branches.append(branch)

    add_word(branch, word[1:])
```

(b) (8 pt) Implement `get_words`, which takes a Trie and returns a list of all the words the Trie is storing.

```
def get_words(trie):
    """
    >>> get_words(make_trie(['this', 'is', 'the', 'trie']))
    ['this', 'the', 'trie', 'is']
    """
    if trie.is_leaf():

        return [trie.label]

    return sum([[trie.label + word for word in get_words(branch)] for branch in trie.branches], [])
```

No more questions.

CS 61A Spring 2020 — Structure and Interpretation of Computer Programs

MIDTERM 1 SOLUTIONS

INSTRUCTIONS

- You have 1 hour and 50 minutes to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except one hand-written 8.5" × 11" crib sheet of your own creation and the official CS 61A midterm 1 study guide.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (`_@berkeley.edu`)	
TA	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, and `abs`.
- You may not use lists, dictionaries, tuples, sets, or the `:=` operator. These features have not been covered.
- You **may not** use example functions defined on your study guide unless a problem clearly states you can.
- For fill-in-the-blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.

1. (8 points) What Would Python Display

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If evaluation would run forever, write "Forever". To display a function value, write "Function". The first two rows have been provided as examples.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have first started `python3` and executed the statements on the left.

```
def tik(tok):
    tik = lambda: tok + 1
    while tok - tik() < 5:
        tok = tik() + 1
        if tok > 100:
            return tik()
    return tok
```

```
snap = lambda chat: lambda: snap(chat)
snap, chat = print, snap(2020)
```

```
def q(q):
    if print(q, q):
        print(q + 1)
    if q:
        q = q + q
    if q > 0:
        return q
    print(q + 2)
```

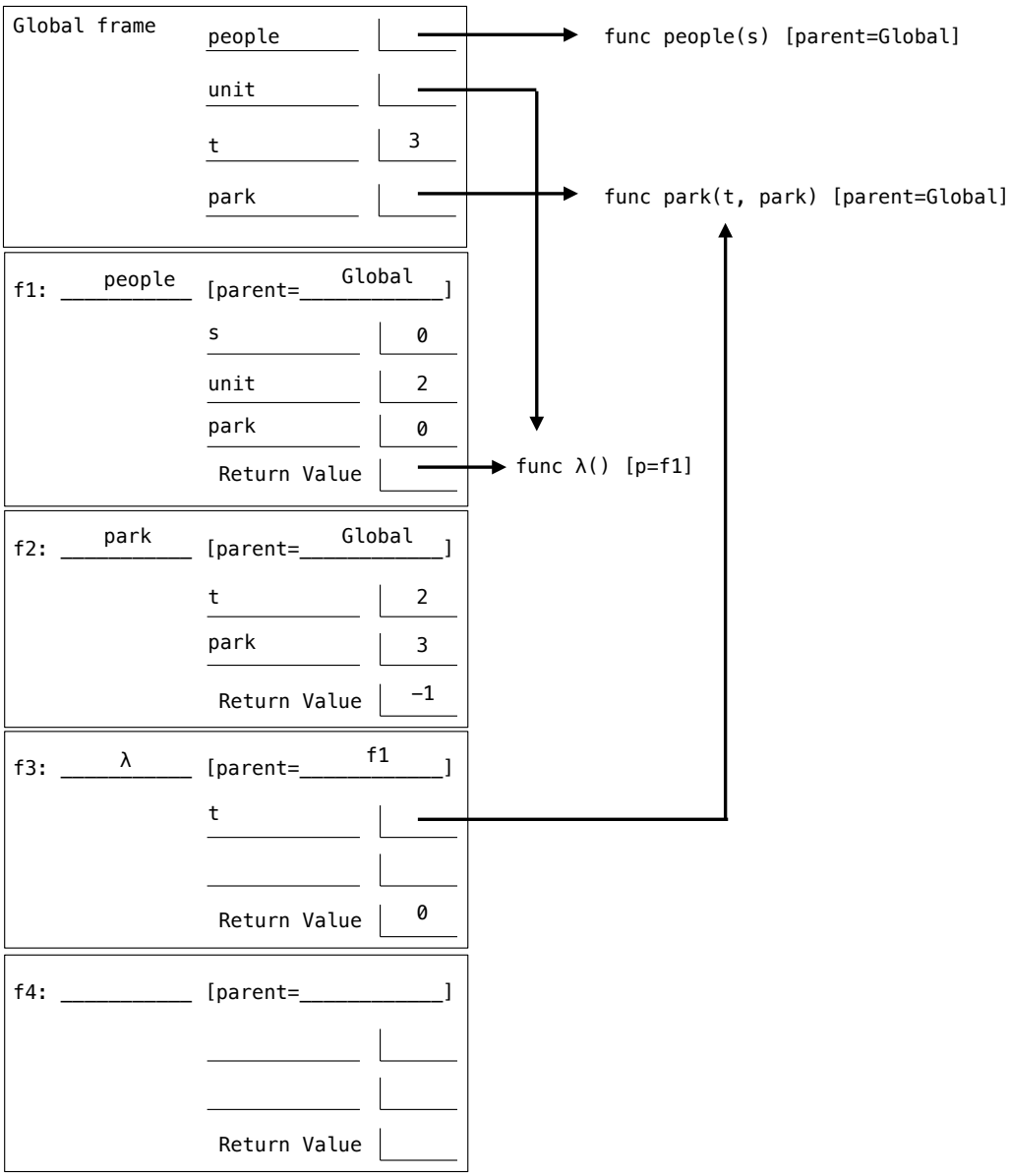
	Expression	Interactive Output
	`pow(10, 2)`	100
	`print(4, 5) + 1`	4 5 Error
(2 pt)	`print(print)(2020)`	Function Error
(2 pt)	`tik(50)`	103
(1 pt)	`chat(2020)`	Error
(1 pt)	`chat()`	2020
(2 pt)	`q(20)`	20 20 40

2. (8 points) People's Park

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled.
You may not need to use all of the spaces or frames.
Do not include frames for calls to built-in functions.
A complete answer will:

- Add all missing names and parents to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.
- Cross out or erase all arrows and values that are not part of the final diagram.

```
1  def people(s):
2      unit = 2
3      park = s(unit, t) + 1
4      s = (lambda t: park)(s)
5      return lambda: abs(unit)
6
7  unit, t = 1, 3
8  def park(t, park):
9      return unit - 2
10 unit = people(park)
```



3. (16 points) Digit Fidget

(a) (6 pt) Implement `same_digits`, which takes two positive integers. It returns whether they both become the same number after replacing each sequence of a digit repeated consecutively with only one of that digit. For example, in 12222321, the sequence 2222 would be replaced by only 2, leaving 12321.

Restriction: You may only write combinations of the following in the blanks:
`a`, `b`, `end`, `10`, `%`, `if`, `while`, `and`, `or`, `==`, `!=`, `True`, `False`, and `return`. (**No division allowed!**)

```
def same_digits(a, b):
    """Return whether a and b become the same number after removing adjacent repeats.

    >>> same_digits(2002200, 2202000) # Ignoring repeats, both are 2020
    True
    >>> same_digits(21, 12)           # Digits must appear in the same order
    False
    >>> same_digits(12, 2212)         # 12 and 212 are not the same
    False
    >>> same_digits(2020, 20)         # 2020 and 20 are not the same
    False
    """
    assert a > 0 and b > 0
    while a and b:

        if a % 10 == b % 10:

            end = a % 10

            while a % 10 == end:

                a = a // 10

            while b % 10 == end:

                b = b // 10

        else:

            return False

    return a == b
```

(b) (3 pt) Implement `no_repeats`, which takes a positive integer `a` and returns the smallest positive integer `b` for which `same_digits(a, b)` returns `True`. Assume `same_digits` is implemented correctly. Watch out for the `assert` statement in the implementation of `same_digits`! You may **not** call `set` or `str`.

```
def no_repeats(a):
    """Remove repeated adjacent digits from a.

    >>> no_repeats(22000200)
    2020
    """
    return search(lambda b: same_digits(a, b), 1)

def search(f, x):
    while not f(x):
        x += 1
    return x
```

(c) (4 pt) Implement `unique_largest`, which takes a positive integer `n`. It returns whether the largest digit in `n` appears only once in `n`. You may assign values to multiple names in an assignment statement.

You may **not** write call expressions or parentheses. You may **not** write `lambda`, `if`, `max`, `set`, `and`, or `or`.

```
def unique_largest(n):
    """Return whether the largest digit in n appears only once.

    >>> unique_largest(132123)  # 3 is largest and appears twice
    False
    >>> unique_largest(1321523) # 5 is largest and appears only once
    True
    >>> unique_largest(5)
    True
    """
    assert n > 0
    top = 0

    while n:

        n, d = n // 10, n % 10

        if d > top:

            top, unique = d, True

        elif d == top:

            unique = False

    return unique
```

(d) (3 pt) Implement `transitive`, which takes a two-argument function `p` that returns `True` or `False`. The `transitive` function returns whether it is the case that for every three digits `a`, `b`, `c` for which `p(a, b)` and `p(b, c)` both return `True`, `p(a, c)` also returns `True`. A digit is an integer between 0 and 9, inclusive.

You may **not** write `str`, `[`, or `]`.

```
def transitive(p):
    """Return whether p is transitive over non-negative single digit integers.

    >>> transitive(lambda x, y: x < y)          # if a < b and b < c, then a < c
    True
    >>> transitive(lambda x, y: abs(x-y) == 1)  # E.g., p(3, 4) and p(4, 5), but not p(3, 5)
    False
    """
    abc = 0
    while abc < 1000:

        a, b, c = abc // 100, (abc // 10) % 10, abc % 10


        if p(b, c) and p(a, b) and not p(a, c):

            return False

        abc = abc + 1

    return True
```

4. (8 points) Composition

(a) (4 pt) Implement `compose`, which takes a positive integer `n`. It returns a function that, when called repeatedly on n one-argument functions $f_1, f_2, \ldots, f_n$, returns a one-argument function of `x` that returns $f_1(f_2(\ldots f_n(x)\ldots))$. You may **not** call the `compose1` function from the Midterm 1 Study Guide.

```
def compose(n):
    """Return a function that, when called n times repeatedly on unary
    functions f1, f2, ..., fn, returns a function g(x) equivalent to
    f1(f2( ... fn(x) ... )).

    >>> add1 = lambda y: y + 1
    >>> compose(3)(abs)(add1)(add1)(-4)  # abs(add1(add1(-4)))
    2
    >>> compose(3)(add1)(add1)(abs)(-4)  # add1(add1(abs(-4)))
    6
    >>> compose(1)(abs)(-4)              # abs(-4)
    4
    """
    assert n > 0


    if n == 1:


        return lambda f: f


    def call(f):


        def on(g):


            return compose(n-1)(lambda x: f(g(x)))


        return on


    return call
```

(b) (4 pt) Complete the final expression below with **only integers and names** so it evaluates to 2020.

```
from operator import add

c = lambda f: lambda x: lambda y: f(x, y)
twice = lambda z: 2 * z


compose(3)(twice)(c(add)(10))(c(pow)(10))(3)
```

CS 61A Structure and Interpretation of Computer Programs
Spring 2021

Midterm 1 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

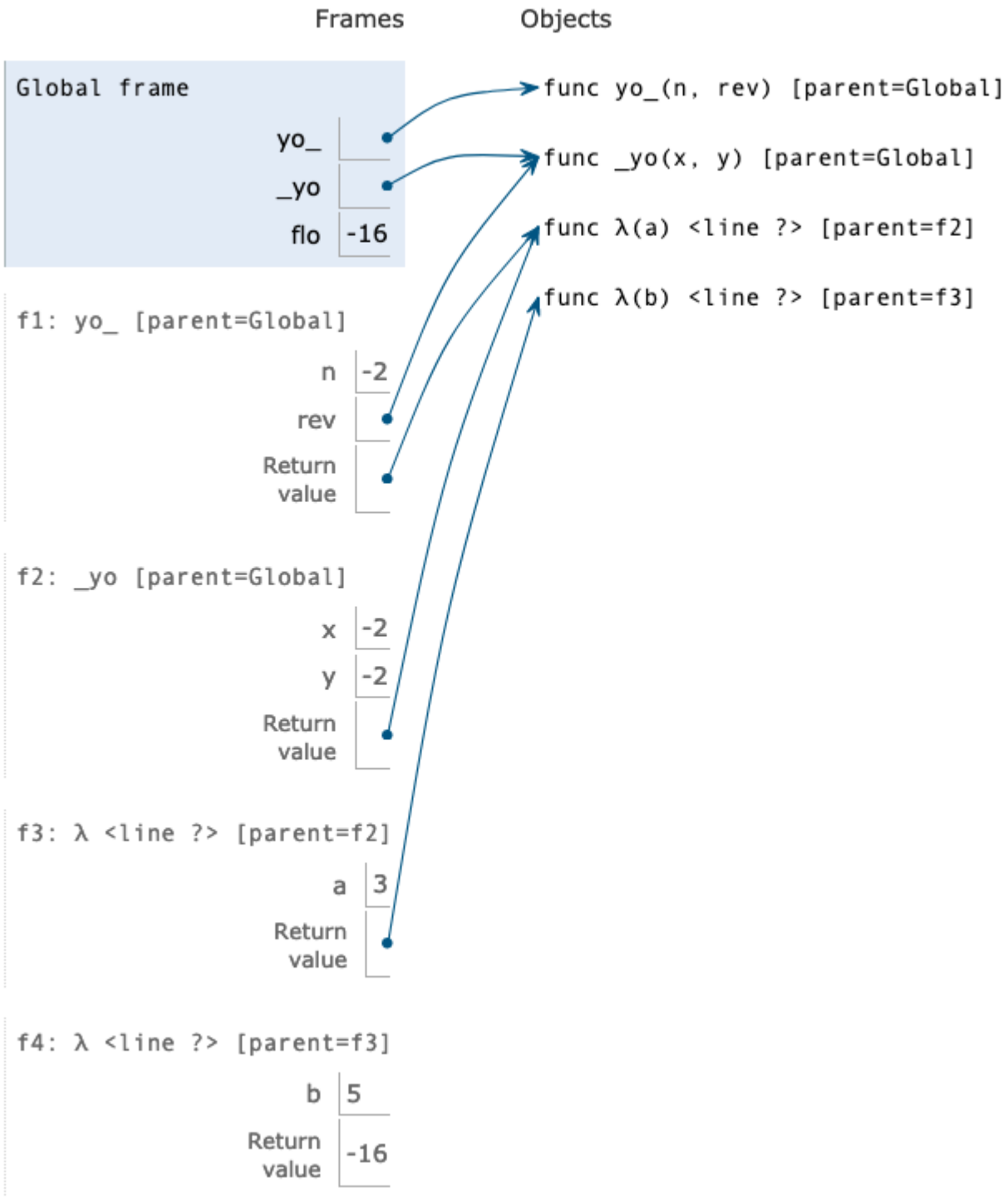
(a) What is your full name?

(b) What is your student ID number?

(c) By writing my name below, I pledge on my honor that I will abide by the rules of this exam and will neither give nor receive assistance. I understand that doing otherwise would be a disservice to my classmates, dishonor me, and could result in me failing the class.

1. (a) (8.0 points) Flow that Yo-Yo

The following environment diagram was generated by a program:



Click here to open the diagram in a new window

In this series of questions, you'll fill in the blanks of the program that follows so that its execution matches the environment diagram. You may want to fill in the blanks in a different order; feel free to answer the questions in whatever order works for you.

```
def yo_(n, rev):
    if n < 0:
        return ______
                 (a)
    elif n == 0:
        return float("inf")
    return n * -2

def _yo(x, y):
    if _______:
         (b)
        y += 1
    if _______:
         (c)
        return lambda a: ______
                              (d)
    return lambda a: lambda b: a + b

flo = yo_(_____, _____)(3)(5)
           (e)    (f)
```

(1.0 pt) Which of these could fill in blank (a)?

- ⓘ `lambda a: a - 3`
- ◯ `lambda a: rev(a, 3)(-1)`
- ◯ `rev`
- ◯ `yo_(rev)`
- ◯ `yo_`
- ● `rev(n, -3)`
- ◯ `lambda a: yo_(rev)`
- ◯ `_yo`

ii. (2.0 pt) Which of these could fill in blank (b)? *Select all that apply.*

- ■ `y < 0`
- ■ `x <= 0`
- ☐ `x == y`
- ■ `y <= 0`
- ☐ `y == 0`
- ■ `x < 0`

iii. (2.0 pt) Which of these could fill in blank (c)? *Select all that apply.*

- ☐ `x > 0`
- ☐ `x == -y`
- ☐ `y == -x`
- ■ `x == y`
- ☐ `y > 0`
- ■ `x < 0 and y < 0`

iv. (1.0 pt) Which of these could fill in blank (d)?

- ○ `x + y`
- ○ `lambda b: a**x + b**y`
- ○ `a*x + b*y`
- ○ `lambda b: a + b`
- ○ `x * y`
- ○ `lambda b: a*x * b*y`
- ○ `a**x + b**y`
- ● `lambda b: a*x + b*y`

v. (1.0 pt) Which of these could fill in blank (e)?

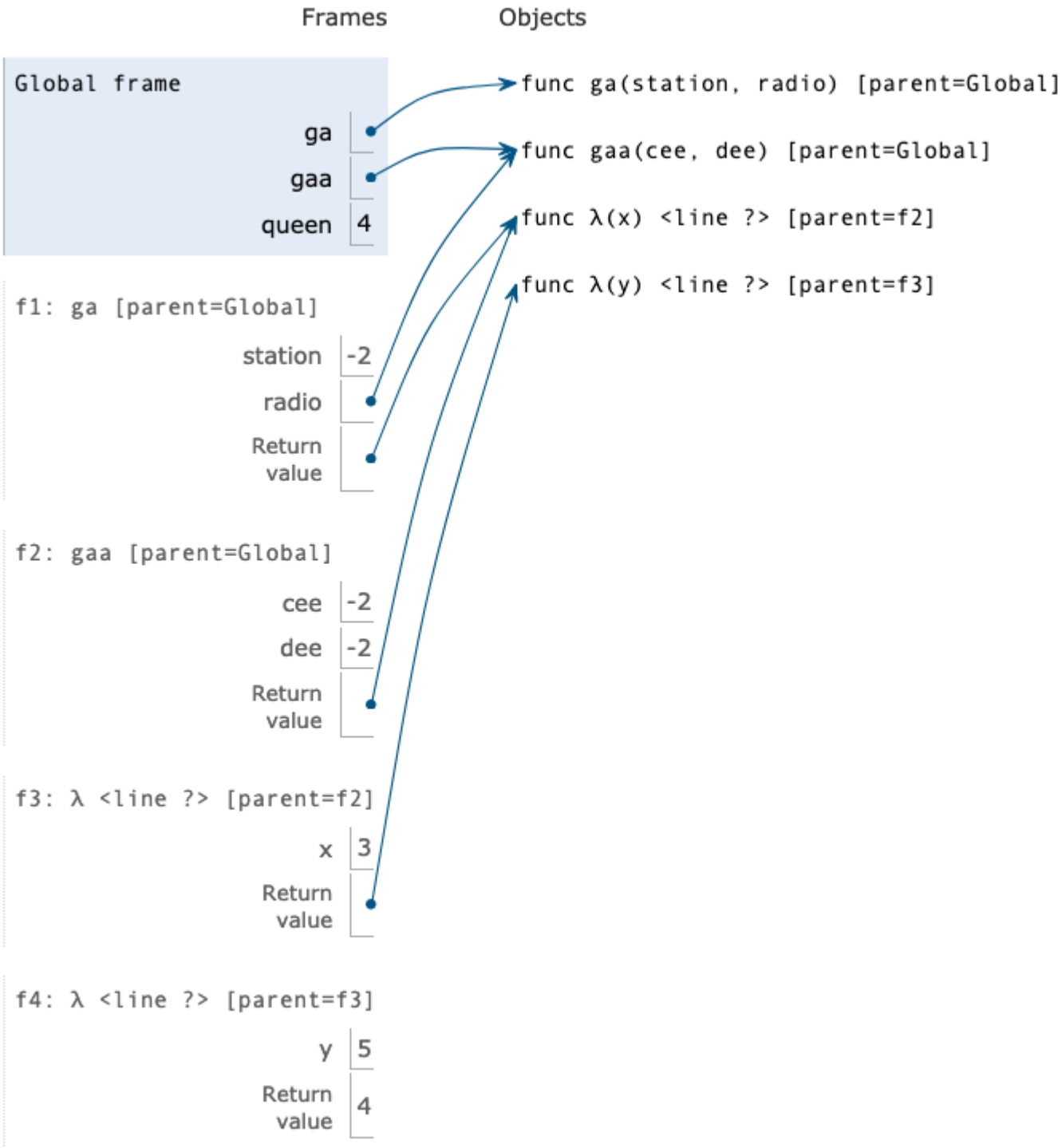
- ○ `-4`
- ○ `_yo`
- ○ `rev`
- ● `-2`
- ○ `yo_`
- ○ `-3`

vi. (1.0 pt) Which of these could fill in blank (f)?

- ○ `yo_`
- ○ `rev`
- ○ `lambda x: x`
- ● `_yo`
- ○ `lambda x: _yo(x)`
- ○ `flo`

(8.0 points) Radio Ga-Ga

The following environment diagram was generated by a program:



Click here to open the diagram in a new window

In this series of questions, you'll fill in the blanks of the program that follows so that its execution matches the environment diagram. You may want to fill in the blanks in a different order; feel free to answer the questions in whatever order works for you.

```
def ga(station, radio):
    if station < 0:
        return ______
                 (a)
    elif station == 0:
        return float("inf")
    return station * -98

def gaa(cee, dee):
    if _______:
         (b)
        dee += 1
    if _______:
         (c)
        return lambda x: ______
                           (d)
    return lambda x: lambda y: x * y

queen = ga(_____, _____)(3)(5)
           (e)    (f)
```

(1.0 pt) Which of these could fill in blank (a)?

(b) i.

- ○ `lambda x: station + x - 3`
- ○ `lambda x: radio(station, 3)(-1)`
- ○ `radio`
- ○ `ga(radio)`
- ○ `ga`
- ● `radio(station, -3)`
- ○ `lambda x: ga(gaa)`
- ○ `gaa`

ii. (2.0 pt) Which of these could fill in blank (b)? *Select all that apply.*

- ■ `cee < 0`
- ■ `dee <= 0`
- ☐ `cee == dee`
- ■ `cee <= 0`
- ☐ `cee == 0`
- ■ `dee < 0`

iii. (2.0 pt) Which of these could fill in blank (c)? *Select all that apply.*

- ☐ `cee > 0`
- ☐ `cee == -dee`
- ☐ `dee == -cee`
- ■ `cee == dee`
- ☐ `dee > 0`
- ■ `cee < 0 and dee < 0`

iv. (1.0 pt) Which of these could fill in blank (d)?

- ○ `cee - dee`
- ● `lambda y: x * cee - y * dee`
- ○ `cee * x - dee * x`
- ○ `lambda y: cee + y`
- ○ `cee * dee`
- ○ `lambda y: cee * x * dee * y`
- ○ `cee**x + dee**y`
- ○ `lambda y: x * cee + y * dee`

v. (1.0 pt) Which of these could fill in blank (e)?

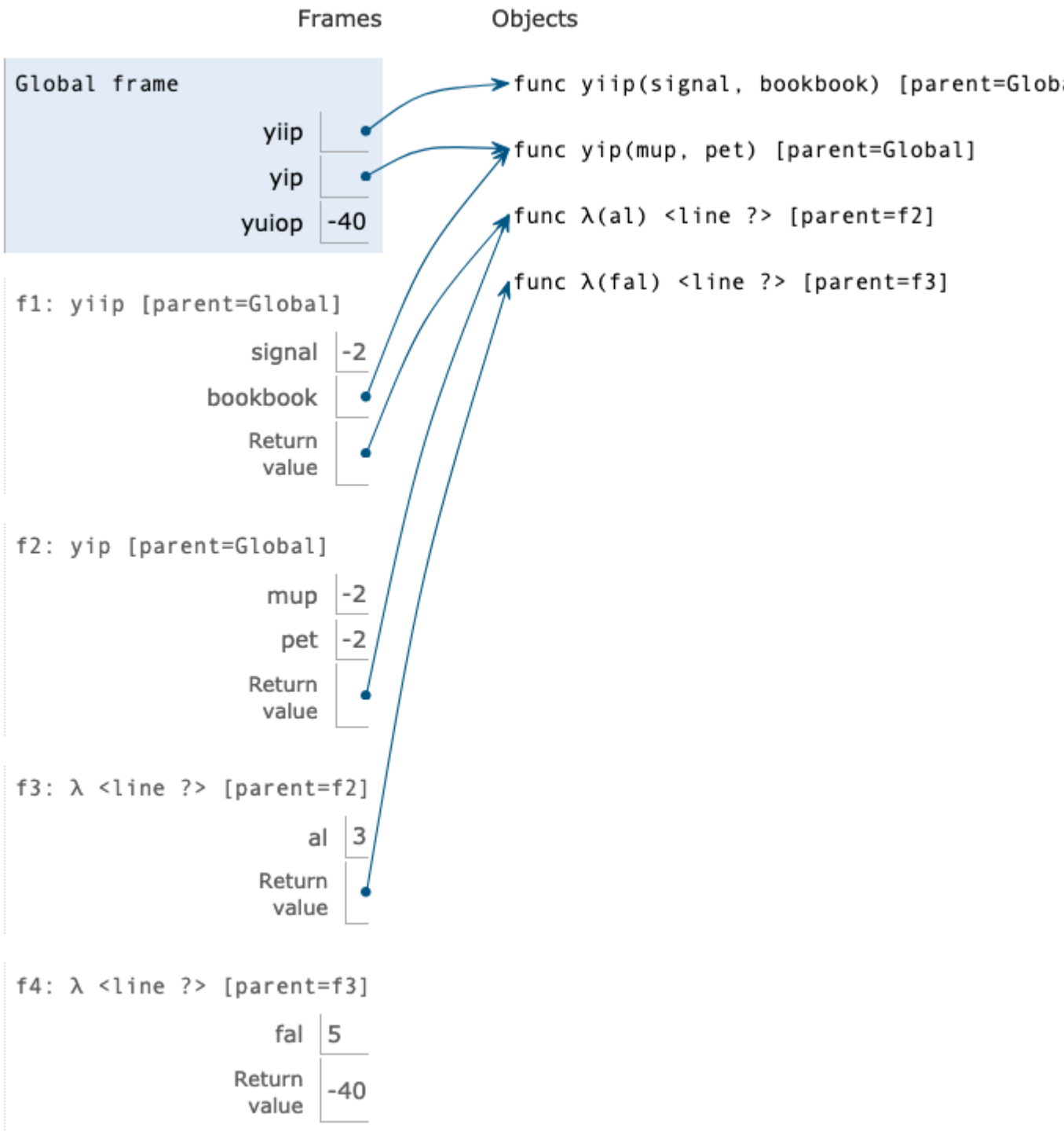
- ○ `gaa(-2)`
- ○ `gaa`
- ○ `radio(ga)`
- ● `-2`
- ○ `ga`
- ○ `station`

vi. (1.0 pt) Which of these could fill in blank (f)?

- ○ `ga`
- ○ `ga()`
- ○ `lambda n: n`
- ● `gaa`
- ○ `gaa()`
- ○ `lambda n: gaa(n)`

(8.0 points) YipYip Book

The following environment diagram was generated by a program:



Click here to open the diagram in a new window

In this series of questions, you'll fill in the blanks of the program that follows so that its execution matches the environment diagram. You may want to fill in the blanks in a different order; feel free to answer the questions in whatever order works for you.

```
def yiip(signal, bookbook):
    if signal < 0:
        return ______
                 (a)
    elif signal == 0:
        return float("inf")
    return signal * -98

def yip(mup, pet):
    if _______:
         (b)
        mup += 1
    if _______:
         (c)
        return lambda al: ______
                             (d)
    return lambda al: lambda fal: al - fal

yuiop = yiip(_____, _____)(3)(5)
             (e)    (f)
```

(1.0 pt) Which of these could fill in blank (a)?

(c) i.
- ○ `lambda al: signal - al * 3`
- ○ `lambda al: bookbook(signal, 3)(al)`
- ○ `bookbook`
- ○ `yiip(bookbook)`
- ○ `yip`
- ● `bookbook(-3, signal)`
- ○ `bookbook(signal + - 3)`

ii. (2.0 pt) Which of these could fill in blank (b)? *Select all that apply.*

- ■ `mup < 0`
- ■ `pet <= 0`
- ☐ `mup == pet`
- ■ `mup <= 0`
- ☐ `pet == 0`
- ■ `pet < 0`

iii. (2.0 pt) Which of these could fill in blank (c)? *Select all that apply.*

- ☐ `mup > 0`
- ☐ `mup == -pet`
- ☐ `pet == -mup`
- ■ `mup == pet`
- ☐ `pet > 0`
- ■ `mup <= 0 and pet <= 0`

iv. (1.0 pt) Which of these could fill in blank (d)?

- ◯ `al - fal**fal`
- ● `lambda fal: mup**al + pet**fal`
- ◯ `mup * al + pet * fal`
- ◯ `lambda fal: mup + pet * fal`
- ◯ `mup * pet`
- ◯ `lambda fal: mup * al * pet * fal`
- ◯ `mup**al + pet**al`
- ◯ `lambda fal: al * mup + fal * pet`

v. (1.0 pt) Which of these could fill in blank (e)?

- ◯ `yiip(2 * -1)`
- ◯ `yiip`
- ◯ `bookbook(yip)`
- ● `-2`
- ◯ `yip`
- ◯ `signal - 2`

vi. (1.0 pt) Which of these could fill in blank (f)?

- ● `yip`
- ◯ `yip()`
- ◯ `lambda y: y`
- ◯ `yiip`
- ◯ `yiip()`
- ◯ `lambda y: yiip(y)`
- ◯ `-2`

2. (1.0 points) The Case of the Missing Docstring

Consider the following function and its doctests:

```
def mystery(l):
  """
  >>> mystery([1, 2, 3, 4])
  2.5
  >>> mystery([2, 4])
  3.0
  >>> mystery([-5, -2, -9])
  0
  >>> mystery([])
  0
  >>> mystery([345])
  345.0
  """
  s = 0
  t = 0
  for item in l:
    if item > 0:
      s += item
      t += 1
  if t == 0:
    return 0
  return s/t
```

(a) Which of the following docstrings would best describe that function?

- ◯ Returns the average of all elements in L or returns zero if no elements exist.
- ◯ Returns the average of elements in L with an odd index or returns zero if no such elements exist.
- ◯ Returns the average of elements in L that are >= 0 or returns zero if no such elements exist.
- ● Returns the average of positive elements in L or returns zero if no such elements exist.

3. (2.0 points) Magical Test Weaver

Consider the following function signature and docstring:

```
def magic_weave(a, b, c):
    """
    Assuming A and B are positive integers with the same number of base-10 digits
    and C is a positive integer < 10, return the number whose base-10
    representation is the interleaving of digits in A and B (alternating
    first one from A then one from B) from all positions where the
    two digits in A and B at that position are both >= C.  Return 0 if there
    are no such positions. Raises an exception if preconditions are not met.
    """
```

Here is one example of a passing doctest:

```
>>> magic_weave(345, 987, 3)
394857
```

(a) Based on the docstring of that function, which of these would be passing doctests? *Select all that apply.*

- ☐ `>>> magic_weave(123, 456, 5)` `56`
- ☐ `>>> magic_weave(234, 456, 5)` `3546`
- ■ `>>> magic_weave(456, 567, 5)` `5667`
- ☐ `>>> magic_weave(0, 0, 5)` `0`
- ☐ `>>> magic_weave(101, 202, 0)` `120012`
- ☐ `>>> magic_weave(567, 899, 10)` `586979`

4. (10.0 points) Domain On the Range

The domain of a function is the set of all possible argument values, while the range of a function is the set of values that it can return. In this two part-question, you will implement higher-order functions to restrict the domain and range of other functions.

(a) (4.0 points) restrict_domain

Implement `restrict_domain`, a function that accepts three parameters (`f`, `low_d`, `high_d`) and returns a higher-order function that returns the same thing as `f` when given an argument between `low_d` and `high_d`, inclusive, and otherwise returns `float("-inf")`.

```
def restrict_domain(f, low_d, high_d):
    """Returns a function that restricts the domain of F,
    a function that takes a single argument x.
    If x is not between LOW_D and HIGH_D (inclusive),
    it returns -Infinity, but otherwise returns F(x).

    >>> from math import sqrt
    >>> f = restrict_domain(sqrt, 1, 100)
    >>> f(25)
    5.0
    >>> f(-25)
    -inf
    >>> f(125)
    -inf
    >>> f(1)
    1.0
    >>> f(100)
    10.0
    """
    ---------
    #   (a)
        ---------
        #   (b)
            ---------
            #  (c)
        ---------
        #  (d)
    return wrapper_method_name
```

i. (1.0 pt) Fill in blank (a).

def wrapper_method_name(n):

ii. (1.0 pt) Fill in blank (b).

if n < low_d or n > high_d:

iii. (1.0 pt) Fill in blank (c).

```
return float("-inf")
```

iv. (1.0 pt) Fill in blank (d).

```
return f(n)
```

(b) (5.0 points) restrict_range

Implement `restrict_range`, a function that accepts three parameters (`f`, `low_r`, `high_r`) and returns a higher-order function that returns the same thing as `f` when that result is between `low_r` and `high_r` (inclusive), and otherwise returns `float("-inf")`.

```
def restrict_range(f, low_r, high_r):
    """Returns a function that restricts the range of F, a function
    that takes a single argument X. If the return value of F(X)
    is not between LOW_R and HIGH_R (inclusive), it returns -Infinity,
    but otherwise returns F(X).

    >>> cube = lambda x: x * x * x
    >>> f = restrict_range(cube, 1, 1000)
    >>> f(1)
    1
    >>> f(-5)
    -inf
    >>> f(5)
    125
    >>> f(10)
    1000
    >>> f(11)
    -inf
    """
     ---------
     #   (a)

         ---------
         #   (b)

         ---------
         #   (c)

             ---------
             #  (d)

         ---------
         #  (e)
    return wrapper_method_name
```

i. (1.0 pt) Fill in blank (a).

def wrapper_method_name(n):

ii. (1.0 pt) Fill in blank (b).

result = f(n)

iii. (1.0 pt) Fill in blank (c).

if result < low_r or result > high_r:

iv. (1.0 pt) Fill in blank (d).

```
return float("-inf")
```

v. (1.0 pt) Fill in blank (e).

```
return result
```

(c) (1.0 points) restrict_both

Now that you have those two functions defined, you'll implement `restrict_both`, a higher-order function that accepts a function `f` and four numeric arguments (`low_d`, `high_d`, `low_r`, `high_r`) and returns the result of applying both `restrict_domain` and `restrict_range` on `f`.

```
def restrict_both(f, low_d, high_d, low_r, high_r):
    """
    Returns a version of F with a domain restricted to (LOW_D, HIGH_D)
    and a range restricted to (LOW_R, HIGH_R).

    >>> diva = lambda x: (10000 // x) * 9
    >>> f = enforce_both(diva, 1, 1000, 100, 999)
    >>> f(0)
    -inf
    >>> f(10000)
    -inf
    >>> f(200)
    450
    >>> f(100)
    900
    >>> f(1000)
    -inf
    """
    -------------
```

i. Fill in the blank. You may use one line or multiple lines, as long as the solution is correct. Your solution should use `restrict_domain` and `restrict_range` somehow, and assume that those were implemented correctly.

If you wrote your code in code.cs61a.org, you can paste it in here (do not worry if it seems too indented, as long as the indentation worked there).

```
return restrict_range(restrict_domain(f, low_d, high_d), low_r, high_r)
```

5. (8.0 points) Digit replacer

The function `digit_replacer(predicate, transformer)` should return a function that replaces all the digits in a number where `predicate(digit)` is true with the result of `transformer(digit)`. The returned function should accept a single argument, the number `n`, and return the number with the digits replaced.

Here is the function signature and doctests:

```
def digit_replacer(predicate, transformer):
    """Returns a function that accepts a single number N (where N > 0) and
    returns a number where all digits that return true for PREDICATE(DIGIT)
    have been replaced by TRANSFORMER(DIGIT). TRANSFORMER is assumed to always
    return a valid digit >= 0 and <= 9.

    >>> is_even = lambda d: d % 2 == 0
    >>> lt_five = lambda d: d < 5
    >>> always_two = lambda d: 2
    >>> floor_divide_two = lambda d: d // 2
    >>> digit_replacer(is_even, floor_divide_two)(21098)
    11094
    >>> digit_replacer(lt_five, always_two)(1064592)
    2262592
    """
```

(a) (4.0 pt) Use iteration (**without recursion**) to implement `digit_replacer`.

Your solution should only use numbers, arithmetic expressions, and booleans. It should *not* use strings, lists, or other data types, and will earn 0 points if it does. It will also earn 0 points if it uses recursion, since that's tested in the second part of this question.

Here's an approximate structure of a solution, if that helps guide your implementation. It does not necessarily reflect the exact number of lines or indentation.

```
def digit_replacer(predicate, transformer):
    _____
        _____
        _____

        while ______:
            _____
                _____
            _____
                _____
            _____
            _____
       return _____
    ______
```

Remember that you can use code.cs61a.org to try out your code and see if it passes the doctests. You can then paste the code here. Please include the function signature.

```
def digit_replacer(predicate, transformer):
    def func(n):
        new_number = 0
        power_of_ten = 0
        while n > 0:
            digit = n % 10
            if predicate(digit):
                digit = transformer(digit)
            new_number += digit * 10**power_of_ten
            power_of_ten += 1
            n = n // 10
        return new_number
    return func
```

(b) (4.0 pt) Use a recursive approach to implement `digit_replacer`.

Your solution should only use numbers, arithmetic expressions, and booleans. It should *not* use strings, lists, or other data types, and will earn 0 points if it does. It will also earn 0 points if it does not use recursion, since that was tested in the first part of this question.

Here's an approximate structure of a solution, if that helps guide your implementation. It does not reflect the exact number of lines or indentation.

```
def digit_replacer(predicate, transformer):
    ______
        ______
            ______
        ______
            ______
        ______
       return ______
    ______
```

Remember that you can use code.cs61a.org to try out your code and see if it passes the doctests. You can then paste the code here. Please include the function signature.

```
def digit_replacer(predicate, transformer):
    def func(n):
        if n == 0:
            return 0
        digit = n % 10
        if predicate(digit):
            digit = transformer(digit)
        return func(n // 10) * 10 + digit
    return func
```

6. (6.0 points) Run checker

Let's use the term "chain function" to mean a function that takes a single numerical argument and returns another chain function (so that if `h` is a chain function, one can call `h(3)(4)(1)(2)(0)...`, which we'll call a "chain of calls.")

The function `run_checker` accepts two functions as arguments (`condition` and `result`) and returns a chain function. Each call in a chain starting with the function returned by `run_checker` first applies `condition` to the two previous arguments in the chain and its own argument. If that returns a false value, it prints "No run!" and otherwise prints the result of `result` applied to the same three arguments. For the first and second calls in the chain, the missing arguments are taken to be -1.

```
def run_checker(condition, result):
    """
    Returns a chain function. Each call in a chain that starts with
    this returned function prints "No run!" if CONDITION returns a false
    value when applied to the previous two arguments and the current argument,
    and otherwise prints the result of applying RESULT to these same
    three arguments.  For calls in the chain where there are fewer than two
    preceding calls in the chain, the missing arguments are taken to be -1.

    >>> f = run_checker(lambda a, b, c: a > b > c and a >= 10, lambda a, b, c: a*(b+c))
    >>> f = f(15)
    No run!
    >>> f = f(10)
    No run!
    >>> f = f(5)
    225
    >>> f = f(2)
    70
    >>> f = f(1)
    No run!
    >>> f = f(11)
    No run!
    >>> f = f(12)
    No run!
    >>> f = f(10)
    No run!
    >>> f = f(2)
    144
    """
    def f(______):
       #   (a)
        def g(______):
           #   (b)

            ------
            ------
            ------
            ------
            ------
            # (c)

        return g
    return f(______)
    #          (d)
```

(a) (1.0 pt) Fill in blank (a).

two_ago, one_ago

(b) (1.0 pt) Fill in blank (b).

input

(c) (3.0 pt) Fill in the (c) blanks. You can use more or fewer lines for your solution than the number suggested by the blanks.

If you use code.cs61a.org to try out your code, you can then paste the code here. Don't worry if it seems too indented, as long as the indentation worked there.

```
if condition(two_ago, one_ago, input):
   print(result(two_ago, one_ago, input))
else:
   print("No run!")
return f(one_ago, input)
```

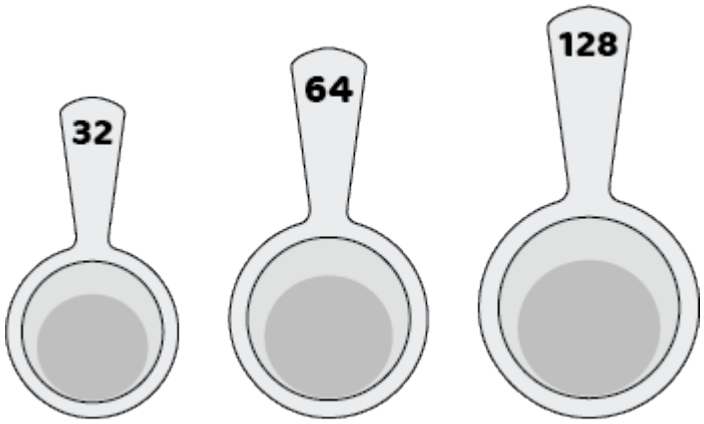
(d) (1.0 pt) Fill in blank (d).

-1, -1

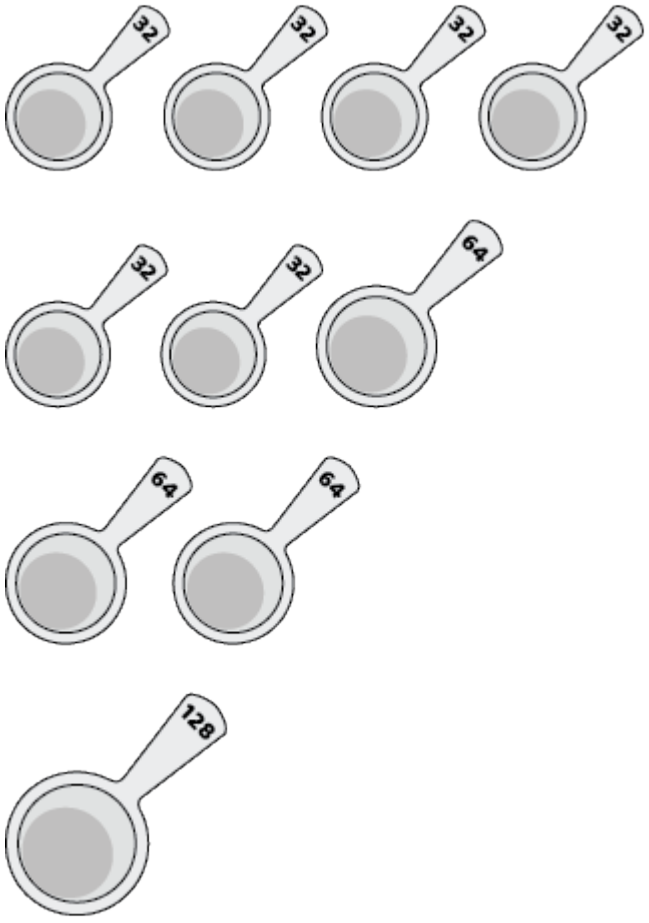
7. (5.0 points) Measure Twice, Cup Once

The function `measure_methods` accepts two arguments (`total_needed`, the total number of grams needed for a recipe, and `cup_sizes`, a list of measuring cup sizes available) and returns the number of possible ways to make exactly `total_needed` using the cup sizes. The `cup_sizes` list is measured in grams, sorted from smallest to largest, and each size is a power of 2. A cup size may be used multiple times in order to come up with `total_needed`.

For example, the list of `[32, 64, 128]` represents these cups:



There are 4 possible ways to make a total of 128 grams from those cups:



Here's the partially defined function:

```
def measure_methods(total_needed, cup_sizes):
```

```
    """Returns the number of ways to make exactly TOTAL_NEEDED with
    the given list of CUP_SIZES (sorted by smallest to largest).

    >>> measure_methods(128, [32, 64, 128])
    4
    >>> measure_methods(256, [32, 64, 128])
    9
    >>> measure_methods(384, [32, 64, 128])
    16
    >>> measure_methods(256, [16, 32, 64])
    25
    >>> measure_methods(125, [32, 64, 128])
    0
    """
    def helper_method_name(______):
        #        (a)
        ------
        ------
        ------
        ------
        ------
        # (b)
    return helper_method_name(______)
    #               (c)
```

(a) (1.0 pt) Fill in blank (a).

total_needed, curr_i

(b) (3.0 pt) Fill in the (b) blanks. You can use more or fewer lines for your solution than the number suggested by the blanks, and your code may involve multiple indentation levels.

If you use code.cs61a.org to try out your code, you can then paste the code here. Don't worry if it seems too indented, as long as the indentation worked there.

```
if total_needed == 0:
   return 1
if total_needed < 0:
   return 0
if len(cup_sizes) <= curr_i:
   return 0
without_cup = helper(total_needed, curr_i + 1)
with_cup = helper(total_needed - cup_sizes[curr_i], curr_i)
return without_cup + with_cup
```

(c) (1.0 pt) Fill in blank (c).

total_needed, 0

8. (1.0 points) Extra Point!

(a) (1.0 pt) "Baked Century" (nine letters, two words).

High Ranks

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Spring 2021

Midterm 2 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) By writing my name below, I pledge on my honor that I will abide by the rules of this exam and will neither give nor receive assistance. I understand that doing otherwise would be a disservice to my classmates, dishonor me, and could result in me failing the class.

1. (2.0 points) Tuple Trouble

Consider this partially implemented two-line program:

```
greeting_from_staff = ("Good Luck", "You Got This", "Just Breathe")
______ = "Once More with Feeling"
```

The full program results in this error:

```
AttributeError: 'tuple' object has no attribute '__setitem__'
```

(a) What code could fill in the blank to cause that error?

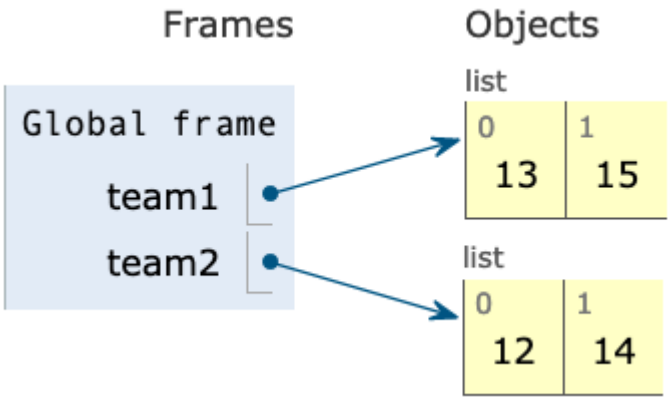
```
greeting_from_staff[2]
```

2. (4.0 points) Teenage Mutant Ninja Lists

A program starts with two list assignments:

```
team1 = [13, 15]
team2 = [12, 14]
```

That code corresponds to this environment diagram:

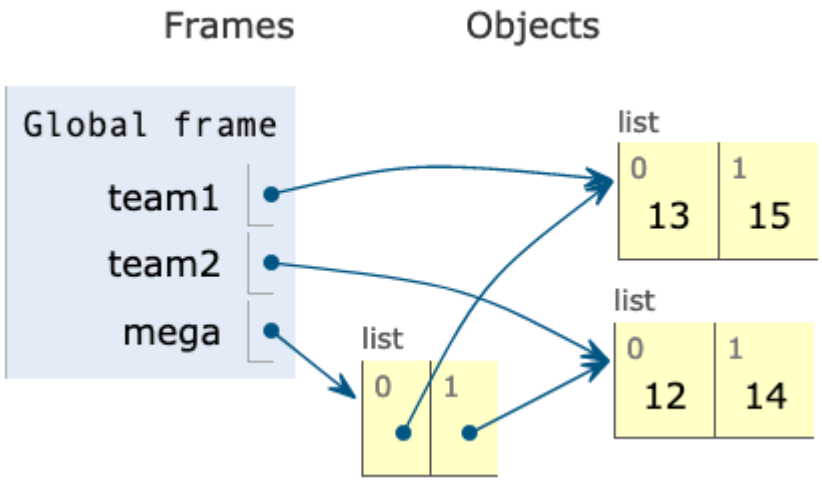


In this question, you'll suggest ways of mutating the lists to result in different environment diagrams.

(a) (1.0 pt)

```
team1 = [13, 15]
team2 = [12, 14]
mega = _____________
```

This modified program corresponds to this diagram:



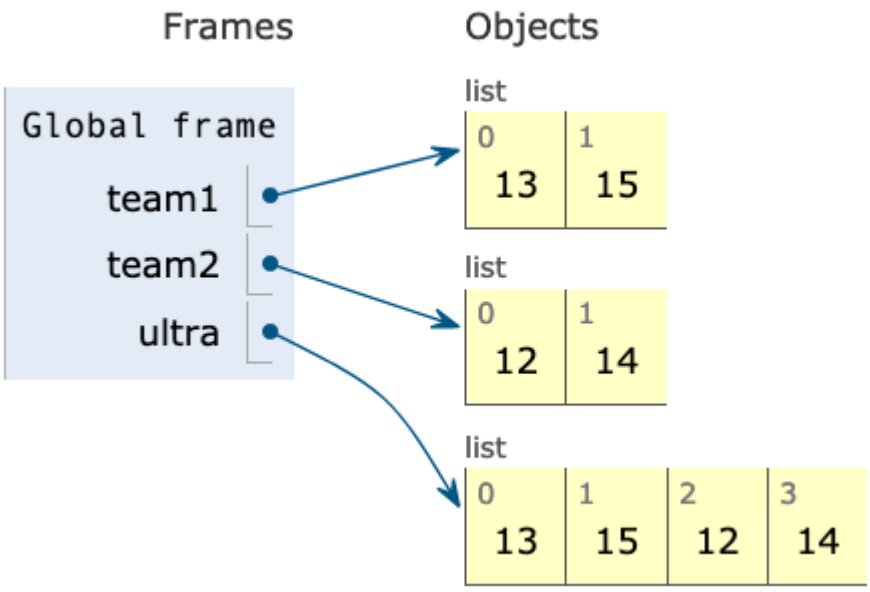
Which of these could fill in the blank? **Select all that apply**

- [x] `[team1, team2]`
- [x] `[team1] + [team2]`
- [ ] `[team1[0:], team2[0:]]`
- [ ] `[team1 + team2]`
- [ ] `[list(team1 + team2)]`
- [ ] `team1, team2`
- [ ] `[list(team1), list(team2)]`
- [ ] `team1 + team2`
- [ ] `team1 - team2`
- [ ] `team1 + team2[:]`
- [ ] `team1[:] + team2`
- [ ] `team1[:] + team2[:]`
- [ ] `team1 += team2`

(b) (1.0 pt)

```
team1 = [13, 15]
team2 = [12, 14]
ultra = _____________
```

This modified program corresponds to this diagram:



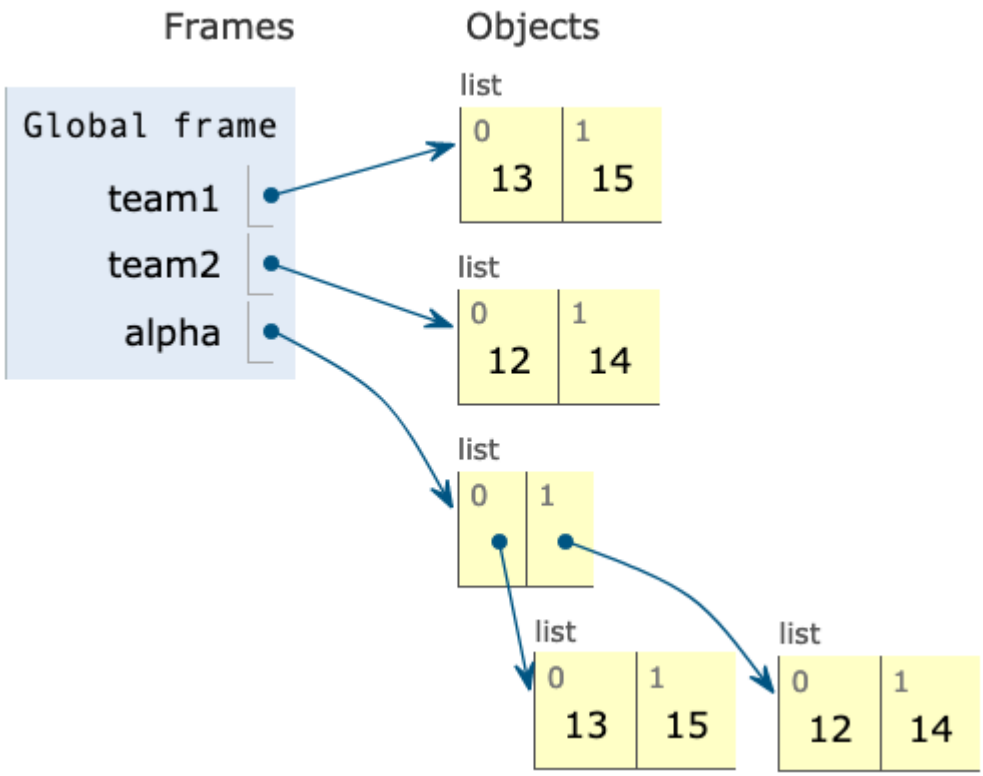
Which of these could fill in the blank? **Select all that apply**

- [x] `team1 + team2`
- [x] `team1[:] + team2[:]`
- [x] `team1 + team2[:]`
- [x] `team1[:] + team2`
- [ ] `[team1, team2]`
- [ ] `[team1[0:], team2[0:]]`
- [ ] `[team1 + team2]`
- [ ] `[list(team1 + team2)]`
- [ ] `team1, team2`
- [ ] `[list(team1), list(team2)]`
- [ ] `team1 - team2`
- [ ] `[team1] + [team2]`
- [ ] `team1 += team2`

(c) (1.0 pt)

```
team1 = [13, 15]
team2 = [12, 14]
alpha = _____________
```

This modified program corresponds to this diagram:

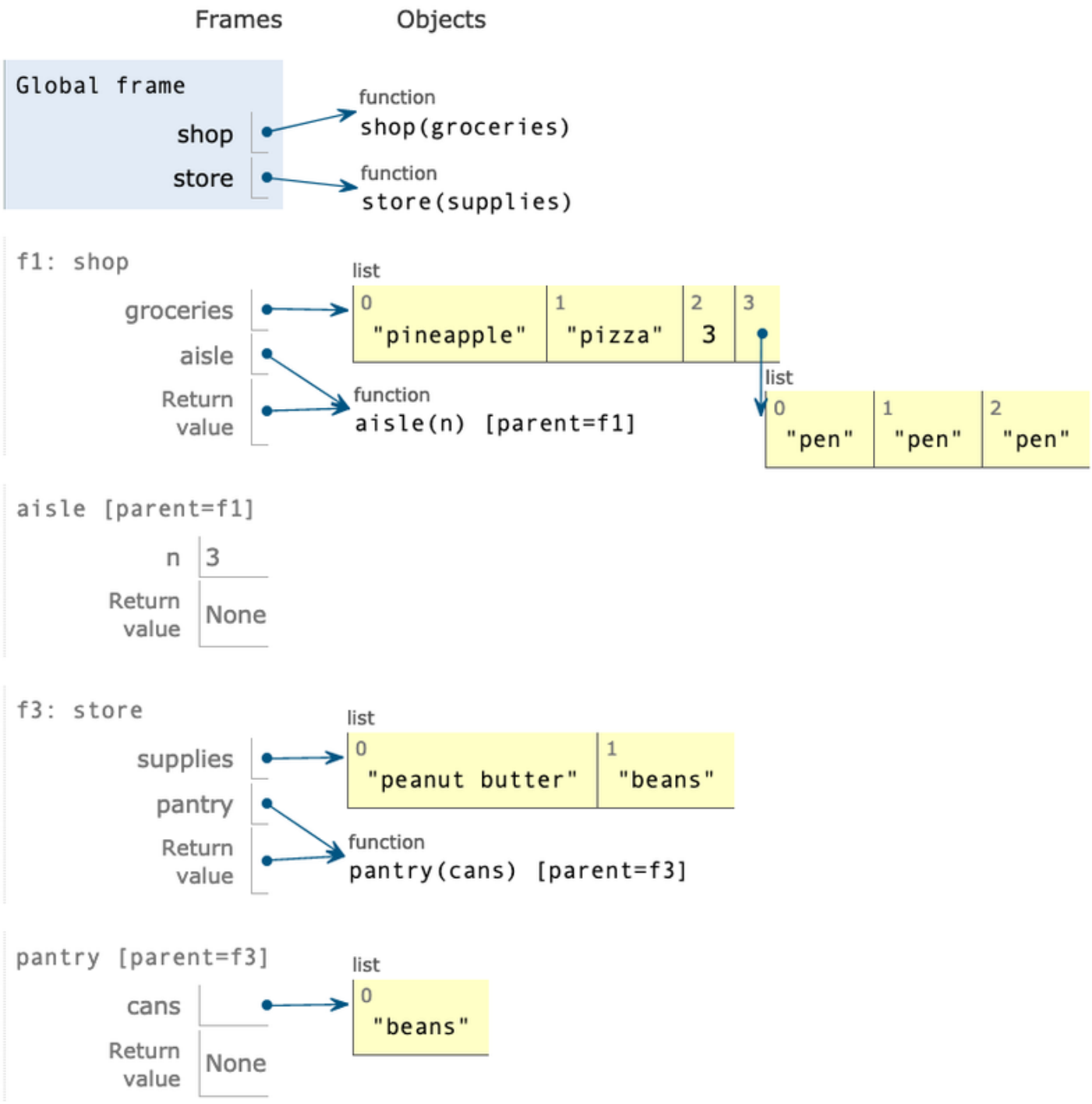


Which of these could fill in the blank? **Select all that apply**

- [x] `[list(team1), list(team2)]`
- [x] `[team1[0:], team2[0:]]`
- [ ] `[team1, team2]`
- [ ] `[team1 + team2]`
- [ ] `[list(team1 + team2)]`
- [ ] `team1, team2`
- [ ] `team1 + team2`
- [ ] `team1 - team2`
- [ ] `[team1] + [team2]`
- [ ] `team1 + team2[:]`
- [ ] `team1[:] + team2`
- [ ] `team1[:] + team2[:]`
- [ ] `team1 += team2`

3. (4.0 points) Higher Order Shopping

The following environment diagram was generated by a program:



Click here to open the diagram in a new window

In this series of questions, you'll fill in the blanks of the program that follows so that its execution matches the environment diagram.

```
def shop(groceries):

    def aisle(n):
```

```
        --------
        #  (a)
        --------
        #  (b)

    return aisle

def store(supplies):

    def pantry(cans):
        --------
        #  (c)
        --------
        #  (d)

    return pantry

shop(["pineapple", "pizza"])(3)
store(["peanut butter"])(["beans"])
```

(a) (2.0 pt) Which of these could fill in blank (a)? **Select all that apply!**

- [x] `groceries.extend([n])`
- [x] `groceries.append(n)`
- [ ] `nonlocal groceries`
- [ ] `nonlocal n`
- [ ] `nonlocal groceries += n`
- [ ] `nonlocal groceries.extend([n])`
- [ ] `nonlocal groceries.append(n)`
- [ ] `nonlocal groceries.extend(n)`
- [ ] `groceries.extend(n)`
- [ ] `groceries.append([n])`
- [ ] `groceries += n`
- [ ] `nonlocal n, groceries`
- [ ] `nonlocal groceries + n`

(b) (1.0 pt) Which of these could fill in blank (b)?

- [x] `groceries.append(["pen"] * n)`
- [ ] `groceries.append("pen" * n)`
- [ ] `groceries.extend("pen" * n)`
- [ ] `groceries.extend(["pen" * n])`
- [ ] `groceries += "pen" * n`
- [ ] `groceries += ["pen" for x in n]`
- [ ] `groceries.extend(["pen" for x in n])`
- [ ] `groceries.append(["pen" for x in n])`
- [ ] `groceries[3:n] = ["pen"]`
- [ ] `groceries[:3:n] = ["pen"]`
- [ ] `groceries[0:n] = ["pen"]`

(c) (1.0 pt) Which of these could fill in blank (c)?

- [x] `nonlocal supplies`
- [ ] `nonlocal store`
- [ ] `nonlocal n`
- [ ] `supplies = list(supplies)`
- [ ] `supplies = supplies[:]`
- [ ] `nonlocal supplies = list(cans)`
- [ ] `nonlocal supplies = list()`
- [ ] `nonlocal list supplies`
- [ ] `nonlocal(supplies)`

(d) (1.0 pt) Which of these could fill in blank (d)?

- [x] `supplies += cans`
- [ ] `nonlocal supplies += cans`
- [ ] `supplies.append(cans)`
- [ ] `nonlocal supplies.append(cans)`
- [ ] `supplies[:] = cans`
- [ ] `nonlocal supplies[:] = cans`
- [ ] `supplies = cans`
- [ ] `nonlocal supplies = cans`
- [ ] `supplies.extend([cans])`
- [ ] `nonlocal supplies.extend([cans])`

4. (8.0 points) Gotta Keep 'Em Separated

Implement `separate()`, a function that accepts two parameters (`separator` and `lnk`) and returns a new linked list where any two consecutive repeated numbers in `lnk` are separated by a link with the value of `separator`. All values in `lnk` will be numbers.

The function should assume the standard CS61A Link definition, viewable here: code.cs61a.org/link_class

```
def separate(separator, lnk):
    """Returns a new linked list (using Link) that separates
    any two consecutive repeated numbers in non-empty LNK
    by inserting a Link with the value of SEPARATOR.
    All values in LNK will be numbers.

    # Case 1: Separates list that starts with pair
    >>> separate(999, Link(1, Link(1)))
    Link(1, Link(999, Link(1)))

    # Case 2: Separates list with overlapping pairs
    >>> separate(999, Link(1, Link(1, Link(1))))
    Link(1, Link(999, Link(1, Link(999, Link(1)))))

    # Case 3: Doesn't mutate the input list
    >>> link = Link(1, Link(1))
    >>> separate(999, link)
    Link(1, Link(999, Link(1)))
    >>> link
    Link(1, Link(1))

    # Case 4: Only separates pairs, not non-pairs
    >>> separate(-999, Link(2, Link(2, Link(4, Link(5)))))
    Link(2, Link(-999, Link(2, Link(4, Link(5)))))

    # Case 5: Separates pairs at end of list
    >>> separate(-1, Link(2, Link(3, Link(4, Link(4)))))
    Link(2, Link(3, Link(4, Link(-1, Link(4)))))

    # Case 6: Returns same-valued list if no pairs found
    >>> separate(999, Link(1, Link(2)))
    Link(1, Link(2))

    # Case 7: Handles single element lists correctly
    >>> lnk = Link(1)
    >>> lnk2 = separate(999, lnk)
    >>> lnk
    Link(1)
    >>> lnk is not lnk2
    True
    """
```

(a) Here's an approximate structure of a recursive solution, if that helps guide your implementation. It does not reflect the exact number of lines or indentation. **Your solution must use a recursive approach, not an iterative approach. An iterative approach will receive 0 points.**

```
if ______:
  _____________
elif ____:
  _____________
elif ____:
  _____________
_______________
```

You can use code.cs61a.org to try out your code and see if it passes the doctests. You can then paste the code here.

```
def separate(separator, lnk):
   if lnk is Link.empty:
      return Link.empty
   elif lnk.rest is Link.empty:
      return Link(lnk.first)
   elif lnk.first == lnk.rest.first:
      return Link(lnk.first, Link(separator, separate(separator,
lnk.rest)))
   return Link(lnk.first, separate(separator, lnk.rest))
```

5. (8.0 points) Online Classes

This question is inspired by online education websites that offer a range of materials such as videos, articles, exercises, and quizzes.

We will represent two kinds of educational content as a Python class: `Video` and `Exercise`. Each of those classes inherits from the base class `LearnableContent`.

The partially implemented classes are shown below. You'll fill out the class definitions using inheritance and attributes. **Always use the definition from the base class if it's reasonable to do so.**

```
class LearnableContent:
    """A base class for specific kinds of learnable content.
    All kinds have title and author attributes,
    but each kind may have additional attributes.
    """

    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __str__(self):
        return f"{self.title} by {self.author}"

class Exercise(LearnableContent):
    """
    >>> lambda_calc = Exercise("Lambda Calculus", "Rosie F", 5)
    >>> lambda_calc.title
    'Lambda Calculus'
    >>> lambda_calc.author
    'Rosie F'
    >>> lambda_calc.num_questions
    5
    >>> str(lambda_calc)
    'Lambda Calculus by Rosie F'
    """

    def __init__(self, title, author, num_questions):
        ________________________
        #          (a)
        ________________________
        #          (b)

class Video(LearnableContent):
    """
    >>> vid = Video("The Golden Ratio", "Sal Khan", 881)
    >>> Video.license
    'CC-BY-NC-SA'
    >>> vid.title
    'The Golden Ratio'
    >>> vid.author
    'Sal Khan'
    >>> vid.num_seconds
    881
    >>> str(vid)
    'The Golden Ratio by Sal Khan (881 seconds)'
    """
```

```
____________________
#      (c)

def __init__(self, title, author, num_seconds):
    ________________________
    #          (d)
    ________________________
    #          (e)

def __str__(self):
    ________________________
    #          (f)
```

Please check your work on code.cs61a.org and run the doctests there.

(a) (1.0 pt) What line of code could go in blank (a)? **You only need a single line of code.**

```
super().__init__(title, author)
```

(b) (1.0 pt) What line of code could go in blank (b)? **You only need a single line of code.**

```
self.num_questions = num_questions
```

(c) (1.0 pt) What line of code could go in blank (c)? **You only need a single line of code.**

```
license = 'CC-BY-NC-SA'
```

(d) (2.0 pt) What line of code could go in blank (d)? **You only need a single line of code.**

```
super().__init__(title, author)
```

(e) (1.0 pt) What line of code could go in blank (e)? **You only need a single line of code.**

```
self.num_seconds = num_seconds
```

(f) (2.0 pt) What line of code could go in blank (f)? **You only need a single line of code.**

You may use any form of string formatting that you are comfortable with, but your solution must use `super()`.

```
return super().__str__() + f" ({self.num_seconds} seconds)"
```

6. (6.0 points) Gas Composition

This question is inspired by research projects that are tracking the emissions of greenhouse gases into the environment so that they can identify the biggest sources of greenhouse gases and find ways to sustainably reduce them.

We have written two classes to help track gas emissions: `EmissionSource` and `EmissionsTracker`.

The `EmissionSource` class is fully implemented. Each instance stores a name, a dictionary of emission rates per hour per gas type, and the number of hours it's been running (initialized to 0). Read through the implementation below:

```
class EmissionSource:

    def __init__(self, name, co2, ch4, n2o):
        self.name = name
        self.emissions = {
            "carbon dioxide": co2,  # kg CO2/ton/hour
            "methane": ch4,         # g CH4/ton/hour
            "nitrous oxide": n2o,   # g N2O/ton/hour
        }
        self.hours = 0

    def run_for(self, num_hours):
        self.hours += num_hours

    def calc_emissions(self, gas):
        return self.emissions.get(gas, 0) * self.hours
```

The `EmissionsTracker` class is partially implemented, but has doctests that demonstrate how it should work. It's job is to track emission_source_list and calculate statistics about their emissions. Read through the doctests, docstrings, and partial implementation below:

```
class EmissionsTracker:
    """
    >>> tracker = EmissionsTracker()
    >>> pp1 = EmissionSource("Anthracite Coal", 2602, 276, 40)
    >>> pp2 = EmissionSource("Lignite Coal", 1389, 156, 23)
    >>> tracker.add_sources([pp1, pp2])
    >>> pp1.run_for(5)
    >>> pp2.run_for(6)

    # Case 1: Calculate max for 2 added so far
    >>> tracker.calc_emissions("methane")
    [('Anthracite Coal', 1380), ('Lignite Coal', 936)]
    >>> tracker.calc_max_source("methane")
    ('Anthracite Coal', 1380)

    # Case 3: Calculate emissions for all 3
    >>> pp3 = EmissionSource("Plastics", 2850, 1216, 160)
    >>> pp3.run_for(3)
    >>> tracker.add_sources([pp3])
    >>> tracker.calc_emissions("carbon dioxide")
    [('Anthracite Coal', 13010), ('Lignite Coal', 8334), ('Plastics', 8550)]
    >>> tracker.calc_emissions("methane")
    [('Anthracite Coal', 1380), ('Lignite Coal', 936), ('Plastics', 3648)]
    >>> tracker.calc_emissions("nitrous oxide")
    [('Anthracite Coal', 200), ('Lignite Coal', 138), ('Plastics', 480)]
```

```
# Case 4: Calculate max for all 3
>>> tracker.calc_max_source("carbon dioxide")
('Anthracite Coal', 13010)
>>> tracker.calc_max_source("methane")
('Plastics', 3648)
>>> tracker.calc_max_source("nitrous oxide")
('Plastics', 480)
"""
def __init__(self):
    self.emission_source_list = []

def add_sources(self, sources_to_add):
    """ Adds the list of SOURCES_TO_ADD to the end of self.emission_source_list"""
    ___________________________
    #         (a)

def calc_emissions(self, gas):
    """ Returns a list of tuples where the first element is the name of
    the emission_src and the second element is the total amount of emissions
    from that type of GAS."""
    ___________________________
    #         (b)

def calc_max_source(self, gas):
    """ Returns a tuple where the first element is the name of the emission_src
    with the highest emissions and the second element is the total amount
    of emissions from that emission_src for that type of GAS."""
    return max(_______________)
    #               (c)
```

The following questions will ask you to fill in the blanks for `EmissionsTracker`. You can try out the code on code.cs61a.org and run the doctests to verify that your proposed code does indeed work.

(a) **(2.0 pt)** What line of code could go in blank (a)? **You only need a single line of code.**

```
self.emission_source_list.extend(sources_to_add)
```

(b) **(2.0 pt)** What line of code could go in blank (b)? **You only need a single line of code. Your code must use methods from the `EmissionSource` class if it's reasonable to do so.**

```
return [ (source.name, source.calc_emissions(gas)) for source in
self.emission_source_list]
```

(c) **(2.0 pt)** What expression could go in blank (c)? You may want to consult the Python documentation for the max function.

```
self.calc_emissions(gas), key=lambda tup: tup[1]
```

7. (4.0 points) Business as Usual

Modern office supplies companies use software to track their sales inventory and can use object oriented programming to represent the types of products they sell.

We will represent one type of product as a Python class: `PaperReam`.

The class is partially implemented below. Please read through the code and doctests.

```
class PaperReam:
    """
    >>> ream = PaperReam("red", 200)
    >>> ream.color_name
    'red'
    >>> ream.num_sheets
    200
    >>> ream
    PaperReam('red', 200)
    """

    def __init__(self, color_name, num_sheets):
        self.color_name = color_name
        self.num_sheets = num_sheets

    --------------------
    #          (a)
        ----------------
        #      (b)
```

The following questions will ask you to fill in the blanks for `PaperReam`. You can try out the code on code.cs61a.org and run the doctests to verify that your proposed code does indeed work.

(a) (2.0 pt) What line of code could go in blank (a)? **You only need a single line of code, since it should be a function header.**

```
def __repr__(self):
```

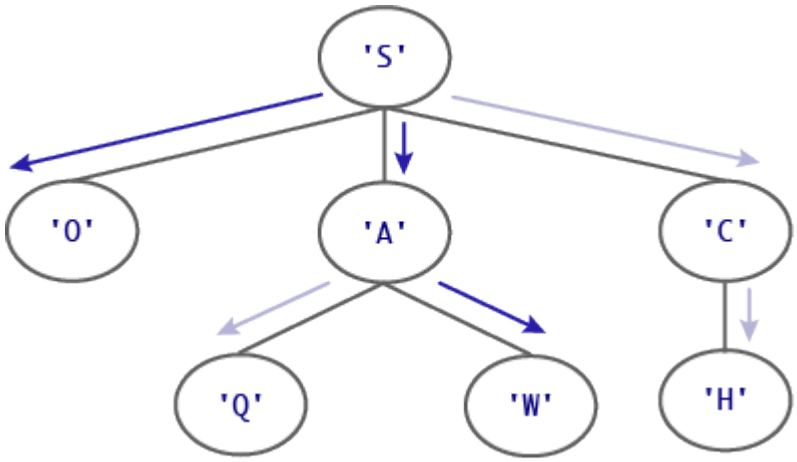
(b) (2.0 pt) What line of code could go in blank (b)? **You only need a single line of code.**

```
return f"PaperReam({repr(self.color_name)}, {repr(self.num_sheets)})"
OR
return f"PaperReam('{self.color_name}', {self.num_sheets})"
```

8. (7.0 points) The Tree of L-I-F-E

Implement `word_finder`, a generator function that yields each word that can be formed by following a path in a tree from the root to a leaf, where the words are specified in a list.

When given the tree shown in the diagram below and a word list that includes 'SO' and 'SAW', the function should first yield 'SO' and then yield 'SAW'.



Please read through the function header and doctests below. We have provided quite a few doctests to test different situations and demonstrate how the function should work. You can always call `draw(t)` on a particular tree object on code.cs61a.org to help you visualize its structure and understand the results of a doctest.

```
def word_finder(letter_tree, words_list):
    """ Generates each word that can be formed by following a path
    in TREE_OF_LETTERS from the root to a leaf,
    where WORDS_LIST is a list of allowed words (with no duplicates).

    # Case 1: 2 words found
    >>> words = ['SO', 'SAT', 'SAME', 'SAW', 'SOW']
    >>> t = Tree("S", [Tree("O"), Tree("A", [Tree("Q"), Tree("W")]), Tree("C", [Tree("H")])])
    >>> gen = word_finder(t, words)
    >>> next(gen)
    'SO'
    >>> next(gen)
    'SAW'
    >>> list(word_finder(t, words))
    ['SO', 'SAW']

    # Case 2: No words found
    >>> t = Tree("S", [Tree("I"), Tree("A", [Tree("Q"), Tree("E")]), Tree("C", [Tree("H")])])
    >>> list(word_finder(t, words))
    []

    # Case 3: Same word twice
    >>> t = Tree("S", [Tree("O"), Tree("O")] )
    >>> list(word_finder(t, words))
    ['SO', 'SO']

    # Case 4: Words that start the same
    >>> words = ['TAB', 'TAR', 'BAT', 'BAR', 'RAT']
    >>> t = Tree("T", [Tree("A", [Tree("R"), Tree("B")])])
    >>> list(word_finder(t, words))
```

```
['TAR', 'TAB']

# Case 5: Single letter words
>>> words = ['A', 'AN', 'AH']
>>> t = Tree("A")
>>> list(word_finder(t, words))
['A']

# Case 6: Words end in leaf
>>> words = ['A', 'AN', 'AH']
>>> t = Tree("A", [Tree("H"), Tree("N")])
>>> list(word_finder(t, words))
['AH', 'AN']

# Case 7: Words start at root
>>> words = ['GO', 'BEARS', 'GOB', 'EARS']
>>> t = Tree("B", [Tree("E", [Tree("A", [Tree("R", [Tree("S")])])])])
>>> list(word_finder(t, words))
['BEARS']

# Case 8: This special test ensures that your solution does *not*
#  pre-compute all the words before yielding the first one.
#  If done correctly, your solution should error only when it
#  tries to find the second word in this tree.
>>> words = ['SO', 'SAM', 'SAT', 'SAME', 'SAW', 'SOW']
>>> t = Tree("S", [Tree("O"), Tree("A", [Tree("Q"), Tree(1)]), Tree("C", [Tree(1)])])
>>> gen = word_finder(t, words)
>>> next(gen)
'SO'
>>> try:
...     next(gen)
... except TypeError:
...     print("Got a TypeError!")
... else:
...     print("Expected a TypeError!")
Got a TypeError!
"""
```

(a) Here is a skeleton of a correct solution:

```
def word_finder(letter_tree, words_list):
    def _________(____):
        _______________ # Optional
        if ____________:
            yield _______________
        for ____________:
            yield from _________

    yield from __________
```

We highly encourage you to follow the skeleton. You may diverge from the skeleton if your approach is correct and passes the doctests, but a non-working solution that doesn't follow the skeleton will not receive partial credit.

A correct solution should **not** first find all the words and then yield them; it should instead yield whenever it finds a new word. **Your solution may receive 0 points if it pre-computes the words.**

Your function should use the standard CS61A Tree definition, viewable here: code.cs61a.org/tree_class

Please use code.cs61a.org to try out your code. You can then paste the code here.

```
def word_finder(letter_tree, words_list):
    def string_builder(t, str):
        str += t.label
        if t.is_leaf() and str in words_list:
            yield str
        for b in t.branches:
            yield from string_builder(b, str)
    yield from string_builder(letter_tree, "")
```

9. (4.0 points) Light It Up

An individually addressed LED light strip allows programmers to write programs to change the color of each individual LED light.

We could represent a strip of lights using a `SingleLED` class and an `LEDLightStrip` class:

```
class SingleLED:

    def __init__(self, brightness, color):
        self.brightness = brightness
        self.color = color

class LEDLightStrip:

    def __init__(self, leds):
        self.leds = leds
        self.length = len(leds)
        self.is_on = False

    def toggle(self):
        self.is_on = not self.is_on
```

Then a short 3-light strip could be constructed like so:

```
light_strip = LEDLightStrip([SingleLED(0, "blue"), SingleLED(0, "red"), SingleLED(0, "green")])
```

We would like to be able to iterate through the light object, one light at a time, like so:

```
for led in light_strip:
    led.brightness += 20
```

But with our current code, we see this error:

```
TypeError: 'LEDLightStrip' object is not iterable
```

(a) (2.0 pt) We need to define a new method on `LEDLightStrip` to make it iterable. Here's a skeleton of that method:

```
def _______________(self):
    #     (a)
    ______________
    ______________
    #     (b)
```

What method name should go in blank (a)?

`__iter__`

(b) (2.0 pt) What code could go in blank (b) such that the for loop works? *You can use a single line of code or multiple lines of code, as long as the code is correct.*

```
for led in self.leds:
yield led
OR:
return iter(self.leds)
```

10. (3.0 points) May I Take Your Order?

The following two functions can be used for inserting items into a linked list. The first function `insert_front()` always inserts at the front of the list while the second function `insert_back()` always inserts at the back of the list. Both functions use the standard CS61A Link definition, viewable here: code.cs61a.org/link_class

```
def insert_front(link, new_val):
    """Add NEW_VAL to the front of non-empty LINK, mutating the list.

    >>> link = Link(-1, Link(-3, Link(-5)))
    >>> insert_front(link, -0)
    >>> link
    Link(-0, Link(-1, Link(-3, Link(-5))))
    >>> insert_front(link, -3)
    >>> link
    Link(-3, Link(-0, Link(-1, Link(-3, Link(-5)))))
    """
    original_first = link.first
    link.first = new_val
    link.rest = Link(original_first, link.rest)

def insert_back(link, new_val):
    """Add NEW_VAL to the end of non-empty LINK, mutating the list.

    >>> link = Link(-1, Link(-3, Link(-5)))
    >>> insert_back(link, -0)
    >>> link
    Link(-1, Link(-3, Link(-5, Link(-0))))
    >>> insert_back(link, -3)
    >>> link
    Link(-1, Link(-3, Link(-5, Link(-0, Link(-3)))))
    """
    while link.rest is not Link.empty:
        link = link.rest
    link.rest = Link(new_val)
```

(a) (1.0 pt) What is the order of growth of `insert_front()` as the input list size changes?

- (●) (1)
- ( ) $(\log_2 n)$
- ( ) (n)
- ( ) $(n \log_2 n)$
- ( ) (n^2)
- ( ) (n^3)
- ( ) (2^n)
- ( ) $(n!)$

(b) (1.0 pt) What is the order of growth of `insert_back()` as the input list size changes?

- ○ (1)
- ○ $(\log_2 n)$
- ● (n)
- ○ $(n \log_2 n)$
- ○ (n^2)
- ○ (n^3)
- ○ (2^n)
- ○ $(n!)$

(c) (1.0 pt) Which of the following correctly describes the two functions?

- ● Both functions perform destructive operations on the input parameter, a mutable object.
- ○ Both functions perform non-destructive operations on the input parameter, a mutable object.
- ○ The first function performs a non-destructive operation while the second function performs a destructive operation.
- ○ The first function performs a destructive operation while the second function performs a non-destructive operation.

11. (1.0 points) Extra Point!

(a) (1.0 pt) What phrase might be suggested by "6 = Q. in the S. M."?

6 quarks in the standard model.

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Spring 2022

Midterm 1 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

1. (7.0 points) What Would Python Dip?

Assume the following code has been executed.

```
def dipping(dots):
    if print("you dip"):
        return print("i dip")
    else:
        return print(dots) or dots or print("we dip")
```

What would the Python interpreter display? If the interpreter would include a new line, please enter a new line in your answer.

(a) (2.0 pt) `>>> dipping(0)`

you dip
0
we dip

(b) (2.0 pt) `>>> dipping(555)`

```
you dip
555
555
```

(c) (1.0 pt) What is the return value of `dipping(0)`?

None

(d) (1.0 pt) What is the return value of `dipping(-666)`?

-666

(e) (1.0 pt) Rewrite the body of the `dipping` function in a single line of code.

```
def dipping(dots):
    ---------------------
```

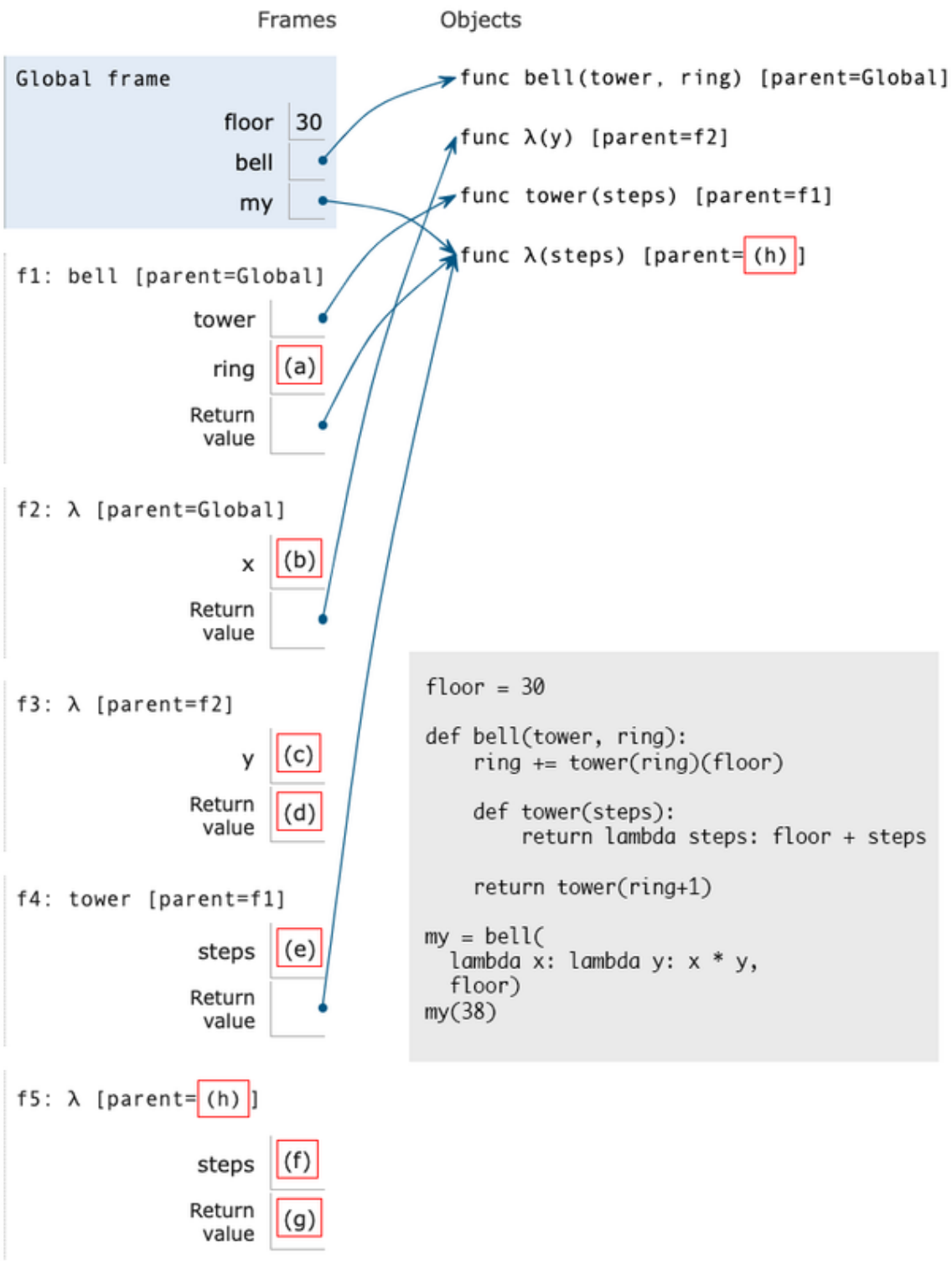
```
return print("i dip") if print("you dip") else (print(dots) or dots or
print("we dip"))
```

2. (8.0 points) Ring My Bell Tower

The following code was used to generate the environment diagram below:

```
floor = 30

def bell(tower, ring):
    ring += tower(ring)(floor)

    def tower(steps):
        return lambda steps: floor + steps

    return tower(ring+1)

my = bell(lambda x: lambda y: x * y, floor)
my(38)
```

The environment diagram below represents the *final* state of the environment. The code is also provided to the right of the diagram, for convenience. Line numbers have been omitted intentionally.



(a) (1.0 pt) Fill in blank (a)

930

(b) **(1.0 pt)** Fill in blank (b)

30

(c) **(1.0 pt)** Fill in blank (c)

30

(d) **(1.0 pt)** Fill in blank (d)

900

(e) **(1.0 pt)** Fill in blank (e)

931

(f) **(1.0 pt)** Fill in blank (f)

38

(g) **(1.0 pt)** Fill in blank (g)

68

(h) **(1.0 pt)** Fill in blank (h)

f4

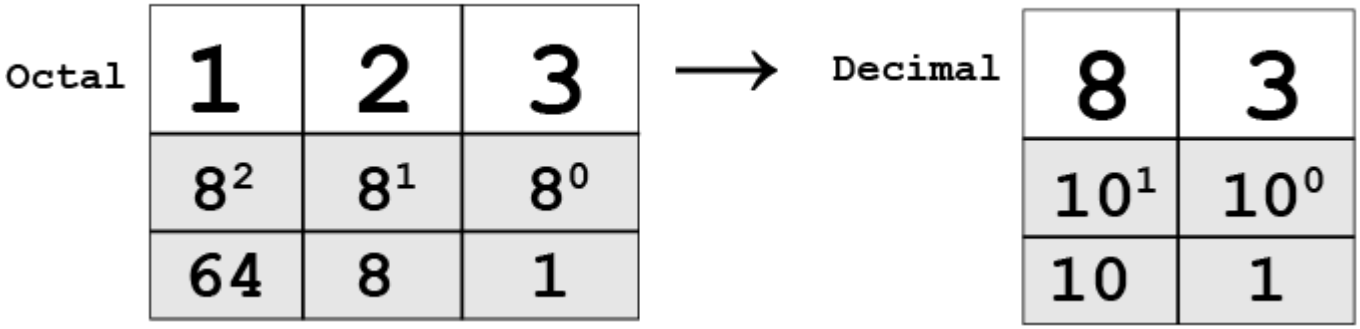
3. (6.0 points) Doctor Octalpus, Reborn

The standard number representation system is the decimal system, where each digit in a number represents a power of ten. The right-most digit is the ones' place, the next digit is the tens' place, etc.

In the octal system, each digit in a number represents a power of eight. The right-most digit is still the 1's place, but the next digit is the 8's place, the next digit is the 64's place, etc. Each digit ranges from 0-7, so octal numbers will never contain the digits 8 or 9.

To convert a number represented in octal to a number represented in decimal, each digit must be multipled by the appropriate power of eight. For example, `123` is actually `(1 * 64) + (2 * 8) + (3 * 1)`, resulting in a decimal representation of 83.

The diagram visualizes the equivalence between the octal and decimal numbers:



Implement `convert_to_decimal`, which takes an octal number and returns the decimal equivalent. The octal number will always start with a non-0 digit, and the number will always be positive.

```
def convert_to_decimal(octal):
    """
    >>> convert_to_decimal(3)   # (8^0 * 3)
    3
    >>> convert_to_decimal(23)  # (8^1 * 2) + (8^0 * 3)
    19
    >>> convert_to_decimal(123) # (8^2 * 1) + (8^1 * 2) + (8^0 * 3)
    83
    """
    decimal = 0
    curr_place = _________
                     (a)
    _________:
       (b)
        curr_digit = _________
                         (c)
        decimal = _________
                      (d)
        curr_place = _________
                         (e)
        octal = _________
                    (f)
    return decimal
```

(a) (1.0 pt) Fill in blank (a).

```
1
```

(b) (1.0 pt) Fill in blank (b).

```
while octal > 0
```

(c) (1.0 pt) Fill in blank (c).

```
octal % 10
```

(d) (1.0 pt) Fill in blank (d).

```
decimal + (curr_digit * curr_place)
```

(e) (1.0 pt) Fill in blank (e).

```
curr_place * 8
```

(f) (1.0 pt) Fill in blank (f).

```
octal // 10
```

4. (6.0 points) Forbidden Digits

Implement `forbid_digit`, a higher-order function which takes two arguments, a function `f` and a digit `forbidden`, and returns another function. If the returned function is passed a number where the digit in the 1s place is equal to the forbidden digit, it should return the result of calling the given function on the number without that final digit. Otherwise, it should return the result of calling the given function on the number.

```
def forbid_digit(f, forbidden):
    """
    >>> g = forbid_digit(lambda y: 200 // (y % 10), 0)
    >>> g(11)
    200
    >>> g(10)
    200
    >>> g = forbid_digit(lambda x: f'{x}a', 6)
    >>> g(61)
    '61a'
    >>> g(66)
    '6a'
    >>> g = forbid_digit(g, 3)
    >>> g(43)
    '4a'
    >>> g(63)
    '0a'
    >>> g(44)
    '44a'
    """
    def forbid_wrapper(n):
        if ________:
              (a)
            --------
              (b)
        else:
            --------
              (c)
    --------
       (d)
```

(a) (1.0 pt) Fill in blank (a).

```
n % 10 == forbidden
```

(b) (1.0 pt) Fill in blank (b).

```
return f(n // 10)
```

(c) (1.0 pt) Fill in blank (c).

```
return f(n)
```

(d) **(1.0 pt)** Fill in blank (d).

```
return forbid_wrapper
```

(e) **(2.0 pt)** Rewrite the body of the `forbid_digit` function in a single line of code.

```
def forbid_digit(f, forbidden):
    ____________________
```

```
return lambda n: f(n//10) if n % 10 == forbidden else f(n)
```

5. (7.0 points) The Floor is Lava

Implement `lava_hopper`, a function that "hops" from one number to the next computed number and tries to avoid any number detected as "lava". When it does land on "lava", it steps backwards by one number until it finds a non-lava number and then keeps hopping.

The function takes four arguments: `start_number` (the initial number), `goal_number` (the target number), `next_hop` (a function that computes the next number based on the current), and `is_lava` (a function that returns a boolean indicating if a number is lava), and it returns the minimum number of hops required to get from `start_number` to at least `goal_number`. The number of hops does *not* include steps backwards. If either the `start_number` or `goal_number` spots are lava, it returns the string 'No lava allowed there!'.

For example, consider this call

```
lava_hopper(1, 8, lambda x: x * 2, lambda x: x == 4)
```

The function starts from the number 1 and then hops to the numbers 2, 4, realizes that's lava, steps back to 3, hops to 6, hops to 12, and returns 4 (the number of hops required to get to/past 8).

Note that depending on the functions passed in for `next_hop` and `is_lava`, it is possible for a correct `lava_hopper` implementation to result in an infinite loop.

```
def lava_hopper(start_number, goal_number, next_hop, is_lava):
    """
    >>> # hops from 1->2, 2->4, 4->8
    >>> lava_hopper(1, 8, lambda x: x * 2, lambda x: False)
    3
    >>> # hops from 1->2, 2->4, steps to 3, hops 3->6, hops 6->12
    >>> lava_hopper(1, 8, lambda x: x * 2, lambda x: x == 4)
    4
    >>> # hops from 1->2, 2->4, 4->8, steps to 7, then 6, then 5, hops to 10
    >>> lava_hopper(1, 10, lambda x: x * 2, lambda x: 6 <= x <= 8)
    4
    >>> # hops from 3->6, 6->12, steps to 11, hops 11->22
    >>> lava_hopper(3, 20, lambda x: x * 2, lambda x: x % 10 == 2)
    3
    >>> lava_hopper(1, 8, lambda x: x * 2, lambda x: x == 1)
    'No lava allowed there!'
    >>> lava_hopper(1, 8, lambda x: x * 2, lambda x: x == 8)
    'No lava allowed there!'
    """
    if __________:
           (a)
        return 'No lava allowed there!'
    num_hops = 0
    while __________:
              (b)
        ________________:
              (c)
            ----------
                (d)
        start_number = __________
                    (e)
        ----------
            (f)
    return num_hops
```

(a) (1.0 pt) Fill in blank (a).

```
is_lava(start_number) or is_lava(goal_number)
```

(b) (1.0 pt) Fill in blank (b).

```
start_number < goal_number
```

(c) (1.0 pt) Fill in blank (c).

```
while is_lava(start_number)
```

(d) (1.0 pt) Fill in blank (d).

```
start_number -= 1
```

(e) (1.0 pt) Fill in blank (e).

```
next_hop(start_number)
```

(f) (1.0 pt) Fill in blank (f).

```
num_hops += 1
```

(g) (1.0 pt) Write a call to `lava_hopper` that would result in an infinite loop.

```
lava_hopper(______________________)
```

```
1, 8, lambda x: x + 1, lambda x: x == 6
```

6. (6.0 points) Curry Up Now

The function `order_meal` takes three arguments, `item_price`, `item_quantity`, and `ordered_at`, and either returns the total cost of the meal or reports how many hours should be waited before ordering. Only the doctests are shown below, as the implementation is not necessary for completing the question.

```
def order_meal(item_price, item_quantity, ordered_at):
    """
    >>> order_meal(5.99, 5, 11)
    29.95
    >>> order_meal(9.99, 5, 20)
    49.95
    >>> order_meal(8.99, 5, 7)
    'Wait!'
    """
    # Code intentionally omitted
```

Implement `curry_up_now`, a function that curries `order_meal` into a chain of three functions that each take a single argument. Once the third function is called, it should attempt to order the meal and print out the result. If the meal was successfully ordered during business hours, it should then return another curried function that can re-order the same item with a 50% discount.

```
def curry_up_now(item_price):
    """
    >>> curry_up_now(2.99)(2)(15)
    5.98
    <function <lambda>>
    >>> lunch_special = curry_up_now(8.99)
    >>> lunch_special(5)(11)
    44.95
    <function <lambda>>
    >>> lunch_special(3)(13)(2)(14)
    26.97
    8.99
    >>> no_discount = curry_up_now(10.99)(4)(7)
    Wait!
    >>> print(no_discount)
    None
    """
    def order_quantity(item_quantity):
        def by(ordered_at):
            result =  ________
                        (a)
            ________
               (b)
            ________:
               (c)
                return ________
                          (d)
        return by
    return order_quantity
```

(a) (1.0 pt) Fill in blank (a).

```
order_meal(item_price, item_quantity, ordered_at)
```

(b) (1.0 pt) Fill in blank (b).

```
print(result)
```

(c) (1.0 pt) Fill in blank (c).

```
if result != "Wait!"
```

(d) (1.0 pt) Fill in blank (d).

```
lambda item_quantity: lambda ordered_at: print(order_meal(item_price*0.50,
item_quantity, ordered_at))
```

(e) (2.0 pt) Rewrite the body of the inner `by` function in a single line of code, such that it behaves the same and passes the same doctests. Note that two calls to `order_meal` with the same parameters will always have the same result.

```
def curry_up_now(item_price):
    def order_quantity(item_quantity):
        def by(ordered_at):
            ----------------------
        return by
    return order_quantity
```

A large text area is provided to give you ample space to write, but your answer must be a valid *single* line of code.

```
print(order_meal(item_price, item_quantity, ordered_at)) or (
(lambda q: lambda h: print(order_meal(item_price*0.50, q, h))) if
order_meal(item_price, item_quantity, ordered_at) != "Wait!" else None)
```

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Spring 2022

Midterm 2 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) Below is a breakdown of the points per question, to help you budget your time.

Question	Points
Q1	5 pts
Q2	3 pts
Q3	2 pts
Q4	6 pts
Q5	4 pts
Q6	3 pts
Q7	6 pts
Q8	21 pts

1. (5.0 points) A Bug's Life

Thank you to our course staff members Richard, Melanie, and Amritansh for donating their bug collection to the midterm.

(a) (1.0 points) Wasp

Richard is trying to write a generator function that maps a given iterable using a one-argument function. Here's what he comes up with:

```
1 def map_gen(fn, iterable):
2     for it in iterable:
3         yield from fn(it)
```

i. (0.5 pt) Richard tests out his function by running the code below. He expected to see `[2, 4, 6, 8]` but instead gets an error when he runs these lines in an interpreter:

```
>>> data = [1, 2, 3, 4]
>>> fn = lambda x: x * 2
>>> list(map_gen(fn, data))
```

Which line of code in the function caused the code to break?

- ○ 1
- ○ 2
- ● 3

ii. (0.5 pt) Rewrite the buggy line of code you identified above to fix the error.

```
yield fn(it)
```

(b) (1.5 points) Moth

Melanie is trying to write some code to represent a cylinder in Python. This is her code:

```
1  class Cylinder:
2      def __init__(self, radius, height):
3          self.radius = radius
4          self.height = height
5
6      def volume(self):
7          volume = 3.14 * self.radius ** 2 * self.height
8          return volume
9
10     def percent_filled(self, volume_of_water):
11         return (volume_of_water / self.volume) * 100
```

i. (0.5 pt) Melanie runs the following code expecting to see it return 50.0, since 392.5 is a half-empty `cyl` (or is it half-full?).

```
>>> cyl = Cylinder(5, 10)
>>> cyl.percent_filled(392.5)
```

Unfortunately, the code results in this error instead:

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/Users/melanie/cylinder.py", line 11, in percent_filled
    return (volume_of_water / self.volume) * 100
TypeError: unsupported operand type(s) for /: 'int' and 'method'
```

According to this traceback, which method contains an error?

- ○ `__init__`
- ○ `volume`
- ● `percent_filled`

ii. (1.0 pt) Rewrite the buggy line of code from the method that you identified above to fix the error.

```
return (volume_of_water / self.volume()) * 100
```

(c) (2.5 points) Bumblebee

Amritansh steps up with a challenge of his own. He's writing some code to flatten lists, but he seems to be running into some errors. Here's his code:

```
1 def flatten(lst):
2     res = []
3     for el in lst:
4         if isinstance(el, list):
5             res += [res.extend(flatten(el))]
6         else:
7             res += [el]
8     return res
```

i. (1.0 pt) He runs this on a nested list, as below:

```
>>> my_data = ["Mialy", ["Daphne", "Jordan"]]
>>> flatten(my_data)
```

The result he expects is `["Mialy", "Daphne", "Jordan"]`. What is the actual result?

```
["Mialy", "Daphne", "Jordan", None]
```

ii. (0.5 pt) What line number causes the unexpected output to appear?

```
5
```

iii. (1.0 pt) Rewrite the buggy line you identified above so that Amritansh gets the expected output.

```
res += flatten(el) or res.extend(flatten(el))
```

2. (3.0 points) What Would Python Display?

Assume the following code has been run (you may find a box-and-pointer diagram useful):

```
>>> lst1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> number_one = lst1.remove(1)
>>> some_number = lst1.pop()
>>> lst2 = lst1
>>> lst3 = lst2[:]
>>> lst1.insert(4, lst3)
>>> lst3.extend([lst1.pop() for _ in range(3)])
>>> lst2.append(lst1.remove(lst3))
```

For each part below, write the output of running the given code. Assume that none of the lines below will error.

(a) (1.0 pt)

```
>>> lst1[4:] + [number_one, some_number]
```

```
[6, None, None, 10]
```

(b) (1.0 pt)

```
>>> lst3[5:9]
```

```
[7, 8, 9, 9]
```

(c) (0.5 pt)

```
>>> next(reversed(lst3[:5]))
```

```
6
```

(d) (0.5 pt)

```
>>> lst1 is lst2
```

```
True
```

3. (2.0 points) A Different Kind of Dictionary

The `DictionaryEntry` class stores entries for an English dictionary, using instance variables for the `word` and its `definition`.

```
class DictionaryEntry:
    """
    >>> euph = DictionaryEntry("euphoric", "intensely happy")
    >>> avid = DictionaryEntry("avid", "enthusiastic")
    >>> [euph, avid]
    [DictionaryEntry('euphoric', 'intensely happy'), DictionaryEntry('avid', 'enthusiastic')]
    >>> f'Today we are learning {euph}'
    'Today we are learning euphoric: "intensely happy"'
    """
    def __init__(self, w, d):
        self.word = w
        self.definition = d

    def __repr__(self):
        ----------
           (a)

    def __str__(self):
        ----------
           (b)
```

(a) (1.0 pt) Fill in blank (a).

```
return f"DictionaryEntry({repr(self.word)}, {repr(self.definition)})"
```

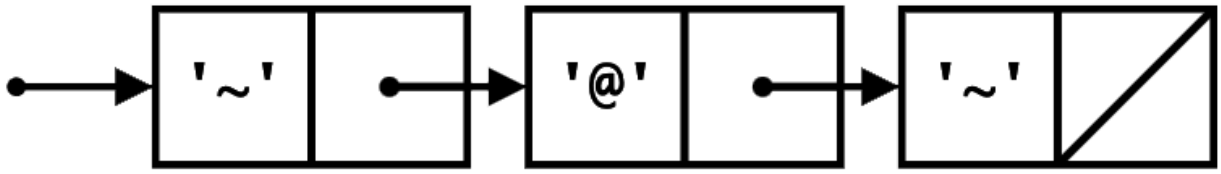
(b) (1.0 pt) Fill in blank (b).

```
return f"{self.word}: \"{self.definition}\""
```

4. (6.0 points) Beadazzled

Implement `make_necklace`, a function that creates a linked list where each value comes from a given list of `beads`, and the beads are repeated in order to make a necklace of a given `length`.

For example, if `make_necklace` is called with `["~", "@"]` and a `length` of 3, then the linked list will contain `'~'`, then `'@'`, then `'~'`. Here's a diagram of that list:



See the docstring and doctests for further details on how the function should behave.

```
def make_necklace(beads, length):
    """
    Returns a linked list where each value is taken from the BEADS list,
    repeating the values from the BEADS list until the linked list has reached
    LENGTH. You can assume that LENGTH is greater than or equal to 1, there is
    at least one bead in BEADS, and all beads are string values.

    >>> wavy_ats = make_necklace(["~", "@"], 3)
    >>> wavy_ats
    Link('~', Link('@', Link('~')))
    >>> print(wavy_ats)
    <~ @ ~>
    >>> wavy_ats2 = make_necklace(["~", "@"], 4)
    >>> print(wavy_ats2)
    <~ @ ~ @>
    >>> curly_os = make_necklace(["}", "O", "{"], 9)
    >>> print(curly_os)
    <} O { } O { } O {>
    """
    if __________:
           (a)
        return __________
                    (b)
    return Link(__________, make_necklace(__________, __________))
                    (c)                        (d)         (e)
```

(a) **(1.0 pt)** Fill in blank (a).

```
length == 0
```

(b) **(1.0 pt)** Fill in blank (b).

```
Link.empty
```

(c) **(1.0 pt)** Fill in blank (c).

```
beads[0]
```

(d) **(2.0 pt)** Fill in blank (d).

```
beads[1:] + [beads[0]]
```

(e) **(1.0 pt)** Fill in blank (e).

```
length - 1
```

5. (4.0 points) File Tree-tionaries

We can represent files and folders on a computer with Trees:

```
Tree("C:", [Tree("Documents", [Tree("hw05.py")]), Tree("pwd.txt")])
```

We can also represent the same folder layout with dictionaries:

```
{"C:": {"Documents": {"hw05.py": "FILE"}, "pwd.txt": "FILE"}}
```

Notice that in this model, we still treat files as dictionary keys, but with the value `"FILE"`.

Complete the implementation of `filetree_to_dict` below, which takes in a Tree `t` representing files and folders, and converts it to the dictionary representation.

```
def filetree_to_dict(t):
    """
    >>> filetree_to_dict(Tree("hw05.py"))
    {"hw05.py": "FILE"}
    >>> filetree = Tree("C:", [Tree("Documents", [Tree("hw05.py")]), Tree("pwd.txt")])
    >>> filetree_to_dict(filetree)
    {"C:": {"Documents": {"hw05.py": "FILE"}, "pwd.txt": "FILE"}}
    """
    res = {}
    if t.is_leaf():
        res[t.label] = ______________
                            (a)
    else:
        nested = {}
        for branch in t.branches:
            nested[branch.label] = ______________(branch)[______________]
                                        (b)                    (c)
        ______________ = nested
             (d)
    return res
```

(a) (1.0 pt) Fill in blank (a)

```
"FILE"
```

(b) (1.0 pt) Fill in blank (b)

```
filetree_to_dict
```

(c) (1.0 pt) Fill in blank (c)

```
branch.label
```

(d) (1.0 pt) Fill in blank (d)

```
res[t.label]
```

6. (3.0 points) Mapping Time and Space

(a) The goal of the `maplink_to_list` function below is to map a linked list `lnk` into a Python list, applying a provided function `f` to each value in the list along the way. The function is fully written and passes all its doctests.

```
def maplink_to_list1(f, lnk):
    """Returns a Python list that contains f(x) for each x in Link LNK.
    >>> square = lambda x: x * x
    >>> maplink_to_list1(square, Link(3, Link(4, Link(5))))
    [9, 16, 25]
    """
    new_lst = []
    while lnk is not Link.empty:
        new_lst.append(f(lnk.first))
        lnk = lnk.rest
    return new_lst
```

i. (1.0 pt) What is the order of growth of `maplink_to_list1` in respect to the size of the input linked list `lnk`?

- ◯ Constant
- ◯ Logarithmic
- ● Linear
- ◯ Quadratic
- ◯ Exponential

(b) The next function, `maplink_to_list2`, serves the same purpose but is implemented slightly differently. This alternative implementation also passes all the doctests.

```
def maplink_to_list2(f, lnk):
    """Returns a Python list that contains f(x) for each x in Link LNK.
    >>> square = lambda x: x * x
    >>> maplink_to_list2(square, Link(3, Link(4, Link(5))))
    [9, 16, 25]
    """
    def map_link(f, lnk):
        if lnk is Link.empty:
            return Link.empty
        return Link(f(lnk.first), map_link(f, lnk.rest))

    mapped_lnk = map_link(f, lnk)
    new_lst = []
    while mapped_lnk is not Link.empty:
        new_lst.append(mapped_lnk.first)
        mapped_lnk = mapped_lnk.rest
    return new_lst
```

i. (1.0 pt) What is the order of growth of the alternative function, `maplink_to_list2`, in respect to the size of the input linked list `lnk`?

- ◯ Constant
- ◯ Logarithmic
- ● Linear
- ◯ Quadratic
- ◯ Exponential

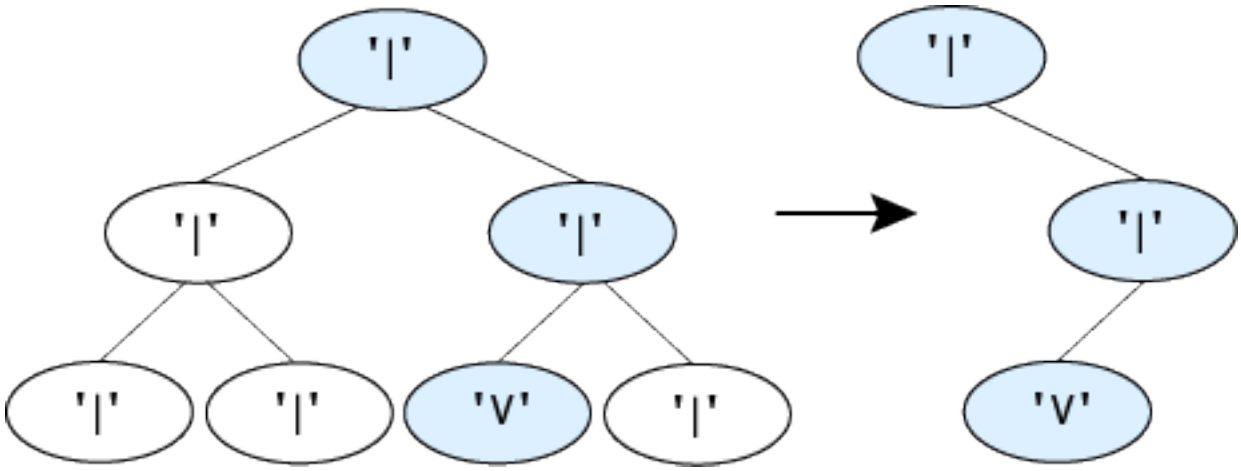
(c) (1.0 pt) Which of the functions requires more **space** to run?

- ○ `maplink_to_list1`
- ● `maplink_to_list2`
- ○ They both require the same amount of space.

7. (6.0 points) Tulip Mania

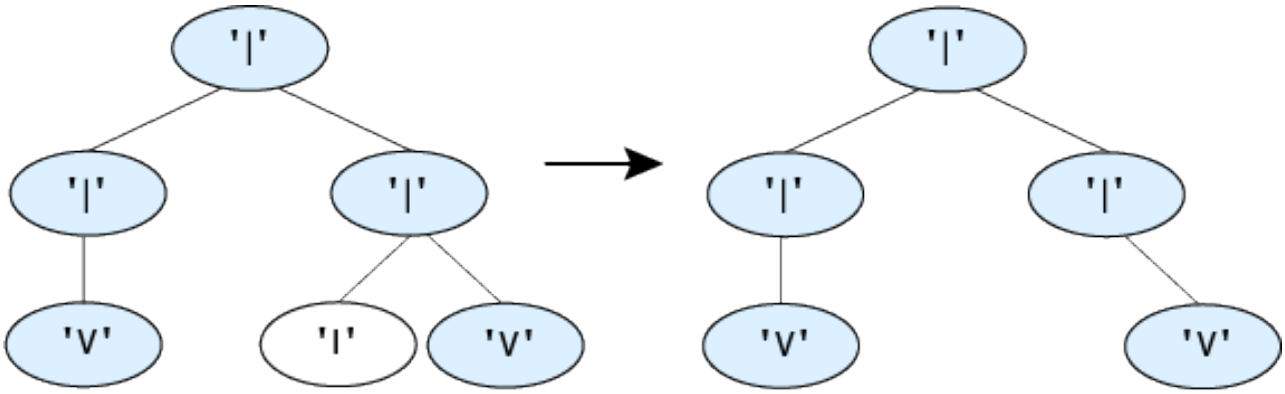
Implement `flower_keeper`, a function that mutates a tree `t` so that the only paths that remain are ones which end in a leaf node with a Tulip flower ('V').

For example, consider this tree where only one path ends in a flower. After calling `flower_keeper`, the tree has only three nodes left, the ones that lead to the flower.

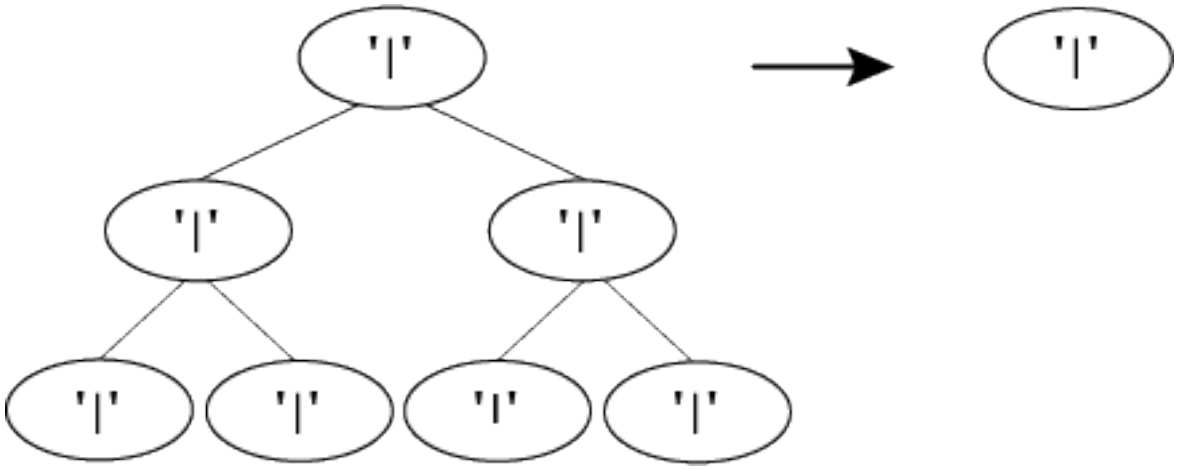


The shaded nodes in the diagram indicate paths that end in flowers.

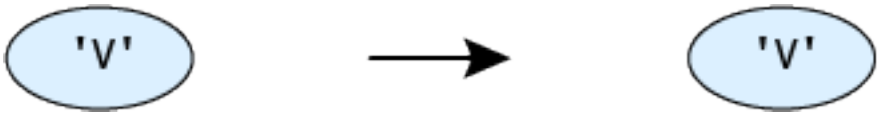
For this tree where two paths end in flowers, the tree keeps both paths that lead to flowers.



For a tree where none of the nodes are flowers, the function removes every branch except the root node.



For a tree with only a single node that is a flower, the function does not remove anything.



Read through the docstring and doctests for additional guidance on completing the implementation of the function. The examples shown above are from the doctests.

```
def flower_keeper(t):
    """
    Mutates the tree T to keep only paths that end in flowers ('V').
    If a path consists entirely of stems ('|'), it must be pruned.
    If T has no paths that end in flowers, the root node is still kept.
    You can assume that a node with a flower will have no branches.

    >>> one_f = Tree('|', [Tree('|', [Tree('|'), Tree('|')]), Tree('|', [Tree('V'), Tree('|')])])
    >>> print(one_f)
    |
      |
        |
        |
      |
        V
        |
    >>> flower_keeper(one_f)
    >>> one_f
    Tree('|', [Tree('|', [Tree('V')])])
    >>> print(one_f)
    |
      |
        V
    >>> no_f = Tree('|', [Tree('|', [Tree('|'), Tree('|')]), Tree('|', [Tree('|'), Tree('|')])])
    >>> flower_keeper(no_f)
    >>> no_f
    Tree('|')
    >>> just_f = Tree('V')
    >>> flower_keeper(just_f)
    >>> just_f
    Tree('V')
    >>> two_f = Tree('|', [Tree('|', [Tree('V')]), Tree('|', [Tree('|'), Tree('V')])])
    >>> flower_keeper(two_f)
    >>> two_f
    Tree('|', [Tree('|', [Tree('V')]), Tree('|', [Tree('V')])])
    """
    for b in __________:
                (a)
        __________
            (b)
    __________ = [__________ for b in __________ if __________]
        (c)           (d)                 (e)           (f)
```

(a) **(0.5 pt)** Fill in blank (a).

```
t.branches
```

(b) **(1.0 pt)** Fill in blank (b).

```
flower_keeper(b)
```

(c) **(1.0 pt)** Fill in blank (c).

```
t.branches
```

(d) **(1.0 pt)** Fill in blank (d).

```
b
```

(e) **(0.5 pt)** Fill in blank (e).

```
t.branches
```

(f) **(2.0 pt)** Fill in blank (f).

```
b.label == 'V' or not b.is_leaf()
```

8. (21.0 points) CS 61A Presents The Game of Hoop.

When we introduced the Hog project to you, we designed the rules of the game in a way that would make it possible to implement without using more advanced concepts like OOP and iterators/generators. Now that we've learned these concepts, you can rewrite parts of the game and expand it! The new version, called Hoop, will support *any* number of players greater than or equal to 2.

Assume the following changes from Hog:

- All rules have been taken out except for the Sow Sad rule, which is reiterated below.
- Each player's `strategy` is a function `strategy(own_score, other_scores)` that takes in that player's score and a *list* of the other players' scores, and outputs the number of times that player wishes to roll the dice during their turn. Order doesn't matter for the list of other players' scores.
- You don't have to keep track of the leader, and there is no commentary mechanism.

Sow Sad: If a player rolls any number of 1s in their turn, their score for that set of rolls is a 1 regardless of whatever else they rolled.

To start, take a look at the `HoopPlayer` class, which represents one player in our game:

```
class HoopPlayer:
    def __init__(self, strategy):
        """Initialize a player with STRATEGY, and a starting SCORE of 0. The
        STRATEGY should be a function that takes this player's score and a list
        of other players' scores.
        """
        self.strategy = strategy
        self.score = 0
```

Every game of Hoop will involve rolling a dice as well. Take a look at the `HoopDice` class, which represents a rollable dice that takes a set of values. You'll fill in the blanks in the first part of this question.

```
class HoopDice:
    def __init__(self, values):
        """Initialize a dice with possible values VALUES, and a starting INDEX
        of 0. The INDEX indicates which value from VALUES to return when the
        dice is rolled next.
        """
        self.values = values
        self.index = 0

    def roll(self):
        """Roll this dice. Advance the index to the next step before returning.
        """
        value = ______________
        ______________ = (______________) % ______________
        return value
```

(a) (4.0 points) The Dice: Making it Rollable

Begin by making the `HoopDice` rollable. Fill in the skeleton for the `HoopDice.roll` method below. See above for the rest of `HoopDice`.

```
class HoopDice:
    ...
    def roll(self):
        """Roll this dice. Advance the index to the next step before
        returning.
        >>> five_six = HoopDice([5, 6])
        >>> five_six.roll()
        5
        >>> five_six.index
        1
        >>> five_six.roll()
        6
        >>> five_six.index
        0
        """
        value = _______________
                    (1a)
        _______________ = (_______________) % _______________
            (1b)                (1c)                (1d)
        return value
```

i. (1.0 pt) Fill in blank (1a)

```
self.values[self.index]
```

ii. (1.0 pt) Fill in blank (1b)

```
self.index
```

iii. (1.0 pt) Fill in blank (1c)

```
self.index + 1
```

iv. (1.0 pt) Fill in blank (1d)

```
len(self.values)
```

(b) (2.0 points) The Game: Looping Through Players

For the rest of the question, assume that `HoopDice.roll` is implemented correctly, regardless of your answers above.

In the rest of the question, you will work on making the game itself. Take a look at the `HoopGame` class, which takes in the setup for your game and makes it playable. You'll fill in the blanks shortly.

```
class HoopGame:
    def __init__(self, players, dice, goal):
        """Initialize a game with a list of PLAYERS, which contains at least one
        HoopPlayer, a single HoopDice DICE, and a goal score of GOAL.
        """
        self.players = players
        self.dice = dice
        self.goal = goal

    def next_player(self):
        """Infinitely yields the next player in the game, in order."""
        yield from ______________
        yield from ______________

    def get_scores(self):
        """Collects and returns a list of the current scores for all players
        in the same order as the SELF.PLAYERS list.
        """
        # Implementation omitted. Assume this method works correctly.

    def get_scores_except(self, player):
        """Collects and returns a list of the current scores for all players
        except PLAYER.
        """
        return [______________ for pl in ______________ if ______________]

    def roll_dice(self, num_rolls):
        """Simulate rolling SELF.DICE exactly NUM_ROLLS > 0 times. Return sum of
        the outcomes unless any of the outcomes is 1. In that case, return 1.
        """
        outcomes = [______________ for x in ______________]
        ones = [______________ for outcome in outcomes]
        return 1 if ______________(ones) else ______________(outcomes)

    def play(self):
        """Play the game while no player has reached or exceeded the goal score.
        After the game ends, return all players' scores.
        """
        player_gen = self.next_player()
        while max(self.get_scores()) < self.goal:
            player = ______________(player_gen)
            other_scores = self.get_scores_except(player)
            num_rolls = ______________(player.score, other_scores)
            outcome = self.roll_dice(num_rolls)
            ______________ += outcome
        return self.get_scores()
```

To see what the expected behavior of each method is, take a look at its doctests. These will be provided for the relevant methods in each subpart for the rest of the question.

In all following subparts, assume that the following has already been run to set up the game:

```
>>> roll_once_strategy = lambda pl, ops: 1
>>> roll_twice_strategy = lambda pl, ops: 2
>>> always_5 = HoopDice([5])
>>> player1 = HoopPlayer(roll_twice_strategy)
>>> player2 = HoopPlayer(roll_once_strategy)
>>> player3 = HoopPlayer(lambda pl, ops: 6)
>>> game = HoopGame([player1, player2, player3], always_5, 55)
>>> # since we omit the implementation of HoopGame.get_scores, here's what it
>>> # should output:
>>> game.get_scores()
[0, 0, 0]
```

The first `HoopGame` method you'll implement is `HoopGame.next_player`, which gives the game a way to iterate over all of the players playing, in order to give them each a turn in order.

```
class HoopGame:
    ...
    def next_player(self):
        """Infinitely yields the next player in the game, in order.
        >>> player_gen = game.next_player()
        >>> next(player_gen) is player1
        True
        >>> next(player_gen) is player3
        False
        >>> next(player_gen) is player3
        True
        >>> next(player_gen) is player1
        True
        >>> next(player_gen) is player2
        True
        >>> new_player_gen = game.next_player()
        >>> next(new_player_gen) is player1
        True
        >>> next(player_gen) is player3
        True
        """
        yield from ______________
                        (2a)
        yield from ______________
                        (2b)
```

i. (1.0 pt) Fill in blank (2a)

```
self.players
```

ii. (1.0 pt) Fill in blank (2b)

```
self.next_player()
```

(c) (3.0 points) The Game: Getting Scores

The `HoopGame.get_scores` method is already fully implemented and provides a way to get a list of the scores of *all* players.

However, each player's strategy relies on the player's *opponents'* scores, so `HoopGame` also needs a method that gives the scores of all players *except* the current one. Fill in the skeleton for `HoopGame.get_scores_except` below. Keep in mind that multiple players may have the same score.

```
class HoopGame:
    ...
    def get_scores_except(self, player):
        """Collects and returns a list of the current scores for all players
        except PLAYER.
        >>> game.get_scores_except(player2)
        [0, 0]
        """
        return [_______________ for pl in _______________ if _______________]
                    (3a)                    (3b)              (3c)
```

As a reminder, here's what the `HoopPlayer` class looks like:

```
class HoopPlayer:
    def __init__(self, strategy):
        """Initialize a player with STRATEGY, and a starting SCORE of 0. The
        STRATEGY should be a function that takes this player's score and a list
        of other players' scores.
        """
        self.strategy = strategy
        self.score = 0
```

i. (1.0 pt) Fill in blank (3a)

```
pl.score
```

ii. (1.0 pt) Fill in blank (3b)

```
self.players
```

iii. (1.0 pt) Fill in blank (3c)

```
pl is not player
```

(d) (5.0 points) The Game: Rolling Dice

Next, give `HoopGame` the ability to roll its dice some given number of times and return the total from rolling all those times. Recall that the dice in the game is just a single dice, represented by the `HoopDice` instance at `self.dice`. Fill in the skeleton for `HoopGame.roll_dice` below:

```
class HoopGame:
    ...
    def roll_dice(self, num_rolls):
        """Simulate rolling SELF.DICE exactly NUM_ROLLS > 0 times. Return sum of
        the outcomes unless any of the outcomes is 1. In that case, return 1.
        >>> game.roll_dice(4)
        20
        """
        outcomes = [_____________ for x in _____________]
                        (4a)                    (4b)
        ones = [_____________ for outcome in outcomes]
                    (4c)
        return 1 if _____________(ones) else _____________(outcomes)
                        (4d)                    (4e)
```

i. (1.0 pt) Fill in blank (4a)

```
self.dice.roll()
```

ii. (1.0 pt) Fill in blank (4b)

```
range(num_rolls)
```

iii. (1.0 pt) Fill in blank (4c)

```
outcome == 1
```

iv. (1.0 pt) Fill in blank (4d)

```
any
```

v. (1.0 pt) Fill in blank (4e)

```
sum
```

(e) (3.0 points) The Game: Putting it all Together

Finally, it's time to play the game! Recall that a player's strategy depends on two inputs: the player's own score, and a list of the scores of the player's opponents. Using existing attributes wherever possible, fill in the skeleton for `HoopGame.play` below:

```
class HoopGame:
    ...
    def play(self):
        """Play the game while no player has reached or exceeded the goal score.
        After the game ends, return all players' scores.
        >>> game.play()
        [20, 10, 60]
        """
        player_gen = self.next_player()
        while max(self.get_scores()) < self.goal:
            player = _______________(player_gen)
                        (5a)
            other_scores = self.get_scores_except(player)
            num_rolls = _______________(player.score, other_scores)
                           (5b)
            outcome = self.roll_dice(num_rolls)
            _______________ += outcome
                (5c)
        return self.get_scores()
```

i. (1.0 pt) Fill in blank (5a)

```
next
```

ii. (1.0 pt) Fill in blank (5b)

```
player.strategy
```

iii. (1.0 pt) Fill in blank (5c)

```
player.score
```

(f) (4.0 points) The Dice: It's Broken!

Now that you've finished implementing the game, you'll implement a new kind of `BrokenHoopDice`. This dice extends `HoopDice`, but is broken! That is, it alternates between behaving like a normal dice and returning `when_broken` value between turns.

For this part of the question, you may not use `=` or `lambda`.

```
class BrokenHoopDice(HoopDice):
    def __init__(self, values, when_broken):
        ______________(values)
             (6a)
        self.when_broken = when_broken
        self._______________ = False
                  (6b)

    def roll(self):
        """
        >>> broken = BrokenHoopDice([5, 6, 7], 11)
        >>> broken.roll()
        5
        >>> [broken.roll() for _ in range(6)]
        [11, 6, 11, 7, 11, 5]
        """
        if self.is_broken:
            self.is_broken = not self.is_broken
            return _______________
                        (6c)
        else:
            self.is_broken = not self.is_broken
            return _______________()
                        (6d)
```

i. (1.0 pt) Fill in blank (6a)

```
super().__init__
```

ii. (1.0 pt) Fill in blank (6b)

```
is_broken
```

iii. (1.0 pt) Fill in blank (6c)

```
self.when_broken
```

iv. (1.0 pt) Fill in blank (6d)

```
super().roll
```

9. (0.0 points) Just for Fun

Draw some cool dice faces for Hoop! This is not for points and will not be graded.

No more questions.

CS 61A Structure and Interpretation of Computer Programs

Spring 2023 MIDTERM 1

INSTRUCTIONS

You have 1 hour and 50 minutes to complete the exam.

- The exam is closed book, closed notes, closed computer, closed calculator, except one 8.5" × 11" page of your own creation and the provided midterm 1 study guide.
- Mark your answers on the exam itself in the spaces provided. We will not grade answers written on scratch paper or outside the designated answer spaces.
- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `pow`, `len`, `abs`, `bool`, `int`, `float`, `str`, `round`, `max`, and `min`.
- You **may not** use example functions defined on your study guide unless a problem clearly states you can.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.
- You may not use ; to place two statements on the same line.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (10.0 points) What Would Python Display?

Assume the following code has been executed.

```
e = 40
def glob(e):
    print(e)
    e = e + 1
    return e

def mid(nights):
    days = 7
    if days or nights:
        days, nights = (days and days + 1), (nights or days)
    print(days)
    return nights

def special(f, x):
    return f(x) + f(x)
```

For each expression below, write the output **displayed by the interactive Python interpreter** when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. To display a function value, write "Function".

(a) (2.0 pt) `print(0, print(1), 2)`

1
0 None 2

(b) (4.0 pt) `print(glob(e), glob(3), e)`

40
3
41 4 40

(c) (2.0 pt) `mid(not 'swift')`

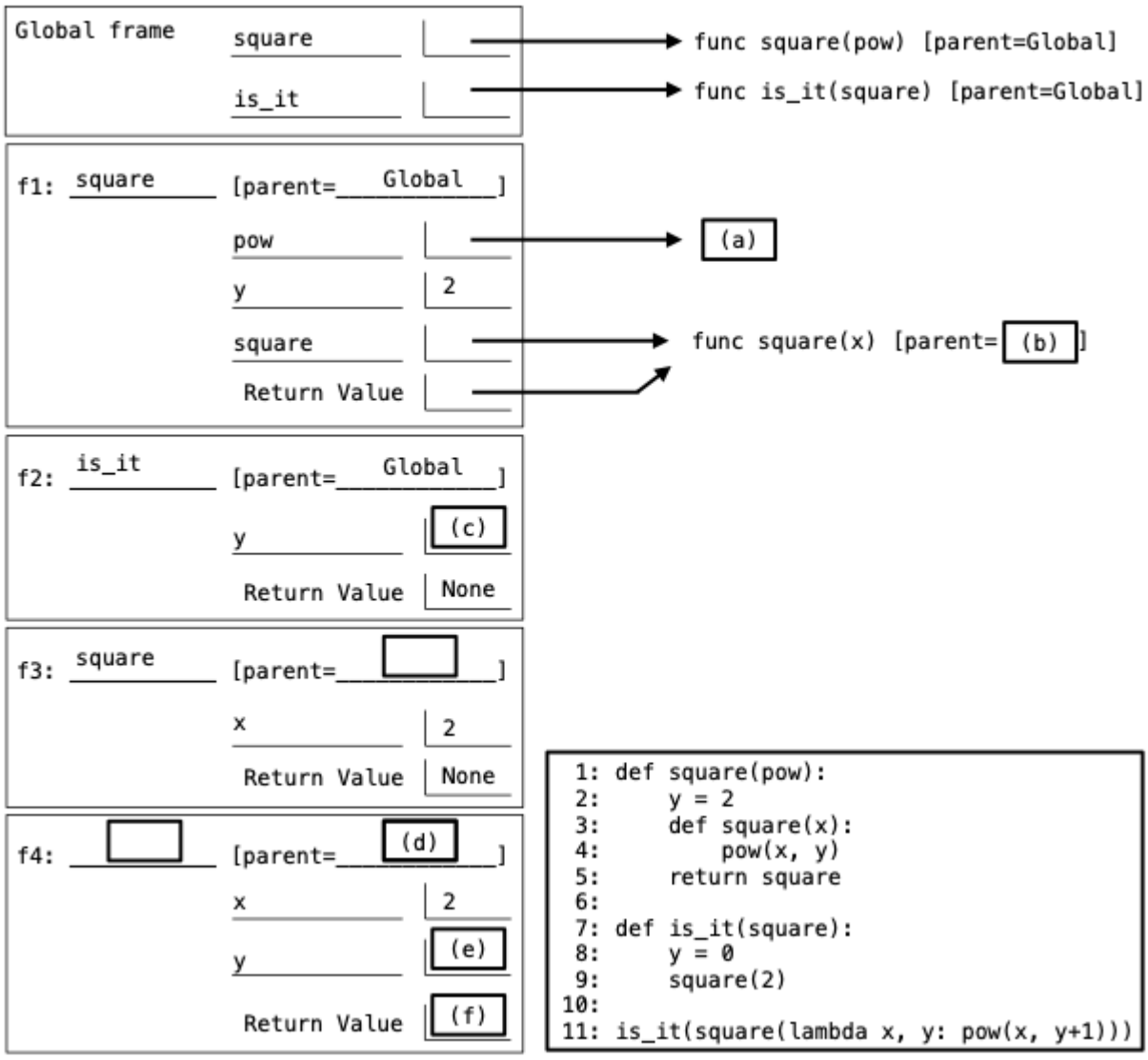
8
7

(d) (2.0 pt) `special(str, 10)`

'1010'

2. (6.0 points) Square the Square

Complete the environment diagram below and then answer the questions that follow. There is a one question for each labeled blank in the diagram. The blanks with no labels have no questions associated with them and are not scored. **If a blank contains an arrow to a function, write the function as it would appear in the diagram. Do not add frames for calls to built-in functions.**



(a) (1.0 pt) Fill in blank (a).

- ○ `func pow(...) [parent=Global]`
- ○ `func pow(...) [parent=f1]`
- ● `func lambda <line 11>(x, y) [parent=Global]`
- ○ `func lambda <line 11>(x, y) [parent=f1]`

(b) (1.0 pt) Fill in blank (b).

- [x] `f1`
- [ ] `Global`
- [ ] `square`
- [ ] None of these

(c) (1.0 pt) Fill in blank (c).

- [x] `0`
- [ ] `1`
- [ ] `2`
- [ ] None of these

(d) (1.0 pt) Fill in blank (d).

- [ ] `f1`
- [x] `Global`
- [ ] `square`
- [ ] None of these

(e) (1.0 pt) Fill in blank (e).

- [ ] `0`
- [ ] `1`
- [x] `2`
- [ ] `3`
- [ ] None of these

(f) (1.0 pt) Fill in blank (f).

- [ ] `1`
- [ ] `2`
- [ ] `4`
- [x] `8`
- [ ] None of these

3. (14.0 points) On Repeat

Definition: A positive integer `n` is a *repeating sequence* of positive integer `m` if `n` is written by repeating the digits of `m` one or more times. For example, 616161 is a repeating sequence of 61, but 61616 is not.

Hint: `pow(b, n)` raises `b` to the power of `n`. For example, `pow(10, 3)` is `1000`, and `654321 % pow(10, 3)` is `321` (the last 3 digits).

(a) (8.0 points)

Implement `repeating` which takes positive integers `t` and `n`. It returns whether `n` is a repeating sequence of some `t`-digit integer. **You may not use `str` or `repr` or `[` or `]` or `for`.**

```
def repeating(t, n):
    """Return whether t digits repeat to form positive integer n.

    >>> repeating(1, 6161)
    False
    >>> repeating(2, 6161)  # repeats 61 (2 digits)
    True
    >>> repeating(3, 6161)
    False
    >>> repeating(4, 6161)  # repeats 6161 (4 digits)
    True
    >>> repeating(5, 6161)  # there are only 4 digits
    False
    """
    if _________ > n:  # check that n has at least t digits
          (a)

        return False

    rest = n

    while rest:

        if rest % pow(10, t) != _________:
                                    (b)

            _________
               (c)


        _________
           (d)

    return _________
              (e)
```

i. (2.0 pt) Fill in blank (a).

- ○ `t`
- ○ `t - 1`
- ○ `10 * t`
- ○ `10 * (t - 1)`
- ○ `pow(10, t)`
- ● `pow(10, t - 1)`

ii. **(2.0 pt)** Fill in blank (b)

```
n % pow(10, t)
```

iii. **(1.0 pt)** Fill in blank (c)

```
return False
```

iv. **(2.0 pt)** Fill in blank (d)

```
rest = rest // pow(10, t)
```

v. **(1.0 pt)** Fill in blank (e).

- (●) `True`
- ( ) `False`
- ( ) `rest == n`
- ( ) `rest != n`

(b) (6.0 points)

Implement `digits`, which takes a positive integer `n`. It returns the smallest `m` for which `n` is a repeating sequence of `m`. **You may not use `str` or `repr` or `[` or `]` or `for`.** Assume `repeating` is implemented correctly (and you may use it).

```
def digits(n):
    """Return the smallest m that repeats to form n.

    >>> digits(6161616161)
    61
    >>> digits(33333333)
    3
    >>> digits(12312312)
    12312312
    >>> digits(123123123)
    123
    """
    k = 1

    while True:

        if _________:
              (f)

            return _________
                       (g)
        _________
           (h)
```

i. (3.0 pt) Fill in blank (f).

```
repeating(k, n)
```

ii. (2.0 pt) Fill in blank (g).

```
n % pow(10, k)
```

iii. (1.0 pt) Fill in blank (h).

- (●) `k = k + 1`
- ( ) `n = n // 10`
- ( ) `return False`
- ( ) `return n`

4. (10.0 points) Perfect Ten

Definitions:

- A *non-negative function* takes one argument and returns a non-negative integer.
- The *nearest* multiple of 10 to a non-negative integer `n` is larger than `n` if the last digit of `n` is 5 or more and less than or equal to `n` otherwise. For example, the nearest multiple of 10 to 25 is 30, but the nearest multiple of 10 to 24 is 20.

(a) (5.0 points)

Implement `nearest_ten`, which takes a non-negative function `f`. The function returned by calling `nearest_ten(f)` takes one argument `n` and returns the nearest multiple of 10 to `f(n)`. (In other words, it rounds `f(n)` to the nearest ten.)

```
def nearest_ten(f):
    """nearest_ten returns a function of n that returns the multiple of 10 nearest to f(n).

    >>> h = nearest_ten(lambda n: n * n)
    >>> h(2)  # The nearest multiple of 10 to 2 * 2 = 4
    0
    >>> h(3)  # The nearest multiple of 10 to 3 * 3 = 9
    10
    >>> h(5)  # The nearest multiple of 10 to 5 * 5 = 25
    30
    >>> h(8)  # The nearest multiple of 10 to 8 * 8 = 64
    60
    >>> h(10)  # The nearest multiple of 10 to 10 * 10 = 100
    100
    """
    def g(n):

        y = _________
              (a)

        rest, ones = y // 10, y % 10

        if _________:
             (b)

            rest = _________
                      (c)

        return _________
                  (d)

    return g
```

i. (2.0 pt) Fill in blank (a). **You may not use `round` or `int` or `//` for this blank.**

`f(n)`

ii. **(1.0 pt)** Fill in blank (b).

- ○ `rest > 5`
- ○ `rest >= 5`
- ○ `ones > 5`
- ● `ones >= 5`
- ○ `y > 5`
- ○ `y >= 5`

iii. **(1.0 pt)** Fill in blank (c).

- ○ `rest + ones`
- ○ `rest * 10 + ones`
- ○ `rest // 10`
- ○ `rest % 10`
- ● `rest + 1`

iv. **(1.0 pt)** Fill in blank (d).

- ○ `rest`
- ● `rest * 10`
- ○ `rest + ones`
- ○ `rest * 10 + ones`
- ○ `rest + round(ones)`
- ○ `rest * 10 + round(ones)`
- ○ `round(rest + ones)`
- ○ `round(rest * 10 + ones)`

(b) (5.0 points)

Implement `distance`, which takes two arguments: a positive integer function `f` and a positive integer `n`. It returns the absolute difference (a non-negative integer) between `f(n)` and the nearest multiple of 10 to `f(n)`. **You may not call `round` or `int` or use `+` or `-`.** Assume `nearest_ten` is implemented correctly (and you may use it).

```
def distance(f, n):
    """Return the absolute value of the difference between f(n)
    and the nearest multiple of 10 to f(n).

    >>> distance(lambda n: n * n, 4)  # 16 is 4 away from 20
    4
    >>> distance(lambda n: n * n, 9)  # 81 is 1 away from 80
    1
    """
    return abs(_________ - _________)
                  (e)         (f)
```

i. (2.0 pt) Fill in blank (e).

- ○ `f`
- ● `f(n)`
- ○ `n`
- ○ `n * n`
- ○ `lambda n: n`
- ○ `lambda n: f(n)`
- ○ `lambda n: n * n`
- ○ `(lambda n: n * n)(n)`

ii. (3.0 pt) Fill in blank (f).

```
nearest_ten(f)(n)
```

(c) This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Use `nearest_ten` to implement `nearest_hundred`, which takes a positive integer function `f`. The function returned by calling `nearest_hundred(f)` takes a positive integer `n` and returns the nearest multiple of 100 to `f(n)`. **Note:** The nearest multiple of 100 to `n` is larger than `n` if and only if the tens digit of `n` is 5 or more, so 300 is nearest to 250, but 200 is nearest to 249. **You may not call `round` or `int`. You may not use `str` or `repr` or `%`.** Assume `nearest_ten` is implemented correctly, and you **must** use it.

```
def nearest_hundred(f):
    """nearest_hundred(f)(n) is the multiple of 100 nearest to f(n).

    >>> h = nearest_hundred(lambda n: n * n)
    >>> h(8)  # The nearest multiple of 100 to 8 * 8 = 64
    100
    >>> h(20)  # The nearest multiple of 100 to 20 * 20 = 400
    400
    >>> h(25)  # The nearest multiple of 100 to 25 * 25 = 625
    600
    """
    return _________
```

i. Fill in the blank.

```
lambda n: nearest_ten(lambda y: f(y) // 10)(n) * 10
lambda n: nearest_ten(lambda y: y // 10)(f(n)) * 10
lambda n: nearest_ten(lambda y: y)(f(n) // 10) * 10
```

5. Just for Fun

(a) Optional: If CS 61A were a sport, what would it look like? (Feel free to draw.)

CS 61A Spring 2023

Structure and Interpretation of Computer Programs

Midterm 2 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (6.0 points) What Would Python Display?

Assume the following code has been executed.

```
def chain(s):
    return [s[0], s[1:]]

silver = [2, chain([3, 4, 5])]
gold = [silver[0], silver[1].pop()]
silver[0] = 1
platinum = chain(chain([6, 7, 8]))
```

Write the output **displayed by the interactive Python interpreter** when each expression below is evaluated.

Hint: Try drawing an environment diagram!

Reminder: `s.pop()` removes and returns the last item in list `s`.

(a) (2.0 pt) `silver`

```
[1, [3]]
```

(b) (2.0 pt) `gold`

```
[2, [4, 5]]
```

(c) (2.0 pt) `platinum`

```
[6, [[7, 8]]]
```

2. (10.0 points) Letter Grade

(a) (4.0 points)

Implement the `Letter` class. A `Letter` has two instance attributes: `contents` (a `str`) and `sent` (a `bool`). Each `Letter` can only be sent once. The `send` method prints whether the letter was sent, and if it was, returns the reply, which is a new `Letter` instance with the same `contents`, but in all caps. *Hint*: `'hi'.upper()` evaluates to `'HI'`.

```
class Letter:
    """A letter receives an all-caps reply when sent successfully.

    >>> hi = Letter('Hello, World!')
    >>> hi.send()
    Hello, World! has been sent.
    HELLO, WORLD!
    >>> hi.send()
    Hello, World! was already sent.
    >>> Letter('Hey').send().send()
    Hey has been sent.
    HEY has been sent.
    HEY
    """
    def __init__(self, contents):
        self.contents = contents
        _________
           (a)
    def send(self):
        if self.sent:
            print(self, 'was already sent.')
        else:
            print(self, 'has been sent.')
            _________
               (b)
            return _________
                       (c)
    def __repr__(self):
        # Note: since no __str__ method is defined, the repr and str strings are the same.
        return self.contents
```

i. (1.0 pt) Fill in blank (a).

```
self.sent = False
```

ii. (1.0 pt) Fill in blank (b)

```
self.sent = True
```

iii. (2.0 pt) Fill in blank (c)

```
Letter(self.contents.upper())
```

(b) (6.0 points)

Implement the `Numbered` class. A `Numbered` letter has a `number` attribute equal to how many numbered letters have previously been constructed. This number appears in its `repr` string. Assume `Letter` is implemented correctly.

```
class Numbered(Letter):
    """A numbered letter has a different repr method that shows its number.

    >>> hey = Numbered('Hello, World!')
    >>> hey.send()
    #0 has been sent.
    HELLO, WORLD!
    >>> Numbered('Hi!').send()
    #1 has been sent.
    HI!
    >>> hey
    #0
    """
    number = 0

    def __init__(self, contents):
        super().__init__(contents)
        _________
           (d)
        _________
           (e)

    def __repr__(self):
        return '#' + _________
                        (f)
```

i. (2.0 pt) Fill in blank (d).

- ○ `Numbered.number = 0`
- ○ `Numbered.number = number`
- ○ `Numbered.number = self.number`
- ○ `self.number = 0`
- ○ `self.number = number`
- ● `self.number = Numbered.number`

ii. (2.0 pt) Fill in blank (e).

```
Numbered.number += 1
```

iii. (2.0 pt) Fill in blank (f).

```
str(self.number)
```

3. (9.0 points) Prefixes

Definition. A *prefix sum* of a sequence of numbers is the sum of the first `n` elements for some positive length `n`.

(a) (4.0 points)

Implement `prefix`, which takes a list of numbers `s` and returns a list of the prefix sums of `s` in increasing order of the length of the prefix.

```
def prefix(s):
    """Return a list of all prefix sums of list s.

    >>> prefix([1, 2, 3, 0, 4, 5])
    [1, 3, 6, 6, 10, 15]
    >>> prefix([2, 2, 2, 0, -5, 5])
    [2, 4, 6, 6, 1, 6]
    """
    return [_________ for k in _________]
               (a)                (b)
```

i. (2.0 pt) Fill in blank (a).

```
sum(s[:k+1])
```

ii. (1.0 pt) Fill in blank (b).

- ○ `s`
- ○ `[s]`
- ○ `s[1:]`
- ○ `range(s)`
- ● `range(len(s))`

iii. (1.0 pt) What is the order of growth of the time to run this alternate implentation in terms of the length of `s`?

Assume that the `append` method of a list takes just 1 step (constant time).

```
def prefix(s):
    """Return a list of all prefix sums of list s."""
    t = 0
    result = []
    for x in s:
        t = t + x
        result.append(t)
    return result
```

- ○ Constant
- ○ Logarithmic
- ● Linear
- ○ Quadratic
- ○ Exponential

(b) (5.0 points)

Implement `tens`, which takes a **non-empty** linked list of numbers `s` represented as a `Link` instance. It prints all of the prefix sums of `s` that are multiples of 10 in increasing order of the length of the prefix.

The `Link` class appears on the midterm 2 study guide.

```
def tens(s):
    """Print all prefix sums of Link s that are multiples of ten.

    >>> tens(Link(3, Link(9, Link(8, Link(10, Link(0, Link(14, Link(6)))))))
    20
    30
    30
    50
    """

    def f(suffix, total):

        if total % 10 == 0:

            print(total)

        if _________:
              (d)

            _________
               (e)

    _________
       (f)
```

i. (1.0 pt) Fill in blank (d).

- ○ `s is not Link.empty`
- ○ `s.rest is not Link.empty`
- ○ `s.rest.rest is not Link.empty`
- ● `suffix is not Link.empty`
- ○ `suffix.rest is not Link.empty`
- ○ `suffix.rest.rest is not Link.empty`

ii. (2.0 pt) Fill in blank (e).

```
f(suffix.rest, total + suffix.first)
```

iii. (2.0 pt) Fill in blank (f).

```
f(s.rest, s.first)
```

4. (10.0 points) Tree Trimming

(a) (7.0 points)

Implement `exclude`, which takes a `Tree` instance `t` and a value `x`. It returns a `Tree` containing the root node of `t` as well as each non-root node of `t` with a label not equal to `x`. The parent of a node in the result is its nearest ancestor node that is not excluded. The input `t` must not be modified.

The `Tree` class appears on the midterm 2 study guide.

```
def exclude(t, x):
    """Return a Tree with the non-root nodes of t whose labels are not equal to x.

    >>> t = Tree(1, [Tree(2, [Tree(2), Tree(3)]), Tree(4, [Tree(1)])])
    >>> exclude(t, 2)
    Tree(1, [Tree(3), Tree(4, [Tree(1)])])
    >>> t              # t is not changed
    Tree(1, [Tree(2, [Tree(2), Tree(3)]), Tree(4, [Tree(1)])])
    >>> exclude(t, 1)  # The root node cannot be excluded
    Tree(1, [Tree(2, [Tree(2), Tree(3)]), Tree(4)])
    """

    filtered_branches = map(lambda y: _________, t.branches)
                                          (a)

    bs = []

    for b in filtered_branches:

        if _________:
              (b)

            bs._________(_________)
                  (c)       (d)

        else:

            bs.append(b)

    return Tree(t.label, bs)
```

i. (2.0 pt) Fill in blank (a).

```
exclude(y, x)
```

ii. (2.0 pt) Fill in blank (b).

```
b.label == x
```

iii. (1.0 pt) Fill in blank (c).

- ○ `remove`
- ○ `pop`
- ○ `append`
- ● `extend`

iv. (2.0 pt) Fill in blank (d).

- ○ `b`
- ● `b.branches`
- ○ `exclude(b, x)`
- ○ `Tree(b.label, [exclude(y, x) for y in b.branches])`

(b) (3.0 points)

Implement `remove`, which takes a `Tree` instance `t` and a value `x`. It removes all non-root nodes from `t` that have a label equal to `x`, then returns `t`. The parent of a node in `t` is its nearest ancestor that is not removed. Assume that `exclude` is implemented correctly.

```
def remove(t, x):
    """Remove all non-root nodes labeled x from t and return t.

    >>> t = Tree(1, [Tree(2, [Tree(2), Tree(3)]), Tree(4)])
    >>> remove(t, 2)
    Tree(1, [Tree(3), Tree(4)])
    >>> remove(t, 3)  # t was changed, so both 2 and 3 are now removed.
    Tree(1, [Tree(4)])
    """

    ---------
       (e)

    return t
```

i. (3.0 pt) Fill in blank (e).

```
t.branches = exclude(t, x).branches
```

5. (10.0 points) Parking

Definition. When parking vehicles in a row, a *motorcycle* takes up 1 parking spot and a *car* takes up 2 adjacent parking spots. A string of length `n` can represent `n` adjacent parking spots using `%` for a motorcycle, `<>` for a car, and `.` for an empty spot. For example `'.%%.<><>'` represents an empty spot, then two spots containing motorcycles, then another empty spot, then four spots containing two cars.

Thanks to the Berkeley Math Circle for introducing this question.

(a) (4.0 points)

Implement `count_park`, which returns the number of ways that vehicles can be parked in `n` adjacent parking spots for positive integer `n`. Some or all spots can be empty.

```
def count_park(n):
    """Count the ways to park cars and motorcycles in n adjacent spots.

    >>> count_park(1)  # '.' or '%'
    2
    >>> count_park(2)  # '..', '.%', '%.', '%%', or '<>'
    5
    >>> count_park(4)  # some examples: '<><>', '.%%.', '%<>%', '%.<>'
    29
    """

    if n < 0:

        return 0

    if n == 0:

        return __________
                  (a)

    return __________
              (b)
```

i. (1.0 pt) Fill in blank (a).

```
1
```

ii. (3.0 pt) Fill in blank (b).

```
2 * count_park(n-1) + count_park(n-2)
```

(b) (6.0 points)

Implement `park`, a generator function that takes a non-negative integer `n`. It yields (in any order) all possible strings representing `n` adjacent parking spots.

```
def park(n):
    """Yield the ways to park cars and motorcycles in n adjacent spots.

    >>> sorted(park(1))
    ['%', '.']
    >>> sorted(park(2))
    ['%%', '%.', '.%', '..', '<>']
    >>> sorted(park(3))
    ['%%%', '%%.', '%.%', '%..', '%<>', '.%%', '.%.', '..%', '...', '.<>', '<>%', '<>.']
    >>> len(list(park(4)))
    29
    """
    if n == 0:
        yield ''
    elif n > 0:
        for s in park(n-1):
            ---------
               (c)
            ---------
               (d)

        for s in _________:
                    (e)
            ---------
               (f)
```

i. (3.0 pt) Fill in blanks (c) and (d) with two lines of code. You may **not** use `yield from`.

```
yield s + '.' yield s + '%'
```

ii. (1.0 pt) Fill in blank (e).

```
park(n-2)
```

iii. (2.0 pt) Fill in blank (f).

```
yield s + '<>'
```

(c) This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Implement `pack`, a generator function that takes a non-negative integer `n`. It yields (in any order) all possible strings representing `n` adjacent parking spots in which an equal number of cars and motorcycles are parked, no motorcycle is adjacent to a car, and no empty spot is adjacent to another empty spot.

```
def pack(n):
    """Yield the ways to park an equal number of cars and motorcycles
    in n adjacent spots with no motorcycle adjacent to a car and no
    empty spot adjacent to another empty spot.

    >>> sorted(pack(4))
    ['%.<>', '<>.%']
    >>> sorted(pack(7))
    ['%%.<><>', '<><>.%%']
    >>> sorted(pack(8))
    ['%%.<>.<>', '%%.<><>.', '%.%.<><>', '%.<><>.%', '.%%.<><>',
     '.<><>.%%', '<>.%%.<>', '<>.<>.%%', '<><>.%%.', '<><>.%.%']
    """
    def f(n, k):
        if n == 0 and k == 0:
            yield ''
        elif n > 0:
            yield from g(n-1, k-1, '<', '%' )
            yield from g(n-1, k,   '.', '.' )
            yield from g(n-2, k+1, '%', '<>')

    def g(n, k, no, yes):

        yield from _________

    yield from f(n, 0)
```

i. Fill in the blank.

```
[yes + s for s in f(n, k) if not s or s[0] != no]
```
(Omitting the surrounding `[]` is ok and in fact more efficient.)

No more questions.

CS 61A Spring 2024

Structure and Interpretation of Computer Programs

Midterm 1 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (8.0 points) What Would Python Display?

Assume the following code has been executed already.

```
one = 1

def choose(one):
    if big(one):
        print('A')
        if huge(one):
            print('B')
    elif big(one) or huge(one):
        print('C')
    if big(one) or print('D'):
        print('E')
    else:
        print('F')

big = lambda x: x >= one
huge = lambda x: x > one

def which():
    one = 3
    def this():
        return one
        return one + 1
    return this
    one = 4
```

(a) (6.0 pt) Which lines are displayed by the interactive Python interpreter after evaluating `choose(one + one)`? **Select all that apply.**

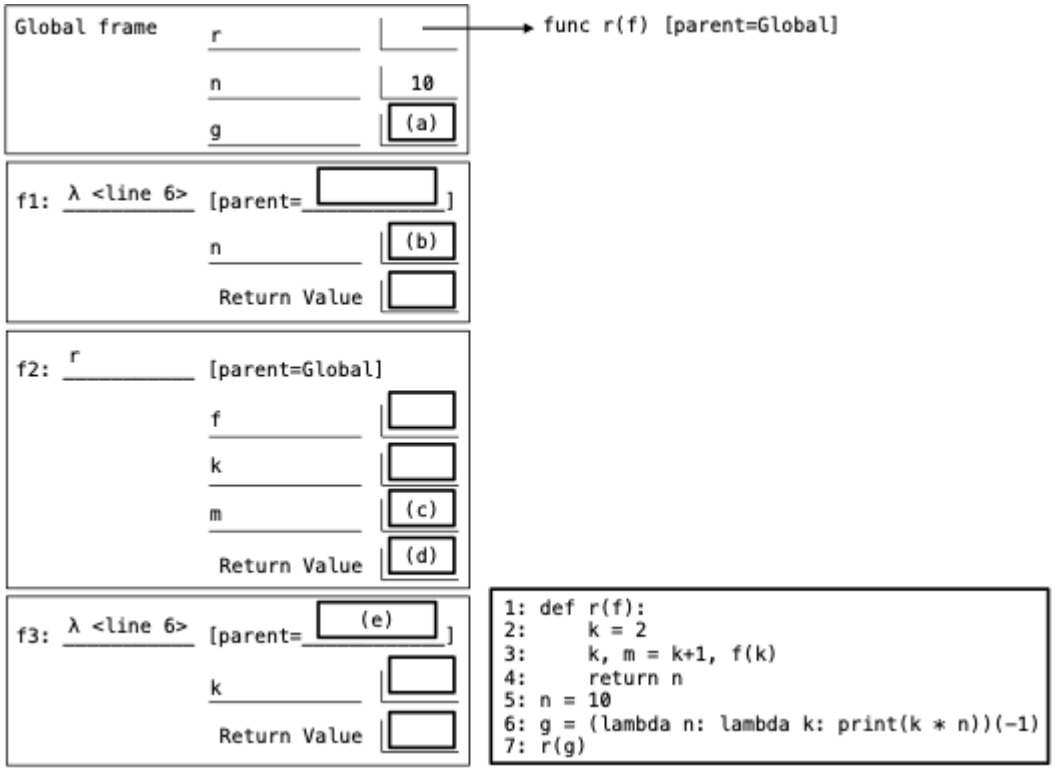
- [x] `A`
- [x] `B`
- [ ] `C`
- [ ] `D`
- [x] `E`
- [ ] `None`
- [ ] None of the above

(b) (2.0 pt) What is displayed by the interactive Python interpreter after evaluating `which()()`?

- ○ 2
- ● 3
- ○ 4
- ○ 5
- ○ A function
- ○ An error occurs before anything is displayed

2. (8.0 points) Silence of the Lambda

Complete the environment diagram below and then answer the questions that follow. There is one question for each labeled blank in the diagram. The blanks with no labels have no questions associated with them and are not scored. If a blank contains an arrow to a function, write the function as it would appear in the diagram.



(a) (2.0 pt) Fill in blank (a). The line number has been omitted for simplicity.

- ○ -1
- ○ 1
- ○ -10
- ○ `func lambda(n) [parent = Global]`
- ○ `func lambda(k) [parent = Global]`
- ○ `func lambda(n) [parent = f1]`
- ● `func lambda(k) [parent = f1]`

(b) (1.0 pt) Fill in blank (b).

- ● -1
- ○ 2
- ○ 3
- ○ 10

(c) **(1.0 pt)** Fill in blank (c).

- ○ `-3`
- ○ `-2`
- ○ `20`
- ○ `30`
- ● `None`

(d) **(1.0 pt)** Fill in blank (d).

- ○ `-1`
- ○ `2`
- ○ `3`
- ● `10`

(e) **(1.0 pt)** Fill in blank (e).

- ○ `Global`
- ● `f1`
- ○ `f2`
- ○ `f3`

(f) **(2.0 pt)** What is printed by the call to `print` on line 6?

- ○ `-3`
- ● `-2`
- ○ `20`
- ○ `30`
- ○ `None`

3. (8.0 points) Nearly Square

Implement `near_square`, which takes positive integer `n` and non-negative integer `k`. It returns the largest integer less than or equal to `n` which is the product of two positive integers that differ by `k` or less. You may use `solve`, which is provided.

```
def near_square(n, k):
    """Return the largest integer that is less than or equal to n and
    equals a * b for some positive integers a and b where abs(a - b) <= k.

    >>> near_square(125, 0)  # 11 * 11 = 121 and abs(11 - 11) = 0
    121
    >>> near_square(120, 3)  # 10 * 12 = 120 and abs(10 - 12) = 2
    120
    >>> near_square(120, 1)  # 10 * 11 = 110 and abs(10 - 11) = 1
    110
    """
    while True:

        gap = k

        while _______:
                (a)
            x = _______
                  (b)
            if _______:  # Check if x is a whole number
                 (c)

                return _______
                         (d)

            _______
              (e)

        _______
          (f)

def solve(b, c):
    """Returns the largest x for which x * (x + b) = c

    >>> solve(2, 120)  # x=10 solves x * (x + 2) = 120
    10.0
    >>> solve(2, 121)  # x=10.045... solves x * (x + 2) = 121
    10.045361017187261
    """
    return (b*b/4 + c) ** 0.5 - b/2
```

(a) (2.0 pt) Fill in blank (a). Select **all** that apply.

- ☐ `gap`
- ☐ `gap != 0`
- ☐ `gap > 0`
- ■ `gap >= 0`

(b) (2.0 pt) Fill in blank (b).

```
solve(gap, n)
```

(c) (1.0 pt) Fill in blank (c). The `round` function is demonstrated on the Midterm 1 Study Guide (page 2 righthand side).

- ◯ `round(x)`
- ◯ `x % 10`
- ● `x == round(x)`
- ◯ `x == x % 10`

(d) (1.0 pt) Fill in blank (d).

- ● `n`
- ◯ `k`
- ◯ `gap`
- ◯ `True`

(e) (1.0 pt) Fill in blank (e).

- ◯ `n = n - 1`
- ◯ `n = k - 1`
- ◯ `n = n - k`
- ● `gap = gap - 1`
- ◯ `gap = k - 1`
- ◯ `k = k - 1`

(f) (1.0 pt) Fill in blank (f).

- ● `n = n - 1`
- ◯ `n = k - 1`
- ◯ `n = n - k`
- ◯ `gap = gap - 1`
- ◯ `gap = k - 1`
- ◯ `k = k - 1`

4. (16.0 points) Nice Dice

A **dice integer** is a positive integer whose digits are all greater than or equal to 1 and all less than or equal to 6.

(a) (8.0 points)

Implement `streak`, which takes a positive integer `n`. It returns `True` if `n` is a **dice integer** whose digits are all the same and `False` otherwise.

```
def streak(n):
    """Return whether positive n is a dice integer in which all the digits are the same.

    >>> streak(22222)
    True
    >>> streak(4)
    True
    >>> streak(22322)  # 2 and 3 are different digits.
    False
    >>> streak(99999)  # 9 is not allowed in a dice integer.
    False
    """
    d = n % 10

    if _______:
         (a)

        return False

    while n:

        if _______:
             (b)

            _______
              (c)

        n = n // 10

    return _______
             (d)
```

i. (2.0 pt) Fill in blank (a).

- ◯ `d < 1 and d > 6`
- ● `d < 1 or d > 6`
- ◯ `n < 1 and n > 6`
- ◯ `n < 1 or n > 6`
- ◯ `min(n) < 1 and max(n) > 6`
- ◯ `min(n) < 1 or max(n) > 6`

The loop checks the remaining digits.

ii. **(2.0 pt)** Fill in blank (b).

```
d != n % 10
```

iii. **(1.0 pt)** Fill in blank (c).

- ◯ `d = n % 10`
- ◯ `d = (n // 10) % 10`
- ◯ `result = False`
- ● `return False`

iv. **(1.0 pt)** Fill in blank (d).

- ● `True`
- ◯ `result`
- ◯ `d == n`
- ◯ `d == n % 10`

v. (2.0 pt) Fill in the blank of `changes`, which takes a dice number `n`. It returns the number of digits of `n` that are different from the digit to their left.

```
def changes(n):
    """Return the number of adjacent digits that are different in dice number n.

    >>> changes(22222)
    0
    >>> changes(222255)
    1
    >>> changes(22322)
    2
    >>> changes(22366666622)
    3
    >>> changes(52431)
    4
    """
    count = 0
    while n >= 10:

        if _______:

            count = count + 1
        n = n // 10
    return count
```

- ○ `n != n % 100`
- ○ `n != n // 10`
- ○ `n != (n // 10) % 10`
- ○ `n != (n // 10) % 100`
- ○ `n % 10 != n % 100`
- ○ `n % 10 != n // 10`
- ● `n % 10 != (n // 10) % 10`
- ○ `n % 10 != (n // 10) % 100`

(b) (2.0 pt) Fill in the blank of `count_if`, which takes a one-argument function `f`. It returns a function that takes a positive integer `n` and returns the count of its digits for which `f` returns a true value.

```
def count_if(f):
    """Return a function that takes a positive integer n and returns
    the count of its digits for which f returns a true value.

    >>> is_three = lambda d: d == 3
    >>> count_threes = count_if(is_three)  # count_threes returns the number of threes in n
    >>> count_threes(431663334231)         # 3 appears 5 times
    5
    """
    def g(n):
        count = 0
        while n:
            if _______:
                count += 1
            n = n // 10
        return count
    return g
```

```
f(n % 10)
```

(c) (2.0 pt) Fill in the blank of `count_at_least`, which takes a positive integer `k`. It returns a function that takes a positive integer `n` and returns the count of its digits that are greater than or equal to `k`.

```
def count_at_least(k):
    """Return a function that returns how many of the digits of its argument are at least k.

    >>> above_3 = count_at_least(3)  # above_3 returns the # of digits greater than or equal to 3
    >>> above_3(431663334231)
    9
    """
    return count_if(_______)
```

- ○ `lambda: n % 10 >= k`
- ○ `lambda: n >= k`
- ○ `lambda: d >= k`
- ○ `lambda: d >= n % 10`
- ○ `lambda d: n % 10 >= k`
- ○ `lambda d: n >= k`
- ● `lambda d: d >= k`
- ○ `lambda d: d >= n % 10`
- ○ `lambda d: lambda k: n % 10 >= k`
- ○ `lambda d: lambda k: n >= k`
- ○ `lambda d: lambda k: d >= k`
- ○ `lambda d: lambda k: d >= n % 10`

(d) (4.0 pt) Fill in the blank of `times`, which takes a non-negative integer `k`, a positive integer `n`, and a digit `d`. It returns `True` if `n` contains digit `d` exactly `k` times and `False` otherwise. You may use `count_if`.

```
def times(k, n, d):
    """Returns whether n contains digit d exactly k times.

    >>> times(3, 6132344561, 4)   # 4 only appears 2 times
    False
    >>> times(3, 61323445614, 4)  # 4 appears exactly 3 times
    True
    """
    return _______
```

```
k == count_if(lambda i: i == d)(n)
```

(e) (0.0 pt) This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Fill in the blank to implement `streak` using `count_if`. You may not write `else` or `*` or `**`.

The expression is long, so you may use multiple lines to write it.

```
def streak(n):
    """Return whether positive n is a dice integer in which all the digits are the same.

    >>> streak(22222)
    True
    >>> streak(4)
    True
    >>> streak(22322)  # 2 and 3 are different digits.
    False
    >>> streak(99999)  # 9 is not allowed in a dice integer.
    False
    """
    return ______
```

```
count_if(lambda d: True)(n) == count_if(lambda d: d > 0 and d < 7 and d
== n % 10)(n)
```

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Spring 2024

Midterm 2 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (7.0 points) What Would Python Display?

Assume the following code has been executed. The `Link` class appears on the midterm 2 study guide (page 2, left side).

```
def shake(it):
    if it is not Link.empty and it.rest is not Link.empty:
        if it.first + 1 < it.rest.first:
            it.rest = Link(it.rest.first-1, it.rest)
            shake(it)
        else:
            shake(it.rest)
it = Link(2, Link(5, Link(7)))
off = Link(1, it.rest)
shake(it)

def cruel(summer):
    while summer is not Link.empty:
        yield summer.first
        summer = summer.rest
        if summer is not Link.empty:
            summer = summer.rest
summer = Link(1, Link(2, Link(3, Link(4))))
```

Write the output printed for each expression below or *Error* if an error occurs.

(a) (2.0 pt) `print(it)`

- ○ `<2 5 7>`
- ○ `<2 4 5 7>`
- ○ `<2 4 5 6 7>`
- ○ `<2 3 4 5 7>`
- ○ `<2 4 3 5 7>`
- ● `<2 3 4 5 6 7>`
- ○ `<2 4 3 5 6 7>`

(b) (2.0 pt) `print(off)`

`<1 5 6 7>`

(c) (2.0 pt) `print([x*x for x in cruel(summer)])`

`[1, 9]`

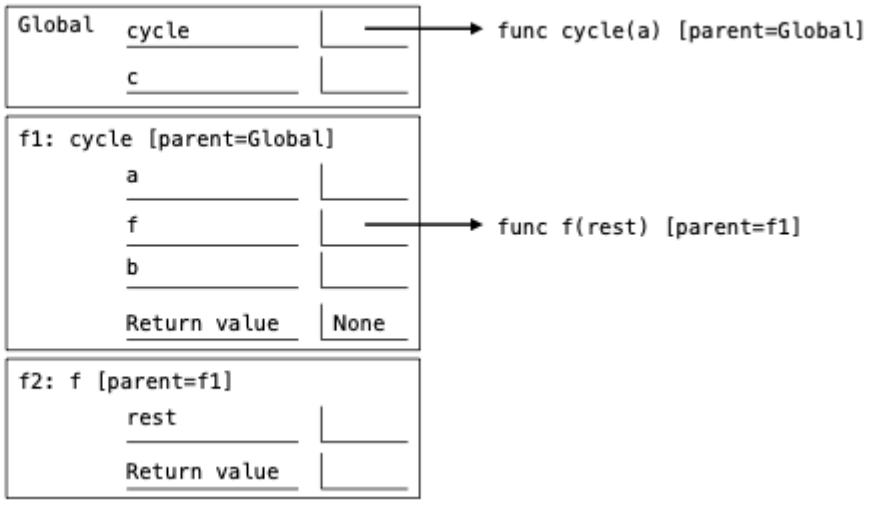
(d) (1.0 pt) What is the order of growth of the time it takes to evaluate `shake(Link(1, Link(n)))` in terms of `n`?

- ◯ exponential
- ◯ quadratic
- ● linear
- ◯ constant

2. (6.0 points) Spin Cycle

Complete the environment diagram below and then answer the questions that follow. Do not add frames for calls to built-in functions (such as `print`).

```
1: def cycle(a):
2:     def f(rest):
3:         rest.append(a.pop())
4:         return [rest.append(a)]
5:     b = []
6:     a = f(b)
7:     b = b + [5]
8:     print(a)
9:     print(b)
10:
11: c = [2, 3, 4]
12: cycle(c)
13: print(c)
```



(a) (2.0 pt) What would be printed by the expression `print(a)` on line 8?

`[None]` **https://i.postimg.cc/Zn1D5MZM/cycle-sol.png**

(b) (2.0 pt) What would be printed by the expression `print(b)` on line 9?

```
[4, [2, 3], 5]
```

(c) (2.0 pt) What would be printed by the expression `print(c)` on line 13?

```
[2, 3]
```

3. (8.0 points) Fearless

If you sing lyrics into a mic, every connected speaker repeats them.

A `Mic` instance has a dictionary `speakers` containing `Speaker` instances as values, each with its location (`str`) as its key. Its `sing` method takes a string `lyrics` and invokes the `repeat` method of each `Speaker` instance connected to it.

A `Speaker` takes a `transform` function that takes and returns a string. To `connect` a `Speaker` instance to `m` (`Mic`) in a `location` (`str`), add that instance to the `speakers` dictionary of `m` in that `location`. To `repeat` a signal `s` (`str`), return the result of calling the speaker's `transform` function on `s`.

Every `Mic` starts connected to a `Speaker` in the `Front` location that repeats the exact same signal it receives.

Implement the `Mic` and `Speaker` classes to match the doctests. The `str.lower` and `str.upper` functions return lowercased and uppercased versions of a string, respectively.

```
class Mic:
    """A microphone connected to speakers.

    >>> m = Mic()  # Front is connected automatically
    >>> m.sing('Is this thing on?')
    Front - Is this thing on?
    >>> Speaker(str.lower).connect(m, 'Left Side')
    >>> Speaker(str.upper).connect(m, 'Right Side')
    >>> m.sing("You belong with me.")
    Front - You belong with me.
    Left Side - you belong with me.
    Right Side - YOU BELONG WITH ME.
    """
    def __init__(self):
        self.speakers = _______
                          (a)

    def sing(self, lyrics):
        for k in self.speakers.keys():  # iterate over the keys of a dictionary
            print(k, '-', _______.repeat(lyrics))
                            (b)

class Speaker:
    def __init__(self, transform):
        self.transform = transform

    def connect(self, m, location):
        _______
          (c)

    def repeat(self, s):
        return _______
                 (d)
```

(a) (3.0 pt)

```
{'Front': Speaker(lambda x: x)}
```

(b) (2.0 pt)

- ○ `self`
- ○ `self.speaker`
- ○ `self.speakers`
- ● `self.speakers[k]`
- ○ `self.speakers[k].self`
- ○ `speaker`
- ○ `speakers`
- ○ `speakers[k]`
- ○ `speakers[k].self`

(c) (2.0 pt)

```
m.speakers[location] = self
```

(d) (1.0 pt)

- ○ `s`
- ○ `lyrics`
- ○ `transform`
- ○ `transform(s)`
- ○ `transform(lyrics)`
- ○ `self.transform`
- ● `self.transform(s)`
- ○ `self.transform(lyrics)`

4. (29.0 points) Who's counting?

Definition. A *strip* is a list of integers in which each integer is one more than the last. For example, `[3, 4, 5, 6]` is a strip. Empty and one-element lists are strips.

(a) (4.0 points)

Implement `is_strip`, which takes a list of integers `s` and returns whether it is a strip.

```
def is_strip(s):
    """Return whether list s is a strip.

    >>> is_strip([3, 4, 5, 6])
    True
    >>> is_strip([3, 3, 3])        # 3 after 3
    False
    >>> is_strip([3, 4, 5, 4, 6])  # 4 after 5
    False
    >>> is_strip([3, 4, 5, 5, 6])  # 5 after 5
    False
    >>> is_strip([3, 4, 5, 6, 8])  # 8 after 6
    False
    >>> is_strip([5])
    True
    >>> is_strip([])
    True
    """
    assert type(s) == list
    return _______ or s == list ( range ( _______ , _______ ))
             (a)                              (b)       (c)
```

i. (1.0 pt) Fill in blank (a).

- (●) `len(s) == 0`
- ( ) `s[0] + 1 == s[1]`
- ( ) `s[0] + 1 in s`
- ( ) `s[0] + 1 in s[1:]`

ii. (1.0 pt) Fill in blank (b).

`s[0]`

iii. (2.0 pt) Fill in blank (c).

- ( ) `s`
- ( ) `s[0]`
- ( ) `s[1]`
- ( ) `s[0] + 1`
- (●) `s[0] + len(s)`
- ( ) `s[len(s) - 1]`
- ( ) `s[1] - s[0]`

(b) (7.0 points)

Implement `drip`, which takes two non-empty lists of integers `s` and `t`. It returns `True` if there is a strip containing all and only the elements of `s` and `t` **starting with** `s[0]` in which the elements of `s` appear in order and the elements of `t` appear in order. It returns `False` otherwise.

```
def drip(s, t):
    """Return whether there is a strip made out of interleaving s and t.

    >>> drip([1, 3, 5], [2, 4, 6])    # 1 2 3 4 5 6
    True
    >>> drip([1, 4, 5], [2, 3, 6])    # 1 2 3 4 5 6
    True
    >>> drip([1, 2, 3], [4, 5, 6])    # 1 2 3 4 5 6
    True
    >>> drip([2, 4, 5], [1, 3, 6])    # No strip starting with 2 can contain 1
    False
    >>> drip([1, 2, 4, 5], [1, 3, 6]) # No strip can contain 1 and 1
    False
    >>> drip([1, 4, 5], [2, 3, 7])    # No strip can contain 5 and 7 but no 6
    False
    >>> drip([1, 5, 4], [2, 3, 6])    # No strip can contain 5 before 4
    False
    >>> drip([2], [3, 4, 5])          # 2 3 4 5
    True
    >>> drip([1], [2, 3, 5])          # No strip can contain 3 and 5 but no 4
    False
    """
    while s and t:
        if s[0] + 1 == t[0]:
            s, t = _______  # The next element of the strip is in t
                     (a)
        elif _______:
               (b)
            s = s[1:]       # The next element of the strip is in s
        else:
            return _______
                     (c)
    return _______
             (d)
```

i. (1.0 pt) Fill in blank (a).

- ○ `t, s`
- ● `t, s[1:]`
- ○ `t[1:], s`

ii. (3.0 pt) Fill in blank (b).

```
len(s) > 1 and s[0] + 1 == s[1]
```

iii. (1.0 pt) Fill in blank (c).

- (●) `False`
- ( ) `s[0] != t[0]`
- ( ) `is_strip(s) or is_strip(t)`
- ( ) `not (is_strip(s) or is_strip(t))`
- ( ) `is_strip(s) and is_strip(t)`
- ( ) `not (is_strip(s) and is_strip(t))`

iv. (2.0 pt) Fill in blank (d).

- ( ) `True`
- ( ) `is_strip(s)`
- ( ) `is_strip(t)`
- ( ) `is_strip(s) or is_strip(t)`
- ( ) `not (is_strip(s) or is_strip(t))`
- (●) `is_strip(s) and is_strip(t)`
- ( ) `not (is_strip(s) and is_strip(t))`

(c) (6.0 points)

Implement `longest`, which takes a list of integers `s`. It returns the longest strip whose elements appear in s in order. If two such strips exist, return the one that starts earlier in `s`. **Hint:** `s[-1]` is the last element in list `s`.

```
def longest(s):
    """Return the longest strip whose elements appear in s in order.
    >>> longest([4, 2, 3, 5, 6, 4, 6, 5])  # 2 3 4 5 is the longest strip in s
    [2, 3, 4, 5]
    >>> longest([4, 2, 3, 5, 6, 4])  # 4 5 6 is as long as 2 3 4 and is earlier
    [4, 5, 6]
    >>> longest([2, 4, 6])
    [2]
    """
    if len(s) == 0:
        return []
    return max([longest_with_s0(s), _______ ], key= _______ )
                                       (a)             (b)
def longest_with_s0(s):
    """Return the longest strip in s that starts with s[0]."""
    result = _______
               (c)
    for k in s[1:]:
        if _______:
             (d)
            result.append(k)
    return result
```

i. (2.0 pt) Fill in blank (a).

```
longest(s[1:])
```

ii. (1.0 pt) Fill in blank (b).

- (●) `len`
- ( ) `len(s)`
- ( ) `s[0]`
- ( ) `lambda i: s[i]`

iii. (1.0 pt) Fill in blank (c).

- ( ) `[]`
- ( ) `[s]`
- ( ) `s[0]`
- (●) `s[:1]`

iv. (2.0 pt) Fill in blank (d).

```
k == result[-1] + 1
```

(d) (5.0 points)

Implement `has_strip`, which takes a `Tree` of integers `t`. It returns whether there is a strip containing the labels along some path from the root to a leaf of `t`. The `Tree` class is on the midterm 2 study guide (page 2 left side).

```
def has_strip(t):
    """Return whether the elements of some root-to-leaf path form a strip.

    >>> has_strip(Tree(1, [Tree(3, [Tree(4)]), Tree(2, [Tree(2), Tree(3)])])) # 1, 2, 3
    True
    >>> has_strip(Tree(1, [Tree(3, [Tree(4)]), Tree(2, [Tree(2, [Tree(3)])])]))
    False
    >>> has_strip(Tree(1, [Tree(3, [Tree(4)]), Tree(2, [Tree(2)])]))
    False
    >>> has_strip(Tree(1, [Tree(2, [Tree(4)]), Tree(3, [Tree(3)])]))
    False
    """
    if t.is_leaf():
        return _______
                 (a)
    for b in t.branches:
        if _______
             (b)
            return True
    return _______
             (c)
```

i. (1.0 pt) Fill in blank (a).

- (●) `True`
- ( ) `False`
- ( ) `t`
- ( ) `t.label`
- ( ) `t.label == t + 1`
- ( ) `t + 1 == t.label`

ii. (3.0 pt) Fill in blank (b).

```
b.label == t.label + 1 and has_strip(b)
```

iii. (1.0 pt) Fill in blank (c).

- ( ) `True`
- (●) `False`
- ( ) `t`
- ( ) `t.label`
- ( ) `t.label == t + 1`
- ( ) `t + 1 == t.label`

(e) (7.0 points)

Implement `strips`, a generator function that takes a `Tree` of integers `t`. It yields all strips that contain the labels along a path from the root to a leaf of `t`.

```
def strips(t):
    """Yield the paths from the root to a leaf of Tree t that form strips.

    >>> list(strips(Tree(1, [Tree(3, [Tree(4)]), Tree(2, [Tree(2), Tree(3)])])))
    [[1, 2, 3]]
    >>> list(strips(Tree(1, [Tree(2), Tree(2, [Tree(2), Tree(3, [Tree(4)]), Tree(3)])])))
    [[1, 2], [1, 2, 3, 4], [1, 2, 3]]
    """
    if t.is_leaf():
        yield _______
                (a)

    for b in t.branches:
        if _______:
             (b)

            for s in _______:
                       (c)

                yield _______
                        (d)
```

i. (2.0 pt) Fill in blank (a).

- ◯ `None`
- ◯ `t`
- ◯ `t.label`
- ◯ `[t]`
- ● `[t.label]`

ii. (1.0 pt) Fill in blank (b).

- ◯ `t.label == b.label + 1`
- ● `b.label == t.label + 1`
- ◯ `t.label == b.label + 1 and is_strip(b)`
- ◯ `b.label == t.label + 1 and is_strip(b)`

iii. (2.0 pt) Fill in blank (c).

- ◯ `strips(t)`
- ● `strips(b)`
- ◯ `t.branches`
- ◯ `b.branches`
- ◯ `range(len(t.branches))`
- ◯ `range(len(b.branches))`

iv. (2.0 pt) Fill in blank (d).

- ○ `s`
- ○ `t`
- ○ `t + s`
- ○ `t.label + s`
- ○ `[t] + s`
- ● `[t.label] + s`
- ○ `[t] + [s]`
- ○ `[t.label] + [s]`

v. (0.0 pt) This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Fill in the blank of `only_strips`, which takes a Tree of integers `t`. It removes all nodes that are **not** on a *strip path*. A *strip path* is a path from the root to a leaf whose labels form a strip. You may use functions implemented earlier in this question.

```
def test_only_strips(t):
    """Call only_strips(t) and then return t. Assume has_strip(t) is True.

    >>> test_only_strips(Tree(1, [Tree(2), Tree(2, [Tree(2), Tree(3, [Tree(4)]), Tree(3)])]))
    Tree(1, [Tree(2), Tree(2, [Tree(3, [Tree(4)]), Tree(3)])])
    >>> test_only_strips(Tree(5, [Tree(6), Tree(6, [Tree(7, [Tree(9)])]), Tree(7)]))
    Tree(5, [Tree(6)])
    """
    only_strips(t)
    return t

def only_strips(t):
    """Remove all nodes of Tree t that are not on a path from the root to a leaf
    whose labels form a strip. Assume has_strip(t) is True.
    """
    t.branches = [b for b in t.branches if _______]
```

```
has_strip(b) and (only_strips(b) or t.label + 1 == b.label)
```

No more questions.

CS 61A Spring 2025

Structure and Interpretation of Computer Programs

Midterm 1 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (6.0 points) What Would Python Print?

Answer the questions about this code. No errors occur while it is executed.

```
def double(x):
    return 2 * x

def square(f):
    return lambda x: f(x) * f(x)

def inc(f):
    return lambda x: f(x + 1)

def triple(f):
    return lambda x: f(f(f(x)))

def put(x):
    return lambda f: f(x)

one = put(1)
triple(print)(5)
```

(a) (2.0 pt) What is displayed by the last line?

5
None
None

(b) (2.0 pt) What would be displayed by evaluating `print(one(square(double)))`

4

(c) (2.0 pt) What would be displayed by evaluating `print(inc(put)(1)(triple(double)))`

16

(d) (0.0 pt) This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Write one line of code just after `triple(print)(5)` that would double and add one to all numbers in the answers to parts (b) and (c) above. For example, for an answer containing 100, it would now contain 201 instead.

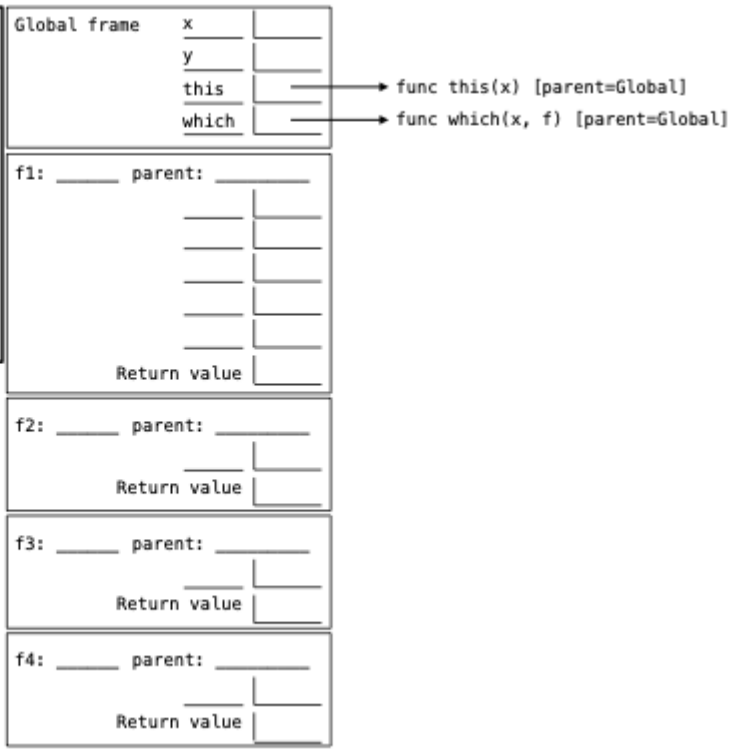
You **may not** use any numbers or arithmetic symbols (such as `+` or `*`) or arithmetic functions (such as `add` or `mul`). You **may not** reassign `triple` or `square`. You may use multiple assignment: `x, y = ___, ___`

Four possible answers:
`print = (lambda f: lambda x: f(double(x)))(inc(print))`
`print = (lambda f: lambda x: inc(f)(double(x)))(print)`
`f, print = inc(print), lambda x: f(double(x))`
`f, print = print, lambda x: inc(f)(double(x))`

2. (8.0 points) Which One

Complete the environment diagram below to answer the questons about the calls to `print`. Only the questions will be scored, not the diagram. Each call to `print` is evaluated once, and there are no errors caused by running this code.

```
 1: x, y = 1, 2
 2:
 3: def this(x):
 4:     x = 3
 5:     print(y)
 6:     return 4
 7:
 8: def which(x, f):
 9:     def this(x):
10:         x = 5
11:         return y
12:     y = 6
13:     y = f(x)
14:     print(this(y))
15:     return lambda: (print(x) or y)
16:
17: print(which(7, this)())
```



(a) (2.0 pt) What is displayed by the call to `print` on line 5?

2

(b) (2.0 pt) What is displayed by the call to `print` on line 14?

4

(c) (2.0 pt) What is displayed by the call to `print` on line 15?

7

(d) (2.0 pt) What is displayed by the call to `print` on line 17?

4

3. (13.0 points) Legit Digit

(a) (4.0 points)

Implement `all_digits`, which takes a positive integer `n` and one-argument function `cond` that always returns `True` or `False`. It returns `True` if `cond(d)` returns `True` when called on every digit `d` in `n`, and `False` otherwise.

```
def all_digits(n, cond):
    """Return whether cond returns true for every digit of positive n.

    >> odd = lambda d: d % 2 == 1
    >>> all_digits(123, odd)  # not all digits are odd
    False
    >>> all_digits(357, odd)  # all digits are odd
    True
    """

    while n:

        if _______:
             (a)

            _______
              (b)

        n = n // 10

    return _______
             (c)
```

i. (2.0 pt) Fill in blank (a).

```
not cond(n % 10)
```

ii. (1.0 pt) Fill in blank (b).

- ◯ `n = n % 10`
- ◯ `n = n * 10`
- ◯ `return True`
- ● `return False`
- ◯ `return odd`
- ◯ `return cond`

iii. (1.0 pt) Fill in blank (c).

- ◯ `n`
- ● `True`
- ◯ `False`
- ◯ `odd(n)`
- ◯ `cond(n)`

(b) (3.0 points)

Definition. A *prefix* of a positive integer `n` is the value of `n//pow(10, p)` for some non-negative integer `p`. For example, 3456 has prefixes 3456, 345, 34, 3, and 0.

Implement `prefix_digits`, which takes a positive integer `n` and a one-argument function `cond` that always returns `True` or `False`. It returns the largest prefix of `n` for which `cond` returns `True` when called on every digit of the prefix. You may call `all_digits` and `process`.

```
def process(n, check):
    """A function to help implement prefix_digits."""
    while n:
        if check(n):
            return n
        n = n // 10
    return 0

def prefix_digits(n, cond):
    """Return the largest prefix of positive n for which cond returns true for every digit.

    >>> odd = lambda d: d % 2 == 1
    >>> prefix_digits(94720, odd)
    9
    >>> prefix_digits(919321, odd)
    9193
    >>> prefix_digits(2025, odd)
    0
    >>> prefix_digits(20252025, lambda d: d < 4)
    202
    >>> prefix_digits(20252025, lambda d: True)
    20252025
    """
    return process(n, lambda k: _______ )
                                  (d)
```

i. (3.0 pt) Fill in blank (d).

- ○ `all_digits(n, cond)`
- ● `all_digits(k, cond)`
- ○ `all_digits(n, cond(n))`
- ○ `all_digits(k, cond(k))`
- ○ `lambda cond: all_digits(n, cond)`
- ○ `lambda cond: all_digits(k, cond)`
- ○ `lambda cond: all_digits(n, cond(n))`
- ○ `lambda cond: all_digits(k, cond(k))`
- ○ `all_digits(n, lambda n: cond)`
- ○ `all_digits(k, lambda n: cond)`
- ○ `all_digits(n, cond(lambda n: n))`
- ○ `all_digits(k, cond(lambda n: k))`

(c) (6.0 points)

Re-implement `prefix_digits`, which takes a positive integer `n` and a one-argument function `cond` that always returns `True` or `False`. It returns the largest prefix of `n` for which `cond` returns `True` when called on every digit of the prefix. You **may not** call `all_digits` or `process`.

```
def prefix_digits(n, cond):
    """Return the largest prefix of positive n for which cond returns true for every digit.

    >>> odd = lambda d: d % 2 == 1
    >>> prefix_digits(94720, odd)
    9
    >>> prefix_digits(919321, odd)
    9193
    >>> prefix_digits(2025, odd)
    0
    >>> prefix_digits(20252025, lambda d: d < 4)
    202
    """
    k = 0
    while n >= ________:
                   (e)
        if cond(_______):
                   (f)
            k += 1
        else:
            n = n // 10
            -------
              (g)
    return n
```

i. (2.0 pt) Fill in blank (e).

- ○ `0`
- ○ `k`
- ○ `k // 10`
- ● `pow(10, k)`

ii. (2.0 pt) Fill in blank (f).

- ○ `n % 10`
- ○ `n // 10 % 10`
- ○ `n % pow(10, k)`
- ○ `n // pow(10, k)`
- ○ `n // 10 % pow(10, k)`
- ● `n // pow(10, k) % 10`

iii. (2.0 pt) Fill in blank (g). You may not call `all_digits` or `process`.

k = 0

4. (4.0 points) Hailstone Returns

Definition. A *hailstone sequence* is formed by picking a positive integer `n` as the start then repeatedly updating `n` until it is 1 using this rule: if `n` is even, divide it by 2; if `n` is odd, multiply it by 3 and add 1.

Implement `hailstone`, which prints each update in a hailstone sequence by displaying: the update number (counting up from 1), the current `n`, an `->` arrow symbol, and the updated `n`.

This implementation may not be possible using the template. If that's the case, respond *Impossible* to the following questions.

```
def hailstone(n):
    """Print numbered updates in the hailstone sequence.

    >>> hailstone(10)
    1 10 -> 5
    2 5 -> 16
    3 16 -> 8
    4 8 -> 4
    5 4 -> 2
    6 2 -> 1
    """
    def f():
        if n % 2 == 1:
            m = 3 * n + 1
        else:
            m = n // 2
        -------
          (a)
        -------
          (b)
    k = 1
    while n > 1:
        k, n = k + 1, f()
```

(a) (2.0 pt) Fill in blank (a).

- ○ `print(k, n, ->, m)`
- ● `print(k, n, '->', m)`
- ○ `print(k - 1, n, ->, m)`
- ○ `print(k - 1, n, '->', m)`
- ○ *Impossible*

(b) (2.0 pt) Fill in blank (b). If the problem is not possible using the template, write *Impossible*.

```
return m
```

5. (4.0 points) It Takes Two

Implement `at_least_two`, which takes three arguments. It returns `True` if at least two of them are true values and `False` otherwise. The built-in `bool` function takes one argument and returns `True` if it is a true value and `False` if it is a false value.

```
def at_least_two(x, y, z):
    """Returns whether at least two of the arguments are true values.
    >>> at_least_two(1 + 1, 3 + 3, 5 + 5)
    True
    >>> at_least_two(1 + 1, 3 - 3, 5 + 5)
    True
    >>> at_least_two(1 - 1, 3 + 3, 5 + 5)
    True
    >>> at_least_two(1 - 1, 3 + 3, 0)
    False
    >>> at_least_two(1 - 1, 0, 0)
    False
    """
    if _______:
         (a)
        return _______
    else:         (b)
        return _______
                  (c)
```

(a) (2.0 pt) Fill in blank (a).

- (●) `x`
- ( ) `x == True`
- ( ) `x and y`
- ( ) `x == True and y == True`

(b) (1.0 pt) Fill in blank (b). *Select all correct answers.*

- [ ] `y or z`
- [ ] `y and z`
- [■] `bool(y or z)`
- [ ] `bool(y and z)`
- [ ] `not(y or z)`
- [ ] `not(y and z)`

(c) (1.0 pt) Fill in blank (c). *Select all correct answers.*

- [ ] `y or z`
- [ ] `y and z`
- [ ] `bool(y or z)`
- [■] `bool(y and z)`
- [ ] `not(y or z)`
- [ ] `not(y and z)`

No more questions.

CS 61A Spring 2025

Structure and Interpretation of Computer Programs

Midterm 2 Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (6.0 points) What Would Python Display?

Assume the following code has been executed. The `pop` method of a list removes and returns its last element.

```
s = [2, [5, 8], 11]
t = [1, 3, 5, 7, 9]

def add_t(x):
    if isinstance(x, list):
        return [add_t(y) for y in x]
    else:
        return x + t.pop()

m = map(add_t, s)
```

Write the value of each expression below or *Error* if an error occurs. Assume that each expression is evaluated in order and sequentially, so evaluating the first could affect the value of the second. Write *Map* for a map object and *Function* for a function.

(a) (2.0 pt) `[x[1] for x in s if isinstance(x, list)]`

```
[8]
```

(b) (1.0 pt) `next(m)`

- ○ 3
- ○ 5
- ○ 7
- ○ 9
- ● 11

(c) (1.0 pt) `t.pop()`

- ○ 1
- ○ 3
- ○ 5
- ● 7
- ○ 9

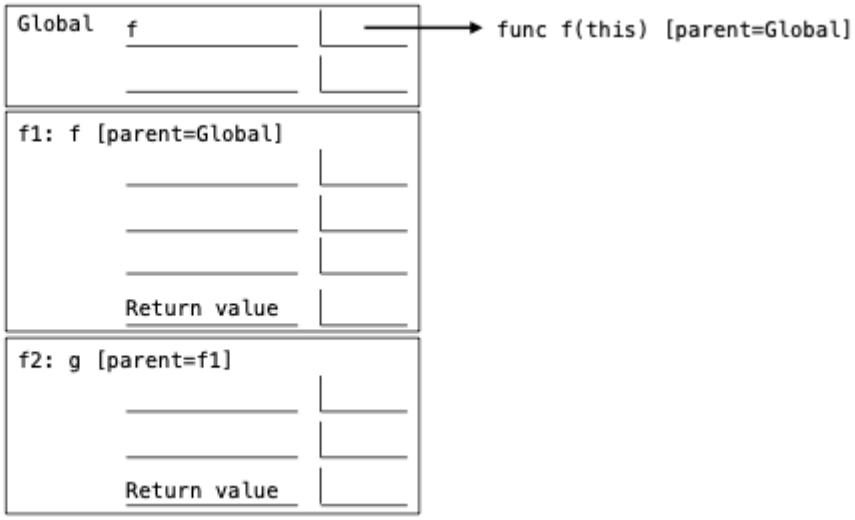
(d) (2.0 pt) `[next(m) for z in range(2)]`

```
[[10, 11], 12]
```

2. (7.0 points) This or That

Answer the questions about the code below. Draw an environment diagram, but the diagram itself will not be scored.

```
 1: def f(this):
 2:     that = this[1]
 3:     def g(copy):
 4:         that[0] = 4
 5:         copy[0] = 5
 6:         copy[1].append(6)
 7:         print(copy)
 8:     g(list(this))
 9:     this.append(that)
10:     that.append(7)
11:
12: t = [1, [2], 3]
13: f(t)
14: print(t)
```



(a) (3.0 pt) What is displayed by the call to `print` on line 7?

```
[5, [4, 6], 3]
```

(b) (3.0 pt) What is displayed by the call to `print` on line 14?

```
[1, [4, 6, 7], 3, [4, 6, 7]]
```

(c) (1.0 pt) What is the order of growth of the time it takes to execute `result = nest(n)` in terms of positive integer `n`? Assume that `append` always takes one step (constant time).

```
def nest(n):
    """Create a nested list.
    >>> nest(3)
    [[[[], []], [[], []]], [[[], []], [[], []]]]
    """
    s = []
    first = s
    for x in range(n):
        next_s = []
        s.append(next_s)
        s.append(next_s)
        s = next_s
    return first
```

- ◯ exponential
- ◯ quadratic
- ● linear
- ◯ logarithmic
- ◯ constant

3. (10.0 points) Berkeley Time

(a) (5.0 points)

An `Event` instance has `start` and `end` attributes that are positive integers representing the number of minutes after midnight that the `Event` begins and concludes.

Implement the `Event` class. The `split` method takes an integer `k` that **evenly divides** the difference between the `end` and `start` times. It is a generator function that yields `k` sequential events of equal length that together span the same time interval as the original event. Calling `split` **does not** change the event.

```
class Event:
    """An Event has a start and end time and can be split into equal-length shorter events.
    >>> calapalooza = Event(980, 1180)
    >>> list(calapalooza.split(4))  # Split it into 4 smaller events that go from 980 to 1180.
    [Event(980, 1030), Event(1030, 1080), Event(1080, 1130), Event(1130, 1180)]
    """
    def __init__(self, start, end):
        self.start, self.end = start, end
    def __repr__(self):
        return f'Event({self.start}, {self.end})'
    def split(self, k):
        assert (self.end - self.start) % k == 0  # k evenly divides the duration of the event.
        length = (self.end - self.start) // k

        start = _______
                  (a)
        for i in range(k):

            yield _______
                    (b)

            _______
              (c)
```

i. (1.0 pt) Fill in blank (a).

- ○ `0`
- ○ `None`
- ○ `self`
- ○ `start`
- ● `self.start`

ii. (3.0 pt) Fill in blank (b).

```
Event(start, start + length)
```

iii. (1.0 pt) Fill in blank (c).

- ● `start = start + length`
- ○ `start = self.start + length`
- ○ `self.start = start + length`
- ○ `self.start = self.start + length`

(b) (5.0 points)

A `BerkeleyEvent` is an `Event` with a modified `split` method. The first `Event` yielded by `split` starts at the same time as the original event, but all events after the first one are 10 minutes shorter and start 10 minutes after the previous event ends. Again, assume `k` evenly divides the difference between the `end` and `start` attributes.

```
class BerkeleyEvent(Event):
    """When a BerkeleyEvent is split, there is time between events.
    >>> list(BerkeleyEvent(840, 930).split(3))
    [Event(840, 870), Event(880, 900), Event(910, 930)]
    """
    time = 10  # How much time between events
    def split(self, k):
        first = True
        for e in _______:
                    (d)
            if first:
                first = False
            else:
                _______.start = _______ + _______
                  (e)             (f)       (g)
            yield e
```

i. (2.0 pt) Fill in blank (d).

`Event.split(self, k)` **or** `super().split(k)`

ii. (1.0 pt) Fill in blank (e).

- (●) `e`
- ( ) `self`
- ( ) `self.e`

iii. (1.0 pt) Fill in blank (f).

- ( ) `self`
- ( ) `self.start`
- ( ) `self.end`
- ( ) `e`
- (●) `e.start`
- ( ) `e.end`

iv. (1.0 pt) Fill in blank (g). **Select all that apply.**

- ☐ `time`
- ■ `self.time`
- ☐ `self.BerkeleyEvent.time`
- ■ `BerkeleyEvent.time`
- ☐ `Event.time`

4. (11.0 points) DeepSeek

Definition. The *depth* of the root node of a tree is 0. The depth of each other node is one plus the depth of its parent.

(a) (5.0 points)

Implement `deepest`, which takes a `Tree` instance `t` and a value `x`. It returns the largest depth of a node labeled `x` in `t`. If there are no nodes labeled `x` in `t`, then `deepest(t, x)` returns `None`.

The `Tree` class appears on the Midterm 2 study guide (page 2, left column).

```
def deepest(t, x):
    """Return the maximum depth of a node labeled x in Tree t.
    Return None if x is not a label in t.

    >>> example = Tree(30, [Tree(20), Tree(20, [Tree(30, [Tree(20)])])])
    >>> deepest(example, 30)
    2
    >>> deepest(example, 20)
    3
    >>> print(deepest(example, 10))
    None
    """
    below = [deepest(b, x) for b in t.branches]
    depths = [ _______ ]
                (a)
    if _______:
         (b)
        depths._______
                (c)
    if depths:
        return max(depths)
    else:
        return None
```

i. (3.0 pt) Fill in blank (a). Include `for` and `in` and `if` in your answer.

```
y+1 for y in below if y is not None
```

ii. (1.0 pt) Fill in blank (b).

- ○ `x in depths`
- ○ `depths`
- ○ `not depths`
- ○ `t == x`
- ● `t.label == x`
- ○ `t.is_leaf()`

iii. (1.0 pt) Fill in blank (c).

```
append(0)
```

(b) (6.0 points)

Implement `deep`, which takes a `Tree` instance `t`. It returns a dictionary containing each unique node label of `t` as a key. The value for each key `k` is the largest depth of any node in `t` labeled `k`.

IMPORTANT: You may not call `deepest` in your implementation.

Hint: For a dictionary `d`, `d.get(1, 2)` returns the value for the key `1` if that key appears in `d` and 2 otherwise.

```
def deep(t):
    """Return a dictionary containing each unique label of t as a key
    and the maximum depth of a node labeled with that key as its value.

    >>> deep(Tree(30, [Tree(20), Tree(20, [Tree(30, [Tree(20)])])]))
    {30: 2, 20: 3}
    """
    result = {}
    def visit(node, depth):
        _______ = max(_______, result.get(_______, 0))
          (d)           (e)                 (f)
        for b in node.branches:
            -------
              (g)
    visit(t, 0)
    return result
```

i. (2.0 pt) Fill in blank (d).

```
result[node.label]
```

ii. (1.0 pt) Fill in blank (e).

- ○ 0
- ○ 1
- ● `depth`
- ○ `t.label`
- ○ `node.label`

iii. (1.0 pt) Fill in blank (f).

- ○ `depth`
- ○ `t`
- ○ `t.label`
- ○ `node`
- ● `node.label`

iv. (2.0 pt) Fill in blank (g).

```
visit(b, depth + 1)
```

5. (16.0 points) Just Add and Multiply

Definition. A plus-times-expression for a list (or linked list) of numbers inserts either `+` or `*` between each adjacent pair of numbers. For example, `2*3+4+5` is one possible plus-times-expression for `[2, 3, 4, 5]`.

(a) (5.0 points)

Implement `muladd`, which takes a **linked list** of numbers `s`. It returns the value of the plus-times-expression that starts with `*` and then alternates between `+` and `*`. If `s` has only one element, `muladd` returns that element.

The `Link` class is defined on the left column of page 2 of the Midterm 2 Study Guide.

```
def muladd(s):
    """Combine the numbers in linked list s by alternately multiplying and adding.

    >>> example = Link(9, Link(4, Link(7, Link(2, Link(0)))))  # <9 4 7 2 0>
    >>> muladd(example)           # 9 * 4 + 7 * 2 + 0, not 9 * (4 + 7 * (2 + 0))
    50
    >>> muladd(Link(2, example))  # 2 * 9 + 4 * 7 + 2 * 0
    46
    >>> muladd(Link(2))
    2
    """
    if s is Link.empty:

        return 0

    elif s.rest is Link.empty:

        return _______
                 (a)
    else:

        return _______
                 (b)
```

i. (1.0 pt) Fill in blank (a).

- ○ `0`
- ○ `1`
- ● `s.first`
- ○ `s.rest`
- ○ `s.rest.first`

ii. (4.0 pt) Fill in blank (b). If your answer is too long, you can continue on another line.

```
s.first * s.rest.first + muladd(s.rest.rest)
```

(b) (5.0 points)

Implement `ways`, which takes positive integers `k` and `n`. It returns the number of possible plus-times-expressions for a list of length `n` that have at most `k` `*`-symbols in a row. For example, `2+3*4*5*6+7*8*9+10+11*12*13*14+15*16` has 3 in a row `3*4*5*6`, then 2 in a row `7*8*9`, then 3 in a row `11*12*13*14`, then 1 in a row `15*16`.

```
def ways(k, n):
    """Return the number of plus-times-expressions with at most k consecutive *'s for n numbers.

    >>> ways(1, 4) # For [2, 3, 4, 5], these 5 ways: 2+3+4+5, 2+3+4*5, 2+3*4+5, 2*3+4+5, 2*3+4*5
    5
    >>> ways(2, 4) # For [2, 3, 4, 5], the 7 ways would also include 2+3*4*5 and 2*3*4+5
    7
    >>> ways(2, 5) # Some examples for [2, 3, 4, 5, 6]: 2+3*4+5*6 & 2*3*4+5*6 (but not 2+3*4*5*6)
    13
    """
    def f(left, n):

        if _______:
             (c)

            return 1

        result = f( _______ , n - 1)
                      (d)
        if left:

            result += _______
                        (e)
        return result

    return f(k, n)
```

i. (1.0 pt) Fill in blank (c).

- ○ `left == 0`
- ○ `left == 1`
- ○ `n == 0`
- ● `n == 1`

ii. (1.0 pt) Fill in blank (d).

- ○ `left`
- ○ `left - 1`
- ● `k`
- ○ `k - 1`

iii. (3.0 pt) Fill in blank (e).

```
f(left - 1, n - 1)
```

(c) (6.0 points)

Implement `close`, which takes a list of numbers `s` and a number `x`. It returns the number closest (in absolute value) to `x` that is the value of some plus-times-expression for `s`.

Hint: A slice that starts past the end of a list is empty. For example, `[2, 3, 4][3:]` evalautes to `[]`.

The `product` function used to implement `close` is defined below.

```
def close(s, x):
    """Return the value of a plus-times-expression for s that is closest to x.

    >>> print(close([1, 2, 3, 4], 10))  # 1 + 2 + 3 + 4 = 10
    10
    >>> print(close([1, 2, 3, 4], 20))  # 1 * 2 * 3 * 4 = 24 (4 away from 20)
    24
    >>> print(close([1, 2, 3, 4], 30))  # 1 + 2 * 3 * 4 = 25 (5 away from 30)
    25
    >>> print(close([3, 7, 5, 4, 2, 6], 66))  # 3 * 7 + 5 * 4 * 2 + 6 = 67 (1 away from 66)
    67
    """
    if not s:
        return 0
    choices = [product(s[:i]) + _______ for i in range(1, len(s) + 1)]
                                  (f)

    return min(choices, key=_______)
                              (g)

def product(s):
    """Return the result of multiplying together the elements of a non-empty list of numbers s."""
    if len(s) == 1:
        return s[0]
    return s[0] * product(s[1:])
```

i. (4.0 pt) Fill in blank (f).

```
close(s[i:], x - product(s[:i]))
```

ii. (2.0 pt) Fill in blank (g). **Select all that apply.** Assume `sub` has been imported from the `operator` module. The `sub` function takes two arguments and subtracts the second from the first.

- ☐ `abs`
- ☐ `sub`
- ☐ `abs(sub)`
- ☐ `sub(abs)`
- ■ `lambda y: abs(x - y)`
- ☐ `lambda x, y: abs(x - y)`

(d) (0.0 points)

This A+ question is not worth any points. It can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Fill in the blank of `close_exp`, which takes a list of numbers `s` and a number `x`. It returns a string containing a plus-times-expression for `s` that evaluates to a number that is as close (in absolute value) as possible to `x`. (If there is more than one such expression, return any of them.)

The built-in `eval` function takes a string containing an expression and returns its value. For example, `eval('2+2')` evaluates to 4.

```
def close_exp(s, x):
    """Return the plus-times-expression of s that has a value closest to x.

    >>> print(close_exp([1, 2, 3, 4], 10))  # 1 + 2 + 3 + 4 = 10
    1+2+3+4
    >>> print(close_exp([1, 2, 3, 4], 20))  # 1 * 2 * 3 * 4 = 24 (4 away from 20)
    1*2*3*4
    >>> print(close_exp([1, 2, 3, 4], 30))  # 1 + 2 * 3 * 4 = 25 (5 away from 30)
    1+2*3*4
    >>> print(close_exp([3, 7, 5, 4, 2, 6], 66))  # 3 * 7 + 5 * 4 * 2 + 6 = 67 (1 away from 66)
    3*7+5*4*2+6
    """
    goal = close(s, x)
    term = ''
    for i in range(len(s)):
        if term:
            term += '*'
        term += str(s[i])
        candidate = term
        if i < len(s) - 1:
            candidate += _________________________
        if eval(candidate) == goal:
            return candidate
```

i. (0.0 pt) Fill in the blank. If your answer is too long, you can continue on another line.

```
'+' + close_exp(s[i+1:], x - eval(term))
```

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Summer 2019

MIDTERM SOLUTIONS

INSTRUCTIONS

- You have 3 hours to complete the exam individually.
- The exam is closed book, closed notes, closed computer, and closed calculator, except for two hand-written 8.5" × 11" crib sheet of your own creation.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last (Family) Name	
First (Given) Name	
Student ID Number	
Berkeley Email	
Teaching Assistant	
Exam Room	
Name of the person to your left	
Name of the person to your right	
All the work on this exam is my own. **(please sign)**	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, and `abs`.
- You **may not** use example functions defined on your study guides unless a problem clearly states you can.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.

1. (10 points) WWPD

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. Each expression has at least one line of output.

- If an error occurs, write **Error**, but include all output displayed before the error.
- To display a function value, write **Function**.
- If an expression would take forever to evaluate, write **Forever**.
- If nothing is displayed, write **Nothing**

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have started `python3` and executed the code shown on the left first, then you evaluate each expression on the right in the order shown. Expressions evaluated by the interpreter have a cumulative effect.

```
1  d = {}
2  x = 2
3  y = 9
4  p = lambda : print('crikey')
5
6  def f(a, b):
7      if b or print(a):
8          print(a, b)
9      if b > a:
10         return a + b
11     else:
12         return "less"
13
14 def g(f, x, y):
15     def m(n):
16         if f(n):
17             m = f
18             print('bonk')
19         else:
20             m = p
21             print('ers')
22         return m(n - 1)
23     return m(x) and f(x)
24
25 def h(x):
26     if x not in d:
27         d[x] = 'yes'
28         return False
29     else:
30         return True
31
32 def m(x):
33     d = {}
34
35 def e(x):
36     print(h(x))
37     m(x)
38     print(h(x))
```

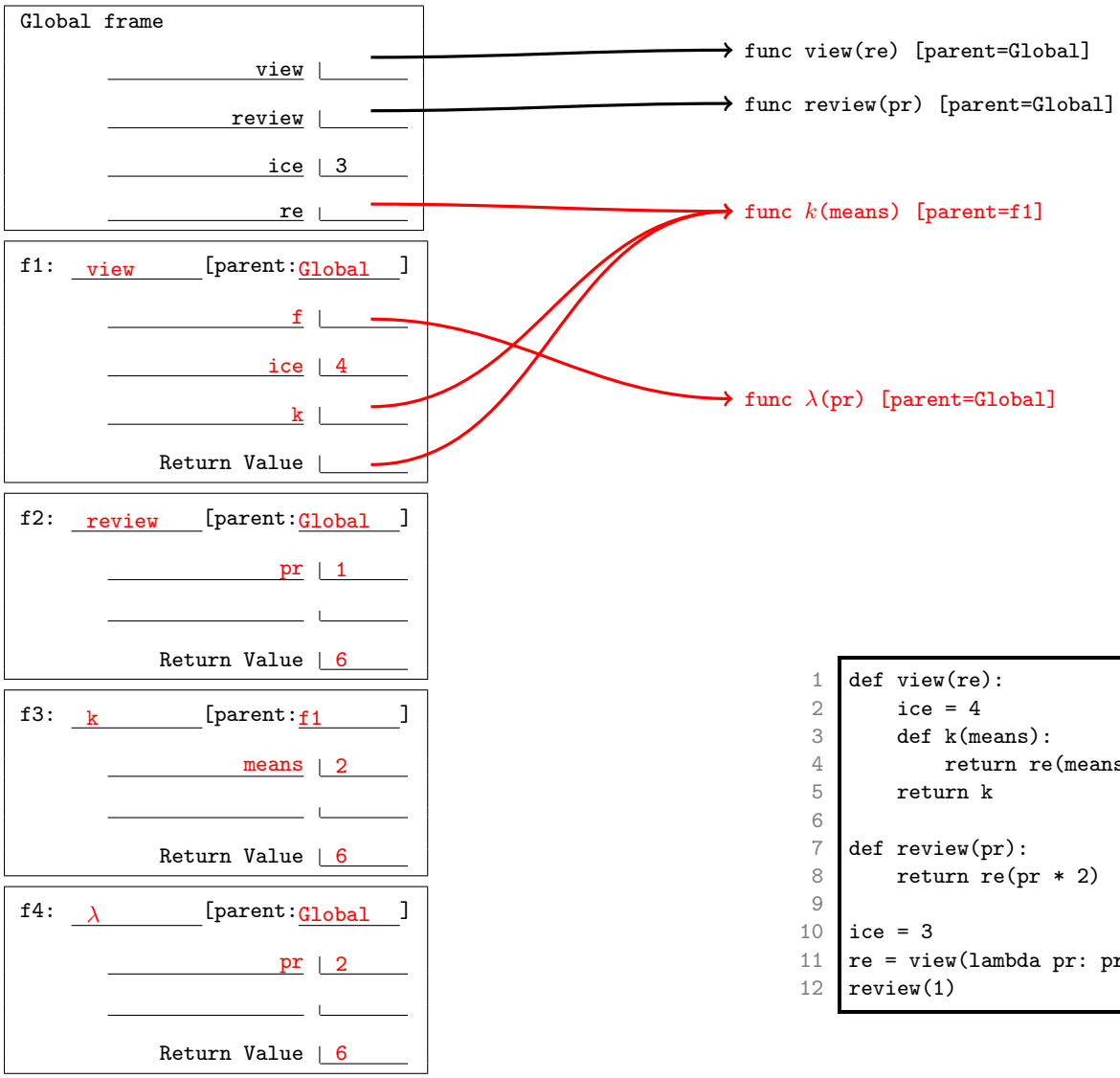
Expression	Interactive Output
`print(4, 5) + 1`	4 5 **Error**
`(p() and p()) or p()`	crikey crikey
`print(e(5))`	False True None
`f(x, y)`	2 9 11
`g(h, 4, 10)`	ers **Error**
`g(lambda x: x % 2, y, x)`	bonk 0

2. (10 points) Environment Diagram

Fill in the environment diagram that results from executing the code below until the entire program is finished, an error occurs, or all frames are filled. *You may not need to use all of the spaces or frames.*

A complete answer will:

- Add all missing names and parent annotations to all local frames.
- Add all missing values created or referenced during execution.
- Show the return value for each local frame.




```
1  def view(re):
2      ice = 4
3      def k(means):
4          return re(means)
5      return k
6
7  def review(pr):
8      return re(pr * 2)
9
10 ice = 3
11 re = view(lambda pr: pr * ice)
12 review(1)
```

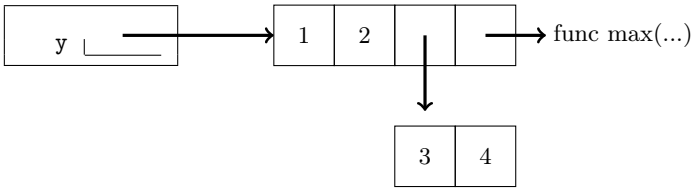
3. (8 points) Boxing Day

Draw box-and-pointer diagrams for the state of the lists after executing each block of code below. Each box should contain a value or a pointer to an object, as in an environment diagram.

You don't need to write index numbers or the word `list`. Please erase or cross out any boxes or pointers that are not part of a final diagram.

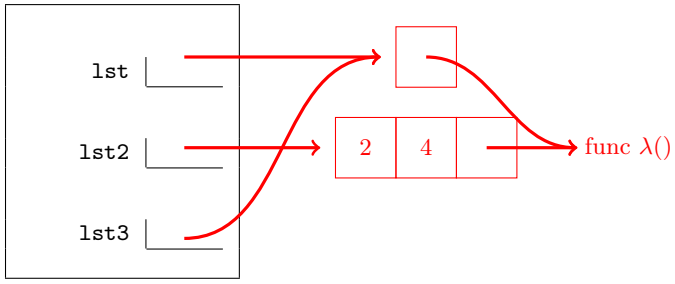
An example is given below:

```
1  y = [1, 2, [3, 4], max]
```



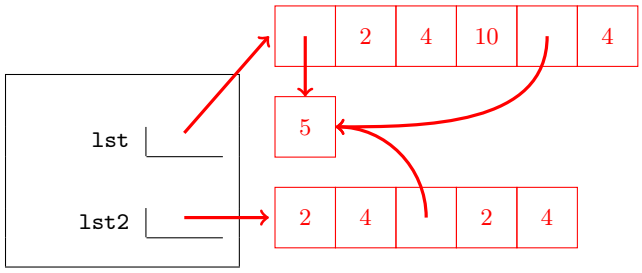
(a) (3 pt)

```
1  lst = [2, 4, lambda: lst]
2  lst2 = lst
3  lst = lst[2:]
4  lst3 = lst[0]()
```



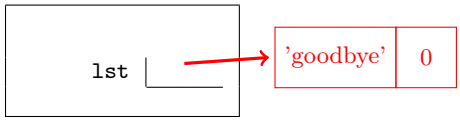
(b) (3 pt)

```
1  lst = [[5], 2, 4, 10]
2  lst2 = lst[1:3] + lst[:3]
3  for n in lst2[:2]:
4      lst.append(lst2[n])
```



(c) (2 pt)

```
1  lst = ['goodbye', 0, None, 8, 'hello', 1]
2  while lst.pop():
3      x = lst.pop()
4      if x:
5          lst.pop()
6      else:
7          lst.append('three')
```



4. (2 points) Conceptual Questions

There are three problems here, which cover topics from this week's lecture. Each problem is worth 1 point, but you can only earn a maximum of 2 points on this problem, so you only need to know two answers.

Please make sure to fill in the bubble completely when answering. Each question has only one right answer.

(a) (1 pt) Mutable Functions

Declaring a variable nonlocal allows you to modify a binding for that variable in:

◯ Any parent frame ◯ The global frame ● Any parent frame other than global ◯ The current frame

(b) (1 pt) Growth

Out of these options, what's the best way to measure the performance of a program?

◯ Run the program many times and calculate the average time it takes to run
● Count the number of operations executed by the program for different inputs
◯ Count how many while and for loops the code contains

(c) (1 pt) Iterators and Generators

Select the statement that is ***not*** true. A generator function is a function that...

● is an iterator ◯ returns an iterator ◯ returns a generator ◯ contains the "yield" keyword

5. (4 points) Negative Feedback

Write a function `clear_negatives` that takes in a list containing integers, and returns a list which is identical to the original, but in each case where a negative integer appears, delete a number of elements from the list equivalent to the magnitude of the negative integer.

For example, if -4 occurs in the list, remove a total of four elements: -4 and the next three elements. See the doctests for examples.

Your solution ***must*** use recursion, and should not modify the input list.

```
def clear_negatives(lst):
    """
    >>> clear_negatives([1, 0, 6])
    [1, 0, 6]
    >>> clear_negatives([3, -2, 1, 6, 2])
    [3, 6, 2]
    >>> lst = [-3, -4, 5, 5]
    >>> clear_negatives(lst)
    [5]
    >>> lst
    [-3, -4, 5, 5]
    """

    if not lst:

        return []

    elif lst[0] < 0:

        return clear_negatives(lst[-lst[0]:])

    else:

        return [lst[0]] + clear_negatives(lst[1:])
```

6. (6 points) Your Guess Is As Good As Mine

Let's play a guessing game! In order to do this, we'll use higher order functions.

Write a function, `make_guess`, which takes in a number that we want someone to try to guess, and returns a guessing function.

A *guessing function* is a one-argument function which takes in a number. If the number passed in is the number we wanted to guess, it will return the number of incorrect guesses made prior to the correct guess. Otherwise, it returns another guessing function, which keeps track of the fact that we've made an incorrect guess.

Solutions which use lists, object mutation, nonlocal, or global will receive **no credit.**

```
def make_guess(n):
    """Makes a guessing function for n.

    >>> guesser = make_guess(10)
    >>> guess1 = guesser(6)
    >>> guess2 = guess1(7)
    >>> guess3 = guess2(8)
    >>> guess3(10)
    3
    >>> guess2(10)
    2
    >>> a = make_guess(5)(1)(2)(3)(4)(5)
    >>> a
    4
    """
    def update_guess(num_incorrect):
        def new_guess(x):
            if x == n:
                return num_incorrect
            else:
                return update_guess(num_incorrect + 1)
        return new_guess
    return update_guess(0)
```

7. (12 points) Scrabbled Eggs

In this problem, we will build (part of) the ultimate Scrabble AI! Scrabble is a game where, given a collection of letters that each have a point value assigned, you try to make the word with the greatest score.

For example, if "a" and "s" are both worth 1 point, and "b" and "c" are both worth 2 points, the word "cab" would get 5 points, and "sass" would get 4 points.

(a) (8 pt) Implement the function `is_subseq` which takes two strings, `w1` and `w2`, and returns `True` if `w1` is a subsequence of `w2`.

Recall from Lab 5 that a string is a subsequence of another string if all the letters in the first string appear in the second string in the same order, although not necessarily directly next to each other. Put another way, you can transform the first string to the second string just by adding letters at any position in the first string.

For example, "put" and "cot" are both subsequences of "computer", but "car" and "rot" are not.

```
def is_subseq(w1, w2):
    """ Returns True if w1 is a subsequence of w2 and False otherwise.

    >>> is_subseq("word", "word")
    True
    >>> is_subseq("compute", "computer")
    True
    >>> is_subseq("put", "computer")
    True
    >>> is_subseq("computer", "put")
    False
    >>> is_subseq("sin", "science")
    True
    >>> is_subseq("nice", "science")
    False
    >>> is_subseq("boot", "bottle")
    False
    >>> is_subseq("a", "bottle")
    False
    """

    if not w1:
        return True
    elif not w2:
        return False
    else:
        with_elem = w1[0] == w2[0] and is_subseq(w1[1:], w2[1:])
        without_elem = is_subseq(w1, w2[1:])
        return with_elem or without_elem
```

(b) (4 pt) Now, implemment `scrabbler` which takes in a string `chars`, a list of strings `words`, and a dictionary `values` which maps letters to point values. It should return a dictionary where each key is a word in `words` which can be formed from the letters in `chars` and each value is the point value of that word.

For the purposes of this problem, we will only consider words we can form by using letters in `chars` in the same order they appear in the string. Assume `values` contains all the letters across valid words as keys. You may additionally assume `is_subseq` from the previous part has been implemented correctly.

```
def scrabbler(chars, words, values):

    """
    >>> words = ["easy", "as", "pie"]
    >>> values = {"e": 2, "a": 2, "s": 1, "p": 3, "i": 2, "y": 4}
    >>> scrabbler("heuaiosby", words, values)
    {'easy': 9, 'as': 3}
    >>> scrabbler("piayse", words, values)
    {'pie': 7, 'as': 3}
    """

    result = {}

    for w in words:

        if is_subseq(w, chars):

            total = sum([values[c] for c in w])

            result[w] = total

    return result
```

8. (8 points) Leaf It To Me

Write a function `max_path` which takes in a tree `t` with positive labels and a number `k`, and returns the path with length at most `k` for which the sum of the labels in the path is greatest. If there are multiple paths with the greatest sum, return the leftmost one.

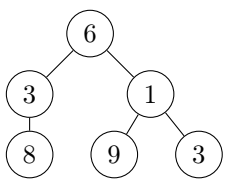
The path does *not* have to start at the root of the tree - the path can contain any top-to-bottom sequence of nodes.

```
def tree(label, branches=[]):
    return [label] + list(branches)

def is_leaf(t):
    return not branches(t)

def label(t):
    return t[0]

def branches(t):
    return t[1:]
```

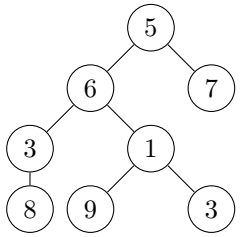
The tree data abstraction is provided here for your reference. Do not violate the abstraction barrier!

```
def max_path(t, k):
    """ Return a list of the labels on any path in tree t of length at most k with the greatest sum

    >>> t1 = tree(6, [tree(3, [tree(8)]), tree(1, [tree(9), tree(3)])])
    >>> max_path(t1, 3)
    [6, 3, 8]
    >>> max_path(t1, 2)
    [3, 8]
    >>> t2 = tree(5, [t1, tree(7)])
    >>> max_path(t2, 1)
    [9]
    >>> max_path(t2, 2)
    [5, 7]
    >>> max_path(t2, 3)
    [6, 3, 8]
    >>> max_path(t1, 4)
    [6, 3, 8]
    """
    def helper(t, k, on_path):
        if k == 0:
            return []
        elif is_leaf(t):
            return [label(t)]
        a = [[label(t)] + helper(b, k - 1, True) for b in branches(t)]
        if on_path:
            return max(a, key = sum)
        else:
            b = [helper(b, k, False) for b in branches(t)]
            return max(a + b, key = sum)
    return helper(t, k, False)
```

t1



t2



9. (0 points) The Prisoner's Dilemma

In this extra credit problem, you may choose one of two options.

- Mark the choice to "Betray" and write a positive integer in the blank below. The one student who writes the *smallest, unique positive integer* will receive *two* (2) extra credit points but only if fewer than 90% of students choose the next option.
- Mark the choice to "Work Together". If at least 90% of students choose this option, all students who chose this option will receive *one* (1) extra credit point and those who marked the choice to "Betray" will receive zero (0) extra credit points.

Will you *work together*? Or will you *betray* your fellow students? It is up to you.

○ Betray _____

● Work Together

10. (0 points) The End

(a) (0 pt) Any feedback for us on how this exam went / how the course is going so far?

(b) (0 pt) Use this space to draw a picture, write a note, or otherwise express yourself.

CS 61A Structure and Interpretation of Computer Programs
Summer 2021

Diagnostic Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number? A regex restricts inputs to numerical responses only.

1. (1.0 points) Applications are Closed

Write a higher order function `apply_until_n` that takes in `f`, a one-argument function, and a positive integer `n` > 0. `apply_until_n` should return a function that takes in a number `x` and applies `f` onto `x` as many times as possible before the result exceeds `n`.

This function should return the latest result that doesn't exceed `n`. You may assume that the return value of `f` is always strictly greater than the input.

```
def apply_until_n(f, n):
    """
    >>> square = lambda x: x * x
    >>> add_one = lambda x: x + 1
    >>> apply_until_n(add_one, 5)(3)
    5
    >>> apply_until_n(square, 10)(2)
    4
    >>> apply_until_n(square, 20)(2)
    16
    """
    def applier(x):
        while ________:
        #        (a)
            ____________
            #    (b)
        return x
    return ________
    #         (c)
```

(a) Fill in blank (a)?

f(x) <= n

(b) Fill in blank (b)?

x = f(x)

(c) Fill in blank (c)?

applier

2. (1.0 points) Catch that Bug!

Your classmates are trying to complete a coding assignment but are struggling. The assignment is to write a function `digit_counter`, which takes in `f`, a one-argument function, and `n`, a non-negative integer, and returns the number of digits in `n` for which `f(digit)` returns `True`.

Here's a doctest showing the intended behavior:

```
>>> is_even = lambda x: (x % 2) == 0
>>> digit_counter(is_even, 1112)
1
>>> digit_counter(is_even, 9843)
2
>>> greater_than_three = lambda x: x > 3
>>> digit_counter(greater_than_three, 123456789)
6
```

In all below parts of this problem, you don't need to indent your answer when writing code. Each solution can be made correct by changing exactly one line.

(a) Albert has decided to use a while loop to complete the assignment, but it's not working!

```
1 def digit_counter(f, n):
2     counter = 0
3     while n >= 0:
4         if f(n % 10):
5             counter += 1
6         n = n // 10
7     return counter
```

Which line number is Albert's error on?

3

(b) What should that line be replaced with?

n > 0

(c) Alex has decided to use recursion instead, but it's also not working!

```
1 def digit_counter(f, n):
2     if n < 10 and f(n):
3         return n
4     if f(n % 10):
5         return 1 + digit_counter(f, n // 10)
6     return digit_counter(f, n // 10)
```

Which line number is Alex's error on?

2

(d) What should that line be replaced with?

if n == 0:

(e) Catherine has taken an unconventional approach and has decided to use a helper function – and it's still wrong!

```
1 def digit_counter(f, n):
2     def helper(x, sofar):
3         if x > n:
4             return sofar
5         last = (n // x) % 10
6         return helper(x * 10, sofar + f(last))
7     return helper(0, 0)
```

Which line number is Catherine's error on?

7

(f) What should that line be replaced with?

return helper(1, 0)

3. (1.0 points) Oh, Camel!

Definition: A *camel sequence* is an integer in which each digit is either strictly less than or strictly greater than both of its adjacent digits. Write a function `is_camel_sequence` that takes in a nonnegative integer `n` and returns whether `n` is a *camel sequence*.

Note: Any single digit integer is a valid camel sequence.

Restrictions: You may not use `int`, `str`, `[` or `]` in your solution.

```
def is_camel_sequence(n):
    """
    >>> is_camel_sequence(15263) # 1 < 5, 5 > 2, 2 < 6, 6 > 3
    True
    >>> is_camel_sequence(98989)
    True
    >>> is_camel_sequence(123) # 1 < 2, but 2 is not greater than 3.
    False
    >>> is_camel_sequence(4114) # 1 is not strictly less than 1
    False
    >>> is_camel_sequence(1)
    True
    >>> is_camel_sequence(12)
    True
    >>> is_camel_sequence(11)
    False
    """
    def helper(n, incr):
        if _____________:
        #      (a)
            return True
        elif incr:
            return _______________ and helper(_____________)
            #            (b)                       (c)
        else:
            return _______________ and helper(_____________)
            #            (d)                       (e)
    return ____________ or _____________
    #          (f)             (g)
```

(a) Fill in blank (a)?

> **n < 10**

(b) Which of the following could fill in blank (b)?

- ● `n % 10 < n // 10 % 10`
- ○ `n % 10 <= n // 10 % 10`
- ○ `n % 10 < n // 10`
- ○ `n % 10 <= n // 10`
- ○ `n % 10 < n % 100`
- ○ `n % 10 <= n % 100`
- ○ `n % 10 < n % 10 // 10`
- ○ `n % 10 <= n % 10 // 10`

(c) Fill in blank (c)?

n // 10, not incr

(d) Which of the following could fill in blank (d)?

- ● `n % 10 > n // 10 % 10`
- ○ `n % 10 >= n // 10 % 10`
- ○ `n % 10 > n // 10`
- ○ `n % 10 >= n // 10`
- ○ `n % 10 > n % 100`
- ○ `n % 10 >= n % 100`
- ○ `n % 10 > n % 10 // 10`
- ○ `n % 10 >= n % 10 // 10`

(e) Fill in blank (e)?

n // 10, not incr

(f) Fill in blank (f)?

helper(n, True)

(g) Fill in blank (g)?

helper(n, False)

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Summer 2021

Midterm Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

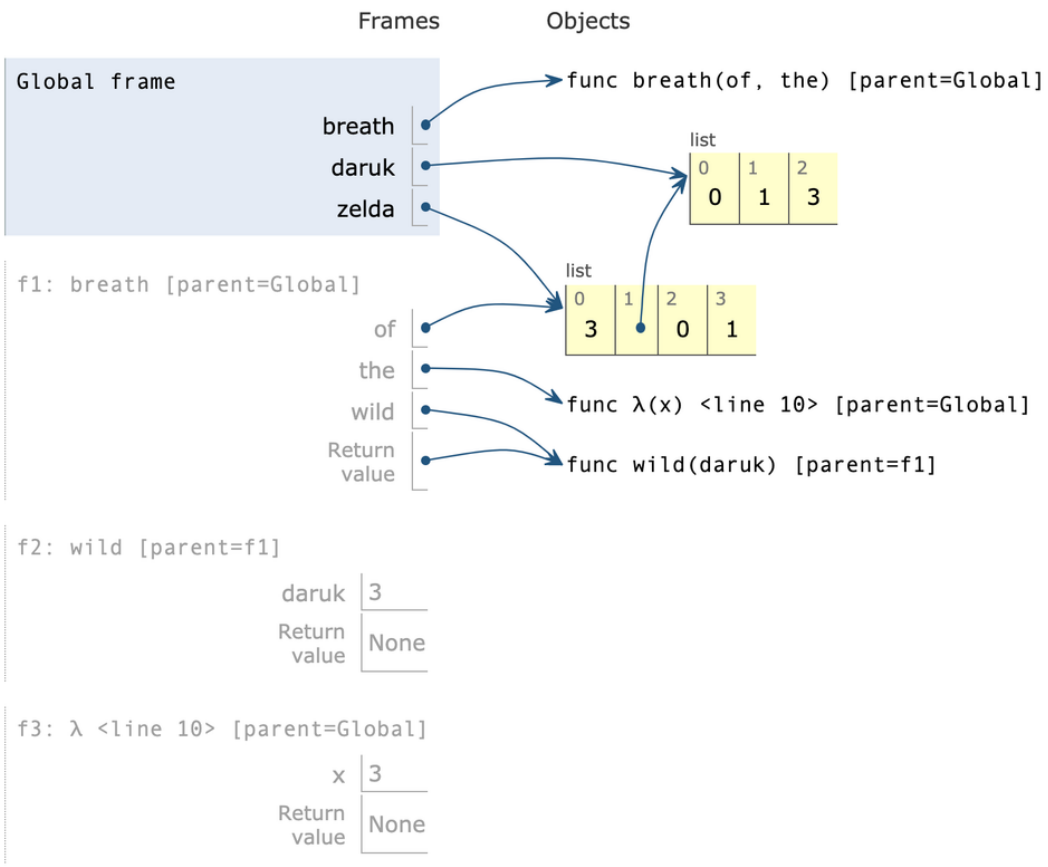
(a) What is your full name?

(b) What is your student ID number?

(c) By writing my name below, I pledge on my honor that I will abide by the rules of this exam and will neither give nor receive assistance. I understand that doing otherwise would be a disservice to my classmates, dishonor me, and could result in me failing the class.

1. Breath of the Environment

(a) The following environment diagram was generated by a program:



Click here to open the diagram in a new window

In this series of questions, you'll fill in the blanks of the program that follows so that its execution matches the environment diagram.

```
def breath(of, the):
    def wild(daruk):
        of.extend(______(a)______)
        ____(b)_____(____(c)_______)
    return wild


daruk = [0, 1]
zelda = [3, daruk]
breath(zelda, _________(d)_________)(3)
```

i. Which of these could fill in blank (a)? **Select all that apply!**

- [x] `zelda[1]`
- [ ] `daruk`
- [ ] `daruk[1]`
- [x] `of[1]`
- [ ] `of[1:]`
- [ ] `[1, 3]`
- [ ] `zelda`

ii. Which of these could fill in blank (b)?

- ( ) `breath`
- ( ) `(lambda x: zelda[1].extend([daruk]))`
- ( ) `daruk`
- ( ) `wild`
- (•) `the`
- ( ) `zelda`
- ( ) `(lambda x: x)(3)`

iii. Which of these could fill in blank (c)? **Select all that apply!**

- [x] `3`
- [ ] `(lambda x: x)(3)`
- [x] `zelda[0]`
- [ ] `zelda.pop(0)`
- [ ] `zelda.append(3)`
- [x] `of[0]`
- [x] `daruk`

iv. Which of these could fill in blank (d)?

- ( ) `lambda x: None`
- ( ) `[z + 1 for z in daruk]`
- (•) `lambda x: zelda[1].append(x)`
- ( ) `lambda x: zelda[1].extend([daruk])`
- ( ) `the(daruk)`
- ( ) `lambda x: zelda[1] + [3]`

2. LIST-en Up!

(a) Order in the Court!

Write a function `order_order` which takes a non-empty list of 2-argument functions `operators` and returns a new 2-argument function. When called, this returned function should print the result of applying the first function in operators to the two parameters passed in. It should then return another function that applies the second function in `operators` to the parameters, and so on. When the returned function has called the last function in the `operators` list, it should cycle back to the beginning of the list and use the first function again on the next call.

See the doctest for an example.

```
def order_order(operators):
    """
    >>> from operator import add, mul, sub
    >>> ops = [add, mul, sub]
    >>> order = order_order(ops)
    >>> order = order(1, 2) # applies add and returns mul
    3
    >>> order = order(1, 2) # applies mul and returns sub
    2
    >>> order = order(1, 2) # applies sub and cycles back to return add
    -1
    >>> order = order(1, 2) # cycles back to applying add
    3
    >>> order = order(1, 2)
    2
    >>> order = order(1, 2)
    -1
    """
    def apply(x, y):
        print(_________________)
        #           (a)
        return order_order(_________________)
        #                       (b)
    return ___________________
    #              (c)
```

i. What line of code could go in blank (a)?

operators [0] (x, y)

ii. What line of code could go in blank (b)?

operators[1:] + [operators[0]]

iii. What line of code could go in blank (c)?

apply

(b) Skipping Around

A skip list is defined as a sublist of a list such that each element in the sublist is non adjacent in the original list. For original list `[5, 6, 8, 2]`, the lists `[5, 8]`, `[5, 2]`, `[6, 2]`, `[5]`, `[6]`, `[8]`, `[2]`, `[]` are all skip lists of the original list. The empty list is always a skip list of any list.

Given a list `int_lst` of unique integers, return a list of all unique skip lists of `int_lst` where **each skip list contains integers in strictly increasing order**. The order in which the skip lists are returned does not matter.

```
def list_skipper(int_lst):
    """
    >>> list_skipper([5,6,8,2])
    [[5, 8], [5], [6], [8], [2], []]
    >>> list_skipper([1,2,3,4,5])
    [[1, 3, 5], [1, 3], [1, 4], [1, 5], [1], [2, 4], [2, 5], [2], [3, 5], [3], [4], [5], []]
    >>> list_skipper([])
    [[]]
    """
    if len(int_lst) == 0:
        return _________________
                    #  (a)
    with_first = ____________________
                       #  (b)
    without_first = ______________________
                          #   (c)
    with_first = [ _________ for x in with_first if x == [] or _____________]
                    # (d)                                              (e)
    return with_first + without_first
```

i. What line of code could go in blank (a)?

> [[]]

ii. Which of these could fill in blank (b)?

- ○ `list_skipper(int_lst)`
- ○ `list_skipper(int_lst[1:])`
- ● `list_skipper(int_lst[2:])`
- ○ `list_skipper(int_lst[0])`
- ○ `list_skipper(int_lst[:-1])`

iii. What line of code could go in blank (c)?

> **list_skipper(int_lst[1:])**

iv. Which of these could fill in blank (d)?

- ◯ `x`
- ◯ `x[0]`
- ◯ `[x[0]] + int_lst`
- ● `[int_lst[0]] + x`
- ◯ `int_lst.append(x)`
- ◯ `x.append(int_lst)`

v. Which of these could fill in blank (e)?

- ◯ `x[0] < int_lst[0]`
- ◯ `x < int_lst[0]`
- ● `x[0] > int_lst[0]`
- ◯ `x > int_lst[0]`
- ◯ `x[0] == int_lst[1]`
- ◯ `len(x) < len(int_lst)`

3. Growing Up

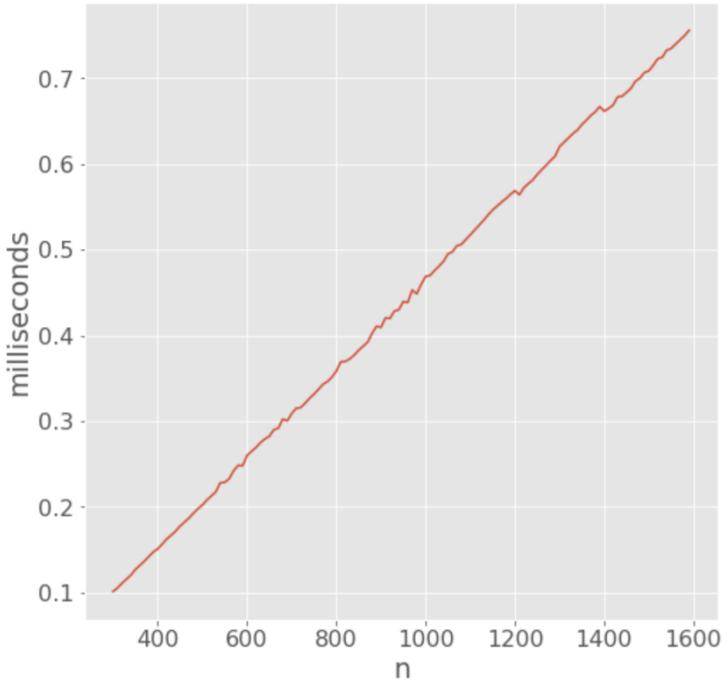
(a) Examine the `mystery1` function below which takes in a single positive integer parameter `n`:

```
def mystery1(n):
    i = 1
    total = 0
    while i <= n:
        if len([j for j in range(2, n + 1) if i % j == 0]) > 1:
            total = total + 1
        i = i + 1
    return total
```

i. Select the option indicating the *worst-case* order of growth of the runtime of `mystery1`.

- ◯ Constant: $O(1)$
- ◯ Logarithmic: $O(log_2 n)$
- ◯ Linear: $O(n)$
- ● Quadratic: $O(n^2)$
- ◯ Exponential: $O(2^n)$

(b) Assume that there exists a `mystery2` function which takes in a single positive integer parameter `n`. The below graph compares `n` to the *worst-case* time it takes to evaluate `mystery2(n)`.



You may assume that `mystery2(n)`'s runtime follows a similar trend for all values of `n`.

i. Select the option best matching the *worst-case* order of growth of the runtime of `mystery2`.

- ◯ Constant: $O(1)$
- ◯ Logarithmic: $O(log_2 n)$
- ● Linear: $O(n)$
- ◯ Quadratic: $O(n^2)$
- ◯ Exponential: $O(2^n)$

4. Maximum Exponen-tree-ation

(a) For a list of integers `values`, define the **maximum exponentiation** of `values` to be the largest single number that can be achieved by repeatedly combining pairs of consecutive elements from `values` through exponentiation. Two possible exponentiations for the list `[3, 1, 2, 3]` are shown below:

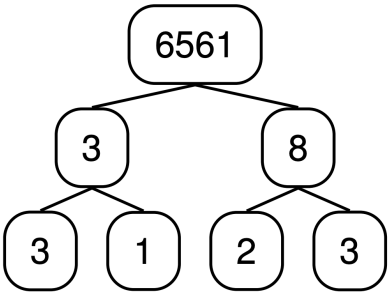
```
(3 ** 1) ** (2 ** 3) = 6561
(3 ** (1 ** 2)) ** 3 = 27
```

Of all the exponentiations of `[3, 1, 2, 3]`, the maximum exponentiation is `6561`, which can be calculated through the below steps:

```
[3, 1, 2, 3]
[3 ** 1, 2, 3]
[3, 2, 3]
[3, 2 ** 3]
[3, 8]
[3 ** 8]
6561
```

Implement `exp_tree`, which accepts a non-empty list of integers `values` and returns a tree whose label is the maximum exponentiation of `values`. Given that the maximum exponentiation is `b ** n` for some two integers `b` and `n`, the two branches of this tree should be the `exp_tree`s of the sublists of `values` used to exponentiate to `b` and `n`, respectively. You may assume that `values` has a unique exponentiation tree.

For example, below is a drawing of the `exp_tree` for `values = [3, 1, 2, 3]`:



The function should assume the standard CS61A Tree ADT definition, which is already loaded into code.cs61a.org, and is viewable here: code.cs61a.org/tree_adt

Note: Exponentials in Python are expressed as `base ** exponent`.

```
def exp_tree(values):
    """
    Returns the exponential tree that can be made from VALUES
    with the greatest possible root label

    >>> print_tree(exp_tree([5]))
    5
    >>> print_tree(exp_tree([3, 2]))
    9
      3
      2
    >>> print_tree(exp_tree([2, 3, 2]))
    512
      2
      9
        3
        2
    >>> lst = [3, 1, 2, 3]
```

```
>>> print_tree(exp_tree(lst))
6561
  3
    3
    1
  8
    2
    3
"""
if __________:
    #  (a)
   return __________
            # (b)
else:
    def tree_at_split(i):
        base = exp_tree(__________)
                         #  (c)
        exponent = exp_tree(__________)
                            #  (d)
        return tree(__________, [base, exponent])
                    #  (e)
    trees = [tree_at_split(i) for i in range(1, len(values))]
    return max(trees, key=__________)
                           #  (f)
```

i. Which of these could fill in blank (a)?

- ◯ `len(values) == 0`
- ● `len(values) == 1`
- ◯ `values`
- ◯ `len(values) == 2`
- ◯ `values[0] < values[1]`
- ◯ `len(values) <= 2`
- ◯ `len(values) <= 3`

ii. Which of these could fill in blank (b)?

- ◯ `0`
- ◯ `tree(0)`
- ◯ `values[0]`
- ● `tree(values[0])`
- ◯ `values[1]`
- ◯ `tree(values[1])`
- ◯ `values[0] ** values[1]`
- ◯ `tree(values[0] ** values[1])`
- ◯ `values[1] ** values[0]`
- ◯ `tree(values[1] ** values[0])`

iii. What line of code could go in blank (c)?

values[:i]

iv. What line of code could go in blank (d)?

values[i:]

v. What line of code could go in blank (e)?

label(base) ** label(exponent)

vi. What line of code could go in blank (f)?

label

5. You've Seen it All

(a) Wait 'til You See What's in Store

Implement `memory_store`, a function that will return two functions: `add_val` and `times_seen`. When called on a number `var1`, `times_seen` will return the number of times `add_val` has been called on that particular value `var1`.

```
def memory_store():
    """
    >>> add_val, times_seen = memory_store()
    >>> add_val(4)
    >>> for _ in range(3):
    ...     add_val(3)
    >>> times_seen(3)
    3
    >>> times_seen(4)
    1
    >>> times_seen(2)
    0
    >>> add_val(3)
    >>> times_seen(3)
    4
    """
    memory = {}
    def add_val(var1):
        if var1 in memory:
            ----------------
            #      (a)
        else:
            ------------------
            #      (b)
    def times_seen(var1):
        return ------------------
        #             (c)
    return add_val, times_seen
```

i. Which of these could fill in blank (a)?

- ◯ `memory[var1] = 0`
- ◯ `memory[var1] = var1`
- ◯ `memory.append(var1)`
- ● `memory[var1] = memory[var1] + 1`
- ◯ `return memory[var1]`
- ◯ `return memory[0]`
- ◯ `return memory.get(var1, 0)`

ii. What line of code could go in blank (b)?

memory[var1] = 1

iii. What line of code could go in blank (c)?

```
memory.get(var1, 0)
```

(b) XKSeenD

Definition: A repeatable function is a function that returns another repeatable function, which also returns a repeatable function, and so on infinitely.

Implement the function `add_seen_d`, which takes in an integer `d` and returns a repeatable function `inner`.

When `inner` is called on an integer `innervar`, it prints the sum of all integers that have been passed to `inner` at least `d` times so far among the repeated calls.

You will need to implement the helper function `make_seen_function`, which returns a seen function. A seen function is a function that takes in a value and returns the number of times the number has been seen already.

You may not use lists, dictionaries, or the function defined in 4.1 in this part.

```
initial_seen = lambda x: 0
def add_seen_d(d, seen_fn=initial_seen, total=0):
    """
    >>> a = add_seen_d(2)
    >>> a = a(3)
    0
    >>> a = a(4)
    0
    >>> a = a(3) # 3 seen for the 2nd time, thus added
    3
    >>> a = a(3) # 3 seen for the 3rd time, and won't be added again
    3
    >>> a = a(4) # 4 seen for the 2nd time, thus added
    7
    >>> a = a(5) # 5 is seen for the 1st time, so not added
    7
    """
    def inner(innervar):
        new_seen = ____________________________
                            #  (a)
        if new_seen(innervar) == d:
            print(____________________________)
                        #  (b)
            return ____________________________
                          #  (c)
        else:
            print(____________________________)
                        #  (d)
            return add_seen_d(d, new_seen, total)
    return inner

def make_seen_function(f, msvar):
    def seen(seenvar):
        if msvar == seenvar:
            return ____________________________
                      #  (e)
        return ____________________________
                   #  (f)
    return seen
```

i. Which of these could fill in blank (a)?

- ◯ `seen_fn`
- ◯ `lambda x: seen_fn(x) + 1`
- ◯ `make_seen_function(seen_fn, d)`
- ● `make_seen_function(seen_fn, innervar)`
- ◯ `make_seen_function(lambda x: seen_fn(x) + 1, d)`
- ◯ `make_seen_function(lambda x: seen_fn(x) + 1, innervar)`

ii. What line of code could go in blank (b)?

(You may not use lists, dictionaries, or the function defined in 4.1 in this part)

total + innervar

iii. What line of code could go in blank (c)?

(You may not use lists, dictionaries, or the function defined in 4.1 in this part)

add_seen_d(d, new_seen, total + innervar)

iv. What line of code could go in blank (d)?

(You may not use lists, dictionaries, or the function defined in 4.1 in this part)

total

v. Which of these could fill in blank (e)? **Select all that apply!**

- ☐ `f(msvar)`
- ☐ `f(seenvar)`
- ■ `1 + f(msvar)`
- ■ `1 + f(seenvar)`
- ☐ `f(msvar) + f(seenvar)`
- ☐ `f(seenvar) - msvar`

vi. What line of code could go in blank (f)?

(You may not use lists, dictionaries, or the function defined in 4.1 in this part)

f(seenvar)

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Summer 2022

Midterm Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

1. (7.0 points) What Would Python Display?

For each of the expressions below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines.

- If an error occurs, write `Error`, but include all output displayed before the error.
- If evaluation would run forever, write `Forever`.
- To display a function value, write `Function`.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`. Assume that you have first started `python3` and executed the below statements.

```
def f(x):
    return not x

def my_pow(x, n):
    print(x, n)
    if f(n):
        return 1
    elif n < 0:
        return 1 // my_pow(x, -n)
    elif n % 2:
        return x * my_pow(x, n - 1)
    return my_pow(x * x, n // 2)


def hero(spider):
    def man(home):
        def marvel(home):
            return None
        print(spider)
        print(marvel)
        return spider - home
    return man

goat = lambda m: lambda n: m - n
bleat = (lambda a, b, c, d: b or a(d)(c))(goat, 5 == 6, 7, 4)
```

(a) (2.0 pt) `my_pow(2, -1)`

```
2 -1
2 1
2 0
0
```

(b) (1.0 pt) `hero(1, 2, 3)`

```
Error
```

(c) (1.0 pt) `hero(1)(2)(3)`

```
1
Function
Error
```

(d) (2.0 pt) `print(1, print(4, goat(5)(4 // 2)))`

```
4 3
1 None
```

(e) (1.0 pt) `bleat`

```
-3
```

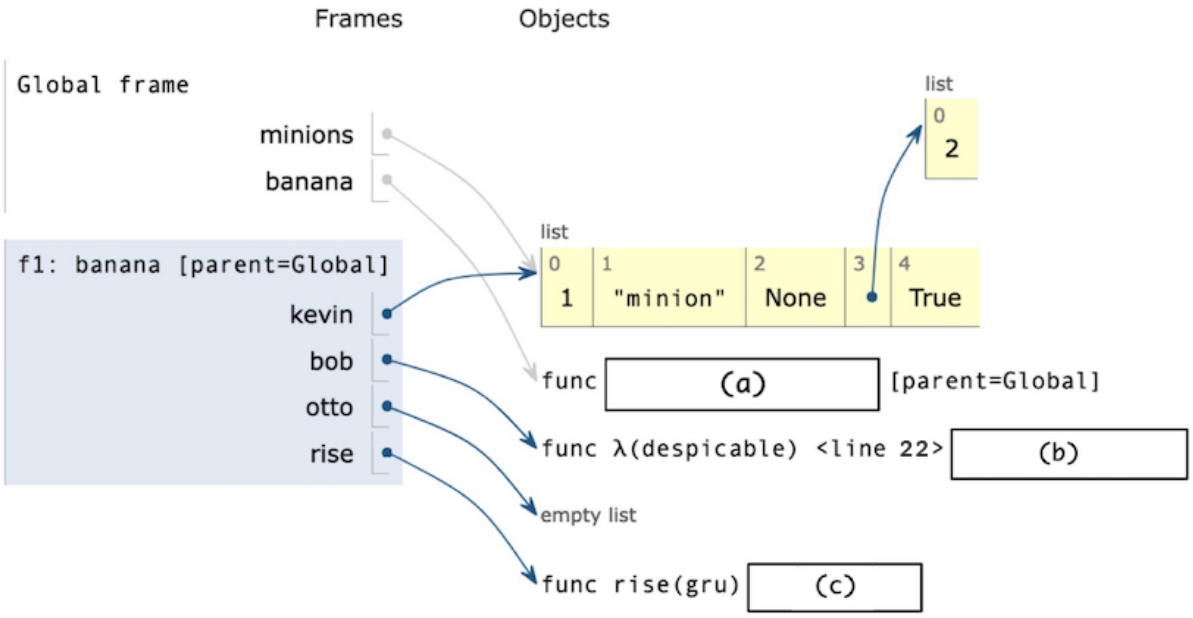
2. (6.0 points) Minions

Answer the following questions to fill in the blanks in the environment diagram, and answer what is printed when the function is run. Line numbers are included for convenience.

```
 1| minions = [1, "minion", None, [2], True]
 2|
 3| def banana(kevin, bob):
 4|     otto = []
 5|
 6|     def rise(gru):
 7|         gru.extend([gru.append([gru[1]])])
 8|         print(gru[2])
 9|         return gru[-1]
10|
11|     # STOP EXECUTION HERE FOR PART I
12|
13|     while kevin.pop():
14|         stuart = kevin.pop(0)
15|         otto.append(bob(stuart))
16|
17|     print(otto)
18|     print(minions)
19|
20|     print(rise(otto))
21|
22| banana(minions, lambda despicable: despicable * 2)
```

(a) (1.5 points) Part I

The following environment diagram shows the execution of the program until, but not including, the `while` loop beginning on line 13.



i. (0.5 pt) Fill in blank (a)

```
banana(kevin, bob)
```

ii. (0.5 pt) Fill in blank (b)

```
[parent=Global]
```

iii. (0.5 pt) Fill in blank (c)

```
[parent=f1]
```

(b) (4.0 points) Part II

Answer the following questions assuming the remaining code has been executed.

i. (1.5 pt) What will be printed to the terminal as a result of executing `print(otto)` on line 17?

```
[2, 'minionminion']
```

ii. (1.0 pt) What will be printed to the terminal as a result of executing `print(minions)` on line 18?

```
[]
```

iii. (1.0 pt) What will be printed to the terminal as a result of executing `print(gru[2])` on line 8?

```
['minionminion']
```

iv. (1.0 pt) What will be printed to the terminal as a result of executing `print(rise(otto))` on line 20?

```
None
```

3. (3.0 points) Bite-Size HOFs

(a) (1.5 points) Inverse Checker

Implement `inverse_checker`, a function that takes in two functions `f` and `g` and returns a function that returns True if `g` is the inverse function of `f` on input `n`. That is, `g` undoes the effect of `f` called on `n`.

```
def inverse_checker(f, g):
    """
    >>> checker0 = inverse_checker(lambda x: x + 1, lambda x: x - 1)
    >>> all([checker0(n) for n in range(100)])
    True
    >>> # `g` is the inverse `f`, but `f` is not the inverse of `g`
    >>> checker1 = inverse_checker(lambda x: x * 2, lambda x: x // 2)
    >>> all([checker1(n) for n in range(100)])
    True
    >>> checker2 = inverse_checker(lambda x: x ** 3, lambda x: x ** -3)
    >>> all([checker2(n) for n in range(1, 100)])
    False
    """
    def checker(n):

        return __________
                    (a)
    return __________
               (b)
```

i. (1.0 pt) Fill in blank (a)

```
return g(f(n)) == n
```

ii. (0.5 pt) Fill in blank (b)

```
checker
```

(b) (1.5 points) Force Truthy

Implement `force_truthy`, a function that takes in a function `f` and returns a function that returns the same thing as `f` when given an argument `n` such that `f(n)` outputs a truthy value, and otherwise returns `True`.

```
def force_truthy(f):
    """
    >>> truthy = force_truthy(lambda x: x // 10)
    >>> all([truthy(n) for n in range(10)])
    True
    >>> truthy(9)
    True
    >>> truthy(10)
    1
    >>> truthy(20)
    2
    """
    def truthy(n):

        return __________
                    (a)
    return __________
               (b)
```

i. (1.0 pt) Fill in blank (a)

```
f(n) or True
```

ii. (0.5 pt) Fill in blank (b)

```
truthy
```

4. (6.0 points) Least Resistance

Fill in the definition of the function `least_resistance`, which takes in three parameters, `m`, `n`, and `f`. `m` and `n` are integers which specify a coordinates position on a grid, and `f` is a two-argument function that takes in coordinates and returns a number. Your goal is to find the path of "least resistance" from the position `(m, n)` to the position `(0, 0)` on the grid, relative to `f`, which defines the resistance of each square, and return the total resistance met along that path.

A path is a series of consecutive steps from a coordinate position on the grid to `(0, 0)`, where at each step you may either take one step down, or one step to the left. The total resistance of a path is defined as the sum of `f` called on each coordinate position visited. For example, the below graphic visualizes the paths and of least resistance, and total resistance met, for the first two doctests.

5	25	26	29	34	41	50
4	16	17	20	25	32	41
3	9	10	13	18	25	34
2	4	5	8	13	20	29
1	1	2	5	10	17	26
0	0	1	4	9	16	25
	0	1	2	3	4	5

```
f = lambda x, y: x ** 2 + y ** 2
```

```
50 + 41 + 32 + 25 + 18 + 13 +
  8 + 5 + 2 + 1 + 0 = 195
```

5	5	5	5	5	5	5
4	4	4	4	4	4	4
3	3	3	3	3	3	3
2	2	2	2	2	2	2
1	1	1	1	1	1	1
0	0	0	0	0	0	0
	0	1	2	3	4	5

```
g = lambda x, y: y
```

```
5 + 4 + 3 + 2 + 1 + 0 +
0 + 0 + 0 + 0 + 0 = 15
```

Note: In the skeleton, you are provided a line that uses `float('inf')`. This will return the Python equivalent of infinity. That is, for any number `n`, `float('inf') > n` will be `True`, no matter the value of `n`.

```
def least_resistance(m, n, f):
    """
    >>> f = lambda x, y: x ** 2 + y ** 2
    >>> least_resistance(5, 5, f)
    195
    >>> g = lambda x, y: y
    >>> least_resistance(5, 5, g)
    15
    """
    if __________:
          (a)
        return __________
                  (b)
    elif __________:
            (c)
        return float('inf')
    else:
        r1 = least_resistance(_________________________________)
                                              (d)
        r2 = least_resistance(_________________________________)
                                              (e)
        return __________(r1, r2) + __________
                  (f)                   (g)
```

(a) (1.0 pt) Fill in blank (a)

```
m == 0 and n == 0
```

(b) (0.5 pt) Fill in blank (b)

```
f(0, 0) # or f(m, n)
```

(c) (1.0 pt) Fill in blank (c)

```
m < 0 or n < 0
```

(d) (1.0 pt) Fill in blank (d)

```
m - 1, n, f # can swap blanks (d) and (e)
```

(e) (1.0 pt) Fill in blank (e)

```
m, n - 1, f
```

(f) (0.5 pt) Fill in blank (f)

```
min
```

(g) (1.0 pt) Fill in blank (g)

```
f(m, n)
```

5. (7.0 points) Conditional Curry

Implement `cond_curry`, a function that takes in two functions, `f` and `cond`. `f` is a function that takes in two arguments, and `cond` is a predicate function that will take in a single argument and return either `True` or `False`. `cond_curry` returns a curried version of `f` that only "accepts" an argument `x` if calling `cond(x)` would return True. Otherwise, `x` is not accepted as an argument. Once the curried function has accepted two arguments, it will behave exactly as `f` would when called on those arguments.

```
def cond_curry(f, cond):
    """
    >>> from operator import add
    >>> curried = cond_curry(add, is_prime) # assume `is_prime` is implemented
    >>> curried(11)(13) # 11 + 13 = 24
    24
    >>> curried(10)(11)(12)(13) # 10 and 12 are not prime, and so are ignored
    24
    >>> curried(7)(4)(4)(4)(4)(4)(4)(4)(4)(4)(7)
    14
    """
    __________:
        (a)
        __________:
            (b)
            __________:
                (c)
                return __________
                            (d)
            return __________
                        (e)
        __________:
            (f)
            return __________
                        (g)
        return __________
                    (h)
    return __________
                (i)
```

(a) (0.5 pt) Fill in blank (a)

- ○ `if cond(f)`
- ● `def g(x)`

(b) (1.0 pt) Fill in blank (b)

- ○ `if cond(f)`
- ● `def h(y)`
- ○ `if cond(x)`
- ○ `if f(x, cond)`
- ○ `if f(cond, x)`

(c) **(0.75 pt)** Fill in blank (c)

- ◯ `if cond(x)`
- ● `if cond(y)`
- ◯ `if cond(f)`
- ◯ `def i(z)`

(d) **(1.0 pt)** Fill in blank (d)

- ● `f(x, y)`
- ◯ `f(x, z)`
- ◯ `f(y, z)`
- ◯ `h`
- ◯ `i`

(e) **(0.75 pt)** Fill in blank (e)

- ◯ `f(x, y)`
- ◯ `g`
- ● `h`
- ◯ `i`

(f) **(1.0 pt)** Fill in blank (f)

- ● `if cond(x)`
- ◯ `if cond(f)`
- ◯ `def h(y)`

(g) **(0.75 pt)** Fill in blank (g)

- ◯ `f(x, y)`
- ◯ `cond(x)`
- ◯ `cond(y)`
- ● `h`
- ◯ `g`

(h) **(0.75 pt)** Fill in blank (h)

- ● `g`
- ◯ `h`
- ◯ `f(x, cond)`
- ◯ `cond(x)`

(i) (0.5 pt) Fill in blank (i)

- ○ `cond(f)`
- ● `g`

6. (9.0 points) Blob Sum

(a) (2.0 points) Count Digits

Implement `count_digits`, a function that takes in a number `n` and returns the number of digits `n` contains. You should treat the number 0 as having no digits.

```
def count_digits(n):
    """
    >>> count_digits(0) # 0 has no digits
    0
    >>> count_digits(618)
    3
    >>> count_digits(2022)
    4
    """
    if __________:
          (a)
        return __________
                    (b)
    return __________
              (c)
```

i. (0.5 pt) Fill in blank (a)

```
n == 0
```

ii. (0.5 pt) Fill in blank (b)

```
0
```

iii. (1.0 pt) Fill in blank (c)

```
1 + count_digits(n // 10)
```

(b) (7.0 points) Blob Sum

Implement `blob_sum`, a function that takes in two positive integers, `n` and `k`, and returns `True` if there exists a way to add together the digits of `n` to equal `k`, where every digit of `n` is used exactly once. However, in `blob_sum`, multiple consecutive digits can be considered as a single multi-digit number (a blob), or as multiple individual digits.

Digits are read left-to-right. For example, `123` can `blob_sum` to `15` (`= 12 + 3`) but not `33` (`= 1 + 32`).

You may assume `count_digits` is implemented correctly.

```
def blob_sum(n, k):
    """
    >>> blob_sum(123, 15) # 12 + 3 = 15
    True
    >>> blob_sum(123, 6) # 1 + 2 + 3 = 6
    True
    >>> blob_sum(123, 33) # digits of `n` must be read left-to-right
    False
    >>> blob_sum(123, 24) # 1 + 23 = 24
    True
    >>> blob_sum(123, 12) # every digit of `n` must be used
    False
    >>> blob_sum(123, 35) # every digit of `n` can only be used once
    False
    """
    def helper(n, k, blob):
        if __________:
              (a)
            return __________
                        (b)
        if __________:
              (c)
            return False
        rest, last = __________
                       (d)
        new_blob = __________
                       (e)
        return __________
                   (f)
    return helper(_______________________________)
                               (g)
```

i. (0.5 pt) Fill in blank (a)

```
n == 0
```

ii. (1.0 pt) Fill in blank (b)

```
k == blob
```

iii. **(0.5 pt)** Fill in blank (c)

```
k <= 0
```

iv. **(0.5 pt)** Fill in blank (d)

```
n // 10, n % 10
```

v. **(2.0 pt)** Fill in blank (e)

```
blob + (last * (10 ** count_digits(blob)))
```

vi. **(2.0 pt)** Fill in blank (f)

```
helper(rest, k - new_blob, 0) or helper(rest, k, new_blob)
```

vii. **(0.5 pt)** Fill in blank (g)

```
n, k, 0
```

7. (6.0 points) Even Out

Fill in the definition for the function `even_out`. `even_out` takes in two parameters: `lst`, which is a list containing only the numbers `1` and `0` as elements, and `d`, which is a non-negative integer. `even_out` mutates `lst` such that exactly `d` instances of `0` occur between each instance of `1`. It also returns two values: the number of zeros it had to add to accomplish this, and the number of zeros it had to remove. You may assume that the first and last elements of `lst` will always be `1`.

```
def even_out(lst, d):
    """
    >>> lst = [1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1]
    >>> a, r = even_out(lst, 2)
    >>> lst
    [1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1]
    >>> a
    3
    >>> r
    3
    >>> a, r = even_out(lst, 0)
    >>> lst
    [1, 1, 1, 1, 1]
    >>> a
    0
    >>> r
    8
    """
    i = 0
    count = d
    added, removed = 0, 0
    while i < len(lst):
        if __________:
              (a)
            i += 1
            count = 0
        elif __________:
                (b)
            __________
               (c)
            i += 1
            count += 1
            added += 1
        elif __________:
                (d)
            __________
               (e)
            removed += 1
        else:
            i += 1
            count += 1
    return added, removed
```

(a) (1.0 pt) Fill in blank (a)

```
lst[i] == 1 and count == d
```

(b) (1.0 pt) Fill in blank (b)

```
lst[i] == 1 and count < d
```

(c) (1.5 pt) Fill in blank (c)

```
lst.insert(i, 0)
```

(d) (1.0 pt) Fill in blank (d)

```
count == d
```

(e) (1.5 pt) Fill in blank (e)

```
lst.pop(i)
```

8. (20.0 points) Boba Cafe OOPerations

(a) (9.0 points) Part I: Makin' Drinks

Tyler and Chris want to open a new restaurant, but they disagree on what style of restaurant would be most successful in Berkeley. Tyler believes that a boba restaurant will attract more sales, while Chris argues that a cafe will attract more customers. As a compromise, they've decided to open a combined boba cafe that will sell a variety of different drinks and food.

Tyler and Chris have decided to represent their restaurant's sales with different Food classes in order to track their sales.

```
class Item:

    all_items = {}

    def __init__(self, name, cost, in_stock=True):
        self.name = name
        self.cost = cost
        self.in_stock = in_stock
```

Complete the implementations for the class Boba and the class Coffee. Boba and Coffee are items in the restaurant. Whenever a new item is created, that item's name should be added to the class attribute dictionary `all_items` with a value of 1 in the class `Item`. If the name is already in the dictionary, the value should increase by 1. `Boba` should have a list `all_boba` that stores all the Boba items that have been sold. `Coffee` should have a list `all_coffee` that stores all Coffee items that have been sold.

Boba should have an additional attribute called `topping` that keeps track of what topping that drink had, if any.

Coffee should have an additional attribute called `temp` that defaults to 'hot' and a method called `add_ice` that sets the temp to 'cold' when it is called.

```
class Boba(_____):
            (a)
    all_boba = ________
                 (b)

    def __init__(self, name, cost, in_stock, topping):

        ________________________
                   (c)
        __________ = _________
            (d)          (e)
        ________________________
                   (f)
        if ___________ not in ____________________:
                (g)                    (h)
            ___________________[_________] = 1
                   (i)           (j)
        else:
            ___________________[_________] += 1
                   (i)           (j)


class Coffee(_____):
              (a)
    all_coffee = ________
                   (b)

    def __init__(self, name, cost, in_stock):

        ________________________
                   (c)
        _________ = 'hot'
            (k)
        ________________________
                   (l)
        if ___________ not in ____________________:
                (g)                    (h)
            ___________________[_________] = 1
                   (i)           (j)
        else:
            ___________________[_________] += 1
                   (i)           (j)

    def add_ice(__________):
                   (m)
        __________ = 'cold'
            (n)
```

i. (0.5 pt) Fill in blank (a)

Item

ii. **(0.5 pt)** Fill in blanks labeled (b)

```
[]
```

iii. **(1.5 pt)** Fill in blanks labeled (c)

```
super().__init__(name, cost, in_stock)
```

iv. **(0.5 pt)** Fill in blank (d)

```
self.topping
```

v. **(0.5 pt)** Fill in blank (e)

```
topping
```

vi. **(0.75 pt)** Fill in blank (f)

```
Boba.all_boba.append(self)
```

vii. **(0.5 pt)** Fill in blanks labeled (g)

```
self.name
```

viii. **(1.0 pt)** Fill in blanks labeled (h)

```
super().all_items
```

ix. **(0.5 pt)** Fill in blanks labeled (i)

```
super().all_items
```

x. **(0.5 pt)** Fill in blanks labeled (j)

```
self.name
```

xi. (0.5 pt) Fill in blank (k)

```
self.temp
```

xii. (0.75 pt) Fill in blank (l)

```
Coffee.all_coffee.append(self)
```

xiii. (0.5 pt) Fill in blank (m)

```
self
```

xiv. (0.5 pt) Fill in blank (n)

```
self.temp
```

(b) (5.0 points) Part II: Who Wins?

Tyler is feeling a bit competitive still and wants to prove to Chris that the boba part of their restaurant is the more successful part. Write a method called `more_sold` that returns whether more Boba or more Coffee has been sold. Remember Item stores a dictionary with keys as names of drinks and values as the number of that drink sold. If sales are equal, then make sure neither Tyler nor Chris think they are selling more than the other.

```
def more_sold():
        """
        >>> bubble_tea = Boba('Bubble Tea', 4, True, 'Tapioca pearls')
        >>> black_coffee = Coffee('Black', 2, True)
        >>> latte = Coffee('Latte', 4, True)
        >>> Item.more_sold()
        'Coffee'
        """
        boba_names = [__________ for boba in ____________________]
                           (a)                        (b)
        coffee_names = [__________ for coffee in ____________________]
                             (c)                          (d)
        boba_total = [_________________ for name in _____________________________]
                             (e)                                  (f)
        coffee_total = [_________________ for name in _____________________________]
                             (e)                                    (g)
        if _____________________:
                    (h)
            return 'Boba'
        elif _____________________:
                     (i)
            return 'Coffee'
        else:
            return 'Neither'
```

i. (0.5 pt) Fill in blank (a)

```
boba.name
```

ii. (0.5 pt) Fill in blank (b)

```
Boba.all_boba
```

iii. (0.5 pt) Fill in blank (c)

```
coffee.name
```

iv. (0.5 pt) Fill in blank (d)

```
Coffee.all_coffee
```

v. **(0.5 pt)** Fill in blanks labeled (e)

```
Item.all_items[name]
```

vi. **(0.75 pt)** Which of these could fill blanks labeled (f)? Check all that apply.

- [x] `Item.all_items.keys() if name in boba_names`
- [ ] `Item.all_items.values() if name in boba_names`
- [ ] `Item.all_items.items() if name in boba_names`
- [ ] `Item.all_items if name`
- [ ] `Item.all_items if boba_names`
- [x] `Item.all_items if name in boba_names`
- [ ] `Item.all_items[]`
- [x] `boba_names`
- [ ] `boba_names if name in coffee_names`
- [ ] `name`

vii. **(0.75 pt)** Which of these could fill blanks labeled (g)? Check all that apply.

- [x] `Item.all_items.keys() if name in coffee_names`
- [ ] `Item.all_items.values() if name in coffee_names`
- [ ] `Item.all_items.items() if name in coffee_names`
- [ ] `Item.all_items if name`
- [ ] `Item.all_items if coffee_names`
- [x] `Item.all_items if name in coffee_names`
- [ ] `Item.all_items[]`
- [x] `coffee_names`
- [ ] `coffee_names if name in boba_names`
- [ ] `name`

viii. **(0.5 pt)** Fill in blank (h)

```
sum(boba_total) > sum(coffee_total) # or len(Boba.all_boba) >
len(Coffee.all_coffee)
```

ix. **(0.5 pt)** Fill in blank (i)

```
sum(coffee_total) > sum(boba_total) # or len(Coffee.all_coffee) >
len(Boba.all_boba)
```

(c) (6.0 points) Part III: Optimize Cost

This question was removed.

9. (1.0 points) Extra Credit

Here are three questions about lecture. You must get all three correct to earn one point of extra credit.

(a) Songs from which artist were put into Richard's playlist in his Sequences lecture (lecture 8)?

Olivia Rodrigo

(b) Which algorithm was demoed in Laryn's Recursion lecture (lecture 6) as a method of verifying credit card numbers?

Luhn Algorithm

(c) What did Cooper say was Richard's favorite Taylor Swift album in the Objects lecture (lecture 10)?

Red (Taylor's Version)

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Summer 2023

Midterm Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

> Oski the Bear

(b) What is your student ID number?

> 123456789

(c) What is your @berkeley.edu email address?

> oski@berkeley.edu

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

> Oski Bear

1. (8.0 points) What Would Python Display

For each expression below, choose the correct option or write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines.

- If an error occurs, write "Error", but include all output displayed before the error.
- If evaluation would run forever, write "Forever".
- To display a function value, write "Function".
- If the evaluated expression wouldn't display anything, write "Nothing". The interactive interpreter displays the value of a successfully evaluated expression, unless it is None.

(a) i. (0.5 pt)

```
>>> (1 or 2) and (3 or 4)
```

- ○ `True`
- ○ `False`
- ○ `0`
- ○ `1`
- ○ `2`
- ● `3`
- ○ `4`

ii. (0.5 pt)

```
>>> ('Barbie' or 'Ken') * 2
```

- ○ `'Barbie'`
- ● `'BarbieBarbie'`
- ○ `'Ken'`
- ○ `'KenKen'`
- ○ `'BarbieKen'`
- ○ `'KenBarbie'`
- ○ `'BarbieBarbieKenKen'`
- ○ `2`
- ○ `Error`
- ○ `Nothing`

iii. (0.5 pt) For the rest of this question, assume the following code has been executed.

```
>>> it = iter([2, ['a', 'b'], 'c', 3])
>>> next(it)
```

```
2
```

iv. (0.5 pt)

```
>>> next(it)
```

```
['a', 'b']
```

v. (1.0 pt)

```
>>> list(it)
```

```
['c', 3]
```

vi. (1.0 pt)

```
>>> next(it)
```

```
Error
```

vii. (1.0 pt) For the rest of this question, assume the following code has been executed.

```
dreamhouse = 101
beach = lambda pink: pink(dreamhouse)
dreamhouse = 102

def is_even(n):
    return n % 2 == 0
```

```
>>> beach(is_even)
```

- (●) True
- ( ) False
- ( ) 101
- ( ) 102
- ( ) 1
- ( ) 0
- ( ) Function
- ( ) Error
- ( ) Nothing

viii. (1.0 pt)

```
>>> beach(dreamhouse)
```

- ( ) True
- ( ) False
- ( ) 101
- ( ) 1
- ( ) 0
- ( ) Function
- (●) Error
- ( ) Nothing

ix. (1.0 pt)

```
>>> pink = beach(print)
```

- ◯ `True`
- ◯ `False`
- ◯ `101`
- ● `102`
- ◯ `1`
- ◯ `0`
- ◯ `Function`
- ◯ `Error`
- ◯ `Nothing`

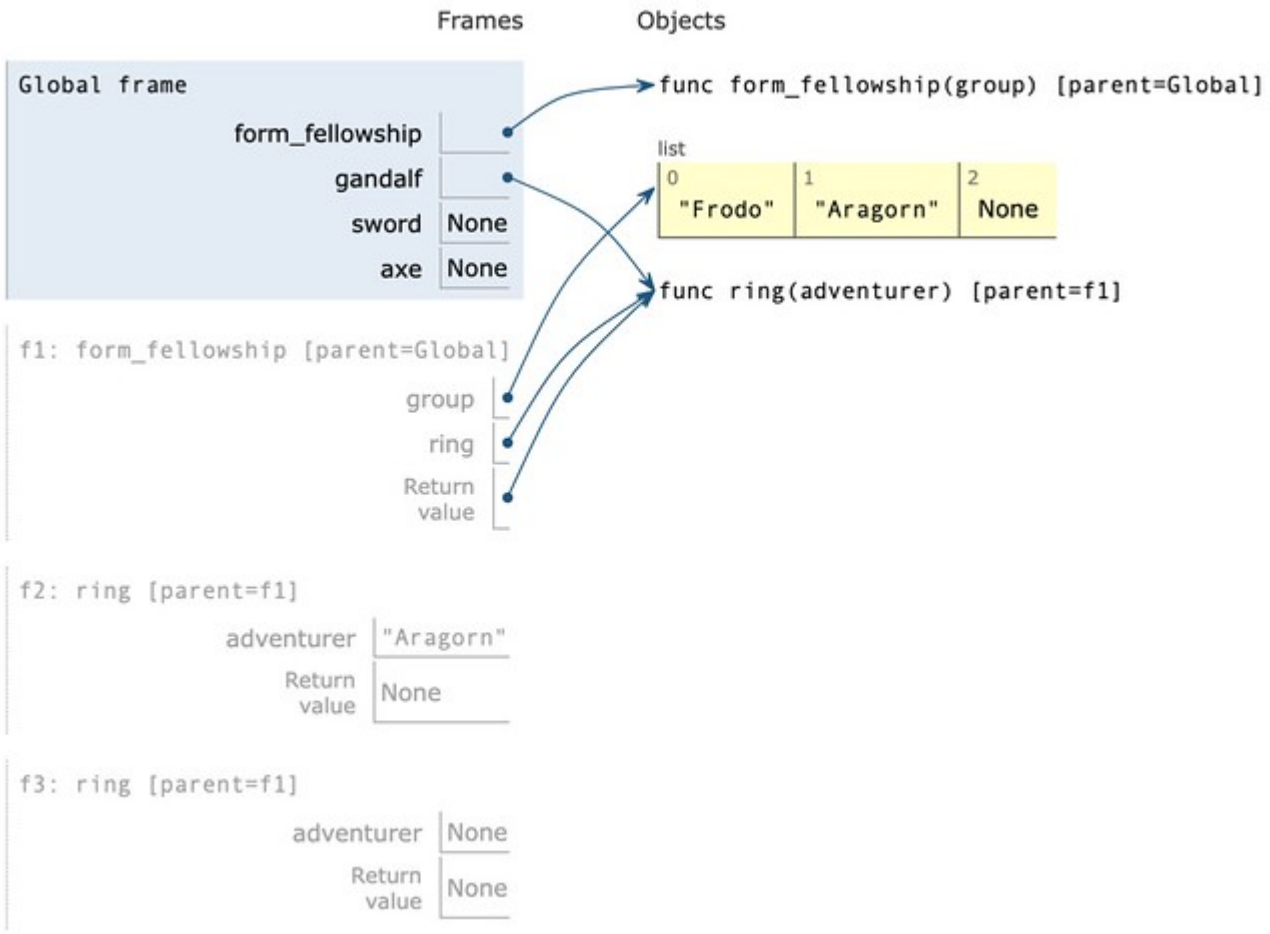
x. (1.0 pt)

```
>>> pink
```

- ◯ `True`
- ◯ `False`
- ◯ `101`
- ◯ `102`
- ◯ `1`
- ◯ `0`
- ◯ `Function`
- ◯ `Error`
- ● `Nothing`

2. (10.0 points) The Fellowship of the List

Given the environment diagram and skeleton code, answer the questions below.




```
def form_fellowship(__1__):
    def ring(__2__):
        return ____3____
    return ____4____

gandalf = form_fellowship(____5____)
sword = _____6_____
_____7_____ = _____8_____
```

(a) i. (1.0 pt) Fill in blank (1)

```
group
```

ii. (1.0 pt) Fill in blank (2)

```
adventurer
```

iii. **(1.5 pt)** Which of these could go in 3? (select all that apply)

- ☐ `group + adventurer`
- ☐ `adventurer + group`
- ☐ `group.extend(adventurer)`
- ■ `group.extend([adventurer])`
- ☐ `adventurer.extend(group)`
- ☐ `adventurer.extend([group])`
- ■ `group.append(adventurer)`
- ☐ `group.append([adventurer])`
- ☐ `adventurer.append(group)`
- ☐ `adventurer.append([group])`
- ☐ `group + adventurer[:]`
- ☐ `group[:] + adventurer`

iv. **(1.0 pt)** Fill in blank (4)

```
ring
```

v. **(1.5 pt)** Which of these could go in 5? (select one)

- ● `['Frodo']`
- ○ `'Frodo'`
- ○ `['Frodo', 'Aragorn']`
- ○ `'Aragorn'`
- ○ `['Frodo', 'Aragorn', None]`
- ○ `None`

vi. **(2.0 pt)** Fill in blank (6). You may only make one call expression and may not use `or`, `and`, `if`, or `else`.

```
gandalf("Aragorn")
```

vii. **(1.0 pt)** Fill in blank (7)

```
axe
```

viii. (1.0 pt) Which of these could go in 8? (select all that apply)

- [x] `gandalf(sword)`
- [ ] `gandalf(sword.append(None))`
- [ ] `gandalf(group)`
- [ ] `gandalf(adventurer)`
- [x] `gandalf(None)`
- [ ] `sword('Aragorn')`
- [ ] `sword(None)`

3. (13.0 points) Digit Widgets

In this question, we will be testing your understanding of both iterative and recursive solutions.

Hint: `pow(b, n)` raises `b` to the power of `n`. For example, `pow(10, 3)` is `1000`.

(a) (5.0 points)

Implement the recursive version of `digit_widget` which takes digit `d`. It returns a function with positive integer parameter `n` that removes all instances of `d` from `n`. **You may not use `str` or `repr` or `[` or `]` or `for`.**

```
def digit_widget(d):
    """
    Given a digit D, returns a function which accepts positive integer N that removes
    all instances of D from N. If there are no digits remaining in N, return 0.
    >>> remove_5s = digit_widget(5)
    >>> remove_5s(1234512345)
    12341234
    >>> remove_5s(55555)
    0
    >>> remove_0s = digit_widget(0)
    >>> remove_0s(102001)
    121
    >>> remove_0s(900)
    9
    >>> remove_0s(0)
    0
    """
    def remove_digit(n):
        if _________:
              (a)
            return _________
                       (b)
        if n % 10 == _________:
                         (c)
            return _________
                       (d)
        return _________
                   (e)
    return _________
              (f)
```

i. (0.5 pt) Fill in blank (a).

- ◯ `n == d`
- ● `n == 0`
- ◯ `n % 10 == 0`
- ◯ `n > 0`
- ◯ `n >= 0`
- ◯ `n < 0`

ii. **(0.5 pt)** Fill in blank (b)

- (●) `0`
- ( ) `1`
- ( ) `d`
- ( ) `n - 1`
- ( ) `pow(10, d)`
- ( ) `pow(10, n)`

iii. **(0.5 pt)** Fill in blank (c)

- ( ) `0`
- ( ) `1`
- (●) `d`
- ( ) `n - 1`
- ( ) `pow(10, d)`
- ( ) `pow(10, n)`

iv. **(1.0 pt)** Fill in blank (d)

- ( ) `remove_digit(n % d)`
- (●) `remove_digit(n // 10)`
- ( ) `remove_digit(n % 10)`
- ( ) `remove_digit(n - 1)`
- ( ) `remove_digit(n % pow(10, d))`
- ( ) `remove_digit(n // pow(10, d))`

v. **(2.0 pt)** Fill in blank (e).

```
remove_digit(n//10) * 10 + n % 10
```

vi. **(0.5 pt)** Fill in blank (f).

```
remove_digit
```

(b) (5.0 points)

Implement the iterative version of `digit_widget`, `digit_widget_iter`, which takes digit `d`. It returns a function with positive integer parameter `n` that removes all instances of `d` from `n`. **You may not use `str` or `repr` or `[` or `]` or `for`.**

```
def digit_widget_iter(d):
    """
    Given a digit D, returns a function which accepts positive integer N that removes
    all instances of D from N. If there are no digits remaining in N, return 0.
    >>> remove_5s = digit_widget_iter(5)
    >>> remove_5s(1234512345)
    12341234
    >>> remove_5s(55555)
    0
    >>> remove_0s = digit_widget_iter(0)
    >>> remove_0s(102001)
    121
    >>> remove_0s(900)
    9
    >>> remove_0s(0)
    0
    """
    def remove_digit(n):
        result, i = 0, 0
        while _________:
                  (a)
            if _________:
                   (b)
                result += _________
                               (c)

                _________
                   (d)

            _________
               (e)
        return result
    return _________
              (f)
```

i. (0.5 pt) Fill in blank (a).

- ◯ `n == d`
- ◯ `n == 0`
- ◯ `n % 10 == 0`
- ● `n > 0`
- ◯ `n >= 0`
- ◯ `n < 0`

ii. **(0.5 pt)** Fill in blank (b)

- ◯ `n % 10 == 0`
- ◯ `n // 10 != 0`
- ◯ `n % 10 == d`
- ● `n % 10 != d`
- ◯ `n % pow(10, d) == 0`
- ◯ `n // 10 == pow(10, d)`

iii. **(2.0 pt)** Fill in blank (c)

```
pow(10, i) * n % 10
```

iv. **(0.5 pt)** Fill in blank (d).

- ◯ `n = n % pow(10, i)`
- ◯ `i = pow(10, i)`
- ● `i += 1`
- ◯ `n = n % 10`
- ◯ `n = n // 10`
- ◯ `n -= 1`

v. **(1.0 pt)** Fill in blank (e).

- ◯ `n = n % pow(10, i)`
- ◯ `i = pow(10, i)`
- ◯ `i += 1`
- ◯ `n = n % 10`
- ● `n = n // 10`
- ◯ `n -= 1`

vi. **(0.5 pt)** Fill in blank (f).

```
remove_digit
```

(c) (3.0 points)

Implement `digit_machine`, which takes a positive integer `n`. It returns `n` where all instances of 4 and 8 have been removed. **You may not use `str` or `repr` or `[` or `]` or `for`.** Assume `digit_widget` is implemented correctly (and you may use it).

```
def digit_machine(n):
    """
    Given an integer N, return a modified N such that all instances of 4 and 8 have been removed.
    >>> digit_machine(484848)
    0
    >>> digit_machine(123456789)
    1235679
    >>> digit_machine(208)
    20
    """
    return _________
              (a)
```

i. (3.0 pt) Fill in blank (a).

```
return digit_widget(4)(digit_widget(8)(n))
```

4. (8.0 points) Goatda

This question will test the debugging skills you've learned in the class so far. We have included an implementation of the `goatda` function which accepts a single argument function `lamb` and a non-negative integer `n`. It returns a single argument function that applies `lamb` to the argument `n` times. However, **the included implementation is buggy**!

```
def goatda(lamb, n):
    """
    Implement goatda which accepts LAMB, a single argument function, and N the number of times
    to apply LAMB. Return a single argument function that applies LAMB to the argument N times.
    >>> add3 = goatda(lambda x: x+1, 3)
    >>> add3(10) # (((10 + 1) + 1) + 1)
    13
    >>> add3(2) # (((2 + 1) + 1) + 1)
    5
    >>> print2 = goatda(print, 2)
    >>> print2("hi") # print(print("hi"))
    hi
    None
    >>> identity = goatda(lambda x: x+3, 0)  # The function is applied 0 times
    >>> identity(0)
    0
    """
    if n == 0:                                # line 1
        return lambda x: x                    # line 2
    return lambda x: lamb(goatda(lamb, n-1)) # line 3
```

(a) After implementing `goatda`, we execute the following lines of code in the terminal.

```
>>> add3 = goatda(lambda x: x+1, 3)
>>> add3(10)
```

The call to `add3(10)` results in an error!

i. (2.0 pt) What kind of error does this result in?

- ○ `RecursionError: too much recursion`
- ● `TypeError: unsupported operand type(s)`
- ○ `TypeError: 'NoneType' object is not callable`
- ○ `IndentationError`
- ○ `IndexError`
- ○ `NameError`

ii. (2.0 pt) Let's modify the code, so that it no longer errors. Select how to replace the 3 lines, so that the function passes the doctests and no longer errors. Select `No change` if you do not want to modify the line. Indentation will remain the same.

How would you modify line 1?

- ○ `if n > 0:`
- ○ `if n < 0:`
- ○ `if n >= 0:`
- ○ `if n % 10 > 0:`
- ○ `if n % 10 >= 0:`
- ○ `if lamb(n) == 0:`
- ○ `if lamb(n):`
- ● `No change`

iii. (2.0 pt) How would you modify line 2?

- ○ `return lamb(n)`
- ○ `return n`
- ○ `return lambda x: n`
- ○ `return lambda x: 0`
- ○ `return lambda x: lamb(x)`
- ○ `return lambda x: lambda y: lamb(y)`
- ● `No change`

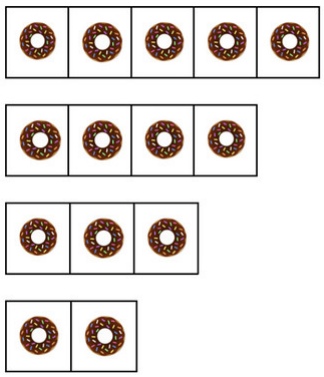
iv. (2.0 pt) How would you modify line 3?

- ○ `return goatda(lamb, lamb(n-1))`
- ○ `return lamb(goatda(lamb, n-1))`
- ○ `return lambda x: lamb(x) + goatda(lamb, n-1)`
- ○ `return lambda x: goatda(lamb(x), n-1)`
- ● `return lambda x: lamb(goatda(lamb, n-1)(x))`
- ○ `return lambda x: goatda(lamb, n-1)(n)`
- ○ `No change`

5. (13.0 points) Sweetness Overload

(a) (3.0 points)

Implement `donut_tower` which takes in the number of donuts `n` and positive integer `k`. It returns the number of layers one can fill using `n` donuts, given the smallest layer consists of `k` donuts and each subsequent layers consists of one more donut than the previous one. For instance, if `n` is 16 and `k` is 2, then you can create 4 layers (as shown below) using 14 donuts.



```
def donut_tower(n, k):
    """
    Given non-zero numbers N and K, returns the number of donut layers
    that can be formed using N donuts and K-donut smallest layer
    >>> donut_tower(0, 3)
    0
    >>> donut_tower(1, 1)
    1
    >>> donut_tower(10, 2) # 2 + 3 + 4 = 9 donuts
    3
    >>> donut_tower(20, 3)  # 3 + 4 + 5 + 6 = 18 donuts
    4
    """
    if _________:
          (a)
       return _________
                 (b)
    else:
       return 1 + _________
                    (c)
```

i. (1.0 pt) Fill in blank (a)

- (●) `n < k`
- ( ) `n >= k`
- ( ) `n > k`
- ( ) `n == 1`
- ( ) `n == 0`
- ( ) `n < 1`

ii. **(1.0 pt)** Fill in blank (b)

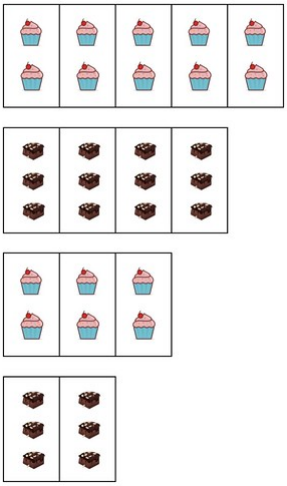
```
0
```

iii. **(1.0 pt)** Fill in blank (c)

- ◯ `max(donut_tower(n, k + 1), donut_tower(n - k, k))`
- ◯ `donut_tower(n, k + 1)`
- ● `donut_tower(n - k, k + 1)`
- ◯ `donut_tower(n - k, k)`
- ◯ `donut_tower(n - k + 1, k)`
- ◯ `donut_tower(n - 1, k)`

(b) (4.0 points)

Implement `alternating_tower`, which takes a positive integer `c`, the number of cupcakes, positive integer `b`, the number of brownies, and `k`, the size of the smallest layer that can hold `k` donuts. It returns the maximum numbers of dessert layers that can be formed using brownies and cupcakes. Since those items are smaller than donuts, each k-sized donut layer can hold twice as many cupcakes and thrice as many brownies. Also, a dessert tower has alternating layers of brownies and cupcakes, but **the first layer must always consist of brownies.** For instance, if `b` is 20, `c` is 20 and `k` is 2, then you can create a maximum of 4 layers (as shown below) using 18 brownies and 16 cupcakes.



```
def alternating_tower(b, c, k):
    """
    >>> alternating_tower(5, 4, 1) # 1st layer has 3 brownies, 2nd layer has 4 cupcakes
    2
    >>> alternating_tower(10, 15, 3) # 1st layer has 9 brownies, 2nd layer has 8 cupcakes
    2
    >>> alternating_tower(16, 6, 5) # 1st layer has 15 brownies
    1
    """
    def helper(b_item, c_item, curr_layer, is_cupcakes):
        if _________:
              (a)
            return 0
        elif is_cupcakes:
            return _________
                      (b)
        else:
            return 1 + helper(_________, c_item, curr_layer + 1, not is_cupcakes)
                                    (c)
    return helper(_________, c, k + 1, True)
                     (d)
```

i. **(1.0 pt)** Fill in blank (a)

- (●) `b_item < 0 or c_item < 0`
- ( ) `b_item`
- ( ) `b_item == 0 or c_item == 0`
- ( ) `b_item > 0 or c_item > 0`
- ( ) `c_item`
- ( ) `b_item < 0 and c_item < 0`
- ( ) `c_item * 2 == k or b_item * 3 == k`
- ( ) `b_item * 3 <= k`

ii. **(1.0 pt)** Fill in blank (b)

- ( ) `helper(b_item - 3 * curr_layer, c_item, curr_layer + 1, is_cupcakes) - 1`
- ( ) `1 + helper(b_item - 3 * curr_layer, c_item - 2 * curr_layer, curr_layer - 1, not is_cupcake`
- (●) `1 + helper(b_item, c_item - 2 * curr_layer, curr_layer + 1, not is_cupcakes)`
- ( ) `helper(b_item - 3 * curr_layer, c_item - 2 * curr_layer, curr_layer - 1, is_cupcakes)`
- ( ) `helper(b_item, b_item - 2 * curr_layer, curr_layer + 1, not is_cupcakes)`
- ( ) `helper(b_item - 3 * curr_layer, c_item, curr_layer + 1, not is_cupcakes) + helper(b_item, c_item - 2 * curr_layer, curr_layer + 1, is_cupcakes)`

iii. **(1.0 pt)** Fill in blank (c)

- ( ) `b_item`
- ( ) `b_item - 3 * (curr_layer + 1)`
- (●) `b_item - 3 * curr_layer`
- ( ) `3 * b_item`
- ( ) `b_item + c_item * 3 * curr_layer`
- ( ) `b_item // 3 * (curr_layer + 1)`
- ( ) `b_item // 3 * curr_layer`
- ( ) `b_item - 3 // curr_layer`

iv. **(1.0 pt)** Fill in blank (d)

```
b - 3 * k
```

(c) (6.0 points)

Implement `sweetest_tower`, which takes a positive integer `c`, the number of cupcakes, positive integer `b`, the number of brownies, and `k`, the size of the smallest layer that can hold `k` donuts. It returns the maximum number of dessert layers that can be formed using brownies and cupcakes. As previously stated, since those items are smaller than donuts, each k-sized donut layer can hold twice as many cupcakes and thrice as many brownies. **Unlike `alternating_tower`, layers can be in any order.**

```
def sweetest_tower(b, c, k):
    """
    >>> sweetest_tower(0, 2, 1) # 1st layer with 2 cupcakes
    1
    >>> sweetest_tower(5, 4, 4)
    0
    >>> sweetest_tower(17, 19, 3) # layer 1 has 6 cupcakes, 2nd has 8 cupcakes, 3rd has 15 brownie
    3
    >>> sweetest_tower(30, 8, 4)    # 1st layer has 12 brownies, 2nd has 15 brownies
    2
    """
    with_brownies, with_cupcakes = 0,0
    if _________:
          (a)
        with_brownies = _________
                            (b)
    if _________:
          (c)
        with_cupcakes = _________
                            (d)
    return _________(with_brownies, with_cupcakes)
              (e)
```

i. (1.0 pt) Fill in blank (a)

- (●) `b >= 3 * k`
- ( ) `b > 3 * k`
- ( ) `b`
- ( ) `b == 3 * k`
- ( ) `b > c`
- ( ) `b // 3 < k`
- ( ) `b // 3 == k`
- ( ) `b >= 0`

ii. **(1.0 pt)** Fill in blank (b)

- ○ `1 + sweetest_tower(b // 3 - k, c, k - 1)`
- ○ `sweetest_tower(b - 3 * k, c, k + 1)`
- ● `1 + sweetest_tower(b - 3 * k, c, k + 1)`
- ○ `sweetest_tower(b - 3 * k, c, k - 1)`
- ○ `3 * sweetest_tower(b, c - 2 * k, k + 1)`
- ○ `3 + sweetest_tower(b, c - 2 * k, k + 1)`
- ○ `sweetest_tower(b - 3 // k, c - 2 * k, k + 1)`

iii. **(1.0 pt)** Fill in blank (c)

```
c >= 2 * k
```

iv. **(1.0 pt)** Fill in blank (d)

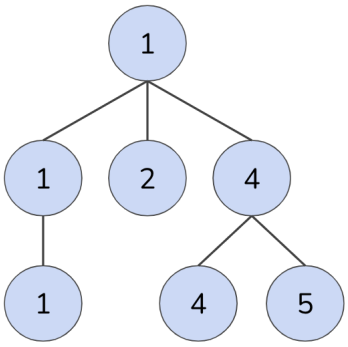
```
1 + sweetest_tower(b, c - 2*k, k + 1)
```

v. **(1.0 pt)** Fill in blank (e)

```
max
```

6. (12.0 points) All Treeils Lead to Rome

Definition: A **Treeil** is a tree where *exactly* one of the child node(s) of each non-leaf node has a label equal to the label of the node. Alternatively, a treeil is a tree where every node has a singular path from the node to a leaf consisting entirely of nodes with the same label. Here is an example of a valid treeil:



(a) (7.0 points) Treeil Trial

Implement `is_treeil` which takes in a tree `t` and determines whether `t` is a treeil.

```
def is_treeil(t):
    """
    Returns whether a tree is a treeil. A treeil is a tree where exactly one of
    the branches of each non-leaf node has a label equal to the label of the node.
    >>> t1 = tree(1)
    >>> is_treeil(t1)
    True
    >>> t2 = tree(1, [tree(2), tree(3)])
    >>> is_treeil(t2)
    False
    >>> t3 = tree(1, [tree(1, [tree(1), tree(3)]), tree(2)])
    >>> is_treeil(t3)
    True
    >>> t4 = tree(1, [tree(1, [tree(1), tree(1)]), tree(2)])
    >>> is_treeil(t4)
    False
    >>> t5 = tree(2, [tree(3, [tree(1)]), tree(2, [tree(2)])])
    >>> is_treeil(t5)
    False
    """
    if _________:
          (a)
        return True
    else:
        match_one = _________([b for b in branches(t) if _________]) == 1
                       (b)                                  (c)
        return _________ and _________([_________ for b in branches(t)])
                  (d)           (e)        (f)
```

i. **(1.0 pt)** Fill in blank (a). Select **all** that apply.

- ☐ `True`
- ☐ `False`
- ■ `is_leaf(t)`
- ☐ `branches(t)`
- ■ `not branches(t)`
- ☐ `label(t)`
- ☐ `not label(t)`
- ■ `branches(t) == []`

ii. **(1.0 pt)** Fill in blank (b).

- ○ `sum`
- ○ `max`
- ○ `min`
- ● `len`
- ○ `any`
- ○ `all`

iii. **(2.0 pt)** Fill in blank (c).

```
label(t) == label(b)
```

iv. **(1.0 pt)** Fill in blank (d).

```
match_one
```

v. **(1.0 pt)** Fill in blank (e).

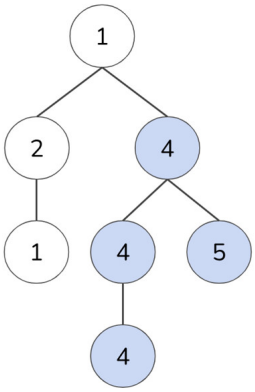
- ○ `sum`
- ○ `max`
- ○ `min`
- ○ `len`
- ○ `any`
- ● `all`

vi. **(1.0 pt)** Fill in blank (f).

```
is_treeil(b)
```

(b) (5.0 points) Treeil Trail

Implement `max_treeil` which takes in a tree `t` and determines the highest number of nodes in a treeil that is a subtree of `t`. You may assume `is_treeil` is implemented correctly from the previous question. The following diagram displays a tree `t`, which is not a treeil. The treeil subtree of `t` with the highest number of nodes is highlighted. Calling `max_treeil` on this tree `t` should give a result of `4`.




```
def max_treeil(t):
    """
    Given a tree (that is not necessarily a treeil), returns
    the highest number of nodes in a treeil in the tree.
    >>> t1 = tree(1)
    >>> max_treeil(t1)
    1
    >>> t2 = tree(1, [tree(2), tree(3)])
    >>> max_treeil(t2)
    1
    >>> t3 = tree(1, [tree(1, [tree(1), tree(3)]), tree(2)])
    >>> max_treeil(t3)
    5
    >>> t4 = tree(1, [tree(1, [tree(1), tree(1)]), tree(2)])
    >>> max_treeil(t4)
    1
    >>> t5 = tree(2, [tree(3, [tree(1)]), tree(2, [tree(2)])])
    >>> max_treeil(t5)
    2
    """
    if _________:
          (a)
        return 1
    elif _________:
            (b)
        return 1 + _________
                      (c)
    else:
        return max([_________ for b in branches(t)])
                       (d)
```

i. **(1.0 pt)** Fill in blank (a).

- ○ `is_treeil(t)`
- ○ `not is_treeil(t)`
- ● `is_leaf(t)`
- ○ `is_treeil(t) or is_leaf(t)`
- ○ `not is_treeil(t) and is_leaf(t)`

ii. **(1.5 pt)** Fill in blank (b).

```
is_treeil(t)
```

iii. **(1.0 pt)** Fill in blank (c).

- ○ `max([max_treeil(b) for b in branches(t)])`
- ○ `len([max_treeil(b) for b in branches(t)])`
- ● `sum([max_treeil(b) for b in branches(t)])`
- ○ `sum([b for b in branches(t) if is_treeil(b)])`
- ○ `len([b for b in branches(t) if is_treeil(b)])`

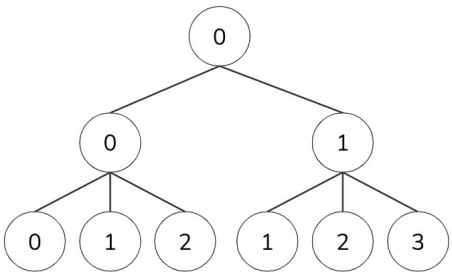
iv. **(1.5 pt)** Fill in blank (d).

```
max_treeil(b)
```

(c) (0.0 points) Treeil Treeatment (A+ Question)

This A+ question is not worth any points. This can only affect your course grade if you have a high A and might receive an A+. Finish the rest of the exam first!

Implement `make_treeil` which takes in a non-negative integer `n` and returns a treeil of depth `n`. Each node of the treeil has exactly one more child than its parent node. The first branch of a node will have value equal to its parent. Subsequent branches will have a label of exactly one more than the previous branch. The root has a label of 0 and has exactly two children for a tree of depth 1. Here's an example of a treeil of depth 2.




```
def make_treeil(n):
    """
    Returns a treeil of depth n where each node has one more child than its parent.
    The first branch of a node will have label equal to its parent with subsequent
    branches having label of exactly one more than the previous branch.
    >>> t2 = make_treeil(2)
    >>> print_tree(t2)
    0
      0
        0
        1
        2
      1
        1
        2
        3
    """
    def treeil_tracker(k, parent):
        if n == k:
            return tree(parent)
        return _________
                   (a)
    return treeil_tracker(0, 0)
```

i. (0.0 pt) Fill in blank (a).

```
tree(parent, [treeil_tracker(k+1, i) for i in range(parent,
parent+k+2)])
```

No more questions.

CS 61A Structure and Interpretation of Computer Programs
Summer 2024

Midterm Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign your name to confirm that all work on this exam will be your own.

The penalty for academic misconduct on an exam is an F in the course.

1. (12.0 points) Generiterator

Assume the following code has been executed. No error occurs when executing this code block.

```
def f(x):
    yield from map(lambda x: x[::-1], x)

def next_next(i):
    print(next(i))
    return next(i)

def generiterator1(s):
    yield f(s)
    yield from f(s)
    print('warmed up!')

def generiterator2(s):
    while s:
        yield f(s)
        yield iter(s[0])
        s = s[1:]
    print('generiterating complete!')

g_str = generiterator1(['i', '<3', '61a', '!'])
g_num = generiterator2([[1], [1, 2], [1, 2, 3]])
```

`What Would Python Display`? Write the output displayed by evaluating each expression below. If an error occurs, write `"Error"`, but include all output displayed before the error. If evaluation would run forever, write `"Forever"`. To display an iterator object, write `"Iterator"`. To display a generator object, write `"Generator"`.

Assume the expressions are evaluated in order in the same interactive session, and so evaluating an earlier expression may affect the result of a later one.

Hint 1: Draw it out!

Hint 2: When a string is passed into `print`, no quotation marks are displayed. When a string is the value of an expression evaluated by the interpreter, quotation marks *are* displayed. A list of strings always displays quotation marks.

(a) (5.0 points) Jester

i. (1.0 pt) `next(g_str)`

Generator

ii. (1.0 pt) `next(g_str)`

```
'i'
```

iii. (1.0 pt) `next_next(g_str)`

```
3<
'a16'
```

iv. (2.0 pt) `next_next(g_str)`

```
!
warmed up!
Error
```

(b) (7.0 points) Genome

i. (1.0 pt) `list(next(g_num))`

```
[[1], [2, 1], [3, 2, 1]]
```

ii. (2.0 pt) `next_next(next(g_num))`

```
1
Error
```

iii. (2.0 pt) `next_next(next(g_num))`

```
[2, 1]
[3, 2, 1]
```

iv. (2.0 pt) `len(list(g_num))`

```
generiterating complete!
3
```

2. (8.0 points) Conveyor Belt

Draw the environment diagram for the code below using box and pointer notation and then answer the questions that follow. Your diagram will not be graded.

Hint: If you pass an argument into `pop`, it uses the argument as the index of the element to remove. If you do not pass an argument into `pop`, it defaults to popping the last element.

```
def conveyor_belt(s):
    k = -1
    while s[k]:
        k = s[0].pop(chain_pop(s[0]))
        s = s[k:]
        if s[0] == box:
            s.append(box or s.append(box))
        else:
            s.append(s.extend(s[:1]))
    return s

def chain_pop(s):
    return s.pop(s.pop(s.pop()))

box = [1, 1, 0, 0]

result = conveyor_belt([box, box[:]])
```

Blank Space for Diagram:

(a) (4.0 pt) What would be displayed by evaluating `print(result)` in the Global frame?

```
[[], None, [], None]
```

(b) (0.5 pt) What is the result of evaluating `box in result` in the Global frame?

- (●) True
- ( ) False

(c) (0.5 pt) What is the result of evaluating `[] in result` in the Global frame?

- (●) True
- ( ) False

(d) (0.5 pt) What is the result of evaluating `None in result` in the Global frame?

- (●) True
- ( ) False

(e) (0.5 pt) What is the result of evaluating `result[0] == result[1]` in the Global frame?

- ( ) True
- (●) False
- ( ) Error

(f) (0.5 pt) What is the result of evaluating `result[0] is result[1]` in the Global frame?

- ( ) True
- (●) False
- ( ) Error

(g) (0.5 pt) What is the result of evaluating `result[0] == result[2]` in the Global frame?

- (●) True
- ( ) False
- ( ) Error

(h) (0.5 pt) What is the result of evaluating `result[0] is result[2]` in the Global frame?

- ( ) True
- (●) False
- ( ) Error

(i) (0.5 pt) What is the result of evaluating `not result[0]` in the Global frame?

- (●) True
- ( ) False
- ( ) Error

3. (4.0 points) Tree Sum

Laryn, Charlotte, and Raymond are working together on a CS 61A problem, but they can't agree on a solution. Help them determine which implementation(s) are correct, if any.

`tree_sum` is a function that takes in a tree `t` and a one-argument function `cond` and returns the sum of all the labels, `x`, in `t` for which `cond(x)` returns `True`.

```
>>> t2 = tree(5, [tree(6), tree(7)])
>>> t1 = tree(3, [tree(4), t2])
>>> tree_sum(t1, lambda x: x >= 4)
22
>>> tree_sum(t1, lambda x: x >= 6)
13
>>> tree_sum(t2, lambda x: x >= 6)
13
```

```
# Attempt 1
def tree_sum1(t, cond):
    if cond(label(t)):
        total = label(t)
    else:
        total = 0
    for b in branches(t):
        if cond(label(b)):
            total += tree_sum1(b, cond)
    return total
```

```
########################################
```

```
# Attempt 2
def tree_sum2(t, cond):
    if cond(label(t)):
        total = label(t)
    else:
        total = 0
    for b in branches(t):
        total += tree_sum2(b, cond)
    return total
```

```
########################################
```

```
# Attempt 3
def tree_sum3(t, cond):
    total = label(t)
    for b in branches(t):
        if cond(label(b)):
            total += tree_sum3(b, cond)
    return total
```

(a) (4.0 pt) Which implementation(s) are correct, if any? **Select all** that apply.

- ☐ `tree_sum1` is correct
- ■ `tree_sum2` is correct
- ☐ `tree_sum3` is correct
- ☐ `tree_sum1`, `tree_sum2`, and `tree_sum3` are all incorrect

4. (17.0 points) Add Consecutive

Implement `add_consecutive`, a function that takes in a positive integer `n` and returns a list of integers. Each element of the returned list is the sum of adjacent consecutive digits in `n`. Two digits are adjacent if they are directly beside each other. Two digits are consecutive if the absolute difference between them is exactly 1. Two of the same digit are **not** considered consecutive.

You may not use `str` or `repr` or `[` or `]` or `for`.

(a) (9.0 points) Iterative Add Consecutive

Implement `add_consecutive` iteratively.

```
def add_consecutive(n):
    """
    >>> add_consecutive(123456789)
    [45]
    >>> add_consecutive(567231) # [5 + 6 + 7, 2 + 3, 1]
    [18, 5, 1]
    >>> add_consecutive(111) # repeated digits are not consecutive
    [1, 1, 1]
    >>> add_consecutive(1235689)
    [6, 11, 17]
    >>> add_consecutive(3216598)
    [6, 11, 17]
    >>> add_consecutive(13579)
    [1, 3, 5, 7, 9]
    >>> add_consecutive(12321) # [1 + 2 + 3 + 2 + 1]
    [9]
    >>> add_consecutive(4)
    [4]
    >>> add_consecutive(105)
    [1, 5]
    >>> add_consecutive(135797531)
    [1, 3, 5, 7, 9, 7, 5, 3, 1]
    """
    result = []

    subtotal = 0

    while __________:
              (a)

        rest, last = n // 10, n % 10

        subtotal = __________
                       (b)
        if __________:
               (c)
            result = __________
                          (d)
            subtotal = __________
                             (e)
        n = rest

    result = __________
                 (f)
    return result
```

i. (2.0 pt) Select all of the expressions below that could fill in blank (a).

- ☐ `n`
- ☐ `n > 0`
- ☐ `n >= 0`
- ☐ `n != 0`
- ☐ `n > 10`
- ■ `n >= 10`
- ■ `n > 9`
- ☐ `n >= 9`
- ■ `n // 10`
- ☐ `n % 10`

ii. (1.0 pt) Fill in blank (b).

```
subtotal + last
```

iii. (2.0 pt) Fill in blank (c).

```
abs((rest % 10) - last) != 1
```

iv. (1.0 pt) Fill in blank (d).

```
[subtotal] + result
```

v. (1.0 pt) Select all of the expressions below that could fill in blank (e).

- ■ `0`
- ☐ `rest`
- ☐ `last`
- ☐ `n`
- ☐ `subtotal`
- ☐ `subtotal + rest`
- ☐ `subtotal + last`
- ☐ `subtotal + n`

vi. (2.0 pt) Fill in blank (f).

```
[subtotal + n] + result
```

(b) (8.0 points) Recursive Add Consecutive

Implement `add_consecutive` recursively.

```
def add_consecutive(n):
    """
    >>> add_consecutive(123456789)
    [45]
    >>> add_consecutive(567231) # [5 + 6 + 7, 2 + 3, 1]
    [18, 5, 1]
    >>> add_consecutive(111) # repeated digits are not consecutive
    [1, 1, 1]
    >>> add_consecutive(1235689)
    [6, 11, 17]
    >>> add_consecutive(3216598)
    [6, 11, 17]
    >>> add_consecutive(13579)
    [1, 3, 5, 7, 9]
    >>> add_consecutive(12321) # [1 + 2 + 3 + 2 + 1]
    [9]
    >>> add_consecutive(4)
    [4]
    >>> add_consecutive(105)
    [1, 5]
    >>> add_consecutive(135797531)
    [1, 3, 5, 7, 9, 7, 5, 3, 1]
    """
    def helper(n, subtotal):
        rest, last = n // 10, n % 10

        subtotal = __________
                       (a)
        if __________:
               (b)
            return __________
                       (c)
        if __________:
               (d)
            return helper(__________)
                              (e)
        else:
            return __________
                       (f)
    return helper(n, __________)
                          (g)
```

i. (1.0 pt) Select all of the expressions below that could fill in blank (a).

- ☐ `0`
- ☐ `rest`
- ☐ `last`
- ☐ `n`
- ☐ `subtotal`
- ☐ `subtotal + rest`
- ■ `subtotal + last`
- ☐ `subtotal + n`

ii. (1.0 pt) Fill in blank (b).

```
n < 10
```

iii. (1.0 pt) Fill in blank (c).

```
[subtotal]
```

iv. (1.0 pt) Fill in blank (d).

```
abs((rest % 10) - last) == 1
```

v. (1.0 pt) Fill in blank (e).

```
rest, subtotal
```

vi. (2.0 pt) Fill in blank (f).

```
helper(rest, 0) + [subtotal]
```

vii. (1.0 pt) Fill in blank (g).

```
0
```

5. (7.0 points) Combine Tree

Implement `combine_tree`, a function that takes in a tree `t` and a two-argument function `f` and returns a new tree where the label of any node `b` is the result of calling `f` on the labels of all the nodes in the subtree rooted at `b` (including the label of `b` itself).

You may assume that `f` is an associative function. That is, `f(x, y) == f(y, x)` for all `x` and `y`.

```
def combine_tree(t, f):
    """
    >>> from operator import add, mul
    >>> t = tree(1, [tree(2), tree(3, [tree(4), tree(5), tree(6)])])
    >>> sum_tree = combine_tree(t, add)
    >>> print_tree(sum_tree)
    21
        2
        18
            4
            5
            6
    >>> product_tree = combine_tree(t, mul)
    >>> print_tree(product_tree)
    720
        2
        360
            4
            5
            6
    >>> max_tree = combine_tree(t, max)
    >>> print_tree(max_tree)
    6
        2
        6
            4
            5
            6
    """
    if is_leaf(t):

        return __________
                   (a)
    total = __________
              (b)
    new_branches = []

    for b in branches(t):

        new_b = __________
                    (c)
        new_branches.append(new_b)

        total = __________
                    (d)
    return tree(total, new_branches)
```

(a) (1.0 pt) Fill in blank (a).

- ● `t`
- ○ `label(t)`
- ○ `0`
- ○ `1`

(b) (1.0 pt) Select all of the expressions below that could fill in blank (b).

- ☐ `t`
- ■ `label(t)`
- ☐ `0`
- ☐ `1`
- ☐ `f(label(t))`
- ☐ `f(label(t), label(t))`
- ☐ `f(0, label(t))`
- ☐ `f(1, label(t))`
- ☐ `combine_tree(t, f)`
- ☐ `f(sum([combine_tree(b, f) for b in branches(t)]), label(t))`
- ☐ `sum(map(f, [combine_tree(b, f) for b in branches(t)]))`

(c) (2.0 pt) Fill in blank (c).

- ○ `t`
- ○ `b`
- ○ `f(t)`
- ○ `f(b)`
- ○ `f(label(t))`
- ○ `f(label(b))`
- ○ `f(label(t), label(b))`
- ○ `f(label(b), label(t))`
- ○ `combine_tree(t, f)`
- ● `combine_tree(b, f)`

(d) (3.0 pt) Fill in blank (d).

```
f(total, label(new_b))
```

6. (11.0 points) Multi Compose

Implement `multi_compose`, a function that takes in a list of functions and two integers `x` and `y`. It returns a composite function that on input `x` outputs `y` by applying a subsequence of functions in the input list. Functions can only be applied in the order in which they appear in the list. That is, the ith-indexed function must be applied before the i+1th-indexed function. Each function can be used 0 or 1 times.

If no such composite function exists, return `None`. If more than one such composite function exists, return any one of them.

You may assume `x != y`. Your solution must use `safe_compose`, which composes two functions together when called with two functions as arguments and returns `None` when called with `None` as an argument.

```
def safe_compose(f, g):
    """
    >>> composed = safe_compose(lambda x: x + 1, lambda x: x * 2)
    >>> composed(5)
    11
    >>> safe_compose(lambda x: x, None) is None and safe_compose(None, lambda x: x) is None
    True
    """
    if f is None or g is None:
        return None
    def composed(x):
        return f(g(x))
    return composed


def multi_compose(funcs, x, y):
    """
    >>> add_one, double = lambda x: x + 1, lambda x: x * 2
    >>> sub_three, square = lambda x: x - 3, lambda x: x ** 2
    >>> list_of_funcs = [add_one, double, sub_three, square]
    >>> double_then_square = multi_compose(list_of_funcs, 3, 36)
    >>> double_then_square
    Function
    >>> double_then_square(1) # (1 * 2) ** 2 --> 4
    4
    >>> square_then_double = multi_compose(list_of_funcs, 3, 18)
    >>> square_then_double # None
    >>> all_funcs = multi_compose(list_of_funcs, 3, 25)
    >>> all_funcs(1) # (((1 + 1) * 2) - 3) ** 2 --> 1
    1
    >>> double = multi_compose(list_of_funcs, 50, 100)
    >>> double(1) # 1 * 2 --> 2
    2
    >>> sub_two = multi_compose(list_of_funcs, 50, 48)
    >>> sub_two(2) # 2 - 2 --> 0
    0
    >>> double_then_sub = multi_compose(list_of_funcs, 50, 97)
    >>> double_then_sub(1) # (1 * 2) - 3 --> -1
    -1
    >>> negate = multi_compose(list_of_funcs, 100, -100)
    >>> negate # None
    """
```

```
if x == y:
    return lambda x: __________
                         (a)
if __________:
       (b)
    return None

return __________ or multi_compose(__________)
          (c)                         (d)
```

(a) (2.0 pt) Fill in blank (a).

```
x
```

(b) (1.0 pt) Fill in blank (b).

```
not funcs
```

(c) (6.0 pt) Fill in blank (c).

```
safe_compose(multi_compose(funcs[1:], funcs[0](x), y), funcs[0])
```

(d) (2.0 pt) Fill in blank (d).

```
funcs[1:], x, y
```

7. (5.0 points) Memoized Fibonacci Tree

Recall the Fibonacci sequence from lecture. It is defined as follows:

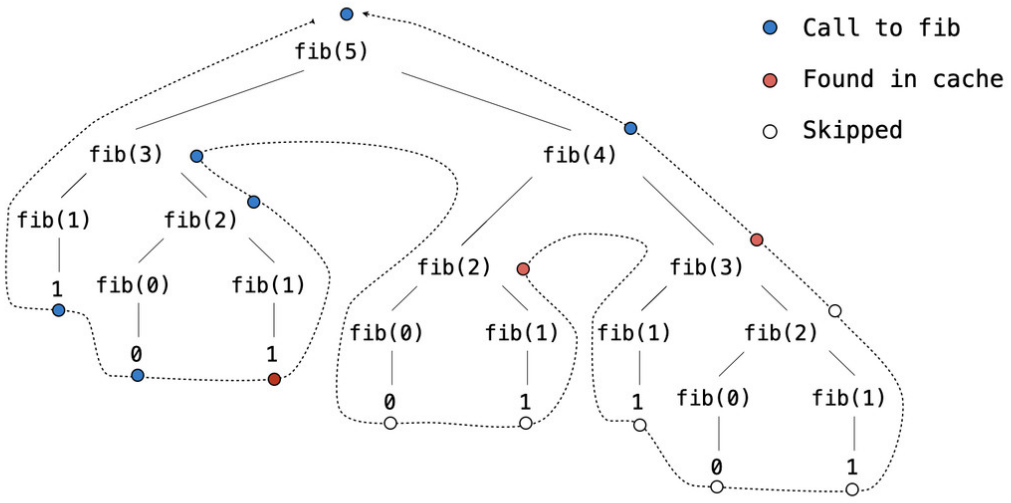
```
         0 if n == 0
fib(n) = 1 if n == 1
         fib(n - 1) + fib(n - 2) else
```

A **Fibonacci Tree** is a tree where each label is a Fibonacci number and each non-leaf node has exactly two children: the two Fibonacci numbers that appear directly before it in the Fibonacci sequence.

The `fib_tree` function below takes in a nonnegative integer `n` and returns a Fibonacci Tree that has root label `fib(n)`. Implement `fib_tree_memo`, a memoized version of `fib_tree`.

```
def fib_tree(n):
    """
    >>> print_tree(fib_tree(5))
    5
        2
            1
            1
                0
                1
        3
            1
                0
                1
            2
                1
                1
                    0
                    1
    """
    # IMPLEMENTATION OMITTED
```

Here is a visual indicating how the call diagram of `fib_tree` should change once you memoize it in `fib_tree_memo`.



```
def fib_tree_memo(n):
    """
    >>> print_tree(fib_tree_memo(5))
    5
        2
            1
            1
                0
                1
        3
            1
            2
    """
    def helper(n, cache):
        if __________:
               (a)
            return tree(__________)
                              (b)
        if n <= 1:

            return tree(n)

        b0, b1 = helper(n - 2, cache), helper(n - 1, cache)

        fib_num = __________
                      (d)
        __________ = fib_num
            (e)
        return tree(fib_num, [b0, b1])

    return helper(n, {})
```

(a) **(1.0 pt)** Fill in blank (a).

```
n in cache
```

(b) **(1.0 pt)** Fill in blank (b).

```
cache[n]
```

(c) **(1.0 pt)** Fill in blank (d).

```
label(b0) + label(b1)
```

(d) **(1.0 pt)** Fill in blank (e).

```
cache[n]
```

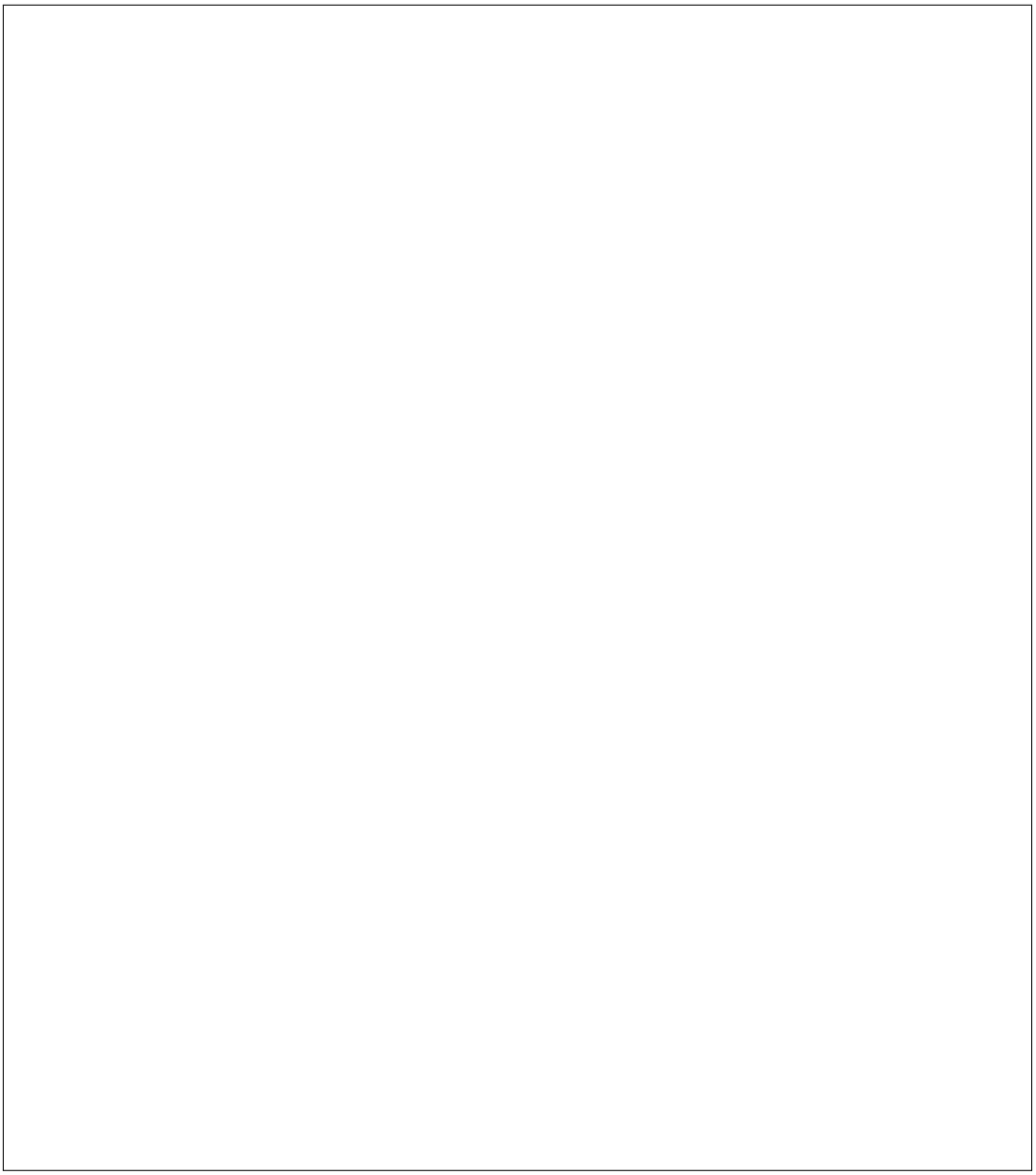
(e) (1.0 pt) What is the order of growth of the run time of `fib_tree_memo` with respect to `n`?

- ○ Constant, $\Theta(1)$, $O(1)$
- ○ Logarithmic, $\Theta(\log n)$, $O(\log n)$
- ● Linear, $\Theta(n)$, $O(n)$
- ○ Quadratic, $\Theta(n^2)$, $O(n^2)$
- ○ Exponential, $\Theta(b^n)$, $O(b^n)$

8. (0.0 points) Just for Fun

This is not for points and will not be graded.

(a) **Optional**: Draw your favorite spot on campus!



No more questions.

CS 61A Structure and Interpretation of Computer Programs
Summer 2025

Midterm Solutions

INSTRUCTIONS

This is your exam. Complete it either at exam.cs61a.org or, if that doesn't work, by emailing course staff with your solutions before the exam deadline.

This exam is intended for the student with email address `<EMAILADDRESS>`. If this is not your email address, notify course staff immediately, as each exam is different. Do not distribute this exam PDF even after the exam ends, as some students may be taking the exam in a different time zone.

For questions with **circular bubbles**, you should select exactly *one* choice.

- ◯ You must choose either this option
- ◯ Or this one, but not both!

For questions with **square checkboxes**, you may select *multiple* choices.

- ☐ You could select this choice.
- ☐ You could select this one too!

You may start your exam now. Your exam is due at <DEADLINE> Pacific Time. Go to the next page to begin.

Preliminaries

You can complete and submit these questions before the exam starts.

(a) What is your full name?

(b) What is your student ID number?

(c) What is your @berkeley.edu email address?

(d) Sign (or type) your name to confirm that all work on this exam will be your own. The penalty for academic misconduct on an exam is an F in the course.

1. (5.0 points) What Would Python Display?

```
1: a = [[13], 8]
2: b = a + [[2]]
3: b[1] -= 2
4: a[1] -= b[1]
5:
6: print(a)
7:
8: a[0] += [7]
9: b[b[2][0]] += [a.pop(1)]
10:
11: print(b)
12:
13: c = b[-1]
14: c.append(a[0])
15: c.append(b[0].extend([c[0]]))
16:
17: print(c)
```

Assume the expressions are evaluated in order in the same interactive session, and so evaluating an earlier expression may affect the result of a later one.

Hint: Draw it out!

(a) (1.0 pt) What is displayed by the call to `print` in line 6?

```
[[13], 2]
```

(b) (2.0 pt) What is displayed by the call to `print` in line 11?

- ○ `[[13], 2, [2]]`
- ● `[[13, 7], 6, [2, 2]]`
- ○ `[[13], 6, [2]]`
- ○ `[[13, 7], 2, [2]]`
- ○ `[[13, 7], 6, [4]]`
- ○ `[[13, 7], 6, [2, [2]]]`
- ○ `[[13], 6, [2, 2]]`

(c) (2.0 pt) What is displayed by the call to `print` in line 17?

- ○ `[2, 2, [13, 7], None]`
- ○ `[2, 2, [13], [2, 2]]`
- ○ `[2, 2, [13, 2], [13]]`
- ○ `[2, 2, [13], 2]`
- ● `[2, 2, [13, 7, 2], None]`
- ○ `[2, 2, [13, 7, 2], [None]]`

2. (10.0 points) Surf's Up!

Answer the questions about the code below. Use the free space or scratch paper to draw the diagram to help you answer the questions, however any drawn diagrams will not be graded.

```
1: def wave(twin):
2:     n = 3
3:     def twin():
4:         print(wave, n)
5:         return print('Shred') or n
6:         print('Wipeout')
7:     return twin()
8:
9: def twin():
10:     def twin(wave):
11:         while wave(twin):
12:             return print('Riptide')
13:         print('Tide')
14:     return twin
15:
16: twin = twin()(lambda twin: print('Twin Wave') or wave(twin))
```

If an error occurs, write `"Error"`. If a function is printed, write `"Function"`.

(a) (4.0 pt) What is printed after evaluating the above code?

Twin Wave
Function 3
Shred
Riptide

(b) (1.0 pt) How many frames are opened (not including the Global Frame)?

- ○ `3`
- ○ `4`
- ● `5`
- ○ `6`

(c) (2.0 pt) What is the parent frame of `lambda twin:` in line 16?

- ● `Global`
- ○ `f1`
- ○ `f2`
- ○ `None`

(d) (1.0 pt) How many calls are made to the function `wave` defined in line 1?

- ○ `0`
- ● `1`
- ○ `2`
- ○ `3`
- ○ `None`

(e) (2.0 pt) What is `twin` assigned to in the Global Frame?

`None`

3. (17.0 points) Python in python?

Definition. A **python** is a sequence of digits where the following are true:

- All digits are equal to some digit `d`
- All digits are two places apart (difference between indices equals 2)

In other words, a python is a sequence of digits where a digit `d` occurs in every other digit place, and the length of the python is the number of times `d` appears consecutively.

(a) (9.0 points)

Implement `python`, which takes in a number `n` and a digit `d`, and returns the length of the longest **python** in `n` where all digits equal `d`

Assume `n > 0`

```
def python(n, d):
    """
    Return the length of the longest python of digit d in integer n.
    >>> python(2125262, 2)      # positions of 2s: [0,2,4,6] -> all 2 apart -> length 4
    4
    >>> python(2020211202, 2)   # positions of 2s: [0,2,4,7,9] -> longest python is 3 (at 0,2,4)
    3
    >>> python(20211120202, 2)  # positions of 2s: [0,2,6,8,10] -> longest python is 3 (at 6,8,10)
    3
    >>> python(1010, 0)
    2
    >>> python(123456, 7)       # no 7 exists within n
    0
    """
    lengths = []
    _______:
      (a)
        copied_n = _______
                     (b)
        length = 0
        _______ and copied_n > 0:
          (c)
            length += 1
            _______
              (d)
        lengths._______
                  (e)
        _______
          (f)
    return max(lengths)
```

i. (1.0 pt) Fill in blank (a). Select all that apply.

- ☐ `while n // 10 < 0:`
- ■ `while n > 0:`
- ☐ `while n >= 10:`
- ☐ `while d == n:`

ii. **(1.0 pt)** Fill in blank (b).

```
n
```

iii. **(3.0 pt)** Fill in blank (c).

```
while copied_n % 10 == d
```

iv. **(1.0 pt)** Fill in blank (d).

- ◯ `n = n // 100`
- ◯ `n = copied_n // 100`
- ● `copied_n = copied_n // 100`
- ◯ `copied_n = n // 100`

v. **(2.0 pt)** Fill in blank (e). Select all that apply.

- ■ `insert(0,length)`
- ☐ `append([length])`
- ☐ `extend(list(length))`
- ■ `extend([length])`

vi. **(1.0 pt)** Fill in blank (f).

```
n = n // 10
```

(b) (8.0 points)

Implement `longest_python`, which takes in a number n and returns the digit and length for the digit with the longest **python** in n. This function explicity returns the digit, followed by its corresponding length.

If there is a tie for longest python, return the numerically greater digit.

Assume `python` from part (3a) is implemented correctly.

```
def longest_python(n):
    """
    Return the digit and length of the longest python for any digit d
    If there is a tie, return the larger digit.
    >>> longest_python(2324252)   # 2 has len 4, others have len 1
    (2, 4)
    >>> longest_python(1212121)   # 1 has len 4, 2 has len 3,
    (1, 4)
    >>> longest_python(56565656)  # 5 has len 4, 6 has len 4, 6 > 5
    (6, 4)
    >>> longest_python(67676565)  # 6 has len 4, 7 has len 2, 5 has len 2
    (6, 4)
    >>> longest_python(12345)     # all have len 1, return largest digit
    (5, 1)
    """
    d = {}
    for x in _______:
                (g)
        _______
          (h)
    key = _______
            (i)
    return d[key], _______
                     (j)
```

i. (1.0 pt) Fill in blank (g).

- ○ `n`
- ○ `range(n)`
- ○ `range(len(n))`
- ● `range(10)`

ii. (3.0 pt) Fill in blank (h). You may not use `for`, `in`, `if`, or `lambda` in your solution.

```
d[python(n,x)] = x
```

iii. (2.0 pt) Fill in blank (i). You may not use `for`, `in`, `if`, or `lambda` in your solution.

```
max(d) OR max(d.keys())
```

iv. (2.0 pt) Fill in blank (j).

`key` **OR** `max(d)`

4. (17.0 points) Valid Number

Definition. A number is valid if it satisfies either of the following critera:

- It has 2 or fewer digits, or
- Excluding the last two digits, for every digit, there is another digit 1 or 2 places to the right with the same **parity** (even or odd)

For example, 3412 is valid as 3 and 1 have the same parity, and 4 and 2 have the same parity

(a) (7.0 points)

Implement `is_valid`, which takes in a number `n` and returns True if `n` is valid, `False` otherwise.

Assume `n > 0`

```
def is_valid(n):
    """
    >>> is_valid(3412)   # 1 is two digits to the right of 3, 2 is two digits to the right of 4.
    True
    >>> is_valid(43)     # 2 or fewer digits
    True
    >>> is_valid(3443)   # No odd number within two digits of leftmost 3.
    False
    >>> is_valid(213)    # No even number to the right of 2.
    False
    """
    def helper(last, second, n):
        if _______:
             (a)
            return True
        curr = n % 10
        if _______ and _______:
             (b)          (c)
            return False
        return helper(_______)
                       (d)

    return helper(n % 10, n // 10 % 10, n // 100)
```

i. (1.0 pt) Fill in blank (a).

- (●) `n == 0`
- ( ) `n >= 0`
- ( ) `n < 0`
- ( ) `n == 1`

ii. (1.5 pt) Fill in blank (b).

- ( ) `curr % 2 == 0`
- ( ) `last % 2 == 0`
- (●) `curr % 2 != last % 2`
- ( ) `(curr + last) % 2 == 0`

iii. (1.5 pt) Fill in blank (c).

```
curr % 2 != second % 2 OR (curr + second) % 2 == 1
```

iv. (3.0 pt) Fill in blank (d).

```
second, curr, n // 10 OR second, n % 10, n // 10
```

(b) (10.0 points)

Implement `valid_subseq`, which takes in some number `n`, and returns the number of subsequences of n that are valid numbers.

A subsequence of a number is a combination of digits whose relative order remains the same as in the original number. In other words, a subsequence of a number is the number with some (or no) digits removed.

For example, 213 has the following subsequences: 2, 1, 3, 21, 23, 13, 213

Assume `is_valid` from part (4a) is implemented correctly.

Additionally, we have provided you with a function `reverse`, which takes a number and reverses its digits.

Hint True + True == 2

```
def valid_subseq(n):
    """
    Return the number of valid subsequences of n.
    >>> valid_subseq(12)    # possible subseqs: 1, 2, 12 - all are valid
    3
    >>> valid_subseq(123)
    7
    >>> valid_subseq(213)   # possible subseqs: 2, 1, 3, 21, 23, 13, 213 - 6 of which are valid.
    6
    >>> valid_subseq(3412)
    14
    """

    def reverse(x):
        """
        This function gives the reverse of a number
        >>> reverse(3412)
        2143
        """
        # IMPLEMENTATION NOT SHOWN

    def helper(num, rest):
        if _______ and rest == 0:
             (e)
            return _______
                      (f)
        elif _______:
               (g)
            return _______
                      (h)
        return helper(_______, _______) + helper(_______,_______)
                        (i)      (j)               (k)     (l)
    return helper(0, reverse(n))
```

i. (1.0 pt) Fill in blank (e).

- ◯ `n >= 0`
- ◯ `num > 0`
- ● `num == 0`
- ◯ `n < 0`

ii. **(1.0 pt)** Fill in blank (f).

- [x] `0`
- [ ] `1`
- [x] `num`
- [x] `rest`

The intended solution was `0`. However, since the `if` statement ensures that both `num` and `rest` are `0`, they are also valid options.

iii. **(1.0 pt)** Fill in blank (g).

```
rest == 0
```

iv. **(2.0 pt)** Fill in blank (h).

- [ ] `0`
- [ ] `1`
- [ ] `is_valid(rest)`
- [x] `is_valid(num)`

v. **(2.0 pt)** Fill in blank (i).

```
num * 10 + rest % 10
```

vi. **(1.0 pt)** Fill in blank (j).

```
rest // 10
```

vii. **(1.0 pt)** Fill in blank (k).

- [ ] `n`
- [x] `num`
- [ ] `rest`
- [ ] `num // 10`

viii. **(1.0 pt)** Fill in blank (l).

```
rest // 10
```

5. (15.0 points) Family Trees

(a) (7.0 points)

Implement `descendants`, which takes in a tree `t` and depth `d`, and returns a list of labels for all children at least depth `d` away from the root.

```
def descendants(t, d):
    """Return a list of all labels of descendants at least d levels away from the root node.

    >>> t1 = tree(1, [tree(2, [tree(3), tree(4)]), tree(5, [tree(6), tree(7)])])
    >>> print(t1)
    1
        2
            3
            4
        5
            6
            7
    >>> descendants(t1, 1)
    [2, 3, 4, 5, 6, 7]
    >>> descendants(t1, 2)
    [3, 4, 6, 7]
    >>> descendants(t1, 3)
    []
    """
    lst = []
    if _______:
         (a)

        _______
          (b)
    for b in branches(t):
        lst = _______
                (c)
    return _______
             (d)
```

i. (1.0 pt) Fill in blank (a).

- ○ `d == 0`
- ○ `d >= 0`
- ○ `d < 0`
- ● `d <= 0`

ii. (2.0 pt) Fill in blank (b). Select all that apply.

- ☐ `lst + label(t)`
- ☐ `lst.append(t)`
- ☐ `lst += [t]`
- ■ `lst.extend([label(t)])`

iii. (3.0 pt) Fill in blank (c).

```
lst + descendants(b, d-1)
```

iv. (1.0 pt) Fill in blank (d)

```
lst
```

(b) (8.0 points)

Definition. A tree is an ancestor if all paths from the root node to a depth `d` have strictly decreasing labels.

Implement `is_ancestor`, which takes in a tree `t` and a depth `d` and returns `True` if the tree is an ancestor, `False` otherwise.

```
def is_ancestor(t, d):
    """
    Return True if t is an ancestor with respect to a depth d, False otherwise.
    >>> t1 = tree(2, [tree(1)])
    >>> print_tree(t1)
    2
        1

    >>> is_ancestor(t1, 0)
    True
    >>> is_ancestor(t1, 1)    # 2 -> 1: strictly decreasing
    True

    >>> t2 = tree(4, [t1, tree(3, [tree(5), tree(3)])])
    >>> print_tree(t2)
    4
        2
            1
        3
            5
            3
    >>> is_ancestor(t2, 0)
    True     # 4
    >>> is_ancestor(t2, 1)    # 4 -> 2 and 4 -> 3 are both strictly decreasing
    True
    >>> is_ancestor(t2, 2)    # 4 -> 3 -> 5 and 4 -> 3 -> 3 are not strictly decreasing
    False
    """
    if _______:
         (e)
        return True
    for b in branches(t):
        if _______ _______ _______:
             (f)     (g)     (h)
            return _______
                      (i)
    return _______
             (j)
```

i. (1.0 pt) Fill in blank (e).

- (●) `d == 0`
- ( ) `d < 0`
- ( ) `d == 1`
- ( ) `d >= 0`

ii. **(2.0 pt)** Fill in blank (f). You may not use `and`, `or`, or `not`.

```
label(b) >= label(t)
```

iii. **(1.0 pt)** Fill in blank (g).

- ◯ `and`
- ● `or`
- ◯ `not`
- ◯ `==`

iv. **(2.0 pt)** Fill in blank (h).

```
not is_ancestor(b, d-1)
```

v. **(1.0 pt)** Fill in blank (i).

- ◯ `not b`
- ◯ `True`
- ◯ `is_ancestor(b, d-1)`
- ● `False`

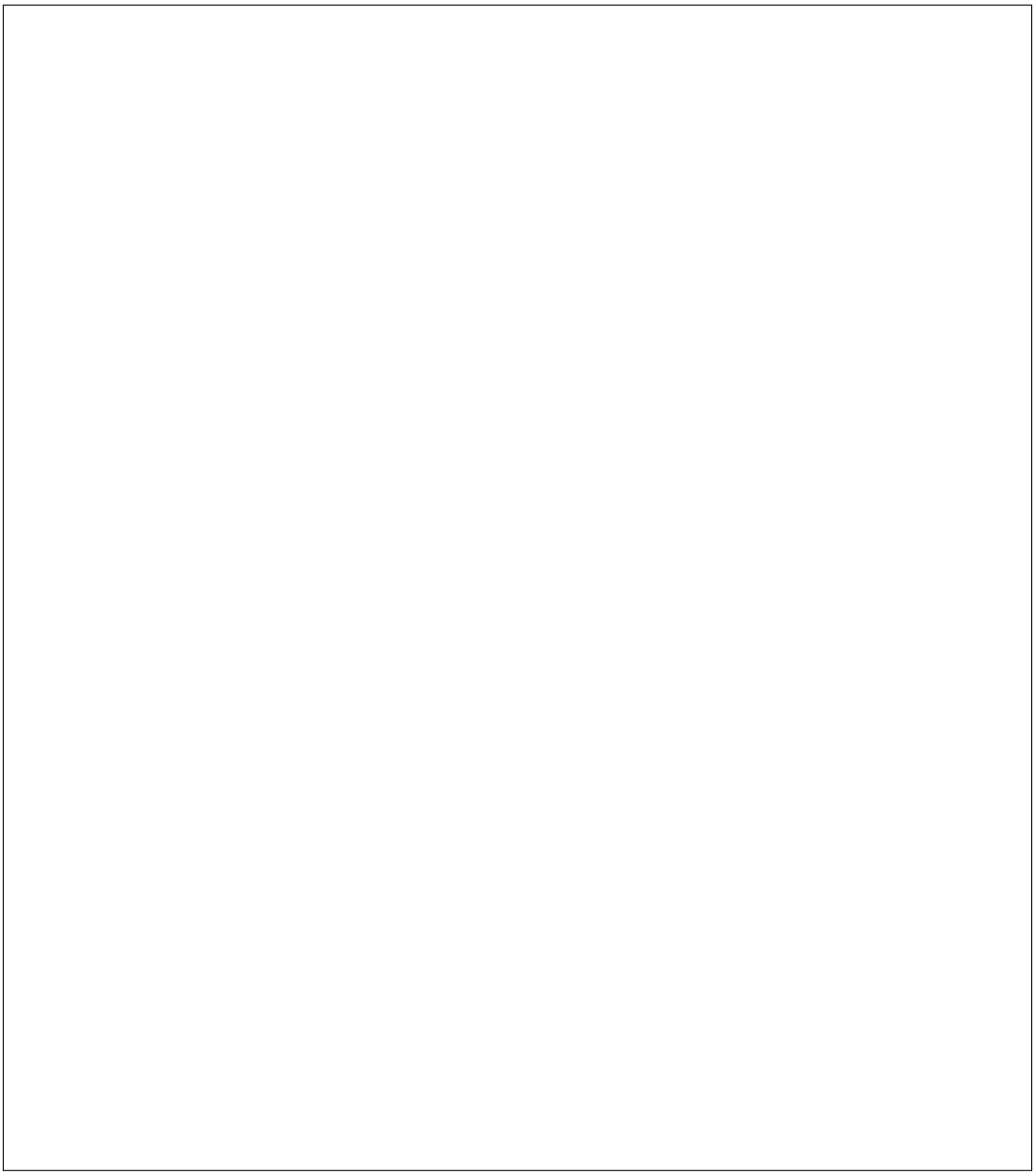
vi. **(1.0 pt)** Fill in blank (j).

```
True
```

6. (0.0 points) Just for Fun

This is not for points and will not be graded.

(a) Optional: Draw how this exam made you feel!



No more questions.